

Switching and Finite Automata Theory

Third Edition

Zvi Kohavi

Technion–Israel Institute of Technology

Niraj K. Jha

Princeton University



CAMBRIDGE
UNIVERSITY PRESS

Information on this title: www.cambridge.org/9780521857482

© Z. Kohavi and N. Jha 2010

First published in print format 2009

ISBN-13 978-0-511-65824-2 eBook (NetLibrary)

ISBN-13 978-0-521-85748-2 Hardback

Contents

Preface

page xi

Part 1 Preliminaries

1	Number systems and codes	3
1.1	Number systems	3
1.2	Binary codes	10
1.3	Error detection and correction	13
	Notes and references	19
	Problems	20
2	Sets, relations, and lattices	23
2.1	Sets	23
2.2	Relations	25
2.3	Partially ordered sets	28
2.4	Lattices	30
	Notes and references	33
	Problems	33

Part 2 Combinational logic

3	Switching algebra and its applications	37
3.1	Switching algebra	37
3.2	Switching functions	44
3.3	Isomorphic systems	52
3.4	Electronic-gate networks	57
*3.5	Boolean algebras	58
	Notes and references	60
	Problems	61

4 Minimization of switching functions	67
4.1 Introduction	67
4.2 The map method	68
4.3 Minimal functions and their properties	78
4.4 The tabulation procedure for the determination of prime implicants	81
4.5 The prime implicant chart	86
4.6 Map-entered variables	93
4.7 Heuristic two-level circuit minimization	95
4.8 Multi-output two-level circuit minimization	97
Notes and references	100
Problems	101
5 Logic design	108
5.1 Design with basic logic gates	108
5.2 Logic design with integrated circuits	112
5.3 NAND and NOR circuits	125
5.4 Design of high-speed adders	128
5.5 Metal-oxide semiconductor (MOS) transistors and gates	132
5.6 Analysis and synthesis of MOS networks	135
Notes and references	143
Problems	144
6 Multi-level logic synthesis	151
6.1 Technology-independent synthesis	151
6.2 Technology mapping	162
Notes and references	169
Problems	170
7 Threshold logic for nanotechnologies	173
7.1 Introductory concepts	173
7.2 Synthesis of threshold networks	181
Notes and references	200
Problems	202
8 Testing of combinational circuits	206
8.1 Fault models	206
8.2 Structural testing	212
8.3 I_{DDQ} testing	220
8.4 Delay fault testing	224
8.5 Synthesis for testability	232
8.6 Testing for nanotechnologies	250
Notes and references	254
Problems	257

Part 3 Finite-state machines

9 Introduction to synchronous sequential circuits and iterative networks	265
9.1 Sequential circuits – introductory example	265
9.2 The finite-state model – basic definitions	269
9.3 Memory elements and their excitation functions	272
9.4 Synthesis of synchronous sequential circuits	280
9.5 An example of a computing machine	293
9.6 Iterative networks	296
Notes and references	300
Problems	300
10 Capabilities, minimization, and transformation of sequential machines	307
10.1 The finite-state model – further definitions	307
10.2 Capabilities and limitations of finite-state machines	309
10.3 State equivalence and machine minimization	311
10.4 Simplification of incompletely specified machines	317
Notes and references	330
Problems	330
11 Asynchronous sequential circuits	338
11.1 Modes of operation	338
11.2 Hazards	339
11.3 Synthesis of SIC fundamental-mode circuits	346
11.4 Synthesis of burst-mode circuits	358
Notes and references	363
Problems	365
12 Structure of sequential machines	372
12.1 Introductory example	372
12.2 State assignments using partitions	375
12.3 The lattice of closed partitions	380
12.4 Reduction of the output dependency	383
12.5 Input independency and autonomous clocks	386
12.6 Covers, and the generation of closed partitions by state splitting	388
12.7 Information flow in sequential machines	395
12.8 Decomposition	404
*12.9 Synthesis of multiple machines	413
Notes and references	418
Problems	419

13 State-identification experiments and testing of sequential circuits	431
13.1 Experiments	431
13.2 Homing experiments	435
13.3 Distinguishing experiments	439
13.4 Machine identification	440
13.5 Checking experiments	442
*13.6 Design of diagnosable machines	448
13.7 Alternative approaches to the testing of sequential circuits	453
13.8 Design for testability	458
13.9 Built-in self-test (BIST)	461
Appendix 13.1 Bounds on the length of synchronizing sequences	464
Appendix 13.2 A bound on the length of distinguishing sequences	467
Notes and references	467
Problems	468
 14 Memory, definiteness, and information losslessness of finite automata	 478
14.1 Memory span with respect to input–output sequences (finite-memory machines)	478
14.2 Memory span with respect to input sequences (definite machines)	483
14.3 Memory span with respect to output sequences	488
14.4 Information-lossless machines	491
*14.5 Synchronizable and uniquely decipherable codes	504
Appendix 14.1 The least upper bound for information losslessness of finite order	510
Notes and references	512
Problems	513
 15 Linear sequential machines	 523
15.1 Introduction	523
15.2 Inert linear machines	525
15.3 Inert linear machines and rational transfer functions	532
15.4 The general model	537
15.5 Reduction of linear machines	541
15.6 Identification of linear machines	550
15.7 Application of linear machines to error correction	556
Appendix 15.1 Basic properties of finite fields	559
Appendix 15.2 The Euclidean algorithm	561
Notes and references	562
Problems	563
 16 Finite-state recognizers	 570
16.1 Deterministic recognizers	570
16.2 Transition graphs	572

16.3	Converting nondeterministic into deterministic graphs	574
16.4	Regular expressions	577
16.5	Transition graphs recognizing regular sets	582
16.6	Regular sets corresponding to transition graphs	588
*16.7	Two-way recognizers	595
	Notes and references	601
	Problems	602
	<i>Index</i>	608

Preface

Topics in switching and finite automata theory have been an important part of the curriculum in electrical engineering and computer science departments for several decades. The third edition of this book builds on the comprehensive foundation provided by the second edition and adds: significant new material in the areas of CMOS logic; modern two-level and multi-level logic synthesis methods; logic design for emerging nanotechnologies; test generation, design for testability and built-in self-test for combinational and sequential circuits; modern asynchronous circuit synthesis techniques; etc. We have attempted to maintain the comprehensive nature of the earlier edition in providing readers with an understanding of the structure, behavior, and limitations of logical machines. At the same time, we have provided an up-to-date context in which the presented techniques can find use in a variety of applications. We start with introductory material and build up to more advanced topics. Thus, the technical background assumed on the part of the reader is minimal.

This edition maintains the style of the previous edition in providing a logical and rigorous discussion of various topics with minimal formalism. Thus, theorems and algorithms are preceded by several intuitive examples to ease understanding. The original references for various topics are provided. Of course, readers who want to dig deeper into a subject would need to consult later works also.

The book is divided into three parts. The first part consists of Chapters 1 and 2. It provides introductory background. The second part consists of Chapters 3 through 8. It deals with combinational logic. The third part consists of Chapters 9 through 16. It is concerned with finite automata. Several chapters contain specific topics that are not prerequisites for subsequent chapters, e.g. Chapters 6, 7, 11–16. Such chapters can be selected at the preference of instructors. Sections marked with a star may be omitted without loss of continuity.

The book can be used for courses at the junior or senior levels in electrical engineering and computer science departments as well as at the beginning graduate level. It is intended as a text for a two-semester sequence. The first semester can be devoted to switching theory (Chapters 1, 3–11) and the second

semester to finite automata theory (Chapters 2, 12–16). Other partitions into two semesters are also possible, keeping in mind that Chapters 3–5 are prerequisites for the rest of the book and Chapters 9 and 10 are prerequisites for Chapters 12–16.

Some chapters have undergone major revision and others only minor revision. Two sections have been added to Chapter 4, on heuristic and multi-output two-level circuit minimization. A section has been added to Chapter 5 on CMOS circuit realizations. Chapter 6 has been completely rewritten with an emphasis on technology-independent multi-level logic synthesis as well as on technology mapping. Chapter 7 has been updated with synthesis techniques geared towards emerging nanotechnologies that can efficiently implement threshold, majority, and minority logic. Chapter 8 has also been completely rewritten to include a discussion of fault models, structural testing, I_{DDQ} testing, delay fault testing, synthesis for testability, and testing for nanotechnologies. All these topics provide the underpinning for the testing of modern integrated circuits. Minor changes have been made to the flip-flop section in Chapter 9. Chapter 11 has been updated with material on the synthesis of asynchronous circuits that allow multiple input changes, including burst-mode circuits. The substantial revisions of Chapter 13 include the addition of material on sequential test generation, design for testability, and built-in self-test. These concepts are also important for understanding how modern integrated circuits are tested. The problem sets have been expanded in all the above chapters.

The previous edition has been used at many universities, which encouraged us to undertake the task of revising the book. We are grateful for the feedback and comments from Professors Sudhakar Reddy, Israel Koren, and Robert Dick. We are also indebted to students and colleagues at Technion and at Princeton University for providing a stimulating environment that made this revision possible.

Last, but not the least, Niraj would like to thank his father, Dr Chintamani Jha, and his wife, Shubha, without whose encouragement and understanding this edition would not have been possible.

Zvi Kohavi
Niraj K. Jha

1

Number systems and codes

This chapter deals with the representation of numerical data, with emphasis on those representations that use only two symbols, 0 and 1. Described are special methods of representing numerical data that afford protection against various transmission errors and component failures.

1.1 Number systems

Convenient as the decimal number system generally is, its usefulness in machine computation is limited because of the nature of practical electronic devices. In most present digital machines, the numbers are represented, and the arithmetic operations performed, in a different number system called the binary number system. This section is concerned with the representation of numbers in various systems and with methods of conversion from one system to another.

Number representation

An ordinary decimal number actually represents a polynomial in powers of 10. For example, the number 123.45 represents the polynomial

$$123.45 = 1 \times 10^2 + 2 \times 10^1 + 3 \times 10^0 + 4 \times 10^{-1} + 5 \times 10^{-2}.$$

This method of representing decimal numbers is known as the *decimal number system*, and the number 10 is referred to as the *base* (or *radix*) of the system. In a system whose base is b , a positive number N represents the polynomial

$$\begin{aligned} N &= a_{q-1}b^{q-1} + \cdots + a_0b^0 + \cdots + a_{-p}b^{-p} \\ &= \sum_{i=-p}^{q-1} a_i b^i, \end{aligned}$$

where the base b is an integer greater than 1 and the a 's are integers in the range $0 \leq a_i \leq b - 1$. The sequence of digits $a_{q-1}a_{q-2} \cdots a_0$ constitutes the *integer*

Table 1.1 Representation of integers

Base				
2	4	8	10	12
0000	0	0	0	0
0001	1	1	1	1
0010	2	2	2	2
0011	3	3	3	3
0100	10	4	4	4
0101	11	5	5	5
0110	12	6	6	6
0111	13	7	7	7
1000	20	10	8	8
1001	21	11	9	9
1010	22	12	10	α
1011	23	13	11	β
1100	30	14	12	10
1101	31	15	13	11
1110	32	16	14	12
1111	33	17	15	13

part of N , while the sequence $a_{-1}a_{-2}\cdots a_{-p}$ constitutes the *fractional part* of N . Thus, p and q designate the number of digits in the fractional and integer parts, respectively. The integer and fractional parts are usually separated by a *radix point*. The digit a_{-p} is referred to as the *least significant digit* while a_{q-1} is called the *most significant digit*.

When the base b equals 2, the number representation is referred to as the *binary number system*. For example, the binary number 1101.01 represents the polynomial

$$1101.01 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2},$$

that is,

$$1101.01 = \sum_{i=-2}^3 a_i 2^i,$$

where $a_{-2} = a_0 = a_2 = a_3 = 1$ and $a_{-1} = a_1 = 0$.

A number N in base b is usually denoted $(N)_b$. Whenever the base is not specified, base 10 is implicit. Table 1.1 shows the representations of integers 0 through 15 in several number systems.

The *complement* of a digit a , denoted a' , in base b is defined as

$$a' = (b - 1) - a.$$

That is, the complement a' is the difference between the largest digit in base b and digit a . In the binary number system, since $b = 2$, $0' = 1$ and $1' = 0$.

In the decimal number system, the largest digit is 9. Thus, for example, the complement¹ of 3 is $9 - 3 = 6$.

Conversion of bases

Suppose that some number N , which we wish to express in base b_2 , is presently expressed in base b_1 . In converting a number from base b_1 to base b_2 , it is convenient to distinguish between two cases. In the first case $b_1 < b_2$, and consequently base- b_2 arithmetic can be used in the conversion process. The conversion technique involves expressing number $(N)_{b_1}$ as a polynomial in powers of b_1 and evaluating the polynomial using base- b_2 arithmetic.

Example We wish to express the numbers $(432.2)_8$ and $(1101.01)_2$ in base 10. Thus

$$\begin{aligned}(432.2)_8 &= 4 \times 8^2 + 3 \times 8^1 + 2 \times 8^0 + 2 \times 8^{-1} = (282.25)_{10}, \\ (1101.01)_2 &= 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 0 \\ &\quad \times 2^{-1} + 1 \times 2^{-2} = (13.25)_{10}.\end{aligned}$$

In both cases, the arithmetic operations are done in base 10.

When $b_1 > b_2$ it is more convenient to use base- b_1 arithmetic. The conversion procedure will be obtained by considering separately the integer and fractional parts of N . Let $(N)_{b_1}$ be an integer whose value in base b_2 is given by

$$(N)_{b_1} = a_{q-1}b_2^{q-1} + a_{q-2}b_2^{q-2} + \cdots + a_1b_2^1 + a_0b_2^0.$$

To find the values of the a 's, let us divide the above polynomial by b_2 .

$$\frac{(N)_{b_1}}{b_2} = \underbrace{a_{q-1}b_2^{q-2} + a_{q-2}b_2^{q-3} + \cdots + a_1}_{Q_0} + \frac{a_0}{b_2}.$$

Thus, the least significant digit of $(N)_{b_2}$, i.e., a_0 , is equal to the first remainder. The next most significant digit, a_1 , is obtained by dividing the quotient Q_0 by b_2 , i.e.,

$$\left(\frac{Q_0}{b_2}\right)_{b_1} = \underbrace{a_{q-1}b_2^{q-3} + a_{q-2}b_2^{q-4} + \cdots}_{Q_1} + \frac{a_1}{b_2}.$$

The remaining a 's are evaluated by repeated divisions of the quotients until Q_{q-1} is equal to zero. If N is finite, the process must terminate.

¹ In the decimal system, the complement is also referred to as the 9's complement. In the binary system, it is also known as the 1's complement.

Example The above conversion procedure is now applied to convert $(548)_{10}$ to base 8. The r_i in the table below denote the remainders. The first entries in the table are 68 and 4, corresponding, respectively, to the quotient Q_0 and the first remainder from the division $(548/8)_{10}$. The remaining entries are found by successive division.

Q_i	r_i
68	$4 = a_0$
8	$4 = a_1$
1	$0 = a_2$
	$1 = a_3$

Thus, $(548)_{10} = (1044)_8$. In a similar manner we can obtain the conversion of $(345)_{10}$ to $(1333)_6$, as illustrated in the table below.

Q_i	r_i
57	$3 = a_0$
9	$3 = a_1$
1	$3 = a_2$
	$1 = a_3$

Indeed, $(1333)_6$ can be reconverted to base 10, i.e.,

$$(1333)_6 = 1 \times 6^3 + 3 \times 6^2 + 3 \times 6^1 + 3 \times 6^0 = 345$$

If $(N)_{b_1}$ is a fraction, a dual procedure is employed. It can be expressed in base b_2 as follows:

$$(N)_{b_1} = a_{-1}b_2^{-1} + a_{-2}b_2^{-2} + \cdots + a_{-p}b_2^{-p}.$$

The most significant digit, a_{-1} , can be obtained by multiplying the polynomial by b_2 :

$$b_2 \cdot (N)_{b_1} = a_{-1} + a_{-2}b_2^{-1} + \cdots + a_{-p}b_2^{-p+1}.$$

If the above product is less than 1 then a_{-1} equals 0; if the product is greater than or equal to 1 then a_{-1} is equal to the integer part of the product. The next most significant digit, a_{-2} , is found by multiplying the fractional part of the above product part by b_2 and determining its integer part; and so on. This process does not necessarily terminate since it may not be possible to represent the fraction in base b_2 with a finite number of digits.

Example To convert $(0.3125)_{10}$ to base 8, find the digits as follows:

$$0.3125 \times 8 = 2.5000, \quad \text{hence} \quad a_{-1} = 2;$$

$$0.5000 \times 8 = 4.0000, \quad \text{hence} \quad a_{-2} = 4.$$

Thus $(0.3125)_{10} = (0.24)_8$.

Similarly, the computation below proves that $(0.375)_{10} = (0.011)_2$:

$$0.375 \times 2 = 0.750, \quad \text{hence} \quad a_{-1} = 0;$$

$$0.750 \times 2 = 1.500, \quad \text{hence} \quad a_{-2} = 1;$$

$$0.500 \times 2 = 1.000, \quad \text{hence} \quad a_{-3} = 1.$$

Example To convert $(432.354)_{10}$ to binary, we first convert the integer part and then the fractional part. For the integer part we have

Q_i	r_i
216	$0 = a_0$
108	$0 = a_1$
54	$0 = a_2$
27	$0 = a_3$
13	$1 = a_4$
6	$1 = a_5$
3	$0 = a_6$
1	$1 = a_7$
	$1 = a_8$

Hence $(432)_{10} = (110110000)_2$. For the fractional part we have

$$0.354 \times 2 = 0.708, \quad \text{hence} \quad a_{-1} = 0,$$

$$0.708 \times 2 = 1.416, \quad \text{hence} \quad a_{-2} = 1,$$

$$0.416 \times 2 = 0.832, \quad \text{hence} \quad a_{-3} = 0,$$

$$0.832 \times 2 = 1.664, \quad \text{hence} \quad a_{-4} = 1,$$

$$0.664 \times 2 = 1.328, \quad \text{hence} \quad a_{-5} = 1,$$

$$0.328 \times 2 = 0.656, \quad \text{hence} \quad a_{-6} = 0,$$

$$a_{-7} = 1,$$

etc.

Consequently $(0.354)_{10} = (0.0101101 \cdots)_2$. The conversion is usually carried up to the desired accuracy. In our example, reconversion to base 10 shows that

$$(110110000.0101101)_2 = (432.3515)_{10}$$

Table 1.2 Elementary binary operations

Bits		Sum		Difference		Product
a	b	$a + b$	Carry	$a - b$	Borrow	
0	0	0	0	0	0	0
0	1	1	0	1	1	0
1	0	1	0	1	0	0
1	1	0	1	0	0	1

A considerably simpler conversion procedure may be employed in converting octal numbers (i.e., numbers in base 8) to binary and vice versa. Since $8 = 2^3$, each octal digit can be expressed by three binary digits. For example, $(6)_8$ can be expressed as $(110)_2$, etc. The procedure of converting a binary number into an octal number consists of partitioning the binary number into groups of three digits, starting from the binary point, and to determine the octal digit corresponding to each group.

Example

$$(123.4)_8 = (001\ 010\ 011.100)_2,$$

$$(1010110.0101)_2 = (001\ 010\ 110.010\ 100) = (126.24)_8.$$

A similar procedure may be employed in conversions from binary to hexadecimal (base 16), except that four binary digits are needed to represent a single hexadecimal digit. In fact, whenever a number is converted from base b_1 to base b_2 , where $b_2 = b_1^k$, k digits of that number when grouped may be represented by a single digit from base b_2 .

Binary arithmetic

The binary number system is widely used in digital systems. Although a detailed study of digital arithmetic is beyond the scope of this book, we shall present the elementary techniques of binary arithmetic. The basic arithmetic operations are summarized in Table 1.2, where the sum and carry, difference and borrow, and product are computed for every combination of binary digits (abbreviated *bits*) 0 and 1. For a more comprehensive discussion of computer arithmetic, the reader may consult [2].

Binary addition is performed in a manner similar to that of decimal addition. Corresponding bits are added and if a carry 1 is produced then it is added to the binary digits at the left.

Example The addition of $(15.25)_{10}$ and $(7.50)_{10}$ in binary proceeds as follows:

$$\begin{array}{r}
 1111 \quad \text{carries of 1} \\
 1111.01 = (15.25)_{10} \\
 + \\
 0111.10 = (7.50)_{10} \\
 \hline
 10110.11 = (22.75)_{10}
 \end{array}$$

In subtraction, if a borrow of 1 occurs and the next left digit of the minuend (the number from which a subtraction is being made) is 1 then the latter is changed to 0 and subtraction is continued in the usual manner. If, however, the next left digit of the minuend is 0 then it is changed to 1, as is each successive minuend digit to the left which is equal to 0. The first minuend digit to the left, which is equal to 1, is changed to 0, and subtraction is continued.

Example The subtraction of $(12.50)_{10}$ from $(18.75)_{10}$ in binary proceeds as follows:

$$\begin{array}{r}
 1 \quad \text{borrows of 1} \\
 10010.11 = (18.75)_{10} \\
 - \\
 01100.10 = (12.50)_{10} \\
 \hline
 00110.01 = (6.25)_{10}
 \end{array}$$

Just as with decimal numbers, the multiplication of binary numbers is performed by successive addition while division is performed by successive subtraction.

Example Multiply the binary numbers below:

$$\begin{array}{r}
 11001.1 = (25.5)_{10} \\
 \times \\
 110.1 = (6.5)_{10} \\
 \hline
 110011 \\
 000000 \\
 110011 \\
 110011 \\
 \hline
 10100101.11 = (165.75)_{10}
 \end{array}$$

Example Divide the binary number 1000100110 by 11001.

$$\begin{array}{r}
 10110 \text{ quotient} \\
 11001 \overline{)1000100110} \\
 11001 \\
 \underline{00100101} \\
 11001 \\
 \underline{0011001} \\
 11001 \\
 \underline{00000} \text{ remainder}
 \end{array}$$

1.2 Binary codes

Although the binary number system has many practical advantages and is widely used in digital computers, in many cases it is convenient to work with the decimal number system, especially when the communication between human being and machine is extensive, since most numerical data generated by humans is in terms of decimal numbers. To simplify the problem of communication between human and machine, several codes have been devised in which decimal digits are represented by sequences of binary digits.

Weighted codes

In order to represent the 10 decimal digits 0, 1, . . . , 9, it is necessary to use at least four binary digits. Since there are 16 combinations of four binary digits, of which 10 combinations are used, it is possible to form a very large number of distinct codes. Of particular importance is the class of *weighted codes*, whose main characteristic is that each binary digit is assigned a decimal “weight,” and, for each group of four bits, the sum of the weights of those binary digits whose value is 1 is equal to the decimal digit which they represent. If w_1, w_2, w_3 , and w_4 are the given weights of the binary digits and x_1, x_2, x_3, x_4 the corresponding digit values then the decimal digit $N = w_4x_4 + w_3x_3 + w_2x_2 + w_1x_1$ can be represented by the binary sequence $x_4x_3x_2x_1$. The sequence of binary digits that represents a decimal digit is called a *code word*. Thus, the sequence $x_4x_3x_2x_1$ is the code word for N . Three weighted four-digit binary codes are shown in Table 1.3.

The binary digits in the first code in Table 1.3 are assigned weights 8, 4, 2, 1. As a result of this weight assignment, the code word that corresponds to each decimal digit is the binary equivalent of that digit; e.g., 5 is represented by 0101, and so on. This code is known as the *binary-coded-decimal* (BCD)

Table 1.3 The code words $x_4x_3x_2x_1$ for the decimal digits N in three weighted binary codes

Decimal digit N	$w_4w_3w_2w_1$											
	8	4	2	1	2	4	2	1	6	4	2	-3
0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1	0	1	0	1
2	0	0	1	0	0	0	1	0	0	0	1	0
3	0	0	1	1	0	0	1	1	1	0	0	1
4	0	1	0	0	0	1	0	0	0	1	0	0
5	0	1	0	1	1	0	1	1	1	0	1	1
6	0	1	1	0	1	1	0	0	0	1	1	0
7	0	1	1	1	1	1	0	1	1	1	0	1
8	1	0	0	0	1	1	1	0	1	0	1	0
9	1	0	0	1	1	1	1	1	1	1	1	1

code. For each code in Table 1.3, the decimal digit that corresponds to a given code word is equal to the sum of the weights in those binary positions that are 1's rather than 0's. Thus, in the second code, where the weights are 2, 4, 2, 1, decimal 5 is represented by 1011, corresponding to the sum $2 \times 1 + 4 \times 0 + 2 \times 1 + 1 \times 1 = 5$. The weights assigned to the binary digits may also be negative, as in the code (6, 4, 2, -3). In this code, decimal 5 is represented by 1011, since $6 \times 1 + 4 \times 0 + 2 \times 1 - 3 \times 1 = 5$.

It is apparent that the representations of some decimal numbers in the (2, 4, 2, 1) and (6, 4, 2, -3) codes are not unique. For example, in the (2, 4, 2, 1) code, decimal 7 may be represented by 1101 as well as 0111. Adopting the representations shown in Table 1.3 causes the codes to become self-complementing. A code is said to be *self-complementing* if the code word of the “9's complement of N ”, i.e., $9 - N$, can be obtained from the code word of N by interchanging all the 1's and 0's. For example, in the (6, 4, 2, -3) code, decimal 3 is represented by 1001 while decimal 6 is represented by 0110. In the (2, 4, 2, 1) code, decimal 2 is represented by 0010 while decimal 7 is represented by 1101. Note that the BCD code (8, 4, 2, 1) is not self-complementing. It can be shown that a necessary condition for a weighted code to be self-complementing is that the sum of the weights must equal 9. There exist only four positively weighted self-complementing codes, namely, (2, 4, 2, 1), (3, 3, 2, 1), (4, 3, 1, 1), and (5, 2, 1, 1). In addition, there exist 13 self-complementing codes with positive and negative weights.

Nonweighted codes

There are many nonweighted binary codes, two of which are shown in Table 1.4. The Excess-3 code is formed by adding 0011 to each BCD code word.

Table 1.4 Nonweighted binary codes

Decimal digit	Excess-3				Cyclic			
0	0	0	1	1	0	0	0	0
1	0	1	0	0	0	0	0	1
2	0	1	0	1	0	0	1	1
3	0	1	1	0	0	0	1	0
4	0	1	1	1	0	1	1	0
5	1	0	0	0	1	1	1	0
6	1	0	0	1	1	0	1	0
7	1	0	1	0	1	0	0	0
8	1	0	1	1	1	1	0	0
9	1	1	0	0	0	1	0	0

Table 1.5 Decimal numbers in the complete four-bit Gray code and in binary

Decimal number	Gray				Binary			
	g_3	g_2	g_1	g_0	b_3	b_2	b_1	b_0
0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
2	0	0	1	1	0	0	1	0
3	0	0	1	0	0	0	1	1
4	0	1	1	0	0	1	0	0
5	0	1	1	1	0	1	0	1
6	0	1	0	1	0	1	1	0
7	0	1	0	0	0	1	1	1
8	1	1	0	0	1	0	0	0
9	1	1	0	1	1	0	0	1
10	1	1	1	1	1	0	1	0
11	1	1	1	0	1	0	1	1
12	1	0	1	0	1	1	0	0
13	1	0	1	1	1	1	0	1
14	1	0	0	1	1	1	1	0
15	1	0	0	0	1	1	1	1

Thus, for example, the representation of decimal 7 in Excess-3 is given by $0111 + 0011 = 1010$. The Excess-3 code is self-complementing and possesses a number of properties that made it practical in early decimal computers.

In many practical applications, e.g., analog-to-digital conversion, it is desirable to use codes in which the code words for successive decimal integers differ in only one digit. Codes that have such a property are referred to as *cyclic codes*. The second code in Table 1.4 is an example of such a code. (Note that in this, as in all cyclic codes, the code word representing the decimal digits 0 and 9 differ in only one digit.) A particularly important cyclic code is the *Gray code*. A four-bit Gray code is shown in Table 1.5. The feature that makes this cyclic

code useful is the simplicity of the procedure for converting from the binary number system into the Gray code, as follows.

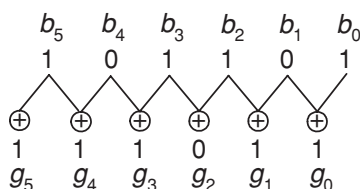
Let $g_n \cdots g_2 g_1 g_0$ denote a code word in the $(n + 1)$ th-bit Gray code, and let $b_n \cdots b_2 b_1 b_0$ designate the corresponding binary number, where the subscripts 0 and n denote the least significant and most significant digits, respectively. Then, the i th digit g_i can be obtained from the corresponding binary number as follows:

$$g_i = b_i \oplus b_{i+1}, \quad 0 \leq i \leq n - 1, \\ g_n = b_n,$$

where the symbol $\oplus$ denotes the *modulo-2 sum*, which is defined as follows:

$$0 \oplus 0 = 0, \quad 1 \oplus 1 = 0, \quad 0 \oplus 1 = 1, \quad 1 \oplus 0 = 1.$$

For example, the Gray code word that corresponds to the binary number 101101 is found to be 111011 in a manner indicated in the following diagram:



Thus, to convert from Gray code to binary, start with the leftmost digit and proceed to the least significant digit, setting $b_i = g_i$ if the number of 1's preceding g_i is even and setting $b_i = g'_i$ if the number of 1's preceding g_i is odd. (Note that zero 1's counts as an even number of 1's.) For example, the Gray code word 1001011 represents the binary number 1110010. The proof that the preceding conversion procedures does indeed work is left to the reader as an exercise.

The n -bit Gray code is a member of a class called *reflected codes*. The term “reflected” is used to designate codes which have the property that the n -bit code can be generated by reflecting the $(n - 1)$ th-bit code, as illustrated in Fig. 1.1. The two-bit Gray code is shown in Fig. 1.1a. The three-bit Gray code (Fig. 1.1b) can be obtained by reflecting the two-bit code about an axis at the end of the code and assigning a most significant bit of 0 above the axis and 1 below the axis. The four-bit Gray code is obtained in the same manner from the three-bit code, as shown in Fig. 1.1c.

1.3 Error detection and correction

In the codes presented so far, each code word consists of four binary digits, which is the minimum number needed to represent the 10 decimal digits. Such

Fig. 1.1 Reflection of Gray codes.

00	0	00	0	000
01	0	01	0	001
11	0	11	0	011
10	0	10	0	010
	1	10	0	110
	1	11	0	111
	1	01	0	101
	1	00	0	100
			1	100
			1	101
			1	111
			1	110
			1	010
			1	011
			1	001
			1	000

(a)

(b)

(c)

codes, although adequate for the representation of decimal digits, are very sensitive to the transmission errors that may occur because of equipment failure or noise in the transmission channel. In any practical system there is always a finite probability of occurrence of a single error. The probability that two or more errors will occur simultaneously, although nonzero, is substantially smaller. We, therefore, restrict our discussion mainly to the detection and correction of single errors.

Error-detecting codes

In a four-bit binary code, the occurrence of a single error in one of the binary digits may result in another, incorrect but valid, code word. For example, in the BCD code (see above), if an error occurs in the least significant digit of 0110 then the code word 0111 results and, since it is a valid code word, it is incorrectly interpreted by the receiver. If a code possesses the property that the occurrence of any single error transforms a valid code word into an invalid code word, it is said to be a (*single-*)*error-detecting code*. Two error-detecting codes are shown in Table 1.6.

Error detection in either code in Table 1.6 is accomplished by a *parity check*. The basic idea in a parity check is to add an extra digit to each code word of a given code so as to make the number of 1's in each code word either odd or even. In the codes of Table 1.6 we have used *even parity*. The even-parity BCD code is obtained directly from the BCD code of Table 1.3. The added bit, denoted *p*, is called the *parity bit*. The *2-out-of-5 code* consists of all 10 possible combinations of two 1's in a five-bit code word. With the exception

Table 1.6 Error-detecting codes

Decimal digit	Even-parity BCD					2-out-of-5				
	8	4	2	1	p	0	1	2	4	7
0	0	0	0	0	0	0	0	0	1	1
1	0	0	0	1	1	1	1	0	0	0
2	0	0	1	0	1	1	0	1	0	0
3	0	0	1	1	0	0	1	1	0	0
4	0	1	0	0	1	1	0	0	1	0
5	0	1	0	1	0	0	1	0	1	0
6	0	1	1	0	0	0	0	1	1	0
7	0	1	1	1	1	1	0	0	0	1
8	1	0	0	0	1	0	1	0	0	1
9	1	0	0	1	0	0	0	1	0	1

of the code word for decimal 0, the 2-out-of-5 code of Table 1.6 is a weighted code and can be derived from the (1, 2, 4, 7) code.

In each of the codes in Table 1.6 the number of 1's in a code word is even. Now, if a single error occurs it transforms the valid code word into an invalid one, thus making the detection of the error straightforward. Although parity check is intended only for the detection of single errors, it, in fact, detects any odd number of errors and some even numbers of errors. For example, if the code word 10100 is received in an even-parity BCD message, it is clear that the message is erroneous, since such a code word is not defined although the parity check is satisfied. We cannot determine, however, the original transmitted word.

In general, to obtain an n -bit error-detecting code, no more than half the possible 2^n combinations of digits can be used. The code words are chosen in such a manner that, in order to change one valid code word into another valid code word, at least two digits must be complemented. In the case of four-bit codes this constraint means that only eight valid code words can be formed of the 16 possible combinations. Thus, to obtain an error-detecting code for the 10 decimal digits, at least five binary digits are needed. It is useful to define the *distance* between two code words as the number of digits that must change in one word so that the other word results. For example, the distance between 1010 and 0100 is three, since the two code words differ in three bit positions. The *minimum distance* of a code is the smallest number of bits in which any two code words differ. Thus, the minimum distance of the BCD or the Excess-3 codes is one, while that of the codes in Table 1.6 is two. Clearly, *a code is an error-detecting code if and only if its minimum distance is two or more.*

Error-correcting codes

For a code to be error-correcting, its minimum distance must be further increased. For example, consider the three-bit code which consists of only two

valid code words, 000 and 111. If a single error occurs in the first code word, it could become 001, 010, or 100. The second code word could be changed by a single error to 110, 101, or 011. Note that in each case the invalid code words are different. Clearly, this code is error-detecting since its minimum distance is three. Moreover, if we assume that only a single error can occur then this error can be located and corrected, since every error results in an invalid code word that can be associated with only one of the valid code words. Thus, the two code words 000 and 111 constitute an error-correcting code whose minimum distance is three. In general, a code is said to be *error-correcting* if the correct code word can always be deduced from the erroneous word. In this section, we shall discuss a type of single-error-correcting codes known as *Hamming codes*.

If the minimum distance of a code is three, then any single error changes a valid code word into an invalid one, which is distance one away from the original code word and distance two from any other valid code word. Therefore, in a code with minimum distance three, any single error is correctable *or* any double error detectable. Similarly, a code whose minimum distance is four may be used for either single-error correction *and* double-error detection *or* triple-error detection. The key to error correction is that it must be possible to *detect* and *locate* erroneous digits. If the location of an error has been determined then, by complementing the erroneous digit, the message is corrected.

The basic principles in constructing a Hamming error-correcting code are as follows. To each group of m *information* or *message digits*, k *parity-checking digits*, denoted $p_1, p_2, \dots, p_k$, are added to form an $(m + k)$ -digit code. The location of each of the $m + k$ digits within a code word is assigned a decimal value; one starts by assigning a 1 to the most significant digit and $m + k$ to the least significant digit. Then k parity checks are performed on selected digits of each code word. The result of each parity check is recorded as 1 or 0, depending, respectively, on whether an error has or has not been detected. These parity checks make possible the development of a binary number, $c_1 c_2 \dots c_k$, whose value is equal to the decimal value assigned to the location of the erroneous digit when an error occurs and is equal to zero if no error occurs. This number is called the *position* (or *location*) *number*.

The number k of digits in the position number must be large enough to describe the location of any of the $m + k$ possible single errors, and must in addition take on the value zero to describe the “no error” condition. Consequently, k must satisfy the inequality $2^k \geq m + k + 1$. Thus, for example, if the original message is in BCD where $m = 4$ then $k = 3$ and at least three parity-checking digits must be added to the BCD code. The resultant error-correcting code thus consists of seven digits. In this case, if the position number is equal to 101, it means that an error has occurred in position 5. If, however, the position number is equal to 000, the message is correct.

In order to be able to specify the checking digits by means of only message digits and independently of each other, they are placed in positions

Table 1.7 Position numbers $c_1c_2c_3$

Error position	Position number		
	c_1	c_2	c_3
0 (no error)	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

1, 2, 4, $\dots$, 2^{k-1} . Thus, if $m = 4$ and $k = 3$ then the checking digits are placed in positions 1, 2, and 4 while the remaining positions contain the original (BCD) message bits. For example, in the code word **1100**110, the checking digits (in boldface) are $p_1 = 1$, $p_2 = 1$, $p_3 = 0$, while the message digits are 0, 1, 1, 0, which correspond to decimal 6.

We shall now show how the Hamming code is constructed, by constructing the code for $m = 4$ and $k = 3$. As discussed above, the parity-checking digits must be specified in such a way that, when an error occurs, the position number will take on the value assigned to the location of the erroneous digit. Table 1.7 lists the seven error positions and the corresponding values of the position number. It is evident that if an error occurs in position 1, or 3, or 5, or 7, the least significant digit, i.e., c_3 , of the position number must be equal to 1. If the code is constructed so that in every code word the digits in positions 1, 3, 5, and 7 have even parity, then the occurrence of a single error in any of these positions will cause an odd parity. In such a case, the least significant digit of the position number is recorded as 1. If no error occurs among these digits, a parity check will show an even parity and the least significant digit of the position number is recorded as 0.

From Table 1.7, we observe that an error in positions 2, 3, 6, or 7 should result in the recording of a 1 in the center of the position number. Hence, the code must be designed so that the digits in positions 2, 3, 6, and 7 have even parity. Again, if the parity check of these digits shows an odd parity then the corresponding position-number digit, i.e., c_2 , is set to 1; otherwise it is set to 0. Finally, if an error occurs in positions 4, 5, 6, or 7 then the most significant digit of the position number, i.e., c_1 , should be a 1. Therefore, if digits 4, 5, 6, and 7 are designed to have even parity, an error in any of these digits will be recorded as a 1 in the most significant digit of the position number. To summarize the situation regarding the checking digits p_i :

- p_1 is selected so as to establish even parity in positions 1, 3, 5, 7;
- p_2 is selected so as to establish even parity in positions 2, 3, 6, 7;
- p_3 is selected so as to establish even parity in positions 4, 5, 6, 7.

Table 1.8 Hamming code for BCD

Decimal digit	Digit position and symbol						
	1 p_1	2 p_2	3 m_1	4 p_3	5 m_2	6 m_3	7 m_4
0	0	0	0	0	0	0	0
1	1	1	0	1	0	0	1
2	0	1	0	1	0	1	0
3	1	0	0	0	0	1	1
4	1	0	0	1	1	0	0
5	0	1	0	0	1	0	1
6	1	1	0	0	1	1	0
7	0	0	0	1	1	1	1
8	1	1	1	0	0	0	0
9	0	0	1	1	0	0	1

The code can now be constructed by adding the appropriate checking digits to the message digits. Consider, for example, the message 0100 (i.e., decimal 4), as shown in the table below.

Digit position:	1	2	3	4	5	6	7
Digit symbol:	p_1	p_2	m_1	p_3	m_2	m_3	m_4
Original BCD message:			0		1	0	0
Parity check in positions 1, 3, 5, 7 requires $p_1 = 1$:	1		0		1	0	0
Parity check in positions 2, 3, 6, 7 requires $p_2 = 0$:	1	0	0		1	0	0
Parity check in positions 4, 5, 6, 7 requires $p_3 = 1$:	1	0	0	1	1	0	0
Coded message:	1	0	0	1	1	0	0

Thus checking digit p_1 is set equal to 1 so as to establish even parity in positions 1, 3, 5, and 7. Similarly, it is evident that p_2 must be 0 and p_3 must be 1, so that even parity is established, respectively, in positions 2, 3, 6, and 7 and 4, 5, 6, and 7. The Hamming code for the decimal digits coded in BCD is shown in Table 1.8.

Error location and correction are performed for the Hamming code in the following manner. Suppose, for example, that the sequence 1101001 is transmitted but, owing to an error in the fifth position, the sequence 1101101 is received. The location of the error can be determined by performing three parity checks as follows:

Digit position:	1	2	3	4	5	6	7	
Message received:	1	1	0	1	1	0	1	
4-5-6-7 parity check:				1	1	0	1	$c_1 = 1$ since parity is odd
2-3-6-7 parity check:		1	0			0	1	$c_2 = 0$ since parity is even
1-3-5-7 parity check:	1		0		1		1	$c_3 = 1$ since parity is odd

Thus, the position number formed as $c_1c_2c_3$ is 101, which means that the location of the error is in position 5. To correct the error, the digit in position 5 is complemented and the correct message 1101001 is obtained.

It is easy to prove that the Hamming code constructed as shown above is a code whose distance is three. Consider, for example, the case where the two original four-bit (code) words differ in only one position, e.g., 1001 and 0001. Since each message digit appears in at least two parity checks, the parity checks that involve the digit in which the two code words differ will result in different parities and hence different checking digits will be added to the two words, making the distance between them equal to three. For example, consider the two words below.

Digit position:	1	2	3	4	5	6	7
Digit symbol:	p_1	p_2	m_1	p_3	m_2	m_3	m_4
First word:			1		0	0	1
Second word:			0		0	0	1
First word with parity bits:	0	0	1		0	0	1
Second word with parity bits:	1	1	0		0	0	1

The two words differ in only m_1 (i.e., position 3). Parity checks 1-3-5-7 and 2-3-6-7 for these two words will give different results. Therefore, the parity-checking digits p_1 and p_2 must be different for these words. Clearly, the foregoing argument is valid in the case where the original code words differ in two of the four positions. Thus, the Hamming code has a distance of three.

If the distance is increased to four, by adding a parity bit to the code in Table 1.8 in such a way that all eight digits have even parity, the code may be used for single-error correction and double-error detection in the following manner. Suppose that two errors occur; then the overall parity check is satisfied but the position number (determined as before from the first seven digits) will indicate an error. Clearly, such a situation indicates the existence of a double error. The error positions, however, cannot be located. If only a single error occurs, the overall parity check will detect it. Now, if the position number is 0 then the error is in the last parity bit; otherwise, it is in the position given by the position number. If all four parity checks indicate even parities then the message is correct.

Notes and references

The material on number systems is available in almost all elementary texts on algebra, switching theory, and digital computers. An extensive discussion of computer arithmetic is available in Koren [2]. Binary codes have been studied by numerous authors. A listing of many four-bit weighted codes is given in Richards [3]. The material on error-correcting codes is due to Hamming [1].

[1] Hamming, R. W.: "Error detecting and error correcting codes," *Bell System Tech. J.*, vol. 29, pp. 147–160, April 1950.
[2] Koren, I.: *Computer Arithmetic Algorithms*, A. K. Peters, Natick MA, 2002.

- [3] Richards, R. K.: *Arithmetic Operations in Digital Computers*, Van Nostrand, Princeton NJ, 1955.

Problems

Problem 1.1. Convert the following numbers in the way specified:

- (a) $(1431)_8$ to base 10
- (b) 11001010.0101 to base 10
- (c) 11001101.0101 to base 8 and base 4
- (d) $(1984)_{10}$ to base 8
- (e) $(1776)_{10}$ to base 6
- (f) $(53.1575)_{10}$ to base 2
- (g) $(3.1415 \dots)_{10}$ to base 8 and base 2

Problem 1.2

- (a) Given that $(16)_{10} = (100)_b$, determine the value of b .
- (b) Given that $(292)_{10} = (1204)_b$, determine the value of b .

Problem 1.3. Given binary numbers $a = 1010.1$, $b = 101.01$, and $c = 1001.1$, perform the following binary operations:

- (a) $a + c$
- (b) $a - b$
- (c) $a \cdot c$
- (d) a/b

Problem 1.4. Each of the following arithmetic operations is correct in at least one number system. Determine the possible bases of the numbers in each operation.

- (a) $1234 + 5432 = 6666$
- (b) $41/3 = 13$
- (c) $33/3 = 11$
- (d) $23 + 44 + 14 + 32 = 223$
- (e) $302/20 = 12.1$
- (f) $\sqrt{41} = 5$

Problem 1.5. In the following series, the same integer is expressed in different number systems. Determine the missing member of the series.

$$10\,000, 121, 100, ?, 24, 22, 20, \dots$$

Problem 1.6

- (a) Encode each of the 10 decimal digits 0, 1, ..., 9 by means of the following weighted binary codes:

6	3	1	-1
7	3	2	-1
7	3	1	-2
5	4	-2	-1
8	7	-4	-2

- (b) Determine which of the above codes is self-complementing.

Problem 1.7

- (a) Prove that, in every positively weighted code, one of the weights must be 1, a second weight must be either 1 or 2, and the sum of the weights must be equal to or greater than 9.
- (b) Show by listing all such codes that there are only 17 positively weighted codes, of which only four are self-complementing.

Problem 1.8

- (a) Prove that in a self-complementing code the sum of the weights must be 9.
- (b) Obtain the weights of three different four-bit self-complementing codes whose only negative weight is -4 .

Problem 1.9. The following were suggested as the first few code words in four cyclic codes. In each case, either complete the code or show that it cannot be completed. Each code sequence must contain the set of all possible code words, and the last code word must be distance one from the first.

- (a) 000, 001, 011, 111
- (b) 000, 010, 011, 111, 101
- (c) 000, 010, 110, 111
- (d) 0000, 0100, 0101, 1101, 1111, 1011, 1010

Problem 1.10. Given a Gray code word $g_n \cdots g_2 g_1 g_0$, prove that the i th digit of the corresponding binary number $b_n \cdots b_2 b_1 b_0$ is given by

$$b_i = g_n \oplus g_{n-1} \oplus g_{n-2} \oplus \cdots \oplus g_i,$$

$$b_n = g_n.$$

Hint: Prove first that if $x \oplus y = z$ then $x \oplus z = y$ and $y \oplus z = x$, where x , y , and z are binary variables.

Problem 1.11. The message below has been coded in the Hamming code of Table 1.8 and transmitted through a noisy channel. Decode the message assuming that at most a single error has occurred in each code word:

1001001011100111101100011011

Problem 1.12. Construct a seven-bit error-correcting code to represent the decimal digits by augmenting the Excess-3 code and by using an *odd-1* parity check.

Problem 1.13. Consider the following four codes:

Code A	Code B	Code C	Code D
0001	000	01011	000000
0010	001	01100	001111
0100	011	10010	110011
1000	010	10101	
	110		
	111		
	101		
	100		

- (a) Which of the following properties is satisfied by each of the above codes?
- (i) Detects single errors

- (ii) Detects double errors
 - (iii) Detects triple errors
 - (iv) Corrects single errors
 - (v) Corrects double errors
 - (vi) Corrects single and detects double errors
- (b) How many words can be added to code A without changing its error-detection and correction capabilities? Give a possible set of such words. Is this set unique?

2

Sets, relations, and lattices

The objective of this chapter is twofold: to develop the properties of partially ordered sets and lattices in an informal manner, and to furnish algebraic concepts necessary for the understanding of later chapters. The chapter develops in an intuitive manner the notions of sets, relations, and partial ordering, which together form the basis for the presentation of some results from lattice theory and, in Chapter 3, Boolean algebras. The chapter is by no means a complete treatment of the subjects but rather a survey of some results that bear upon material presented in later chapters.

2.1 Sets

A set S is intuitively defined as a collection of distinct objects. The readers of this book and prime numbers are examples of sets. The objects that form a set are called *elements*, or *members*, of that set, and the set is said to *contain* them. The membership of an element a in a set A is denoted by $a \in A$ to mean “ a is an element of A .” A set which has no element is called an *empty* or *null set* and is denoted ϕ . The elements contained in a set are either listed explicitly or described by their properties. This is accomplished by placing the elements or the describing property in braces.

Example The set of all even numbers between 1 and 10 is written as

$$\{2, 4, 6, 8, 10\}.$$

The infinite set of all positive even numbers can be described by

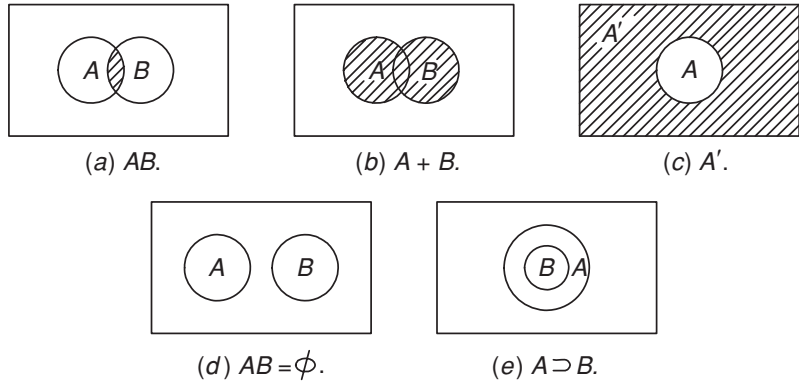
$$\{2, 4, 6, \dots\}.$$

The set

$$\{\text{all readers of this book who live in Antarctica}\}$$

is in all likelihood empty.

Fig. 2.1 Venn diagrams.



Two sets A and B are *equal*, or *identical*, if they contain precisely the same elements. The equality of two sets is denoted by $A = B$. A set A is said to be a *subset* of B if every element of A is also an element of B . If B contains at least one element which is not contained in A , then A is said to be a *proper subset* of B . We use the notation $A \subseteq B$ to indicate that A is a subset of B , and $A \subset B$ to indicate that A is a proper subset of B . Thus, the collection of female students in a university is a proper subset of the set of all students. The subset of students who understand the lecture in a class, on the other hand, is not necessarily a proper subset of all the students sitting in that class, since it may happen that they all understand the lecture. The sets that we shall consider in each particular discussion are subsets of a corresponding set U , which we shall call the *universe*.

Example In the rolling of a die, the universe of the possible outcomes is the set consisting of all six faces of the die, $f_1, f_2, \dots, f_6$, i.e.,

$$U = \{f_1, f_2, f_3, f_4, f_5, f_6\}.$$

Clearly, U has $2^6 = 64$ subsets, namely,

$$\phi, \{f_1\}, \dots, \{f_6\}, \{f_1, f_2\}, \dots, \{f_5, f_6\}, \{f_1, f_2, f_3\}, \dots, U.$$

New sets can be generated by operating on existing sets. The *union*, or *sum*, of two sets A and B , designated $A + B$ or $A \cup B$, is the set containing all elements which are members of either A or B or both. The *intersection*, or *product*, of two sets A and B , designated AB or $A \cap B$, is the set containing precisely those elements which are members of both A and B . The *absolute complement* (or simply *complement*) A' of a set A is the set containing the elements of the universe that are not contained in A .

Two sets A and B are *disjoint*, or *mutually exclusive*, if they have no common element, i.e., $AB = \phi$. For example, if we let A be the set of female students, and B be the set of male students, then union $A + B$ yields the entire student

body. The intersection $AB = \phi$ is the null set, for obvious reasons, and since $U = A + B$ then $A' = B$ and $B' = A$.

A common way of describing various sets graphically is by a *Venn diagram*, shown in Fig. 2.1, where the universe is represented by a rectangle, and the elements of the sets are represented by the interiors of the corresponding circles. The intersection and union of A and B are shown by the shaded areas of Figs. 2.1a, b, respectively.

2.2 Relations

The concepts of equivalence relations and partitions, which are presented in this section, are very useful in the study of finite-state machines and are essential for the understanding of their structural properties.

An *ordered pair* (a, b) is a pair of elements with a specific order associated with them. A father and his son, a teacher and a student, are examples of ordered pairs. The first element a is the first *coordinate* of the pair, while the second element b is its second coordinate. A convenient way of describing a set of ordered pairs is by means of a directed graph.

Example The graph of Fig. 2.2 describes the set of ordered pairs

$$\{(a, a), (a, b), (b, a), (b, c), (c, a)\}.$$

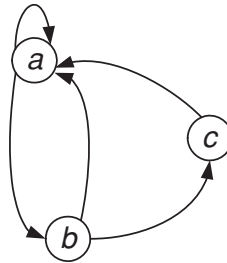


Fig. 2.2 Graphical representation of a set of ordered pairs.

In a similar manner, we define the notion of an *ordered triple* (a, b, c) , where a is the first coordinate, b the second, and c the third. Extending the definition to n elements yields the notion of an *ordered n -tuple* $(a_1, a_2, \dots, a_n)$. The i th element a_i of an ordered n -tuple is referred to as its *i th coordinate*.

It is often necessary to consider sets whose members are ordered pairs. Such a set of ordered pairs is called a *binary relation*. If R is a binary relation and the pair (a, b) is an element of R , we write $a R b$ to indicate that a is *related to* b by R . We often specify relation R by the property that relates the members

of each of its ordered pairs. For example, the binary relation “is less than” is denoted by $a < b$, “is equal to” is denoted by $a = b$, and so on.

If A and B are two sets then the *Cartesian product* of A and B , denoted $A \times B$, is the set containing all ordered pairs (a, b) such that $a \in A$ and $b \in B$. It is evident that any subset of $A \times B$ is a binary relation; it is referred to as a *relation from A to B* .

Example Let $A = \{p, q\}$ and $B = \{r, s, t\}$; then

$$A \times B = \{(p, r), (p, s), (p, t), (q, r), (q, s), (q, t)\}$$

A relation from a set A to A is called a *relation in A* , and it is a subset of the Cartesian product $A \times A$, that is, $R \subseteq A \times A$. The Cartesian product $A \times A$ is usually denoted A^2 , $A \times A \times A$ denoted A^3 , etc. A relation R in a set A is *reflexive* if it contains (a, a) for every $a \in A$; it is *symmetric* if the existence of the ordered pair (a, b) in R implies the existence of (b, a) . A relation is *antisymmetric* if for every ordered pair (a, b) that it contains, where $a \neq b$, it does not contain pair (b, a) . In other words, if both (a, b) and (b, a) are contained in an antisymmetric relation then $a = b$. A relation R is *transitive* if the existence of (b, a) and (a, c) in R implies the existence of (b, c) .

Example The relation $\{(a, a), (b, b), (a, b)\}$ in the set $\{a, b\}$ is reflexive and transitive but not symmetric. The relation $\{(a, b), (b, a)\}$ is symmetric but not transitive, since it does not contain the pair (a, a) that would be implied by the existence of the pairs (a, b) and (b, a) if it were transitive.

A binary relation R in a set S is called an *equivalence relation (in S)* if it is reflexive, symmetric, and transitive. Two elements related by an equivalence relation are said to be *equivalent*.

Example The relation $=$ is an equivalence relation, since it satisfies the following for all a, b , and c in R :

$$\begin{array}{ll} a = a & (\text{reflexivity}) \\ \text{if } a = b \text{ then } b = a & (\text{symmetry}) \\ \text{if } a = b \text{ and } b = c \text{ then } a = c & (\text{transitivity}) \end{array}$$

An equivalence relation actually partitions the elements of a set into *disjoint* subsets such that all members of a subset are equivalent and members of different subsets are not equivalent. These disjoint subsets are called *equivalence classes*, and they play an important role in the study of finite-state machines.

Example The relation of parallelism between lines in a plane is an equivalence relation. In particular, the equivalence relation for the lines in Fig. 2.3 is

$$R = \{(a, a), (b, b), (c, c), (d, d), (e, e), (f, f), (a, b), (b, a), (a, c), (c, a), (b, c), (c, b), (d, e), (e, d)\}$$

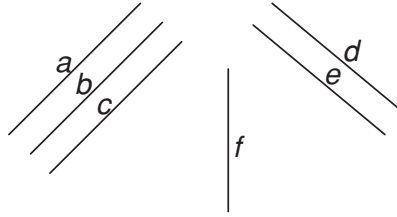


Fig. 2.3 Lines in a plane.

The equivalence classes are $\{a, b, c\}$, $\{d, e\}$, and $\{f\}$, and are together denoted $\{\overline{a, b, c}; \overline{d, e}; \overline{f}\}$.

A relation that is reflexive and symmetric but not transitive is called a *compatibility relation*. Two elements related by a compatibility relation are said to be *compatible*. A consequence of the nontransitivity of a compatibility relation is that it classifies the elements of a set into *nondisjoint* subsets such that all members of a subset are compatible. These subsets are called *compatibility classes*.

Definition 2.1 A *partition* π on a set S is a collection of disjoint subsets whose set union is S . The disjoint subsets are called the *blocks* of π .

The number of blocks in π is denoted $\#(\pi)$, and $\rho(\pi)$ denotes the number of elements in the largest block. If every block of π contains precisely the same number of elements, the partition is said to be *uniform*.

Since an equivalence relation partitions the elements of a set into disjoint subsets, it defines, or *induces*, a partition on that set. For example, the equivalence relation corresponding to Fig. 2.3 induces the partition $\pi = \{\overline{a, b, c}; \overline{d, e}; \overline{f}\}$. It is quite obvious that the converse is also true and that every partition on S defines an equivalence relation in that set.

A binary relation F in a set S of ordered pairs is called a *function* if and only if the existence of two pairs (a, b) and (a, c) in F such that their first coordinates are identical implies that $b = c$. In other words, a function is a set of ordered pairs in which no two pairs have the same first coordinate. A *function from set A to set B* is one which associates with each element a in A exactly one element b in B such that $(a, b) \in F$.

Example If $A = \{a_1, a_2, a_3\}$ and $B = \{b_1, b_2\}$ then $\{(a_1, b_1), (a_2, b_2), (a_3, b_1)\}$ is a function from A to B , while $\{(a_1, b_1), (a_2, b_2), (a_3, b_1), (a_3, b_2)\}$ is not, since it assigns two elements of B to a_3 .

A function from set A to itself is called a *unary operation* in A and serves to assign to every element in A a unique element from A . Similarly, a *binary operation* is a function from A^2 to A and assigns to every ordered pair of A^2 a unique element from A . In general, an *n -ary operation* in A is a function from A^n to A .

Example Consider a set S of positive real numbers. The function *square root* is a unary operation which assigns to each a in S an element $\sqrt{a}$ from S . Addition and multiplication are examples of binary operations.

2.3 Partially ordered sets

A reflexive, antisymmetric, and transitive binary relation is called a *partial ordering*. A set S together with a partial ordering relation is referred to as a *partially ordered set*. A very useful example of partial ordering is the “is less than or equal to” relation. If (a, b) is an element of a partially ordered set, we usually say that a is less than or equal to b even if no numerical values are associated with a or b .

Example The partial ordering $\leq$ satisfies the following for all a, b , and c in S :

$$\begin{aligned} a &\leq a && (\text{reflexivity}) \\ a \leq b \text{ and } b \leq a &\text{ imply } a = b && (\text{antisymmetry}) \\ \text{if } a \leq b \text{ and } b \leq c, &\text{ then } a \leq c && (\text{transitivity}) \end{aligned}$$

A partition π_1 on S is said to be smaller than or equal to π_2 on S , denoted $\pi_1 \leq \pi_2$, if and only if each pair of elements that are in a common block of π_1 are also in a common block of π_2 . Two partitions π_1 and π_2 are said to be *incomparable* if neither $\pi_1 \leq \pi_2$ nor $\pi_2 \leq \pi_1$ is true.

Example Consider a set S and three partitions on S :

$$\begin{aligned} S &= \{a, b, c, d, e, f, g, h, i\}, \\ \pi_1 &= \{\overline{a, b}; \overline{c, d}; \overline{e, f}; \overline{g, h, i}\}, \\ \pi_2 &= \{\overline{a, f}; \overline{b, c}; \overline{d, e}; \overline{g, h, i}\}, \\ \pi_3 &= \{\overline{a, b, e, f}; \overline{c, d}; \overline{g, h, i}\}. \end{aligned}$$

Clearly $\pi_1 \leq \pi_3$, but π_1 and π_2 are incomparable as are π_2 and π_3 .

If, for every pair of elements $a, b \in S$, either $a \leq b$ or $b \leq a$ then set S is *totally ordered* by the binary relation $\leq$. For example, the set of all prime numbers is totally ordered by the $\leq$ relation. However, the set of partitions $\{\pi_1, \pi_2, \pi_3\}$ defined in the preceding example is partially ordered, since no ordering by the relation $\leq$ exists between π_1 and π_2 .

A convenient way of displaying the ordering relation among the elements of an ordered set S is by means of a graph whose vertices represent the elements of the set. Vertex a is drawn at a higher level than vertex b whenever $b < a$, that is, $b \leq a$ but $b \neq a$. Vertex a is at a higher level immediately adjacent to vertex b if $b < a$ and there is no element c in S such that $b < c < a$. In such cases, a is said to *cover* b . The graph is called a *Hasse graph* or *Hasse diagram*.

It is always possible to reconstruct a partial ordering from the Hasse diagram. This is accomplished by observing that each upward path from vertex b to vertex a corresponds to $b < a$, which in turn may be denoted $b \leq a$.

Example The partial ordering displaying the divisibility relation among all positive integers dividing number 45, such that the quotient is an integer, is shown in Fig. 2.4a.

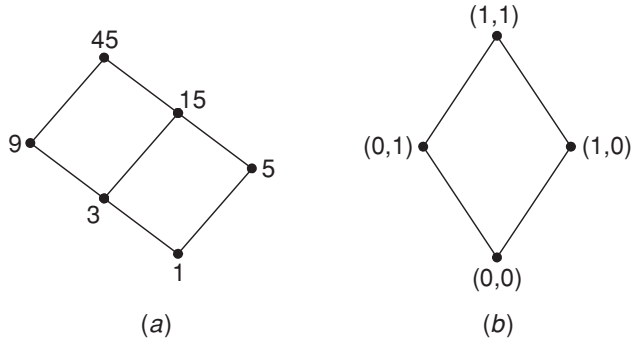


Fig. 2.4 Hasse diagrams for partially ordered sets.

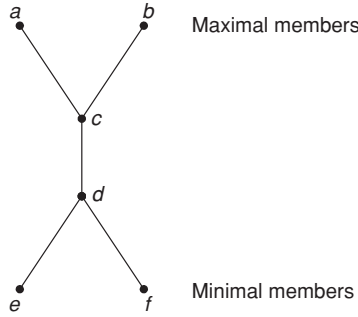
Example Let $S = \{(0, 0), (0, 1), (1, 0), (1, 1)\}$ and define an ordering relation as follows:

$$(a_1, a_2) \leq (b_1, b_2) \text{ if and only if } a_1 \leq b_1 \text{ and } a_2 \leq b_2$$

Clearly, S is not a totally ordered set under this ordering, since $(0,1)$ and $(1,0)$ are not related. The graphical description of the partial ordering is given in Fig. 2.4b.

Consider a partially ordered set S and a given relation $\leq$. If $a \leq b$ for every element b in S then a is said to be the *least member* of the set S . Not every set has a least member (see, for example, Fig. 2.5), but whenever it does exist

Fig. 2.5 A Hasse diagram without least or greatest elements.



it is unique. In order to prove the uniqueness of the least member, assume that for some S , there exist two least members, a_1 and a_2 . Since $a_1 \leq b$ for every element b in S , then $a_1 \leq a_2$. Similarly, since $a_2 \leq b$, then $a_2 \leq a_1$. Consequently, $\leq$ being an antisymmetric relation, $a_1 = a_2$. Similarly, if $b \leq a$ for all b in S , then a is said to be the *greatest member* of S and, if such a member exists, it is unique. In the two graphs of Fig. 2.4, the least and greatest elements are shown at the lowest and highest levels, respectively.

Whenever a least member does not exist, it is convenient to define a *minimal member* a in S such that for no b in S is $b < a$; that is, there is no smaller element but there may exist another unrelated minimal member in S . A *maximal member* in S is similarly defined (see Fig. 2.5).

Let S be a partially ordered set, and let P be a subset of S ; then an element s in S is an *upper bound* of P if and only if, for every p in P , $p \leq s$. An element s in S is a *lower bound* of P if and only if, for every p in P , $s \leq p$. Note that s is not necessarily a member of P . An upper bound s of P is said to be the *least upper bound* (lub) if $s \leq s'$ for all upper bounds s' of P . Similarly, the lower bound s in S is called the *greatest lower bound* (glb) if and only if, for all lower bounds s' of P , $s' \leq s$.

Example Consider the subset $P = \{3, 5\}$ of the set $S = \{1, 3, 5, 9, 15, 45\}$ illustrated in Fig. 2.4a. The upper bounds are 15 and 45; the lub is 15. The glb is 1. In the partially ordered set illustrated in Fig. 2.5, the subset $P = \{a, b\}$ has no upper bound but four lower bounds, c, d, e , and f , of which c is the glb. For subset $P = \{b, f\}$, b is the lub while f is the glb.

2.4 Lattices

Lattices play an important role in the characterization of various computation models. In particular, it will be shown later that a Boolean algebra is nothing other than a lattice with a few specific properties.

Definition 2.2 A partially ordered set in which every pair of elements has a unique glb and a unique lub is called a *lattice*.

Example The partially ordered sets described in Fig. 2.4 are lattices, while the partially ordered set described in Fig. 2.5 is not.

A consequence of Definition 2.2 is that each finite lattice has both a least and a greatest element, which are denoted 0 and 1, respectively. Thus, for each element a of the lattice,

$$a \leq 1 \quad \text{and} \quad 0 \leq a.$$

Example The lattice of all subsets of set $S = \{a, b, c\}$, under the ordering relation of set inclusion, is shown in Fig. 2.6, where $\{a, b, c\} = 1$ and $\phi = 0$.

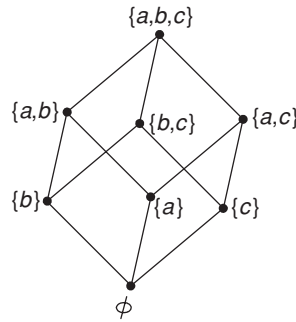


Fig. 2.6 Lattice of the subsets of $\{a, b, c\}$.

Because of the uniqueness of the lub and glb, they may be viewed as binary operations that assign to each ordered pair of elements their lub and glb. The first operation, called the *sum* or *join*, is denoted by $+$ or $\vee$; the second operation, called the *product* or *meet*, is denoted by $\wedge$ or $\cdot$. Thus,

$$a + b = \text{lub}(a, b),$$

$$a \cdot b = \text{glb}(a, b).$$

By definition, the lub and glb satisfy the idempotent and commutative laws, since

$$\begin{aligned} a \cdot a &= a + a = a && (\text{idempotency}), \\ a \cdot b &= b \cdot a \quad \text{and} \quad a + b = b + a && (\text{commutativity}). \end{aligned}$$

In addition, they satisfy the absorption law and are associative, since

$$\begin{aligned} a + a \cdot b &= a \quad \text{and} \quad a \cdot (a + b) = a && (\text{absorption}), \\ a \cdot (b \cdot c) &= (a \cdot b) \cdot c \quad \text{and} \quad a + (b + c) = (a + b) + c && (\text{associativity}). \end{aligned}$$

In order to prove the validity of the absorption law, recall that $a \cdot b$ defines the glb of a and b , and thus $a \cdot b \leq a$. Hence $a + a \cdot b$, which defines the lub of a and $a \cdot b$, is clearly a . The dual property is verified in an analogous manner. The proof that the operations are associative is left to the reader as an exercise (see Problem 2.3).

The following properties are valid for every finite lattice:

$$a + 0 = a, \quad a \cdot 0 = 0,$$

and

$$a \cdot 1 = a, \quad a + 1 = 1.$$

The duality of the idempotent through associative laws, as well as that of the foregoing operations with the least and greatest elements, is apparent and will be further discussed in conjunction with the subject of Boolean algebras.

Now consider the partially ordered set whose elements are partitions. Define as the *greatest partition* that containing just a single block and as the *least partition* that containing as many blocks as elements, i.e., where each block contains just a single element. These partitions are designated $\pi(I)$ and $\pi(0)$, respectively. The binary operations of lub and glb are applied to the partitions in the following manner. The sum (or join) $\pi_1 + \pi_2$ is obtained by including in every block those elements of π_1 and π_2 that are chain-connected;¹ the product (or meet) $\pi_1 \cdot \pi_2$ is obtained by finding the intersection of the blocks of individual partitions. As a consequence, under the above-defined operations the set of all partitions constitutes a lattice. It can be shown that these sum and product operations follow directly from the partition inclusion relation and indeed yield the lub and glb, respectively. However, the proof is beyond the scope of this book.

Example Let $\pi_1 = \{\overline{a, b}; \overline{c, d, e}; \overline{f, h}; \overline{g, i}\}$ and $\pi_2 = \{\overline{a, b, c}; \overline{d, e}; \overline{f, g}; \overline{h, i}\}$; then

$$\pi_1 + \pi_2 = \{\overline{a, b, c, d, e}; \overline{f, g, h, i}\}$$

and

$$\pi_1 \cdot \pi_2 = \{\overline{a, b}; \overline{c}; \overline{d, e}; \overline{f}; \overline{g}; \overline{h, i}\}.$$

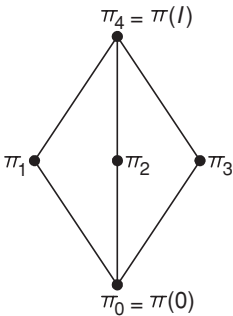


Fig. 2.7 A nondistributive lattice.

The distributive law is not necessarily valid for arbitrary lattices, as shown by the lattice in Fig. 2.7. A lattice is said to be *distributive* if and only if

$$\begin{aligned} a \cdot (b + c) &= a \cdot b + a \cdot c, \\ a + (b \cdot c) &= (a + b)(a + c). \end{aligned}$$

¹ Two subsets (or blocks) S_1 and S_n are said to be *chain-connected* if and only if there exists a sequence of subsets $S_1, S_2, \dots, S_n$ such that $S_i \cdot S_{i+1} \neq \phi$, $i = 1, 2, \dots, n - 1$.

Example Consider the following set of partitions:

$$\begin{aligned}\pi_0 &= \{\overline{a}; \overline{b}; \overline{c}\} = \pi(0), & \pi_1 &= \{\overline{a}, \overline{b}; \overline{c}\}, & \pi_2 &= \{\overline{a}; \overline{b}, \overline{c}\}, \\ \pi_3 &= \{\overline{a}, \overline{c}; \overline{b}\}, & \pi_4 &= \{\overline{a}, \overline{b}, \overline{c}\} = \pi(I).\end{aligned}$$

The product $\pi_1 \cdot (\pi_2 + \pi_3) = \pi_1$, but $\pi_1 \cdot \pi_2 + \pi_1 \cdot \pi_3 = \pi_0$; consequently, the lattice, which is shown in Fig. 2.7, is not distributive.

If, for each element a in the lattice, there exists an element a' such that

$$a \cdot a' = 0 \quad \text{and} \quad a + a' = 1$$

then the lattice is said to be *complemented*. The element a' is said to be a *complement* of a , and vice versa. For example, the lattice of subsets of $\{a, b, c\}$ shown in Fig. 2.6 is complemented as well as distributive.

Notes and references

The material covered in this chapter is available in many good books on algebra; among these are Birkhoff and MacLane [2] and Mostow, Sampson, and Meyer [3]. A classical reference, though an advanced one, is *Lattice Theory* by Birkhoff [1].

- [1] Birkhoff, G.: *Lattice Theory*, American Mathematical Society Colloquium Publications, vol. 25, Providence RI, 1948.
- [2] Birkhoff, G., and S. MacLane: *A Survey of Modern Algebra*, third edition, Macmillan, New York, 1965.
- [3] Mostow, G. D., J. H. Sampson, and J. Meyer: *Fundamental Structures of Algebra*, McGraw-Hill, New York, 1963.

Problems

Problem 2.1. In an examination there are three problems, A , B , and C . The following tabulation gives the percentages of students who received credit for solving one or more problems:

$$\begin{array}{lll} A, & 40; & A, B, & 12; & A, B, C, & 4. \\ B, & 30; & A, C, & 8; \\ C, & 30; & B, C, & 6; \end{array}$$

(For example, “ $A, B, 12$ ” means that 12% of the students received credit for both problem A and problem B). What percent of students received no credit at all for any of the three problems?

Hint: Use a Venn diagram.

Problem 2.2. Consider a set of triangles $S = \{A, B, \dots\}$ in a plane. What kind of relations are the following, and what properties do they have, e.g., are they reflexive, symmetric, etc.?

For every two triangles A and B in S , A and B are related if and only if:

- (a) A is congruent to B ;
- (b) A has area in common with B ;
- (c) A is similar to B ;
- (d) A is entirely inside, or the same as, B ;
- (e) A has a side equal to or smaller than the smallest side of B ;
- (f) A has a side equal to or smaller than the smallest side of B , but has at least as much area as B .

Problem 2.3. Prove that the lub and glb operations are associative; that is, for all a, b , and c of any lattice,

$$a + (b + c) = (a + b) + c \quad \text{and} \quad a \cdot (b \cdot c) = (a \cdot b) \cdot c.$$

Hint: Use the uniqueness of the lub and glb of (a, b, c) .

Problem 2.4. The set $\{a, b, c, d, e, f, g, h, i, j, k\}$ has the partitions

$$\pi_1 = \{\overline{a, b, c}, \overline{d, e}, \overline{f}, \overline{g, h, i}, \overline{j, k}\},$$

$$\pi_2 = \{\overline{a, b}, \overline{c, g, h}, \overline{d, e, f}, \overline{i, j, k}\},$$

$$\pi_3 = \{\overline{a, b, c, f}, \overline{d, e}, \overline{g, h, i, j, k}\}.$$

- (a) Find $\pi_1 + \pi_2$ and $\pi_1 \cdot \pi_2$.
- (b) Find $\pi_1 + \pi_3$ and $\pi_1 \cdot \pi_3$.
- (c) Find a partition that is greater than π_1 and smaller than π_3 .
- (d) Can you find a partition that is greater than π_2 and smaller than π_3 ?

Problem 2.5. Prove that if a complemented lattice is not distributive then the complements of its elements are not necessarily unique. Conversely, if for some element in the lattice the complement is not unique then the lattice is not distributive.

Problem 2.6. For each lattice given in Fig. P2.6, determine whether it is distributive and/or complemented. If the lattice is complemented, identify the complementary elements. Which diagram corresponds to a total ordering?

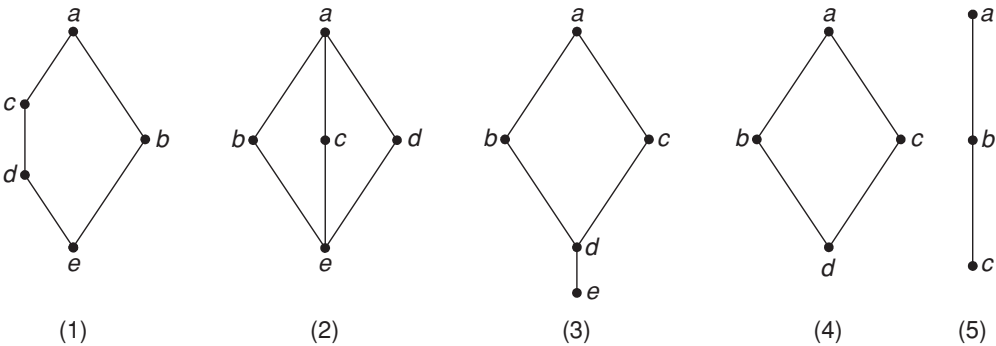


Fig. P2.6

Part 2 **Combinational logic**

3

Switching algebra and its applications

The second part of this book is devoted to *combinational logic* and deals with various aspects of the analysis and design of combinational switching circuits. The particular characteristic of a *combinational switching circuit* is that its outputs are functions of only the present circuit inputs. First, switching algebra is introduced as the basic mathematical tool essential for dealing with problems encountered in the study of switching circuits. Switching expressions are defined and are found to be instrumental in describing the logical properties of switching circuits. Systematic simplification procedures of these expressions are next presented; these lead to more economical circuits. Logical design is studied with special attention to conventional logic, complementary metal-oxide semiconductor (CMOS) circuits, and threshold logic. Finally, problems related to the testing of combinational circuits for various fault models, and synthesis-for-testability techniques are discussed.

In the current chapter, after developing a switching algebra from the simplest set of basic postulates we show its applications to the study of switching circuits as well as to the calculus of propositions. Finally, this switching algebra is shown to be a special case of Boolean algebra.

3.1 Switching algebra

The basic concepts of switching algebra will be introduced by means of a set of postulates, from which we shall derive useful theorems and develop necessary tools that will enable us to manipulate and simplify algebraic expressions.

Fundamental postulates

The basic postulate of switching algebra is the existence of a two-valued switching variable that can take either of two distinct values, 0 and 1. Precisely stated,

if x is a switching variable then

$$x \neq 0 \quad \text{if and only if} \quad x = 1,$$

$$x \neq 1 \quad \text{if and only if} \quad x = 0.$$

These values are often referred to as the *truth values* of x .

A *switching algebra* is an algebraic system consisting of the set $\{0, 1\}$, two binary¹ operations called OR and AND, denoted by the symbols $+$ and $\cdot$ respectively, and one unary operation called NOT, denoted by a prime.

The definitions of the OR and AND operations are as follows:

<i>OR operation</i>	<i>AND operation</i>
$0 + 0 = 0,$	$0 \cdot 0 = 0,$
$0 + 1 = 1,$	$0 \cdot 1 = 0,$
$1 + 0 = 1,$	$1 \cdot 0 = 0,$
$1 + 1 = 1.$	$1 \cdot 1 = 1.$

Thus the OR combination of two switching variables $x + y$ is equal to 1 if the value of either x or y is 1 or if the values of both x and y are 1. The AND combination of these variables $x \cdot y$ is equal to 1 if and only if the values of x and y are both equal to 1. The result of the OR operation is very often called the (*logical*) *sum* or *union* and may be denoted by $\cup$ or $\vee$. The result of the AND operation is referred to as the (*logical*) *product* or *intersection*, and is denoted by $\cap$ or $\wedge$. We shall generally omit the dot $\cdot$ and write xy to mean $x \cdot y$.

The NOT operation, which is also known as *complementation*, is defined as follows:

$$0' = 1,$$

$$1' = 0.$$

The preceding postulates and definitions of switching operations enable us to derive many useful theorems and develop an entire algebraic structure that may be advantageously applied to switching circuits.

Basic properties

The first property that drastically differs from the algebra of real numbers and accounts for the special characteristics of switching algebra, is the idempotent law for a switching variable x :

$$x + x = x, \quad (\text{idempotency}). \quad (3.1)$$

$$x \cdot x = x \quad (3.2)$$

¹ A *binary operation* on a set of elements is a rule that assigns a unique element from the set to each ordered pair of elements from the set. A *unary operation* is a rule which assigns to every element in the set another element from the set (see Section 2.2).

To prove this property, we shall employ *perfect induction*. Perfect induction is a method of proof whereby a theorem is verified for every possible combination of values that the variables may assume. Since x is a two-valued variable, $x + x = x$ may assume the values $1 + 1 = 1$ and $0 + 0 = 0$. These equations, being identities, clearly verify the validity of Eq. (3.1), and similarly for Eq. (3.2) we have $1 \cdot 1 = 1$ and $0 \cdot 0 = 0$.

If x is a switching variable, then

$$x + 1 = 1, \quad (3.3)$$

$$x \cdot 0 = 0, \quad (3.4)$$

$$x + 0 = x, \quad (3.5)$$

$$x \cdot 1 = x. \quad (3.6)$$

The following two pairs of relations establish the commutativity and associativity of switching operations. The convention adopted for parenthesizing is that of ordinary algebra, where $x + y \cdot z$ means $x + (y \cdot z)$ and not $(x + y) \cdot z$. Let x , y , and z be switching variables. Then

$$x + y = y + x, \quad (3.7)$$

$$x \cdot y = y \cdot x \quad (\text{commutativity}). \quad (3.8)$$

$$(x + y) + z = x + (y + z), \quad (3.9)$$

$$(x \cdot y) \cdot z = x \cdot (y \cdot z) \quad (\text{associativity}). \quad (3.10)$$

In addition, for every switching variable x ,

$$x + x' = 1, \quad (3.11)$$

$$x \cdot x' = 0 \quad (\text{complementation}). \quad (3.12)$$

The properties established by Eqs. (3.2) through (3.12) can be proved by the method of perfect induction. The actual proofs are left to the reader as exercises. It is the associative law which enables us to extend the definitions of the AND and OR operations to more than two variables, i.e., we write $T = x + y + z$ to mean that T equals 1 if any of x , y , or z , or any combination thereof, equals 1.

In switching algebra, multiplication distributes over addition and addition distributes over multiplication – a property known as the distributive law:

$$x \cdot (y + z) = x \cdot y + x \cdot z, \quad (3.13)$$

$$x + y \cdot z = (x + y) \cdot (x + z) \quad (\text{distributivity}). \quad (3.14)$$

To verify Eq. (3.13) for every possible combination of values of x , y , and z , it is convenient to tabulate these combinations in a table called a *truth table* or *table of combinations*. Since every variable may assume one of two values, 0 or 1, the truth table for the three variables contains $2^3 = 8$ combinations. These combinations are tabulated in the leftmost column of Table 3.1. The value of $x(y + z)$ is computed for every possible combination of x and $y + z$. The value

Table 3.1 Proof by perfect induction of Eq. (3.13)

x	y	z	xy	xz	$y + z$	$x(y + z)$	$xy + xz$
0	0	0	0	0	0	0	0
0	0	1	0	0	1	0	0
0	1	0	0	0	1	0	0
0	1	1	0	0	1	0	0
1	0	0	0	0	0	0	0
1	0	1	0	1	1	1	1
1	1	0	1	0	1	1	1
1	1	1	1	1	1	1	1

of $xy + xz$ is computed independently by adding the entries in columns xy and xz . Since the two different methods of computation yield identical results, as shown in the two rightmost columns, Eq. (3.13) is verified.

We observe that all the preceding properties are grouped in pairs. Within each pair, one statement can be obtained from the other by interchanging the OR and AND operations and replacing the constants 0 and 1 by 1 and 0, respectively. Any two statements or theorems that have this property are called *dual*, and this quality of duality that characterizes switching algebra is known as the *principle of duality*. It stems from the symmetry of the postulates and definitions of switching algebra with respect to the two operations and two constants. The implication of the concept of duality is that it is necessary to prove only one of each pair of statements because its dual is, henceforth, proved.

Switching expressions and their manipulation

By a *switching expression* we mean the combination of a finite number of switching variables (x , y , etc.) and constants (0, 1) by means of switching operations (+, ·, and '). More precisely, any switching constant or variable is a switching expression, and if T_1 and T_2 are switching expressions then so are T_1' , T_2' , $T_1 + T_2$, and $T_1 T_2$. No other combinations of variables and constants are switching expressions.

The properties to be presented below in Eqs. (3.15) through (3.20) provide the basic tools for the simplification of switching expressions. They establish the notion of redundancy and, like all the preceding properties, they appear in dual forms. Equation (3.15) and its dual (3.16) express the *absorption law* of switching algebra.

$$x + xy = x, \quad (3.15)$$

$$x(x + y) = x \quad (\text{absorption}). \quad (3.16)$$

The method of proof by perfect induction is efficient, as long as the number of combinations for which the statement is to be verified is small. In other

cases, algebraic procedures are more appropriate, such, for example, as are demonstrated in the following proof of Eq. (3.15).

Proof We have

$$\begin{aligned}
 x + xy &= x1 + xy && \text{(by Eq. (3.6))} \\
 &= x(1 + y) && \text{(by Eq. (3.13))} \\
 &= x1 && \text{(by Eqs. (3.3) and (3.7))} \\
 &= x && \text{(by Eq. (3.6)).}
 \end{aligned}
 \quad \diamond$$

Another property of switching expressions, important in their simplification, is the following:

$$x + x'y = x + y, \quad (3.17)$$

$$x(x' + y) = xy. \quad (3.18)$$

Equation (3.17) is proved as follows.

Proof We have

$$\begin{aligned}
 x + x'y &= (x + x')(x + y) && \text{(by Eq. (3.14))} \\
 &= 1(x + y) && \text{(by Eq. (3.11))} \\
 &= x + y && \text{(by Eqs. (3.6) and (3.8)).}
 \end{aligned}
 \quad \diamond$$

The **consensus theorem** is noteworthy in that it is used frequently in the simplification of switching expressions. It is stated in the following two equations:

$$\begin{aligned}
 &\boxed{xy + x'z + yz = xy + x'z}, && \text{(consensus theorem).} && (3.19) \\
 (x + y)(x' + z)(y + z) &= (x + y)(x' + z) && (3.20)
 \end{aligned}$$

The extra term yz in Eq. (3.19) is known as the *consensus*.

Proof We can manipulate the left-hand side of Eq. (3.19) as follows:

$$\begin{aligned}
 xy + x'z + yz &= xy + x'z + yz1 \\
 &= xy + x'z + yz(x + x') \\
 &= xy + x'z + xyz + x'yz \\
 &= xy(1 + z) + x'z(1 + y) \\
 &= xy + x'z.
 \end{aligned}
 \quad \diamond$$

The preceding properties permit a variety of manipulations on switching expressions. In particular, they enable us (whenever possible) to convert an expression into an equivalent one with fewer literals, where by a *literal* we mean an appearance of a variable or its complement. For example, while the left-hand side of Eq. (3.19) consists of six literal appearances, its right-hand side consists of only four appearances. If the value of a switching expression is independent of the value of some literal x_i , then x_i is said to be *redundant*. Equations (3.1) through (3.20) provide, among other things, the tools for manipulating expressions so as to eliminate redundant literals.

Example Simplify the expression $T(x, y, z) = x'y'z + yz + xz$ by eliminating redundant literals.

$$\begin{aligned}
 x'y'z + yz + xz &= z(x'y' + y + x) \\
 &= z(x' + y + x) \\
 &= z(y + 1) \\
 &= z1 \\
 &= z.
 \end{aligned}$$

Hence, $T(x, y, z)$ is actually independent of the values of x and y and depends only on z .

It is important to observe that no inverse operations are defined in switching algebra and, consequently, no cancellations are allowed. For example, if $A + B = A + C$, the equality of B and C is not implied; in fact, if $A = B = 1$ and $C = 0$ then $1 + 1 = 1 + 0$, but $B \neq C$. Similarly, B is not necessarily equal to C if $AB = AC$.

De Morgan's theorems

The rules governing complementation operations are summarized by three theorems. The first is the involution theorem:

$$(x')' = x \quad (\text{involution}). \quad (3.21)$$

Proof Equation (3.21) is obvious by perfect induction. $\diamond$

De Morgan's theorems for two variables are

$$(x + y)' = x' \cdot y', \quad (3.22)$$

$$(x \cdot y)' = x' + y'. \quad (3.23)$$

Proof The proof of Eq. (3.22) follows by perfect induction, using the truth table of Table 3.2; $(x + y)'$ and $x' \cdot y'$ are computed independently and are shown to be identical for all possible combinations of values of x and y . The proof of Eq. (3.23) then follows by the principle of duality. $\diamond$

Table 3.2 Truth table for the proof of Eq. (3.22)

x	y	x'	y'	$x + y$	$(x + y)'$	$x' \cdot y'$
0	0	1	1	0	1	1
0	1	1	0	1	0	0
1	0	0	1	1	0	0
1	1	0	0	1	0	0

For n variables, Eqs. (3.22) and (3.23) can be expressed as follows: *the complement of any expression can be obtained by replacing each variable and element with its complement and, at the same time, interchanging the OR and AND operations*, that is,

$$[f(x_1, x_2, \dots, x_n, 0, 1, +, \cdot)]' = f(x_1', x_2', \dots, x_n', 1, 0, \cdot, +). \quad (3.24)$$

Equation (3.24) is known as the *general De Morgan's theorem* and its proof follows immediately from Eq. (3.22) and mathematical induction on the number of operations.

Example In order to simplify the expression

$$T(x, y, z) = (x + y)[x'(y' + z')]' + x'y' + x'z',$$

it is necessary first to apply De Morgan's theorem and then to multiply out the expressions in parentheses:

$$\begin{aligned} T(x, y, z) &= (x + y)(x + yz) + x'y' + x'z' \\ &= (x + xyz + yx + yz) + x'y' + x'z' \\ &= x + yz + x'y' + x'z' \\ &= x + yz + y' + z' \\ &= x + z + y' + z' \\ &= x + y' + 1 \\ &= 1. \end{aligned}$$

Hence, $T = 1$ independently of the values of the variables.

Example Prove the following identity:

$$xy + x'y' + yz = xy + x'y' + x'z.$$

From the application of Eq. (3.19) to $x'y' + yz$, it follows that the term $x'z$ may be added to the left-hand side of the equation; i.e., the equation becomes

$$xy + x'y' + yz + x'z = xy + x'y' + x'z.$$

Another application of Eq. (3.19) to the first, third, and fourth terms in the augmented left-hand side of the equation shows that yz is redundant. After elimination of yz , the left-hand side of the equation is identical to its right-hand side (i.e., both consist of identical terms), and thus the proof is complete.

Table 3.3 Truth table for $T(x, y, z) = x'z + xz' + x'y'$

x	y	z	T
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

3.2 Switching functions

Definitions

Let $T(x_1, x_2, \dots, x_n)$ be a switching expression. Since each of the variables $x_1, x_2, \dots, x_n$ can independently assume either of the two values 0 or 1, there are 2^n combinations of values to be considered in determining the values of T . In order to determine the value of an expression for a given combination, it is only necessary to substitute the values for the variables in the expression. For example, if $T(x, y, z) = x'z + xz' + x'y'$ then, for the combination $x = 0, y = 0, z = 1$, the value of the expression is 1 because $T(0, 0, 1) = 0'1 + 01' + 0'0' = 1$. In a similar manner, the value of T may be computed for every combination, as shown in the right-hand column of Table 3.3.

If we now repeat the above procedure and construct the truth table for the expression $x'z + xz' + y'z'$, we find that it is identical to that of Table 3.3. Hence, for every possible combination of variables, the value of the expression $x'z + xz' + x'y'$ is identical to the value of $x'z + xz' + y'z'$. Thus different switching expressions may represent the same assignment of values specified by the right-hand column of a truth table. The values assumed by an expression for all the combinations of variables $x_1, x_2, \dots, x_n$ define a switching function. In other words, a *switching function* $f(x_1, x_2, \dots, x_n)$ is a correspondence that associates an element of the algebra with each of the 2^n combinations of variables $x_1, x_2, \dots, x_n$. This correspondence is best specified by means of a truth table. Note that each truth table defines only one switching function, although this function may be expressed in a number of ways.

The complement $f'(x_1, x_2, \dots, x_n)$ is a function whose value is 1 whenever the value of $f(x_1, x_2, \dots, x_n)$ is 0, and 0 whenever the value of f is 1. The sum of two functions $f(x_1, x_2, \dots, x_n)$ and $g(x_1, x_2, \dots, x_n)$ is 1 for every combination in which either f or g or both equal 1, while their product is equal

to 1 if and only if both f and g equal 1. If a function $f(x_1, x_2, \dots, x_n)$ is specified by means of a truth table, its complement is obtained by complementing each entry in the column headed f . New functions that are equal to the sum $f + g$ and the product fg are obtained by adding or multiplying the corresponding entries in the f and g columns.

Example Two functions $f(x, y, z)$ and $g(x, y, z)$ are specified in columns f and g of Table 3.4. The complement f' , the sum $f + g$, and the product fg are specified in the corresponding columns.

Table 3.4 Illustration of the addition, multiplication, and complementation of switching functions

x	y	z	f	g	f'	$f + g$	fg
0	0	0	1	0	0	1	0
0	0	1	0	1	1	1	0
0	1	0	1	0	0	1	0
0	1	1	1	1	0	1	1
1	0	0	0	1	1	1	0
1	0	1	0	0	1	0	0
1	1	0	1	1	0	1	1
1	1	1	1	0	0	1	0

Simplification of expressions

The truth table assigns to each combination of variable values a specific switching element. Consequently, all the properties of switching elements (Eqs. (3.1) through (3.24)) are valid when the elements are replaced by expressions. For example, $xy + xyz = xy$ by virtue of the property established in Eq. (3.15).

Example Simplify the expression

$$T(A, B, C, D) = A'C' + ABD + BC'D + AB'D' + ABCD'.$$

First, apply the consensus theorem, Eq. (3.19), to the first three terms of T , letting x , y , and z replace A' , C' , and BD , respectively. As a result the third term, $BC'D$, is redundant. Next, apply the distributive law, Eq. (3.13), to the fourth and fifth terms. This gives the expression $AD'(B' + BC)$. Letting x and y replace B' and C , respectively, and applying Eq. (3.17) yields $AD'(B' + C)$. No other literal is redundant; thus the simplest expression for T is

$$T = A'C' + A[BD + D'(B' + C)].$$

Example Simplify the expression

$$T(A, B, C, D) = A'B + ABD + AB'CD' + BC.$$

First apply Eq. (3.17) to the first two terms and to the last two terms. This yields

$$T = A'B + BD + ACD' + BC.$$

The next step in the simplification is not as obvious; in order to simplify T , it is first necessary to expand it. Since $BC = (A + A')BC$ we have

$$T = A'B + BD + ACD' + ABC + A'BC.$$

The application of Eq. (3.15) to the first and last terms results in the elimination of the last term. Now apply Eq. (3.19) to the second, third, and fourth terms, letting x , y , and z replace D , B , and AC , respectively. This step eliminates ABC and yields

$$T = A'B + BD + ACD'.$$

Canonical forms

Truth tables have been shown to be the means for describing switching functions. An expression representing a switching function is derived from the table by finding the sum of all the terms that correspond to those combinations (i.e., rows) for which the function assumes the value 1. Each term is a product of the variables on which the function depends. Variable x_i appears in uncomplemented form in the product if it has value 1 in the corresponding combination, and it appears in complemented form if it has value 0. For example, the product term that corresponds to row 3 of Table 3.5, where the values of x , y , and z are 0, 1, and 1, is $x'yz$. The sum of all product terms for the function defined by Table 3.5 is

$$f(x, y, z) = x'y'z' + x'yz' + x'yz + xyz' + xyz.$$

A product term that, as for each term in the above expression, contains each of the n variables as factors in either complemented or uncomplemented form is called a *minterm*. Its characteristic property is that it assumes the value 1 for exactly one combination of variables. If we assign to each of the n variables a fixed arbitrary value, either 0 or 1, then, of the 2^n minterms, one and only one minterm will have value 1 while all the remaining $2^n - 1$ minterms will have value 0, because they differ by at least one literal, whose value is 0, from the minterm whose value is 1. The sum of all minterms derived from those rows for which the value of the function is 1 takes on the value 1 or 0 according to the value assumed by f .

Table 3.5 Truth table for function $f(x, y, z) = x'y'z' + x'yz' + x'yz + xyz' + xyz$

Decimal code	x	y	z	f
0	0	0	0	1
1	0	0	1	0
2	0	1	0	1
3	0	1	1	1
4	1	0	0	0
5	1	0	1	0
6	1	1	0	1
7	1	1	1	1

Therefore, this sum is in fact an algebraic representation of f . An expression of this type is called a *canonical sum of products* or *disjunctive normal expression*.

Switching functions are usually expressed in a compact form, obtained by listing the decimal codes associated with the minterms for which $f = 1$. The decimal codes are derived from the truth tables by regarding each row as a binary number; e.g., the minterm $x'y'z'$ is associated with row 010, which, when interpreted as a binary number, is equal to 2. The function defined by Table 3.5 can thus be expressed as

$$f(x, y, z) = \sum(0, 2, 3, 6, 7)$$

where $\sum()$ means that $f(x, y, z)$ is the sum of all the minterms whose decimal code is one of the numbers given within the parentheses.

A switching function can also be expressed as a product of sums. This is accomplished by considering those combinations for which the function is required to have the value 0. For example, the sum term $x + y + z'$ has the value 1 for all combinations of x , y , and z , except for $x = 0$, $y = 0$, and $z = 1$, when it has the value 0. Any similar term assumes the value 0 for only one combination. Consequently, a product of such sum terms will assume the value 0 for precisely those combinations for which the individual terms are 0. For all other combinations, the product-of-sum terms will have the value 1. A sum term that contains each of the n variables in either a complemented or an uncomplemented form is called a *maxterm*. An expression formed of the product of all maxterms for which the function takes on the value 0 is called a *canonical product of sums* or *conjunctive normal expression*.

In each maxterm, a variable x_i appears in uncomplemented form if it has the value 0 in the corresponding row in the truth table, and it appears in complemented form if it has the value 1. For example, the maxterm that corresponds to the row whose decimal code is 1 in Table 3.5 is $x + y + z'$. The canonical

product-of-sums expression for the function defined by Table 3.5 is given by

$$f(x, y, z) = (x + y + z')(x' + y + z)(x' + y + z').$$

This function can also be expressed in a compact form by listing the combinations for which f is to have value 0, i.e.,

$$f(x, y, z) = \prod(1, 4, 5),$$

where $\prod()$ means the product of all maxterms whose decimal code is given within the parentheses.

One way of obtaining the canonical forms of any switching function is by means of *Shannon's expansion theorem* (also called *Shannon's decomposition theorem*), which states that any switching function $f(x_1, x_2, \dots, x_n)$ can be expressed as either

$$f(x_1, x_2, \dots, x_n) = x_1 \cdot f(1, x_2, \dots, x_n) + x_1' \cdot f(0, x_2, \dots, x_n) \quad (3.25)$$

or

$$f(x_1, x_2, \dots, x_n) = [x_1 + f(0, x_2, \dots, x_n)] \cdot [x_1' + f(1, x_2, \dots, x_n)]. \quad (3.26)$$

Proof This proceeds by perfect induction. Let x_1 be equal to 1; then x_1' equals 0 and Eq. (3.25) becomes an identity, i.e.,

$$f(1, x_2, \dots, x_n) = 1 \cdot f(1, x_2, \dots, x_n).$$

Similarly, substituting $x_1 = 0$ and $x_1' = 1$ also reduces Eq. (3.25) to an identity and thus the theorem is proved. $\diamond$

If we now apply the expansion theorem with respect to variable x_2 to each of the two terms in Eq. (3.25), we obtain

$$\begin{aligned} f(x_1, x_2, \dots, x_n) &= x_1 x_2 f(1, 1, x_3, \dots, x_n) + x_1 x_2' f(1, 0, x_3, \dots, x_n) \\ &\quad + x_1' x_2 f(0, 1, x_3, \dots, x_n) + x_1' x_2' f(0, 0, x_3, \dots, x_n). \end{aligned}$$

The expansion of the function about the remaining variables yields the disjunctive normal form. In a similar manner, repeated applications of the dual expansion theorem, Eq. (3.26), to $f(x_1, x_2, \dots, x_n)$ about its variables $x_1, x_2, \dots, x_n$ yield the conjunctive normal form.

A simpler and faster procedure for obtaining the canonical sum-of-products form of a switching function is summarized as follows.

1. Examine each term; if it is a minterm, retain it, and continue to the next term.
2. In each product that is not a minterm, check the variables that do not occur; for each x_i that does not occur, multiply the product by $(x_i + x_i')$.
3. Multiply out all products and eliminate redundant terms.

Example Determine the canonical sum-of-products form for $T(x, y, z) = x'y + z' + xyz$. Applying rules 1–3, we obtain

$$\begin{aligned}
 T &= x'y + z' + xyz \\
 &= x'y(z + z') + (x + x')(y + y')z' + xyz \\
 &= x'yz + x'y'z' + xyz' + x'y'z' + x'yz' + x'y'z' + xyz \\
 &= x'yz + x'y'z' + xyz' + x'y'z' + x'y'z' + xyz.
 \end{aligned}$$

The canonical product-of-sums form is obtained in a dual manner by expressing the function as a product of factors and adding the product $x_i x_i'$ to each factor in which the variable x_i is missing. The expansion into canonical form is obtained by repeated applications of Eq. (3.14).

Example Let us determine the canonical product-of-sums form of $T(x, y, z) = x'(y' + z)$. Using the above procedure,

$$\begin{aligned}
 T &= x'(y' + z) \\
 &= (x' + yy' + zz')(y' + z + xx') \\
 &= [(x' + y + z)(x' + y + z')(x' + y' + z)(x' + y' + z')] \\
 &\quad \cdot [(x + y' + z)(x' + y' + z)] \\
 &= (x' + y + z)(x' + y + z')(x' + y' + z)(x' + y' + z')(x + y' + z).
 \end{aligned}$$

In some instances, it is desirable to transform a function from one form to another. This transformation can be accomplished by writing down the truth table and using the previously described techniques. An alternative method, which is based on the involution theorem $(x')' = x$, is illustrated by the following example.

Example Find the canonical product-of-sums form for the function

$$T(x, y, z) = x'y'z' + x'y'z + x'yz + xyz + xy'z + xy'z'.$$

Using the involution theorem,

$$T = (T')' = [(x'y'z' + x'y'z + x'yz + xyz + xy'z + xy'z')']'.$$

The complement T' consists of those minterms that are not contained in the expression for T , i.e.,

$$\begin{aligned}
 T &= [x'yz' + xyz']' \\
 &= (x + y' + z)(x' + y' + z).
 \end{aligned}$$

Functional properties

From the foregoing discussion, we may conclude that *the canonical sum-of-products form of a switching function is unique (up to commutation)*. In order to prove this assertion, suppose that there exist two different canonical sum-of-products forms expressing f . Since we are assuming the forms to be different, they must differ by at least one minterm; that is, there must be at least one set of values for the variables $x_1, x_2, \dots, x_n$ for which one form results in $f(x_1, x_2, \dots, x_n) = 0$ while the other form yields $f(x_1, x_2, \dots, x_n) = 1$, a result which contradicts the assumption that both forms express the same function. (Note that according to the commutativity law there actually exist more than one such canonical form, but we shall regard them all as identical.)

Two switching functions $f_1(x_1, x_2, \dots, x_n)$ and $f_2(x_1, x_2, \dots, x_n)$ are said to be *logically equivalent* (or simply *equivalent*) if and only if both functions have the same value for each combination of variables $x_1, x_2, \dots, x_n$. Thus, we have the following property.

- Two switching functions are equivalent if and only if their canonical sum-of-products forms are identical.

Consequently, in order to prove an identity of two functions it is sufficient to expand both functions to their canonical forms and to compare the outcomes.

In a similar manner, it can be shown that every switching function may be expressed *uniquely* in a canonical product-of-sums form and that two switching functions are equivalent if and only if their canonical product-of-sums forms are identical. From here on, we shall confine our discussion to the sum-of-products form since the applicability of subsequent results to the dual form is understood.

Let a binary constant a_i be the value of the function $f(x_1, x_2, \dots, x_n)$ for the combination of variables whose decimal code is i . Then every switching function can be expressed in the form

$$f(x_1, x_2, \dots, x_n) = a_0 x_1' x_2' \cdots x_n' + a_1 x_1' x_2' \cdots x_n + \cdots + a_r x_1 x_2 \cdots x_n.$$

A factor a_i is set to 1 (0) if the corresponding minterm is (is not) contained in the canonical form of the function. There are 2^n coefficients, each of which can have two values, 0 and 1. Hence, there are 2^{2^n} possible assignments of values to the coefficients, and thus *there exist 2^{2^n} switching functions of n variables*.

Example Tabulate the functions of two variables. The results are given in Table 3.6.

The canonical sum-of-products form of a function of two variables is given by

$$f(x, y) = a_0 x' y' + a_1 x' y + a_2 x y' + a_3 x y.$$

Table 3.6 List of switching functions $f(x, y)$ of two variables, x and y

a_3	a_2	a_1	a_0	$f(x, y)$	Name of function	Symbol
0	0	0	0	0	Inconsistency	
0	0	0	1	$x'y'$	NOR	$x \downarrow y^a$
0	0	1	0	$x'y$		
0	0	1	1	x'	NOT	x'
0	1	0	0	xy'		
0	1	0	1	y'		
0	1	1	0	$x'y + xy'$	EXCLUSIVE-OR (modulo-2 addition)	$x \oplus y$
0	1	1	1	$x' + y'$	NAND	$x y^b$
1	0	0	0	xy	AND	$x \cdot y$
1	0	0	1	$xy + x'y'$	Equivalence	$x \equiv y$
1	0	1	0	y		
1	0	1	1	$x' + y$	Implication	$x \rightarrow y$
1	1	0	0	x		
1	1	0	1	$x + y'$	Implication	$y \rightarrow x$
1	1	1	0	$x + y$	OR	$x + y$
1	1	1	1	1	Tautology	

^a The downward-pointing arrow is referred to as a dagger.

^b The vertical is referred to as a Sheffer stroke.

There are $2^{2^2} = 16$ functions corresponding to the 16 possible assignments of 0's and 1's to a_0, a_1, a_2 , and a_3 . There are six nonsimilar functions: $f = 0$, $f = 1$, and $f = x$, which are known as trivial functions, while $f = xy$, $f = x + y$, and $f = xy + x'y'$ are known as nontrivial functions. Any other function may be obtained from these six by complementation or the interchange of variables. For example, $x'y + xy'$ can be obtained from $xy + x'y'$ by interchanging x and x' .

The EXCLUSIVE-OR operation

The EXCLUSIVE OR, denoted $\oplus$, is a binary operation on the set of switching elements. It assigns value 1 to two arguments if and only if they have complementary values; that is, $A \oplus B = 1$ if either A or B is 1 but not when both A and B are 1. It is evident that the EXCLUSIVE-OR operation assigns to each pair of elements its modulo-2 sum; consequently, it is often called the *modulo-2 addition* operation. The following properties of the EXCLUSIVE OR are direct consequences of its definition:

$$\begin{aligned}
 A \oplus B &= B \oplus A && (\text{commutativity}), \\
 (A \oplus B) \oplus C &= A \oplus (B \oplus C) \\
 &= A \oplus B \oplus C && (\text{associativity}), \\
 (AB) \oplus (AC) &= A(B \oplus C) && (\text{distributivity}).
 \end{aligned}$$

$$\text{if } A \oplus B = C \text{ then } \begin{cases} A \oplus C = B, \\ B \oplus C = A, \\ A \oplus B \oplus C = 0. \end{cases}$$

In general, the modulo-2 addition of an even number of elements whose value is 1 gives 0 and the modulo-2 addition of an odd number of elements whose value is 1 gives 1. The usefulness of the modulo-2-addition operation will become evident in subsequent chapters, and especially in the analysis and design of linear sequential machines.

Functionally complete operations

It has been demonstrated that every switching function can be expressed in a canonical sum-of-products form, where each expression consists of a finite number of switching variables, constants, and the operations $+$, $\cdot$, $'$.

Definition 3.1 A set of operations is said to be *functionally complete* (or *universal*) if and only if every switching function can be expressed entirely by means of operations from this set.

The set $\{+, \cdot, '\}$ is clearly functionally complete. Moreover, by means of De Morgan's theorems, it can be shown that the set $\{+, '\}$ is also functionally complete. Since $x \cdot y = (x' + y')'$, the operations $+$ and $'$ can together replace the operation $\cdot$ in any switching function, and therefore the set $\{+, '\}$ is functionally complete. In a similar way, it can be shown that the set $\{\cdot, '\}$ is also functionally complete. Many functionally complete sets of operations exist, among the more important of which are NAND and NOR operations.

Example Prove that the NOR operation is functionally complete.

A common method for proving the completeness of an operation is to show that it is capable of generating each operation of a set that is already known to be functionally complete, for example, $\{+, '\}$ or $\{\cdot, '\}$.

Since $x \downarrow y = x'y'$ (see Table 3.6), then

$$\begin{aligned} x \downarrow x &= x'x' = x', \\ (x \downarrow y) \downarrow (x \downarrow y) &= (x'y')' = x + y. \end{aligned}$$

In order to implement switching functions, it is sufficient to find a set of devices capable of implementing a functionally complete set of operations. In general, it is desirable to reduce the implementation cost by selecting a minimal set of such devices. Since NAND and NOR operations are functionally complete, devices implementing them currently serve as major building blocks in logic design.

3.3 Isomorphic systems

In this section, we shall discuss the relationship between switching algebra (see Section 3.1), the calculus of propositions, and the algebra of series-parallel switching circuits.

Two algebraic systems, each consisting of a set of elements and one or more operations that satisfy a given set of postulates, are said to be *isomorphic* if the following are satisfied. First, for every operation in one system there exists a corresponding operation in the second system, although it may be denoted in a different way. Second, to each element x_i in one system there corresponds a unique element y_i in the second system, and vice versa. Consequently, if both systems have finite sets of elements then they have the same number of elements. Finally, if in every postulate of the first system each x_i is replaced by the corresponding y_i , and every operation is replaced by the corresponding operation from the second system, then the resulting postulate must be valid for the second system. In other words, two algebraic systems are isomorphic if and only if they are identical except for the labels and symbols used to represent the operations and elements.

The algebra of series–parallel circuits and the calculus of propositions will be shown to be isomorphic to switching algebra; therefore all the properties of the latter system are valid for the former ones.

Series–parallel switching circuits

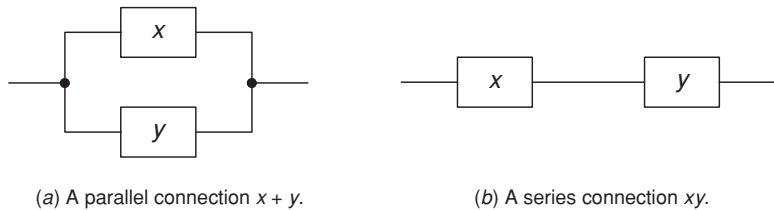
A switching circuit consists of “gates” through which information flows. This information may take the form of electric signals, water, pressure, or some other quantity. A *gate* is a two-state device capable of switching from one state, which permits the flow of information, to the other state, which blocks it, and vice versa. Physically, this gate may be an electrical switch that is either open or closed, a pneumatic device that may be in either a compressed or released state, and so on.

We shall associate with each gate a two-valued variable (with symbol x , y , etc.), which is in a primed form if the gate normally permits the flow of information and is in an unprimed form if the gate normally blocks that flow. If two gates operate in such a way that they are always in the same state, they are associated with the same variable and denoted by the same letter. If they operate in such a way that one always permits the flow of information when the other is blocking it, and vice versa, the first is denoted by a primed letter, say x' , while the second is denoted by the same unprimed letter, i.e., x . In general, primed letters are reserved for those gates that normally, i.e., before the circuit is activated, allow the flow of information, while unprimed letters are assigned to gates that normally block that flow. If a gate permits the flow of information, the literal associated with it takes on the value 1, and if it blocks that flow, the literal takes on the value 0.

The parallel connection of two gates is denoted by $x + y$ and their series connection by xy , as shown in Fig. 3.1. The circuits of Fig. 3.1, as well as a circuit that consists of a single gate, are said to be *elementary series–parallel* circuits. Any switching circuit constructed of either a series or parallel connection of two or more elementary series–parallel circuits is called *series–parallel*. In other words, a circuit is series–parallel if it can be decomposed into

Table 3.7 Definition of transmission functions

x	y	$x + y$	xy
0	0	0	0
0	1	1	0
1	0	1	0
1	1	1	1

Fig. 3.1 Basic connections of switching circuits. Both are termed elementary series–parallel circuits.

either two subcircuits in series or two subcircuits in parallel; these subcircuits, which are also series–parallel, may be again decomposed as before, and so on until each subcircuit consists of only an elementary series–parallel circuit.

For each circuit, we define a *transmission function*, which assumes the value 1 when there is a path from one terminal to the other terminal through which information flows and assumes the value 0 if there is no such path. The transmission function is said to *represent* the circuit, and the circuit is said to be a *realization* of the function. A transmission function is usually denoted by the letter T .

In order to determine the value of the transmission function representing the parallel circuit in Fig. 3.1a, we observe that a path exists between the two circuit terminals if either gate x or gate y or both allow the flow of information, that is, T is 1 if either x or y is 1 or both x and y are 1. The circuit blocks the flow of information if both x and y block such a flow, i.e., if both x and y are 0. These properties of the transmission function are tabulated in Table 3.7. In a similar manner, we observe that the transmission function, which represents the series circuit of Fig. 3.1b, is 1 if and only if both gates x and y permit the flow of information, i.e., x is 1 and y is 1.

From Table 3.7 and from the preceding discussion, it is evident that a complete analogy exists between the OR and AND operations defined in Section 3.1 and, respectively, the operations $x + y$ and xy that define the transmission functions of parallel and series switching circuits. Moreover, since the transmission function of a gate must be either 0 or 1, it follows that $x = 1$ if and only if $x' = 0$, and that $x = 0$ if and only if $x' = 1$. Thus, the *complement* of a given circuit is a circuit which blocks all paths of information flow whenever the given circuit permits any. Clearly, the algebraic system defined in this section for switching circuits is isomorphic to the switching algebra defined in

Section 3.1. Consequently, all the properties of switching functions apply to transmission functions as well and may be used in the analysis and synthesis of switching circuits. In particular, since the previous properties of switching elements (Eqs. (3.1) through (3.26)) hold true when expressions replace the elements, we may conclude that the transmission function of a circuit consisting of a series connection of two subcircuits whose transmission functions are T_1 and T_2 is $T_1 T_2$. Similarly, the transmission function of a circuit composed of two parallel subcircuits T_1 and T_2 is $T_1 + T_2$.

Example The transmission function of the circuit in Fig. 3.2a is given by

$$T = xy' + (x' + y)z.$$

Simple algebraic manipulation yields the reduced form

$$T = xy' + z,$$

which represents the simpler circuit shown in Fig. 3.2b.

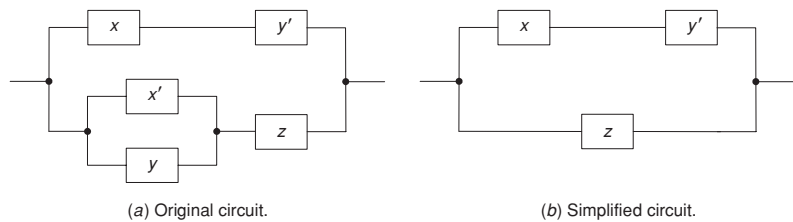


Fig. 3.2 Simplification of a switching circuit.

An important application of the theory of switching circuits is to CMOS circuits in which transistors allow the transmission of information. The properties of CMOS circuits and their analysis and design, are studied in Chapter 5.

Propositional calculus

A *proposition* is a declarative statement that may be either true or false but never both. For example, the temperature is 100 degrees, the turtle runs faster than the hare, the sum of 2 and 3 equals 4, etc. With every proposition we associate a variable, denoted p , q , etc., that assumes the value 1 if the proposition is true and the value 0 if it is false. Thus, a proposition of value 0 is always false, while a proposition of value 1 is always true.

New propositions may be derived from existing ones. Consider, for example, the propositions “the sun is shining” and “the sun is not shining.” It seems evident that if the first proposition is true then the second one is false, and vice versa. A proposition is said to be a *negation* of another proposition if when one

Table 3.8 Definitions of the
conjunction pq and disjunction
 $p + q$ of p and q

p	q	pq	$p + q$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

is false the other is true. Thus, negation p' of a proposition p is defined to be 1 if p is 0 and to be 0 if p is 1.

Two propositions p and q may be combined to form new propositions. For example, if p designates proposition “the temperature is above 60 degrees” and q designates “the humidity is over 50 percent,” then we may form a proposition “the temperature is above 60 degrees *and* the humidity is over 50 percent” by combining p and q with a connective *and*. In general, the *conjunction* of p and q , denoted pq , is the proposition “ p and q .” Proposition pq is true whenever both p and q are true and is false whenever either one or both p and q is or are false.

Propositions may also be combined by means of a connective *or*. For example, the preceding propositions, when thus combined, yield the proposition “either the temperature is above 60 degrees or the humidity is over 50 percent.” In general, the *disjunction* of p and q , denoted $p + q$, means the proposition “either p or q or both,” where the words “or both” are omitted and “or” is defined to be the *inclusive or*. From its definition, it follows that the proposition $p + q$ is true whenever either p or q or both is or are true and is false whenever both p and q are false. The conjunction and disjunction of p and q are defined in Table 3.8.

The analogy between the calculus of propositions and switching algebra is now apparent. In fact, they are isomorphic algebraic systems. Consequently, we may speak of variables and functions in precisely the same way as before.

Example An air-conditioning system in a storage warehouse is to be turned on if one or more of the following three conditions occurs:

1. the weight of the stored material is less than 100 tonnes, the relative humidity is at least 60 percent, and the temperature is above 60 degrees;
2. the weight of the stored material is 100 tonnes or more and the temperature is above 60 degrees;
3. the weight of the stored material is less than 100 tonnes and the barometer stands at 30 inches of mercury (about 1 atmosphere) or over.

Let A denote the proposition that the air conditioning is turned on. It is our objective to specify A in terms of the following four propositions:

W designates a weight of 100 tonnes or more;
 H designates a relative humidity of at least 60 percent;
 T designates a temperature above 60 degrees;
 P designates a barometric pressure of 30 or more.

From condition 1, we find that A is 1, i.e. the air conditioning is turned on, if $W'HT$ is 1; from condition 2, we conclude that A is 1 if WT is 1; and condition 3 is represented by $W'P$. Consequently, an expression for A is

$$A = W'HT + WT + W'P.$$

This expression may be simplified by applying Eq. (3.17) to yield

$$\begin{aligned}
 A &= HT + WT + W'P \\
 &= T(H + W) + W'P.
 \end{aligned}$$

Hence the air-conditioning system is turned on if the temperature is above 60 degrees and either the weight is at least 100 tonnes or the humidity is at least 60 percent, or if the weight is less than 100 tonnes and the barometer stands at 30 or over.

3.4 Electronic-gate networks

In the previous sections of the chapter, we have studied methods of deriving switching functions, manipulating them, and eliminating all redundancies from them. We consider now the problem of realizing switching functions by means of electronic devices. We shall introduce briefly the building blocks of these devices, deferring reference to their actual physical properties to Chapter 5.

Electronic gates generally receive voltages as inputs and produce output voltages. The precise values of these voltages are not significant in determining the logic operation of the gates; in fact, they vary from circuit to circuit and from device to device. The significant point is that the voltages are restricted to two ranges of values, “high” and “low.” Thus, two-valued variables may be used to represent them. By convention we shall associate the switching constants 1 and 0 with the higher and lower voltages, respectively.

Electronic gates are constructed of two-state switching devices, each capable of either permitting a flow of current or blocking it. In order to implement any switching function, these gates must be capable of implementing a functionally complete set of operations.

One set of basic gates, capable of implementing the three operations AND, OR, and NOT, is shown in Fig. 3.3. The *AND gate* has two or more inputs, and one output that assumes the value 1 if and only if all the inputs assume the value 1. Thus, if the input values are a , b , and c then the output value is given by $T_1 = abc$. Moreover, the *OR gate* produces an output value 1 if one or more of its input values is 1 and thus its output may be characterized by

Fig. 3.3 Gate symbols.

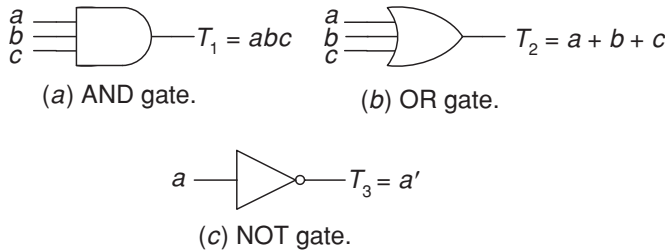
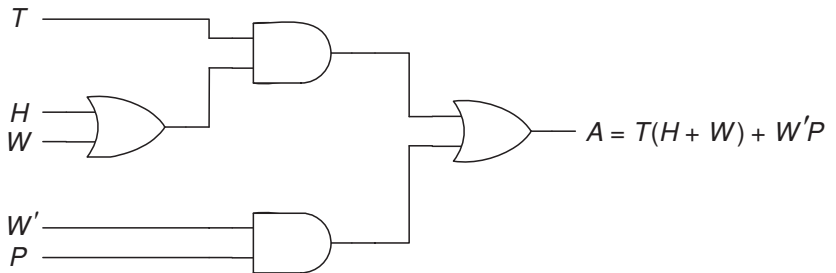


Fig. 3.4 Gate network.



$T_2 = a + b + c$. The *NOT gate* has one input, and one output whose value is the complement of the input value; i.e., its output value is 1 if its input value is 0, and 0 if its input value is 1.

Gate networks are constructed by the use of interconnecting gates, where the output of one gate is used to drive the inputs of others. As an example, consider the network of Fig. 3.4, which implements the function $A = T(H + W) + W'P$ describing the preceding air-conditioning control system. The inputs to this network may come from various thermometers, humidity-measurement devices, a barometer, and a scale, while its output turns on (or off) the air conditioner.

The purpose of the preceding discussion was to introduce the basic electronic-gate logic. A more comprehensive study of the analysis and synthesis of switching circuits is deferred to Chapter 5.

*3.5 Boolean algebras

In Chapter 2 we established the properties of partially ordered sets and lattices. We shall now define a Boolean algebra and subsequently show its relationship to the switching algebra defined in Section 3.1.

Definition 3.2 A *Boolean algebra* B is a distributive and complemented lattice.

Since a Boolean algebra is defined as a special lattice, all the lattice properties derived in Chapter 2 are applicable to any Boolean algebra. Accordingly, we can now summarize the properties of Boolean algebras as follows:

- a boolean algebra B is a set of elements $a, b, c, \dots$, together with two binary operations, $+$ and $\cdot$, that satisfy the idempotent, commutative, absorption, and associative laws and are mutually distributive;
- B contains two bounds, 0 and 1, which are the least and greatest elements, respectively;
- B has a unary operation of complementation that assigns to every element its complement.

We shall now prove that *the complement a' of any element a in B is unique*; that is, there exists only one element a' such that $a + a' = 1$ and $a \cdot a' = 0$. Suppose that there exists some element a that possesses two complements, b_1 and b_2 , satisfying the above properties, i.e.,

$$\begin{aligned} a + b_1 &= 1, & a + b_2 &= 1, \\ a \cdot b_1 &= 0, & a \cdot b_2 &= 0. \end{aligned}$$

Then

$$b_1 = b_1 \cdot 1 = b_1 \cdot (a + b_2) = b_1 \cdot a + b_1 \cdot b_2 = 0 + b_1 \cdot b_2 = b_1 \cdot b_2.$$

A similar argument shows that $b_2 = b_1 \cdot b_2$. Consequently $b_1 = b_2$, which proves the uniqueness of the complement and provides the justification for defining the unary complement operation. An immediate corollary is that the complement of a' is a , i.e., $(a')' = a$.

To find the complements of elements 0 and 1, note that by definition $0 + 0' = 1$ but by virtue of the definition of the lub, it follows that $0' = 1$. Thus,

$$0' = 1 \quad \text{and} \quad 1' = 0.$$

Example Using the above, prove De Morgan's theorems (see Section 3.1) for two variables:

$$\begin{aligned} (a + b)' &= a' \cdot b', \\ (a \cdot b)' &= a' + b'. \end{aligned}$$

We need to show that $(a + b)(a' \cdot b') = 0$ and that $(a + b) + a' \cdot b' = 1$. (As before, we shall subsequently omit the $\cdot$ symbol.) Expanding the parentheses on the left-hand side of the former equation, we obtain

$$(a + b)(a' \cdot b') = aa'b' + ba'b' = 0 + a'bb' = 0 + 0 = 0.$$

Applying the distributive law to $(a + b) + a' \cdot b'$ yields

$$(a + b) + a' \cdot b' = (a + b + a')(a + b + b') = (b + 1)(a + 1) = 1.$$

The dual property is verified in an analogous manner.

We shall now show that the switching algebra defined in Section 3.1 is a two-valued Boolean algebra. Define a Boolean algebra that consists of just

Table 3.9 Definition of a Boolean algebra that is isomorphic to the switching algebra. Each entry in the left-hand and middle blocks of the table gives the result of combining a row label with a column label by means of the operation specified in the top left-hand corner of the block

+	0 1		·	0 1	
0	0	1	0	0	0
1	1	1	1	0	1

two elements, 0 and 1, with the usual binary operations $+$ and $\cdot$ and the complementation operation $'$. If the algebra is to satisfy all lattice properties and Definition 3.2 above, it must follow the operations shown in Table 3.9. For example, to show that the operation $\cdot$ is commutative it is necessary to show that it is commutative for each of the four ways of selecting values for the two elements, that is, for every combination of values $ab = ba$. It is evident that Table 3.9 defines a Boolean algebra which is isomorphic to the switching algebra defined in Section 3.1.

Example The algebraic system defined in Table 3.10 is a Boolean algebra. The elements 0 and 1 satisfy the definitions of the least and greatest bounds, namely, that, for every element x in B , $x + 0 = x$ and $x + 1 = 1$. The elements a and b are complements of each other since they satisfy the requirements that $a + b = 1$ and $a \cdot b = 0$. Finally, it is easy to verify that this system defines a distributive lattice by showing that, for every combination of elements, the operations are idempotent, commutative, and associative and that they distribute over each other.

Table 3.10 A Boolean algebra (see Table 3.9)

+	0	1	a	b	·	0	1	a	b
0	0	1	a	b	0	0	0	0	0
1	1	1	1	1	1	0	1	a	b
a	a	1	a	1	a	0	a	a	0
b	b	1	1	b	b	0	b	0	b

Notes and references

The first significant contribution in the area of switching theory was made by Shannon [3] in 1938. He developed the algebra of switching circuits and showed its relation to the calculus of propositions and Boolean algebra [1]. Further developments of switching theory were made by numerous authors in the 1940s and 1950s, in particular in a second

paper by Shannon [4], in a book by Keister, Ritchie, and Washburn [2], and in a report by the staff of the Harvard University Computation Laboratory [5].

- [1] Boole, G.: *An Investigation of the Laws of Thought*, Dover, New York, 1854.
- [2] Keister, W., S. A. Ritchie, and S. Washburn: *The Design of Switching Circuits*, Van Nostrand, New York, 1951.
- [3] Shannon, C. E.: "A symbolic analysis of relay and switching circuits," *Trans. AIEE*, vol. 57, pp. 713-723, 1938.
- [4] Shannon, C. E.: "The synthesis of two-terminal switching circuits," *Bell System Tech. J.*, vol. 28, pp. 59-98, 1949.
- [5] Staff of the Computation Laboratory: "Synthesis of electronic computing and control circuits," *Annals*, vol. 27, Harvard University Press, Cambridge MA, 1951.

Problems

Problem 3.1. Prove the properties in Eqs. (3.3) through (3.12).

Problem 3.2. Using mathematical induction, prove De Morgan's theorem for n variables,

$$[f(x_1, x_2, \dots, x_n, 0, 1, +, \cdot)]' = f(x_1', x_2', \dots, x_n', 1, 0, \cdot, +).$$

Problem 3.3. Simplify the following algebraic expressions:

- (a) $x' + y' + xyz'$
- (b) $(x' + xyz') + (x' + xyz')(x + x'y'z)$
- (c) $xy + wxyz' + x'y$
- (d) $a + a'b + a'b'c + a'b'c'd + \dots$
- (e) $xy + y'z' + wxz'$
- (f) $w'x' + x'y' + w'z' + yz$

Problem 3.4. Find, by inspection, the complement of each of the following expressions and then simplify it.

- (a) $x'(y' + z')(x + y + z')$
- (b) $(x + y'z')(y + x'z')(z + x'y')$
- (c) $w' + (x' + y + y'z')(x + y'z)$

Problem 3.5. Demonstrate, without using perfect induction, whether each of the following equations is valid.

- (a) $(x + y)(x' + y)(x + y')(x' + y') = 0$
- (b) $xy + x'y' + x'yz = xyz' + x'y' + yz$
- (c) $xyz + wy'z' + wxz = xyz + wy'z' + wxy'$
- (d) $xy + x'y' + xy'z = xz + x'y' + x'yz$

Problem 3.6. Given $AB' + A'B = C$, show that $AC' + A'C = B$.

Problem 3.7. Find the values of two-valued variables A , B , C , and D by solving the following set of simultaneous equations:

$$\begin{aligned} A' + AB &= 0, \\ AB &= AC, \\ AB + AC' + CD &= C'D. \end{aligned}$$

Problem 3.8. Prove that if $w'x + yz' = 0$, then

$$wx + y'(w' + z') = wx + xz + x'z' + w'y'z.$$

Problem 3.9. Define a connective operator $*$ for two-valued variables A , B , and C as follows:

$$A * B = AB + A'B'.$$

Let $C = A * B$. Determine which of the following is valid:

- (a) $A = B * C$
- (b) $B = A * C$
- (c) $A * B * C = 1$

Problem 3.10. Determine the canonical sum-of-products representation of the following functions:

- (a) $f(x, y, z) = z + (x' + y)(x + y')$
- (b) $f(x, y, z) = x + (x'y' + x'z)'$

Problem 3.11. Show the truth table for each of the following functions and find its simplest product-of-sums form (i.e., the form with the minimum number of literals).

- (a) $f(x, y, z) = xy + xz$
- (b) $f(x, y, z) = x' + yz'$

Problem 3.12. By adding redundant factors or terms to the expression $uvw + uwx + uvxz + xyz$, it may be simplified as follows:

$$\begin{aligned} uvw + uwx + uvxz + xyz &= uw(v + xy) + xz(uv + y) \\ &= uw(uv + xy) + xz(uv + xy) \\ &= (uw + xz)(uv + xy). \end{aligned}$$

Factor each of the following expressions into a product of two factors such that the resulting expression has the least number of literals:

- (a) $wxyz + w'x'y'z' + w'xy'z + wx'y'z'$
- (b) $vwx + vwy + wxy + vxyz$

Problem 3.13. The dual f_d of a function $f(x_1, x_2, \dots, x_n)$ is obtained by interchanging the operations of logical addition and multiplication and by interchanging constants 0 and 1 within any expression for that function.

- (a) Show that $f_d = f'(x'_1, x'_2, \dots, x'_n)$.
- (b) Find a three-variable function that is its own dual. Such a function is called *self-dual*.
- (c) Prove that for any function f and any two-valued variable A , which may or may not be a variable in f , the function

$$g = Af + A'f_d$$

is self-dual.

Problem 3.14

- (a) Show that $f(A, B, C) = A'BC + AB' + B'C'$ is a universal operation.
- (b) Assuming that a constant value 1 is available, show that $f(A, B) = A'B$ (together with the constant) is a universal operation.

Problems

Problem 3.15. For each of the following, prove or show a counter-example.

- (a) If $A \oplus B = 0$ then $A = B$.
- (b) If $A \oplus C = B \oplus C$ then $A = B$.
- (c) $A \oplus B = A' \oplus B'$.
- (d) $(A \oplus B)' = A' \oplus B = A \oplus B'$.
- (e) $A \oplus (B + C) = (A \oplus B) + (A \oplus C)$.
- (f) If $A \oplus B \oplus C = D$ then $A \oplus B = C \oplus D$ and $A = B \oplus C \oplus D$.

Problem 3.16. Any function of two variables can be represented, with proper choice of truth values for the a 's, as

$$f(x, y) = a_0x'y' + a_1x'y + a_2xy' + a_3xy.$$

- (a) Prove that each representation below can also be used to specify any function of two variables. Show how to obtain the b 's and c 's from the a 's.

$$f(x, y) = b_0 \oplus b_1y \oplus b_2x \oplus b_3xy,$$

$$f(x, y) = c_0x'y' \oplus c_1x'y \oplus c_2xy' \oplus c_3xy.$$

Hint: Compare coefficients by choosing appropriate values for x and y .

- (b) Prove that if a function $f(x_1, x_2, \dots, x_n)$ is represented in a canonical sum-of-products form then all OR operations may be replaced by EXCLUSIVE-OR operations.

Problem 3.17. Prove that any function $f(x_1, x_2, \dots, x_n)$ can be expressed in a complement-free form as follows:

$$\begin{aligned} f(x_1, x_2, \dots, x_n) = & d_0 \oplus d_1x_1 \oplus d_2x_2 \oplus \dots \oplus d_nx_n \\ & \oplus d_{n+1}x_1x_2 \oplus d_{n+2}x_1x_3 \oplus \dots \oplus d_{n(n+1)/2}x_{n-1}x_n \\ & \oplus d_{[n(n+1)/2]+1}x_1x_2x_3 \oplus \dots \oplus d_{2^n-1}x_1x_2 \dots x_n, \end{aligned}$$

where $d_0, d_1, \dots, d_{2^n-1}$ are two-valued variables.

Problem 3.18. Prove that the expansion of any switching function of n variables $f(y_1, y_2, \dots, y_s, z_1, z_2, \dots, z_{n-s})$ with respect to the variables $z_1, z_2, \dots, z_{n-s}$ is given by

$$\begin{aligned} f(y_1, y_2, \dots, y_s, z_1, z_2, \dots, z_{n-s}) \\ = \sum_{i=1}^{2^{n-s}-1} f_i(y_1, y_2, \dots, y_s) g_i(z_1, z_2, \dots, z_{n-s}), \end{aligned}$$

where

$$\begin{aligned} f_0(y_1, y_2, \dots, y_s) &= f(y_1, y_2, \dots, y_s, 0, 0, \dots, 0), \\ f_1(y_1, y_2, \dots, y_s) &= f(y_1, y_2, \dots, y_s, 0, 0, \dots, 1), \\ &\vdots \\ f_{2^{n-s}-1}(y_1, y_2, \dots, y_s) &= f(y_1, y_2, \dots, y_s, 1, 1, \dots, 1) \end{aligned}$$

and where $g_i(z_1, z_2, \dots, z_{n-s})$ is the product term whose decimal representation is i , e.g., $g_0 = z'_1z'_2 \dots z'_{n-s}$. Note that the distinction between the y 's and the z 's is only for convenience and has no other significance.

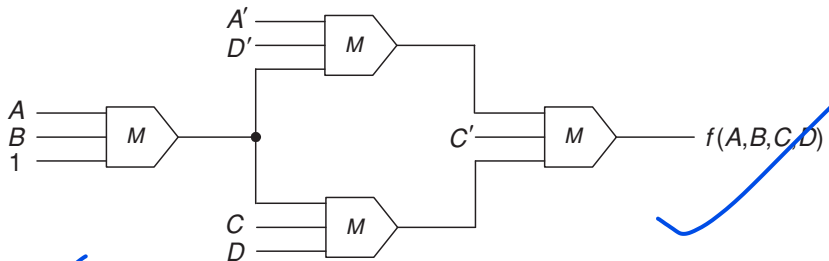
Hint: Use Shannon's expansion theorem as given in Eq. (3.25) and finite induction on s .

Problem 3.19. The majority function $M(x, y, z)$ is equal to 1 when two or three of its arguments equal 1, that is,

$$M(x, y, z) = xy + xz + yz = (x + y)(x + z)(y + z)$$

- Show that $M(a, b, M(c, d, e)) = M(M(a, b, c), d, M(a, b, e))$.
- Show that $M(x, y, z)$, the complementation operation, and the constant 0 form a functionally complete set of operations.
- Find the simplest switching expression $f(A, B, C, D)$ corresponding to the network of Fig. P3.19.

Fig. P3.19



Problem 3.20. A safe has five locks, v, w, x, y , and z , all of which must be unlocked for the safe to open. The keys to the locks are distributed among five executives in the following manner:

- A has keys for locks v and x ;
- B has keys for locks v and y ;
- C has keys for locks w and y ;
- D has keys for locks x and z ;
- E has keys for locks v and z .

- Determine the minimum number of executives required to open the safe.
- Find all the combinations of executives that can open the safe. Write an expression $f(A, B, C, D, E)$ which specifies when the safe can be opened as a function of which executives are present.
- Who is the "essential executive" without whom the safe cannot be opened?

Problem 3.21. You are presented with a set of requirements under which an insurance policy will be issued. The applicant must be

- a married female 25 years old or over, or
- a female under 25, or
- a married male under 25 who has not been involved in a car accident, or
- a married male who has been involved in a car accident, or
- a married male 25 years or over who has not been involved in a car accident.

Variables w, x, y , and z assume truth value 1 in the following cases:

- $w = 1$ if the applicant has been involved in a car accident;
- $x = 1$ if the applicant is married;
- $y = 1$ if the applicant is a male;
- $z = 1$ if the applicant is under 25.

Problems

- (a) Find an algebraic expression that assumes the value 1 whenever the policy should be issued.
- (b) Simplify algebraically the above expression and suggest a simpler set of requirements.

Problem 3.22. Five soldiers, A , B , C , D , and E , volunteer to perform an important military task if the following conditions are satisfied.

1. Either A or B or both must go.
2. Either C or E , but not both, must go.
3. Either both A and C go or neither goes.
4. If D goes then E must also go.
5. If B goes then A and D must also go.

Define variables A , B , C , D , E such that an unprimed variable will mean that the corresponding soldier has been selected to go. Determine the expression that specifies the combinations of volunteers that can get the assignment.

Problem 3.23

- (a) Show a series-parallel network that realizes the transmission function $T = A(B + C'D') + A'B'$.
- (b) Show an AND, OR, NOT gate network that realizes the function $T = A'B + AB'C + B'C'$, assuming that only unprimed inputs are available.

Problem 3.24. Prove that a Boolean algebra of three elements $B = \{0, 1, a\}$ cannot exist.

Problem 3.25. Prove that for every Boolean algebra:

- (a) $a + a'b = a + b$;
- (b) if $a + b = a + c$ and $a' + b = a' + c$ then $b = c$;
- (c) if $a + b = a + c$ and $ab = ac$ then $b = c$.

Problem 3.26. Prove that the partial ordering of all positive integers dividing number 30 is a Boolean algebra of eight elements, $B = \{1, 2, 3, 5, 6, 10, 15, 30\}$.

- (a) Draw the corresponding Hasse diagram.
- (b) Define the binary operations by their operations on the integers.
- (c) For each element a in B , specify its complement a' .

Problem 3.27. An alternative definition of Boolean algebra is by means of the *Huntington postulates*, which are given as follows:

Definition A Boolean algebra is a set B of elements $a, b, c, \dots$ with the following properties.

1. B has two binary operations $+$ and $\cdot$, which satisfy the idempotent laws $a + a = a$ and $a \cdot a = a$, the commutative laws $a + b = b + a$ and $a \cdot b = b \cdot a$, the associative laws $a + (b + c) = (a + b) + c$ and $a \cdot (b \cdot c) = (a \cdot b) \cdot c$, and the absorption laws $a + (a \cdot b) = a$ and $a \cdot (a + b) = a$.
2. The operations are mutually distributive:

$$a \cdot (b + c) = (a \cdot b) + (a \cdot c) \quad \text{and} \quad a + (b \cdot c) = (a + b) \cdot (a + c).$$

3. There exist in B two universal bounds 0 and 1, which satisfy

$$0 + a = a, \quad 0 \cdot a = 0, \quad 1 + a = 1, \quad 1 \cdot a = a.$$

4. The Boolean algebra B has a unary operation of complementation, which assigns to every element a in B an element a' in B such that

$$a \cdot a' = 0, \quad a + a' = 1.$$

Derive the following properties of Boolean algebras directly from the above Huntington postulates.

- (a) For each a in B , there exists a *unique* a' in B .
- (b) For every a in B , $(a')' = a$.
- (c) For every Boolean algebra, $0' = 1$ and $1' = 0$.
- (d) In any Boolean algebra,

$$(a + b)' = a' \cdot b' \quad \text{and} \quad (a \cdot b)' = a' + b'.$$

.

4

Minimization of switching functions

A switching function can usually be represented by a number of expressions. Our aim in this chapter will be to develop procedures for obtaining a minimal expression for any such function, after establishing some criteria for minimality. In the preceding chapter, we dealt with simplification of switching expressions by means of algebraic manipulations. The deficiency of this method is that it does not constitute an algorithm and is ineffective for expressions of even a small number of variables (e.g., four or five). The methods to be introduced in this chapter partly overcome these limitations. The presented map method is very effective for the simplification by hand of expressions of up to five or six variables, while the tabulation procedure is suitable for machine computation and yields minimal expressions.

4.1 Introduction

Our aim in simplifying a switching function $f(x_1, x_2, \dots, x_n)$ is to *find an expression $g(x_1, x_2, \dots, x_n)$ which is equivalent to f and which minimizes some cost criteria*. There are various criteria to determine minimal cost. The most common are:

1. the minimum number of appearances of literals (recall that a *literal* is a variable in complemented or uncomplemented form);
2. the minimum number of literals in a sum-of-products (or product-of-sums) expression;
3. the minimum number of terms in a sum-of-products expression, provided that there is no other such expression with the same number of terms and fewer literals.

In subsequent discussions, we shall adopt the third criterion and restrict our attention to the sum-of-products form. Of course, dual results can be obtained by employing the product-of-sums form instead. Note that the expression

$xy + xz + x'y'$ is minimal according to criterion 3, although it may be written as $x(y + z) + x'y'$, which requires fewer literals.

Consider the minimization of the function $f(x, y, z)$ given below. A combination of the first and second product terms yields $x'z'(y + y') = x'z'$. Similarly, combinations of the second and third, fourth and fifth, and fifth and sixth terms yield a reduced expression for f :

$$\begin{aligned} f(x, y, z) &= x'yz' + x'y'z' + xy'z' + x'yz + xyz + xy'z \\ &= x'z' + y'z' + yz + xz. \end{aligned}$$

This expression is said to be in an irredundant form, since any attempt to reduce it, either by deleting any of the four terms or by removing a literal, will yield an expression that is not equivalent to f . In general, a sum-of-products expression, from which no term or literal can be deleted without altering its logic value, is called an *irredundant*, or *irreducible*, expression.

The above reduction procedure is not unique, and a different combination of terms may yield different reduced expressions. In fact, if we combine the first and second terms of f , the third and sixth, and the fourth and fifth, we obtain the expression

$$f(x, y, z) = x'z' + xy' + yz.$$

In a similar manner, by combining the first and fourth terms, the second and third, and the fifth and sixth, we obtain a third irredundant expression,

$$f(x, y, z) = x'y + y'z' + xz.$$

While all three expressions are irredundant, only the latter two are minimal. Consequently, *an irredundant expression is not necessarily minimal, nor is the minimal expression always unique*. It is, therefore, desirable to develop procedures for generating the set of all minimal expressions, so that the appropriate one may be selected according to other criteria (e.g., the distribution of gate loads, etc.).

4.2 The map method

The algebraic procedure of combining various terms and applying to them the rule $Aa + Aa' = A$ becomes very tedious as the number of terms and variables increases. The map method presented in this section and the tabulation procedure in Section 4.4 provide systematic methods for combining terms and obtaining minimal expressions.

Representation of functions

A *Karnaugh map*, hereafter usually referred to simply as a map, is actually a modified form of truth table in which the arrangement of combinations is

4.2 The map method

Fig. 4.1 Karnaugh maps for three and four variables.

		xy			
		00	01	11	10
z	0	0	2	6	4
	1	1	3	7	5

(a) Location of minterms in a three-variable map.

		xy			
		00	01	11	10
z	0		1	1	
	1			1	

(b) Map for function $f(x, y, z) = \sum(2, 6, 7) = yz' + xy$.

		wx			
		00	01	11	10
yz	00	0	4	12	8
	01	1	5	13	9
	11	3	7	15	11
	10	2	6	14	10

(c) Location of minterms in a four-variable map.

		wx			
		00	01	11	10
yz	00		1	1	1
	01		1	1	
	11			1	
	10			1	

(d) Map for function $f(w, x, y, z) = \sum(4, 5, 8, 12, 13, 14, 15) = wx + xy' + wy'z'$.

particularly convenient. The maps for functions of three and four variables are shown in Fig. 4.1. The column headings are labeled with the four combinations of the two corresponding variables. The row headings correspond to the binary values of z in the three-variable map and to the values of yz in the four-variable map. Each n -variable map consists of 2^n cells (squares), representing all possible combinations of these variables. The decimal codes that correspond to these combinations are shown in Figs. 4.1a, c. We shall subsequently refer to particular cells by these decimal codes.

The function value associated with a particular combination is entered in the corresponding cell. For example, the map of the function $f(x, y, z) = \sum(2, 6, 7)$ is shown in Fig. 4.1b, where the value 1 is entered in cells 2, 6, and 7 (see Fig. 4.1a). A blank cell means that for the corresponding combination, the value of the function is 0. The minterm that corresponds to a particular cell is determined as in the truth table. The variable x_i appears in uncomplemented form in the product if it has value 1 in the corresponding cell, and in complemented form if it has value 0. For example, cell 6 in the three-variable map corresponds to xyz' , and in the four-variable map it corresponds to $w'xyz'$. Fig. 4.1d shows the map for function $f(w, x, y, z) = \sum(4, 5, 8, 12, 13, 14, 15)$.

The cyclic code used in listing the combinations as column and row headings is of particular importance. As a result of this coding, cells that have a common side correspond to combinations that differ by the value of just a single variable. In general, two cells that differ in just one variable value are said to

be *adjacent* and play a major role in the simplification process, because they may be combined by means of the rule $Aa + Aa' = A$, where A denotes a product of literals and a denotes a single literal. For the purpose of determining adjacencies, it is useful to regard the three-variable map as the surface of a cylinder formed by joining the left and right sides of the map. Similarly, the four-variable map is regarded as an open face of a torus; that is, the left and right sides of the map are joined, as are its top and bottom. This has the result, for example, that cell 8 is adjacent to cells 0 and 10 in addition to its obvious adjacency to cells 9 and 12.

The product term corresponding to two adjacent cells for which the function has the value 1 is obtained by writing down the product of all those variables whose values are the same in the two cells and deleting the variable which is complemented in one cell and uncomplemented in the other. For example, the term that corresponds to cells 2 and 6 of Fig. 4.1b is yz' , since $x'yz' + xyz' = yz'$.

For each minterm of n literals, there are n other minterms that have $n - 1$ literals in common with it, differing from it in just one literal. Utilizing the geometrical properties of the map, it is easy to verify that in the three-variable map each cell is adjacent to three other cells and in the four-variable map each cell is adjacent to four other cells.

Simplification and minimization of functions

A collection of 2^m cells, each adjacent to m cells of the collection, is called a *cube* and the cube is said to *cover* these cells. Each cube can be expressed by a product containing $n - m$ literals, where n is the number of variables on which the function depends. The m literals that are not contained in the product can be eliminated, because each of their 2^m combinations appear in the product with the same factor. For example, the square array of four 1's in Fig. 4.1d corresponds to

$$\begin{aligned} w'xy'z' + w'xy'z + wxy'z' + wxy'z &= xy'(w'z' + w'z + wz' + wz) \\ &= xy'. \end{aligned}$$

Similarly, the product expressing the linear array of four 1's is wx , since the values of both w and x are the same in the four cells while the value of yz is different in each cell.

Now consider the function f defined by the map of Fig. 4.1b. We could express f as the sum of three minterms. However, observing that the map consists of two pairs of adjacent cells, we can express f as the sum of two product terms:

$$f = yz' + xy$$

The use of cell 6 in forming both cubes is justified by the idempotent law (cf. Section 3.1). In this example, the corresponding algebraic manipulations

leading to the above result are

$$\begin{aligned}
 f &= x'yz' + xyz' + xyz \\
 &= x'yz' + xyz' + xyz' + xyz \\
 &= yz'(x' + x) + xy(z' + z) \\
 &= yz' + xy.
 \end{aligned}$$

In general, by the idempotent law any cell may be included in as many cubes as desired. For example, the function f defined by the map of Fig. 4.1d can be expressed as the sum of three products, corresponding to the three cubes indicated on the map, i.e.,

$$f = wx + xy' + wy'z'.$$

From the preceding discussion, we observe that a function f can be expressed as a sum of those product terms that correspond to the cubes necessary to cover all its 1-cells. The number of product terms in the expression for f is equal to the number of cubes, while the number of literals in each term is determined by the size of the corresponding cube. In order to obtain a minimal expression, we must cover all the 1-cells with the smallest number of cubes in such a way that each cube is as large as possible. Hence, a cube contained in a larger cube must never be selected. If there is more than one way of covering the map (i.e., its 1-cells) with the minimal number of cubes, we must select a covering that consists of larger cubes. Such a selection guarantees that the corresponding expression is indeed minimal and that no other expression containing the same number of terms, but fewer literals, exists. A cube contained in any combination of other cubes already selected in the covering of the map is redundant by virtue of the consensus theorem, Eq. (3.19). 

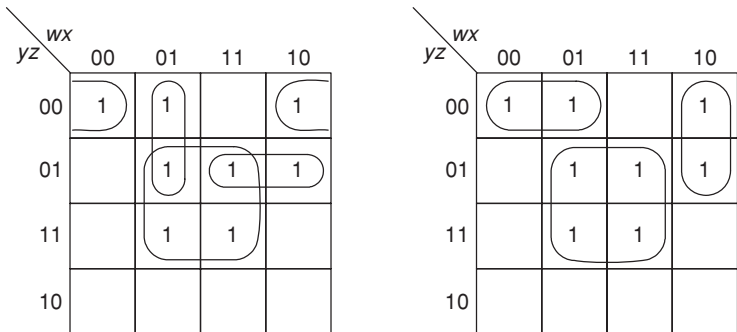
The foregoing discussion suggests the following rules for obtaining simple expressions for f .

1. Start by covering with cubes those 1-cells that cannot be combined with any other 1-cell, and continue to those which have only a single adjacent 1-cell and thus can form cubes of only two 1-cells.
2. Next, combine those 1-cells that yield cubes of four but are not part of any cube of eight cells, and so on.
3. A minimal expression is one that corresponds to a collection of cubes that are as large and as few in number as possible, such that every 1-cell in the map of the function is covered by at least one cube.

Example Two irredundant expressions for

$$f(w, x, y, z) = \sum(0, 4, 5, 7, 8, 9, 13, 15)$$

can be derived from the maps of Fig. 4.2. The expression derived from Fig. 4.2a is $f = x'y'z' + w'xy' + wy'z + xz$. Since none of the cubes is



(a) $f = x'y'z' + w'xy' + wy'z + xz$ is an irredundant expression.

(b) $f = w'y'z' + wx'y' + xz$ is the unique minimal expression.

Fig. 4.2 Two irredundant expressions for $f(w, x, y, z) = \sum(0, 4, 5, 7, 8, 9, 13, 15)$.

contained either within a combination of other cubes or within a larger cube, this expression is irredundant. However, since it does not contain the smallest possible number of terms, it is not a minimal expression. The expression derived from the map of Fig. 4.2b, $f = w'y'z' + wx'y' + xz$, is the unique minimal expression for f . There exist two more irredundant expressions for f , but neither of them is minimal.

Example The function $f(w, x, y, z) = \sum(1, 5, 6, 7, 11, 12, 13, 15)$ has only one irredundant form, as opposed to the preceding example. This unique minimal expression is derived from Fig. 4.3 and is found to be $f = wx'y' + wyz + w'xy + w'y'z$. Note that the dotted cube xz of four 1's becomes redundant if rule 1 is followed, since all its cells are covered by the other cubes.

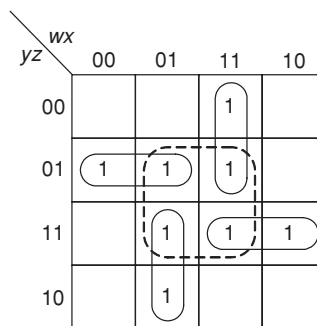
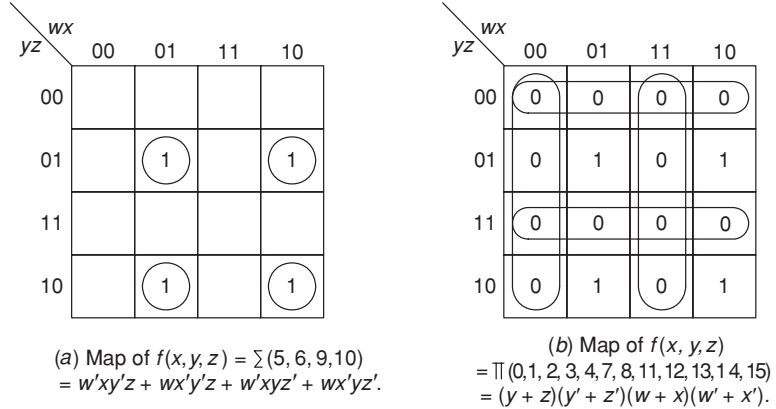


Fig. 4.3 Map for $f = wx'y' + wyz + w'xy + w'y'z$.

Fig. 4.4 Minimal sum-of-products and product-of-sums forms.



So far, we have specified a switching function by combining the 1-cells. Clearly, it may equally well be specified by the 0-cells. In the latter case, the expression yields the complement f' , whose 1's are the 0's of f and vice versa.

Determination of the minimal product of sums

The minimization of functions expressed as product of sums is the dual procedure of that just developed for the sum-of-products form. An immediate question arises as to whether the number of literals required in the minimal expressions of both forms is the same. Supposing that we have obtained a minimal sum-of-products expression for f , does this imply that the minimal product-of-sums expression will require at least as many literals? The answer to this question is negative, as is shown subsequently.

Consider the function $f(w, x, y, z) = \sum(5, 6, 9, 10)$. From the cubes shown in Fig. 4.4a, it is evident that no two 1-cells are adjacent. Thus f cannot be reduced, and its minimal sum-of-products form consists of 16 literals in four minterms:

$$f(w, x, y, z) = w'xy'z + w'xyz' + wx'y'z + wx'yz'.$$

The minimal product-of-sums expression for a function f is defined in an analogous manner to the minimal sum of products. It consists of the product of a minimum number of sum factors, provided that there is no other such product with the same number of factors and with fewer literals. The product-of-sums expression is obtained from the map in the same way as from the truth table. A variable corresponding to a 1 is complemented, and a variable corresponding to a 0 is uncomplemented. Cubes are formed of 0-cells instead of 1-cells and are selected in exactly the same manner as in the sum-of-products case. The minimal product-of-sums expression for f is derived from the map of Fig. 4.4b, i.e.,

$$f(w, x, y, z) = (y + z)(y' + z')(w + x)(w' + x').$$

This expression consists of only eight literals as against 16 in the sum-of-products form. Hence, if a minimal expression is sought, regardless of its form, both forms must be determined and the one with a smaller number of literals selected.

Don't-care combinations

So far, the functions considered have been completely specified for every combination of variables. There exist situations, however, where, while a function is to assume the value 1 for some combinations and the value 0 for others, it may assume either value for a number of combinations. Such situations may occur when the variables are not mutually independent; that is, dependency among the variables may preclude the occurrence of certain combinations, for which, consequently, the value of the function will not be specified. Combinations for which the value of the function is not specified are called *don't-care combinations*. The value of the function for such combinations is denoted by ϕ (or d).

In practice, when $x_1, x_2, \dots, x_n$ are variables designating the inputs to a switching circuit and when $f(x_1, x_2, \dots, x_n)$ designates its output, it often happens that for certain input combinations the value of the output is unspecified, either because these input combinations are invalid or because the precise value of the output is of no importance. Since each don't-care combination can be specified in either of two ways, i.e., 0 or 1, an incompletely specified function containing k don't-care combinations actually corresponds to a class of 2^k distinct functions. Our task is thus to choose the function (or functions) having the minimal representation.

When employing the map of an incompletely specified function, we assign the value 1 to selected don't-care combinations and the value 0 to others, in such a way as to increase the size of the selected cubes whenever possible. No cube containing only don't-care cells can be formed, because it is not required that the function equal 1 for these combinations.

Example Design a code converter that converts BCD messages into Excess-3 code. The converter has four input lines carrying signals labeled w, x, y , and z , and four output lines carrying signals f_1, f_2, f_3 , and f_4 . The inputs and outputs correspond, respectively, to BCD and Excess-3 coded messages. If the system operates properly then the input combinations will correspond to the decimal values 0 through 9, while the remaining six combinations, 10 through 15, will never occur and thus may be regarded as don't-care combinations. The code converter is designed by considering each output function separately. The truth table specifying the codes is shown in Fig. 4.5a and the resulting output functions in Fig. 4.5b.

The simplification of output functions is accomplished by use of the corresponding maps, as shown in Fig. 4.6. Don't-care combinations are

Decimal number	BCD inputs				Excess-3 outputs			
	w	x	y	z	f_4	f_3	f_2	f_1
0	0	0	0	0	0	0	1	1
1	0	0	0	1	0	1	0	0
2	0	0	1	0	0	1	0	1
3	0	0	1	1	0	1	1	0
4	0	1	0	0	0	1	1	1
5	0	1	0	1	1	0	0	0
6	0	1	1	0	1	0	0	1
7	0	1	1	1	1	0	1	0
8	1	0	0	0	1	0	1	1
9	1	0	0	1	1	1	0	0

(a) Truth table for BCD and Excess-3 codes

$$f_1 = \sum(0, 2, 4, 6, 8) + \sum_{\phi}(10, 11, 12, 13, 14, 15)$$

$$f_2 = \sum(0, 3, 4, 7, 8) + \sum_{\phi}(10, 11, 12, 13, 14, 15)$$

$$f_3 = \sum(1, 2, 3, 4, 9) + \sum_{\phi}(10, 11, 12, 13, 14, 15)$$

$$f_4 = \sum(5, 6, 7, 8, 9) + \sum_{\phi}(10, 11, 12, 13, 14, 15)$$

(b) Output functions

Fig. 4.5 Specifications of a code converter.

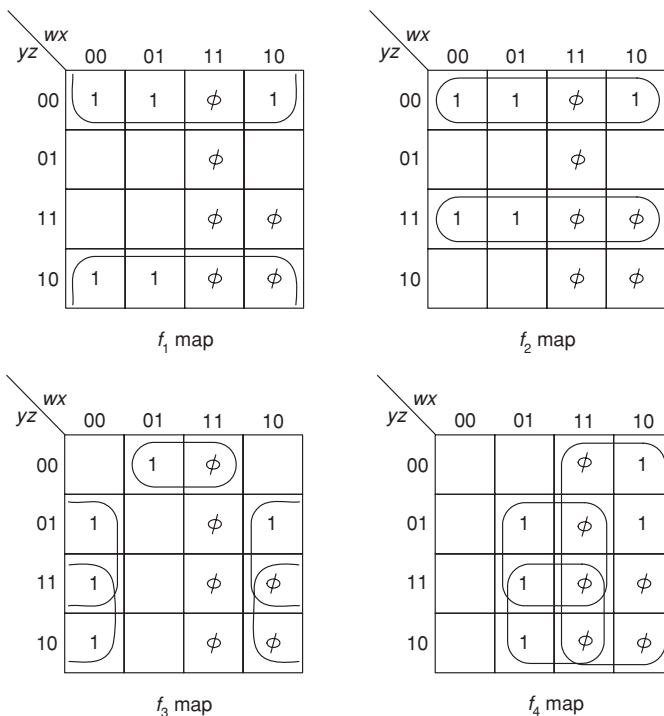


Fig. 4.6 Maps for a BCD-to-Excess-3 code converter.

considered and specified in each function regardless of the specification in other functions. Generally, the specification is done in such a way as to increase the size of the cubes in the map without making it necessary to select more cubes than would be necessary if fewer don't-cares were made 1's. The minimal functions derived from the maps are

$$\begin{aligned} f_1 &= z', \\ f_2 &= y'z' + yz, \\ f_3 &= x'y + x'z + xy'z', \\ f_4 &= w + xy + xz. \end{aligned}$$

A gate network¹ realizing the code translator is shown in Fig. 4.7. Note that if, owing to a malfunction in the message, an invalid input combination occurs then the output of the code converter will also be erroneous.

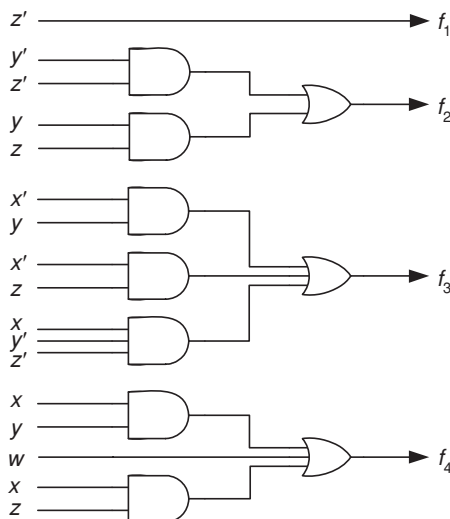


Fig. 4.7 A BCD-to-Excess-3 code converter.

A switching circuit in which a set of n input variables determines the values of two or more outputs is called a *multi-output circuit*. The above code converter is a four-output circuit.

¹ Any gate network that realizes a sum-of-products (or a product-of-sums) expression is called a *two-level* realization, since it consists of one level of AND (OR) gates driving a second-level OR (AND) gate. Thus, the longest path through which any input signal must pass until it reaches the output consists of two gates. A measure of the complexity of a network is either the overall number of gates or the total number of gate inputs. For example, the network of Fig. 4.7 consists of 10 gates and 23 gate inputs.

Fig. 4.8 Five-variable map with the locations of minterms.

$yz \backslash vwx$	000	001	011	010	110	111	101	100
00	0	4	12	8	24	28	20	16
01	1	5	13	9	25	29	21	17
11	3	7	15	11	27	31	23	19
10	2	6	14	10	26	30	22	18

The five-variable map

The minimization procedure described so far with respect to functions of three or four variables can be extended to the case of five or six variables. For functions of seven or more variables, the map is very large and its value as an effective tool in the minimization procedure decreases, since it becomes very difficult to keep track of adjacencies.

A five-variable map contains $2^5 = 32$ cells, as shown in Fig. 4.8. Each cell, in addition to being adjacent to four other cells, can be combined with a fifth cell on the other side of the center symmetry line. Thus, cell 9 in the map of Fig. 4.8 is adjacent to (and therefore may be combined with) cell 25, cell 15 is adjacent to 31, 4 to 20, and so on.

Example With the aid of a map, minimize the function

$$f(v, w, x, y, z) = \sum(1, 2, 6, 7, 9, 13, 14, 15, 17, 22, 23, 25, 29, 30, 31).$$

From the cubes shown in Fig. 4.9, we obtain the minimal sum-of-products expression

$$f(v, w, x, y, z) = x'y'z + wxz + xy + v'w'yz'$$

$yz \backslash vwx$	000	001	011	010	110	111	101	100
00								
01	1		1	1	1	1		1
11		1	1			1	1	
10	1	1	1			1	1	

Fig. 4.9 Map for $f(v, w, x, y, z) = x'y'z + wxz + xy + v'w'yz'$.

The extension of the map to six variables is accomplished in a similar manner. The map is a square consisting of 64 cells, where each cell is adjacent to six other cells. The actual construction of the map, the determination of the appropriate row and column headings, and the locations of the minterms are left to the reader as an exercise.

4.3 Minimal functions and their properties

In Section 4.1, we observed that there exists a distinction between irredundant and minimal expressions and that neither is necessarily unique. We shall now investigate the properties of these expressions and determine the characteristics of the product terms contained in a minimal sum-of-products expression.

Prime implicants

A switching function $f(x_1, x_2, \dots, x_n)$ is said to *cover* another function $g(x_1, x_2, \dots, x_n)$, this action being denoted by $f \supseteq g$, if f assumes the value 1 whenever g does. Thus, if f covers g then it has a 1 in every row of the truth table in which g has a 1. If f covers g and at the same time g covers f , then f and g are equivalent.

Let $f(x_1, x_2, \dots, x_n)$ be a switching function and $h(x_1, x_2, \dots, x_n)$ be a product of literals. If f covers h then h is said to *imply* f ; h is said to be an *implicant* of f . The implication is often denoted by $h \rightarrow f$.

Example If $f = wx + yz$ and $h = wxy'$ then f covers h and h implies f .

Definition 4.1 A *prime implicant* p of a function f is a product term covered by f such that the deletion of any literal from p results in a new product that is not covered by f . Alternatively stated, p is a prime implicant if and only if p implies f but does not imply any product with fewer literals that in turn also implies f . The set of all prime implicants of f will be denoted by P .

Example A prime implicant of $f = x'y + xz + y'z'$ is $x'y$, since it is covered by f and neither x' nor y alone implies f .

Theorem 4.1 Every irredundant sum-of-products equivalent to f is a union of prime implicants of f .

Proof Let f^* be an irredundant sum-of-products expression equivalent to f and suppose that f^* contains a product term q that is not a prime implicant. Since q is not a prime implicant, it is possible to replace it with another product that consists of fewer literals. Hence f contains redundant literals, which contradicts the initial assumption. $\diamond$

The next task is to generate the set of all prime implicants of f and from this set to select those prime implicants whose union yields a minimal expression for f . Suppose that f is given in a canonical sum-of-products form; then, by applying the *combining theorem* $Aa + Aa' = A$ to a pair of minterms, we obtain a product that implies f . Repeated applications of this theorem to all pairs of terms that differ in the value of just one variable yield a set of products, each of which implies f . A product that cannot be combined with any other product to yield a still smaller product, i.e., one with fewer literals, is a prime implicant of f . Thus, our first step in the determination of the minimal expression is a systematic combination of terms. The second step, that of selecting the minimal set of prime implicants, is in general more complicated, as will be demonstrated in the next section.

On the map for f , an irreducible product corresponds to a cube that is not contained in any larger cube. Consequently, the set P of all prime implicants can be obtained by writing down the products corresponding to all the cubes that are not contained in any larger cubes.

Example Consider the map of $f(w, x, y, z) = \sum(0, 4, 5, 7, 8, 9, 13, 15)$ given in Fig. 4.2. The set of all prime implicants of f is

$$P = \{xz, w'y'z', wx'y', x'y'z', w'xy', wy'z\}.$$

Note that xyz is not a prime implicant since it implies xz .

Deriving minimal expressions

An inspection of the maps in Fig. 4.2 reveals that the prime implicant xz *must* be contained in any irredundant expression equivalent to f , since it is the only product that covers the combinations 7 and 15. However, any other 1-cell is covered by two prime implicants and, consequently, none of them is essential for the specification of an irredundant expression.

A prime implicant p of a function f is said to be an *essential prime implicant* if it covers at least one minterm of f that is not covered by any other prime implicant. Since every minterm of f must be covered by an expression for f , all essential prime implicants must be contained in any irredundant expression for this function.

Example The prime implicants of the function

$$f(w, x, y, z) = \sum(4, 5, 8, 12, 13, 14, 15)$$

are all essential, as demonstrated by the map of Fig. 4.1d.

Example The map for the function $f(x, y, z) = \sum(0, 2, 3, 4, 5, 7)$ is shown in Fig. 4.10; it is known as a *cyclic* prime implicant map since no prime implicant is essential, all prime implicants have the same size, and every cell is covered by exactly two prime implicants. The reader can verify by means of this map the results obtained in an algebraic manner in Section 4.1.

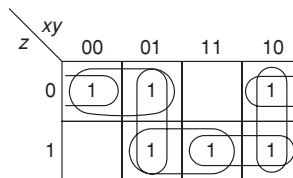


Fig. 4.10 A map for the function $f(x, y, z) = \sum(0, 2, 3, 4, 5, 7)$.

Since every minterm covered by a nonessential prime implicant is covered by at least two prime implicants, any nonessential prime implicant is covered by the sum of some prime implicants. For example, the prime implicant $w'xy'$ of the function whose map is shown in Fig. 4.2 is covered by the sum of the prime implicants xz and $w'y'z'$. An essential prime implicant, however, is not covered by any such sum.

When simplifying expressions by means of a map, we start by selecting essential prime implicants, if any. This is accomplished by first forming maximal cubes of those 1's that can be combined to form only one cube. Any other cube whose 1's are contained in one or more of these cubes corresponds to a redundant term and need not be considered further. We thus arrive at the following conclusion.

- The set of all essential prime implicants must be contained in any irredundant sum-of-products expression, while any prime implicant covered by the sum of the essential prime implicants must not be contained in an irredundant expression.

For example, the prime implicant xz of function f of Fig. 4.3 is covered by the sum of four essential prime implicants and, therefore, must not be contained in any irredundant expression for f . We can thus summarize the procedure for obtaining a minimal sum-of-products expression for a function f .

1. Determine all essential prime implicants and include them in the minimal expression.
2. Remove from the list of prime implicants all those that are covered by the essential prime implicants.
3. If the set derived in step 1 covers all the minterms of f then it is the unique minimal expression. Otherwise, select additional prime implicants such that

f is covered completely and such that the total number and size of the prime implicants thus added are minimal.

The execution of step 3 is not always straightforward. While in most cases with only a small number of variables this execution can be done by inspecting the map, in more complicated cases, and when the number of variables is large, a more systematic method is needed. The prime implicant chart presented in the next section is a possible tool aiding the search for a minimal expression.

4.4 The tabulation procedure for the determination of prime implicants

The Karnaugh map method described in the preceding sections is very useful for functions of up to six variables. In order to manipulate functions of a larger number of variables a more systematic procedure, preferably one that can be carried out by a computer, is necessary. The tabulation procedure, known also as the Quine–McCluskey method of reduction, satisfies the above requirements. It is suitable for hand computation and is also easily programmable.

The binary representation

The fundamental idea on which this procedure is based is that repeated applications of the combining theorem $Aa + Aa' = A$ to all adjacent pairs of terms yield the set of all prime implicants, from which a minimal sum may be selected. The technique will be introduced by minimizing the function

$$f_1(w, x, y, z) = \sum(0, 1, 8, 9) = w'x'y'z' + w'x'y'z + wx'y'z' + wx'y'z.$$

The first two and last two terms of f_1 can be combined to yield

$$\begin{aligned} f_1(w, x, y, z) &= w'x'y'(z' + z) + wx'y'(z' + z) \\ &= w'x'y' + wx'y'. \end{aligned}$$

These two terms can be combined in turn, and we obtain

$$\begin{aligned} f_1(w, x, y, z) &= x'y'(w' + w) \\ &= x'y'. \end{aligned}$$

In the first step we obtained, for each of the two pairs of adjacent terms, consisting of four literals per term, one term that consists of three literals. In the second step, these two terms were combined again and reduced to a single two-literal product. A similar result could have been obtained by initially combining the first and third and the second and fourth terms in the original function. However, no combination of the first and fourth or the second and third terms is possible because they are not adjacent. Therefore our first task is

to determine, in a simple and systematic way, which terms can (or cannot) be combined and to carry out all possible such combinations.

Two k -variable terms can be combined into a single $(k - 1)$ -variable term if and only if they have in common $k - 1$ identical literals and differ in just a single literal. The combined term consists of the product of the $k - 1$ identical literals while the variable, which is uncomplemented in one term and complemented in the other, is deleted. Thus, the terms $w'x'y'z'$ and $w'x'y'z$ can be combined to $w'x'y'$, while $w'x'y'z$ and $wx'y'z'$ cannot be combined, since they differ in two variables (i.e., w and z). If we consider the binary representation of the minterms, we observe that the necessary and sufficient condition for two minterms to be combinable is that their binary representations differ in just one position. For example, the representations for $w'x'y'z$ and $wx'y'z$ are 0001 and 1001, respectively. The combined term is denoted -001 , where the dash indicates that variable w has been absorbed and the combined term is $x'y'z$. The terms $w'x'y'z$ and $wx'y'z'$, however, cannot be combined since their binary representations 0001 and 1000 differ in two positions, i.e., in the first and fourth digits.

For the binary representations of two minterms to be different in just one position, it is necessary that their numbers of 1's differ by exactly one. Consequently, to facilitate the combination process the minterms are arranged in groups according to the number of 1's in their binary representation. With the following steps, the procedure becomes systematic.

1. Arrange all minterms in groups, such that all terms in the same group have the same number of 1's in their binary representation. Start with the least number of 1's and continue with groups of increasing numbers of 1's. The number of 1's in a term is called the *index* of that term.
2. Compare every term of the lowest-index group with each term in the successive group; whenever possible, combine the two terms being compared by means of the combining theorem $Aa + Aa' = A$. Repeat this by comparing each term in a group of index i with every term in the group of index $i + 1$ until all possible applications of the combining theorem have been exhausted.

Two terms from adjacent groups are combinable if their binary representations differ by just a single digit in the same position; the combined term consists of the original fixed representation, the different digit being replaced by a dash ($-$). A check mark ($\checkmark$) is placed next to every term which has been combined with at least one term. (Note that each term may be combined with several terms, but only a single check is required.)

3. Now compare the terms generated in step 2, in the same fashion: a new term is generated by combining two terms that differ by only a single 1 and whose dashes are in the same position. The process continues until no further combinations are possible. The remaining unchecked terms constitute the set of prime implicants of the function.

Fig. 4.11 Determination of the set of prime implicants for the function $f_2(w, x, y, z) = \sum(0, 1, 2, 5, 7, 8, 9, 10, 13, 15)$.

Step 1					Step 2					Step 3							
	w	x	y	z		w	x	y	z		w	x	y	z			
0	0	0	0	0	✓	0,1	0	0	0	–	✓	0,1,8,9	–	0	0	–	A
1	0	0	0	1	✓	0,2	0	0	–	0	✓	0,2,8,10	–	0	–	0	B
2	0	0	1	0	✓	0,8	–	0	0	0	✓	1,5,9,13	–	–	0	1	C
8	1	0	0	0	✓	1,5	0	–	0	1	✓	5,7,13,15	–	1	–	1	D
5	0	1	0	1	✓	1,9	–	0	0	1	✓						
9	1	0	0	1	✓	2,10	–	0	1	0	✓						
10	1	0	1	0	✓	8,9	1	0	0	–	✓						
7	0	1	1	1	✓	8,10	1	0	–	0	✓						
13	1	1	0	1	✓	5,7	0	1	–	1	✓						
15	1	1	1	1	✓	5,13	–	1	0	1	✓						
						9,13	1	–	0	1	✓						
						7,15	–	1	1	1	✓						
						13,15	1	1	–	1	✓						

The entire procedure is, actually, a mechanized process for combining and reducing all adjacent pairs of terms. The unchecked terms are the prime implicants of f , since each implies f and is not covered by any other term with fewer literals. We shall illustrate the procedure by applying it to the function

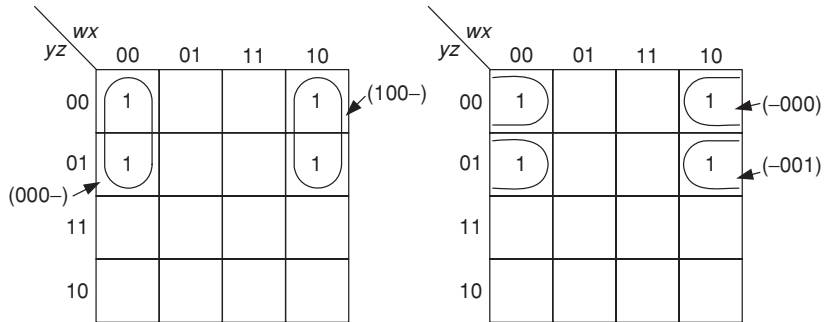
$$f_2(w, x, y, z) = \sum(0, 1, 2, 5, 7, 8, 9, 10, 13, 15).$$

The left-hand part of Fig. 4.11, corresponding to the application of step 1, consists of all minterms, arranged in groups of increasing indices. The reduced terms, after the first application of step 2, are given in the center part. For example, the combination of the terms 0000 and 0001 is recorded by writing 000– in its first row, where the dash indicates that variable z is redundant. The terms 0000 and 0001 in the left-hand part of the figure are now checked off. The same rule is applied repeatedly until all combinable terms are recorded in the center part.

The entire procedure is now repeated for the groups just formed in the center part of the figure. Again, only adjacent groups need be compared, and a new term is generated whenever two terms that differ in only one position and have their dashes in the same position are found. This procedure guarantees that the two combined terms actually consist of the same variables; that is, the same variable was deleted from both terms in the previous step. The new terms are recorded in the right-hand part of the figure, while the appropriate terms are checked off. For example, the term 000– can be combined with 100– to form –00–, which is recorded in the first row.

While recording the terms in the right-hand part of the figure, we observe that each term is generated in two ways. For example, the term –00– is generated in the preceding manner as well as by combining –000 and –001. Clearly it is

Fig. 4.12 Illustration of the two ways of generating a term.



sufficient to record it once, but checks must be placed next to each of the four terms 000-, 100-, -000, and -001. The cause of this phenomenon is that every four-cell cube can be formed by combining two adjacent two-cell cubes in two ways, as illustrated for the preceding example in Fig. 4.12.

The terms recorded in the right-hand part of Fig. 4.11 and labeled *A*, *B*, *C*, and *D* cannot be combined with any other term and, therefore, form the set of prime implicants of f_2 . From this set, we must now select a minimal subset whose union is equivalent to f_2 . This is accomplished by means of the prime implicant chart presented in the Section 4.5.

The decimal representation

The tabulation procedure can be simplified further by adopting the decimal code for the minterms rather than their binary representation. Two minterms can be combined only if they differ by a power of 2, that is, only if the difference between their decimal codes is 2^i . The combined term consists of the same literals as the minterms with the exception of the variable whose weight is 2^i , which is deleted. For example, if we consider the function $f_1(w, x, y, z) = \sum(0, 1, 8, 9)$, the minterms 1 and 9 differ by $2^3 = 8$ and consequently the variable w , whose weight is 8, is deleted. This combining process, which is recorded by placing the weight of the redundant variable in parentheses, e.g., 1, 9 (8), is simply a numerical way of describing the algebraic manipulation $w'x'y'z + wx'y'z = x'y'z$. Similarly, the combination of the minterms 0 and 8 is written as 0, 8 (8).

The condition that the decimal codes of two combinable terms must differ by a power of 2 is necessary but not sufficient. Two terms whose codes differ by a power of 2 but which have the same index cannot be combined, since they differ by more than one variable. Similarly, if a term with a smaller index has a higher decimal value than another term whose index is higher, then the two terms cannot be combined although they may differ by a power of 2. For example, the terms 9 and 7 in Fig. 4.11, whose indices are 2 and 3, respectively, cannot be combined since they differ in the values of three variables. Except for the

Fig. 4.13 Tabulation procedure for $f_3(v, w, x, y, z)$ using decimal notation. The tables are derived in the order (a)–(d).

1 ✓	1, 17 (16) <i>H</i>	17, 19, 21, 23 (2, 4) ✓
2 ✓	2, 18 (16) <i>G</i>	17, 19, 25, 27 (2, 8) ✓
12 ✓	12, 13 (1) <i>F</i>	17, 21, 25, 29 (4, 8) ✓
17 ✓	17, 19 (2) ✓	13, 15, 29, 31 (2, 16) <i>B</i>
18 ✓	17, 21 (4) ✓	19, 23, 27, 31 (4, 8) ✓
20 ✓	17, 25 (8) ✓	21, 23, 29, 31 (2, 8) ✓
24 ✓	18, 19 (1) <i>E</i>	25, 27, 29, 31 (2, 4) ✓
13 ✓	20, 21 (1) <i>D</i>	(c)
19 ✓	24, 25 (1) <i>C</i>	
21 ✓	13, 15 (2) ✓	17, 19, 21, 23, 25, 27, 29, 31 (2, 4, 8) <i>A</i>
25 ✓	13, 29 (16) ✓	(d)
15 ✓	19, 23 (4) ✓	
23 ✓	19, 27 (8) ✓	
27 ✓	21, 23 (2) ✓	
29 ✓	21, 29 (8) ✓	
31 ✓	25, 27 (2) ✓	
(a)	25, 29 (4) ✓	
	15, 31 (16) ✓	
	23, 31 (8) ✓	
	27, 31 (4) ✓	
	29, 31 (2) ✓	
	(b)	

above phenomenon, the tabulation procedure using the decimal representation is completely analogous to that using the binary representation.

The tabulation procedure can easily handle the case of don't-care combinations. During the process of generating the set of prime implicants, don't-care combinations are regarded as true combinations, that is, combinations for which the function assumes value 1. This, in effect, increases to the maximum the number of possible prime implicants. The don't-care terms are, however, not considered in the next step, that of selecting a minimal set of prime implicants, as will be shown in the following section.

The tabulation procedure for generating the set P of prime implicants for the function

$$f_3(v, w, x, y, z) = \sum (13, 15, 17, 18, 19, 20, 21, 23, 25, 27, 29, 31) + \sum_{\phi} (1, 2, 12, 24)$$

is shown in Fig. 4.13. This set consists of eight prime implicants, denoted A through H , i.e.,

$$P = \{vz, wxz, vwx'y', vw'xy', vw'x'y, v'wxy', w'x'yz', w'x'y'z'\}.$$

Fig. 4.14 Prime implicant chart for $f_2(w, x, y, z)$ of Fig. 4.11.

	0	1	2	5	7	8	9	10	13	15
$A = x'y'$	x	x				x	x			
$B = x'z'$	x		⊗			x		⊗		
$C = y'z$		x		x			x		x	
$D = xz$				x	⊗				x	⊗

The selection of the prime implicants to be used in the minimal sum is accomplished with the aid of the prime implicant chart presented in the next section.

4.5 The prime implicant chart

The *prime implicant chart* displays pictorially the covering relationships between the prime implicants and minterms of a function. It consists of an array of u columns and v rows, where u and v designate the number of minterms for which the function takes on the value 1 and the number of prime implicants, respectively. The entries of the i th row in the chart consist of $\times$'s placed at its intersections with columns corresponding to minterms covered by the i th prime implicant. For example, the prime implicant chart of $f_2(w, x, y, z) = \sum(0, 1, 2, 5, 7, 8, 9, 10, 13, 15)$ is shown in Fig. 4.14. It consists of 10 columns corresponding to the minterms of f_2 , and four rows which correspond to the prime implicants A , B , C , and D generated in Fig. 4.11. Row C contains four $\times$'s at the intersections with columns 1, 5, 9, and 13, because these minterms are covered by the prime implicant C . A row is said to *cover* the columns in which it has $\times$'s.

The problem now is to select a minimal subset of prime implicants such that each column contains at least one $\times$ in the rows corresponding to the selected subset and the total number of literals in the prime implicants selected is as small as possible. These requirements guarantee that the union of the selected prime implicants is indeed equivalent to the original function f , and that no other expression containing fewer literals and equivalent to f can be found.

Essential rows

If any column contains just a single $\times$ then the prime implicant corresponding to the row in which this $\times$ appears is essential and consequently must be included in any irredundant expression for f . The $\times$ is circled, and a check mark is placed next to the essential prime implicant. The row that corresponds to an essential prime implicant is referred to as an *essential row*. Once an essential prime implicant has been selected, all the minterms it covers are checked off. For example, essential prime implicant B covers, in addition to columns 2 and

10, columns 0 and 8. Consequently columns 0, 2, 8, and 10 are checked off. If, after all essential prime implicants and their corresponding columns have been checked, the entire function is covered, i.e., every column is checked off, then the union of all essential prime implicants yields the minimal expression. If this is not the case then additional prime implicants are necessary.

The two essential prime implicants B and D of f_2 cover all the minterms except 1 and 9. These minterms may be covered by either prime implicant A or C , and since both are expressed with the same number of literals, we obtain two minimal expressions for f_2 , namely,

$$f_2(w, x, y, z) = x'z' + xz + x'y'$$

and

$$f_2(w, x, y, z) = x'z' + xz + y'z.$$

Don't-care combinations

Don't-care minterms need not be listed as column headings in the prime implicant chart, since they do not have to be covered by the minimal expression. By not listing them, we actually leave the specification of the don't-care terms open; that is, if a minimal expression contains a prime implicant derived from a don't-care combination, this amounts to specifying that combination as 1; otherwise, the don't-care combination is, in effect, assigned the value 0. The prime implicant chart thus yields a minimal expression of a function which covers all the *specified* minterms.

The prime implicant chart for the function

$$f_3(v, w, x, y, z) = \sum (13, 15, 17, 18, 19, 20, 21, 23, 25, 27, 29, 31) + \sum_{\phi} (1, 2, 12, 24),$$

whose prime implicants have been computed in Fig. 4.13, is shown in Fig. 4.15.

Fig. 4.15 The prime implicant chart for $f_3(v, w, x, y, z)$ of Fig. 4.13.

[illegible]

The selection of nonessential prime implicants is facilitated by the initial listing of prime implicants in a descending order, according to the number of minterms they cover. Thus, prime implicants that are located in a higher group in the chart are expressed with fewer literals than those located in a lower group. A horizontal line across the chart separates one group from the other.

The essential prime implicants in the chart of Fig. 4.15 are A , B , and D . They cover all the specified minterms with the exception of 18. This last minterm can be covered by either of prime implicants E and G and, since both have the same number of literals, two minimal expressions can be found, namely,

$$f_3(v, w, x, y, z) = vz + wxz + vw'xy' + vw'x'y$$

and

$$f_3(v, w, x, y, z) = vz + wxz + vw'xy' + w'x'yz'.$$

Determination of the set of all irredundant expressions

So far, we have been able to determine minimal sum-of-products expressions by inspecting the prime implicant chart. In more complex cases, however, the inspection process becomes prohibitively time consuming, and different techniques are in order. As an illustration, consider the minimization of the function

$$f_4(v, w, x, y, z) = \sum(0, 1, 3, 4, 7, 13, 15, 19, 20, 22, 23, 29, 31).$$

The corresponding prime implicant chart is shown in Fig. 4.16a, where the essential prime implicants and all minterms covered by them have been checked off. While every irredundant expression must contain the prime implicants A and C , none may contain B , since B covers only terms already covered by A and C . The reduced chart, which results after the removal of rows A , B , and C and all columns covered by them, is shown in Fig. 4.16b. Every column of the reduced chart contains two $\times$'s, and our task is to select a minimal number of additional prime implicants so as to cover the entire function.

Utilizing the techniques of propositional calculus, we associate a two-valued variable with each remaining prime implicants. The truth value of such a variable is 1 if the corresponding prime implicant is included in the irredundant expression, and is 0 if it is not. Define a *prime implicant function* p to be equal to 1 if each column is covered by at least one of the chosen prime implicants and 0 if it is not. For example, column 0 can be covered by either row H or row I . Consequently, either H or I must be included in any irredundant expression. Similarly, either G or I must also be included, since only they have $\times$'s in column 1. Deriving the appropriate expressions from the remaining columns

Fig. 4.16 Determination of all irredundant expressions for $f_4 = \sum (0, 1, 3, 4, 7, 13, 15, 19, 20, 22, 23, 29, 31)$.

	0	1	3	4	7	13	15	19	20	22	23	29	31
$\checkmark A = wxz$						⊗	x					⊗	x
$B = xyz$					x		x				x		x
$\checkmark C = w'yz$			x		x			⊗			x		
$D = vw'xy$										x	x		
$E = vw'xz'$									x	x			
$F = w'xy'z'$				x					x				
$G = v'w'x'z$	x	x											
$H = v'w'y'z'$	x			x									
$I = v'w'x'y'$	x	x											

(a) Prime implicant chart.

	0	1	4	20	22
D					x
E				x	x
F			x	x	
G		x			
H	x		x		
I	x	x			

(b) Reduced prime implicant chart.

of Fig. 4.16b, we obtain the expression for p ,

$$p = (H + I)(G + I)(F + H)(E + F)(D + E),$$

which can also be written as a sum of products,

$$p = EHI + EFI + DFI + EGH + \textcolor{red}{DFGH}.$$

From this expression for p we find that at least three rows are needed to cover the reduced chart, for example rows E , H , and I , or rows E , F , and I . There are five irredundant expressions for f_4 , corresponding to the five product terms for which p assumes the value 1. Also, since all the prime implicants that correspond to the rows of the reduced chart have the same number of literals, there are only four minimal expressions, corresponding to the first four terms in p . Each of these minimal expressions is obtained by forming the sum of the essential prime implicants A and C and a minimal number of prime implicants necessary to set p equal 1. Thus we have

$$\begin{aligned} f_4(v, w, x, y, z) &= wxz + w'yz + vw'xz' + v'w'y'z' + v'w'x'y', \\ f_4(v, w, x, y, z) &= wxz + w'yz + vw'xz' + w'xy'z' + v'w'x'y', \\ f_4(v, w, x, y, z) &= wxz + w'yz + vw'xy + w'xy'z' + v'w'x'y', \\ f_4(v, w, x, y, z) &= wxz + w'yz + vw'xz' + v'w'x'z + v'w'y'z'. \end{aligned}$$

The foregoing method for determining the irredundant sets of prime implicants can be applied directly to the prime implicant chart, instead of to the reduced chart. However, the prime implicant function will be, in most cases, considerably simpler if first the essential rows and columns covered by them are removed. Note that in deciding whether a product term in p corresponds to a minimal expression, two factors must be considered: the number of prime implicants and the number of literals in each such prime implicant.

Reduction of the chart

In general, prime implicant charts are not as simple as the examples we have given, and more elaborate techniques for manipulating them are required. Whenever our aim is limited to finding just one minimal expression rather than all minimal expressions, the selection of prime implicants may be considerably simplified. Consider the minimization of the function

$$f_5(v, w, x, y, z) = \sum (1, 3, 4, 5, 6, 7, 10, 11, 12, 13, 14, 15, 18, 19, 20, 21, 22, 23, 25, 26, 27).$$

Its prime implicant chart is shown in Fig. 4.17a, where the essential prime implicants A , B , J , and K and all minterms covered by them have been checked off.

The reduced chart, which is obtained by removing the essential rows and the columns covered by them, is shown in Fig. 4.17b. Although none of the rows in the reduced chart is essential, some of them may be removed. For example, row H has an $\times$ in column 19, while row G has $\times$'s in columns 19 and 11. Since both prime implicants G and H belong to the same group in the chart, i.e., both are expressed with the same number of literals, the removal of row H cannot prevent us from finding at least one minimal expression. In other words, two expressions that are identical except that one contains G while the other contains H will have the same number of literals; and since G covers the minterm covered by H , it can replace H in every expression for f_5 without affecting its logic value or its number of literals. Note that the converse is not true, since the removal of row G may leave column 11 without any $\times$ in a row whose corresponding prime implicant must be contained in the minimal expression.

A row U of a prime implicant chart is said to *dominate* another row V of that chart if U covers every column covered by V . Generalizing the preceding arguments, we conclude that, if row U dominates row V and the prime implicant corresponding to row U does not have more literals than the prime implicant corresponding to row V , then row V can be deleted from the chart. Thus, row I of Fig. 4.17b can be deleted because it is dominated by row G and, similarly, rows D and F are removed because they are dominated by rows C and E , respectively. The final reduced chart is shown in Fig. 4.17c. It contains three

Fig. 4.17 Minimization of $f_5 = \sum(1, 3, 4, 5, 6, 7, 10, 11, 12, 13, 14, 15, 18, 19, 20, 21, 22, 23, 25, 26, 27)$.

	1	3	4	5	6	7	10	11	12	13	14	15	18	19	20	21	22	23	25	26	27
$\vee A = w'x$			x	x	x	x										⊗	⊗	x	x		
$\vee B = v'x$			x	x	x	x			⊗	⊗	x	x									
$C = vx'y$														x	x					x	x
$D = vw'y$														x	x			x	x		
$E = wx'y$							x	x												x	x
$F = v'wy$							x	x			x	x									
$G = x'yz$		x						x							x						x
$H = w'yz$		x					x							x				x			
$I = v'yz$		x					x		x				x								
$\vee J = v'w'z$	⊗	x		x		x															
$\vee K = vwx'z$																			⊗		x

(a) Prime implicant chart.

	10	11	18	19	26
C			x	x	x
D			x	x	
E	x	x			x
F	x	x			
G		x		x	
H				x	
I		x			

(b) Reduced prime implicant chart.

	10	11	18	19	26
$\vee C$			⊗	x	x
$\vee E$	⊗	x			x
G		x		x	

(c) Final chart.

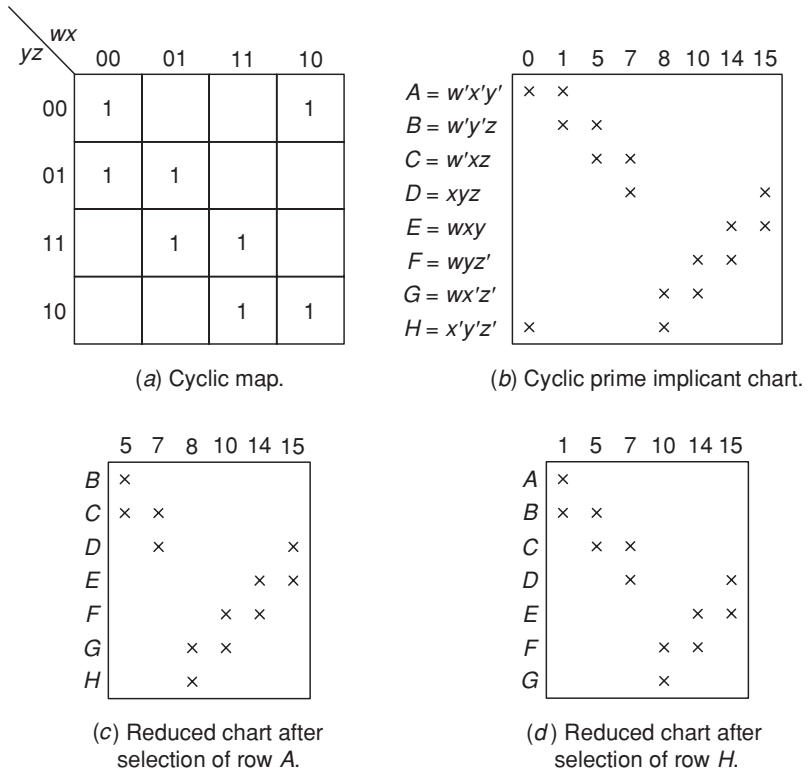
rows, of which two (C and E) must be included in the minimal expression, since only they cover columns 18 and 10, respectively. Clearly, the inclusion of C and E is also sufficient, since they cover all the columns not covered by the essential prime implicants. The minimal expression for f_5 thus consists of the prime implicants A , B , J , K , C , and E , i.e.,

$$f_5(v, w, x, y, z) = w'x + v'x + v'w'z + vwx'z + vx'y + wx'y.$$

Prime implicant charts can also be reduced by removing certain columns. Consider, for example, columns 10 and 11 in Fig. 4.17b. In order to cover column 10, either row E or F must be selected, whereby column 11 will also automatically be covered since it has $\times$'s in rows E and F . The converse is not true, since column 11 can also be covered by row G , but this will not cover column 10.

A column i in a prime implicant chart is said to *dominate* another column j of that chart if i has an $\times$ in every row in which j has an $\times$. Clearly, any minimal expression derived from a chart which contains both columns i and j can be derived from a chart which contains only the dominated column. Hence, *if column i dominates column j , then column i can be deleted from*

Fig. 4.18 Minimization of $f_6 = \sum(0, 1, 5, 7, 8, 10, 14, 15)$ by the branching method.



the chart without affecting the search for a minimal expression. In fact, the removal of dominating columns does not prevent us from finding all minimal expressions.

Note that, when reducing columns the *dominating* ones are removed, while of the rows the *dominated* ones are deleted. The removal of dominated rows and dominating columns may alternate a number of times; that is, we may start by removing dominated rows and dominating columns. This in turn may create new dominated rows that can be removed, and so on.

The branching method

It may happen that a prime implicant chart has no essential prime implicants, dominated rows, or dominating columns. Whenever this happens, a different approach must be taken, called the *branching method*. Consider, for example, the function

$$f_6(w, x, y, z) = \sum(0, 1, 5, 7, 8, 10, 14, 15),$$

whose map, which is cyclic, is given in Fig. 4.18a. Eight prime implicants of equal size are derived from the map and are shown in the chart of Fig. 4.18b,

where each prime implicant covers two minterms and each minterm is covered by two prime implicants. Such a chart is called a *cyclic* prime implicant chart.

In order to find a minimal expression for f_6 , it is necessary to make an arbitrary selection of one row and then apply the above reduction procedure. Consider, for example, column 0 in Fig. 4.18b. It can be covered by either row A or H . Consequently, one of these rows must be included in any minimal expression. If row A is arbitrarily chosen, the chart of Fig. 4.18c results. In this chart row B is dominated by row C and row H is dominated by row G . After removal of these dominated rows, we find that rows C and G must be selected, since only they cover columns 5 and 8, respectively. This selection, in turn, implies the inclusion of row E in the expression for f_6 , i.e.,

$$f_6(w, x, y, z) = w'x'y' + w'xz + wxy + wx'z'.$$

The entire process must now be repeated for row H as the initial selection. The removal of this row results in the chart of Fig. 4.18d. This chart is again reduced by removing the dominated rows A and G and including the prime implicants B , D , and F in the expression for f_6 :

$$f_6(w, x, y, z) = w'y'z + xyz + wyz' + x'y'z'.$$

Since the two expressions for f_6 have the same number of literals, both are minimal.

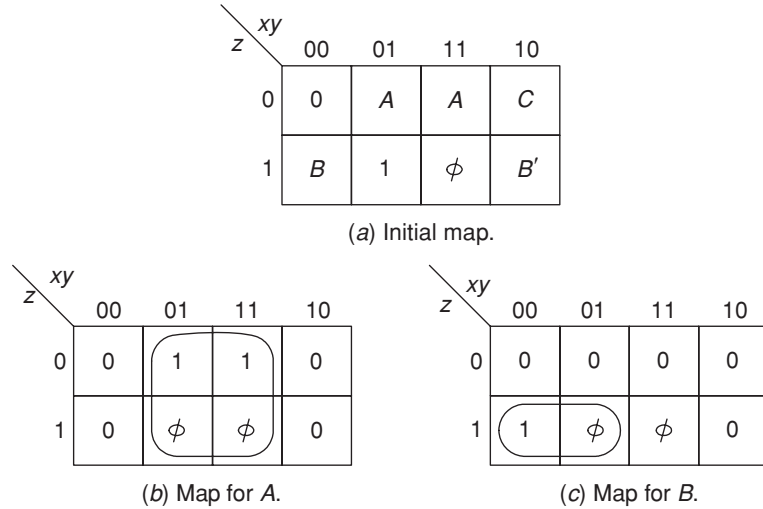
In general, there is no guarantee that the initial arbitrary selection will result in a minimal expression. It is, therefore, necessary to repeat the process for each row that could be substituted for the initially selected one.

Although the prime implicant chart of a function whose map is cyclic is itself always cyclic, it is possible to encounter cyclic charts in the process of reducing a prime implicant chart that corresponds to a noncyclic map. Moreover, a cyclic chart may result while applying the branching process and reducing another cyclic chart. Whenever such a situation occurs, another arbitrary row selection must be made and all alternative expressions must be obtained, such that a minimal one may be selected.

4.6 Map-entered variables

The Karnaugh map can be made a considerably more powerful tool if the variables themselves are entered into the map cells. In the preceding utilization of the map, the function value associated with a particular combination was entered in the corresponding cell, that is, a value of 1, 0, or don't-care is entered into a cell. In practice, it often happens that, for a particular combination, the function value is neither a constant (i.e., 0 or 1) nor a don't care but, rather, depends on the value of some other external variable. For example, the entry in cell $xyz = 010$ in the map of Fig. 4.19a is A . This implies that the value

Fig. 4.19 Deriving expressions from map-entered variables.



of the function f for this combination is a function of the variable A , that is, f is equal to 1 if $A = 1$ and f is equal to 0 if $A = 0$. Similarly, the value of f corresponding to the input combination $xyz = 001$ depends on the value of variable B , while the value of f corresponding to input combination $xyz = 101$ depends on the value of B' .

A map of the type shown in Fig. 4.19a, in which some cell entries are external variables, is said to have *map-entered variables*. A major advantage of such a map is that with an n -variable map (i.e., a map containing 2^n cells), we can specify functions of more than n variables. The three-variable map shown in Fig. 4.19a, for example, specifies a function of six variables, x , y , z and A , B , C .

The product term that corresponds to a cell-entered variable is equal to the product of the variable entered into the cell and the combination that identifies the cell. For example, the product corresponding to cell 010 is $Ax'y'z'$ and the product corresponding to cell 101 is $B'xy'z$.

The procedure for covering such a map and generating a simple expression for the corresponding function can be summarized as follows.

1. Treat all map-entered variables as 0's and find a minimal expression for the resulting map.
2. To cover the first map-entered variable, say A , treat all other map-entered variables as 0's and treat all 1's as don't-cares. Find a minimal cover for the resulting map.
3. Repeat step 2 for each map-entered variable. (Note that, in this context, a variable and its complement are treated as distinct variables, i.e., B and B' in Fig. 4.19a are distinct variables.)

Following this procedure, we can find a minimal expression corresponding to the map in Fig. 4.19a. From step 1, it is evident that the 1 in the map is covered by the cube yz . Step 2 for variable A is illustrated in Fig. 4.19b. Clearly, the corresponding term is Ay . Similarly, from Fig. 4.19c, we obtain the term for B , namely, $Bx'z$. The terms for B' and C are found in a similar manner and the entire function is given by

$$f = yz + Ay + Bx'z + B'xz + Cxy'z'.$$

4.7 Heuristic two-level circuit minimization

The prime implicant chart method requires that first all prime implicants are found and then a minimal subset of these prime implicants that covers all the minterms of the function is chosen. If more than one subset is of minimal cardinality then the one with fewest literals is chosen. The problem with this approach is that it may become impractical for many functions of interest. One reason is that for an n -variable function, the number of prime implicants can be as large as $3^n/n$. The second reason is that prime implicant chart covering can itself be a very time-consuming process.

Heuristic two-level circuit minimization tries to alleviate the above problem by reducing the number of prime implicants that need to be tackled. A very successful computer-aided design tool that encapsulates this approach is called *ESPRESSO*. We shall briefly discuss the minimization approach used in *ESPRESSO* next.

There are three main steps in *ESPRESSO*: expand, reduce and irredundant, which we now describe.

- The *expand* step targets implicants and expands them into prime implicants. Any implicants that are now covered by the expanded prime implicant are omitted from any further consideration.
- The *reduce* step transforms the prime implicants into implicants of the least possible size such that all the minterms of the function are still covered. This actually makes the implementation suboptimal but may lead to better solutions later.
- The *irredundant* step chooses a minimal subset of the prime implicants obtained so far such that the subset covers all the minterms of the function. This is similar to prime implicant chart covering. However, since the number of prime implicants targeted is usually much smaller, the process is not as time consuming.

The direction and order in which an implicant is expanded into a prime implicant has a bearing on the quality of the final result.

Example Consider the circled implicant $x'y'z'$ in the map shown in Fig. 4.20. Suppose that it is expanded in the y direction first. Then we arrive at the prime implicant $x'z'$. However, if we expand it in the x direction first and then the z direction, we arrive at prime implicant y' via the following route: $x'y'z' \rightarrow y'z' \rightarrow y'$. We actually arrive at prime implicant y' by expanding in another order of directions, first z and then x , as follows: $x'y'z' \rightarrow x'y' \rightarrow y'$.

		xy			
		z	00	01	11
z	0	1	1		1
	1	1			1

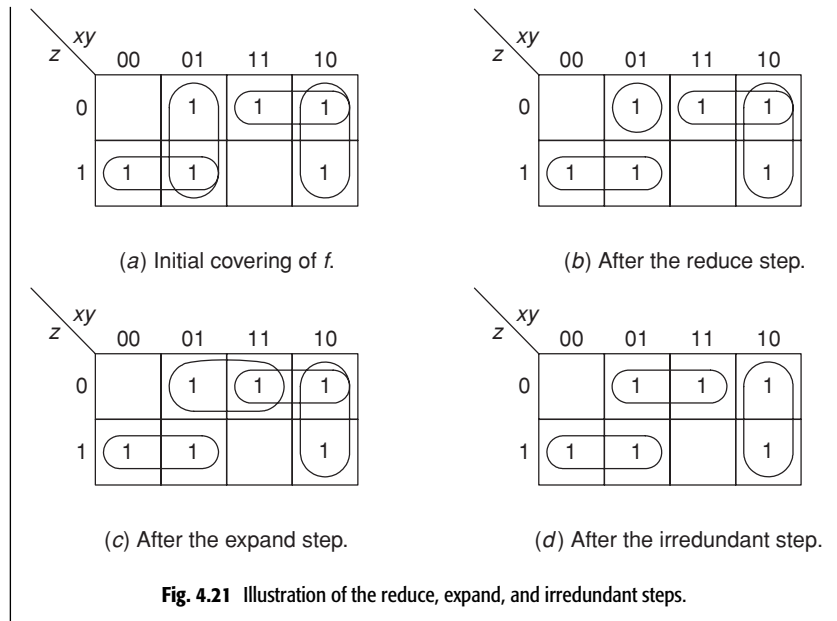
Fig. 4.20 Example illustrating expansion direction and order.

Since all possible prime implicants of the targeted function will not be generated, the quality of the prime implicants generated, i.e., how many minterms they cover, is important. The heuristics for determining a good expansion direction and order are included in *ESPRESSO*.

Since the implicants obtained after the reduce step need not be prime, it is followed by the expand and irredundant steps to derive another, possibly superior, covering of the minterms of the function. This process continues until it is no longer possible to improve on the best solution seen so far regarding the number of product terms or the number of literals included in them. Of course, in order to save time, essential prime implicants can be identified and set aside so that they are not subjected to further transformation.

The different steps of heuristic minimization are illustrated next.

Example Consider the initial set of prime implicants, shown in Fig. 4.21a, that covers all the minterms of function f . Such a set could be obtained by applying expand and irredundant steps to the initial set of minterms. Suppose that the prime implicant $x'y$ is now reduced to the implicant $x'y'z'$, as shown in Fig. 4.21b. When the implicant $x'y'z'$ is now expanded in another direction, the prime implicant yz' is obtained, as shown in Fig. 4.21c. The prime implicant xz' can now be removed in the irredundant step since its minterms are covered by the remaining prime implicants, thus obtaining the covering of minterms shown in Fig. 4.21d. This corresponds to the minimal sum-of-products $x'z + yz' + xy'$. This expression is obviously superior to the original expression, $x'z + x'y + xz' + xy'$.



x	y	z	f		x	y	z	f		x	y	z	f		x	y	z	f		x	y	z	f
0	0	1	1		0	–	1	1		0	–	1	1		0	–	1	1		0	–	1	1
0	1	0	1	expand and	0	1	–	1	reduce	0	1	0	1	expand	–	1	0	1	irredundant	–	1	0	1
0	1	1	1	$\Rightarrow$	1	–	0	1	$\Rightarrow$	1	–	0	1	$\Rightarrow$	1	–	0	1	$\Rightarrow$	1	0	–	1
1	0	0	1	irredundant	1	0	–	1		1	0	–	1		1	0	–	1					
1	0	1	1																				
1	1	0	1																				

Fig. 4.22 Transformations using encoded truth tables.

The input to *ESPRESSO* is typically an encoded truth table, similar to those used in the tabulation procedure. Truth tables equivalent to the set of transformations performed in the example above are shown in Fig. 4.22. The set of minterms of function f is subjected to expand and irredundant steps to obtain the initial covering containing the prime implicants $x'z$, $x'y$, xz' and xy' . Then, the reduction of prime implicant $x'y$ to the implicant $x'yz'$ is depicted by the transformation of 01– to 010. The expansion step converts 010 to –10. Finally, the irredundant step eliminates 1–0.

4.8 Multi-output two-level circuit minimization

In the preceding sections, we have dealt with single-output two-level circuit minimization. However, in general, most circuits that we might want to design have multiple outputs. In this section, we shall see how such multi-output circuits can be minimized.

A trivial way to deal with an n -output circuit is to treat it as n single-output circuits and minimize them separately.

Example Consider the functions f_1 and f_2 shown in Fig. 4.23 and the prime implicants shown in the maps. Since all four prime implicants are essential, the corresponding two-level circuit can be derived as shown in the figure.

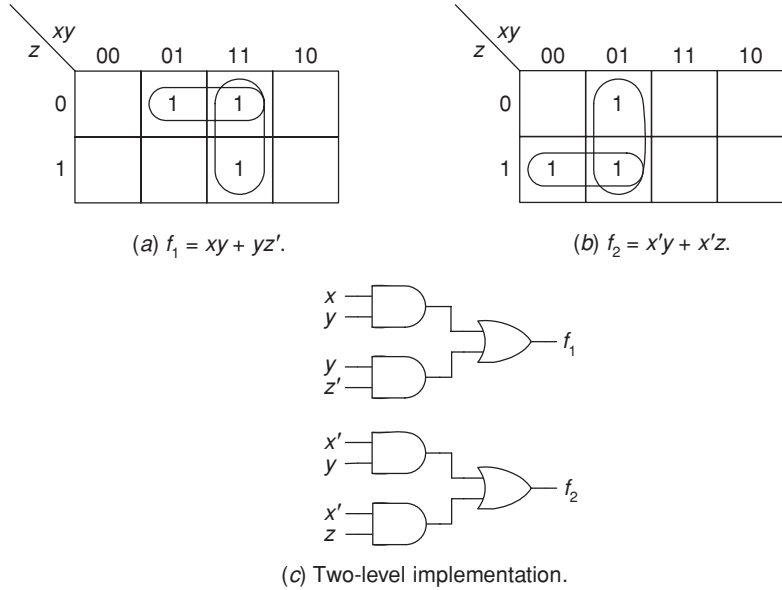


Fig. 4.23 A separately minimized two-level circuit.

The above approach, however, can be suboptimal. The reason is that it does not exploit the possibility of sharing logic among different outputs. To enable sharing, the concept of the *multi-output prime implicant* is needed. Suppose that there are only two output functions, f_1 and f_2 . Then, their multi-output prime implicants are the prime implicants of f_1 and f_2 as well as those of the product $f_1 f_2$. Similarly, if there are three output functions, f_1 , f_2 and f_3 , then their multi-output prime implicants are the prime implicants of f_1 , f_2 , f_3 , $f_1 f_2$, $f_1 f_3$, $f_2 f_3$, and $f_1 f_2 f_3$. In general, for n outputs the number of functions one has to consider is $2^n - 1$.

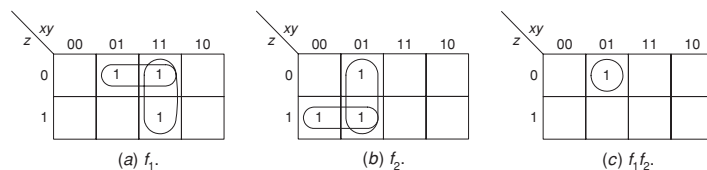
Consider a scenario in which a prime implicant of the function f_1 is also a prime implicant of the function $f_1 f_2$. Then further consideration of this prime implicant is given only for $f_1 f_2$, not for f_1 . The reason is that this enables sharing of the prime implicant among more outputs. In general, if a prime implicant of the function $f_1 f_2 \cdots f_i$ is also a prime implicant of a product function that includes all these individual functions, e.g., $f_1 f_2 \cdots f_i f_j$, the

prime implicant is only considered for the latter, in order to enable greater sharing.

The next step is to obtain an *augmented prime implicant chart*. This augmented chart has rows corresponding to each of the $2^n - 1$ functions that has at least one prime implicant deserving further consideration and columns corresponding to the minterms of each individual function. If the objective is to minimize the number of gates in the multi-output two-level implementation then the usual steps of identifying the essential prime implicants and removing dominated rows and dominating columns can be used to simplify the augmented chart, using the branching method or the prime implicant function when necessary. However, if a secondary objective is to minimize the interconnections then removing dominated rows is not allowed as this sometimes eliminates a solution that has fewer interconnections. The next example illustrates the above method.

Example Consider the functions f_1 and f_2 shown in Fig. 4.23 once again. They are reproduced in Fig. 4.24 along with the product function $f_1 f_2$. Since none of the prime implicants of f_1 and f_2 is also a prime implicant of $f_1 f_2$, all five multi-output prime implicants shown in these maps deserve further consideration. The augmented prime implicant chart is shown in Fig. 4.24d. The essential prime implicants and the minterms they cover are then checked. This leads to the reduced chart shown in Fig. 4.24e. Assuming that we are interested in minimizing the number of gates as a primary objective and the number of interconnections as a secondary objective, we cannot use the concept of dominated rows to reduce this chart further. Thus, we can use the prime implicant function p to resolve the situation as follows:

$$p = (B + E)(C + E) = BC + E.$$



Function	Prime implicant	f_1 2 6 7	f_2 1 2 3
f_1	$\checkmark A = xy$ $B = yz'$	$\times \times$ $\times \times$	
f_2	$C = x'y$ $\checkmark D = x'z$		$\times \times$ $\times \times$
$f_1 f_2$	$E = x'yz'$	$\times$	$\times$

(d) Augmented prime implicant chart.

Function	Prime implicant	f_1 2	f_2 2
f_1	$B = yz'$	$\times$	
f_2	$C = x'y$		$\times$
$f_1 f_2$	$E = x'yz'$	$\times$	$\times$

(e) Reduced chart.

Fig. 4.24 Multi-output prime implicants and augmented prime implicant chart.

Thus, the minimum-gate implementation contains AND gates realizing multi-output prime implicants A , D , and E in the first level. The complete implementation is shown in Fig. 4.25.

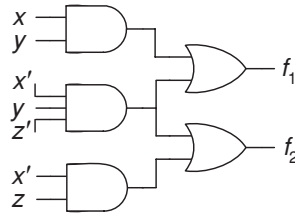


Fig. 4.25 Multi-output minimized two-level circuit.

Fig. 4.26 Using encoded truth tables for minimization.

x	y	z	f_1	f_2		x	y	z	f_1	f_2		x	y	z	f_1	f_2
–	1	0	1	0		–	1	0	1	0		1	1	–	1	0
1	1	–	1	0	reduce	1	1	–	1	0	irredundant	0	–	1	0	1
0	–	1	0	1	$\Rightarrow$	0	–	1	0	1	$\Rightarrow$	0	1	0	1	1
0	1	–	0	1		0	1	–	0	1						
						0	1	0	1	1						

One can perform multi-output two-level minimization using the encoded truth table as well. The equivalent sequence of steps required for the above example is shown in Fig. 4.26. In the initial covering of minterms, there is no way to expand the input part of any row or reduce its output part (by turning a 1 into a 0) and still realize the same set of functions. However, if –10 or 01– is reduced to 010 then its output part can be expanded to 11. Since both –10 and 01– now become redundant they can be eliminated, obtaining the final multi-output minimized implementation.

Notes and references

The problem of minimizing switching expressions has been studied extensively in the literature. The map method was introduced by Veitch [10] in 1952 and modified to its present form by Karnaugh [4]. The tabulation algorithm was developed by Quine [8, 9] and modified by McCluskey [5]. *ESPRESSO* was described in [2]. It built upon prior tools, such as *MINI* [3]. Tabular simplification of multi-output circuits was discussed by Bartee [1] and McCluskey and Schorr [6]. Multi-output two-level minimization is described in greater detail in [7].

- [1] Bartee, T. C.: "Computer design of multiple output logical networks," *IRE Trans. Electron. Computers*, vol. EC-10, no. 1, pp. 21–30, March 1961.

- [2] Brayton, R. K., G. D. Hachtel, C. T. McMullen, and A. L. Sangiovanni-Vincentelli: *Logic Minimization Algorithms for VLSI Synthesis*, Kluwer Academic, Boston, 1984.
- [3] Hong, S. J., R. G. Cain, and D. L. Ostapko: "MINI: a heuristic approach for logic minimization," *IBM J. Research & Development*, vol. 18, pp. 443–458, September 1974.
- [4] Karnaugh, M.: "The map method for synthesis of combinational logic circuits," *Trans. AIEE* part I, vol. 72, no. 9, pp. 593–599, 1953.
- [5] McCluskey, E. J., Jr: "Minimization of Boolean functions," *Bell System Tech. J.*, vol. 35, no. 6, pp. 1417–1444, November 1956.
- [6] McCluskey, E. J., and H. Schorr: "Essential multiple-output prime implicants," in *Mathematical Theory of Automata, Proc. Polytech. Inst. Brooklyn Symp.*, vol. 12, pp. 437–457, 1962.
- [7] Muroga, S.: *Logic Design and Switching Theory*, John Wiley & Sons, New York, 1979.
- [8] Quine, W. V.: "The problem of simplifying truth functions," *Am. Math. Monthly*, vol. 59, no. 8, pp. 521–531, October 1952.
- [9] Quine, W. V.: "A way to simplify truth functions," *Am. Math. Monthly*, vol. 62, no. 9, pp. 627–631, November 1955.
- [10] Veitch, E. W.: "A chart method for simplifying truth functions," in *Proc. ACM*, Pittsburgh, pp. 127–133, May 1952.

Problems

Problem 4.1. With the aid of a four-variable Karnaugh map, derive minimal sum-of-products expressions for each of the following functions:

- (a) $f_1(w, x, y, z) = \sum(0, 1, 2, 3, 4, 6, 8, 9, 10, 11);$
- (b) $f_2(w, x, y, z) = \sum(0, 1, 5, 7, 8, 10, 14, 15);$
- (c) $f_3(w, x, y, z) = \sum(0, 2, 4, 5, 6, 8, 10, 12).$

Problem 4.2

- (a) Find the minimal sum-of-products and minimal product-of-sums expressions for

$$f(w, x, y, z) = \prod(1, 4, 5, 6, 11, 12, 13, 14, 15).$$

Is your answer unique?

- (b) Determine the minimal sum-of-products expression for

$$f(w, x, y, z) = \sum(0, 2, 4, 9, 12, 15) + \sum_{\phi}(1, 5, 7, 10).$$

Problem 4.3. Given the function $T(w, x, y, z) = \sum(1, 2, 3, 5, 13) + \sum_{\phi}(6, 7, 8, 9, 11, 15):$

- (a) find a minimal sum-of-products expression;
- (b) find a minimal product-of-sums expression;
- (c) compare the expressions obtained in (a) and (b); if they do not represent identical functions, explain why.

Problem 4.4. Find all minimal four-variable functions that assume the value 1 when the minterms 4, 10, 11, 13 are equal to 1, and the value 0 when the minterms 1, 3, 6, 7, 8, 9, 12, 14 are equal to 1.

Problem 4.5. Each of the following functions actually represents a set of four functions, corresponding to the possible assignments of the don't-care terms.

$$f_1(w, x, y, z) = \sum(1, 3, 4, 5, 9, 10, 11) + \sum_{\phi}(6, 8),$$

$$f_2(w, x, y, z) = \sum(0, 2, 4, 7, 8, 15) + \sum_{\phi}(9, 12).$$

- (a) Find $f_3 = f_1 \cdot f_2$. How many functions does f_3 represent?
- (b) Find $f_4 = f_1 + f_2$. How many functions does f_4 represent?
- (c) Simplify the above functions, their product, and their sum.

Problem 4.6. Let $f = \sum(5, 6, 13)$ and $f_1 = \sum(0, 1, 2, 3, 5, 6, 8, 9, 10, 11, 13)$. Find f_2 such that $f = f_1 \cdot f_2'$. Is f_2 unique? If not, indicate all possibilities.

Problem 4.7. Given the network of Fig. P4.7, determine the functions f_2 and f_3 if $f_1 = xz' + x'z$ and the overall transmission function is to be

$$f(w, x, y, z) = \sum(0, 4, 9, 10, 11, 12).$$

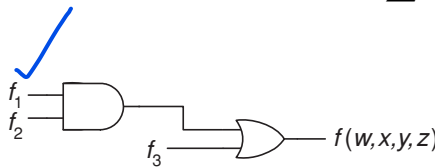


Fig. P4.7

Problem 4.8. A binary-coded-decimal (BCD) message appears in four input lines of a switching circuit. Design an AND, OR, NOT gate network that produces an output value 1 whenever the input combination is 0, 2, 3, 5, or 8.

Problem 4.9. Find the simplest function $g(A, B, C, D)$ that will make the function $f = A'BC + (AC + B)D + g(A, B, C, D)$ self-dual.

Hint: Determine first the properties of maps of self-dual functions.

Problem 4.10. Use the map method to simplify each of the following functions:

- (a) $f_1(v, w, x, y, z)$
 $= \sum(3, 6, 7, 8, 10, 12, 14, 17, 19, 20, 21, 24, 25, 27, 28);$
- (b) $f_2(v, w, x, y, z)$
 $= \sum(0, 1, 2, 4, 5, 9, 11, 13, 15, 16, 18, 22, 23, 26, 29, 30, 31).$

Problem 4.11. The five-variable map can be constructed from two disjoint four-variable maps that correspond to the fifth variable and its complement, as shown in Fig. P4.11.

- (a) Devise an algorithm that specifies the minimization procedure using such maps.
- (b) Simplify the function

$$T(v, w, x, y, z) = \sum(1, 2, 6, 7, 9, 13, 14, 15, 17, 22, 23, 25, 29, 30, 31).$$

whose maps are given in Fig. P4.11.

Fig. P4.11

$yz \backslash wx$	00	01	11	10
00				
01	1		1	1
11		1	1	
10	1	1	1	

$v = 0$

$yz \backslash wx$	00	01	11	10
00				
01	1		1	1
11		1	1	
10		1	1	

$v = 1$

Problem 4.12. Construct a six-variable map and show the representation of

$$T(u, v, w, x, y, z) = u'w'y' + uwy + w'xy'z.$$

Problem 4.13. For the function $T(w, x, y, z) = \sum(0, 1, 2, 3, 4, 6, 7, 8, 9, 11, 15)$:

- Show the map;
- Find all prime implicants and indicate which are essential;
- Find a minimal expression for T and determine whether it is unique.

Problem 4.14. Given the function $T(w, x, y, z) = \sum(1, 3, 4, 5, 7, 8, 9, 11, 14, 15)$:

- use the map to obtain the set of all prime implicants and indicate specifically the essential ones;
- find three distinct minimal expressions for T ;
- find the complement T' directly from the map;
- assume that only unprimed variables are available and construct a circuit that realizes T and requires no more than 13 gate inputs and two NOT gates.

Hint: Use the result obtained in part (c).

Problem 4.15. Show maps for four-variable functions with the following specifications. If this is impossible, explain why.

- A function with eight minterms for which
 - there are no essential prime implicants.
 - all the prime implicants are essential.
- Repeat (a) for functions with nine minterms.
- A function with an even number of prime implicants, of which exactly half are essential.
- A function with six prime implicants, of which four are essential and two are covered by essential ones.

Problem 4.16. Prove or show a counterexample to each of the following statements.

- If a function f has a unique minimal sum-of-products expression then all its prime implicants are essential.
- If a function f has a unique minimal sum-of-products expression then it also has a unique minimal product-of-sums expression.

Minimization of switching functions

- (c) If the pairwise product of all prime implicants of f is 0 then it has a unique minimal expression.
- (d) For every prime implicant p that is not essential, there is an irredundant expression that *does not* contain p .
- (e) If a function f does not have any essential prime implicant then it has at least two minimal sum-of-products forms. $\rightarrow$ *no same length*

Problem 4.17

- (a) Give the map of an irreducible four-variable function whose sum-of-products representation consists of 2^3 minterms.
- (b) Prove that there exists a function of n variables whose minimal sum-of-products form consists of 2^{n-1} minterms and that no function when expressed in sum-of-products form requires more than 2^{n-1} product terms.
- (c) Derive a bound on the number of literals needed to express *any* n -variable function.

Problem 4.18

- (a) Let $f(x_1, x_2, \dots, x_n)$ be equal to 1 if and only if exactly k of the variables equal 1. How many prime implicants does this function have?
- (b) Repeat (a) for the case where f assumes the value 1 if and only if k or more of the variables are equal to 1.

(Note: The above functions are known as *symmetric*.)

Problem 4.19

- (a) Let $T(A, B, C, D) = A'BC + B'C'D$. Prove that *any* expression for T must contain at least one instance of the literal D or of the literal D' .
- (b) If, in a minimal sum-of-products expression, each variable appears either in a primed form or in an unprimed form but not in both then the function is said to be *unate*. Prove that the minimal sum-of-products form of a unate function is unique.
- (c) Is the converse true, i.e., if the minimal sum-of-products expression is unique then the function is unate?

Hint: The function $f = w'z + x'y + x'z$ is unate. If you relabel the variables, the function may be transformed into another function whose variables are all in an unprimed form.

Problem 4.20 Use the tabulation procedure to generate the set of prime implicants and to obtain *all* minimal expressions for the following functions:

- (a) $f_1(w, x, y, z) = \sum(1, 5, 6, 12, 13, 14) + \sum_\phi(2, 4)$
- (b) $f_2(v, w, x, y, z) = \sum(0, 1, 3, 8, 9, 13, 14, 15, 16, 17, 19, 24, 25, 27, 31)$
- (c) $f_3(w, x, y, z) = \sum(0, 1, 4, 5, 6, 7, 9, 11, 15) + \sum_\phi(10, 14)$
- (d) $f_4(w, v, x, y, z) = \sum(1, 5, 6, 7, 9, 13, 14, 15, 17, 18, 19, 21, 22, 23, 25, 29, 30)$
- (e) $f_5(w, x, y, z) = \sum(0, 1, 5, 7, 8, 10, 14, 15)$

Problem 4.21 Apply the branching method to find a minimal expression for

$$f(v, w, x, y, z) = \sum(0, 4, 12, 16, 19, 24, 27, 28, 29, 31).$$

Problem 4.22

- (a) Prove that if x and y are switching variables, then:
 - (i) $x + y = x \oplus y \oplus xy$,
 - (ii) $x' = x \oplus 1$.

- (b) Using the equations in (a), any switching expression can be converted to an equivalent expression containing only the operations EXCLUSIVE OR and AND. Demonstrate the conversion procedure by transforming the expression

$$f = xyz' + xy'z + x'z.$$

- (c) Derive a procedure to transform an expression containing the EXCLUSIVE-OR operation to an equivalent switching expression containing only AND, OR, and NOT operations. Apply your procedure to the expression

$$f = x \oplus y \oplus z.$$

Problem 4.23. Consider the minimization of modulo-2 sum-of-products expressions by means of a Karnaugh map. Since for every such expression the following are valid,

$$x \oplus x \oplus \cdots \oplus x = \begin{cases} 0 & \text{for an even number of } x\text{'s,} \\ x & \text{for an odd number of } x\text{'s,} \end{cases}$$

$$xy \oplus xy' = x,$$

then, when forming cubes, every 1-cell *must* be included in an *odd* number of cubes while any 0-cell *may* be included in selected cubes as long as it is included in an *even* number of such cubes. For example, the map for the function

$$f(x, y, z) = x'y'z' \oplus x'yz \oplus xy'z \oplus xyz'$$

is shown in Fig. P4.23. From the three cubes shown, it is evident that the minimal expression is

$$f = x \oplus y \oplus z'.$$

- (a) Derive an algorithm for simplifying modulo-2 sum-of-products expressions by means of the map.²
- (b) Apply your algorithm to simplify the following expressions:

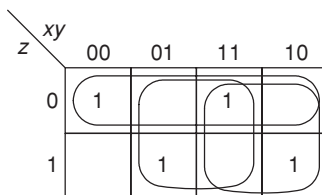
$$f_1(w, x, y, z) = w'xy'z' \oplus w'xyz' \oplus wx'y'z \oplus wx'yz \oplus wxy'z' \oplus wxyz'$$

(note that three terms containing seven literals constitute a minimum);

$$f_2(w, x, y, z) = w'x'yz \oplus w'xy'z \oplus w'xyz' \oplus wx'y'z \oplus wx'yz' \oplus wxy'z'$$

(note that five terms containing 14 literals constitute a minimum).

Fig. P4.23



² For a reference, see Even, S., I. Kohavi, and A. Paz: "On minimal modulo 2 sums of products for switching functions," *IEEE Trans. Electron. Computers*, vol. EC-16, October 1967.

Problem 4.24. Shown in Fig. P4.24 is a prime implicant chart for $f(a, b, c, d)$ in which some of the row and column headings are unknown. It is known, however, that the chart has a row for each prime implicant of f and a column for each minterm for which f has a value 1.

- Find with the aid of a map all the minterms and prime implicants that correspond, respectively, to the columns and rows with unknown headings.
- Is your solution to (a) unique?
- Give the minterms for which f must be equal to 0.
- Find a minimal expression for f .

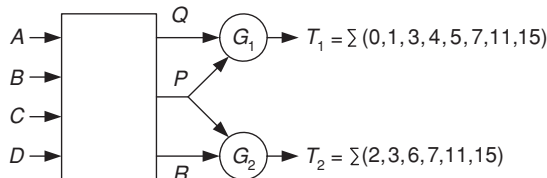
Fig. P4.24

	0	7	8	10	15	?	?
$A = b'd'$	x		x	x			
$B = ?$	x					x	
$C = bcd$		x			x		
$D = ?$					x		x
$E = ?$			x				
$F = ?$							x

Problem 4.25. A combinational network with four inputs A, B, C , and D , three intermediate outputs Q, P , and R , and final two outputs T_1 and T_2 is shown in Fig. P4.25.

- Assuming that G_1 and G_2 are both AND gates, show the map for the smallest function $P_{\min}$ (i.e., with the minimum number of minterms) that makes it possible to produce T_1 and T_2 .
- Show the maps for Q and R that correspond to the above $P_{\min}$. Indicate explicitly the don't-care positions.
- Assuming that G_1 and G_2 are both OR gates, find the largest $P_{\max}$ and show the corresponding maps for Q and R .
- Can both T_1 and T_2 be produced if G_1 is an AND gate and G_2 is an OR gate? Or if G_1 is an OR gate and G_2 is an AND gate?

Fig. P4.25



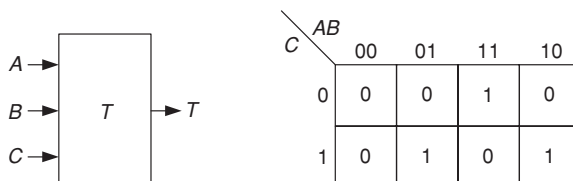
Problem 4.26. A gate T has logical properties that are defined by the map in Fig. P4.26.

- Prove that if the logic value 1 is given then any switching function can be realized by means of T gates, that is, T gates plus the logic value 1 are functionally complete.
- Realize, by means of two T gates, the function

$$f(w, x, y, z) = \sum(0, 1, 2, 4, 7, 8, 9, 10, 12, 15).$$

Hint: Realize the 0's of f .

Fig. P4.26



Problem 4.27. The initial covering of minterms for the function $f = \sum(0, 2, 3, 4, 5, 7)$ is shown on the left in Fig. P4.27. It needs to be converted into the covering shown on the right. Find a sequence of reduce, expand, and irredundant steps needed to do so. This sequence is not unique.

Fig. P4.27

x	y	z	f		x	y	z	f
0	—	0	1	$\Rightarrow$	—	0	0	1
—	1	1	1		0	1	—	1
1	0	—	1		1	—	1	1

Problem 4.28. For the three functions shown below, obtain a multi-output minimized two-level implementation using an augmented prime implicant chart. Assume that minimizing the total number of gates is the sole objective.

$$f_1 = \sum(2, 3);$$

$$f_2 = \sum(2, 3, 4, 5, 6, 7);$$

$$f_3 = \sum(1, 3, 5, 7).$$

Problem 4.29. The initial covering of minterms for two functions, f_1 and f_2 , is shown on the left in Fig. P4.29. It needs to be converted into the covering shown on the right. Find a sequence of reduce, expand, and irredundant steps that will achieve this.

Fig. P4.29

x	y	z	f_1	f_2		x	y	z	f_1	f_2
—	0	1	1	0	$\Rightarrow$	0	—	1	1	0
0	1	—	1	0		1	0	—	1	0
1	—	0	1	0		—	1	0	1	1
0	0	—	0	1		0	0	—	0	1
—	1	0	0	1		1	—	1	0	1
1	—	1	0	1						

5

Logic design

The principal application of switching theory is in the design of digital circuits. The design of such circuits is commonly referred to as *logical* (or *logic*) *design*. Most digital systems are constructed from electronic switching circuits. In this chapter, we describe some components that are typical of the basic building blocks used in constructing digital systems. Switching algebra will be used to describe the logical behavior of networks composed of these building blocks as well as to manipulate and simplify switching expressions, thereby reducing the number of components used in the design. We shall be concerned with the *logic* functions that a circuit performs rather than with its electronic structure or behavior. Special attention will be given to the design of high-speed binary adders. These examples will introduce us to some practical aspects of logic design in which the speed of operation and area limitations require ingenuity in arriving at a proper compromise.

5.1 Design with basic logic gates

Although modern digital systems are composed of a large number of components, they usually employ only a small number of different kinds of elementary circuits, called gates, whose task is to perform logic operations on input signals. In Section 3.2, we showed that in order to implement any switching function, it is necessary to have a set of two-valued switching devices capable of implementing a functionally complete set of operations. The objective of this section is to present some commonly used devices of this type.

Introductory definitions

Switching variables can be represented by either voltage or current. We shall consider only the voltage representation, since that of the current is similar. It is customary to represent the switching constants 1 and 0 by higher and lower voltages, respectively. Such an assignment of voltages to the switching constants is referred to as *positive logic polarity*. The converse, that is, the

representation of 1 and 0 by lower and higher voltages, respectively, is referred to as *negative logic polarity*. Both these representations are valid by virtue of the duality principle in switching algebra.

In practice, 0 and 1 do not correspond to specific, carefully controlled, voltages but to two voltage *ranges*; that is, they may be *nominally* “high” and “low,” but within large tolerances. Consequently, only the range of the signal is important, while its precise value may be subject to changes due to variations in temperature or in the electronic parameters. This flexibility is important because it enables logic devices to employ simple circuits that operate correctly in spite of wide variations in the circuit parameters and the presence of noise on the signal wires.

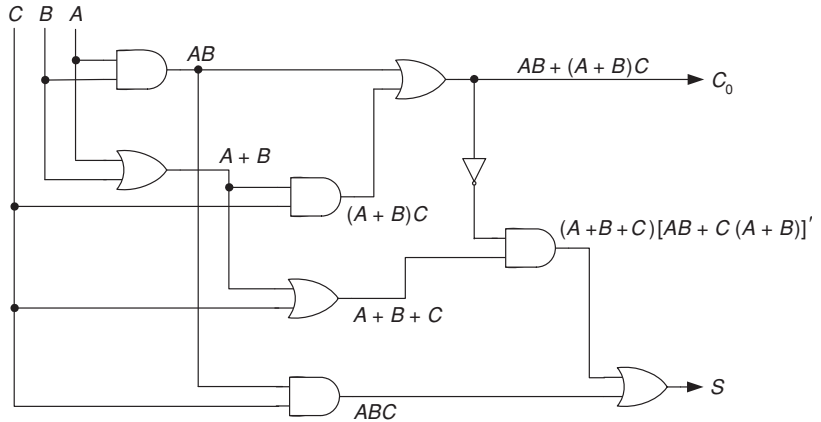
Circuits may be either *synchronous* or *asynchronous*. In the former, synchronization is usually achieved by a timing device called a clock, which produces a train of equally spaced pulses. The clock pulses are fed into the circuit in such a way that the various operations take place only with the arrival of the appropriate synchronization pulses. The clock for a particular circuit may have a number of outputs, on which pulses appear at certain intervals and with a fixed relation between the pulses on the various outputs. This process ensures an orderly execution of the various operations and logical decisions to be made by the circuit. Asynchronous circuits, however, are usually faster because they are almost free-running and do not depend on the frequency of a clock, which in most cases would be well below the speed of operation of a free-running gate. The orderly execution of operations in asynchronous circuits is controlled by a number of *completion* and *initiation* signals, such that the completion signal of one operation initiates the execution of the next consecutive operation, and so on.

In practice there is a maximum amount of current that can be drawn from a gate without affecting its operation. Also, a minimum amount of current is necessary to drive each gate. Consequently, the number of gate inputs that can be driven by the output of a single gate is limited; the maximum such number is called the *fanout* of the gate. The overloading of a gate will cause a serious deterioration in the signal value and may affect circuit performance. A less critical, though still serious, restriction is the bound on the number of inputs that a single gate may have. This bound is referred to as the *fanin* of the gate.

The basic logic gates, which implement the logic operations AND, OR, and NOT, were introduced in Section 3.4. The NOT gate is also called an inverter. In practice, a finite amount of time is required to propagate a signal through a gate, or to switch a gate output from one value to another. This delay, which is known as the *propagation delay*, strongly affects logic design. It may cause hazards or races, which are discussed in Chapters 8 and 11. In this introductory chapter, however, we shall assume that the propagation delay is very small and therefore it will generally be ignored.

In all conventional gates, the output of a gate is either connected to the input of another gate or serves as an external circuit output. It is never connected to the output of another gate since that could lead to nondeterministic operation

Fig. 5.1 Analysis of a full-adder circuit.



or to the destruction of the gate. There are gates, known as *wired-OR* and *wired-AND*, in which special circuitry is provided such that their outputs can be directly connected. However, we shall not consider these gates separately because in most cases they can be handled by using the same procedures that are applicable to conventional gates.

Analysis of combinational circuits

To every combinational switching circuit there corresponds a Boolean function that describes the logic behavior of the circuit. The analysis of a circuit is concerned with determining the function that describes that circuit.

A combinational circuit is analyzed by tracing the output of each gate, starting from the circuit inputs and continuing toward each circuit output. This procedure is illustrated by the analysis of the circuit shown in Fig. 5.1, which is a minimal realization of a *full binary adder*. (A more comprehensive discussion of the properties of this circuit is deferred to Section 5.4). The output, designated C_0 , is given by

$$\begin{aligned} C_0 &= AB + (A + B)C \\ &= AB + AC + BC. \end{aligned}$$

The second output, designated S , is found to be

$$\begin{aligned} S &= (A + B + C)[AB + (A + B)C]' + ABC \\ &= (A + B + C)(A' + B')(A' + C')(B' + C') + ABC \\ &= AB'C' + A'BC' + A'B'C + ABC \\ &= A \oplus B \oplus C. \end{aligned}$$

The circuit shown in Fig. 5.1 is referred to as a *multi-level realization*, because incoming input signals must pass through several levels of gates before they reach the outputs. In this circuit, the signals corresponding to A must pass as many as six levels of gates before reaching output S . Multi-level circuits have several practical limitations. Since a finite delay is associated with each gate,

the propagation time of input signals increases proportionately to the increase in the number of gate levels. The lengths of the various paths in a multi-level circuit are not necessarily the same. Some paths are shorter than others (i.e., they involve fewer gates); e.g., in Fig. 5.1 there is one path going from A to S of length three, while other paths from A to S range in length from four to six levels. Consequently, different propagation times are associated with various paths, which may cause certain hazardous situations. Such situations are discussed in Chapter 8 and 11. A *two-level realization* overcomes these limitations, at the price of considerable increase in the number of gates required for the realization. Two-level realizations of some circuits are shown later. In Chapter 8, we shall also show that the testing of a multi-level circuit for faults is considerably more complicated than the testing of two-level circuits.

Some simple design problems

In the preceding chapters, we have introduced some of the most important tools used in designing switching circuits. These tools include switching algebra, truth tables, and minimization procedures. In this section we shall employ these tools to design and implement some simple circuits.

Example Suppose that we are required to design a *parallel parity-bit generator*. This circuit must produce an output value 1 if and only if an odd number of its inputs have the value 1. As an illustration, we shall design a parity-bit generator for three-bit code words; that is, the circuit has three inputs x , y , and z , and its output p must be 1 whenever either only one of the input values is 1 or all three input values are 1. The map for this function is shown in Fig. 5.2a. Clearly,

$$p = x'y'z + x'yz' + xy'z' + xyz.$$

A simple implementation of p is shown in Fig. 5.2b.

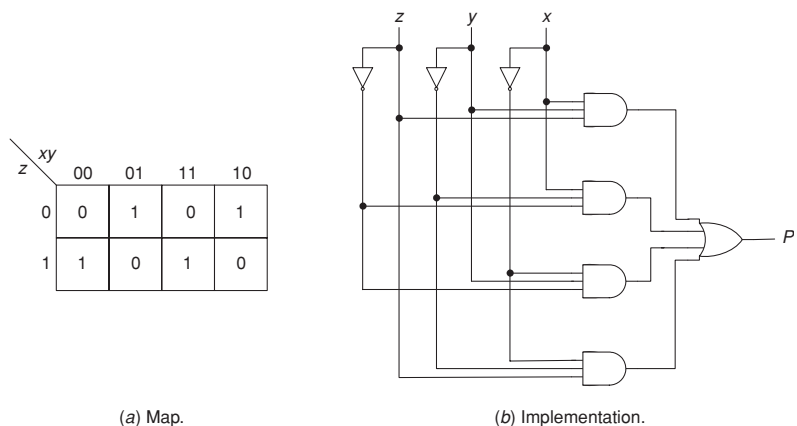


Fig. 5.2 Design of a parallel parity-bit generator.

Example An input line x to a *serial-to-parallel converter* receives a long sequence of binary digits that must be distributed into four different output lines, as specified by external control signals. Let C_1 and C_2 be the two control signals and let L_1, L_2, L_3 , and L_4 denote the output lines. The truth table shown in Table 5.1 specifies the logic values of the output lines for every combination of control signals. For example, if the control signals have values $C_1 = C_2 = 0$ then the input signals must be directed to L_1 , and so on for other control signal values. The resulting logic equations are given in Table 5.1 and a two-level implementation is shown in Fig. 5.3.

Table 5.1 Truth table and logic equations for the serial-to-parallel converter

Control		Output lines				Logic equations
C_1	C_2	L_1	L_2	L_3	L_4	
0	0	x	0	0	0	$L_1 = xC_1'C_2'$
0	1	0	x	0	0	$L_2 = xC_1'C_2$
1	0	0	0	x	0	$L_3 = xC_1C_2'$
1	1	0	0	0	x	$L_4 = xC_1C_2$

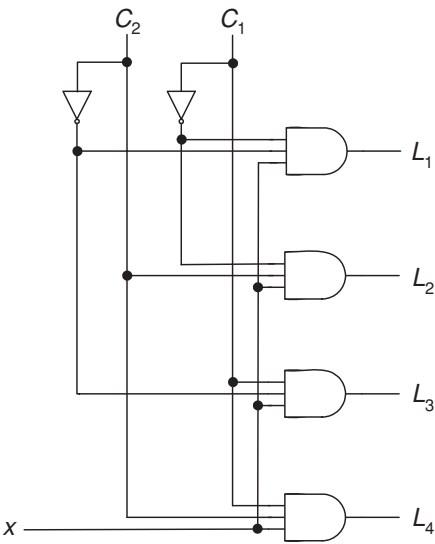


Fig. 5.3 A serial-to-parallel converter.

5.2 Logic design with integrated circuits

Thus far we have developed the traditional techniques of logic design, in which discrete gates are used as basic building blocks for implementing digital

systems. Since the 1950s, more modern devices, called *integrated circuits*, have been developed and now serve as the main building blocks of all logic circuits. Integrated circuits are produced in *packages*, or *chips*, and are historically classified into four categories, as follows.

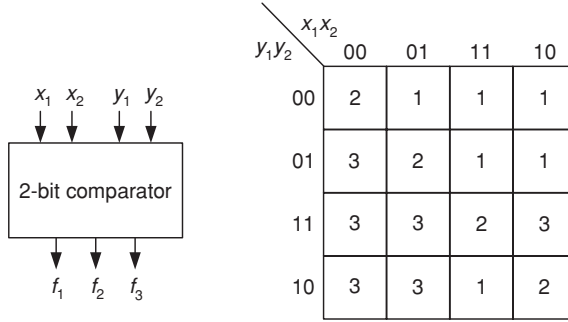
1. *Small-scale integration (SSI)* usually refers to packages containing single gates, e.g., AND, OR, NOT, NAND, NOR, XOR, or small packages containing two or four gates of the same type.
2. *Medium-scale integration (MSI)* refers to intermediate packages containing up to about 100 gates. They usually realize standard circuits that are used often in logic design, e.g., code converters, adders, etc.
3. *Large-scale integration (LSI)*, may contain many hundreds or thousands of gates in a single package. Some LSI circuits are standard, e.g., subsystems for computer control or for a computer arithmetic unit, while other LSI circuits are manufactured to the specification of the logic designer.
4. *Very-large-scale integration (VLSI)* is what we currently observe, in chips in which there may be millions of gates.

Integrated circuits have several important advantages over the older discrete components. First, they are relatively inexpensive; in fact, the integrated circuit cost becomes an increasingly small part of the total cost of a system. Second, they are more reliable and easily available. Presently, a logic designer will make every effort to incorporate as many standard VLSI packages as possible in building a system, since their use will result in a lower cost, at the same time increasing the system's reliability and making it easier to maintain by simple replacement of a defective package by a new one. In this section, we present several standard circuits that used to be available as MSI packages but now constitute parts of VLSI packages. Their design will not only illustrate the design techniques for other, nonstandard, circuits but also enhance our ability to use these circuits, modify them, or enlarge them by connecting several such circuits.

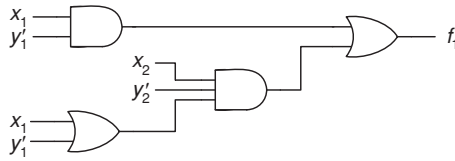
Comparators

An *n-bit comparator* is a circuit that compares the magnitude of two numbers X and Y . It has three outputs f_1 , f_2 , and f_3 , such that: $f_1 = 1$ iff (if and only if) $X > Y$; $f_2 = 1$ iff $X = Y$; $f_3 = 1$ iff $X < Y$. As an example, consider an elementary 2-bit comparator, as in Fig. 5.4a.

The circuit has four inputs x_1 , x_2 , y_1 and y_2 , where x_1 and y_1 denote the most significant digit of X and Y , respectively. The logic equations may be determined with the aid of the map in Fig. 5.4b, where the values 1, 2, and 3 are entered in appropriate cells to denote, respectively, $f_1 = 1$, $f_2 = 1$, and

Fig. 5.4 Designing a 2-bit comparator.

(a) Block diagram.

(b) Map for f_1 , f_2 , and f_3 .(c) Circuit for f_1 .

$f_3 = 1$. Thus

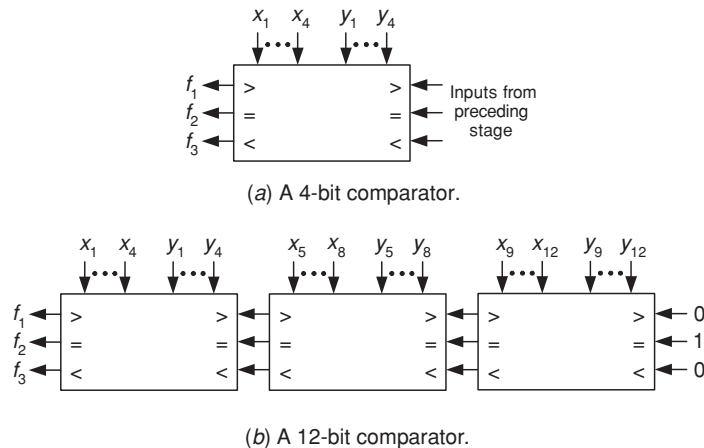
$$\begin{aligned}
 f_1 &= x_1x_2y_2' + x_2y_1'y_2' + x_1y_1' \\
 &= (x_1 + y_1')x_2y_2' + x_1y_1', \\
 f_2 &= x_1'x_2'y_1'y_2' + x_1'x_2y_1'y_2 + x_1x_2'y_1y_2' + x_1x_2y_1y_2 \\
 &= x_1'y_1'(x_2'y_2' + x_2y_2) + x_1y_1(x_2'y_2' + x_2y_2) \\
 &= (x_1'y_1' + x_1y_1)(x_2'y_2' + x_2y_2), \\
 f_3 &= x_2'y_1y_2 + x_1'x_2'y_2 + x_1'y_1 \\
 &= x_2'y_2(y_1 + x_1') + x_1'y_1.
 \end{aligned}$$

The circuit for f_1 is shown in Fig. 5.4c. Similar circuits are obtained for f_2 and f_3 .

The reader can verify that $X > Y$, i.e., $f_1 = 1$, when the most significant bit of X is larger than that of Y , i.e., $x_1 > y_1$, or when the most significant bits are equal but the least significant bit of X is larger than that of Y , namely, $x_1 = y_1$ and $x_2 > y_2$. In a similar way, we can determine the conditions for $f_2 = 1$ and $f_3 = 1$.

This line of reasoning can be further generalized to yield the logic equations for a 4-bit comparator.

Fig. 5.5 Design of a 12-bit comparator using three 4-bit comparators.



A 4-bit comparator is shown in Fig. 5.5a. It has 11 inputs, four representing X , four representing Y , and three connected to the outputs f_1 , f_2 , and f_3 of the preceding 4-bit stage. Three such stages can be connected in cascade, as shown in Fig. 5.5b, to obtain a 12-bit comparator. Initial conditions are inserted at the inputs of the comparator corresponding to the least significant bits in such a way that the outputs of this comparator will depend only on the values of its own x 's and y 's.

Data selectors

A *multiplexer* is essentially an electronic switch that can connect one out of n inputs to the output. The most important application of the multiplexer is as a *data selector*. In general, a data selector has n *data input* lines $D_0, D_1, \dots, D_{n-1}$, m *select digit* inputs $s_0, s_1, \dots, s_{m-1}$, and one output. The m select digits form a binary *select number* ranging from 0 to $2^m - 1$, and when this number has the value k then D_k is connected to the output. Thus this circuit selects one of n data input lines, according to the value of the select number, and in effect connects it to the output. Clearly, the number of select digits must equal $m = \log_2 n$, so that it can identify all the data inputs.

Data selectors have numerous applications. For example, they may be used to connect one out of n input sources of a device to its output. As we shall subsequently show, data selectors may also be used to implement all Boolean functions.

A block diagram for a data selector with eight data input lines is shown in Fig. 5.6a. The select number consists of the three digits $s_2s_1s_0$. Thus, for example, when $s_2s_1s_0 = 101$ then D_5 is to be connected to the output, and so on. The *Enable* (or *Strobe*) input “enables” or turns the circuit on. A logic

Fig. 5.6 Data selector with eight data-input lines.

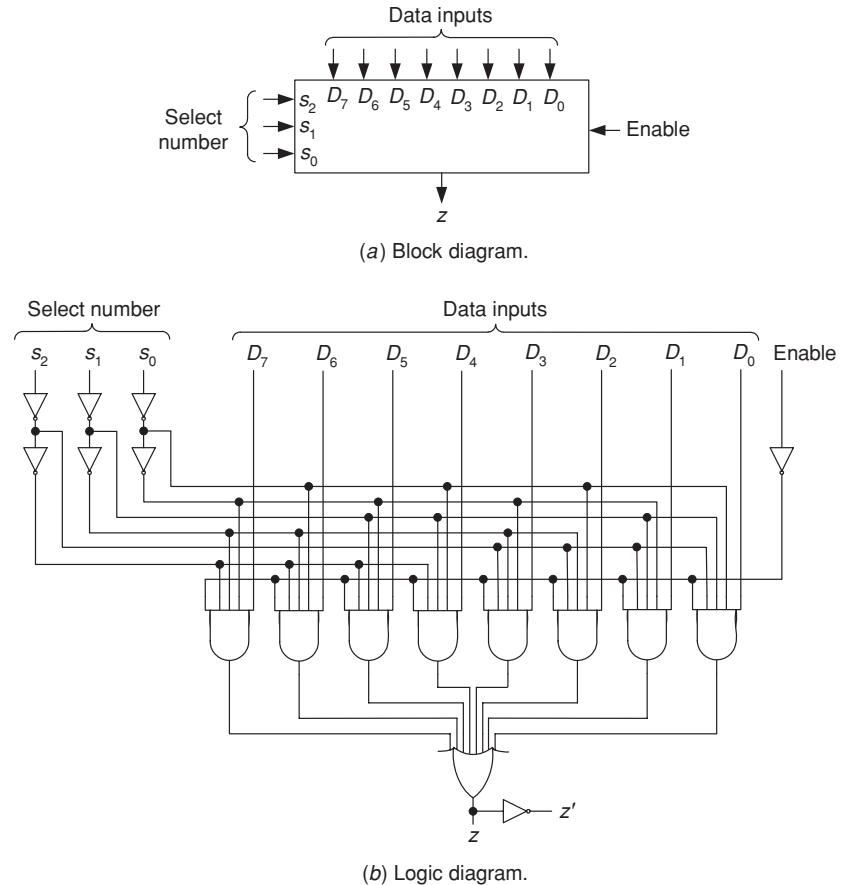
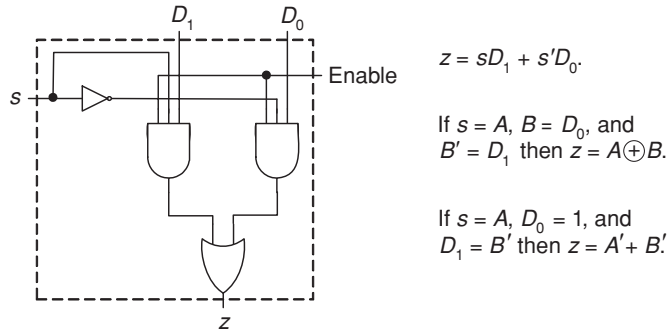


diagram for this data selector is shown in Fig. 5.6*b*. Such a unit provides the complement z' of the output as well as the output z itself. The Enable input turns the circuit on when it assumes the value 0.

Implementing switching functions with data selectors

An important application of data selectors is the implementation of arbitrary switching functions. As an example, we shall show how functions of two variables can be implemented by means of the data selector of Fig. 5.7. Clearly, in this circuit, if $s = 0$ then z assumes the value of D_0 and if $s = 1$ then z assumes the value of D_1 . Thus, $z = sD_1 + s'D_0$. Now, suppose that we want to implement the EXCLUSIVE-OR operation $A \oplus B$. This can be accomplished by connecting variable A to the input s and variables B and B' to D_0 and D_1 , respectively. In this case $z = AB' + A'B = A \oplus B$. Similarly, if we want to implement the NAND operation $z = A' + B'$ then we connect variable A to s and variable B' to D_1 ; D_0 is connected to a constant 1. Clearly, $z = AB' + A'1 = A' + B'$.

Fig. 5.7 Implementing two-variable functions with a data selector.



In a similar manner, a judicious choice of inputs will implement any of the 16 different two-variable functions (see Table 3.6). In general, to implement an n -variable function we require a data selector with $n - 1$ select inputs and 2^{n-1} data inputs. Hence, for example, to implement all three-variable functions we require a data selector with two select inputs, s_1 and s_2 , and $2^{3-1} = 4$ data inputs, D_0 , D_1 , D_2 , and D_3 . The output of such a data selector is

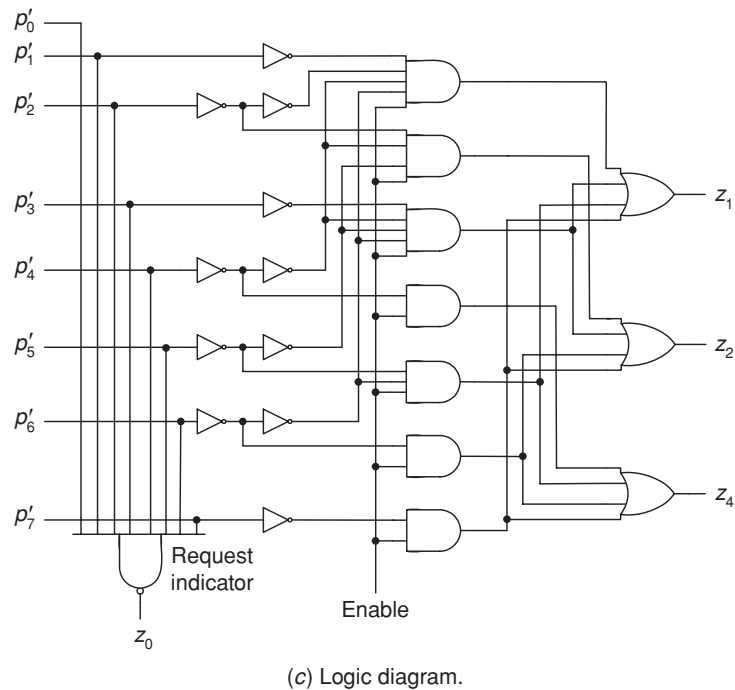
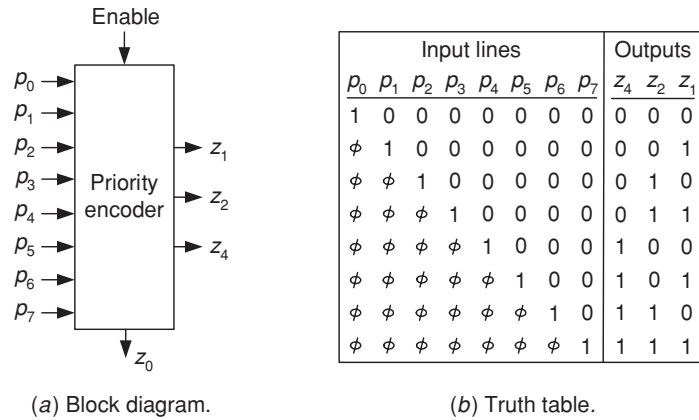
$$z = s'_1 s'_2 D_0 + s_1 s'_2 D_1 + s'_1 s_2 D_2 + s_1 s_2 D_3.$$

The reader can verify that, if we connect variables A and B to s_1 and s_2 , respectively, and variables C and C' to D_0 and D_3 , respectively, and assign constants 1 to D_1 and 0 to D_2 then the circuit will realize the function $z = A'B'C + AB' + ABC' = AC' + B'C$.

In general, then, to implement an n -variable function we assign $n - 1$ variables to the select inputs, one to each such input. The last variable and the constants 0 and 1 are assigned to the data inputs in such a way that together with the select input variables they will yield the required function. Such an implementation is usually possible when at least one variable is available in both its complemented as well as its uncomplemented form; otherwise, a larger data selector may be required. Implementations of functions of five or more variables are usually accomplished by means of a multi-level arrangement of several smaller standard data selectors.

Priority encoders

A *priority encoder* is a device with n input lines and $\log_2 n$ output lines. The input lines represent units which may request service. When two lines p_i and p_j , such that $i > j$, request service simultaneously, line p_i has priority over line p_j . The encoder produces a binary output code indicating which of the input lines requesting service has the highest priority. An input line p_i indicates a request for service by assuming the value 1. A block diagram for an eight-input three-output priority encoder is shown in Fig. 5.8a.

Fig. 5.8 Design of a priority encoder.

The truth table for this encoder is shown in Fig. 5.8b. In the first row, only p_0 requests service and, consequently, the output code should be the binary number zero to indicate that p_0 has priority. This is accomplished by setting $z_4z_2z_1 = 000$. The fourth row, for example, describes the situation where p_3 requests service while p_0 , p_1 , and p_2 each may or may not request service simultaneously. This is indicated by an entry 1 in column p_3 and don't-cares

in columns p_0 , p_1 , and p_2 . No request of a higher priority than p_3 is present at this time. Since in this situation p_3 has the highest priority, the output code must be the binary number three. Therefore, we set z_1 and z_2 to 1 while z_4 is set to 0. (Note that the binary number is given by $N = 4z_4 + 2z_2 + z_1$.) In a similar manner the entire table is completed.

From the truth table, we can derive the logic equations for z_1 , z_2 , and z_4 . Starting with z_4 , we find that

$$z_4 = p_4 p'_5 p'_6 p'_7 + p_5 p'_6 p'_7 + p_6 p'_7 + p_7.$$

This equation can be simplified to

$$z_4 = p_4 + p_5 + p_6 + p_7.$$

For z_2 and z_1 , we find

$$\begin{aligned} z_2 &= p_2 p'_3 p'_4 p'_5 p'_6 p'_7 + p_3 p'_4 p'_5 p'_6 p'_7 + p_6 p'_7 + p_7 \\ &= p_2 p'_4 p'_5 + p_3 p'_4 p'_5 + p_6 + p_7, \\ z_1 &= p_1 p'_2 p'_3 p'_4 p'_5 p'_6 p'_7 + p_3 p'_4 p'_5 p'_6 p'_7 + p_5 p'_6 p'_7 + p_7 \\ &= p_1 p'_2 p'_4 p'_6 + p_3 p'_4 p'_6 + p_5 p'_6 + p_7. \end{aligned}$$

An implementation of such an encoder is given in Fig. 5.8c. In this encoder, the inputs are given in complemented form. The circuit also has an Enable signal and contains an output z_0 that indicates whether any requests are present. Specifically, $z_0 = 0$ if there is no request and $z_0 = 1$ if there are one or more requests present. It is possible to combine several such encoders, by means of external gating, to handle more than eight inputs.

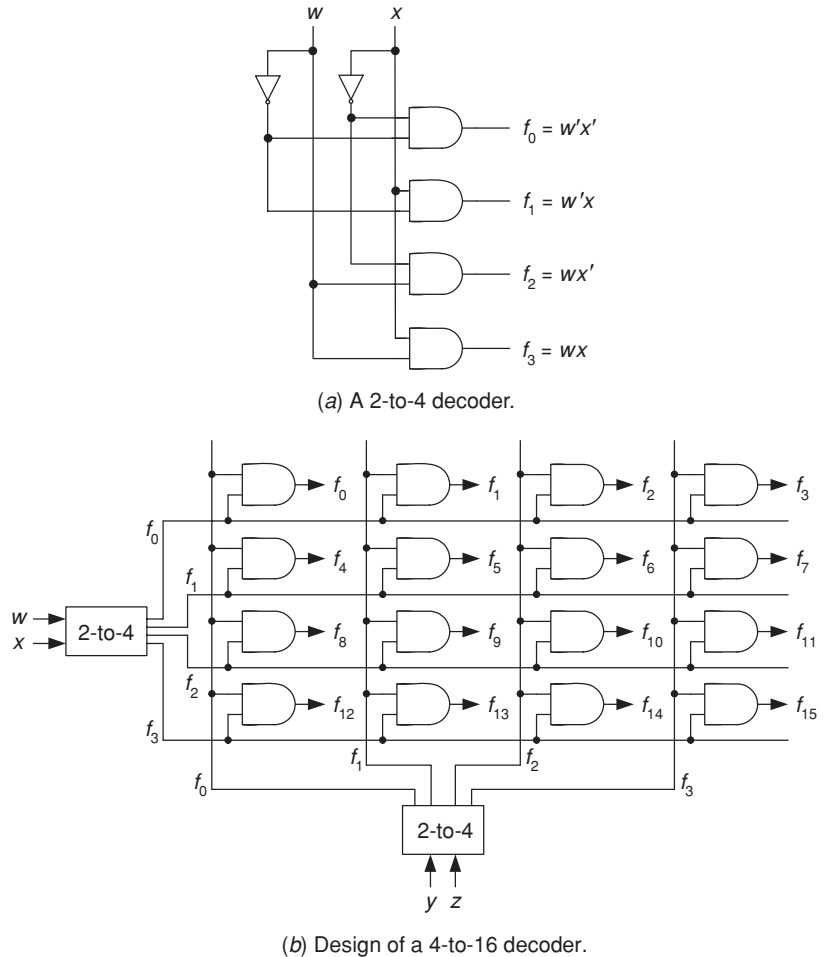
Decoders

A *decoder* is a combinational circuit with n inputs and at most 2^n outputs. Its characteristic property is that *for every combination of input values, only one output value will be equal to 1 at any given time*. Decoders have a wide variety of applications in digital technology. They may be used to route input data to a specified output line, as, for example, is done in memory addressing, where input data are to be stored in (or read from) a specified memory location. They can be used for some code conversions. Or they may be used for data distribution, i.e., demultiplexing, as will be shown later. Finally, decoders are also used as basic building blocks for implementing arbitrary switching functions.

Figure 5.9a illustrates a basic 2-to-4 decoder. Clearly, if w and x are the input variables then each output corresponds to a different minterm of two variables. Two such 2-to-4 decoders plus a gate-switching matrix can be connected, as shown in Fig. 5.9b, to form a 4-to-16 decoder. Switching matrices are very widely used in the design of digital circuits.

Not all decoders have exactly 2^n outputs. Figure 5.10 describes a *decimal decoder* that converts information from BCD to decimal. It has four inputs w ,

Fig. 5.9 Illustration of n -to- 2^n decoders.

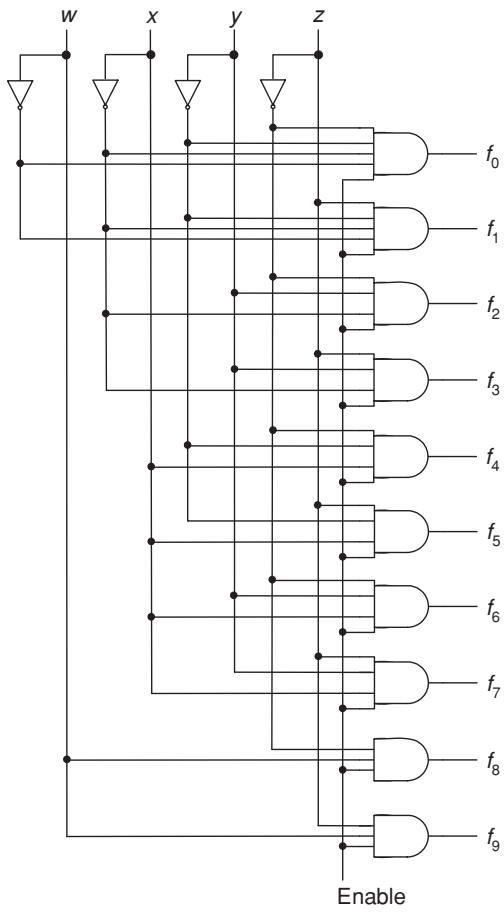
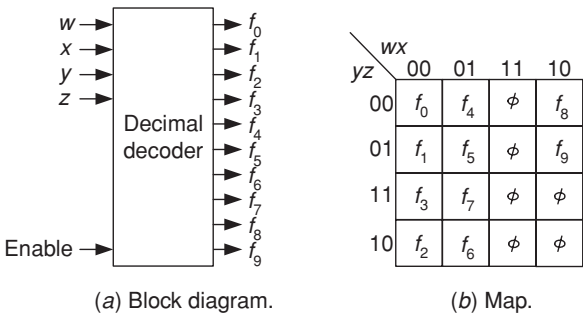


x , y , and z , where w is the most significant and z the least significant digit, and 10 outputs, f_0 through f_9 , corresponding to the decimal numbers. In designing this decoder, we have taken advantage of the don't-care combinations, f_{10} through f_{16} , as can be verified by means of the map in Fig. 5.10b. Another implementation of decimal decoders is by means of a partial-gate matrix, as shown in Fig. 5.11.

A decoder with exactly n inputs and 2^n outputs can also be used to implement any switching function. Each output of such a decoder realizes one distinct minterm. Thus, by connecting the appropriate outputs to an OR gate, the required function can be realized. Figure 5.12 illustrates the implementation of the function $f(A, B, C, D) = \sum(1, 5, 9, 15)$ by means of a complete decoder, i.e., one with n inputs and 2^n outputs.

A decoder with one data input and n address inputs is called a *demultiplexer*. It directs the input data to any one of the 2^n outputs, as specified by the n -bit

Fig. 5.10 Design of a BCD-to-decimal decoder.



(c) Logic diagram.

Fig. 5.11 BCD-to-decimal decoder.

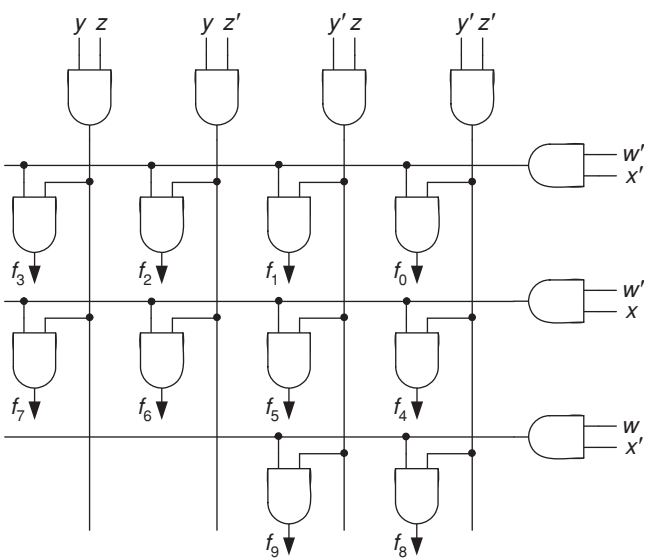


Fig. 5.12 Implementing a switching function with a decoder.

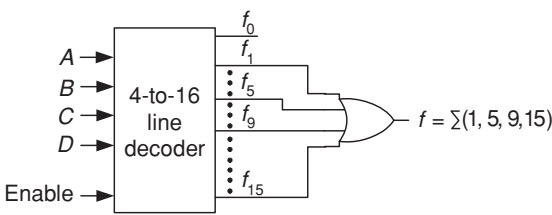
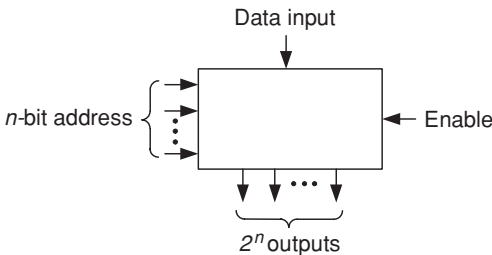


Fig. 5.13 A demultiplexer.



input address. A block diagram for a demultiplexer is shown in Fig. 5.13. A demultiplexer with four outputs is shown in Fig. 5.3.

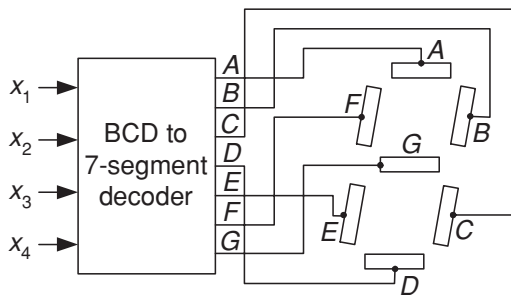
When larger-size decoders are needed, they can usually be formed by interconnecting several smaller decoders with some additional logic.

Seven-segment display

A popular method for displaying decimal digits is by means of the *seven-segment display* shown in Fig. 5.14. The display consists of a BCD-to-seven-segment decoder and seven separate light segments (usually light-emitting

Table 5.2 Seven-segment pattern and code

Decimal digit	BCD code				Seven-segment code						
	x_1	x_2	x_3	x_4	A	B	C	D	E	F	G
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	0	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	0	0	1	1
0	0	0	0	0	1	1	1	1	1	1	0

Fig. 5.14 Seven-segment display.

diodes or crystals) each of which can be turned on and off independently of the others. The display receives its inputs in the form of BCD coded digits and transforms these inputs to obtain the pattern of the corresponding decimal digit.

Table 5.2 can be viewed as the truth table for the output functions of the BCD-to-seven-segment decoder. The seven-segment code corresponding to each digit is directly obtained from the pattern. For example, to display the decimal digit 2, segments A , B , G , E , D are turned on while segments C and F remain off. In a similar manner, the rest of the seven-segment code is obtained. The segment excitation functions can now be determined directly from the table or by using maps. Note that there are six don't-care combinations identical to those in Fig. 5.10*b*. The expressions for the segment excitation functions are thus as follows:

$$A = x_1 + x_2'x_4' + x_2x_4 + x_3x_4,$$

$$B = x_2' + x_3'x_4' + x_3x_4,$$

$$C = x_2 + x_3' + x_4,$$

$$D = x_2'x_4' + x_2'x_3 + x_3x_4' + x_2x_3'x_4,$$

$$E = x_2'x_4' + x_3x_4',$$

$$F = x_1 + x_2x_3' + x_2x_4' + x_3'x_4',$$

$$G = x_1 + x_2'x_3 + x_2x_3' + x_3x_4'.$$

The realization of the decoder is now straightforward. It can be implemented either as a conventional multi-output circuit or using a single 4-to-16 line decoder plus seven OR gates, in a manner similar to that shown in Fig. 5.12.

Sine generators

Trigonometric functions can either be generated sequentially or produced by combinational circuits. Combinational sine generators are used whenever the sine function must be evaluated fast and repeatedly.

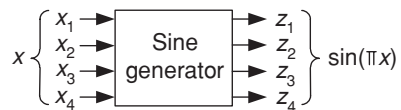
A combinational sine generator receives as its input the angle and as output produces the sine of that angle. The angle is given in radians converted to binary and the sine value is produced in binary. Naturally, the accuracy of the calculation is a function of the number of bits that describe the angles and sine values. In practical applications, at least eight binary digits are required to describe the angles or sine values. In our case, however, in order to simplify the computations we shall consider a *four-bit sine generator*.

Let the sine function be $\sin(\pi x)$, where $0 \leq x < 1$. The angle x will be described by four binary digits x_1, x_2, x_3, x_4 , where x_1 has weight $\frac{1}{2}$, x_2 weight $\frac{1}{4}$, and so on. Thus, for example, to specify an angle of 45° , the input x must equal $\frac{1}{4}$, i.e., $x = 0100$. To specify an angle of 30° , x must equal $\frac{1}{6}$. However, it is impossible to represent this value precisely with four bits; the closest possible value is $\frac{3}{16}$ or $x = 0011$. The truth table of the sine generator is shown in Fig. 5.15a and its block diagram in Fig. 5.15b. The sine is given by the binary

Fig. 5.15 Designing a sine generator.

Angle x				$\sin(\pi x)$			
x_1	x_2	x_3	x_4	z_1	z_2	z_3	z_4
0	0	0	0	0	0	0	0
0	0	0	1	0	0	1	1
0	0	1	0	0	1	1	0
0	0	1	1	1	0	0	0
0	1	0	0	1	0	1	1
0	1	0	1	1	1	0	1
0	1	1	0	1	1	1	0
0	1	1	1	1	1	1	1
1	0	0	0	1	1	1	1
1	0	0	1	1	1	1	1
1	0	1	0	1	1	1	0
1	0	1	1	1	1	0	1
1	1	0	0	1	0	1	1
1	1	0	1	1	0	0	0
1	1	1	0	0	1	1	0
1	1	1	1	0	0	1	1

(a) Truth table.



(b) Block diagram.

number $z = z_1z_2z_3z_4$ such that $0 \leq z < 1$, z_1 has weight $\frac{1}{2}$, z_2 has weight $\frac{1}{4}$, and so on. The sine of 30° is equal to 0.5. Hence, the output values in row $x = 0011$ are specified to be $z = 1000$. Similarly, the sine of 45° is 0.707. Clearly, the closest output value would be $z = 1011$, which is equal to 0.6875. In a similar manner, the entire truth table is constructed.

The logic equations specifying the outputs can be derived from a set of four maps that correspond to the truth tables and are as follows:

$$\begin{aligned} z_1 &= x'_1x_2 + x_1x'_2 + x_2x'_3 + x'_1x_3x_4, \\ z_2 &= x_1x'_2 + x_3x'_4 + x'_1x_2x_4, \\ z_3 &= x_3x'_4 + x_2x_3 + x_2x'_4 + x'_2x'_3x_4 + x_1x'_4, \\ z_4 &= x'_2x'_3x_4 + x_2x'_3x'_4 + x_1x'_2x'_3 + x_1x_3x_4 + x'_1x_2x_4. \end{aligned}$$

The sine generator, which is a special-purpose code converter, can be implemented in a variety of ways, namely, as a conventional multi-output circuit or by using a 4-to-16 line decoder plus the necessary OR gates.

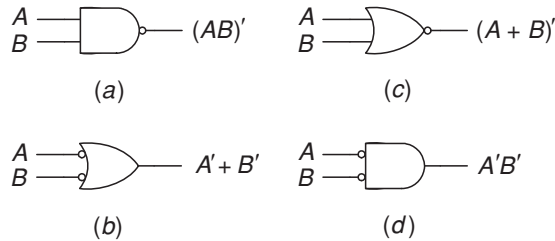
5.3 NAND and NOR circuits

In Section 3.2 we proved that the NAND and NOR operations are each functionally complete. It is highly desirable to construct digital circuits of NAND or NOR gates because of the simplicity and uniformity of such circuits, which have just a single primitive component. NAND gates constitute the major components used today by logic designers. In some future nanotechnologies, NOR gates may play a similar role.

Logic symbols

The analysis and design of NAND and NOR circuits pose difficulties not encountered in AND, OR, NOT logic. Switching algebra, which is a powerful tool for the design of circuits constructed of AND, OR, and NOT gates, is not as directly applicable in the cases of NAND and NOR logic. The main difficulty lies in the fact that, in order to obtain simple NAND (or NOR) circuits, the corresponding algebraic expressions must be factored in such a way that the NAND (or NOR) operation will be the only one in the expression. This step is usually quite complicated because it involves a large number of applications of De Morgan's theorem. For example, the implementation of the function $T = A' + (B + C')(D' + EF')$ with AND, OR, NOT logic is straightforward, but its NAND-logic realization is not as evident. It can be, however, considerably simplified by expressing the function as $T = A|((B'|C)|(D|(E|F')))$. Evidently, the determination of this expression by algebraic means would be quite involved, but it may be avoided through the use of special symbols and simple circuit manipulations.

Fig. 5.16 (a), (b) NAND and (c), (d) NOR gate symbols.



Thus the interpretation and manipulation of logic diagrams, as well as the implementation of switching functions, becomes more evident if we use a system of symbols such that each logic gate can be represented by one of two symbols. This system, known as the MIL-STD-806B, is shown in Fig. 5.16. Each symbol is formed by combining the AND-gate or OR-gate symbol with the inversion symbol, which is a small circle.

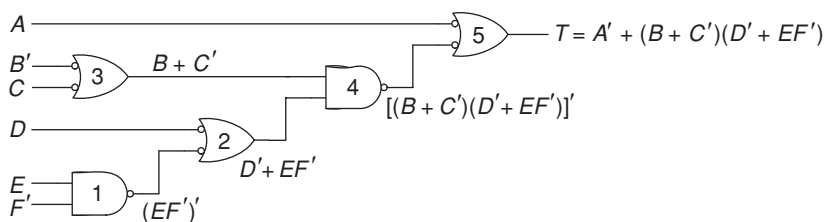
The symbol in Fig. 5.16a represents a circuit that generates the complement of the AND combination of its inputs, i.e., $(AB)'$. The symbol of Fig. 5.16b, however, represents a circuit that generates the OR combination of its inverted inputs, i.e., $A' + B'$. Clearly, both symbols describe the NAND operation but, for reasons that will become more evident later, we prefer to think in terms of AND, OR, and NOT. For example, when realizing the function $P + Q$ it is natural to think in terms of an OR operation; consequently, a gate of the type shown in Fig. 5.16b, whose inputs are P' and Q' , is used to describe the realization of this function. Similar arguments explain the use of the symbols shown in Fig. 5.16c, d for NOR gates.

The assignment of two symbols to represent the same gate circuit is confusing, at first, but very convenient, because it provides a deeper insight into the logic operations taking place within the circuit. It enables the designer to analyze a circuit constructed of NAND or NOR gates by employing the same techniques as those used for circuits consisting of AND, OR, and NOT gates. In other words, the main feature of this notation is that a given circuit may be viewed as either an AND gate or an OR gate, depending on the required logic operation.

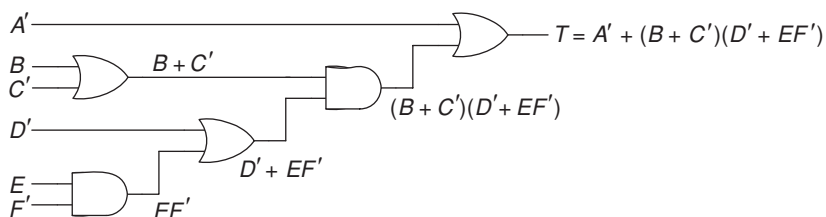
Analysis and synthesis of NAND-NOR network

The usefulness of having two symbols to represent a NAND gate will be demonstrated by analyzing the circuit shown in Fig. 5.17a. Since every small circle represents an inversion, if a line connecting two gates has circles at both ends then *both* circles may be ignored because their net logic effect is nil. Whenever a circuit has a line with a circle at one end and a switching variable (or expression) at the other end (e.g., input or output lines), it is logically equivalent to a circuit that has a connecting line from which the circle has been removed and the variable complemented. This process does not guarantee that

Fig. 5.17 Analysis of a NAND-logic circuit.



(a) NAND-logic circuit.



(b) Logically equivalent AND-OR circuit.

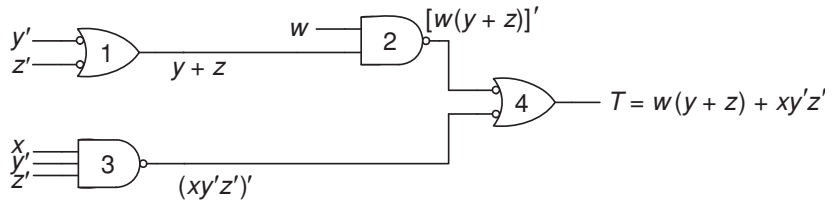
all inversion circles will be removed, but in most cases it ensures a considerably simpler circuit. In the special case in which, each gate output is connected to just a single gate input, the above process yields a circuit with no inversion circles. It follows, for the purpose of analysis, that the circuit of Fig. 5.17a is logically equivalent to the circuit of Fig. 5.17b.

With some experience, circuits consisting of NAND or NOR logic can be analyzed directly, without actually converting the circuit to its equivalent AND-OR form. For example, gate 1 of Fig. 5.17a performs an AND operation and an inversion on its inputs E and F' . This is denoted by $(EF')'$. Gate 2, however, performs an OR operation on the inverted inputs. Its output, therefore, is $D' + [(EF')'] = D' + EF'$. In a similar manner, we find that the output of gate 3 is $B + C'$ while that of gate 4 is the complement of the AND combination of its inputs, as shown in the diagram. The analysis is completed by determining the OR combination of the complemented inputs to gate 5.

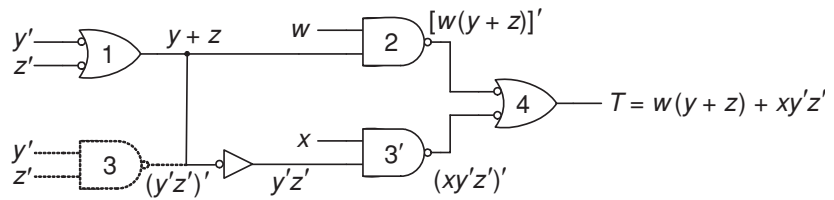
The logic diagram of Fig. 5.17a is characterized by the property that the polarities at all points match completely; that is, if a line connecting two gates has an inversion circle at one end then it also has such a circle at the other end. As a result, the logically equivalent AND-OR circuit contains no inversion circles. In general, however, it may happen that a circled gate output is connected to an uncircled gate input, or vice versa. In such cases, some inversion circles cannot be removed, and the logically equivalent circuit will consist of AND and OR as well as NOT gates, where each NOT gate replaces an inversion circle.

Consider now the function $T = w(y + z) + xy'z'$, whose realization, consisting of four NAND gates, is shown in Fig. 5.18a. The choice of symbol to be used for each gate is dictated by the operation which that gate must perform. For example, the function of gate 1 is to produce the OR combination of $y + z$ and, accordingly, the symbol of Fig. 5.16b is selected. Gate 2, however,

Fig. 5.18 Synthesis of a NAND circuit.



(a) First realization.



(b) Realization with two-input gates.

is to produce the complement of the AND combination of w and $y + z$, and thus the symbol of Fig. 5.16a is chosen. The symbols for the other gates are selected in a similar manner, and we find the output of gate 3 to be $(xy'z')'$, while that of gate 4 is the OR combination of its complemented inputs, that is, $T = w(y + z) + xy'z'$.

This circuit can also be realized with just two-input gates, as shown in Fig. 5.18b. (For the moment, disregard the line connecting the outputs of gates 1 and 3.) In this circuit, the output of gate 3 is the complement of the AND combination of its inputs, i.e., $(y'z')'$. The NOT¹ gate inverts this output, so that the input to gate 3' is $y'z'$. The outputs of gates 3' and 4 are established in a similar manner. At this point, we observe that the inputs and functional operations of gates 1 and 3 are identical. We may, therefore, delete gate 3 after having connected its output to that of gate 1.

It must be emphasized that the assumed logic polarity and symbols used to describe a circuit are important only insofar as the interpretation of the circuit is concerned; the circuit's actual operation is independent of the precise symbol used and the logic polarity assumed. In other words, the circuit "does not know" which symbols are used to describe it and whether we associate the constant 1 or the constant 0 with the high voltage.

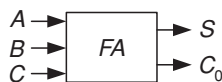
5.4 Design of high-speed adders

The design of high-speed adders serves as an example of the methods of logic design and at the same time illustrates the important and interesting circuits

¹ The NOT gate can be implemented by either joining together the two inputs of a two-input NAND or NOR gate or by providing 1 (0) to one of the inputs of the NAND (NOR) gate.

Fig. 5.19 A full adder *FA*.

<i>A</i>	<i>B</i>	<i>C</i>	<i>S</i>	<i>C</i> ₀
0	0	0	0	0
0	0	1	1	0
0	1	1	0	1
0	1	0	1	0
1	1	0	0	1
1	1	1	1	1
1	0	1	0	1
1	0	0	1	0



(b) Block diagram.

(a) Truth table for *S* and *C*₀.

widely used in most computing machines. Since addition² is one of the most important operations of a computer, the minimization of addition time is an important task of any logic designer. It will subsequently be shown that carry-propagation is the most critical issue in speeding up addition, and the usual trade-off between speed, on the one hand, and simplicity and area, on the other, will become evident.

The full adder

A *full adder* is a device capable of performing the binary addition of three binary digits, arguments *A* and *B* and carry-in *C*, from which it computes the sum *S* and carry-out *C*₀. Consider, for example, the addition of the binary numbers 1011 and 0011:

$$\begin{array}{rcl}
 011 & \text{carry-in} \\
 1011 & \text{augend} \\
 0011 & \text{addend} \\
 \hline
 1110 & \text{sum}
 \end{array}$$

The carry-out produced in the addition of the *i*th significant digits must be incorporated, as a carry-in, in the addition process for the (*i* + 1)th significant digit.

The truth table defining the input–output functional relationship for the full adder is shown in Fig. 5.19, together with its block-diagram representation. The logic equations for the sum and carry-out, derived from the truth table, are

² By “addition,” we shall mean both addition and subtraction in all subsequent discussions, since the latter operation is generally accomplished by the addition of the inverted subtrahend (the term subtracted) in sign-and-magnitude machines, or by the addition of the 2’s complement of the subtrahend in 2’s-complement machines.

given by

$$\begin{aligned}
 S &= A'B'C + A'BC' + AB'C' + ABC \\
 &= A \oplus B \oplus C, \\
 C_0 &= A'BC + ABC' + AB'C + ABC \\
 &= AB + AC + BC.
 \end{aligned}$$

A realization of the full adder was shown in Fig. 5.1. A NAND-logic realization is shown in Fig. P5.15.

The ripple-carry adder

In order to add two n -digit binary numbers, it is necessary to connect n stages of full adders in such a way that each stage computes the corresponding sum and carry. All high-speed adders are basically *parallel* devices, i.e., devices constructed of full adders connected in such a manner that all digits of the augend and addend are fed into them simultaneously. Hence, the number of full adders required for a parallel implementation of an adder is equal to the word length n of the machine.

Let A_i and B_i be the i th digits of the two arguments being added, and let S_i be their sum; C_{0i} and C_i designate the carry-out of the i th full adder and the carry-in of that adder, respectively. The logic equations of the i th full adder are

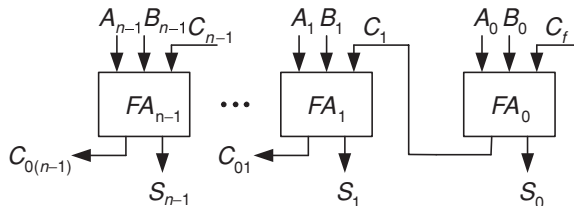
$$\begin{aligned}
 S_i &= A_i \oplus B_i \oplus C_i, \\
 C_{0i} &= A_i B_i + A_i C_i + B_i C_i,
 \end{aligned}$$

where $i = 0, 1, \dots, n-1$. The carry-in C_f into the zeroth (least significant) full adder is zero if the adder is being used for binary addition but can be equal to 1 for other operations, such as incrementing results or subtracting in a 2's-complement machine.

The conventional *ripple-carry adder* consists of a number of stages of full adders, such that the carry-out of the i th stage becomes the carry-in for the $(i+1)$ th stage, i.e., $C_{0i} = C_{i+1}$, as illustrated in Fig. 5.20. The carry C_f is usually referred to as the *forced carry*, while $C_{0(n-1)}$ is the *overflow carry*.

The time required to perform addition in the ripple-carry adder is the time required for the propagation (or ripple) of the carries in the stages. Although a carry will not propagate through all stages in every addition, the time allotted

Fig. 5.20 A ripple-carry adder.



for the addition operation must be at least equal to the longest carry-propagation time (plus the addition time in the last full adder). The adder is assumed to produce the sum in a *fixed time* regardless of the actual carry or the numbers being added. If we assume that two time units are required for generating the carry in one (two-level) full-adder stage then the fixed time that must be allotted to the n -stage ripple-carry adder is at least $2n$ units. This implies that the adder is part of a synchronous system and that the next summands must not be transferred into the adder until at least $2n$ time units have elapsed since the transfer of the current summands. In order to increase the speed of the adder, it is necessary to minimize the fixed time required for carry propagation.

The carry-lookahead adder

The *carry-lookahead adder* is a fixed-time adder in which several stages are simultaneously examined and their carries are generated in parallel.

The carry equation can be rewritten as follows. Define D_i and T_i as the *generated* and *propagated* carry signals for the i th stage, where

$$\begin{aligned} D_i &= A_i B_i, \\ T_i &= A_i \oplus B_i = A_i' B_i + A_i B_i'. \end{aligned}$$

Then

$$C_{0i} = D_i + T_i C_i, \quad (5.1)$$

where D_i equals 1 if a carry is generated in the i th stage, i.e., if $A_i = B_i = 1$; T_i equals 1 if either A_i or B_i , but not both, is equal to 1. If $T_i = 1$ and $C_i = 1$ then $C_{0i} = 1$; that is, the carry-out of the $(i - 1)$ th stage will propagate uninterrupted through the i th stage into the $(i + 1)$ th stage.

In order to generate the carries in a parallel manner, it is necessary to transform the recursive form of the carry function into a nonrecursive form. This can be achieved as follows:

$$\begin{aligned} C_{0i} &= D_i + T_i C_i, \\ C_i &= C_{0(i-1)}, \\ C_{0i} &= D_i + T_i (D_{i-1} + T_{i-1} C_{i-1}) \\ &= D_i + T_i D_{i-1} + T_i T_{i-1} (D_{i-2} + T_{i-2} C_{i-2}) \\ &= D_i + T_i D_{i-1} + T_i T_{i-1} D_{i-2} + T_i T_{i-1} T_{i-2} C_{i-2}. \end{aligned}$$

If we continue this iteration, we are able to express the carry-out of the i th stage directly in terms of external inputs (i.e., excluding carries) of the preceding stages and the forced carry (note that $C_{i-i} = C_f$). Hence,

$$C_{0i} = D_i + T_i D_{i-1} + T_i T_{i-1} D_{i-2} + \cdots + T_i T_{i-1} T_{i-2} \cdots T_0 C_f. \quad (5.2)$$

Equation (5.2) actually defines the i th carry-out C_{0i} to be 1 if it has been generated in the i th stage or originated in a preceding stage and propagated by all subsequent stages.

The implementation of the above lookahead scheme for the entire adder is not practical, because it requires a very large number of gates and, in addition, for each stage of the adder it is necessary to have an OR gate with n inputs and n AND gates with 1 through n inputs. Also, since a modern computer may have 64-bit words, such a complete lookahead scheme cannot be economically accomplished. The limitation can be overcome, though at the expense of computation speed, by dividing the n stages of the adder into groups such that within each group a full carry lookahead, as defined by Eq. (5.2), is achieved while a ripple carry is maintained between groups. For the purpose of illustration, let us consider groups consisting of three full-adder stages, i.e., group 1 consists of stages 0 through 2, group 2 consists of stages 3 through 5, etc. The carry-out of group k (i.e., the carry-in for the group $k + 1$) will be denoted C_{gk} . The first three-stage group with full carry lookahead is shown in Fig. 5.21a, where the block diagram of each full adder is shown with its sum network (SN) and carry network (CN) separated. The details of the carry networks are given in Fig. 5.21b. The sum networks are the conventional ones, i.e., $S_i = T_i \oplus C_i$. The double-arrow inputs to carry network CN_i indicate that A_0 through A_i and B_0 through B_i are the inputs to that carry network.

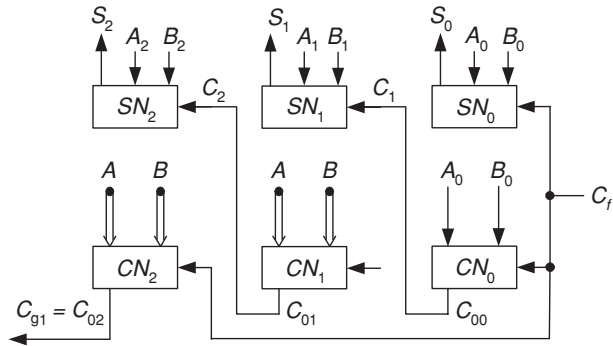
It takes four time units to generate C_{g1} , because there are four levels of gates in CN_2 . (Two units are required to produce T_i and two units to compute C_{g1} in CN_2 .) The generation of C_{g2} and any subsequent group carry requires only two time units, because the necessary generate (D_i) and propagate (T_i) signals are already available. Two additional time units are required in the final sum stage. Consequently, for an n -stage adder divided into three-stage groups with full lookahead within each group and ripple carry between groups, the longest propagation time is $4 + 2n/3$ units as compared with $2n$ units for the ripple-carry adder. A schematic diagram of a 30-digit adder with full lookahead within each three-digit group and ripple carry between groups is shown in Fig. 5.22. The lookahead adder requires about 50% additional hardware, a relatively small price for the threefold increase in speed.

The adder shown in Fig. 5.22 is called *one-level lookahead*. It is also possible to design adders with higher levels of lookahead. This is accomplished by designating a number of groups as a *section* and having a second level of lookahead to speed up the propagation of carries between groups within a section.

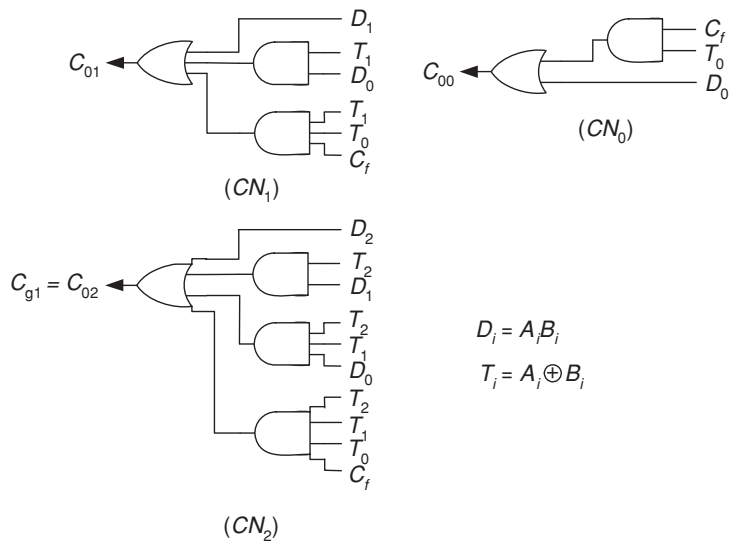
5.5 Metal-oxide semiconductor (MOS) transistors and gates

Currently, complementary metal-oxide semiconductor (CMOS) is the dominant technology for implementing chips. Thus, it would be instructive to see how gates and Boolean functions can be implemented in CMOS technology.

Fig. 5.21 Three-digit adder group with full carry lookahead.



(a) Block diagram of initial three-stage group.



(b) The carry networks.

Fig. 5.22 Schematic diagram of a 30-digit adder with full lookahead within three-digit groups and ripple carry between groups.

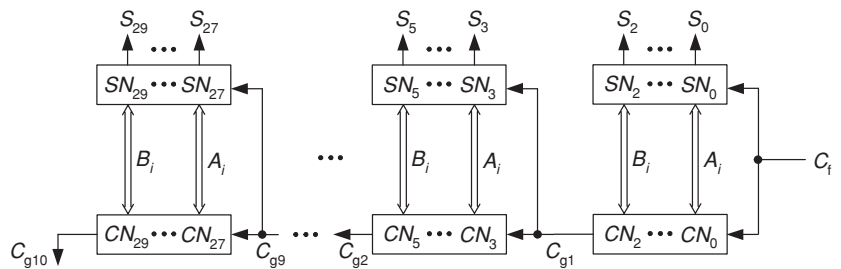
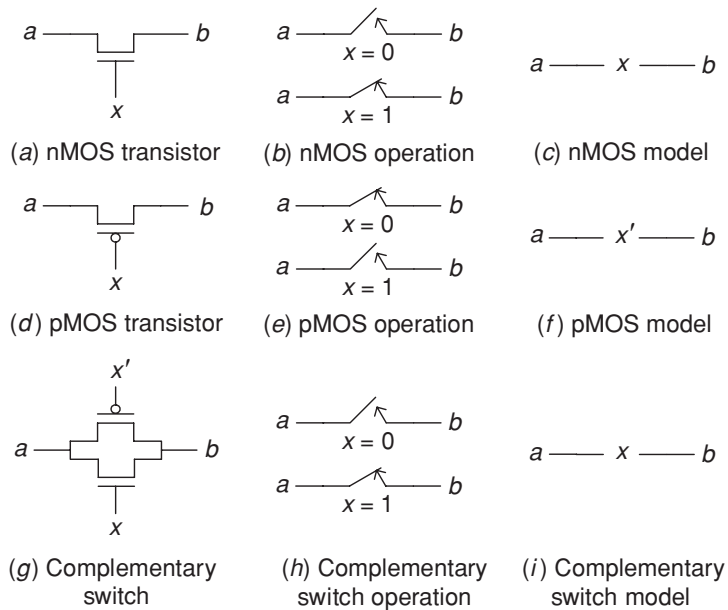


Fig. 5.23 MOS transistor operation.



Two types of transistor are used in CMOS: nMOS and pMOS. Both are three-terminal devices and act like a switch. An nMOS transistor and its switch operation are shown in Fig. 5.23a. The switch is open when $x = 0$ and closed when $x = 1$, as shown in Fig. 5.23b. The opposite is true for the pMOS transistor shown in Fig. 5.23d. It is closed when $x = 0$ and open when $x = 1$, as shown in Fig. 5.24e.

An nMOS transistor passes a 0 perfectly, but a 1 imperfectly. For example, in Fig. 5.23a, if a 0 is placed at terminal a and x is set to 1 then terminal b assumes close to the same voltage as a and thus also has a 0. However, if a 1 is placed at terminal a and x is again set to 1, then the voltage level at terminal b is somewhat lower than at a , although it is still recognized as a 1. The opposite is true for a pMOS transistor. It is good at propagating a 1, but bad at propagating a 0. To overcome this drawback of nMOS and pMOS transistors they can be connected in parallel, as shown in Fig. 5.23g, in what is called a *complementary switch*. This switch is closed when $x = 1$ since both its transistors are closed for this value, as shown in Fig. 5.23h. It is open when $x = 0$ since both its transistors are open in this case.

The analogy of MOS transistors to the gates defined in Section 3.3 is evident. We may, therefore, utilize switching expressions to represent MOS transistors and networks and, conversely, any switching expression can be realized by an appropriate connection of such transistors. The models indicating the condition for transmission for the nMOS transistor, pMOS transistor, and complementary switch are shown in Fig. 5.23c, parts f and i , respectively. The transmission function of a network consisting of a parallel connection of two switches

Fig. 5.24 Basic transmission functions.

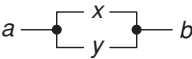
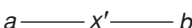
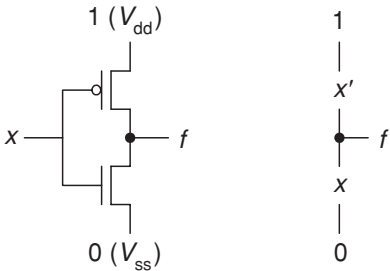
Network	Transmission function
	$T_{ab} = x + y$
	$T_{ab} = xy$
	$T_{ab} = x'$

Fig. 5.25 CMOS NOT gate and its transmission functions.



with symbols x and y is $x + y$, whereas that for a network consisting of a serial connection of these switches is xy . The transmission functions of various networks are shown in Fig. 5.24. Each switch in these networks can be implemented with an nMOS or pMOS transistor or a complementary switch.

Networks of nMOS and pMOS transistors can be connected to form CMOS gates. The simplest is a CMOS NOT gate. Such a gate and the transmission functions of both its transistors are shown in Fig. 5.25. When $x = 0$ the value 1 propagates to output f of the gate and when $x = 1$ the value 0 propagates to f , thus realizing a NOT operation.

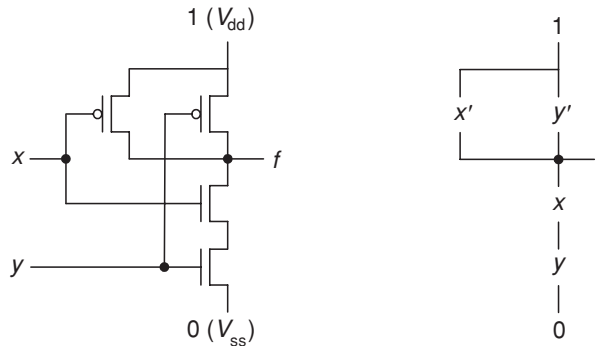
The CMOS NAND and NOR gates and the corresponding transmission functions of their nMOS and pMOS networks are shown in Fig. 5.26. For a NAND gate, we can see that a 0 propagates to output f only if $x = y = 1$. For all other combinations of input values, a 1 propagates to f . For a NOR gate, a 1 propagates to output f only if $x = y = 0$. For all other combinations of input values, a 0 propagates to f .

From the above analysis, it should be obvious that only one of the two networks (nMOS or pMOS) conducts in the steady state for a given set of input values. This is true for all such CMOS gates.

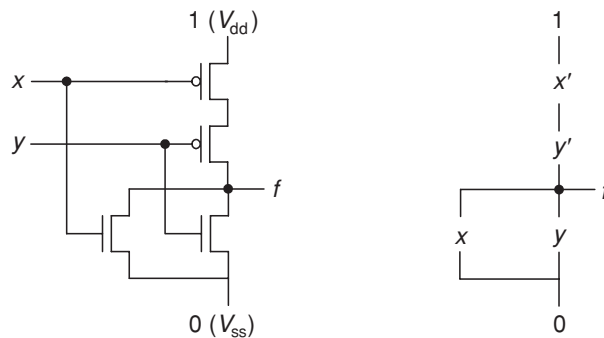
5.6 Analysis and synthesis of MOS networks

By the analysis of a two-terminal MOS network we mean the determination of its transmission function. For networks that have more than two

Fig. 5.26 NAND and NOR gate operation.



(a) CMOS NAND gate and its transmission functions.



(b) CMOS NOR gate and its transmission functions.

terminals the analysis involves the determination of a transmission function for each pair of terminals. The synthesis problem of a MOS network is the converse of its analysis; the desired network performance is specified by a switching expression, from which a corresponding circuit is derived.

Analysis of series-parallel networks

In the preceding section, it was shown that the transmission function of a network that consists of two MOS transistors with transmission functions x and y , connected in parallel, is $x + y$ and that the transmission function of a network consisting of two MOS transistors connected in series is xy . Since the algebra of MOS networks is isomorphic to switching algebra, the transmission function of two networks, T_1 and T_2 , connected in series is $T_1 T_2$ and the transmission function of a parallel connection of these two networks is $T_1 + T_2$. Utilizing these properties, we can determine the transmission function of any series-parallel network.

Example Find the transmission function for the network of Fig. 5.27a. The network consists of a switch x' in series with another network, which contains two parallel subnetworks. The transmission function of the upper subnetwork can be written by inspection as $(y'z + yz')w'$. The lower subnetwork contains three parallel branches. Its transmission function is $w + y' + x'z'$. Thus, the overall transmission function is given by

$$T_{ab}(w, x, y, z) = x'[(y'z + yz')w' + w + y' + x'z'].$$

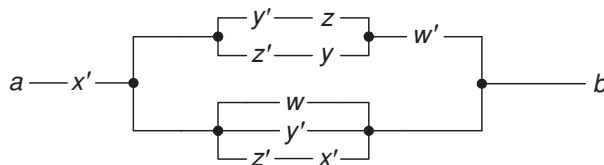
This expression may be simplified to

$$T_{ab}(w, x, y, z) = x'(w + y' + z').$$

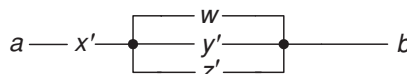
The simplified network is shown in Fig. 5.27b. For some CMOS implementations discussed later, we will also need the complement of the transmission function. From De Morgan's theorem,

$$T'_{ab} = x + w'yz.$$

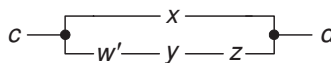
The network corresponding to T'_{ab} is shown in Fig. 5.27c.



$$(a) T_{ab} = x'[(y'z + yz')w' + w + y' + x'z'].$$



$$(b) T_{ab} = x'(w + y' + z').$$



$$(c) T_{cd} = T'_{ab} = x + w'yz.$$

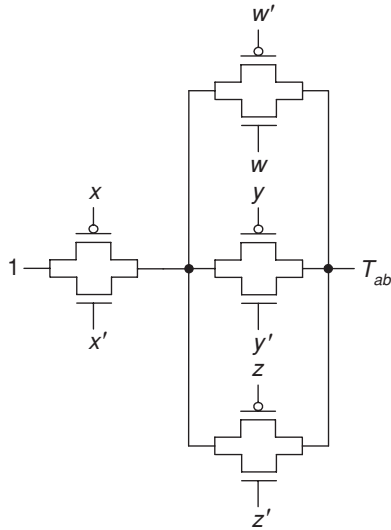
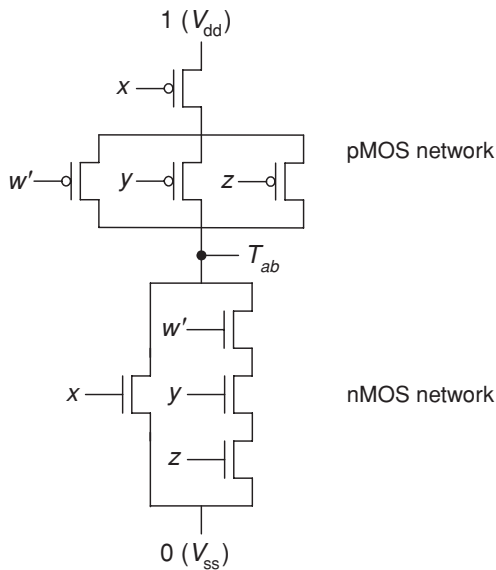
Fig. 5.27 Analysis and simplification of a series-parallel network.

Using the procedure illustrated in the preceding example, we can associate a switching expression with every series-parallel network; conversely, to every switching expression there corresponds a series-parallel network. This example also demonstrates that in order to simplify a network it is advisable first to find its transmission function and then to simplify it wherever possible.

Let us next see how a CMOS implementation can be derived from the simplified network shown in Fig. 5.27b. A complementary-switch-based CMOS

Fig. 5.28

Complementary-switch-based implementation.

**Fig. 5.29** A complex CMOS gate.

implementation for realizing T_{ab} is shown in Fig. 5.28. From the complementary switch and its symbol shown in Fig. 5.23*g, i*, we can see that this simply involves a one-to-one mapping from the network in Fig. 5.27*b* to the one in Fig. 5.28.

A complex CMOS-gate³ implementation for T_{ab} is shown in Fig. 5.29. Its pMOS network is derived from Fig. 5.27*b*. Note that, since a pMOS transistor fed by x conducts when x' is true (see Fig. 5.23*d, f*), a transmission network

³ A CMOS gate is said to be complex if it does not implement a primitive function such as a NOT, NAND or NOR gate.

branch fed by x' is replaced by a pMOS transistor fed by x . This type of straightforward mapping is possible for a pMOS network since it transmits a 1 through it to the output. However, since an nMOS network transmits a 0 through it to the output, we must first derive the network for the complement of the function being synthesized. The network for T'_{ab} is shown in Fig. 5.27c. Since an nMOS transistor fed by x conducts when x is true (see Fig. 5.23a, c), a transmission network branch fed by x is replaced by an nMOS transistor fed by x .

A simpler way to obtain the pMOS network for a complex gate given its nMOS network, or vice versa, is to replace a series (parallel) connection in one network with a parallel (series) connection in the other. One can deduce this from the nMOS and pMOS networks of Fig. 5.29.

Analysis of non-series-parallel networks

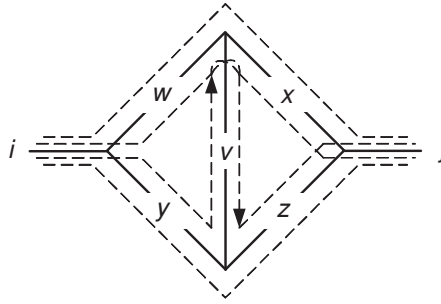
A question now arises as to the relationship between switching expressions and non-series-parallel networks. The previously described analysis procedure is clearly not applicable to bridge-type networks (e.g., Fig. 5.30), and a different, more general, procedure must be developed. In the case of series-parallel networks, switching expressions provide information regarding the structure (or geometry) of the network as well as its transmission. Switching expressions can also be found that reflect the transmission properties, but not the structure, of nonseries-parallel networks.

One way to obtain the transmission function between two terminals of a given network is by tracing all paths from one terminal to the other (see the broken lines). In the bridge network of Fig. 5.30, one path from terminal i to terminal j consists of a series connection of branches w and x . Transmission through this path is 1 if both w and x are 1, i.e., conducting. Hence, this path can be expressed by the product wx . If we associate with each path from terminal i to terminal j a product of literals corresponding to the branches encountered in the path then the sum of all these products is the required transmission function T_{ij} . These paths are known as the *tie sets* of the network. Each tie set represents a minimal path between the two network terminals such that, whenever all the branches in the path are conducting, the transmission through the path is 1 regardless of the state of all other branches in the network. Using this technique, the transmission function for the bridge network of Fig. 5.30a is found to be

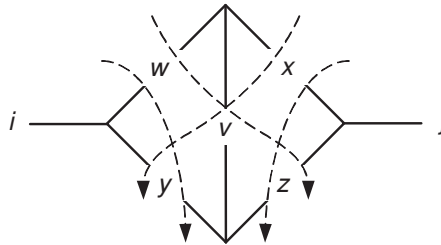
$$T_{ij} = wx + wvz + yvx + yz.$$

A dual technique is illustrated in Fig. 5.30b. Broken lines are drawn through, rather than along, the network branches, so as to separate terminal i from terminal j in all possible ways and thus to render the transmission T_{ij} equal to 0. For example, the transmission T_{ij} is 0 if both branches w and y are open, regardless of the state of the other branches in the network. Similarly, if w , v , and z are open then T_{ij} is 0, and so on. If we express each such “cut” through the network by a sum of literals, e.g., $w + y$, then the product of all these sums is 0 whenever

Fig. 5.30 Analysis of a bridge network.



(a) Tie sets. $T_{ij} = wx + wvz + yvx + yz$.



(b) Cut sets. $T_{ij} = (w + y)(w + v + z)(x + v + y)(x + z)$.

any of its factors is 0. For all other combinations, the product will have the value 1. Consequently, this product is a conjunctive expression for the transmission function of the network. For the bridge network of Fig. 5.30b, we thus have

$$T_{ij} = (w + y)(w + v + z)(x + v + y)(x + z).$$

The minimal sets of switches which, when open, ensure that the network transmission is 0 are known as the *cut sets* of the network. Thus, no conducting path can be found between terminals i and j of a given network when any cut set equals 0.

In determining the tie sets, all paths containing a product of a variable and its complement, e.g., xx' , are ignored. Similarly disregarded are all sums containing a variable and its complement, e.g., $x + x'$, when determining the cut sets.

Synthesis of MOS networks

The synthesis of a network with given properties can be accomplished in several steps. First, the requirements that the network needs to satisfy are expressed algebraically in the form of switching expressions. For simple networks, this can be done directly from a “verbal” description of the required properties. In other cases a truth table must be employed and switching expressions derived from it. Next, these switching expressions are simplified as much as possible,

and a corresponding series–parallel network is obtained. In general, although the expressions may be minimal, the corresponding series–parallel network can be further simplified. Consequently the final step in the synthesis procedure is the simplification of the network.

When simplifying a network, extreme care must be taken to prevent the introduction of undesired paths through the network, which may change its transmission function. Such paths, called *sneak paths*, occur in MOS networks because they are bilateral: they allow the flow of current in both directions.

Example Design a minimal network, with four inputs, w , x , y , and z , that receives BCD numbers and produces a signal whenever the current number is 3 or a multiple of 3.

The map that specifies the transmission function of the desired network is shown in Fig. 5.31a. It contains three 1-cells, in combinations 3, 6, and 9, and six don't-care combinations corresponding to all invalid BCD code words. The minimal sum-of-products expression derived from the map is

$$\begin{aligned} T(w, x, y, z) &= wz + xyz' + x'yz \\ &= z(w + x'y) + xyz'. \end{aligned}$$

The corresponding series–parallel network is shown in Fig. 5.31b. In order to eliminate one of its y branches, we check whether the connection shown by the broken line can be made without introducing any undesired path. If we actually make the connection and eliminate one of the y branches, we obtain the network of Fig. 5.31c where the only sneak path that could be introduced is $z'xx'w$; but, since it is always open, it has no effect on the transmission of the network.

The network of Fig. 5.31c consists of only six branches, as opposed to seven in the series–parallel network of Fig. 5.31b, and is minimal.

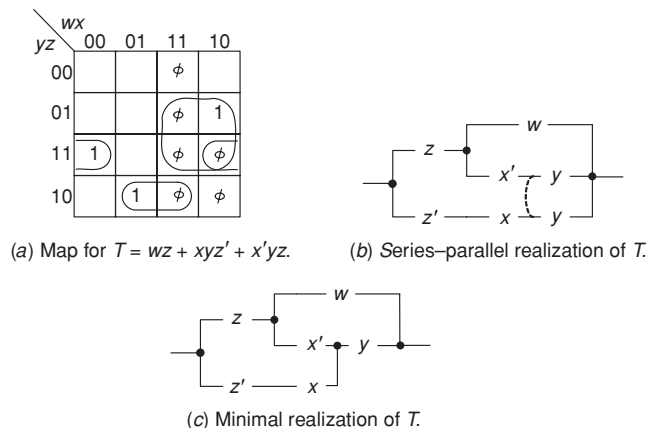


Fig. 5.31 Realization of $T(w, x, y, z) = \sum(3, 6, 9) + \sum_{\phi}(10, 11, 12, 13, 14, 15)$.

A complementary-switch-based CMOS implementation can be derived directly from the nonseries-parallel network in Fig. 5.31c by one-to-one mapping. While complex CMOS gates can also be implemented with non series-parallel nMOS and pMOS networks, in practice most complex gates employ series-parallel networks.

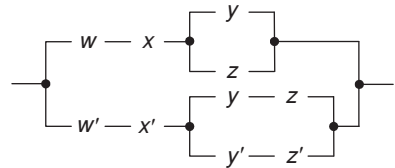
Example Design a minimal network that realizes the function $T(w, x, y, z) = \sum(0, 3, 13, 14, 15)$.

With the aid of the map of Fig. 5.32a, the algebraic expression corresponding to T is found to be

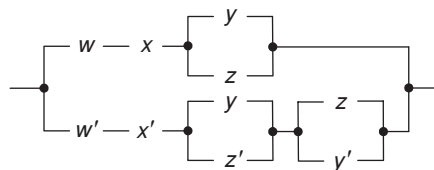
$$\begin{aligned} T &= wxy + wxz + w'x'y'z' + w'x'yz \\ &= wx(y + z) + w'x'(y'z' + yz). \end{aligned}$$

	wx			
yz	00	01	11	10
00	1			
01			1	
11	1		1	
10			1	

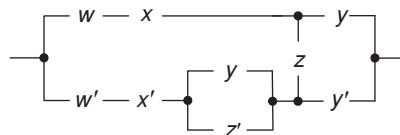
(a) Map for $T = wxy + wxz + w'x'y'z' + w'x'yz$.



(b) Series-parallel realization of T .



(c) An alternative series-parallel realization of T .



(d) A minimum realization of T .

Fig. 5.32 Realization of $T(w, x, y, z) = \sum(0, 3, 13, 14, 15)$.

The corresponding series–parallel network is shown in Fig. 5.32*b*. In Fig. 5.32*c*, the lower branch of the network has been redrawn utilizing the identity $yz + y'z' = (y + z')(y' + z)$. This enables us to combine the two parallel z branches, as shown in Fig. 5.32*d*.

There exist several synthesis procedures for nonseries–parallel networks. Among the more important and interesting of these approaches are applications of the theory of matrices and graph theory to the synthesis problem. These methods are available in various references among which are [2, 4, 9].

Notes and references

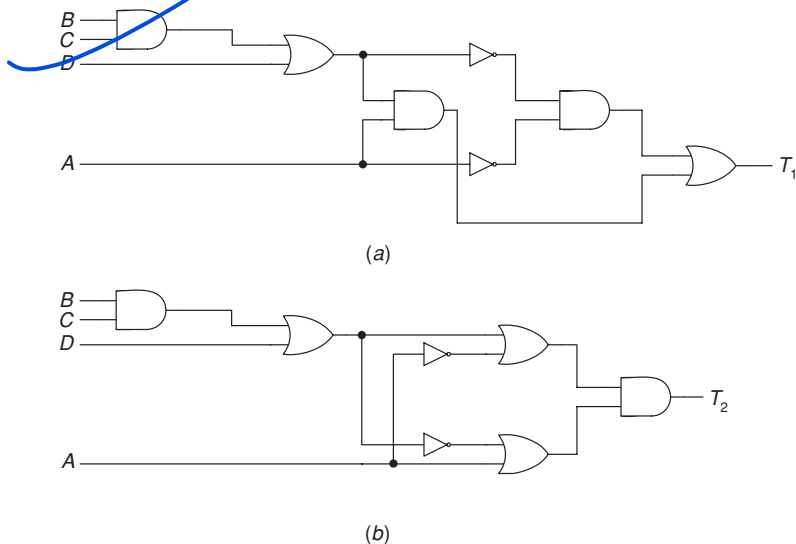
There are numerous books on logic design, among which are Hill and Peterson [3], Katz and Borriello [5], Mano and Ciletti [8], Wakerly [11], and many others. A comprehensive review of high-speed adders is given in MacSorley [7] and Koren [6]. The material on transmission networks dates back to Shannon's original work [10]. An extensive treatment of such networks is available in Caldwell [1].

- [1] Caldwell, S. H.: *Switching Circuits and Logical Design*, John Wiley & Sons, New York, 1958.
- [2] Gould, R.: "Application of graph theory to the synthesis of contact networks," in *Proc. Int. Symp. Theory of Switching*, pp. 244–292, Harvard University Press, Cambridge MA, 1959.
- [3] Hill, F. J., and G. R. Peterson: *Computer Aided Logical Design with Emphasis on VLSI*, fourth edition, John Wiley & Sons, New York, 1993.
- [4] Hohn, F. E., and L. R. Schissler: "Boolean matrices and the design of combinational relay switching circuits," *Bell System Tech. J.*, vol. 34, no. 1, pp. 177–202, 1955.
- [5] Katz, R. H., and G. Borriello: *Contemporary Logic Design*, second edition, Pearson Prentice Hall, Upper Saddle River NJ, 2005.
- [6] Koren, I: *Computer Arithmetic Algorithms*, A. K. Peters, Natick, MA, 2002.
- [7] MacSorley, O. L.: "High-speed arithmetic in binary computers," *Proc. IRE*, vol. 49, no. 1, pp. 67–91, January 1961.
- [8] Mano, M. M., and M. D. Ciletti: *Digital Design*, fourth edition, Prentice Hall, Upper Saddle River NJ, 2007.
- [9] Semon, W.: "Matrix methods in the theory of switching," in *Proc. Int. Symp. Theory of Switching*, pp. 13–50, Harvard University Press, Cambridge MA, 1959.
- [10] Shannon, C. E.: "A symbolic analysis of relay and switching circuits," *Trans. AIEE*, vol. 57, pp. 713–723, 1938.
- [11] Wakerly, J. F.: *Digital Design Principles and Practices*, Prentice Hall, Englewood Cliffs NJ, 1990.

Problems

Problem 5.1. Express T_1 and T_2 (see Fig. P5.1a, b) as functions of A , B , C , and D .

Fig. P5.1



Problem 5.2

- (a) Design a two-level code converter from BCD to the *2-out-of-5* code shown in Table P5.2a.
- (b) Design a two-level code converter from the *Ringtail* code shown in Table P5.2b to BCD.

Table P5.2

Decimal	2-out-of-5						Decimal	Ringtail					
0	1	1	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	1	1	1	1	0	0	0	0	1	1
2	0	0	1	0	1	1	2	0	0	0	1	1	1
3	0	0	1	1	0	1	3	0	0	1	1	1	1
4	0	1	0	0	1	1	4	0	1	1	1	1	1
5	0	1	0	1	0	1	5	1	1	1	1	1	1
6	0	1	1	0	0	0	6	1	1	1	1	0	0
7	1	0	0	0	1	1	7	1	1	1	0	0	0
8	1	0	0	1	0	0	8	1	1	0	0	0	0
9	1	0	1	0	0	0	9	1	0	0	0	0	0

(a)

(b)

Problem 5.3. Design a circuit with four inputs, x_1 , x_2 , x_3 , x_4 , and seven outputs, p_1 , p_2 , m_1 , p_3 , m_2 , m_3 , m_4 , that receives BCD code words and generates the corresponding Hamming code words defined in Table 1.8.

Problems

Problem 5.4. You are supplied with just one NOT gate and an unlimited amount of AND and OR gates and are required to design a circuit that realizes the expression

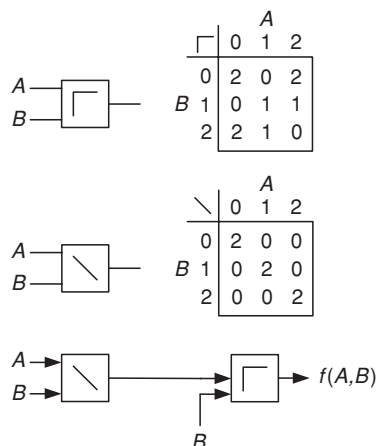
$$T(w, x, y, z) = w'x + x'y + xz'.$$

Only unprimed variables are available as inputs.

Hint: You may find the map of T helpful.

Problem 5.5. The tables shown in Fig. P5.5 define two devices whose inputs and outputs may assume any one of the *three* values 0, 1, or 2.

Fig. P5.5



Give the equivalent of a Karnaugh-map description of the function $f(A, B)$ that is realized by the network of Fig. P5.5.

Problem 5.6. A certain four-input gate, called a LEMON gate, realizes the switching function $LEMON(A, B, C, D) = BC(A + D)$. Assume that the input variables are available in both primed and unprimed form.

(a) Show a realization of the function

$$f(w, x, y, z) = \sum(0, 1, 6, 9, 10, 11, 14, 15)$$

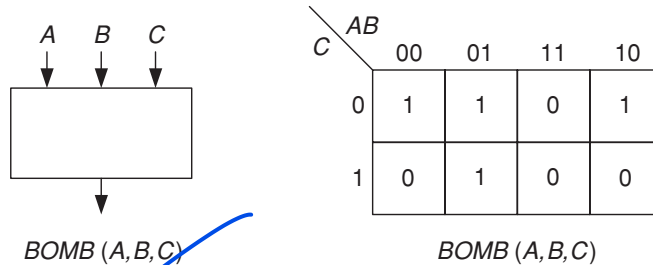
with only three LEMON gates and one OR gate.

(b) Can all switching functions be realized with LEMON and OR logic?

Hint: Draw the map for LEMON and utilize possible “patches” (coverings of the minterms of f with the LEMON function) on the map of f .

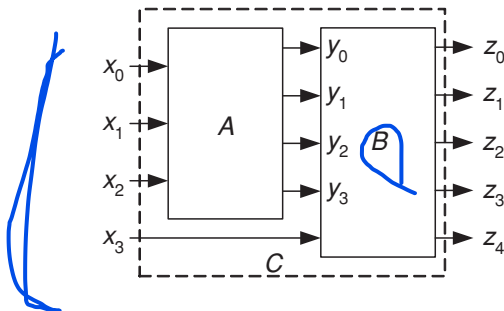
Problem 5.7. A three-input gate, BOMB, whose characteristics are shown in Fig. P5.7, has been mass-produced by an unfortunate company. Experimental evidence shows that input combinations 101 and 010 cause the gate to physically explode. Your task is to determine whether the gate is completely useless or can be externally modified such that it may be efficiently used to implement any switching function without causing explosions.

Fig. P5.7



Problem 5.8. A logic module A , shown in Fig. P5.8, operates as follows: output $y_i = 1$ iff i inputs out of x_0, x_1, x_2 are equal to 1. Design unit B in such a way that the overall logic function of unit C will be to produce an output $z_i = 1$ iff i inputs out of x_0, x_1, x_2, x_3 are equal to 1.

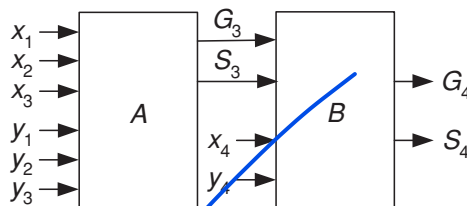
Fig. P5.8



Problem 5.9. Given a logic module A that compares the magnitudes of two 3-bit numbers, $X_3 = x_1x_2x_3$ and $Y_3 = y_1y_2y_3$, where x_3 and y_3 are the least significant bits. Module A has two outputs G_3 and S_3 , such that: $G_3 = 1$ iff $X_3 > Y_3$; $S_3 = 1$ iff $X_3 < Y_3$; and $G_3 = S_3 = 0$ iff $X_3 = Y_3$.

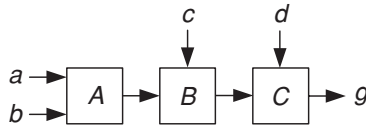
- Design a logic unit B such that together with module A it will serve as a comparator for two four-bit numbers, $X_4 = x_1x_2x_3x_4$ and $Y_4 = y_1y_2y_3y_4$, as shown in Fig. P5.9. Find expressions for G_4 and S_4 in terms of the inputs to unit B and show a realization of these expressions using only NAND gates.
- Show a realization of module A by means of only units of type B . Assume that the constants 0 and 1 are available.

Fig. P5.9



Problem 5.10. Given a function $g(x_1, x_2, x_3, x_4) = \sum(4, 6, 7, 15) + \sum_{\phi}(2, 3, 5, 11)$, realize g in the form shown in Fig. P5.10, i.e., find the correspondence between the x_i and a, b, c, d , and determine the functions A, B , and C .

Fig. P5.10



Problem 5.11. A *half adder* is a device capable of performing the addition of two bits. It has two binary inputs, A and B , and two outputs, S and C_0 . (Note that there is no carry into the half adder.)

- Write truth tables that define the half adder and derive logic expressions for S and C_0 .
- Assuming that only uncomplemented inputs are available, show an implementation of the half adder that requires only three two-input AND or OR gates and one NOT gate.
- Under the above assumption, design the half adder using no more than five NAND gates or NOR gates, but not both together.

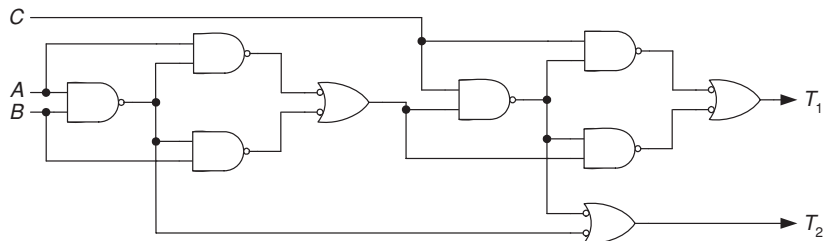
Problem 5.12. Construct a full adder using only two half adders and one OR gate.

Problem 5.13. A *half subtractor* is a device capable of subtracting one binary digit from the other. Show a realization of the half subtractor using AND, OR, NOT logic.

Problem 5.14. Define a *full subtractor*, show its truth tables, and derive logic expressions for difference (D) and borrow (B) outputs.

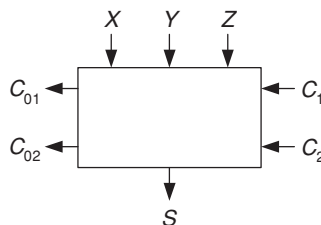
Problem 5.15. Analyze the two-output circuit shown in Fig. P5.15. Indicate the logic expression associated with every gate output.

Fig. P5.15



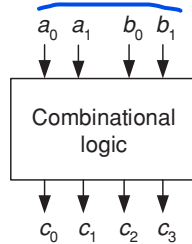
Problem 5.16. Design a device capable of adding *three* binary digits simultaneously. The device has *five* inputs, as shown in Fig. P5.16; X , Y , and Z are the arguments, C_1 is the carry-in from the preceding stage, and C_2 is the carry-in from the next-to-the-preceding stage. The output S designates the sum, while C_{01} and C_{02} designate the carry-outs to the succeeding stage and to the next-to-the-succeeding stage, respectively. Express explicitly the sum and carry-out functions and show a circuit diagram.

Fig. P5.16



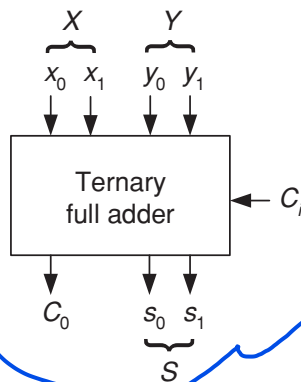
Problem 5.17. The schematic diagram in Fig. P5.17 shows a multiplier capable of multiplying two two-digit binary numbers. The digits of the two numbers are designated a_0 and a_1 , b_0 and b_1 , while c_0 , c_1 , c_2 , and c_3 designate the digits of the product. Design the combinational logic.

Fig. P5.17



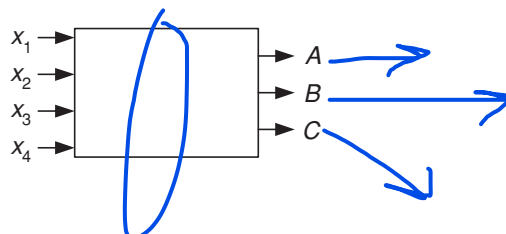
Problem 5.18. The schematic diagram shown in Fig. P5.18 shows a *ternary full adder* that receives two ternary digits X and Y plus a carry-in C_i and produces their sum S in base 3 plus a carry-out C_0 . The ternary digits are coded in binary, that is, each of the three ternary digits 0, 1, 2 is coded by two binary digits: 0 by 00, 1 by 01, and 2 by 10. Thus, for example, if X and Y are each equal to 2 in base 3 and C_i equals 1 then the ternary full adder is required to perform the ternary addition of $(2)_3 + (2)_3 + (1)_3 = (12)_3$. Accordingly, the sum S must be 2 while the carry-out must be 1. Design the circuit assuming that you have as many gates as necessary as well as binary half and full adders.

Fig. P5.18



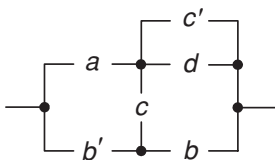
Problem 5.19. A communication system is designed to transmit just two code words, $A = 0010$ and $B = 1101$. However, owing to noise in the system, the received word may have as many as two errors. Design a combinational circuit that receives the words and that can correct one error and detect the existence of two errors. Specifically, design the circuit in Fig. P5.19 in such a way that output A will be equal to 1 if the received word is A , output B will be equal to 1 if the received word is B , and output C will be equal to 1 if the word received has two errors and thus cannot be corrected.

Fig. P5.19

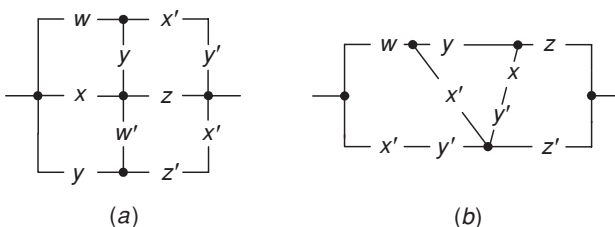


Problem 5.20

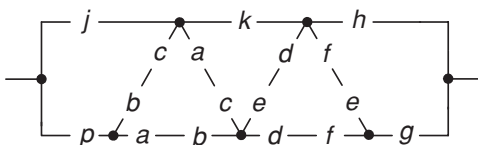
- Find all cut and tie sets for the circuit shown in Fig. P5.20. What function T is realized by this circuit?
- Prove that any network realization of T must contain at least one branch d . Generalize your arguments to determine the necessity of branches for other literals.
- Find a minimum-branch series-parallel network for T .

Fig. P5.20**Problem 5.21**

- Find a minimal network equivalent to that shown in Fig. P5.21a. It requires only five branches.
- Find a minimal complex CMOS gate which realizes a function that is the same as the transmission function realized by the network in Fig. P5.21b. It requires only 14 transistors.

Fig. P5.21

Problem 5.22. For the network of Fig. P5.22, find an equivalent network with only 11 branches.

Fig. P5.22

Problem 5.23. Design a minimal complementary-switch-based CMOS implementation that can turn a lamp on or off from three different locations independently. Denote the switches as x , y , and z .

Problem 5.24. For each of the following functions, find a network realization that requires as few branches as possible:

- $T(w, x, y, z) = \sum(0, 4, 6, 8, 9, 12)$;
- $T(w, x, y, z) = \sum(3, 7, 8, 9, 13)$;
- $T(w, x, y, z) = \sum(5, 6, 7, 9, 10, 11, 13, 14)$;
- $T(w, x, y, z) = \sum(5, 6, 9, 10, 11, 12, 13, 14, 15)$;
- $T(w, x, y, z) = \sum(5, 6, 7, 9, 10, 11, 12)$.

Problem 5.25. In a meeting of a board of directors, four resolutions, A , B , C , and D , are to be put to the vote. The decisions are complicated, however, by the fact that the resolutions are not mutually independent. In fact, voting must be governed by the following rules.

1. Those who vote for resolution B must also vote for resolution C .
2. It is possible to vote for both resolutions A and C only if a vote for either B or D is also cast.
3. Those who vote for either resolution C or D or vote against resolution A must vote for resolution B .

Each member of the board has four switches, A , B , C , and D , which he presses or releases, depending on whether he is in favor of or against the resolution under consideration. The switches of each member are inputs to a complex CMOS gate associated with that member. The gate produces a red signal at the end of the vote if the member *did not* vote according to the rules. Design such a gate with as few transistors as possible.

Problem 5.26. Four people, w , x , y , and z , own a company. Their shares in the company are: w , 40%; x , 30%; y , 20%; z , 10%. A 60% majority of the shares is required to pass a resolution. Around their conference table are mounted four buttons, w , x , y , and z . Each person presses his button to vote in favor of, or releases it to oppose, the resolution under consideration. Design a complex CMOS gate whose output gives a signal whenever a resolution is passed.

Problem 5.27. For $f = w'x' + w'v'z' + v'x'y' + y'z'$, derive a static CMOS complex gate that has a total of only 10 transistors.

Hint: Both nMOS and pMOS networks would need to be nonseries–parallel.

6

Multi-level logic synthesis

In Chapter 4, we discussed techniques for obtaining minimal two-level AND–OR or OR–AND realizations. In the present chapter we generalize the discussion to the synthesis of multi-level realizations, i.e., those that contain more than two levels of logic gates. Such realizations are important since they often require less area and delay compared to the corresponding two-level realizations and hence are more practical. However, unlike two-level realizations, it is difficult to obtain provably optimal multi-level realizations because of the much larger design space available for exploration. Thus, the goal of multi-level logic synthesis is to obtain the best possible realization that targets some design objective such as area reduction while meeting some design constraint such as circuit delay.

There are two phases in multi-level logic synthesis; these are the technology-independent and technology-dependent phases. In the technology-independent phase, the circuit is improved for the targeted design criterion, using the laws of Boolean algebra. In the technology-dependent phase, the resultant circuit is mapped to a library of gates available for the given semiconductor technology. We shall discuss the techniques involved in both phases.

6.1 Technology-independent synthesis

Technology-independent multi-level logic synthesis is carried out with the help of various *logic transformations* that preserve the input–output behavior of the circuit. The most important transformations are factoring, decomposition, extraction, substitution, and elimination. We discuss these transformations next.

Introduction to logic transformations

We begin the discussion of logic transformations with the concept of factoring.

Factoring

In factoring, an expression in sum-of-products form is converted into an expression with multiple levels without introducing any subfunctions.

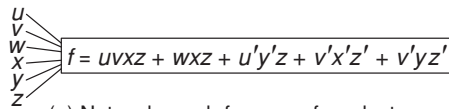
Example Consider the following sum-of-products expression:

$$f = uvxz + wxz + u'y'z + v'x'z' + v'y'z'. \quad (6.1)$$

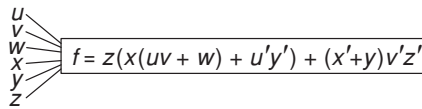
One way to factor it is shown below:

$$f = z(x(uv + w) + u'y') + (x' + y)v'z'. \quad (6.2)$$

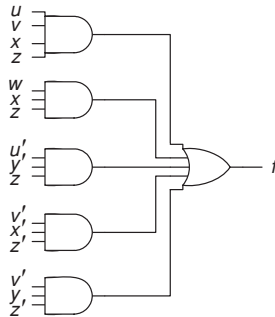
These expressions can be represented by network graphs, as shown in Fig. 6.1a, b. The corresponding two-level and multi-level circuits are shown in Fig. 6.1c, d. As can be seen, the multi-level circuit has six levels of logic.



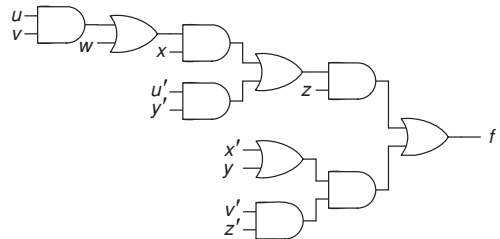
(a) Network graph for sum of products.



(b) Network graph for factored expression.



(c) Two-level circuit.



(d) Multi-level circuit.

Fig. 6.1 Network graphs and corresponding circuits.

The expression in Eq. (6.2) is said to be in factored form, which is a common way to represent a multi-level circuit. A *factored form* is a recursive sum-of-products representation in which the products themselves can consist of a sum of products.

A factored form generally makes the expression more compact. For example, the minimal sum-of-products expression shown in Eq. (6.1) has 16 literals

whereas its factored form has only 11 literals. Since the number of transistors in the complex CMOS-gate implementation of an expression is twice its literal-count (see Section 5.6), the literal-count is a good measure of the implementation complexity.

Decomposition

In decomposition, a factored switching expression is replaced with a set of new expressions.

Example Consider the factored expression in Eq. (6.2). It can be decomposed as follows:

$$\begin{aligned} f_1 &= uv + w, & f_2 &= x' + y, \\ f_3 &= v'z', & f_4 &= xf_1 + u'y', \\ f &= f_2f_3 + zf_4. \end{aligned}$$

The decomposition is depicted by the network graph in Fig. 6.2. One can see that decomposition replaces a network graph node by a set of smaller nodes. Since the functions f_1 , f_2 , f_3 , f_4 , and f are assumed to be separately implemented, the literal-count after the decomposition is the sum of the literal-counts for each function. Thus, the literal-count is now 15.

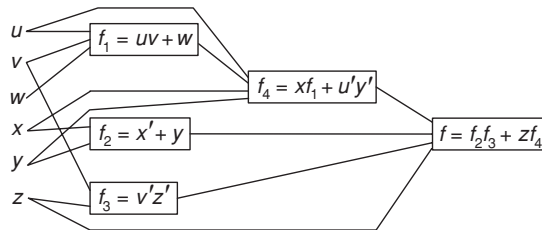


Fig. 6.2 Network graph after decomposition.

Extraction

Extraction is the process of extracting common subexpressions from two or more expressions in factored form.

Example Consider the expressions for f_1 and f_2 below:

$$\begin{aligned} f_1 &= (uv + w)x + u'y', \\ f_2 &= (uv + w)z. \end{aligned}$$

After extracting the subexpression $uv + w$ from the two expressions, we get the following expressions:

$$\begin{aligned} f_1 &= f_3x + u'y', & f_2 &= f_3z, \\ f_3 &= uv + w. \end{aligned}$$

Thus, the literal-count reduces from 10 to 9. The network graphs are shown in Fig. 6.3.

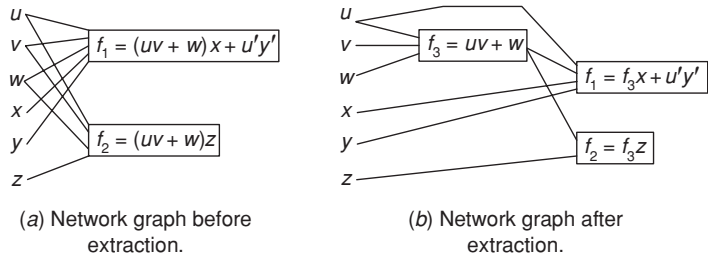


Fig. 6.3 Network graphs depicting extraction.

Substitution

Substitution is the process of replacing a subexpression in an expression f with a variable g corresponding to a node in a network graph. In other words, g is substituted into f or f is expressed in terms of g .

Example Consider f_1 and f_2 below:

$$f_1 = uvx + wx + u'y',$$

$$f_2 = uv + w.$$

The expression f_1 can be given in terms of f_2 as $f_2x + u'y'$. Thus, f_2 has been substituted into f_1 . Figure 6.4 shows the network graphs for this example.

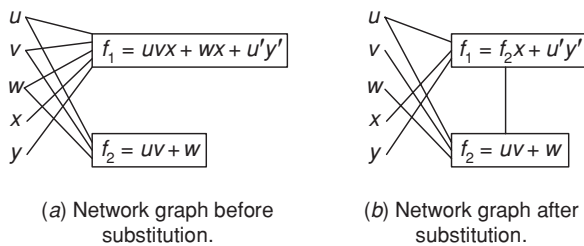


Fig. 6.4 Network graphs depicting substitution.

Elimination

Elimination is the process of removing an internal node from the network graph; it becomes possible if the corresponding expression replaces the variable corresponding to that node. Whenever the elimination step reduces the literal-count, it may be useful to employ it.

Example Consider $f_1 = x + f_2$ and $f_2 = y + z$. If f_2 is not needed elsewhere in the network then it can be eliminated in the expression for f_1 by replacing it with $y + z$, thus obtaining $f_1 = x + y + z$. This reduces the literal-count from four to three.

In multi-level logic synthesis, the above five logic transformations are applied to an initial logic network iteratively (they need not be applied in the given order) until no more improvement is possible in the targeted objective. Examples of synthesis objectives are optimization of area, delay, or power consumption.

Factoring

We next explore the different techniques employed in the factoring step.

There are two kinds of switching expression: algebraic and Boolean. In an *algebraic expression*, no implicant of the expression contains another implicant. An expression that does not satisfy this condition is a *Boolean expression*. For example, $x + xy$ is not an algebraic expression whereas $x + yz$ is.

Operations on algebraic expressions are simpler – they can be treated similarly to the multiplication and division of polynomials. However, this prevents the full exploitation of all the laws of Boolean algebra. For example, idempotency, the dual of distributivity (i.e., $x + yz = (x + y)(x + z)$), and absorption cannot be used to manipulate algebraic expressions because they do not have an analog in conventional polynomial algebra. Similarly, complementation (i.e., $x + x' = 1$ and $xx' = 0$), involution, and De Morgan's theorem cannot be used since complements are not defined in polynomial algebra. Thus, complemented literals are deemed to be unrelated to uncomplemented literals. All laws of Boolean algebra are applicable to Boolean expressions.

A factored form is called *algebraic* if multiplication of its terms yields an algebraic sum-of-products expression without the application of the above-mentioned laws, else it is called *Boolean*.

Example The factored form $(w + x)(y + z)$ is algebraic since multiplying out its factors yields the sum-of-products expression $wy + wz + xy + xz$, which is algebraic. However, $(w + yz)(x + yz)$ is not an algebraic factored form but a Boolean factored form, since multiplying out its factors yields $wx + wyz + xyz + yzyz$, which is not an algebraic expression. Here, $yzyz$ cannot be simplified to yz because the use of idempotency is not allowed, neither can it absorb xyz since the absorption law cannot be used. Similarly, $(x + y)(x' + z)$ is not algebraic since multiplying out its terms yields the term xx' which cannot be simplified further.

Division operation

We next look at the division operation, which is a key operation in multi-level logic synthesis. Given the expressions f and f_d , if f can be expressed as $f = f_d f_q + f_r$ then this is said to be a *division operation* where f_d is the *divisor*, f_q the *quotient*, and f_r the *remainder*. If f_d and f_q have no variables in common then it is said to be an *algebraic division* operation; otherwise it is said to be a *Boolean division* operation. Correspondingly, f_d is either an *algebraic divisor* or a *Boolean divisor*. If $f_r = 0$ then f_d is correspondingly either an *algebraic factor* or a *Boolean factor*.

Example Let $f_1 = vx + vy + wx + wy + z$. Since it has a factored form $(v + w)(x + y) + z$, $(v + w)$ is an algebraic divisor with quotient $(x + y)$ and remainder z .

Consider a slightly different expression, $f_2 = vx + vy + wx + wy = (v + w)(x + y)$. In this case $(v + w)$ is an algebraic factor of f_2 (so, of course, is $(x + y)$).

Next, consider $f_3 = w + xy + z = (w + x)(w + y) + z$. Here, $(w + x)$ is a Boolean divisor of f_3 , not an algebraic divisor, since $(w + x)$ and $(w + y)$ have w in common.

Similarly, for $f_4 = w + xy = (w + x)(w + y)$, $(w + x)$ is a Boolean factor, not an algebraic factor.

Given an expression, there may be more than one way to factor it. For example, $f_5 = xy + xz + yz$ can be factored as $x(y + z) + yz$ or $(x + y)z + xy$. For the first factored form, $(y + z)$ is an algebraic divisor, and for the second factored form, $(x + y)$ is an algebraic divisor.

Algebraic kernels and co-kernels

The concept of kernels and co-kernels helps determine the common subexpressions that can be extracted from switching expressions. In this section, we shall use this concept to factor a single expression. Later, when we discuss extraction, we shall use it to extract subexpressions from two or more expressions.

If an expression cannot be factored by a cube (see Section 4.2), it is said to be *cube-free*. For example, $wx + yz$ is cube-free. However, $xy + xz$ is not cube-free since it can be factored by x . Similarly, xyz is not cube-free since it can be factored by any combination of its literals. Thus, for an expression to be cube-free, it must contain more than one cube.

If, when an expression is divided by a cube, the result is a cube-free quotient then the quotient is called a *kernel* and the cube the corresponding *co-kernel*. If a kernel has no kernel except itself, it is called a *level-0 kernel*. If a kernel has at least one kernel of level $n - 1$ but no kernel of level n or greater except itself, it is called a *level- n kernel*. A co-kernel has the same level as its kernel.

Example Consider the expression $f = uwz + uxz + vwz + vxz + yz + uv$. Its kernels and co-kernels and their levels are shown in Table 6.1. When f is divided by the cube wz , we get $f = (u + v)wz + uxz + vxz + yz + uv$. Thus, its kernel is $u + v$ and wz its co-kernel. Since $u + v$ does not have any kernel but itself, it is a level-0 kernel. When f is divided by u , we get $f = (wz + xz + v)u + vwz + vxz + yz$. Thus, $wz + xz + v$ is its kernel and u its co-kernel. Since $wz + xz + v$ can be factored as $(w + x)z + v$ and $w + x$ is a level-0 kernel, $wz + xz + v$ is a level-1 kernel. If we divide f by w , we obtain the quotient $uz + vz$, which is not cube-free. Thus, w is not a co-kernel. Dividing f by y leads to the quotient z , which is not cube-free. Thus, y is not a co-kernel. However, f is itself cube-free. Thus, it is its own kernel with a co-kernel 1. It has level 2 because it has level-1 kernels.

Table 6.1 Kernels and their co-kernels

Level	Kernel	Co-kernel
0	$u + v$	wz, xz
0	$w + x$	uz, vz
1	$wz + xz + v$	u
1	$wz + xz + u$	v
1	$uw + ux + vw + vx + y$	z
2	$uwz + uxz + vwz + vxz + yz + uv$	1

Rectangle covering

We discuss next a method for computing kernels and co-kernels called *rectangle covering*.

Consider a sum-of-products expression f with p cubes and q distinct literals. A $p \times q$ *cube-literal incidence matrix* can be defined for f in which element (i, j) is 1 if the j th literal is used in the i th cube, and 0 otherwise. A *rectangle* of this matrix denotes a set of rows and columns in which all entries are 1. Let (r, c) denote the row and column subsets of the rectangle. A rectangle (r_1, c_1) is said to *contain* another rectangle (r_2, c_2) if $r_1 \supseteq r_2$ and $c_1 \supseteq c_2$. A rectangle is called *prime* if it is not strictly contained in another rectangle. The co-rectangle of rectangle (r, c) is denoted as $(r, \bar{c})$ where $\bar{c}$ is the complement of the column subset c , i.e., it includes all columns of the matrix not in c .

Example Consider $f = uwz + uxz + yz + uv$. It has four cubes and six distinct literals. Its cube-literal incidence matrix is shown in Table 6.2. $(\{uwz, uxz\}, \{u, z\})$ is a prime rectangle whose co-rectangle is $(\{uwz, uxz\}, \{v, w, x, y\})$. Two other prime rectangles are: $(\{uwz, uxz, uv\}, \{u\})$ and $(\{uwz, uxz, yz\}, \{z\})$.

Table 6.2 Cube–literal incidence matrix for f

Cube	Literal					
	u	v	w	x	y	z
uwz	1	0	1	0	0	1
uxz	1	0	0	1	0	1
yz	0	0	0	0	1	1
uv	1	1	0	0	0	0

A co-kernel of an expression can be derived from a prime rectangle (r, c) that contains at least two rows. Its co-rectangle $(r, \bar{c})$ yields the corresponding kernel, which can be derived as the sum of the cubes in r restricted to the literals in $\bar{c}$.

Example We now continue with the previous example. The prime rectangle $(\{uwz, uxz\}, \{u, z\})$ yields co-kernel uz . Its co-rectangle $(\{uwz, uxz\}, \{v, w, x, y\})$ yields the kernel $w + x$, which is obtained by restricting $uwz + uxz$ to literals in $\{v, w, x, y\}$.

A factoring approach

One factoring approach is to start with a sum-of-products expression and derive a factored form. A possible objective might be to reduce the literal-count of the logic network.

Suppose that a sum-of-products expression f has been given as $f = f_d f_q + f_r$, where f_d is the divisor, f_q the quotient, and f_r the remainder. If the division is algebraic, f_d could be the kernel and f_q the co-kernel. A straightforward approach is to factor f_d , f_q , and f_r recursively until their forms cannot be factored any further. It is possible that, at some stage in the factoring process, the quotient and part of the remainder may have a common subexpression that can be extracted. This is illustrated using the following example. The process of extraction is discussed in more detail in the next subsection.

Example Consider the expression $f = uwz + uxz + vwz + vxz + yz + uv$ once again. Dividing by the kernel $(u + v)$ gives the factored form

$$f = (u + v)(wz + xz) + yz + uv,$$

where $f_d = u + v$, $f_q = wz + xz$, and $f_r = yz + uv$. Here, f_d and f_r cannot be factored any further. However, f_q can be, giving the following factored form at this point:

$$f = (u + v)(w + x)z + yz + uv.$$

Although recursive factoring has been taken as far as it can, we can see that in fact f can be factored further by extracting z from $f_q = (w + x)z$ and yz , which is part of f_r , as follows:

$$f = ((u + v)(w + x) + y)z + uv.$$

This is the final factored form. It reduces the literal count from 16 in the original expression to just 8.

Of course, the above factoring approach is not limited to algebraic factors. Boolean factors can also be used at each step. However, for full-fledged multi-level Boolean optimization, the concepts of the satisfiability don't-care set and the observability don't-care set are useful. We introduce these concepts in Chapter 8 in the context of redundant logic removal.

Extraction

If two or more expressions have common divisors, the divisors can be extracted. The rectangle-covering method, which was used for factoring earlier, can be extended to perform extraction as well.

There are two types of extraction methods: cube extraction and kernel extraction. As the name implies, *cube extraction* refers to the extraction of a cube and *kernel extraction* that of a kernel from two or more expressions. We discuss cube extraction first, then kernel extraction.

To perform cube extraction, the rectangle-covering method requires a minor extension. First, an auxiliary expression f_a is formed as the sum of all the expressions in the logic network. Then a cube–literal incidence matrix is obtained for f_a . Each cube of each expression is tagged with an identifier for that expression. The rest of the approach is the same as before, i.e., it is based on finding a prime rectangle.

Example Suppose the network has two expressions, $f_1 = uwz + uxz + yz + uv$ and $f_2 = vz + wyz$. The auxiliary function $f_a = f_1 + f_2 = uwz + uxz + yz + uv + vz + wyz$. Its cube–literal incidence matrix is shown in Table 6.3. The prime rectangle $(\{yz, wyz\}, \{y, z\})$ has a corresponding cube yz . Thus yz can be extracted from the two expressions, as shown in Fig. 6.5. However, since the literal-count remains at 15 after the extraction, this may not be an attractive step to carry out in logic synthesis.

Note that even though f_a includes the term yz , which absorbs the term wyz , f_a should not be simplified since yz and wyz belong to two different expressions, f_1 and f_2 , in the original logic network.

Table 6.3 Cube–literal incidence matrix for $f_a = f_1 + f_2$. “Id” identifies the expression to which a cube belongs

Cube	Id	Literal					
		u	v	w	x	y	z
uwz	f_1	1	0	1	0	0	1
uxz	f_1	1	0	0	1	0	1
yz	f_1	0	0	0	0	1	1
uv	f_1	1	1	0	0	0	0
vz	f_2	0	1	0	0	0	1
wyz	f_2	0	0	1	0	1	1

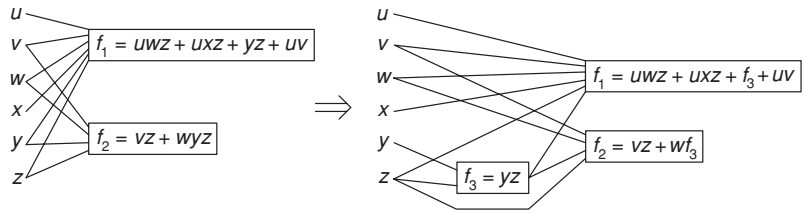


Fig. 6.5 Cube extraction.

To perform kernel extraction, a kernel–cube incidence matrix is defined analogously to the cube–literal incidence matrix. To derive such a matrix, we first represent each cube in a kernel with a new variable and the kernel by a set of such variables. The set of kernels for expression f_i is denoted by $K(f_i)$.

Example Consider the two expressions $f_1 = uwz + uxz + yz$ and $f_2 = vw + vx + vyz$. From their cube–literal incidence matrices we can obtain $K(f_1) = \{(w + x), (uw + ux + y)\}$ and $K(f_2) = \{(w + x + yz)\}$. Let us represent the cubes in these kernels by new variables as follows: we set $a_w = w$, $a_x = x$, $a_y = y$, $a_{uw} = uw$, $a_{ux} = ux$, and $a_{yz} = yz$. The sets of kernels can now be represented in terms of these variables by $K(f_1) = \{\{a_w, a_x\}, \{a_{uw}, a_{ux}, a_y\}\}$ and $K(f_2) = \{\{a_w, a_x, a_{yz}\}\}$.

We next form an auxiliary function f_a as a sum of cubes, where a cube is the product of the new variables corresponding to a kernel for all the expressions under consideration. For the above example, $f_a = a_w a_x + a_{uw} a_{ux} a_y + a_w a_x a_{yz}$.

The row headings in the *kernel–cube incidence matrix* denote the cubes representing the kernels and the column headings denote the new variables. Element (i, j) of this matrix is 1 if the j th new variable is used in the i th cube, and 0 otherwise. A prime rectangle in such a matrix corresponds to a kernel intersection. If the rows of such a rectangle correspond to different

expressions then the kernel intersection corresponds to the subexpression that can be extracted from these expressions.

Example The kernel–cube incidence matrix for the above example is shown in Table 6.4. The first column lists all the kernels in $K(f_1)$ and $K(f_2)$. The second column shows the cube representations corresponding to these kernels. The third column identifies the expression to which the kernel belongs. We can see that a prime rectangle is $(\{a_w a_x, a_w a_x a_{yz}\}, \{a_w, a_x\})$. This corresponds to the kernel intersection $w + x$. Since the two rows of the rectangle correspond to two different expressions this kernel intersection can be extracted from them, as shown in Fig. 6.6. This kernel extraction reduces the literal-count from 15 to 12. Of course, f_1 and f_2 can be factored further in the next synthesis step to reduce the literal-count to 10.

Table 6.4 Kernel-cube incidence matrix for f_a

Kernel	Representation	Id	Literals corresponding to cubes						
			a_w	a_x	a_y	a_{uw}	a_{ux}	a_{yz}	
$w + x$	$a_w a_x$	f_1	1	1	0	0	0	0	
$uw + ux + y$	$a_{uw} a_{ux} a_y$	f_1	0	0	1	1	1	0	
$w + x + yz$	$a_w a_x a_{yz}$	f_2	1	1	0	0	0	1	

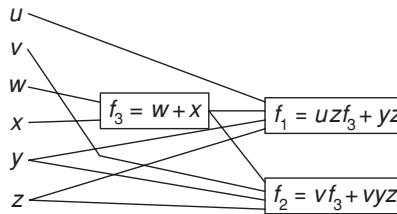


Fig. 6.6 Kernel extraction.

The above example shows that use of new variables makes it possible to treat and manipulate the kernel–cube incidence matrix in the same fashion as a cube–literal incidence matrix.

Decomposition and substitution

The decomposition step helps to reduce a complex expression to a manageable size that can be implemented with standard logic cells. Assuming algebraic division, let us express f as $f_d f_q + f_r$ as before. Decomposition represents the divisor f_d by a variable a , reducing f to $a f_q + f_r$ where $a = f_d$. Just like factoring, the decomposition process can then be carried out recursively on the divisor, quotient, and remainder.

Example Consider the expression $f = xz + yz + wx + wy + vw$. Let the divisor be $x + y$. Using the variable a to represent the divisor gives

$$f = aw + az + vw,$$

$$a = x + y.$$

Next, decomposing the quotient gives

$$f = ab + vw,$$

$$a = x + y,$$

$$b = w + z.$$

The above steps are shown in Fig. 6.7.

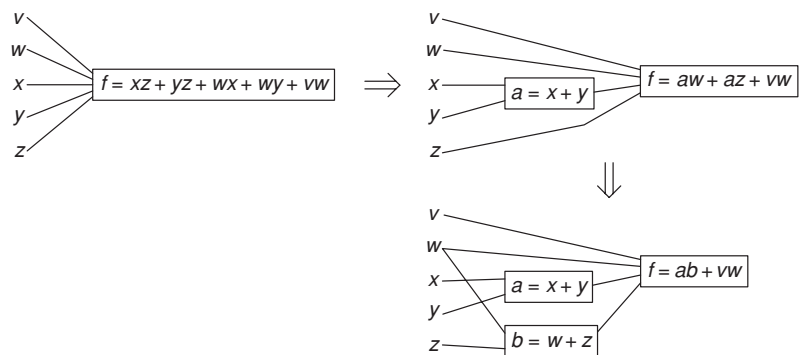


Fig. 6.7 Decomposition.

The end product of decomposition obviously depends on the choice of the divisor. All kernels can be evaluated for this purpose and the one that gives the greatest literal-count reduction chosen. However, this is time consuming. A faster alternative is to consider only level-0 kernels.

As noted earlier, the process of replacing the divisor by the corresponding variable is substitution. In the above example, the divisor $x + y$ has been replaced by the variable a , which has been substituted into f . Thus decomposition and substitution go hand in hand. If a divisor of expression f is also a divisor of expression g then its corresponding variable can be substituted into both f and g .

6.2 Technology mapping

After technology-independent logic synthesis, the circuit components need to be mapped to a set of logic cells constituting the *cell library* that can be implemented in the targeted technology. This process is called *technology mapping*. The area and propagation delay of the logic cells are provided in the

cell library. The objective of technology mapping may be to minimize circuit area or delay, or to minimize area (delay) under delay (area) constraints.

Example Consider the circuit in Fig. 6.8a. Suppose that the cell library has only an inverter, a two-input NAND gate, and a three-input NAND gate, with area costs 1, 2, and 3, respectively. We first implement the circuit with only inverters and two-input NAND gates, as shown in Fig. 6.8b. A trivial technology mapping for this circuit is shown in Fig. 6.8c. The area cost is 9. However, we can take advantage of the three-input NAND gate available in the cell library and obtain the alternative technology mapping shown in Fig. 6.8d. The area cost is now only 7. Thus, the aim of technology mapping is to take full advantage of the cell library and obtain a minimum-cost solution.

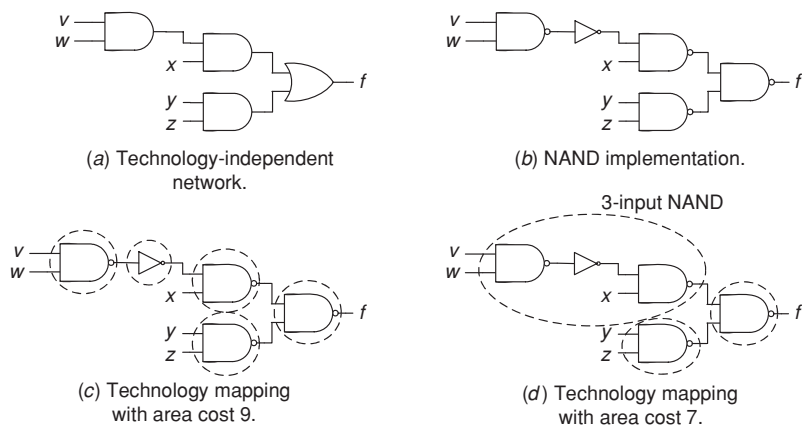


Fig. 6.8 Technology mapping example.

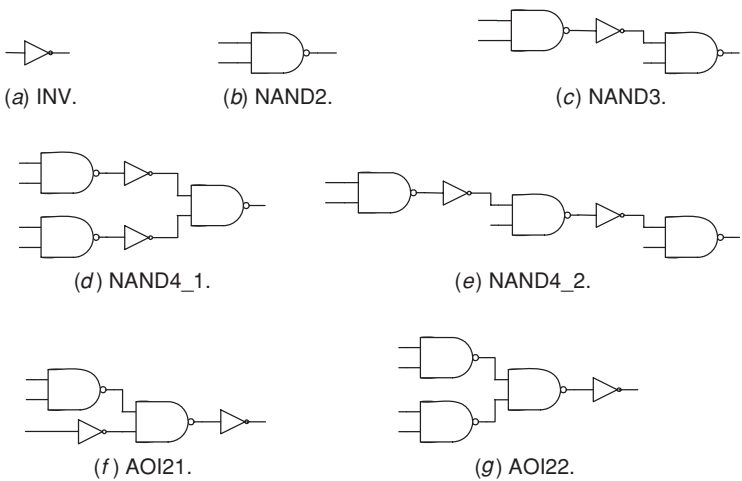
The above example demonstrates a popular approach to technology mapping called *network covering*. *Network covering* refers to the process of replacing subnetworks of the technology-independent logic network with cells from the cell library such that the whole network is covered and the desired objective is met. A cell is said to *match* a subnetwork if they are functionally equivalent.

The technology-independent logic network is first converted into a graph in which each node is derived from a set of *base functions*; for example, a possible set of base functions may consist of an inverter and a two-input NAND gate. Such a graph is called the *subject graph*. In the above example, the original logic network shown in Fig. 6.8a was converted into the subject graph shown in Fig. 6.8b. Similarly, each cell in the cell library is also represented by a graph (or a set of graphs, as we shall see later) in which each node is derived from the set of base functions. Such a graph is called the *pattern graph*. For the above example, we chose a cell library consisting of an inverter, a two-input

Table 6.5 Area and delay costs of the pattern graphs in Fig. 6.9

Pattern graph	Area cost	Delay cost
INV	1	1
NAND2	2	2
NAND3	3	3
NAND4	4	4
AOI21	3	6
AOI22	4	8

Fig. 6.9 Pattern graphs.



NAND gate, and a three-input NAND gate. The corresponding pattern graphs are shown in Figs. 6.9*a, b, c*; the pattern graphs are labeled INV, NAND2, and NAND3, respectively. A cell may have more than one pattern graph. Two decompositions of a four-input NAND cell, labeled NAND4_1 and NAND4_2, into the inverter and two-input NAND base functions are shown in Figs. 6.9*d, e*, respectively. Another common cell is the AND-OR-INVERT (AOI) cell. Figures 6.9*f, g* give the pattern graphs for two versions of an AOI cell, AOI21 and AOI22. The numbers in the labels denote the numbers of inputs of the gates in the first logic level of the cell. Thus, an AOI21 cell implements an expression of the type $(xy + z)'$ and an AOI22 cell implements an expression of the type $(wx + yz)'$.

Typically, area and delay costs are associated with each pattern graph. Table 6.5 presents one such set of costs. Here, we have assumed that the area cost is equal to the number of transistors in the nMOS or pMOS network of the corresponding primitive- or complex-gate CMOS implementation. The delay cost depicts the relative propagation delays through these gates.

A *network cover* refers to an ensemble of pattern graphs with minimum cost that collectively matches every node in the subject graph. Of course, the input of a pattern graph must be the output of another pattern graph or a primary input. Note that a node in the subject graph may be covered by more than one pattern graph. That is why the relevant term is “network cover,” not “network partition.” If area optimization is the objective then the cost to be minimized is the sum of the areas of the ensemble of pattern graphs chosen. If delay optimization is the objective then the critical path delay through the network cover is the cost that needs to be minimized.

Next, we discuss the various steps in technology mapping: decomposition into base functions, partitioning networks into subject graphs, obtaining matches, and obtaining the network cover.

Decomposing a network into base functions

To ensure that any arbitrary network can be decomposed into a set of base functions, obviously the set must be functionally complete. Thus, it could consist of an inverter, a two-input OR, and a two-input AND. The base functions must be supported by the cell library. In this case, the cell library would include INV, OR2 (which implements a two-input OR), and AND2 (which implements a two-input AND). Other possible sets of base functions include: an inverter and a two-input NAND supported by a cell library that includes {INV, NAND2}; an inverter and two-input NOR gate supported by a cell library that includes {INV, NOR2}. Note that even though an inverter can be obtained from a NAND or NOR gate by shorting its inputs, explicitly including an inverter in the set of base functions leads to lower cost. An inverter and two-input NAND constitutes a popular set of base functions since it simplifies the work needed in the subsequent steps. With the above choice of base functions, a trivial network cover always exists in which each node in the subject graph is mapped to the cell that implements that base function.

Partitioning a network into subject graphs

Typically, the technology-independent logic network has multiple inputs and outputs. Let us suppose that the network has been decomposed into the chosen set of base functions. If this network is treated as the starting point then the subsequent technology-mapping steps become quite cumbersome. Thus, usually the network is partitioned into a set of connected subject graphs and each such subject graph is then subjected to the matching and network-covering steps. A popular way of partitioning the network is in terms of subnetworks called leaf-directed-acyclic graphs (leaf-DAGs). A *leaf-DAG* does not have any internal fanouts. Thus, all fanout points in the decomposed network can be marked. Such fanout points form the boundaries of a partition; each partitioned subnetwork forms a subject graph.

Example Consider the network graph shown in Fig. 6.3b. Its technology-independent implementation is shown in Fig. 6.10a. Assuming an inverter and two-input NAND as base functions, the decomposed version of the implementation is shown in Fig. 6.10b. The three subject graphs, s_1 , s_2 , and s_3 , are also shown. Note that the fanout point at f_3 forms a natural boundary between s_1 and s_2 as well as between s_1 and s_3 .

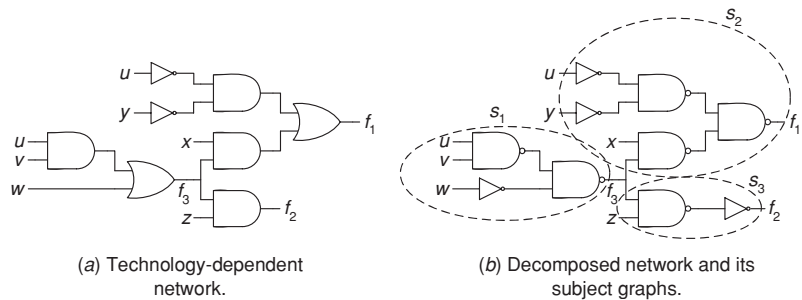


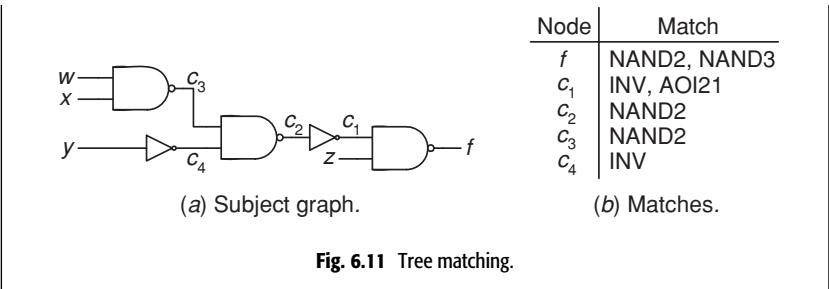
Fig. 6.10 Partitioning the network into subject graphs.

The technology mapping for each subject graph is done separately and the solutions for the subject graphs are connected together to get the technology mapping for the original decomposed network.

Obtaining matches

The third step in technology mapping is to obtain all possible ways in which pattern graphs can match each node in the subject graph. When all the pattern graphs are *trees*, i.e., they do not have a fanout even at their primary inputs, this step is called *tree matching*. For example, all the pattern graphs shown in Fig. 6.9 are trees.

Example Consider the subject graph shown in Fig. 6.11a. Suppose that the cell library has cells whose pattern graphs are those shown in Fig. 6.9. The various matches obtained at the different nodes of the subject graph are shown in Fig. 6.11b. We start at the output and find matches as we go towards the inputs. At node f both NAND2 and NAND3 are matches. Similarly, at node c_1 both INV and AOI21 are matches. However, at nodes c_2 , c_3 , and c_4 , only one match is found. Since the base functions (inverter and two-input NAND) are available as cells, at least one match is guaranteed to be found at each node of the subject graph.



From the above example, we can see that that tree matching entails finding whether a pattern graph is isomorphic to a subgraph of the subject graph. Since the base functions used in the subject graph are an inverter and two-input NAND, each gate in this graph can have either one input or two inputs. To find the pattern graphs that match a particular node in the subject graph, the output of the pattern graph can be matched with this node and the number of inputs of the corresponding nodes can be recursively checked to see if they are equal. If they are not then there is no match for the node in question with that pattern graph. If, in the above process, the primary inputs of the pattern graph have been reached then a match has been found. If the primary inputs of the subject graph have been reached but not the primary inputs of the pattern graph then there is no match.

Obtaining the network cover

In the final step in technology mapping, for each node in the subject graph a possible match needs to be chosen to obtain a network cover such that some given cost, such as area or delay, is minimized. An optimum method for deriving this cover is dynamic programming. This method traverses the subject graph from the primary inputs towards the output and chooses the best match for each node. This is illustrated by the following example.

Example Consider the subject graph in Fig. 6.11a. Suppose that the optimum area cover needs to be obtained from among the matches shown in Fig. 6.11b. The area cost for the pattern graphs is taken from Table 6.5. Table 6.6 shows how the cover is obtained. Traversing forward from the primary inputs, the first set of nodes encountered includes c_3 and c_4 , for each of which there is only one match, NAND2 and INV, respectively. In the second column of Table 6.6, the inputs of these pattern graphs are also shown. The cost of the cover is obtained from the sum of the area cost of the match and the optimum cost of the input nodes of the match. Thus, at c_3 and c_4 the cost of the cover is just the area cost of the corresponding pattern graphs. When we move forward to node c_2 , we again have a single match,

NAND2. The cost of the cover at c_2 is $2 + 1 + 2 = 5$. This includes the cost of all the pattern graphs chosen so far. When we reach c_1 , there are two matches. If INV is chosen then the cost of the cover is $5 + 1 = 6$. However, if AOI21 is chosen then the cost of the cover is simply 3, the area cost of AOI21. The reason is that the inputs of AOI21 are the primary inputs of the subject graph, w , x , and y , and thus covers for the nodes c_3 , c_4 , and c_2 are no longer required. Hence the best cost of the cover obtained at c_1 is 3. Moving forward to f , we again have two matches, NAND2 and NAND3. If NAND2 is chosen then the cost of the cover is $3 + 2 = 5$. However, if NAND3 is chosen then the covers for nodes c_2 and c_1 are no longer required. Thus, the cost of the cover is the sum of those for nodes c_3 , c_4 , and f , i.e., $2 + 1 + 3 = 6$. Thus, at output f the optimum cost of the cover is 5. This cover is shown in Fig. 6.12.

Table 6.6 Covering of the subject graph

Node	Match	Cost of cover
c_3	NAND2(w, x)	2
c_4	INV(y)	1
c_2	NAND2(c_3, c_4)	5
c_1	INV(c_2)	6
	AOI21(w, x, y)	3
f	NAND2(c_1, z)	5
	NAND3(c_3, c_4, z)	6

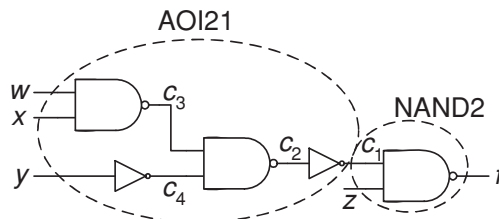


Fig. 6.12 Area cover of the subject graph.

Next, suppose the optimum delay cover needs to be obtained. In this case, the above process can simply be repeated by taking the delay cost of each match into account, instead of the area cost, as illustrated next.

Example Consider the subject graph in Fig. 6.11a once again. For the sake of simplicity, suppose that its primary input values arrive at time 0. The delay cost for each pattern graph is shown in Table 6.5. Again, the best set of matches needs to be selected from those shown in Fig. 6.11b. The different steps are shown in Table 6.7. The delays at nodes c_3 and c_4 are 2 and 1, respectively. In general, the delay at the output of a pattern graph is

the maximum delay at its inputs plus the delay of the pattern graph itself. Thus, the delay at c_2 is $\max(1, 2) + 2 = 4$. At c_1 there are two matches with delays of 5 and 6. Choosing the better of these two, i.e., $\text{INV}(c_2)$, implies the choice of $\text{NAND2}(c_1, z)$ for node f with a delay of $\max(5, 0) + 2 = 7$. However, there is another match at node f , $\text{NAND3}(c_3, c_4, z)$. With this match the delay at f is $\max(2, 1, 0) + 3 = 5$. This is therefore a better match at f . The cover thus obtained is shown in Fig. 6.13.

Table 6.7 Obtaining the optimum delay cover

Node	Match	Cost of cover
c_3	$\text{NAND2}(w, x)$	2
c_4	$\text{INV}(y)$	1
c_2	$\text{NAND2}(c_3, c_4)$	4
c_1	$\text{INV}(c_2)$	5
	$\text{AOI21}(w, x, y)$	6
f	$\text{NAND2}(c_1, z)$	7
	$\text{NAND3}(c_3, c_4, z)$	5

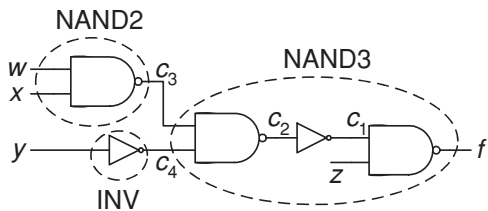


Fig. 6.13 Delay cover of the subject graph.

Notes and references

The material covered in this chapter is addressed in much greater detail in [5, 7]. The concept of kernels is presented in [1]. Rectangle covering and other methods for deriving kernels and co-kernels are given in [2, 4]. Various multi-level logic synthesis steps are discussed in [3]. Network covering is described in [8, 9]. The approach presented in this chapter is derived from the technology mapper presented in [8]. The matching problem is discussed in [6, 10]. In [12], a set of base functions including a two-input NAND and inverter is used. In [11, 13], more comprehensive methods of obtaining delay-oriented technology mapping are given; these include the impact of interconnects on delay. In this chapter, we have presented a technology-mapping technique based on tree matching. However, directed acyclic graphs (DAGs) can be tackled directly using an exact algorithm called binate covering, although this is practical only for small networks. This approach is also discussed in [11].

- [1] Brayton, R. K., and C. McMullen: "The decomposition and factorization of Boolean expressions," in *Proc. IEEE Int. Symp. Circuits & Systems*, pp. 49–54, May 1982.
- [2] Brayton, R. K., R. Rudell, A. L. Sangiovanni-Vincentelli, and A. R. Wang: "Multi-level logic optimization and the rectangular covering problem," in *Proc. IEEE Int. Conf. Computer-Aided Design*, pp. 66–69, November 1987.
- [3] Brayton, R. K., R. Rudell, A. L. Sangiovanni-Vincentelli, and A. R. Wang: "A multi-level logic optimization system," *IEEE Trans. Computer-Aided Design*, vol. CAD-6, no. 6, pp. 1062–1081, November 1987.
- [4] Brayton, R. K., G. D. Hachtel, and A. L. Sangiovanni-Vincentelli: "Multilevel logic synthesis," *Proc. IEEE*, vol. 78, no. 2, pp. 264–300, February 1990.
- [5] De Micheli, G.: *Synthesis and Optimization of Digital Circuits*, McGraw-Hill, New York, 1994.
- [6] Detjens, E., G. Gannot, R. Rudell, A. L. Sangiovanni-Vincentelli, and A. R. Wang: "Technology mapping in MIS," in *Proc. IEEE Int. Conf. Computer-Aided Design*, pp. 116–119, November 1987.
- [7] Hachtel, G. D., and F. Somenzi: *Logic Synthesis and Verification Algorithms*, Kluwer Academic, Boston MA, 1998.
- [8] Keutzer, K.: "DAGON: technology optimization and local optimization by DAG matching," in *Proc. IEEE Design Automation Conf.*, pp. 341–347, June 1987.
- [9] Mailhot, F., and G. De Micheli: "Technology mapping with Boolean matching," *IEEE Trans. Computer-Aided Design*, vol. CAD-12, no. 5, pp. 599–620, May 1993.
- [10] Morrison, C. R., R. M. Jacoby, and G. D. Hachtel: "TECHMAP: technology mapping with delay and area optimization," in G. Saucier and P. M. McLellan (eds.), *Logic and Architecture Synthesis for Silicon Compilers*, pp. 53–64, North-Holland, Amsterdam, Holland, 1989.
- [11] Rudell, R.: "Logic synthesis for VLSI design," Ph.D. thesis, Dept of Electrical Engineering and Computer Sciences, University of California, Berkeley, 1989.
- [12] Sentovich, E. M., K. J. Singh, C. Moon, H. Savoj, R. K. Brayton, and A. L. Sangiovanni-Vincentelli: "Sequential circuit design using synthesis and optimization," in *Proc. IEEE Int. Conf. Computer Design*, pp. 328–333, October 1992.
- [13] Touati, H.: "Performance oriented technology mapping," Ph.D. thesis, Dept of Electrical Engineering and Computer Sciences, University of California, Berkeley, 1990.

Problems

Problem 6.1. Determine which of the following factored forms are algebraic:

- (a) $(v + wx)(w + yz)$,
- (b) $x'(y' + z') + yz'$,
- (c) $y'(w'z + wx) + y(w'x + z) + wyz$.

Problem 6.2

- (a) Obtain all the algebraic and Boolean divisors of the expression

$$v + wx + wy + wz.$$

- (b) Which divisor leads to the least number of literals after factorization?

Problem 6.3. For the following expression, find all level-0, level-1, and level-2 kernels:

$$vwy' + vwz + x'y' + x'z + wx.$$

Problem 6.4. For $f = wxz' + uwx + yz' + uwy + v$:

- obtain the cube-literal incidence matrix;
- obtain all prime rectangles and co-rectangles from the matrix;
- obtain the set of kernels and corresponding co-kernels from the prime rectangles and co-rectangles.

Problem 6.5. For the following three expressions

$$f_1 = uwz + uxz + vwz + vxz + yz + uv,$$

$$f_2 = vw + vx + vyz + uz,$$

$$f_3 = xyz,$$

- derive all the kernels from their cube-literal incidence matrices,
- derive the kernel-cube incidence matrix and identify all its prime rectangles,
- perform a kernel extraction based on each prime rectangle and show the network graph for each extraction.

Problem 6.6. For $f(w, x, y, z) = \sum(1, 3, 5, 7, 8, 11, 13, 15)$, find functions G and H for the decomposition $f(w, x, y, z) = G(H(x, y), w, z)$.

Problem 6.7. The function $f(w, x, y, z) = \sum(4, 7, 8, 11, 13, 14, 23, 27, 28, 29, 30)$ can be decomposed to form $G(H(v, y, z), w, x)$. Determine the functions G and H .

Problem 6.8. For each of the following functions, specify the don't-care combinations and determine functions G and H such that the given function is decomposable, as follows:

$$(a) \quad f(v, w, x, y, z) = \sum(4, 8, 10, 16, 21, 27, 28) + \sum_{\phi}(1, 5, 23, 25, 30, 31) \\ = G(H(v, x, z), w, y);$$

$$(b) \quad f(v, w, x, y, z) = \sum(1, 2, 7, 9, 10, 17, 19, 26, 31) + \sum_{\phi}(0, 15, 20, 23, 25) \\ = G(H(v, w, y), x, z).$$

Problem 6.9. A switching function is said to be *symmetric* if and only if it is invariant under any permutation of its variables. For example, $f(x, y, z) = xyz$ is symmetric since permuting x and y , or x and z , or y and z , yields the same function.

- Show that $f(x, y, z) = xyz + xy'z' + x'yz' + x'y'z$ is symmetric.
- Show that there is a decomposition of f with a total of only eight literals.

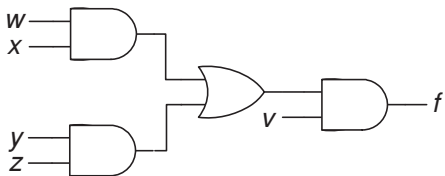
Problem 6.10. Consider a cell library consisting of INV, NAND2, NOR2, AOI21, AOI22, OAI21 (an OR-AND-INVERT cell that implements an expression of the type $[(x + y)z']$), and OAI22 (which implements an expression of the type $[(w + x)(y + z)]'$).

- Assuming an inverter and two-input NOR as base functions, obtain the pattern graphs for all the cells in the library.
- Repeat part (a) assuming an inverter and two-input NAND as base functions.

Problem 6.11

- Decompose the subject graph shown in Fig. P6.11 using an inverter and two-input NAND as base functions.

Fig. P6.11



- (b) Using the library and area costs shown in Table 6.11, find all matches at all nodes of the decomposed subject graph and then obtain the optimum-area network cover.

Table P6.11

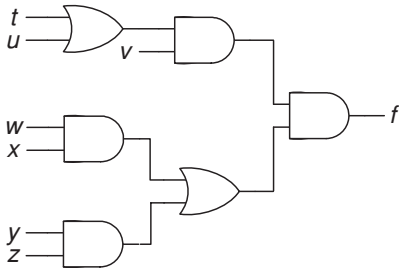
Pattern graph	Area cost	Delay cost
INV	1	1
NAND2	2	2
NOR2	2	3
AOI21	3	6
AOI22	4	8
OAI21	3	7
OAI22	4	9

- (c) Now add a new pseudo-member to the cell library, called an inverter-pair (INVP), with cost 0, which matches two inverters in series. Replace every interconnect in the decomposed subject graph from part (a) with two inverters in series if the interconnect does not already have an inverter. Obtain all matches at all nodes of this modified decomposed subject graph and obtain the optimum-area network cover. What impact did replacing interconnects with inverter pairs have?
- (d) Obtain an optimum-delay network cover for the modified decomposed subject graph derived in part (c). Assume that all primary input values are available at time 0.

Problem 6.12

- (a) Decompose the subject graph shown in Fig. P6.12 using an inverter and two-input NOR as base functions, and assume that all primary input values arrive at time 0.

Fig. P6.12



- (b) Obtain an optimum delay cover for the decomposed subject graph using the cell library shown in Table P6.11. What is its area cost?
- (c) Suppose that the time constraint is relaxed to 11. Find the optimum-area cover under this constraint.

7

Threshold logic for nanotechnologies

We have been concerned with the design of switching circuits constructed of electronic gates or bilateral devices. There exists another type of switching device called a threshold element. With the advent of novel nanotechnologies, threshold elements are attracting attention once again since they form the basic logic primitives in some of these technologies.

Circuits constructed of threshold elements usually consist of fewer components and simpler interconnections than the corresponding circuits implemented with conventional gates. However, while the input–output relations of circuits constructed with conventional gates can be specified by switching algebra, different algebraic means must be developed for threshold circuits. In this chapter, we shall study the properties of threshold elements and present necessary and sufficient conditions for a switching function to be realizable by just a single element. We shall then present a general synthesis procedure for synthesizing switching circuits using only threshold elements. Finally, we shall discuss synthesis methods based on majority or minority logic gates, which are simple threshold elements.

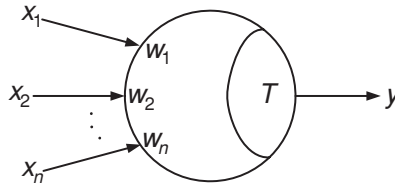
7.1 Introductory concepts

The usefulness of threshold logic, or any other new logic in digital system design, is determined by the availability, cost, and capabilities of the basic building blocks, as well as by the existence of effective synthesis procedures. In this section, we shall study the properties of the threshold element, and discuss its limitations and capabilities.

The threshold element

A *threshold element*, or *gate*, has n two-valued inputs $x_1, x_2, \dots, x_n$ and a single two-valued output y . Its internal parameters are a *threshold* T and *weights* $w_1, w_2, \dots, w_n$, where each weight w_i is associated with a particular input

Fig. 7.1 Symbol for a threshold element.



variable x_i . The values of the threshold T and the weights w_i ($i = 1, 2, \dots, n$) may be any real, finite, positive or negative numbers. The input–output relation of a threshold element is defined as follows:

$$\begin{aligned} y &= 1 \text{ if and only if } \sum_{i=1}^n w_i x_i \geq T, \\ y &= 0 \text{ if and only if } \sum_{i=1}^n w_i x_i < T, \end{aligned} \quad (7.1)$$

where the sum and product operations are the conventional arithmetic ones. The sum $\sum_{i=1}^n w_i x_i$ is called the weighted sum of the element. The symbol representing a threshold element is shown in Fig. 7.1.

Example The input–output relation of the threshold element shown in Fig. 7.2 is given in Table 7.1. The weighted sum is computed in the center column for every input combination. The value 1 is entered in the output

Table 7.1 Input output relation of the gate shown in Fig. 7.2

Input variables			Weighted sum $-x_1 + 2x_2 + x_3$	Output y
x_1	x_2	x_3		
0	0	0	0	0
0	0	1	1	1
0	1	0	2	1
0	1	1	3	1
1	0	0	-1	0
1	0	1	0	0
1	1	0	1	1
1	1	1	2	1

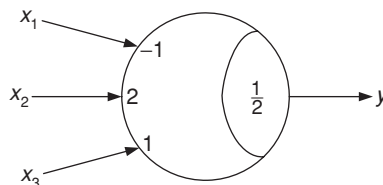
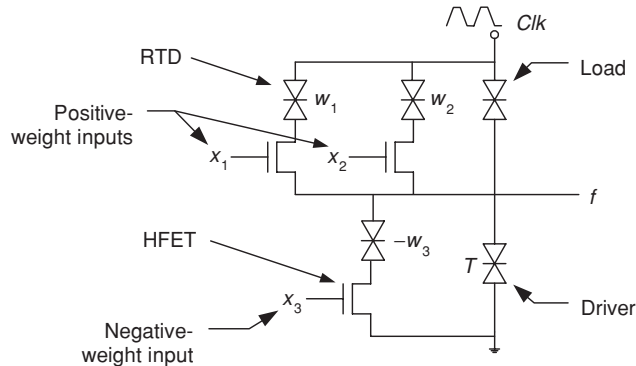


Fig. 7.2 A threshold element.

column in every row for which the weighted sum is greater than or equal to $\frac{1}{2}$ (because $T = \frac{1}{2}$), and the value 0 is entered in all the remaining rows. From the input–output relation (Table 7.1), it is evident that this threshold element realizes the switching function

$$y = f(x_1, x_2, x_3) = \sum(1, 2, 3, 6, 7) \\ = x_1'x_3 + x_2.$$

Fig. 7.3 An RTD–HFET MOBILE
[3] © 1996, IEEE.



The threshold element defined algebraically by Eqs. (7.1) can be constructed physically in various ways. Consider, for example, the threshold element shown in Fig. 7.3, which is based on *resonant tunneling diodes* (RTDs) and *heterostructure field-effect transistors* (HFETs). It is called a *monostable–bistable transition logic element* (MOBILE). A MOBILE is a rising-edge-triggered current-controlled gate. It has serially connected load and driver RTDs. The RTD and HFET structures connected in parallel to the load and driver RTDs perform positive and negative weighting of the inputs, respectively. The area of the RTD in these structures determines the corresponding weight. The difference in the areas of the driver and load RTDs determines the threshold.

Majority and minority gates

A *majority gate* is a special type of threshold element. A three-input majority gate produces an output value 1 if a majority of its inputs (i.e., two or three) are at 1. It implements a *majority function* M given by

$$M(x_1, x_2, x_3) = x_1x_2 + x_2x_3 + x_1x_3.$$

A majority gate can be implemented as a threshold element with $w_i = 1$, $1 \leq i \leq 3$, and $T = 2$. It is the basic logic primitive in various nanotechnologies, such as *quantum cellular automata* (QCA), *single-electron box* (SEB), and *tunneling phase logic* (TPL). By tying one of its inputs to 0 or 1 it can implement

Fig. 7.4 A QCA majority gate [18]

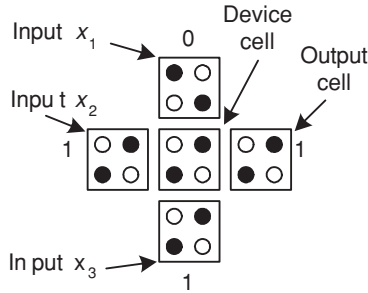
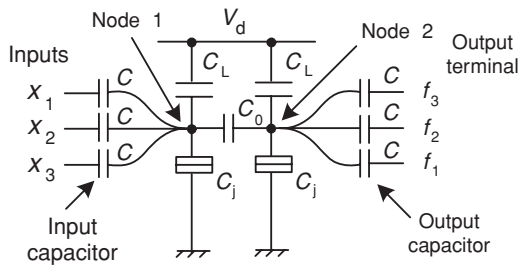


Fig. 7.5 An SEB majority gate [16].



an AND or OR gate, respectively. However, this is a very suboptimal use of a majority gate.

A QCA majority gate is shown in Fig. 7.4. It consists of five QCA cells: three input cells, a device cell, and an output cell. A QCA cell contains four quantum dots at the corners of a square and two electrons that can move to a quantum dot by electron tunneling. Owing to Coulombic interactions, the two electrons can only exist at opposite corners. One such polarization denotes the value 1 and the other the value 0, as shown in the figure. Electron tunneling is controlled by potential barriers that can be raised or lowered across neighboring cells. Computation in a majority gate is performed by driving the device cell to its lowest energy state. This occurs when this cell assumes the polarization of the majority of the three input cells. In this state, the Coulombic repulsions between electrons in the input cells are minimized. The polarization state of the device cell is transferred to the output cell.

An SEB majority gate is shown in Fig. 7.5. An SEB consists of a bias voltage V_d , tunneling junction C_j , and bias capacitor C_L in series. The internal state of the SEB is fully determined by V_d . The majority gate contains a balanced pair of SEBs, three input capacitors and three output capacitors. First, the output terminals are grounded; then V_d is gradually increased to establish the bistability of the balanced pair. With an increase in V_d , electron tunneling occurs and the balanced pair enters the (0, 1) or (1, 0) state, depending on the three input values. If the majority of the three inputs are at 1, the balanced pair goes to the state (1, 0) and produces a positive output voltage at node 2. Otherwise, a negative voltage is produced at node 2.

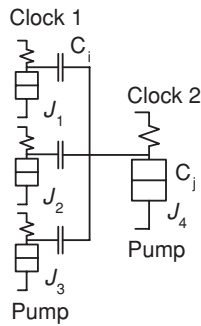


Fig. 7.6 A TPL minority gate [6]
© 1999, IEEE.

A three-input *minority gate* produces an output value 1 if a majority of its inputs are at 0. It implements a *minority function* m given by

$$m(x_1, x_2, x_3) = x'_1 x'_2 + x'_2 x'_3 + x'_1 x'_3.$$

It can be seen that a minority function is just the complement of the majority function. A minority gate can implement a NAND or NOR gate if one of its inputs is set to 0 or 1, respectively.

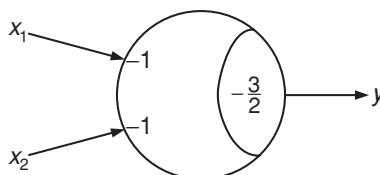
A TPL minority gate is shown in Fig. 7.6. It has two states and uses the phase of a waveform to represent these two logic values. The tunneling junction capacitance is C_j . The TPL operation is based on the phase locking of single electron tunneling oscillations to a pump signal that is distributed throughout the circuit. The pump frequency is set to twice the tunneling frequency. Hence, the electrical phase of the locked oscillation can take on two different values.

Capabilities and limitations of threshold logic

From the definition of threshold elements, it is evident that they are more powerful than conventional gates. Their higher capability is manifested in the ability of *single* threshold elements to realize a larger class of functions than is realizable by any single conventional gate. In fact, a threshold gate can be considered as a generalization of a conventional gate, because any type of the latter can be realized by a single threshold element. A two-input NAND gate, for example, can be realized by a single threshold element with weights -1 , -1 , and threshold $T = -\frac{3}{2}$, as shown in Fig. 7.7. Similarly, a threshold gate whose weights are unity and threshold $T = \frac{1}{2}$ realizes the OR operation, and so on. Since NAND is a functionally complete operation, any switching function can be realized by threshold elements alone.

Because of the wide range of weights and threshold combinations possible, a large class of switching functions can be realized by single threshold elements. As to whether every switching function is realizable by only one threshold element, the answer is *no*, as shown by the following example. Suppose that $f(x_1, x_2, x_3, x_4) = x_1 x_2 + x_3 x_4$ is realizable by a threshold element, with weights w_1, w_2, w_3, w_4 , and threshold T . Then the output value of this element must be 1 for each of the input combinations $x_1 x_2 x'_3 x'_4$ and $x'_1 x'_2 x_3 x_4$ and 0 for each of the input combinations $x'_1 x_2 x'_3 x_4$ and $x_1 x'_2 x_3 x'_4$.

Fig. 7.7 A threshold gate realizing the NAND operation.



Thus

$$\left. \begin{array}{l} w_1 + w_2 \geq T, \\ w_3 + w_4 \geq T \end{array} \right\} \Rightarrow w_1 + w_2 + w_3 + w_4 \geq 2T, \quad (7.2)$$

$$\left. \begin{array}{l} w_2 + w_4 < T \\ w_1 + w_3 < T \end{array} \right\} \Rightarrow w_1 + w_2 + w_3 + w_4 < 2T. \quad (7.3)$$

Clearly, the requirements in inequalities (7.2) and (7.3) are conflicting, and no threshold value can satisfy them. Consequently, $f = x_1x_2 + x_3x_4$ cannot be realized by a single threshold element.

In light of the fact that not every switching function is realizable by just a single threshold element, we now formulate the basic problem of threshold logic as follows.

- Given a switching function $f(x_1, x_2, \dots, x_n)$, determine whether it is realizable by a single threshold element and, if it is, find appropriate weights and threshold.

A switching function that can be realized by a single threshold element is called a *threshold function*.

A straightforward approach to the identification problem of threshold functions is to derive a set of 2^n linear simultaneous inequalities from the truth table and solve them. From the input combinations for which $f = 1$, we derive all the weighted sums that must exceed or equal the threshold T , and from the input combinations for which $f = 0$ we derive all the weighted sums that must be less than T . If a solution (not necessarily unique) to the above inequalities exists, it provides the values for the weights and threshold. If, however, no solution exists then f is not a threshold function.

Example Let $f(x_1, x_2, x_3) = \sum(0, 1, 3)$. The truth table and the corresponding inequalities are given in Table 7.2.

From the inequality which corresponds to combination 0, we observe that T must be negative and so must w_2 and w_1 (see combinations 2 and 4).

Table 7.2 Truth table with linear inequalities for $f = \sum(0, 1, 3)$

Combination	x_1	x_2	x_3	f	Inequality
0	0	0	0	1	$0 \geq T$
1	0	0	1	1	$w_3 \geq T$
2	0	1	0	0	$w_2 < T$
3	0	1	1	1	$w_2 + w_3 \geq T$
4	1	0	0	0	$w_1 < T$
5	1	0	1	0	$w_1 + w_3 < T$
6	1	1	0	0	$w_1 + w_2 < T$
7	1	1	1	0	$w_1 + w_2 + w_3 < T$

From combinations 3 and 5, we conclude that w_2 must be greater than w_1 , and from combination 1 we conclude that w_3 is greater than or equal to T . Thus, we are able at this point to establish the relation

$$w_3 \geq T > w_2 > w_1,$$

where only w_3 may be positive. If we restrict the weights to integer values and want to use weights of the smallest possible magnitude, we obtain

$$w_2 = -1, \quad w_1 = -2, \quad T = -\frac{1}{2}.$$

It is easy to verify that if we choose $w_3 = 1$ then all the inequalities are satisfied; f is therefore a threshold function.

For an n -variable switching function there are 2^n inequalities, some of which may be eliminated because they are implied by others (e.g., if inequalities 2 and 4 in Table 7.2 are satisfied then, since T is negative, inequality 6 is automatically implied and, similarly, inequality 7 is implied by 2 and 5). Although any set of linear inequalities can be either solved or shown by various methods to be inconsistent, it is desirable to explore further those properties of threshold functions that will make possible the development of more effective identification procedures. These properties will be explored in the next section.

The realization of other, nonthreshold, switching functions, whose corresponding AND–OR networks can be quite complex, can often be accomplished with just a few threshold elements. Thus, the use of threshold elements may result, among other things, in a considerable reduction in the number of gates and inputs as well as in the size of the final circuit.

A limitation of threshold logic is its sensitivity to variations in the circuit parameters. Owing to these variations and changes in the input and supply voltages, the weighted sum for a particular combination, especially with a large number of inputs, may deviate from its prescribed value and cause circuit malfunction. Restrictions must, therefore, be imposed on the maximum allowable number of inputs and on the threshold value T . Care must be taken to increase the difference between the values of the weighted sums for which f must equal 1 and for which f must equal 0. One way to do this is to introduce defect tolerances δ_{on} and δ_{off} in the definition of a threshold function, as follows:

$$\begin{aligned} y = 1 \text{ if and only if } \sum_{i=1}^n w_i x_i &\geq T + \delta_{\text{on}}, \\ y = 0 \text{ if and only if } \sum_{i=1}^n w_i x_i &< T - \delta_{\text{off}}. \end{aligned} \tag{7.4}$$

Generally, δ_{on} and δ_{off} take nonnegative values. Higher values of δ_{on} and δ_{off} imply greater tolerances of parametric variations. However, they also imply greater circuit area since larger weights may be required.

Elementary properties

In the discussions to follow, a threshold element will be specified by its input variables and a *weight–threshold vector*

$$V = \{w_1, w_2, \dots, w_n; T\}$$

Thus, the threshold element of Fig. 7.2 is completely specified by its input variables x_1, x_2, x_3 and $V = \{-1, 2, 1; \frac{1}{2}\}$.

Consider a function $f(x_1, x_2, \dots, x_n)$ that is realized by a single threshold element $V_1 = \{w_1, w_2, \dots, w_j, \dots, w_n; T\}$ whose inputs are $x_1, x_2, \dots, x_j, \dots, x_n$. Now suppose that one of the inputs, say x_j , is complemented. Then, as we will show, the same function f is realizable by a single threshold element $V_2 = \{w_1, w_2, \dots, -w_j, \dots, w_n; T - w_j\}$, whose inputs are $x_1, x_2, \dots, x'_j, \dots, x_n$.

From the inequalities in Eq. (7.1) and from V_1 , we find that

$$\text{if } w_j x_j + \sum_{i \neq j} w_i x_i \begin{cases} \geq T & \text{then } f = 1, \\ < T & \text{then } f = 0. \end{cases} \quad (7.5)$$

When V_2 replaces V_1 and x'_j replaces x_j , we find that

$$\text{if } -w_j x'_j + \sum_{i \neq j} w_i x_i \begin{cases} \geq T - w_j & \text{then } g = 1, \\ < T - w_j & \text{then } g = 0, \end{cases} \quad (7.6)$$

where g is the function realized by element V_2 . To prove that g and f are identical functions, let $x_j = 0$ so that $x'_j = 1$. Then Eqs. (7.5) and (7.6) become identical. Next, let $x_j = 1$ so that $x'_j = 0$. Again, Eqs. (7.5) and (7.6) become identical. Consequently, both f and g assume identical values for each input combination and are thus identical functions.

The above property leads to several important conclusions. If a function is realizable by a single threshold element then, by an appropriate selection of complemented and uncomplemented input variables, it is possible to obtain a realization by an element whose weights have any desired *sign* distribution. Therefore, *if a function is realizable by a single threshold element then it is realizable by an element with only positive weights*. Clearly, this assertion is valid only if the input variables are available in both complemented and uncomplemented forms.

We shall next show that if a function $f(x_1, x_2, \dots, x_n)$ is realizable by a single threshold element whose weight–threshold vector is $V_1 = \{w_1, w_2, \dots, w_n; T\}$ then its complement $f'(x_1, x_2, \dots, x_n)$ is realizable by a single threshold element whose weight–threshold vector is $V_2 = \{-w_1, -w_2, \dots, -w_n; -T\}$, under a given condition.¹

¹ The condition requires us to restrict the values of the weights and threshold such that for no input combination will the weighted sum be exactly equal to T .

From the inequalities in Eq. (7.1) and from V_1 we obtain

$$\begin{aligned} \sum_{i=1}^n w_i x_i &> T \quad \text{when } f = 1, \\ \sum_{i=1}^n w_i x_i &< T \quad \text{when } f = 0. \end{aligned} \quad (7.7)$$

Multiplying both sides of Eq. (7.7) by -1 yields

$$\begin{aligned} \sum_{i=1}^n -w_i x_i &< -T \quad \text{when } f = 1 \text{ or } f' = 0, \\ \sum_{i=1}^n -w_i x_i &> -T \quad \text{when } f = 0 \text{ or } f' = 1. \end{aligned} \quad (7.8)$$

Clearly, the inequalities in Eq. (7.8) demonstrate that f' is realizable by the threshold element whose weight-threshold vector is V_2 .

7.2 Synthesis of threshold networks

Our principal goal in this section is the development of methods for the identification and realization of threshold functions as well as for the synthesis of networks of threshold elements, called *threshold networks*. Before proceeding with this general study, we shall present a number of properties of threshold functions that provide the theoretical background necessary for the development of simpler and more effective synthesis methods. We shall be concerned with the synthesis of threshold functions as well as the realization of nonthreshold functions with a network of threshold elements.

Unate functions

A function $f(x_1, x_2, \dots, x_n)$ is said to be positive in a variable x_i if there exists a disjunctive or conjunctive expression for the function in which x_i appears only in uncomplemented form. Analogously, $f(x_1, x_2, \dots, x_n)$ is said to be negative in x_i if there exists a disjunctive or conjunctive expression for f in which x_i appears only in complemented form. If f is either positive or negative in x_i then it is said to be unate in x_i .

Example The function $f = x_1 x_2' + x_2 x_3'$ is positive in x_1 and negative in x_3 but is not unate in x_2 .

If a function $f(x_1, x_2, \dots, x_n)$ is unate in each of its variables then it is called *unate*. Thus, a function is unate if it can be represented by a disjunctive or conjunctive expression in which no variable appears in both its complemented and uncomplemented forms.

Example The function $f = x'_1x_2 + x_1x_2x'_3$ is unate because a disjunctive expression for f exists that satisfies the above definition, namely, $f = x'_1x_2 + x_2x'_3$. However, the function $f = x_1x'_2 + x'_1x_2$ is clearly not unate in either of its variables.

If $f(x_1, x_2, \dots, x_n)$ is positive in x_i then it can be expressed as

$$f(x_1, x_2, \dots, x_n) = x_i g_1(x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_n) + h_1(x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_n). \quad (7.9)$$

Similarly, if $f(x_1, x_2, \dots, x_n)$ is negative in x_i then it can be expressed as

$$f(x_1, x_2, \dots, x_n) = x'_i g_2(x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_n) + h_2(x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_n). \quad (7.10)$$

By definition, if a function f can be expressed by Eq. (7.9) (Eq. (7.10)) then it is positive (negative) in x_i . Hence, *the existence of two such functions, g_1 and h_1 (g_2 and h_2), is a necessary and sufficient condition for f to be positive (negative) in x_i .*

Geometric representation

Unate functions have several interesting properties, which are best illustrated by a geometrical representation. An n -cube contains 2^n vertices, each of which represents an assignment of values to the n variables and thus corresponds to a minterm. A line is drawn between every pair of vertices that differ in just one variable, and no other lines are drawn. The vertices corresponding to true minterms, that is, for which the function assumes the value 1, are called *true vertices* while those for which the function assumes the value 0 are called *false vertices*. The analogy between the n -cube and map methods for representing switching functions is evident.

Example The three-cube representation of the function $f = x'y' + xz$ is shown in Fig. 7.8. The bolder lines connecting the two pairs of true vertices, i.e., the pair (1, 1, 1) and (1, 0, 1) and the pair (0, 0, 1) and (0, 0, 0), represent the cubes xz and $x'y'$, respectively.

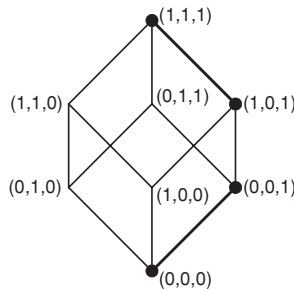


Fig. 7.8 A three-cube (2^3 -vertex) representation of $f = x'y' + xz$.

It is convenient to define a partial-ordering relation between vertices of the n -cube, such that

$$(a_1, a_2, \dots, a_n) \leq (b_1, b_2, \dots, b_n)$$

if and only if, for all i , $a_i \leq b_i$. As shown in Chapter 2, this partially ordered set of vertices is a lattice and the vertices $(0, 0, \dots, 0)$ and $(1, 1, \dots, 1)$ are, respectively, the least vertex and the greatest vertex of the lattice. As in any partial ordering, some pairs of vertices may be incomparable, for example, $(0, 0, \dots, 0, 1)$ and $(1, 0, \dots, 0, 0)$.

Without loss of generality, we shall subsequently restrict our attention to unate functions that are positive in all their variables, that is, functions without any complemented variable. Such a restriction is justified because every complemented variable in a unate function may be relabeled, so that $x'_i \rightarrow y_i$, etc., and obviously, the resulting function is unate if and only if the original one is. For example, the unate function $x'_1 x_2 x'_3 + x_2 x'_3 x_4$ may be converted to $x_1 x_2 x_3 + x_2 x_3 x_4$, using the relabelings $x'_1 \rightarrow x_1$ and $x'_3 \rightarrow x_3$. By reconverting the latter function it is possible to determine the original one.

Theorem 7.1 *A switching function $f(x_1, x_2, \dots, x_n)$ is unate if and only if it is not a tautology² and the above partial ordering exists, such that, for every pair of vertices $(a_1, a_2, \dots, a_n)$ and $(b_1, b_2, \dots, b_n)$, if $(a_1, a_2, \dots, a_n)$ is a true vertex and $(b_1, b_2, \dots, b_n) \geq (a_1, a_2, \dots, a_n)$ then $(b_1, b_2, \dots, b_n)$ is also a true vertex of f .*

Proof Suppose that f is unate. Let us find an expression Φ that represents f as a positive function in all its variables. Obviously, Φ is not a tautology. If $(a_1, a_2, \dots, a_n)$ is a true vertex then it represents an assignment of values to input variables that causes some prime implicant of Φ to be true. If $(b_1, b_2, \dots, b_n) \geq (a_1, a_2, \dots, a_n)$ then, for every $a_i = 1$, the corresponding $b_i = 1$. Therefore, since Φ is positive, $(b_1, b_2, \dots, b_n)$ also represents an assignment of values of the input variables, which causes at least the previously mentioned prime implicant to be true. This proves the “only if” part of the theorem.

Now suppose that f is not a tautology and that, for every pair of its vertices $(a_1, a_2, \dots, a_n)$ and $(b_1, b_2, \dots, b_n)$, if $(a_1, a_2, \dots, a_n)$ is true and $(b_1, b_2, \dots, b_n) \geq (a_1, a_2, \dots, a_n)$ then $(b_1, b_2, \dots, b_n)$ is also true. Since f is not a tautology, $(0, 0, \dots, 0)$ is a false vertex. Consider the k vertices $S_1, S_2, \dots, S_k$, which are the minimal³ true vertices of the lattice. To each vertex S_i there corresponds a product term that consists of just those uncomplemented literals whose corresponding value in S_i is 1; for example, if for a function $f(x_1, x_2, x_3, x_4)$ we have $S_i = (0, 1, 0, 1)$ then the corresponding product term is $x_2 x_4$. The expression formed by the disjunction of the k product terms, which

² A *tautology* is a function which is equal to 1 for all combinations of its variables.

³ A true vertex S_i is said to be *minimal* if no other true vertex $S_j < S_i$. A false vertex S_i is said to be *maximal* if no other false vertex $S_j > S_i$. (See Section 2.3.)

correspond to all the minimal true vertices, is an expression for Φ . Since Φ is positive in all its variables, f is unate. $\diamond$

Example For the unate function $f = x_1x_2 + x_3x_4$ there are two minimal true vertices, namely, $S_1 = (1, 1, 0, 0)$ and $S_2 = (0, 0, 1, 1)$. According to Theorem 7.1, every vertex (a_1, a_2, a_3, a_4) that is greater than S_1 or S_2 must be a true vertex. For example, $(1, 1, 1, 0)$ and $(0, 1, 1, 1)$ are true vertices since $(1, 1, 1, 0) > (1, 1, 0, 0)$ and $(0, 1, 1, 1) > (0, 0, 1, 1)$. Indeed, these vertices correspond, respectively, to the products $x_1x_2x_3$ and $x_2x_3x_4$, which are covered by f .

Linear separability

If we use the n -cube representation for threshold functions and regard the vertices as points in an n -dimensional space, we observe that the linear equation

$$w_1x_1 + w_2x_2 + \cdots + w_nx_n = T \quad (7.11)$$

corresponds to an $(n - 1)$ -dimensional hyperplane that cuts the n -cube. Now, since $f = 0$ when

$$w_1x_1 + w_2x_2 + \cdots + w_nx_n < T$$

and $f = 1$ when

$$w_1x_1 + w_2x_2 + \cdots + w_nx_n \geq T$$

we observe that the hyperplane separates the true vertices from the false ones. A switching function whose true vertices can be separated by a linear equation from its false ones is called a *linearly separable function*, and the functional property that makes such a separation possible is known as *linear separability*. Since by definition every function whose true vertices are separable from its false ones by Eq. (7.11) is a threshold function, we may conclude that all threshold functions are linearly separable and vice versa. Indeed, the terms “threshold function” and “linearly separable function” are used interchangeably to describe the same functional property.

Let $f(x_1, x_2, \dots, x_n)$ be a threshold function that depends upon and is positive in the variable x_i and to which there corresponds the weight–threshold vector $V = \{w_1, w_2, \dots, w_n; T\}$. Since f is positive in x_i , there exists a set of values $a_1, a_2, \dots, a_{i-1}, a_{i+1}, \dots, a_n$ for the input variables $x_1, x_2, \dots, x_{i-1}, x_{i+1}, \dots, x_n$ such that

$$f(a_1, \dots, a_{i-1}, 1, a_{i+1}, \dots, a_n) = 1$$

and

$$f(a_1, \dots, a_{i-1}, 0, a_{i+1}, \dots, a_n) = 0.$$

Hence,

$$w_1a_1 + \cdots + w_{i-1}a_{i-1} + w_i + w_{i+1}a_{i+1} + \cdots + w_na_n > T,$$

$$w_1a_1 + \cdots + w_{i-1}a_{i-1} + w_{i+1}a_{i+1} + \cdots + w_na_n < T,$$

and consequently $w_i > 0$. Since the above argument may be applied to every x_i in $\{x_1, x_2, \dots, x_n\}$, it follows that *the weights associated with a threshold function that is positive in all its variables are all positive*. A threshold function that is positive (negative) in all its variables is called a *positive (negative)* threshold function. Note that if f has a positive expression independent of x_i then $w_i = 0$; but we shall not consider such functions.

Theorem 7.2 *Every threshold function is unate.*

Proof Let $f(x_1, x_2, \dots, x_n)$ be a threshold function whose true vertices can be separated from the false ones by the hyperplane

$$w_1x_1 + w_2x_2 + \cdots + w_nx_n = T.$$

Suppose that f depends upon x_i and $w_i > 0$; then, for every combination $(a_1, a_2, \dots, a_{i-1}, a_{i+1}, \dots, a_n)$ of the variables $x_1, x_2, \dots, x_{i-1}, x_{i+1}, \dots, x_n$, if the vertex $(a_1, a_2, \dots, a_{i-1}, 0, a_{i+1}, \dots, a_n)$ is true then the vertex $(a_1, a_2, \dots, a_{i-1}, 1, a_{i+1}, \dots, a_n)$ must also be true, because

$$\begin{aligned} w_1a_1 + w_2a_2 + \cdots + w_i + \cdots + w_na_n \\ > w_1a_1 + w_2a_2 + \cdots + w_{i-1}a_{i-1} + w_{i+1}a_{i+1} + \cdots + w_na_n. \end{aligned}$$

However, since f is not independent of x_i , the vertex $(a_1, a_2, \dots, a_{i-1}, 0, a_{i+1}, \dots, a_n)$ must be false, proving that f is positive in x_i . Now consider a variable x_i whose weight is negative, i.e., $w_i < 0$; if vertex $(a_1, a_2, \dots, a_{i-1}, 1, a_{i+1}, \dots, a_n)$ is true then so is $(a_1, a_2, \dots, a_{i-1}, 0, a_{i+1}, \dots, a_n)$, because

$$\begin{aligned} w_1a_1 + w_2a_2 + \cdots + w_i + \cdots + w_na_n \\ < w_1a_1 + w_2a_2 + \cdots + w_{i-1}a_{i-1} + w_{i+1}a_{i+1} + \cdots + w_na_n. \end{aligned}$$

Also, since f is not independent of x_i , the vertex $(a_1, a_2, \dots, a_{i-1}, 1, a_{i+1}, \dots, a_n)$ must be false, proving that f is negative in x_i . Consequently f is either positive or negative in each of its variables, and thus it is unate. $\diamond$

The converse of Theorem 7.2 is not true, because there exist many unate functions that are not linearly separable, e.g., $x_1x_2 + x_3x_4$.

Theorem 7.3 *Suppose that, given an expression for a unate switching function $f(x_1, x_2, \dots, x_n)$, literal x_j is replaced by literal x'_k , $j \neq k$, resulting in the expression $g(x_1, x_2, \dots, x_n)$. If g is not a threshold function then neither is f .*

Proof We will prove the contrapositive of the claim. That is, if f is a threshold function then g is a threshold function. Suppose that the weight-threshold

vector of f is $\{w_1, w_2, \dots, w_n; T\}$. We have

$$\begin{aligned} \sum_{i=1}^n w_i x_i \geq T &\Rightarrow f = 1, \\ \sum_{i=1}^n w_i x_i < T &\Rightarrow f = 0. \end{aligned} \quad (7.12)$$

The above equations represent 2^n inequalities for all value combinations of variables $x_1, x_2, \dots, x_n$. By replacing x_j with x'_k , we obtain the following 2^{n-1} inequalities:

$$\begin{aligned} \sum_{i=1, i \neq j}^n w_i x_i + w_j x'_k \geq T &\Rightarrow g = 1, \\ \sum_{i=1, i \neq j}^n w_i x_i + w_j x'_k < T &\Rightarrow g = 0. \end{aligned} \quad (7.13)$$

Since $x'_k = 1 - x_k$, we obtain

$$\begin{aligned} \sum_{i=1, i \neq j, k}^n w_i x_i + (w_k - w_j)x_k \geq (T - w_j) &\Rightarrow g = 1, \\ \sum_{i=1, i \neq j, k}^n w_i x_i + (w_k - w_j)x_k < (T - w_j) &\Rightarrow g = 0. \end{aligned} \quad (7.14)$$

The above inequalities can be satisfied by the weight-threshold vector $\{w_1, w_2, \dots, w_{j-1}, w_{j+1}, \dots, w_{k-1}, w_k - w_j, w_{k+1}, \dots, w_n; T - w_j\}$. The variable sequence corresponding to the weights is $x_1, x_2, \dots, x_{j-1}, x_{j+1}, \dots, x_{k-1}, x_k, x_{k+1}, \dots, x_n$. Thus, g is also a threshold function. $\diamond$

Example Let us apply Theorem 7.3 to $f = x_1 x_2 + x_3 x_4$. To determine whether f is a threshold function, we replace x_2 by x'_3 . This results in $g = x_1 x'_3 + x_3 x_4$. Since g is not unate in x_3 , it is not a threshold function and, therefore, neither is f .

Identification and realization of threshold functions

Our current objective is to present a procedure that will determine whether a given switching function is a threshold function and, if it is, whether it will provide the values of the weights and threshold. The approach to be taken utilizes the linear separability property of threshold functions. In fact, it is a test to determine whether there exists a hyperplane that separates the true vertices of the function from the false ones. This is accomplished in several steps.

First, the given function is tested for unateness. This test is executed by examining a minimal expression of the function. Also, since a unate function has a unique minimal form (see Problem 7.10), if this expression is not unate then the function is not linearly separable. If it is unate, it is converted into another

function that is positive in all its variables. For example, if $f = x_1x_2x'_3x_4 + x_2x'_3x'_4$ then its reduced expression is $f = x_1x_2x'_3 + x_2x'_3x'_4$ and, since it is unate, it is converted to $\Phi = x_1x_2x_3 + x_2x_3x_4$.

Next, one finds all the minimal true and maximal false vertices of Φ . In the above example, there are two minimal true vertices, namely, (1, 1, 1, 0) and (0, 1, 1, 1). The maximal false vertices are found by determining all false vertices with just one variable whose value is 0, then all false vertices with two variables whose value is 0, and so on, leaving out all vertices smaller than the ones already selected. Clearly, the list of minimal true vertices contains all the necessary information for the determination of the maximal false vertices. In our running example, the maximal false vertices are (1, 1, 0, 1), (1, 0, 1, 1), and (0, 1, 1, 0).

To determine whether Φ is linearly separable and, if it is, to find an appropriate set of weights and threshold, it is necessary to determine the coefficients of the separating hyperplane. This is accomplished by deriving and solving a system of pq inequalities, corresponding to the p minimal true and q maximal false vertices. For each pair of vertices $A = \{a_1, a_2, \dots, a_n\}$ and $B = \{b_1, b_2, \dots, b_n\}$, where A and B are, respectively, the minimal true and maximal false vertices, we write the inequality

$$w_1a_1 + w_2a_2 + \dots + w_na_n > w_1b_1 + w_2b_2 + \dots + w_nb_n. \quad (7.15)$$

In our example, since $p = 2$ and $q = 3$, we find six inequalities, as follows:

$$\begin{aligned} w_1 + w_2 + w_3 &> w_1 + w_2 + w_4, \\ w_1 + w_2 + w_3 &> w_1 + w_3 + w_4, \\ w_1 + w_2 + w_3 &> w_2 + w_3, \\ w_2 + w_3 + w_4 &> w_1 + w_2 + w_4, \\ w_2 + w_3 + w_4 &> w_1 + w_3 + w_4, \\ w_2 + w_3 + w_4 &> w_2 + w_3, \end{aligned} \quad (7.16)$$

Since Φ is a positive function, if it is linearly separable then the separating hyperplane, Eq. (7.11), will have positive coefficients. This hyperplane separating the minimal true vertices from the maximal false vertices separates all true vertices from all false ones and thus yields the weight–threshold vector for Φ . Solving the system of inequalities given in Eq. (7.16), we observe that the following are the constraints that must be satisfied:

$$\begin{aligned} w_3 &> w_4, & w_3 &> w_1, \\ w_2 &> w_4, & w_2 &> w_1, \\ w_1 &> 0, & w_4 &> 0. \end{aligned}$$

Letting $w_1 = w_4 = 1$ and $w_2 = w_3 = 2$, we find, by substituting these values into Eq. (7.16), that T must be smaller than 5 but larger than 4. Selecting $T = \frac{9}{2}$ yields the weight–threshold vector for Φ , $V = \{1, 2, 2, 1; \frac{9}{2}\}$.

Finally, it is necessary to convert this weight–threshold vector to one that corresponds to the original function f . The conversion process is based on the

properties established in Eq. (7.6), where, for every input x_j that is complemented in the original function, w_j must be changed to $-w_j$ and T to $T - w_j$. In the above example, the inputs x_3 and x_4 appear in f in complemented form. Thus, in the new weight–threshold vector the weights are 1, 2, -2 , and -1 , and the threshold is $\frac{9}{2} - 2 - 1 = \frac{3}{2}$, which yields $V = \{1, 2, -2, -1; \frac{3}{2}\}$.

Example Determine whether the function

$$f(x_1, x_2, x_3, x_4) = \sum(0, 1, 3, 4, 5, 6, 7, 12, 13)$$

is a threshold function and, if it is, find a weight–threshold vector. Note that $f = x'_1x_2 + x'_1x_4 + x_2x'_3 + x'_1x'_3$ is unate and, therefore, can be converted into the positive function $\Phi = x_1x_2 + x_1x_4 + x_2x_3 + x_1x_3$. The minimal true vertices are

$$(1, 1, 0, 0), \quad (1, 0, 0, 1), \quad (0, 1, 1, 0), \quad (1, 0, 1, 0).$$

The maximal false vertices are

$$(0, 1, 0, 1), \quad (0, 0, 1, 1), \quad (1, 0, 0, 0).$$

Consequently, we obtain a system of 12 inequalities:

$$\left. \begin{array}{l} w_1 + w_2 \\ w_1 + w_4 \\ w_2 + w_3 \\ w_1 + w_3 \end{array} \right\} > \left\{ \begin{array}{l} w_2 + w_4 \\ w_3 + w_4 \\ w_1 \end{array} \right.$$

These inequalities impose several constraints on the weights associated with Φ , namely,

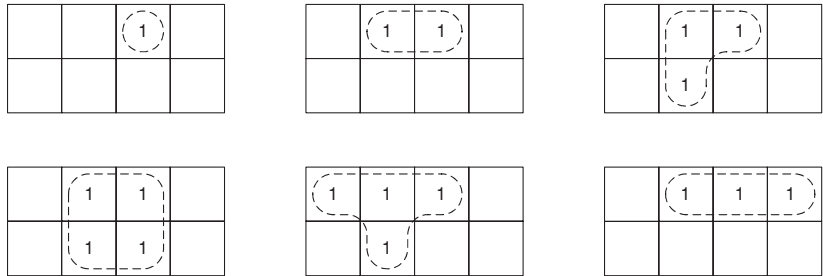
$$\begin{array}{lll} w_1 > w_4, & w_3 > w_4, & w_2 > 0, \\ w_1 > w_2, & w_2 > w_4, & w_3 > 0, \\ w_1 > w_3, & & w_4 > 0. \end{array}$$

If we let $w_4 = 1$ and $w_2 = w_3 = 2$, then it is necessary to make the assignment $w_1 = 3$, because w_1 must be smaller than $w_2 + w_3$.

Now we have, for example, a true vertex $(0, 1, 1, 0)$ whose weighted sum is 4 and a false vertex $(1, 0, 0, 0)$ whose weighted sum is 3. Consequently, $T = \frac{7}{2}$ and the weight–threshold vector for the Φ is $V = \{3, 2, 2, 1; \frac{7}{2}\}$. To find the corresponding vector for the original function f , note that x_1 and x_3 must be complemented. Thus, f is a threshold function whose weight–threshold vector is $V = \{-3, 2, -2, 1; -\frac{3}{2}\}$.

In more complex problems, and when the number of inequalities is large, it becomes necessary to resort to machine computation. By utilizing other properties of threshold functions it is possible to simplify somewhat the identification procedure, but all known methods still involve a solution of some complex system of equations. A listing of all threshold functions of up to seven variables

Fig. 7.9 Admissible patterns for threshold functions of three variables.



can be found in various references; see, for example [19]. Such a listing, which usually contains the weights and threshold corresponding to each linearly separable function, is very helpful in the design of threshold networks.

Map-based synthesis of two-level threshold networks

We have been concerned mainly with the problem of identifying and realizing threshold functions. The next natural problem is that of synthesizing networks constructed of threshold elements to realize any arbitrary switching function. One approach to such synthesis is to develop a procedure for the decomposition of nonthreshold functions into two or more factors, each of which is a threshold function.

For functions of three or four variables, the identification problem may be solved by detecting certain patterns in the corresponding maps. A pattern of 1-cells is said to be an *admissible pattern* if it can be realized by a single threshold element. The admissible patterns for threshold functions of three variables are shown in Fig. 7.9. Each admissible pattern may be in any position on the map, provided that its basic topological structure is preserved. Clearly, any admissible pattern for functions of three variables is also an admissible pattern for functions of four or more variables, and so on. Note that, since the complement of a threshold function is also a threshold function, the patterns formed by 0-cells are also admissible.

Analogously to the synthesis of AND–OR networks, a threshold-logic realization of an arbitrary switching function can now be achieved by selecting a minimal number of admissible patterns such that each 1-cell of the map is covered by at least one admissible pattern.

Example Given a switching function

$$f(x_1, x_2, x_3, x_4) = \sum(2, 3, 6, 7, 10, 12, 14, 15),$$

find a minimal threshold-logic realization. (By a *minimal realization*, we mean one that requires the smallest number of threshold elements.)

The map of f is shown in Fig. 7.10a, where the admissible patterns are marked by broken lines. A quick test (see Problem 7.10) reveals that f

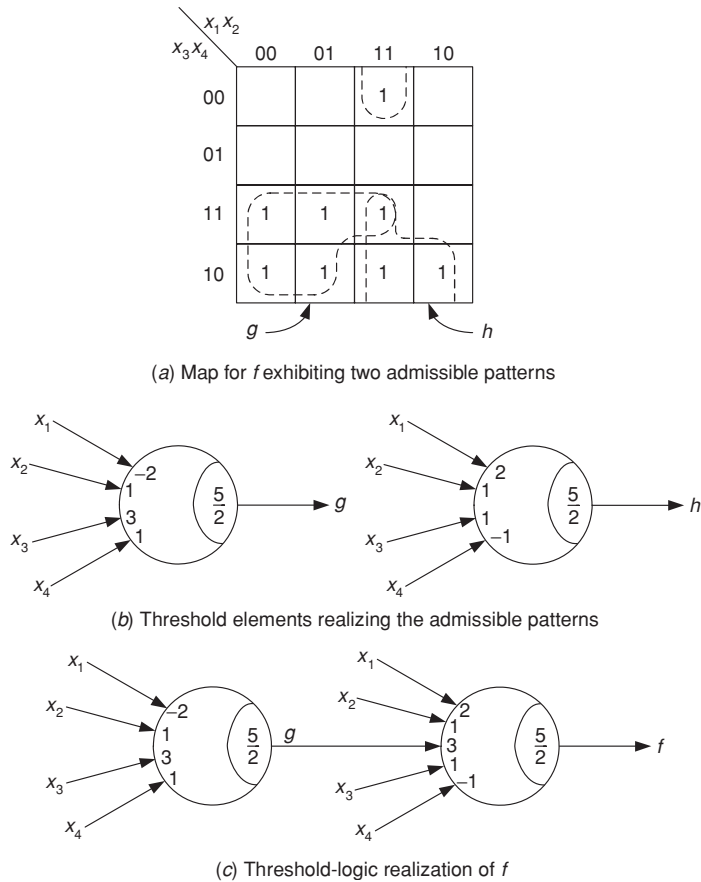


Fig. 7.10 Synthesis of the function $f(x_1, x_2, x_3, x_4) = \sum(2, 3, 6, 7, 10, 12, 14, 15)$.

is not unate and consequently not linearly separable. Hence, we shall attempt to synthesize it as a cascade of two threshold elements, such that the first element realizes an admissible pattern g , and the second element realizes an admissible pattern h . By applying the techniques of the preceding section to the function $g(x_1, x_2, x_3, x_4) = \sum(2, 3, 6, 7, 15)$, the weight–threshold vector for the first element is found to be $V_g = \{-2, 1, 3, 1; \frac{5}{2}\}$. Similarly, the weight–threshold vector for the element that realizes admissible pattern h is found to be $V_h = \{2, 1, 1, -1; \frac{5}{2}\}$. These elements are shown in Fig. 7.10b.

If we select the threshold element that realizes g as the first element then the second element must be such that it will realize h and at the same time allow g to propagate through it uninterrupted. In other words, the second element must, in addition to realizing h , act as an OR gate whose output value is 1 if either g or h or both are 1. This is accomplished by providing it with five inputs, as shown in Fig. 7.10c. The four inputs associated with

the variables x_1, x_2, x_3 , and x_4 have the weights determined earlier, while the fifth input is reserved for g .

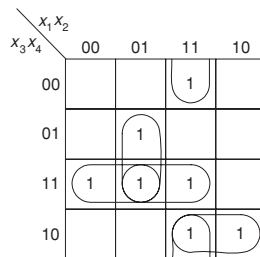
It is now only necessary to determine the weight w_g associated with the input g . This weight can be determined by computing the minimal weighted sum that can occur in the second element when g has the value 1. Since f must have the value 1 whenever g does, this minimal weighted sum must be larger than the threshold of the second element. In our case, the minimal weighted sum is w_g , and it occurs when $x_1 = x_2 = 0$ and $x_3 = x_4 = 1$. Clearly, w_g must be larger than $\frac{5}{2}$ and, therefore, the value $w_g = 3$ has been selected.

To simplify the computation of w_g , it can be set equal to (or larger than) the sum of the threshold and absolute values of all negative weights of the second element. This, however, will not always yield a minimal value for w_g .

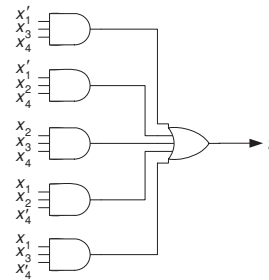
Example Consider the switching function

$$f(x_1, x_2, x_3, x_4) = \sum(3, 5, 7, 10, 12, 14, 15),$$

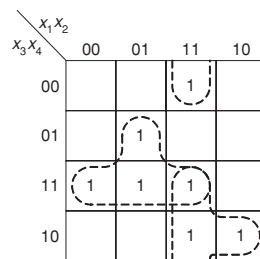
whose map is shown in Fig. 7.11a. Its minimal two-level AND–OR realization, shown in Fig. 7.11b, requires six gates but only two threshold elements, as shown in Fig. 7.11d.



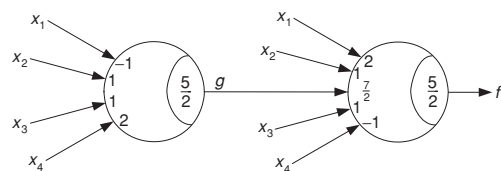
(a) Map showing a minimal set of prime implicants that covers f .



(b) AND–OR realization of f .



(c) Map showing the admissible pattern realized by each threshold element.



(d) A threshold logic realization of f .

Fig. 7.11 Two realizations of $f(x_1, x_2, x_3, x_4) = \sum(3, 5, 7, 10, 12, 14, 15)$.

The admissible patterns realized by the threshold elements are indicated by the patches on the map of Fig. 7.11c. The first element realizes the threshold function $g = \sum(3, 5, 7, 15)$, while the second element realizes the function $g + \sum(10, 12, 14, 15)$. The weight w_g associated with input g has been specified as $\frac{7}{2}$, which is equal to the sum of the threshold and the absolute value of w_4 . This ensures that $f = 1$ whenever $g = 1$, regardless of the weighted sum of the variables within the second element. Hence, $f = 1$ whenever either $g = 1$ or the weighted sum in the second element is greater than $\frac{5}{2}$.

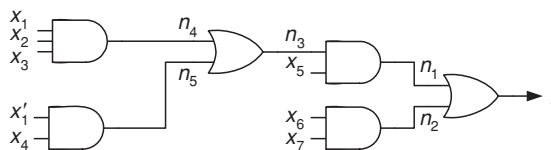
The synthesis procedure outlined in the preceding examples is particularly useful when the number of admissible patterns is small. Whenever the choice of admissible patterns is not obvious, it is necessary to construct a chart of patterns versus true vertices that is analogous to the prime implicant chart and is such that a minimal subset of admissible patterns can be determined. For functions of five or more variables, it is possible to derive the set of all admissible patterns by a tabulation procedure (see [4]) and then to construct the chart for selecting a minimal subset of admissible patterns.

Synthesis of multi-level threshold networks

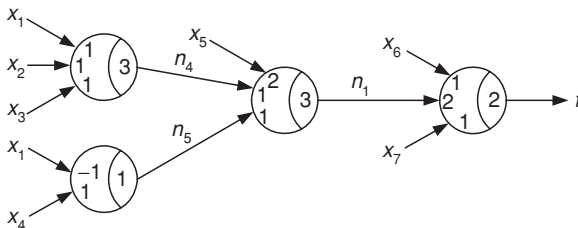
It may be inefficient to implement large switching functions with a two-level threshold network. A multi-level threshold network, consisting of many levels of threshold elements, may be much more compact. As we saw in Chapter 6, traditional multi-level network synthesis is a rich and mature area. We shall see next how traditional synthesis techniques can be enhanced for multi-level threshold network synthesis.

Example Consider the switching network shown in Fig. 7.12a. It has seven gates (including the inverter at x_1) and five levels. If we simply replace each gate with a threshold element, the resulting threshold network will also contain seven threshold elements and five levels. However, this threshold network is suboptimal because some nodes in Fig. 7.12a can be collapsed into a single threshold node. Choosing which node to collapse is critical. If we set the fanin restriction of a node to four, $f = n_1 + n_2$ can be collapsed to get $f = n_3x_5 + x_6x_7$.

We must next determine whether f is a threshold function, using the system of inequalities described earlier. It turns out that f is not a threshold function. Consequently, we must split f into two or more nodes. Suppose we choose to split f into $n_1 + x_6x_7$, where $n_1 = n_3x_5$. Since $n_1 + x_6x_7$ is a threshold function, we proceed to synthesize n_1 . After collapsing the



(a) Switching network.



(b) Equivalent threshold network.

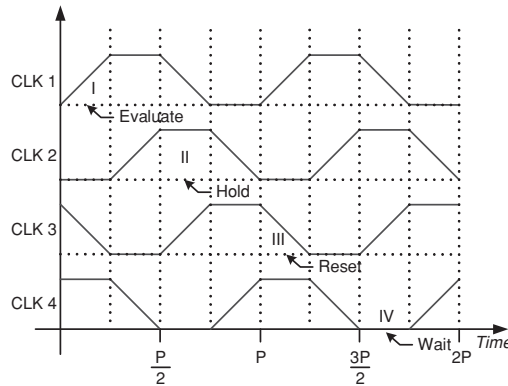
Fig. 7.12 A switching network and an equivalent threshold network [20] © 2005, IEEE.

function, n_1 can be expressed as $n_1 = n_4x_5 + n_5x_5$. Since this is also a threshold function, we next synthesize $n_4 = x_1x_2x_3$ and $n_5 = x_1'x_4$, which are both threshold functions. The corresponding synthesized threshold network shown in Fig. 7.12b contains only four threshold gates and three levels.

We next provide an overview of how multi-level threshold network synthesis can be done. One can start with a multi-output algebraically factored switching network G that implements the given set of switching functions, since its nodes are more likely to be unate and hence possibly threshold functions. The user can specify the maximum number of inputs allowed for any threshold element in the final threshold network that needs to be synthesized.

The synthesis procedure begins by processing each circuit output of G . First, the node representing a circuit output is collapsed. If the node represents a binate function it is split into multiple nodes, which are then processed recursively. If the node is unate and is also a threshold function, it is saved in the threshold network and the inputs of the node processed recursively. Otherwise, the unate node is split into two or more nodes that are threshold functions. The synthesis procedure terminates when all the nodes in the network G are mapped to threshold nodes. Sometimes, for a given node in G , directly mapping the AND and OR gates in the subnetwork implementing it to threshold elements may result in fewer threshold elements for that subnetwork than synthesizing it with

Fig. 7.13 Four-phase clocking for MOBILE circuits [3] © 1996, IEEE.



the above procedure. One can then choose the better of the two subnetworks to implement that node.

Mapping of threshold networks to MOBILEs

A threshold network can be mapped to RTD–HFET structures called MOBILEs, which were introduced earlier (see Fig. 7.3 for an example of a MOBILE). A MOBILE is said to be a self-latching threshold gate, because its output is valid only when the clock is high. One possible clocking scheme for MOBILE circuits consists of four phases, as shown in Fig. 7.13. During the *evaluate* phase, the output of a MOBILE is computed. In the *hold* (i.e., self-latching) phase, the result is valid. In the *reset* phase, the load capacitance is discharged and the MOBILE returns to the monostable mode of operation. Finally, in the *wait* phase, the inputs of the present MOBILE are loaded with the results obtained from the predecessor MOBILE.

In order to make sure that a MOBILE-based threshold network functions correctly under four-phase clocking, we have to make sure that all the input signals of any embedded threshold element arrive in the same clock phase. This can be done by inserting threshold buffers, wherever needed, in the network. Suppose that all primary input signals arrive in the same clock phase. Examining the threshold network from the primary inputs to circuit outputs, if a node fans out to several nodes and those fanout nodes are not at the same level of the network, one needs to insert buffers to make sure that all the input signals of a node arrive in the same clock phase.

Example Consider a threshold network, shown in Fig. 7.14a, that implements a full adder, with switching functions $c_o = ab + ac_i + bc_i$ and $s = c'_o a + c'_o b + c'_o c_i + abc_i$. We observe that the inputs a , b , c_i , and c_o to node s do not arrive in the same clock phase. After inserting three buffers, as shown in Fig. 7.14b, all input signals of each node in the network arrive in the same clock phase. A MOBILE implementation of a threshold buffer and its symbol are shown in Fig. 7.15.

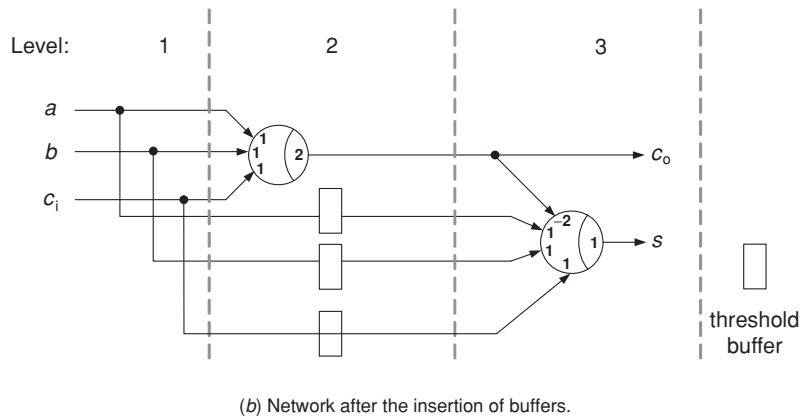
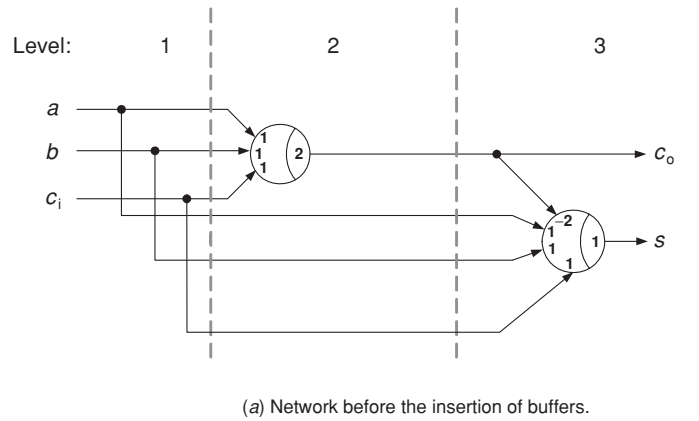


Fig. 7.14 Mapping a threshold network to MOBILEs.

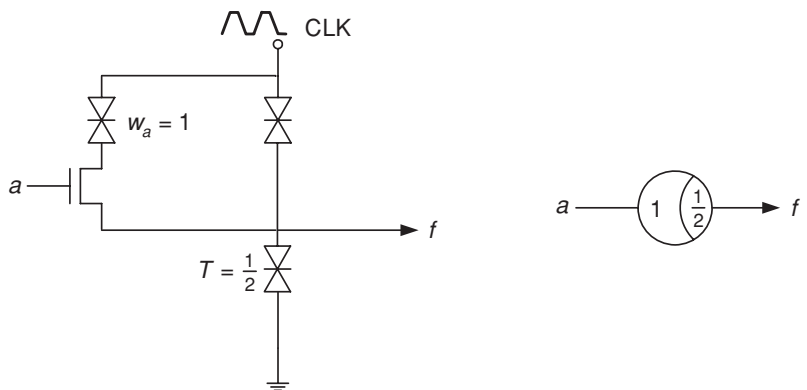


Fig. 7.15 A MOBILE threshold buffer and its symbol.

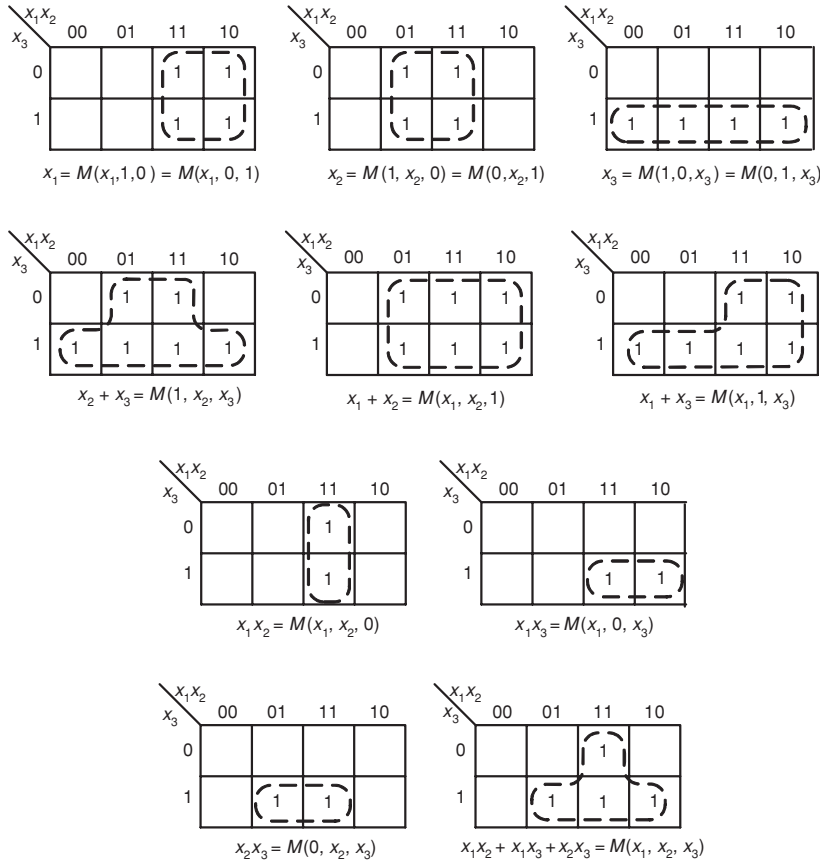


Fig. 7.16 Realizable patterns for majority gates.

Synthesis of multi-level majority and minority networks

Majority and minority gates are also threshold elements. In this section, we shall discuss a synthesis procedure specifically targeted towards multi-level majority network realization. Using De Morgan's theorem, this procedure is trivially applicable to minority network synthesis as well.

Assuming that the constants 0 and 1 are available as inputs, Fig. 7.16 shows all the positive functions that can be realized by a majority gate. A pattern of 1-cells is called a *realizable pattern* if it can be realized by a majority gate. Note that these are slightly different from the admissible patterns shown in Fig. 7.9. Some admissible patterns shown in Fig. 7.9 are realizable by threshold elements but not by a majority gate. Figure 7.16 shows all realizable patterns of three-input positive functions. If we remove the restriction that the function

be positive then there are a total of 38 three-input functions that can be realized by a majority gate.

Example Consider a switching network that implements $f(x_1, x_2, x_3) = x'_1x'_2x'_3 + x'_1x_2x_3 + x_1x_2x'_3 + x_1x'_2x_3$. A straightforward, but naive, approach for constructing a majority network is to decompose the network into two-input AND and OR gates since we know that such gates can be easily implemented by “reduced” majority gates (recall that a majority gate with one input tied to 0 (1) realizes an AND (OR) gate). For this function, the decomposed two-input AND–OR-gate-based network is shown in Fig. 7.17a. It contains 11 gates (each gate being a “reduced” majority gate) and four levels. However, if we can make full use of all three inputs of a majority gate then the number of gates and levels may be reduced. Such an implementation is shown in Fig. 7.17b, which consists of only four majority gates and two levels. An equivalent minority gate implementation is shown in Fig. 7.17c; this can be easily derived from the majority network by using De Morgan’s theorem.

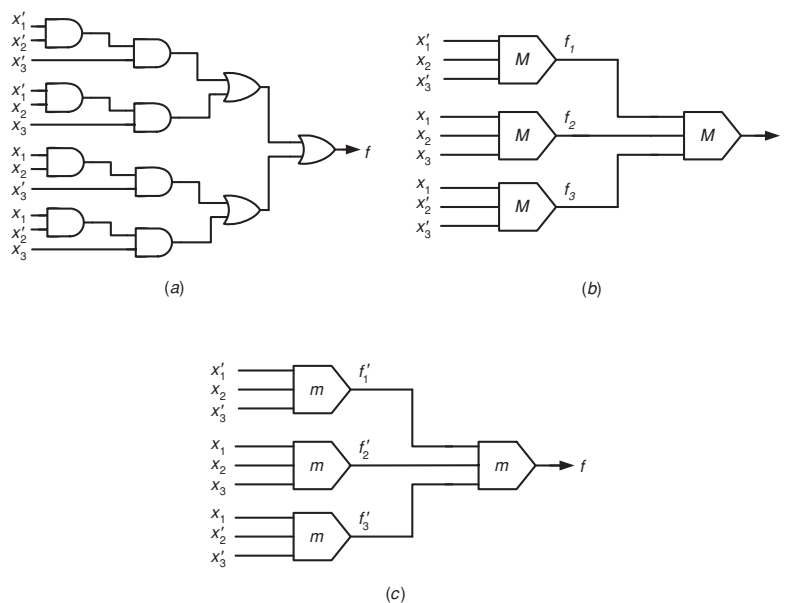


Fig. 7.17 (a) A two-input AND–OR-gate-based network, (b) the majority network, and (c) the minority network [21] © 2007, IEEE.

We next describe a synthesis procedure for multi-level majority networks. Just as in the case of threshold network synthesis, an algebraically factored multi-output combinational network G is also a good starting point for majority network synthesis. The procedure begins by the preprocessing of network G , during which it is decomposed into a network in which no node has more

than three inputs. Then, each node in the decomposed network is checked to determine whether it is a majority function. If it is, we proceed to synthesize the next node. Otherwise we check to see whether there exists a common literal in all the product terms of the node function. If one exists, we factor this literal out. An AND–OR mapping is then performed on the factored node. If no common literal exists, we check to see whether this node can be implemented with fewer than four AND or OR gates. If this is the case, we perform an AND–OR mapping on this node. Otherwise, we map the node onto at most four majority gates using a Karnaugh-map-based method. It is known that all functions of three variables can be realized by at most four majority gates in two levels. The procedure terminates when all the nodes in the decomposed network have been synthesized.

Example Consider $f = x_1x'_2 + x'_2x_3$. If we use AND–OR mapping, three majority gates are needed for f as $f_1 = x_1x'_2$, $f_2 = x'_2x_3$, and $f = f_1 + f_2$. However, since the literal x'_2 appears in both the product terms of f , it can be factored out. Node f can, therefore, be expressed as $f = f_1x'_2$, where $f_1 = x_1 + x_3$, thus requiring only two majority gates.

The map-based method is described next. First, we obtain the map for the logic function of node n , which is a function of at most three inputs. Next we find a realizable pattern in the map, which gives the first majority function f_1 . Then we try to find a second realizable pattern based on the first realizable pattern and the original map of node n . This realizable pattern gives the second majority function f_2 . Finally, from the two previously found realizable patterns and the original map, we find the third realizable pattern. This realizable pattern gives us the third majority function f_3 . These three majority functions are chosen in such a way that original node n can be represented as their majority function, i.e., $n = M(f_1, f_2, f_3) = f_1f_2 + f_2f_3 + f_1f_3$.

The chosen realizable pattern for f_1 can contain “make-up” minterms that are not minterms of n . After finding f_1 , we use the following rule for finding f_2 and f_3 . A minterm (maxterm) of n must also be a minterm (maxterm) of at least two of the three functions f_1 , f_2 , and f_3 . This rule is enforced by defining two sets $\psi 1$ and $\psi 0$. For finding f_2 , the set $\psi 1$ is obtained as follows. If a minterm of n is not a minterm of f_1 , add this minterm to $\psi 1$. Similarly, for finding f_2 , the set $\psi 0$ is obtained as follows. If a maxterm of n is not a maxterm of f_1 , add this maxterm to $\psi 0$. When picking a realizable pattern for f_2 , we need to make sure that the 1's in $\psi 1$ are included in the pattern and that the 0's in $\psi 0$ are not. For finding f_3 , the sets $\psi 1$ and $\psi 0$ are updated as follows. If a minterm (maxterm) of node n is not a minterm (maxterm) of both f_1 and f_2 , add this minterm (maxterm) to $\psi 1$ ($\psi 0$). Again, when picking a realizable pattern for f_3 , we need to make sure that the 1's in $\psi 1$ are included in the pattern and that the 0's in $\psi 0$ are not.

It can be seen that f_3 is not guaranteed to be found on the basis of the two previously chosen functions f_1 and f_2 . Hence, if we fail to find f_3 from the current choices of f_1 and f_2 , backtracking is needed to find a new f_2 . If f_3 can still not be found after a few tries, the AND–OR mapping method can be used to speed up the process.

Example Consider the function $f(x_1, x_2, x_3) = x'_1x'_2x'_3 + x'_1x_2x_3 + x_1x_2x'_3 + x_1x'_2x_3$ once again. The corresponding maps are shown in Fig. 7.18. As can be seen, one make-up minterm is needed for finding the realizable pattern for f_1 . This make-up minterm $x'_1x_2x'_3$ is shown in italic in Fig. 7.18b. Then $\psi 1$ and $\psi 0$ are computed. The second make-up minterm, $x_1x_2x_3$, is needed for the second realizable pattern for f_2 , as shown in Fig. 7.18e. Finally, the third realizable pattern for f_3 is found. We then obtain the majority network shown earlier in Fig. 7.17b.

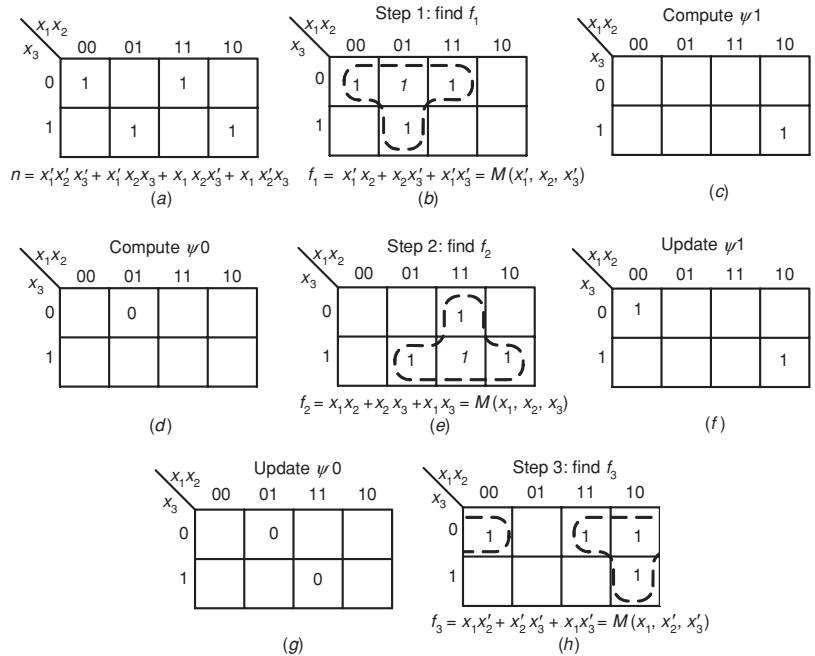
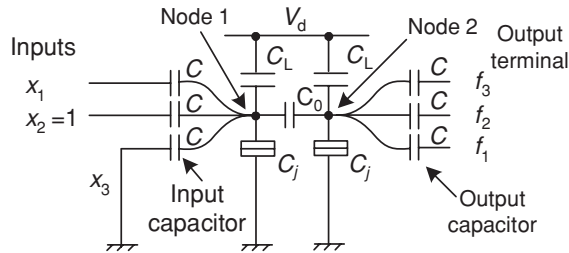


Fig. 7.18 Map-based majority network synthesis [21] © 2007, IEEE.

Mapping of majority networks to QCA, SEB, or TPL

The efficient and automatic mapping of majority networks to networks of quantum cellular automata (QCA) cells is still an ongoing area of research.

A multi-level majority network can be implemented with single-electron boxes (SEBs) by letting the output capacitor of one majority gate act as the

Fig. 7.19 An SEB buffer [16].

input capacitor of the following gate. A three-phase overlapping clock can be used for successive gates. Thus, the mapped majority network needs to be partitioned into three groups, where each group is activated by one phase of the clock. An overlap between successive clock phases allows the output of a stage to be established while the preceding stage maintains its output during its holding period. In order to make an SEB-based majority network function correctly under three-phase overlapping clocking, we have to make sure that all the input signals of any embedded majority gate arrive in the same clock phase. This is a problem similar to that encountered in the mapping of threshold networks to MOBILEs, requiring the insertion of buffers. Figure 7.19 shows an implementation of an SEB buffer.

When mapping a minority network onto tunneling phase logic (TPL) primitives, we have to consider the fanout restriction. So far, only a fanout of at most three has been demonstrated for TPL. This restriction can be satisfied by post-processing the minority network that has been generated without taking into account the fanout restriction. If a node violates the fanout restriction, new nodes are generated by duplicating that node. The inputs and outputs of these nodes are updated to satisfy the fanout restriction.

By now the reader can appreciate the importance of threshold and majority or minority networks in circuit design for various nanotechnologies. This area is likely to attract considerable attention in the coming years.

Notes and references

McNaughton [11] studied the properties of unate functions and established the unateness of a function as a necessary condition for its single-threshold-element realizability. Various properties of threshold functions, as well as synthesis procedures, were studied by Elgot [5], Muroga *et al.* [14], Winder [19], Dertouzos [4], and Lewis and Coates [7]. Synthesis techniques were presented by Oliveira and Sangiovanni-Vincentelli for two-level threshold logic [15] and by Zhang *et al.* [20] for multi-level threshold logic. Synthesis techniques for small majority networks were presented by Akers [1], Miller and Winder [12], and Muroga [13]. Synthesis techniques for large multi-level majority and minority networks were presented by Zhang *et al.* [21]. Very large scale integrated

(VLSI) implementations of threshold logic were surveyed by Beiu *et al.* [2]. An excellent treatment of threshold logic can be found in the book by Muroga [13].

Resonant-tunneling-diode (RTD) based threshold networks were discussed by Pacha *et al.* [17], Chen *et al.* [3], Maezawa *et al.* [8], Mazumder *et al.* [10], and Mathews *et al.* [9], QCA-based majority gates by Tougaw and Lent [18], SEB-based majority networks by Oya *et al.* [16], and TPL-based minority gates by Fahmy and Kiehl [6].

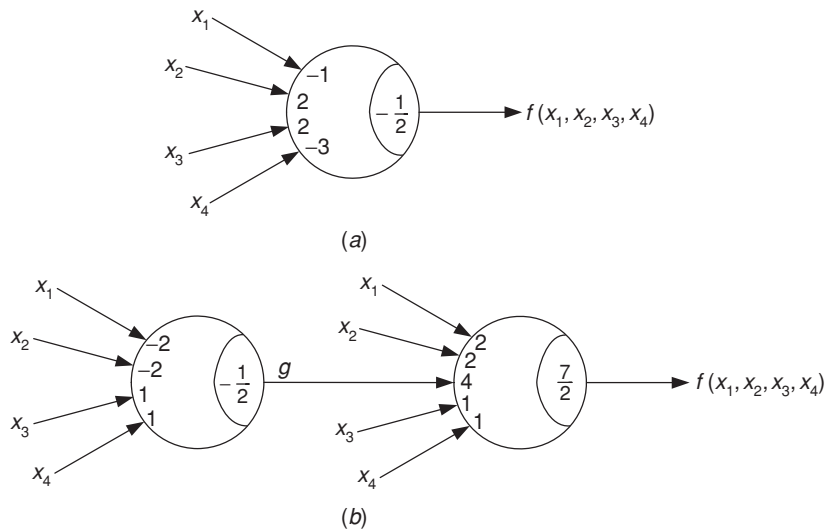
- [1] Akers, S. B.: "Synthesis of combinational logic using three-input majority gates," in *Proc. Third Annual Symp. Switching Circuit Theory & Logical Design*, pp. 149–157, October 1962.
- [2] Beiu, V., J. M. Quintana, and M. J. Avedillo: "VLSI implementations of threshold logic – a comprehensive survey," *IEEE Trans. Neural Networks*, vol. 14, pp. 11 217–11 243, September 2003.
- [3] Chen, K. J., K. Maezawa, and M. Yamamoto: "InP-based high-performance monostable–bistable transition logic elements (MOBILEs) using integrated multiple-input resonant-tunneling devices," *IEEE Electron Device Letters*, vol. 17, no. 3, pp. 127–129, March 1996.
- [4] Dertouzos, M.: *Threshold Logic: A Synthesis Approach*, MIT Press, Cambridge MA, 1965.
- [5] Elgot, C. C.: "Truth functions realizable by single threshold organs," in *Proc. Ann. Symp. Switching Circuit Theory and Logical Design*, 1960; also in *AIEE Publ.*, S-134, pp. 225–245, September 1961.
- [6] Fahmy, H. A., and R. A. Kiehl: "Complete logic family using tunneling-phase-logic devices," in *Proc. Int. Conf. Microelectronics*, pp. 22–24, November 1999.
- [7] Lewis, P. M., and C. L. Coates: *Threshold Logic*, John Wiley & Sons, New York, 1967.
- [8] Maezawa, K., H. Matsuzaki, M. Yamamoto, and T. Otsuji: "High-speed and low-power operation of a resonant tunneling logic gate (MOBILE)," *IEEE Electron Device Letters*, vol. 19, no. 3, pp. 80–82, March 1998.
- [9] Mathews, R. H. *et al.*: "A new RTD–FET logic family," *Proc. IEEE*, vol. 87, no. 4, pp. 596–605, April 1999.
- [10] Mazumder, P., S. Kulkarni, M. Bhattacharya, J. P. Sun, and G. I. Haddad: "Digital circuit applications of resonant tunneling devices," *Proc. IEEE*, vol. 86, no. 4, pp. 664–668, April 1998.
- [11] McNaughton, R.: "Unate truth functions," *IRE Trans. Electronic Computers*, vol. EC-10, pp. 1–6, March 1961.
- [12] Miller, H. S., and R. O. Winder: "Majority logic synthesis by geometric methods," *IRE Trans. Electronic Computers*, vol. EC-11, no. 1, pp. 89–90, February 1962.
- [13] Muroga, S.: *Threshold Logic and its Applications*, John Wiley, New York, 1971.
- [14] Muroga, S., I. Toda, and S. Takasu: "Theory of majority decision elements," *J. Franklin Inst.*, vol. 271, pp. 376–418, May 1961.
- [15] Oliveira, A. L., and A. L. Sangiovanni-Vincentelli: "LSAT – an algorithm for the synthesis of two level threshold gate networks," in *Proc. Int. Conf. Computer-Aided Design*, pp. 130–133, November 1991.
- [16] Oya, T., T. Asai, T. Fukui, and Y. Amemiya: "A majority-logic nanodevice using a balanced pair of single-electron boxes," *J. Nanosci. Nanotech.*, vol. 2, nos. 3–4, pp. 333–342, June–August 2002.

- [17] Pacha, C., W. Prost, F. J. Tegude, P. Glösekötter, and K. F. Gosser: “Resonant tunneling device logic: a circuit designer’s perspective,” in *Proc. European Conf. Circuit Theory & Design*, August 2001.
- [18] Tougaw, P. D., and C. S. Lent: “Logical devices implemented using quantum cellular automata,” *J. Applied Physics*, vol. 75, no. 3, pp. 1811–1817, February 1994.
- [19] Winder, R. O.: “Threshold logic,” doctoral dissertation for the Mathematics Department, Princeton University, May 1962.
- [20] Zhang, R., P. Gupta, L. Zhong, and N. K. Jha: “Threshold network synthesis and optimization and its application to nanotechnologies,” *IEEE Trans. Computer-Aided Design*, vol. 23, no. 1, pp. 107–118, January 2005.
- [21] Zhang, R., P. Gupta, and N. K. Jha: “Majority and minority network synthesis with application to QCA, SET and TPL based nanotechnologies,” *IEEE Trans. Computer-Aided Design*, vol. 25, no. 7, pp. 1233–1245, July 2007.

Problems

Problem 7.1. Find the function $f(x_1, x_2, x_3, x_4)$ realized by each of the threshold networks shown in Fig. P7.1. Show the map of each function.

Fig. P7.1



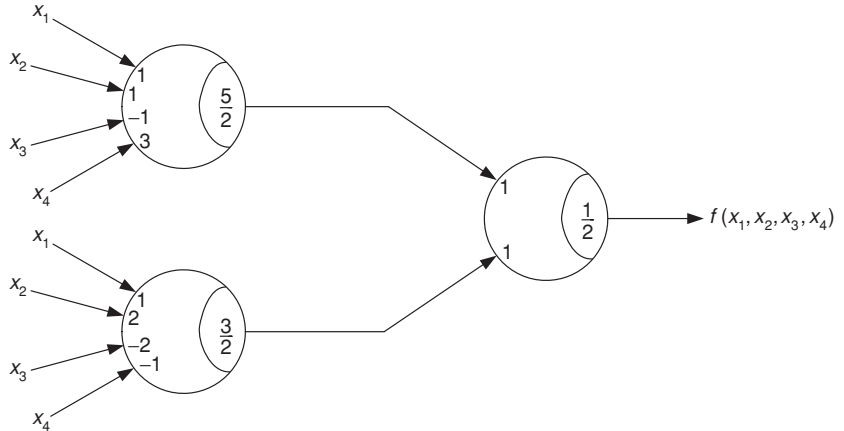
Problem 7.2. By examining the relevant linear inequalities, determine which of the following functions is a threshold function (see the discussion after Eq. (7.3)) and, for each one that is, find the corresponding weight–threshold vector:

- (a) $f_1(x_1, x_2, x_3) = \sum(1, 2, 3, 7)$;
- (b) $f_2(x_1, x_2, x_3) = \sum(0, 2, 4, 5, 6)$;
- (c) $f_3(x_1, x_2, x_3) = \sum(0, 3, 5, 6)$.

Problem 7.3. For each of the functions of Problem 7.2 that is realizable by a single threshold element, find a realization for $f'(x'_1, x_2, x_3)$.

Problem 7.4

- (a) Obtain the function $f(x_1, x_2, x_3, x_4)$ realized by the network shown in Fig. P7.4.
 (b) Show that $f(x_1, x_2, x_3, x_4)$ can be realized by a single threshold element. Find this element.

Fig. P7.4

Problem 7.5. Consider the type of threshold functions for which all the weights are equal, that is, $w_1 = w_2 = \dots = w_n$. In particular, consider those $f(x_1, x_2, \dots, x_n)$ for which

$$f(x_1, x_2, \dots, x_n) = 1 \text{ if and only if } \sum_{i=1}^n x_i \geq T/w,$$

$$f(x_1, x_2, \dots, x_n) = 0 \text{ if and only if } \sum_{i=1}^n x_i < T/w.$$

Determine the value of f when (1) $T/w = 0$, (2) $T/w > n$, (3) $0 < T/w \leq n$.

Problem 7.6

- (a) Prove that if $f(x_1, x_2, \dots, x_n)$ is a threshold function with weight–threshold vector $V_1 = \{w_1, w_2, \dots, w_n; T\}$ then its dual, $f_d(x_1, x_2, \dots, x_n)$, is also a threshold function. Determine its weight–threshold vector.
 (b) Prove that if f is a threshold function then so is

$$g = x'_i f + x_i f_d,$$

where x_i may or may not be a member of set $\{x_1, x_2, \dots, x_n\}$. Find the weight–threshold vector of g .

Problem 7.7

- (a) Prove that if $f(x_1, x_2, \dots, x_n)$ is a threshold function with weight–threshold vector $\{w_1, w_2, \dots, w_n; T\}$ then $G = x_p + f$ and $H = x_p f$ are also threshold functions, where x_p may or may not be a member of the set $\{x_1, x_2, \dots, x_n\}$. Find w_p and the weight–threshold vectors for G and H .

Hint: Define two numbers M and N such that

$$M = \sum_{\text{all positive weights}} w_i, \quad N = \sum_{\text{all negative weights}} w_i,$$

and, if convenient, use them in the expression for w_p .

- (b) Given that $f(x_1, x_2, x_3) = x_1x_3 + x'_3$ is a threshold function, use the result of (a) to show that

$$f_1(x_1, x_2, x_3, x_4) = x_2 + x'_3 + x_4$$

and

$$f_2(x_1, x_2, x_3, x_4) = x_1x_2x_4 + x_2x'_3x_4$$

are threshold functions. Give the weight–threshold vector in each case.

Problem 7.8. The functions $f_1(x_1, x_2, x_3)$ and $f_2(x_1, x_2, x_3)$ are each realizable by a single threshold element. The weight–threshold vectors of these elements are, respectively,

$$V_1 = \{-1, -1, 1; 0\}, \quad V_2 = \{1, 2, -1; 2\}.$$

Is the function

$$f(x_1, x_2, x_3, x_4) = x_4f_1(x_1, x_2, x_3) + x'_4f_2(x_1, x_2, x_3)$$

realizable by a single threshold element? If yes, give its weight–threshold vector. If not, indicate clearly why it is not a threshold function.

Problem 7.9. Prove that if an expression corresponding to a function that is positive (negative) in x_i contains both x_i and x'_i then every occurrence of the literal x'_i (x_i) is redundant.

Problem 7.10

- (a) Prove that a necessary and sufficient condition for a function to be unate is that all its prime implicants intersect in a common implicant. (For example, the common implicant for

$$f_1(x_1, x_2, x_3, x_4) = \sum(0, 1, 3, 4, 5, 6, 7, 12, 13)$$

is the minterm 5.)

- (b) Prove that the minimal sum-of-products form of a unate function is unique and consists of all prime implicants.

Hint: Use Problem 7.9 and the fact that the conjunction of all product prime implicants of a unate function cannot be zero.

Problem 7.11. Use the result of Problem 7.10 to determine which of the following functions is unate and show its minimal form:

- (a) $f_1(x_1, x_2, x_3, x_4) = \sum(1, 2, 3, 8, 9, 10, 11, 12, 14);$
 (b) $f_2(x_1, x_2, x_3, x_4) = \sum(0, 8, 9, 10, 11, 12, 13, 14);$
 (c) $f_3(x_1, x_2, x_3, x_4) = \sum(2, 3, 6, 10, 11, 12, 14, 15).$

Problem 7.12. For each of the following functions, find a two-element cascade realization of the type illustrated in Fig. 7.10c:

- (a) $f_1(x_1, x_2, x_3, x_4) = \sum(2, 3, 6, 7, 8, 9, 13, 15);$
 (b) $f_2(x_1, x_2, x_3, x_4) = \sum(0, 3, 4, 5, 6, 7, 8, 11, 12, 15).$

Problem 7.13. The MOBILE implementation of a full adder, if based on the threshold network in Fig. 7.14*b*, requires five threshold gates. Obtain a MOBILE implementation of a full adder that requires only four threshold gates.

Hint: It does not contain any threshold buffers.

Problem 7.14. Prove that any three-variable function can be implemented with at most four majority gates in a two-level network.

Problem 7.15. Assuming that only uncomplemented inputs are available, implement a full adder with only three majority gates and two inverters.

Problem 7.16. Implement the function $f = x'_1x'_2x_3 + x_1x'_3 + x_2x'_3 + x_1x_2$ with at most four minority gates.

8

Testing of combinational circuits

The problem of determining whether a digital circuit operates correctly is of both theoretical interest and practical concern. Present-day digital systems may be disabled by almost any internal failure. Failures are caused by faults that are initially manifested as errors and finally as failures. In this chapter, we shall study various fault models, techniques for generating tests, and logic synthesis techniques that ensure testability with respect to various types of fault.

8.1 Fault models

In order to alleviate the complexity of test generation, one needs to model the actual defects that may occur in a chip with fault models at higher levels of abstraction. This process of fault modeling considerably reduces the burden of testing because it obviates the need for deriving tests for each possible defect. This is due to the fact that many physical defects map to a single fault at the higher level.

Faults may change the logic values at some internal lines in the integrated circuit, or they may result in a change in the voltage or current levels. They may also change the temporal behavior of the circuit.

Currently, most popular fault models are described at the structure and switch levels of the integrated-circuit design hierarchy. In this section, we shall examine these fault models.

Structural fault models

In structural testing we need to make sure that the interconnections in the given structure are fault-free and are able to carry both 0 and 1 signals. The *stuck-at fault model* is directly derived from these requirements. A line is said to be stuck-at 0 (*s-a-0*) or stuck-at 1 (*s-a-1*) if the line remains fixed at a low or high voltage level, respectively (assuming positive logic). A stuck-at fault does not necessarily imply that the line is shorted to the ground or power line. It could be a model for many other cuts and shorts internal or external to a gate. For

example, a cut on the stem of a fanout may result in an s - a -0 fault on all its fanout branches. However, a cut on just one fanout branch may result in an s - a -0 fault on just that fanout branch. Therefore, stuck-at faults on stems and fanout branches have to be considered separately.

If the stuck-at fault is assumed to occur on only one line in the circuit, it is said to belong to the *single stuck-at fault model*. Otherwise, if stuck-at faults are simultaneously present on more than one line in the circuit, the faults are said to belong to the *multiple stuck-at fault model*. If the circuit has k lines, it can have $2k$ single stuck-at faults, two for each line. However, the number of multiple stuck-at faults is $3^k - 1$ because there are three possibilities for each line (s - a -0, s - a -1, fault-free), and the resultant 3^k cases include the case where all lines are fault-free. Clearly, even for relatively small values of k , testing for all multiple stuck-at faults is impossible. However, as we shall see later in this chapter, synthesis methods exist that can guarantee circuit testability with respect to all multiple stuck-at faults.

Example Consider the circuit shown in Fig. 8.1. Assume first that only the line c_1 has an s - a -0 fault. To test for this single stuck-at fault, we can apply $(x_1, x_2, x_3, x_4) = (1, 1, 0, 1)$ to the circuit. In the fault-free case $f = 1$ and in the presence of the fault $f = 0$. Thus, the fault is detected. If a c_1 s - a -0 fault, a c_2 s - a -0 fault, and an x_3 s - a -1 fault are simultaneously present then we have a multiple stuck-at fault. This multiple stuck-at fault is also detected by the test vector $(1, 1, 0, 1)$. In fact, one can check that any vector that makes $f = 1$ in the fault-free case will detect this fault.

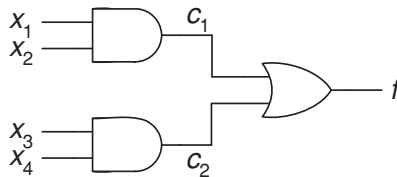
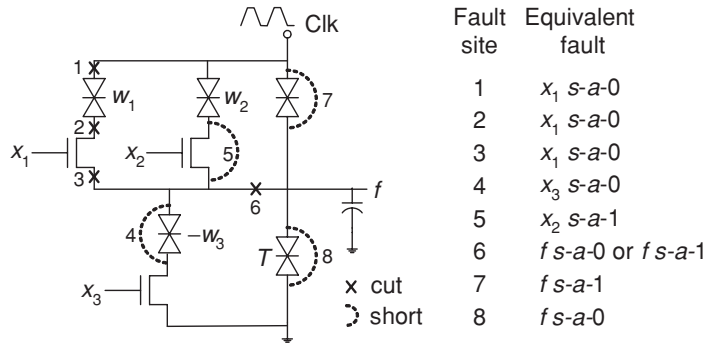


Fig. 8.1 A logic circuit with stuck-at faults.

The stuck-at fault model is not only the most popular one for current technologies but will also be useful for future nanotechnologies. As an example, consider the MOBILE shown in Fig. 8.2.¹ It shows the cuts and shorts that commonly occur in a defective chip. These defects can be modeled as stuck-at faults at the threshold gate level. A cut (e.g., at defect sites 1, 2, and 3) on an HFET or on a line connecting the RTD and HFET makes the line nonconducting and can be modeled as an s - a -0. Similarly, a short across an RTD (site 4) or the driver RTD (site 8) can also be modeled as an s - a -0 fault because in

¹ Recall from Chapter 7 that a MOBILE implements a threshold gate.

Fig. 8.2 Fault modeling for a MOBILE threshold gate [11] © 2008, IEEE.



the former the input weight becomes zero while in the latter there is a direct connection between the output and ground. A cut at site 6 represents an s -a-1 or s -a-0 fault depending on whether the threshold of the gate is less than 0 or greater than or equal to 0. However, defects at sites 5 and 7 can be modeled as s -a-1 faults. A short across the HFET will make it conduct permanently whereas a direct connection between the output and bias voltage makes the fault appear as an s -a-1 when the MOBILE is active.

Switch-level fault models

Switch-level fault modeling deals with faults in transistors and interconnects in a switch-level description of a circuit. This fault model has mostly been used with MOS technologies, specifically CMOS technology. The most prominent members in this category are the stuck-open, stuck-on, and bridging fault models.

The stuck-open fault model

A *stuck-open* fault refers to a transistor that becomes permanently nonconducting owing to some defect.

Example Consider the two-input static CMOS NOR gate shown in Fig. 8.3a. This gate consists of an nMOS network containing transistors Q_1 and Q_2 and a pMOS network containing transistors Q_3 and Q_4 . Recall that an nMOS (pMOS) transistor conducts when the value 1 (0) is fed to its input, otherwise it remains nonconducting. Suppose that a defect d_1 causes an open connection in the gate, as shown. This prevents Q_1 from conducting and is thus said to result in a stuck-open fault in Q_1 . Let us see what happens when we apply an exhaustive set of input vectors to the faulty gate in the sequence $(x_1, x_2) = \{(0, 0), (0, 1), (1, 0), (1, 1)\}$. When $(0, 0)$ is applied, Q_3 and Q_4 conduct and the output $f = 1$. Next, with the application of $(0, 1)$ f gets pulled down to the value 0 through Q_2 . When $(1, 0)$ is applied, there is no conduction path from f to V_{ss} because

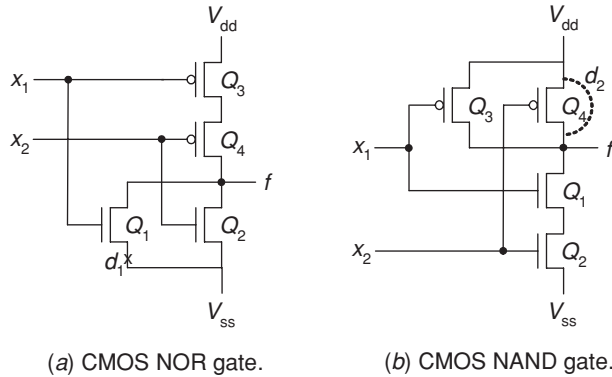


Fig. 8.3 Two-input static CMOS gates.

of the stuck-open fault in Q_1 . Therefore f retains its previous value, which is 0. Finally, with the application of the vector $(1, 1)$, $f = 0$ because of the conduction path through Q_2 . Therefore, we obtain the correct output values at f in the presence of the stuck-open fault even after the application of the exhaustive test set containing all two-bit input vectors. This is due to the fact that the stuck-open fault has forced the gate to behave in a sequential fashion.

Thus, in order to test the circuit for a stuck-open fault, we need a sequence of vectors. Usually *two-pattern tests*, consisting of an *initialization vector* and a *test vector*, are used. Because the CMOS gate can retain its previous value at its output in the presence of a stuck-open fault, the initialization vector is used to initialize the output to the value that is the complement of the value expected when the stuck-open fault is tested.

Example To detect a stuck-open fault caused by the defect d_1 in the NOR gate mentioned above, one needs to activate a conduction path through the faulty transistor without activating any parallel path. There is only one such test vector: $(1, 0)$. Since, in the fault-free case, for this input vector we expect $f = 0$, the initialization vector should make $f = 1$. There is only one such vector: $(0, 0)$. Therefore, $\{(0, 0), (1, 0)\}$ is a unique two-pattern test for this stuck-open fault. When the fault is present, we get the value 1 at the output when $(1, 0)$ is applied. Thus, the fault is detected. Similarly, the two-pattern test $\{(0, 0), (0, 1)\}$ can detect the stuck-open fault in transistor Q_2 . For detecting stuck-open faults in transistors Q_3 or Q_4 , $\{(0, 1), (0, 0)\}$ or $\{(1, 0), (0, 0)\}$ can be used. Therefore, one possible test sequence that detects all four stuck-open faults in the NOR gate is $\{(0, 0), (0, 1), (0, 0), (1, 0)\}$.

The stuck-on fault model

If a transistor has become permanently conducting due to some defect, it is said to have a *stuck-on* fault.

Example Consider the two-input NAND gate shown in Fig. 8.3b. Suppose that owing to a defect d_2 , the source and drain of transistor Q_4 become shorted, as shown. This results in a stuck-on fault in this transistor. In order to try to test for this fault, the only vector we could possibly apply to the NAND gate is (1, 1). In the presence of the fault, transistors Q_1 , Q_2 and Q_4 will conduct. This will result in some intermediate voltage at the output. The exact value of this voltage will depend on the on-resistances of the nMOS and pMOS transistors. If it maps to the value 1 at the output then the stuck-on fault is detected, otherwise it is not. Now suppose that the only fault present in the gate is a stuck-on fault in transistor Q_2 . In order to try to test for this fault, the only vector we could possibly apply is (1, 0). In the presence of the fault, again the same set of transistors, Q_1 , Q_2 and Q_4 , will conduct. However, this time we would like the intermediate voltage to map to the value 0 in order to detect the fault. Since the same set of transistors is activated in both cases, the resultant voltage at the output will be the same. Therefore, because of the contradictory requirements for the detection of the stuck-on faults in Q_4 and Q_2 , only one of these two faults can be detected.

The above example illustrates that simply monitoring the logic value at the output of the gate, called *logic monitoring*, is not enough if we are interested in detecting all single stuck-on faults in it. Fortunately, a method called I_{DDQ} testing is available, which measures the current drawn by the circuit and can ensure the detection of all stuck-on faults. This method is based on the fact that, whenever there is a conduction path from V_{dd} to V_{ss} due to a stuck-on fault, the current drawn by the circuit increases by several orders of magnitude compared to the fault-free case. Thus, with the help of an I_{DDQ} (quiescent drain current) current monitor, such faults can be detected. The disadvantage of I_{DDQ} testing is that it is slow, since it may be possible to feed vectors only at the rate of a few KHz, whereas in logic monitoring, it may be possible to apply vectors at tens or hundreds of MHz.

The bridging fault model

With shrinking geometries, the percentage of chip defects causing shorts, also called *bridging faults*, has been on the increase.

Example Consider the bridging fault between lines c_1 and c_2 in the circuit shown in Fig. 8.4. Such a fault will be denoted by $\langle c_1, c_2 \rangle$. For some input vectors this fault will create a conducting path from V_{dd} to V_{ss} . For example, for $(x_1, x_2, x_3) = (1, 1, 0)$, there is a path from V_{dd} to V_{ss} through

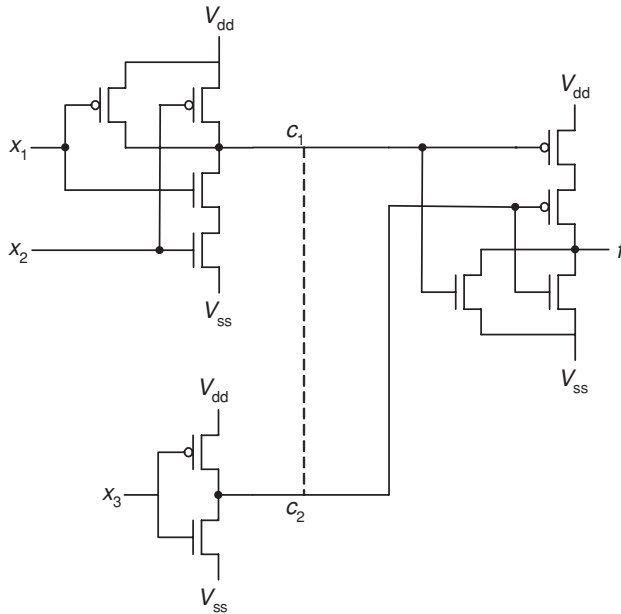


Fig. 8.4 Bridging fault in a static CMOS circuit.

the pMOS network of the inverter, the fault, and the nMOS network of the NAND gate. During fault-free operation, this vector causes opposite values to appear at c_1 and c_2 , i.e., $c_1 = 0$ and $c_2 = 1$. When the fault is present, this will result in an intermediate voltage at the bridged lines. Whether this results in the values 0 or 1 at these lines depends on the relative impedances of the two networks. The resultant value may also differ from one vector to another. For example, (0, 1, 1) also creates a conduction path from V_{dd} to V_{ss} .² However, it is possible that the shorted lines have the value 1 for the vector (1, 1, 0) and the value 0 for vector (0, 1, 1). Furthermore, different gates fed by the shorted lines may interpret the intermediate voltage on these lines as different logic values.

Even though it is clear that bridging faults in CMOS circuits cannot be guaranteed to be detected by logic monitoring, they can be detected by I_{DDQ} testing since they activate a path from V_{dd} to V_{ss} .

Bridging faults are sometimes categorized as feedback or nonfeedback faults. If one or more feedback paths are created in the circuit owing to the fault then it is called a *feedback* fault, otherwise a *nonfeedback* fault.

Delay fault models

Instead of affecting the logical behavior of the circuit, a fault may affect its temporal behavior only. Such faults are called delay faults. Delay faults adversely

² During fault-free operation, this vector causes opposite values to appear at c_1 and c_2 , i.e., $c_1 = 1$ and $c_2 = 0$.

affect the propagation delays of signals in the circuit. Hence, an incorrect value may be latched at the output. With the continuing emphasis on designing circuits for very high performance, delay fault models have attracted wide attention.

Two types of delay fault models are typically used.

- The *transition fault model* A circuit is said to have a transition fault in some gate if the output of the gate has a lumped delay fault that delays its $0 \rightarrow 1$ or $1 \rightarrow 0$ transition by more than the system clock period.
- The *path delay fault model* A circuit is said to have a path delay fault if there exists a path from a primary input to a circuit output in it which is slow to propagate a $0 \rightarrow 1$ or $1 \rightarrow 0$ transition from its input to its output.

Clearly, the path delay fault model is the more general of the two models, as it models the cumulative effect of the delay variations of the gates and wires along the path. However, because the number of paths in a circuit can be very large, the path delay fault model may require much more time for test generation and test application than the transition fault model.

Because of the need to propagate a transition, delay faults, just like stuck-open faults, require two-pattern tests.

Example Consider the circuit shown in Fig. 8.5. A path is shown in bold from x_2 to f_1 . If this path significantly delays the propagation of the $0 \rightarrow 1$ or $1 \rightarrow 0$ transition launched at x_2 then the circuit is said to have a path delay fault.

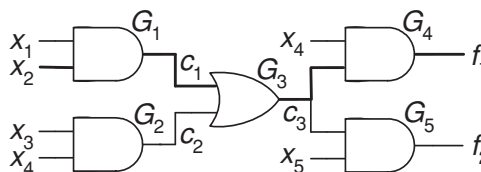


Fig. 8.5 A circuit for illustrating delay faults.

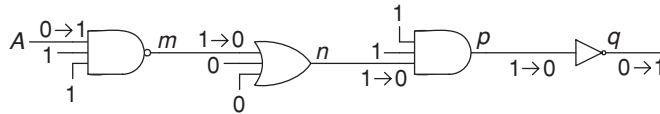
Next, consider gate G_3 . If either logic transition, i.e., $0 \rightarrow 1$ or $1 \rightarrow 0$, through every path going through G_3 gets significantly delayed then G_3 is said to have a transition fault. Note that there are eight such paths, four to f_1 and four to f_2 .

When a $0 \rightarrow 1$ ($1 \rightarrow 0$) transition is delayed, it is said to be a *slow-to-rise* (*slow-to-fall*) transition fault.

8.2 Structural testing

Structural testing refers to the detection of faults on the interconnections in the structure of the circuit. This is done by finding input test vectors that

Fig. 8.6 Part of a circuit describing a sensitized path.



expose the fault at circuit outputs by causing an error (an unexpected output response) to occur. Typically, the structure is assumed to be a gate-level description and the faults targeted are of the single stuck-at kind. In this section we shall first discuss the basic concepts employed in this area and then use them to discuss the *D*-algorithm, which is a complete structural test generation algorithm.

In testing, one frequently comes across the following three terms: the test generation time, the test application time, and the fault coverage. The *test generation time* refers to the time it takes to generate the test set for a circuit on a computer. The *test application time* refers to the time it takes to apply the test vectors in the test set to the circuit under test. The *fault coverage* refers to the percentage of all the targeted faults that are actually detected by the derived test set.

Path sensitization

The main idea behind path sensitization can be illustrated by deriving a test vector that detects an *s-a-1* fault at input *A* of the circuit in Fig. 8.6. Suppose that this path is the only one from *A* to the circuit output. In order to test for an *s-a-1* fault at input *A*, it is necessary to apply a 0 to *A* and 1's to all the remaining inputs of the AND and NAND gates in the path, and 0's to all the remaining inputs of the OR and NOR gates along the path. This ensures that all the gates will allow the propagation of the signal from *A* to the circuit output, and that only this signal will reach the circuit output. This assignment of values is shown in Fig. 8.6. The path is now said to be *sensitized*.

If input *A* is *s-a-1* then *m* has an error that changes its value from 1 to 0, and this change propagates through connections *n* and *p* and causes *q* to change from 0 to 1. Clearly, in addition to detecting an *s-a-1* fault at *A*, this test vector also detects *s-a-0* faults at *m*, *n*, and *p*, and an *s-a-1* fault at *q*. An *s-a-0* fault at *A* is detected in a similar manner. The value 1 is applied to *A*, while the other gate input values remain the same as before. This second test vector will also detect a set of faults on this path that is complementary to the set detected by the previous test vector. Thus, the two test vectors together are sufficient to detect all *s-a-0* and *s-a-1* faults on this path.

The basic principles of the above method, which is also known as *one-dimensional path sensitization*, can be summarized as follows.

1. At the site of the fault, assign a logic value complementary to the fault being tested. That is, to test x_i for *s-a-0* assign $x_i = 1$, and to test it for *s-a-1* assign $x_i = 0$.

2. Select a path from the primary inputs through the site of the fault to a circuit output. The path is said to be sensitized if the inputs to the gates along the path are assigned values so as to propagate to the path output any error on the wires along the path. This process is called *error propagation*.
3. Determine the primary input values that produce all the necessary signal values specified in the preceding steps. This is accomplished by tracing the signals backward from each of the gates along the path to the primary inputs. This process is called *line justification* or *consistency*.

Example Suppose that we want to derive a test vector for an $s-a-1$ fault at line c_1 in the circuit in Fig. 8.7. Error propagation starts with assigning a 0 to c_1 and selecting a path to be sensitized. Let us choose to sensitize the path consisting of gates G_5 , G_7 , and G_9 to the output f_2 . Clearly, since G_5 and G_9 are OR gates, their other inputs (also called *side inputs*) must be 0. This completes error propagation and the path is now sensitized. Next, we need to justify the 0's at lines c_2 and c_7 at the primary inputs. The line c_7 can be made 0 by making $x_3 = x_4 = 0$. To make $c_2 = 0$, we have three choices at (x_1, x_2) , i.e., (0, 0), (0, 1), or (1, 0). If we choose (0, 0) then a test vector for a c_1 $s-a-1$ fault is

$$(x_1, x_2, x_3, x_4, x_5) = (0, 0, 0, 0, 1).$$

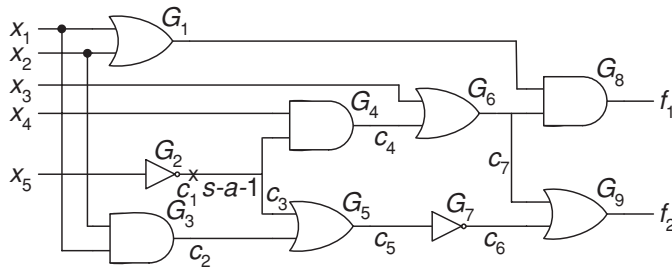


Fig. 8.7 Example of path sensitization.

If, in response to the above test vector, the circuit produces an output value $f_2 = 1$ then the fault in question does not exist. However, if $f_2 = 0$ then the circuit has a fault. This does not necessarily mean that c_1 is $s-a-1$, since such an erroneous output value can be caused by a c_3 or c_5 $s-a-1$ fault or by a c_6 or f_2 or x_5 $s-a-0$ fault.

An important concept that is useful in speeding up the test generation process is called implication. Given the logic value of some line in the circuit, *implication* determines the logic values uniquely implied at other lines in the circuit. This can be done in both the backward and forward directions. In the above example, knowing that the assignment $c_7 = 0$ has been made in the

error propagation step, backward implication determines that $x_3 = c_4 = 0$. The forward implication of $c_7 = 0$ determines that $f_1 = 0$. In this case, only the backward implication is helpful in arriving at a test vector. However, in general both forward and backward implications are helpful in speeding up test generation and should be performed after steps 1 and 2 in the path sensitization procedure given above.

In general there may be several possible choices of sensitized paths from the fault site to a circuit output. In the above example, one could have tried instead to sensitize the path through gates G_4 , G_6 , and G_9 or gates G_4 , G_6 , and G_8 . It may so happen that one choice leads to a conflict in the required logic values and then it may be necessary to *backtrack* and choose another path. Moreover, for a given sensitized path, there may be more than one way of specifying the input values so as to propagate the error along the path. This process may also involve backtracking.

A major advantage of the path sensitization method is that, as illustrated by Fig. 8.6, in many cases a test vector for a primary input is also a test vector for all the lines along the sensitized path to a circuit output. Consequently, if we can select a set of test vectors (called a *test set*) that sensitizes a set of paths containing all the lines in the circuit then it is sufficient to detect just those faults that appear at the primary inputs. However, when a circuit contains fanout, in particular reconvergent fanout, *one-dimensional path sensitization is not guaranteed always to generate a test vector even if one is known to exist* (see Problem 8.1). This has led to a more general *two-dimensional path sensitization* method called the *D*-algorithm that is complete, i.e., it guarantees finding a test vector if one exists.

Fault collapsing

The number of faults that need to be targeted for test generation can be significantly reduced through the process of *fault collapsing*. Consider a circuit whose fault-free output is f . Let f_α denote the circuit output in the presence of fault α . A test vector that detects α must clearly satisfy the condition

$$f \oplus f_\alpha = 1.$$

For example, consider an AND gate with inputs a and b and output f . Suppose that at input a an s - a -0 fault is present, and let the corresponding function be denoted as $f_{a/0}$. Then the only vector that satisfies the above condition is (1, 1). Thus, this is a test vector for an s - a -0 fault at a .

In some circuits, it is possible that $f \oplus f_\alpha = 0$. This would mean that the fault-free and faulty circuits yield the same logic value for each input vector. In this case, fault α cannot be detected and is referred to as *untestable* or *redundant*. A circuit in which all single stuck-at faults are testable is called *fully testable* or *irredundant*. We will deal with untestable faults later.

Next, consider two faults α and β for which the following condition is satisfied:

$$f_\alpha \oplus f_\beta = 0.$$

This means that f_α and f_β are identical. In such a case, faults α and β are said to be equivalent. Consider again the two-input AND gate. Faults $s-a-0$ at a , $s-a-0$ at b , and $s-a-0$ at f are all equivalent since, for all the four input vectors, they produce identical logic values at the output. The vector (1, 1) detects each of these faults. Therefore, it is enough to target only one fault from a set of equivalent faults. This is called *equivalence fault collapsing*.

In general, for an n -input primitive gate, i.e., for AND, OR, NAND or NOR gates, $n + 1$ stuck-at faults are equivalent. This applies to:

- all $s-a-0$ faults at the inputs and output of an AND gate;
- all $s-a-1$ faults at the inputs and output of an OR gate;
- all $s-a-0$ faults at the inputs and an $s-a-1$ fault at the output of a NAND gate;
- all $s-a-1$ faults at the inputs and an $s-a-0$ fault at the output of a NOR gate.

The above result implies that out of the $2(n + 1)$ single stuck-at faults possible in an n -input gate (two faults at each input and output), we need to consider only $n + 2$ faults for test generation on the basis of equivalence fault collapsing. For example, for an AND gate these would be the $n + 1$ $s-a-1$ faults and any $s-a-0$ fault chosen as a representative of the equivalent set of faults containing all the $n + 1$ $s-a-0$ faults.

Next, consider two faults α and β once again. Let the set of test vectors that can detect α (β) be denoted as T_α (T_β). Fault β is said to *dominate* fault α if $T_\alpha \subset T_\beta$. This means that whenever α is detected, so is β . Thus the dominating fault can be removed from the list of faults that need to be targeted. In the case of a two-input AND gate, one can see that an f $s-a-1$ fault dominates both an a $s-a-1$ fault and a b $s-a-1$ fault since $T_{f/1} = \{(0, 0), (0, 1), (1, 0)\}$, $T_{a/1} = \{(0, 1)\}$, and $T_{b/1} = \{(1, 0)\}$. Since either (0,1) or (1,0) will detect an f $s-a-1$ fault, this fault can be omitted from the list of faults (also called the *fault list*).

The above result further reduces the set of faults for an n -input primitive gate from $n + 2$ to $n + 1$. This is called *dominance fault collapsing*. We can see that for an AND (NAND) gate, the output $s-a-1$ ($s-a-0$) fault dominates each of the input $s-a-1$ faults and can thus be omitted. Similarly, for an OR (NOR) gate, the output $s-a-0$ ($s-a-1$) fault dominates each of the input $s-a-0$ faults.

The above fault-collapsing techniques lead to the following theorem.

Theorem 8.1 *A test set that detects all the single stuck-at faults at all the primary inputs and fanout branches of an irredundant combinational circuit detects all the circuit's single stuck-at faults. The primary inputs and fanout branches are referred to as its checkpoints.*

Proof The proof follows from the fact that a stuck-at fault at the output of each gate in the circuit is either equivalent to one of the input stuck-at faults of that gate or dominates it. However, a stuck-at fault at a fanout branch is neither equivalent to nor dominates a stuck-at fault at that fanout stem. Hence, if we scan the circuit from its output to its primary inputs, we can delete all faults not located at the primary inputs or fanout branches. $\diamond$

Corollary A test set that detects all the single stuck-at faults at all the primary inputs of a fanout-free combinational circuit detects all its single stuck-at faults.

Example Consider the circuit in Fig. 8.8. Theorem 8.1 indicates that the checkpoints are the primary inputs x_1, x_2, x_3 , and x_4 and the fanout branches c_1, c_2, c_4 , and c_5 . Thus, only the 16 stuck-at faults on these eight lines need to be considered for test generation. One can, in fact, reduce this fault list even further. Since the x_1 $s-a-0$ and c_1 $s-a-0$ faults are equivalent, one of them can be eliminated. Similarly, the $s-a-0$ faults at x_3 and c_2 are equivalent, as are the $s-a-0$ faults at x_4 and c_5 , and again one from each pair can be eliminated. Finally, the c_4 $s-a-0$ fault is equivalent to the c_3 $s-a-0$ fault, which dominates both the x_1 $s-a-1$ and c_1 $s-a-1$ faults. Thus, the c_4 $s-a-0$ fault can be eliminated. Finally, we end up with only 12 single stuck-at faults. One such fault list contains the $s-a-0$ faults at lines x_1, x_2, x_3 , and x_4 , and the $s-a-1$ faults at lines $x_1, x_2, x_3, x_4, c_1, c_2, c_4$, and c_5 .

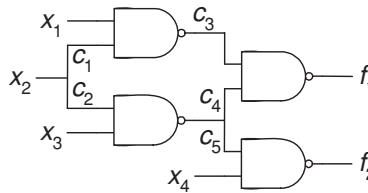
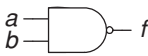


Fig. 8.8 Fault-collapsing example.

The D -algorithm

D -algorithm is a generalization of the one-dimensional path sensitization procedure. It can simultaneously sensitize multiple paths, when necessary. The name of the algorithm is derived from the error symbol D , which is a composite value that represents a 1 on a line in the fault-free circuit and a 0 on that line in the faulty circuit. The symbol D' denotes the complementary situation, 0 in the fault-free circuit and 1 in the faulty circuit. The D -algorithm uses a five-valued algebra composed of $\{0, 1, \phi, D, D'\}$, where ϕ denotes an unknown value. Thus, a line in a circuit can take any of these five values during test generation. The symbols D and D' behave like any Boolean variable in Boolean algebra. For example, $D + 0 = D$, $D \cdot D' = 0$, $D + D' = 1$,

Fig. 8.9 A NAND gate and its tables.

	<u>$a \ b \ f$</u>	<u>$a \ b \ f$</u>	<u>$a \ b \ f$</u>
	0 0 1	0 ϕ 1	D 1 D'
	0 1 1	ϕ 0 1	1 D D'
	1 0 1	1 1 0	D D D'
	1 1 0		

(a) NAND gate. (b) Truth table. (c) Singular cover. (d) Propagation D -cubes.

$D \cdot D = D + D = D$, etc. We next discuss the basic definitions behind the D -algorithm.

The *singular cover* of a gate represents a compacted form of its truth table. For example, the singular cover of a NAND gate is shown in Fig. 8.9c. Each row of a singular cover denotes a *singular cube*. Thus, 0 ϕ 1 is a singular cube. The singular cubes can be seen to represent the prime implicants of f and f' .

A *propagation D -cube* gives the minimal condition for the propagation of error through a gate. It is formed by combining two singular cubes or vectors with opposite output values. For example, combining the first and third singular cubes of the NAND gate yields the propagation D -cube $D' 1 D$. If we combine the third and first singular cubes, in that order, we instead obtain $D 1 D'$. Thus, by interchanging D and D' in a propagation D -cube, we can obtain another propagation D -cube. Three propagation D -cubes for the NAND gate are shown in Fig. 8.9d. Three others can be obtained by interchanging D and D' in each cube.

Different cubes can be combined through the process of *D -intersection* using the following rules:

$$\begin{aligned} 0 \cap 0 &= 0 \cap \phi = \phi \cap 0 = 0, \\ 1 \cap 1 &= 1 \cap \phi = \phi \cap 1 = 1, \\ \phi \cap \phi &= \phi. \end{aligned}$$

The *D -intersection* $C_1 \cap C_2$, of two D -cubes C_1 and C_2 is defined to have the same value in each position where C_1 and C_2 have identical values, and if the value is unknown in one cube then it denotes the value of the other cube in that position. If C_1 and C_2 have known, but different, values in any position then their intersection is null, i.e., it leads to a conflict. For example, let $C_1 = 0 1 \phi D$, $C_2 = \phi 1 D' D$, and $C_3 = 0 0 D' 1$. Then $C_1 \cap C_2 = 0 1 D' D$. However, $C_1 \cap C_3$ is null because of the conflicts in the second and fourth positions.

The *primitive D -cube of a fault* (PDCF) gives the minimal condition for the detection of a fault. For example, 1 1 D' is a PDCF for the output f s -a-1 fault in a NAND gate, as well as its input a s -a-0 and input b s -a-0 faults. This implies that the vector (1, 1) results in a 0 at f in the fault-free case but a 1 in the faulty case. Similarly, there are two PDCFs for the output f s -a-0 fault in a NAND gate: 0 ϕD and $\phi 0 D$. Similarly, the PDCF of a s -a-1

($b\ s\text{-}a\text{-}1$) is $0\ 1\ D\ (1\ 0\ D)$. An $s\text{-}a\text{-}0$ ($s\text{-}a\text{-}1$) fault at a line can be represented by a single-element cube D (D') at that line. Note that a PDCF gives the condition for detecting a fault at a gate whereas a propagation D -cube gives the condition for the propagation of error through a gate.

A *test cube* refers to the collection of all the circuit signals set to a particular value from the five-valued algebra in order to derive a test vector.

We are now in a position to discuss the D -algorithm, whose steps are summarized below.

1. *PDCF selection* Select a PDCF for the targeted fault as the initial test cube and place the gate output that is assigned a D or D' on the D -frontier.
2. *Implication* Perform implication (both forward and backward) of the values assigned in step 1. Do this by intersecting the test cube with the singular cubes of other gates whenever a unique choice exists. If a conflict occurs, backtrack to the previous point where a choice existed and renew the search with the next available choice.
3. *D -drive* Intersect the current test cube with a propagation D -cube of a gate whose input is on the D -frontier. Backtrack when necessary.
4. *Implication of D -drive* Perform implication of the values assigned in the previous step. Repeat the D -drive and its implication until an error signal has propagated to a circuit output.
5. *Line justification* For any gate G whose output is specified as 1 or 0 but whose inputs are not yet justified, perform line justification by intersecting the current test cube with a singular cube of G .
6. *Implication of line justification* Perform implication of the values assigned in step 5. Repeat line justification and its implication until all specified values have been justified. Backtrack when necessary.

Example Suppose that we want to derive a test vector for the $s\text{-}a\text{-}0$ fault shown in Fig. 8.10. The different steps involved in applying the D -algorithm to this example are shown in Table 8.1. Step 1 specifies the PDCF, which is also the initial test cube tc^0 . Steps 2 and 3 involve propagation of the error signal at x_1 through gate G_1 . The implication of the current test cube results in a 1 at line c_3 . At this point the D -frontier becomes empty. Since there is no error signal left to be propagated, we backtrack to step 1. In steps 5

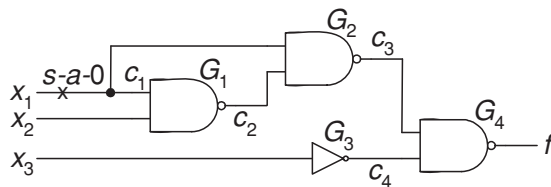


Fig. 8.10 A D -algorithm example.

Table 8.1 Different steps in the D -algorithm example

Step	x_1	x_2	x_3	c_2	c_3	c_4	f	Test cubes
1	D							tc^0 : PDCF – initial test cube
2	D	1		D'				p_1 : propagation D -cube of G_1
3	D	1		D'				$tc^1 = tc^0 \cap p_1$
4	D	1		D'	1			tc^2 : implication of tc^1 ; backtrack
5	D			1	D'			p_2 : propagation D -cube of G_2
6	D			1	D'			$tc^3 = tc^0 \cap p_2$
7	D	0		1	D'			tc^4 : implication of tc^3
8					D'	1	D	p_3 : propagation D -cube of G_4
9	D	0		1	D'	1	D	$tc^5 = tc^4 \cap p_3$
10			0			1		s_1 : singular cube of G_3
11	D	0	0	1	D'	1	D	$tc^6 = tc^5 \cap s_1$; test vector found

and 6, we propagate the D at x_1 through gate G_2 . In step 7, $c_2 = 1$ implies $x_2 = 0$ since x_1 is already specified. In steps 8 and 9, the current error signal at c_3 is propagated through gate G_4 . Since an error signal has reached the circuit output, the D -drive is over. Steps 10 and 11 involve line justification through gate G_3 . At this point, the test vector $(x_1, x_2, x_3) = (1, 0, 0)$ has been found.

The circuit in Fig. 8.10 actually contains an untestable fault. It is left as an exercise to the reader to show that a c_1 s - a -1 fault at the fanout branch of x_1 is untestable. This means that the circuit can be simplified, as we will see later. Interestingly, one can easily ascertain that an x_1 s - a -1 fault is testable.

8.3 I_{DDQ} testing

Quiescent drain current (I_{DDQ}) testing refers to the detection of defects in integrated circuits through the use of supply current monitoring. This is specially suited to CMOS circuits, in which the quiescent supply current is normally very low. Therefore, an abnormally high current indicates the presence of a defect. In order to achieve high quality, it is now well established that integrated circuits need to be tested with structural, delay, and I_{DDQ} tests.

In I_{DDQ} testing, the error effects of the fault no longer have to be propagated to circuit outputs for observation. The faults just have to be activated. Because observability is no longer a problem, it is easier to derive tests for I_{DDQ} -testable faults. Next, we study test generation techniques for such faults.

Test generation for bridging faults

In this subsection, we first discuss conditions for the detection of bridging faults. Then we consider *fault collapsing* methods for such faults, which reduce the

number of bridging faults that need to be targeted. Next, we present a test generation method for bridging faults. We limit ourselves to the consideration of a bridging fault between two nodes only, since if a bridging fault between multiple nodes is present and we activate a path from V_{dd} to V_{ss} through any two nodes involved in the fault then I_{DDQ} testing will detect the multiple-node fault as well. Also, we will consider all two-node bridging faults in the circuit from here on. It becomes necessary to do this in the absence of layout information. However, if layout information is available then the list of faults can be reduced on the basis of their likelihood of occurrence, e.g., the proximity of the two nodes.

Condition for detecting bridging faults

Let $P(r)$ denote the value of node r when vector P is applied to a fault-free circuit. For the nonfeedback bridging fault $\langle r_1, r_2 \rangle$, as discussed earlier, the only requirement for detection is that $P(r_1)$ and $P(r_2)$ assume opposite values. However, this represents an *optimistic condition* for the detection of feedback bridging faults. This can be seen as follows. Suppose that $P(r_1) = 0$ and $P(r_2) = 1$. Because of the feedback bridging fault, node r_2 may be prevented in some cases from being connected to V_{dd} in the faulty circuit. Thus, there may not be conduction between V_{dd} and V_{ss} , which is a prerequisite for I_{DDQ} testing. However, for simplicity of exposition, henceforth we shall assume that fault detection is based on the optimistic condition.

Fault collapsing

In order to reduce the test generation effort, we need to collapse the initial list of bridging faults. Suppose that two nodes r_1 and r_2 exist in the circuit such that, for every input vector P , $P(r_1) = P(r_2)$. Then the bridging fault $\langle r_1, r_2 \rangle$ is *redundant*, i.e., no test exists for it. Furthermore, if a set of vectors T is such that it detects the bridging faults between node r_1 and nodes in set R then T will also detect the bridging faults between node r_2 and nodes in R . Hence, every bridging fault involving node r_2 can be replaced with a corresponding fault involving node r_1 .

The first method for fault collapsing that uses the above arguments involves the identification of logic trees containing inverters and/or buffers in the circuit. Consider a root c_i of such a tree. If there exists at least one inverter with output c_j in this tree such that the path from c_i to c_j in the tree does not contain any other inverters, then we need to consider only those bridging faults for which one node is c_i or c_j . The bridging faults involving the other nodes in the tree can be ignored. If many inverters satisfying the condition for selecting c_j exist then one can be picked randomly. If no such inverter exists (i.e., the tree consists of only buffers) then only node c_i from the tree needs to be considered for the bridging faults in the circuit.

Example Consider the logic circuit shown in Fig. 8.11. Node c_1 is the root of a tree of inverters and a buffer. The path from c_1 to c_2 does not contain any other inverters. Therefore bridging faults involving nodes c_3, c_4, c_5, c_6 , among themselves or with other nodes not be considered.

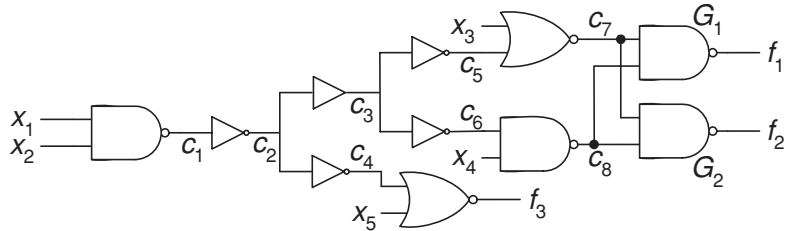


Fig. 8.11 Fault collapsing.

The second method for fault collapsing involves fanout nodes. Consider a set of nodes S such that each node in S has the same fanin nodes and realizes the same function. Then a bridging fault between any pair of nodes in S is redundant, and the above arguments can again be applied.

Example In Fig. 8.11, the fanins of nodes f_1 and f_2 are the same and these two nodes realize the same function. Therefore, only bridging faults involving either f_1 or f_2 , not those involving both, need to be considered. This argument can be extended to the internal nodes of gates G_1 and G_2 as well, i.e., only those bridging faults need to be considered that involve the internal nodes of either gate G_1 or G_2 , but not both.

Test generation

Bridging faults can be detected by applying a stuck-at fault test generator to a transformed circuit, as follows.

For a bridging fault $\langle c_1, c_2 \rangle$ where both c_1 and c_2 are gate outputs, we insert an EXCLUSIVE-OR gate G with inputs c_1 and c_2 . The target fault given to the stuck-at fault test generator is an s -a-0 fault at the output of G . If a test is found for this fault then it would drive c_1 and c_2 to opposite values in the fault-free case and, hence, be a test for the bridging fault. Otherwise, the fault is redundant.

For a bridging fault $\langle c_1, c_2 \rangle$ involving two internal nodes of a gate, we use the detection criterion $\langle c_1 = 0, c_2 = 1 \rangle$ or $\langle c_1 = 1, c_2 = 0 \rangle$, as before.

Example Consider the bridging fault in the CMOS circuit shown in Fig. 8.12. There are two ways to detect this fault, as follows:

- $c_3 = 0$ and $c_5 = 1$ This requires $c_1 = 1$ and $x_4 = 0$, $x_5 = 0$. We can introduce a gate G_1 into a circuit model in such a way that the output value of G_1 is 1 if this condition holds, as shown in Fig. 8.13.
- $c_3 = 1$ and $c_5 = 0$ This requires $x_1 = 1$, $c_1 = 0$ and $x_4 = 1$ or $x_5 = 1$. As before, we can introduce a gate G_2 into a circuit model in such a way that the output value of G_2 is 1 if this condition holds, as shown in Fig. 8.13.

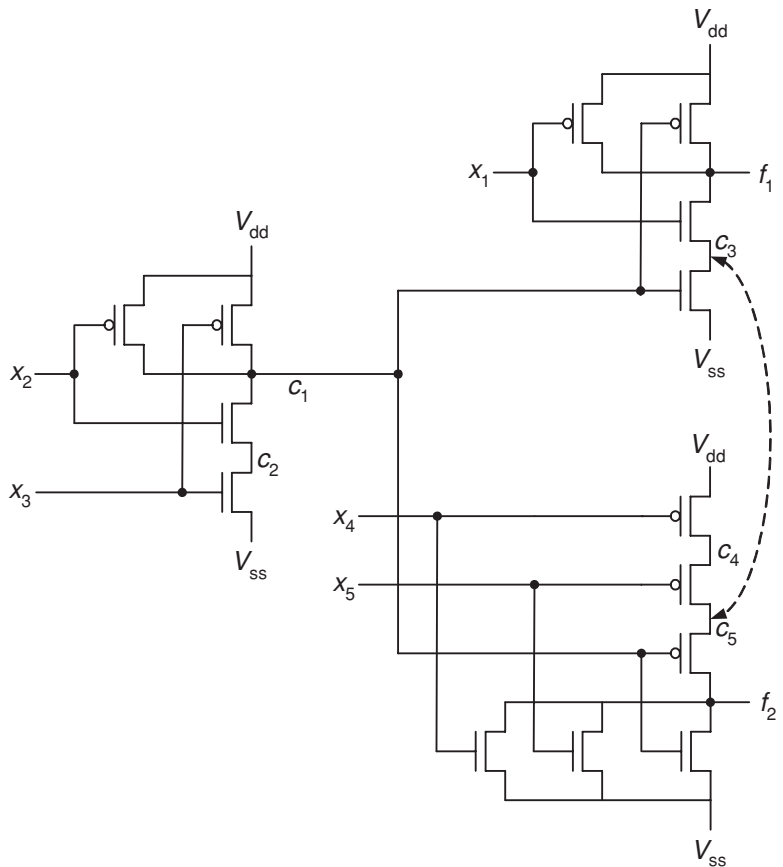


Fig. 8.12 Bridging faults between internal nodes.

Using G_1 and G_2 , we can obtain a test for the bridging fault $\langle c_3, c_5 \rangle$ if the output of either G_1 or G_2 is 1. This is accomplished by adding a two-input OR gate G , as shown in Fig. 8.13. The target stuck-at fault is an $s-a-0$ fault at the output of G . A test vector for such a fault would result

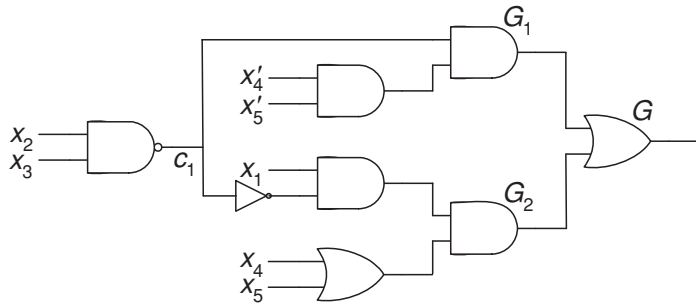


Fig. 8.13 Modeling of bridging faults between internal nodes.

in either (0, 1) or (1, 0) or (1, 1) to appear at the outputs of (G_1 , G_2). Each case would result in the detection of the bridging fault. This approach is, of course, also applicable when one of the shorted nodes is a gate-level node and the other is an internal node.

8.4 Delay fault testing

Delay fault testing exposes temporal defects in an integrated circuit. Even when a circuit performs its logic operations correctly, it may be too slow to propagate signals through certain paths or gates. In such cases, incorrect values may get latched at the circuit outputs.

In this section, we first describe the clocking schemes and basic concepts. We then present test generation methods for path delay faults and transition faults.

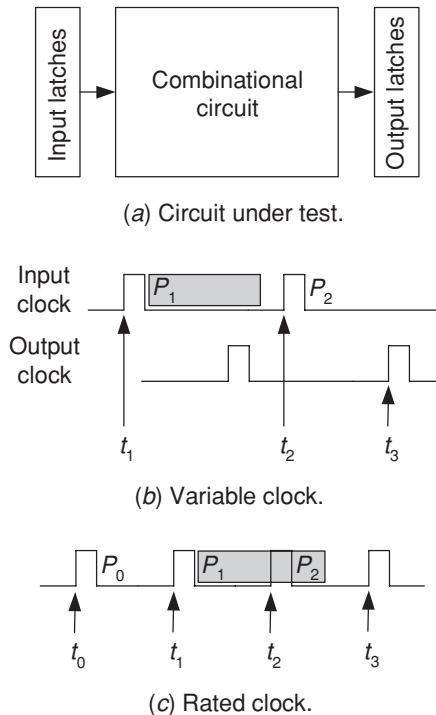
An underlying assumption behind the fault models and most testing methods presented here is that the gate propagation delays are fixed and independent of the input values. Therefore, it is assumed that if a circuit passes a test for a given fault then the fault will not cause an incorrect circuit operation for any other sequence of input patterns. This does not strictly correspond to what happens in actual circuits. Thus, this assumption can lead to some pitfalls in delay fault testing. However, making this assumption keeps the delay fault testing problem tractable.

Clocking schemes for delay fault testing

Delay fault testing techniques are based on either a variable clock scheme or a rated clock scheme. The most commonly used is the variable clock scheme. Consider the combinational circuit shown in Fig. 8.14a.³ In the *variable clock*

³ In this circuit, input and output latches are also shown. These are sequential elements that store logic values; they will be considered in detail in Chapter 9.

Fig. 8.14 Different clocking schemes for combinational circuit testing [4] © 1998, IEEE.



scheme, two clocks are required to separately strobe the primary inputs and circuit outputs, as shown in Fig. 8.14b. In a two-pattern test (P_1 , P_2), the first pattern (or vector) P_1 is applied to the primary inputs at time t_1 and second pattern P_2 at time t_2 . The shaded area represents the amount of time required for the faulty circuit to stabilize after the application of P_1 . The circuit response is observed at time t_3 .

This two-pattern test determines whether the propagation delay of a path, through which a transition or path delay fault is being tested, exceeds the time interval $t_3 - t_2$, which is the maximum allowable path delay for the rated frequency of operation. Owing to the skewed input–output strobing, the interval $t_3 - t_2$ is less than the interval $t_2 - t_1$, which is the time allowed for signal values to stabilize in the faulty circuit. If we assume that no path delay in a faulty circuit exceeds twice the clock period then $t_2 - t_1$ should be at least twice $t_3 - t_2$. This delay fault testing methodology increases the test application time and renders the hardware required for controlling the clock or the test application software more complex. However, it makes test generation easier.

In the *rated clock scheme*, all input vectors are applied at the rated circuit speed using the same strobe for the primary inputs and circuit outputs, as shown in Fig. 8.14c. All the path delays in the fault-free circuit are assumed to be smaller than the interval $t_2 - t_1$. However, paths in the faulty circuit may have delays exceeding this interval. Therefore, logic transitions and

*hazards*⁴ that arise at the time t_1 owing to the vector pair P_0, P_1 may still be propagating through the circuit during the time interval $t_3 - t_2$. This is shown in Fig. 8.14c. In addition, other transitions may originate at time t_2 owing to the vector pair P_1, P_2 . If we assume, as before, that no path delay in the faulty circuit exceeds twice the clock period then signal conditions during the interval $t_3 - t_2$ depend on three vectors P_0, P_1, P_2 . This, in general, makes test generation more complex. However, this is the type of scenario one encounters often in the industry. Contrast the above situation with the one in Fig. 8.14b for the variable clock scheme, where signal conditions during the interval $t_3 - t_2$ depend on the vector pair P_1, P_2 only.

We will assume the use of the variable clock scheme in the rest of this chapter, unless otherwise stated.

Basic definitions

An input of a gate is said to have a *controlling value* if it uniquely determines the output of the gate independently of its other inputs. Otherwise, the value is said to be *noncontrolling*. For example, 1 is the controlling value of an OR or NOR gate, and 0 the noncontrolling value.

A path R in a circuit is a sequence $(g_0 g_1 \cdots g_r)$, where g_0 is a primary input, $g_1, g_2, \dots, g_{r-1}$ are gate outputs, and g_r is a circuit output. Let the gate with output g_j be denoted G_j . An *on-input* of R is a connection between two gates along R . A *side-input* of R is any connection to a gate along R other than its on-input.

There are two path delay faults (or logical paths) for each physical path R , one for each direction of signal transition along R . A path delay fault can be depicted in two equivalent ways: by considering the transition at its input or its output. If the desired transition at the input g_0 of R is a rising (falling) one, the path delay fault is denoted $\uparrow R$ ($\downarrow R$). Alternatively, if the desired transition at output g_r of R is a rising (falling) one then the path delay fault is denoted $R\uparrow$ ($R\downarrow$).

There are various ways to classify path delay faults. However, the most popular approach is to consider only two types of fault: *robustly testable* and *nonrobustly testable*. We consider conditions for detecting such faults next.

A two-pattern test (P_1, P_2) is a *nonrobust test* for a path delay fault if and only if it satisfies the following conditions: (i) it launches the desired logic transition at the primary input of the targeted path, and (ii) all side-inputs of the targeted path settle to noncontrolling values under P_2 .

⁴ Hazards represent a momentary transition to the opposite logic value. They are of two kinds: static and dynamic. A *static hazard* indicates such a transition when the initial and final values are the same, e.g., $0 \rightarrow 1 \rightarrow 0$ or $1 \rightarrow 0 \rightarrow 1$. A *dynamic hazard* indicates such a transition when the initial and final values are different, e.g., $0 \rightarrow 1 \rightarrow 0 \rightarrow 1$ or $1 \rightarrow 0 \rightarrow 1 \rightarrow 0$.

Example Consider the EXCLUSIVE-OR gate implementation shown in Fig. 8.15. It shows the signals at each node when the two-pattern test $\{(0, 1), (1, 1)\}$ is applied. The arrows show how the signal transitions propagate. Signal value $S1$ denotes a steady value 1 at input x_2 . This is a nonrobust test for a path delay fault $x_1c_1c_2f\downarrow$, which is shown in bold. The test is nonrobust because if the observation point were t_2 then this test would be invalidated since we would obtain the correct output value 0 for the second vector even when the above fault is present. This would happen if a path delay fault $x_1c_2f\uparrow$ were also present in the circuit. Thus, a nonrobust test cannot guarantee the detection of the targeted path delay fault.

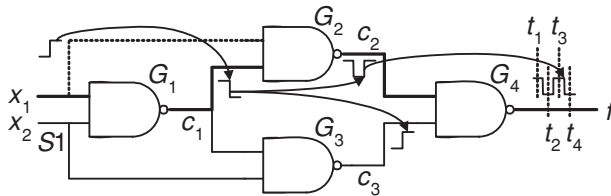


Fig. 8.15 An EXCLUSIVE-OR gate implementation [10] © 1995, IEEE.

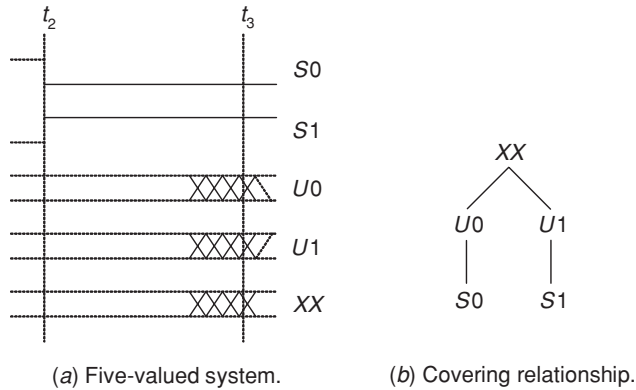
A *robust test* can detect the targeted path delay fault independently of the delays in the rest of the circuit. (i) It must satisfy the conditions of nonrobust tests, and (ii) whenever the logic transition at an on-input is from a noncontrolling to a controlling value, each corresponding side-input should maintain a steady noncontrolling value.

Example Consider the EXCLUSIVE-OR gate implementation in Fig. 8.15 again. The test $\{(0, 0), (1, 0)\}$ is robust for the path delay fault $x_1c_2f\uparrow$, parts of which are shown in the dotted and bold lines.

We saw above that a nonrobust test for a targeted path delay fault may be invalidated by other path delay faults. However, in the presence of tests in the test set that robustly test for invalidating path delay faults, a nonrobust test is called *validatable*.

Example In Fig. 8.15, the rising transition at f just after t_2 corresponds to the path delay fault $x_1c_2f\uparrow$, which was shown to have a robust test $\{(0, 0), (1, 0)\}$ in the previous example. Suppose that this test is in the test set. If the circuit passed this test then the observation time can only be t_3 or t_4 when $\{(0, 1), (1, 1)\}$ is applied. In both cases, this test is valid. Thus, $\{(0, 1), (1, 1)\}$ is a *validatable nonrobust test* for the path delay fault $x_1c_1c_2f\downarrow$, because either the path delay fault is caught, if the observation time is t_3 , or the circuit is free of this fault, if the observation time is t_4 .

Fig. 8.16 Five-valued logic system and the covering relationship [21] © 1987, IEEE.



Test generation for path delay faults

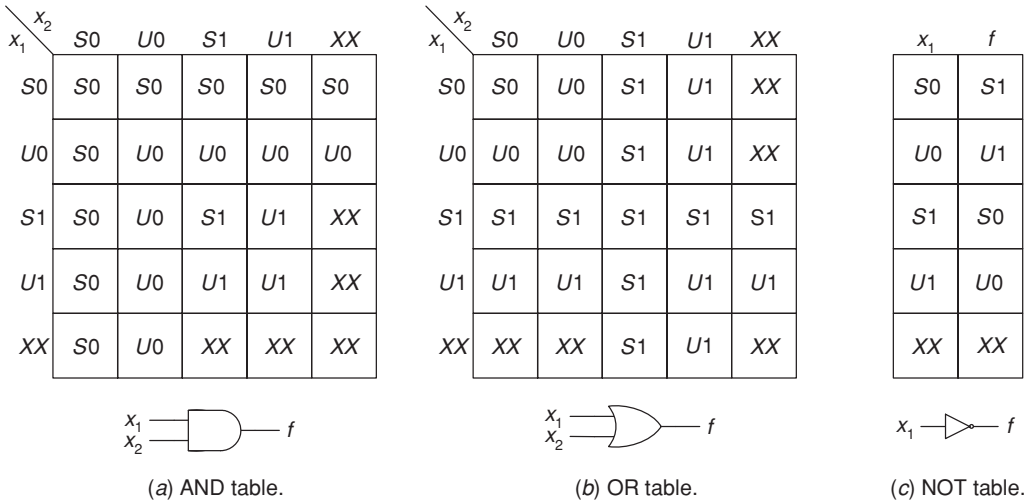
We next present a test generation method for path delay faults based on a five-valued logic system. The number of values determines the time and memory complexity of the methods based on them. The fewer the values, the less complex the implementation. However, in general fewer values also imply less efficiency (i.e., test generation takes longer).

The five values we consider are $\{S0, S1, U0, U1, XX\}$. They are depicted in Fig. 8.16a. Each value represents a type of signal on the lines in a circuit in the time interval $t_3 - t_2$ (see Fig. 8.14b). Let the two-pattern test be (P_1, P_2) . Let the initial value (final value) of a line in the circuit be the binary value on the line after P_1 (P_2) has been applied and the circuit has stabilized. Under the variable clock scheme, recall from Fig. 8.14b that at time t_2 any signal on a line in the circuit has stabilized to the initial value of that line. However, at time t_3 a signal in a faulty circuit may not have stabilized to the final value of the line. The purpose of delay fault testing is to check whether signals do stabilize by time t_3 . The value $S0$ ($S1$) represents signals on lines whose initial and final values are 0 (1). Furthermore, the line remains free of hazards. The value $U0$ ($U1$) represents signals on lines whose final value is 0 (1). The initial values of these lines could be either 0 or 1. In addition, in the time interval $t_3 - t_2$, the lines could have hazards. Obviously, the value $U0$ ($U1$) includes the value $S0$ ($S1$). The value XX represents signals whose initial and final values are unspecified. The covering relationship among the five values is shown in Fig. 8.16b. The value $U0$ covers $S0$, $U1$ covers $S1$, and XX covers both $U0$ and $U1$.

In Fig. 8.17, implication tables for the five-valued logic system are given for AND, OR, and NOT gates. From these tables and the associative law of Boolean algebra, one can determine the output values of multiple-input AND, OR, NAND, and NOR gates, given their input values.

Test generation for robustly testable path delay faults

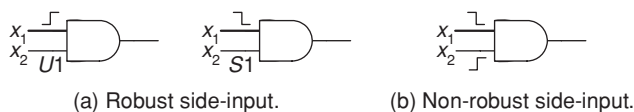
In deriving two-pattern tests for path delay faults, the signals $U0$ and $U1$ are interpreted in two different ways. The $U0$ ($U1$) signal on an on-input is

**Fig. 8.17** Implication tables [21]

© 1987, IEEE.

Fig. 8.18 Robustly sensitizing input values [21] © 1987, IEEE.

On-input transition \ Gate type	AND or NAND	OR or NOR
Rising ($U1$)	$U1$	$S0$
Falling ($U0$)	$S1$	$U0$

Fig. 8.19 Different types of side-inputs for an AND gate.

interpreted as a $1 \rightarrow 0$ ($0 \rightarrow 1$) transition. The $U0$ and $U1$ signals on lines in the circuit, other than on-inputs, are interpreted according to Fig. 8.16a, i.e., with final values of 0 and 1, respectively.

The following key result is used for the robust test generation of path delay faults. A two-pattern test (P_1, P_2) robustly tests such a fault if and only if: (i) it launches the desired transition at the input of the targeted path, and (ii) the side-inputs have values that are covered by the values indicated in Fig. 8.18. Such side-inputs are called robust. This result follows directly from the definition of a robust test. Figure 8.19a shows examples of robust side-inputs for an AND gate; the on-input is shown in bold.

Example Consider the bold path in the circuit in Fig. 8.20. Suppose that the path delay fault with a rising transition at input x_3 of this path needs to be tested. Therefore we place the signal $U1$ at x_3 . The current side-input is x_4 . From Fig. 8.18, we find that signal $S0$ must be placed on x_4 . From the implication table in Fig. 8.17b, we find that the signal on line c_2 is, therefore, $U1$. The side-input of c_2 is c_1 . From Fig. 8.18, we see that we need to place $S0$ at c_1 . This results in $U0$ at line c_3 (from the tables in Figs. 8.17b, c). The side-input now is c_4 . From Fig. 8.18 we need to place $U0$ at c_4 , which allows the propagation of $U0$ to the circuit output f . At this point, the sensitization of the path under test is complete. Next, signals $S0$ at line c_1 and $U0$ at line c_4 need to be justified at the primary inputs. This is accomplished as shown in Fig. 8.20. In general, this step may need backtracking, just as in the case of stuck-at fault test generation. The corresponding two-pattern test is $\{(0, \phi, 0, 0, \phi), (0, \phi, 1, 0, 0)\}$. Note that, for on-input x_3 , $U1$ was interpreted as a $0 \rightarrow 1$ transition, whereas $U0$ on x_5 was interpreted just as a signal with a final value 0. However, if the unknown value for x_5 in the first vector is chosen to be 1, which gives the two-pattern test $\{(0, \phi, 0, 0, 1), (0, \phi, 1, 0, 0)\}$, then readers can check that this two-pattern test also robustly tests for the path delay fault $\downarrow x_5 c_4 f$.

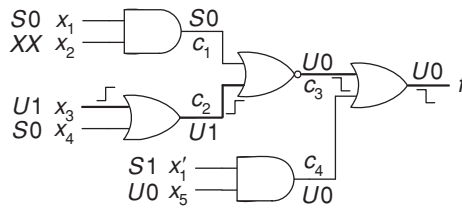


Fig. 8.20 Circuit illustrating robust test generation.

Test generation for nonrobustly testable path delay faults

The method presented above can be very easily modified to perform nonrobust test generation when robust test generation fails. The only modification that is needed is to relax the conditions for side-input values, as shown in Fig. 8.21. This follows directly from the definition of a nonrobust two-pattern test. Figure 8.19b shows an example of a nonrobust side-input for an AND gate.

When the on-input has a transition from a noncontrolling value to a controlling value, nonrobust tests simply require that the side-inputs settle to a noncontrolling value on the second vector, as opposed to the requirement of a steady noncontrolling value for robust tests. The former side-inputs are called nonrobust. Thus, a nonrobustly testable path delay fault has at least one

Fig. 8.21 Nonrobustly sensitizing input values [21] © 1987, IEEE.

On-input transition \ Gate type	AND or NAND	OR or NOR
Rising ($U1$)	$U1$	$U0$
Falling ($U0$)	$U1$	$U0$

nonrobust side-input, the other side-inputs being robust for any two-pattern test. The number of nonrobust side-inputs can be different for different two-pattern tests. To reduce the chance of test invalidation, we aim to reduce the number of nonrobust side-inputs. The time elapsed between when the noncontrolling value on the side-input becomes stable and before the on-input becomes stable is called the *slack* of the side-input. If the slacks of all the side-inputs for the fault under test are positive then there can be no test invalidation. Thus, another objective of test generation is to maximize the slack of the nonrobust side-inputs. Such two-pattern tests can tolerate larger timing variations at these side-inputs.

The quality of nonrobust tests can be improved further by converting them into validatable nonrobust tests, if possible. A two-pattern test P , obtained as above with a minimal number of nonrobust side-inputs and maximal slack, can be processed further as follows. If P has don't-cares at some primary inputs, we should specify the don't-cares in such a fashion that the number of transitions at the primary inputs is minimized (i.e., $U1$ is specified as 11, $U0$ is specified as 00, and XX as 00 or 11). After performing the implications of the new two-pattern test, P_{new} , we examine the nonrobust side-inputs and identify the path delay faults that need to be robustly tested to validate P_{new} . If these identified paths are indeed robustly testable then the nonrobust two-pattern test in question is validatable.

Test generation for transition faults

In transition fault testing, since the delay defect size is assumed to be larger than the system clock period, the delay fault can be exposed by appropriately sensitizing an arbitrary path through the faulty gate. Consider a two-pattern test (P_1, P_2) for a slow-to-rise transition fault at the output g_i of some gate in a circuit. The two-pattern test needs to satisfy two conditions: (i) $g_i(P_1) = 0$ and (ii) $g_i(P_2) = 1$ and a path should be sensitized from g_i to some circuit output under P_2 . Thus, the two-pattern test launches a rising transition at the fault site and makes the effect observable at a circuit output for the second vector. In the presence of the transition fault, both the fault site and the circuit output under question will have an error for the vector P_2 . Note that P_2 is simply a test for an $s-a-0$ fault at g_i . For testing a slow-to-fall fault at g_i the conditions are similar, except that $g_i(P_1) = 1$ and $g_i(P_2) = 0$. In this case, P_2 is an $s-a-1$ fault test for g_i . In this method, possible test invalidation due to hazards at the

circuit output under question is ignored. However, such a possibility can be reduced by choosing two-pattern tests in which P_1 and P_2 differ in only one bit, whenever possible.

Example Consider the circuit in Fig. 8.22. Suppose there is a slow-to-rise transition fault at line c_3 . First, we need to derive a vector P_1 that makes $c_3 = 0$. Such a vector is $(\phi, \phi, 0, \phi, \phi)$. Then, we need to derive a vector P_2 that makes $c_3 = 1$ and sensitizes any path from c_3 to f . One such vector is $(0, \phi, 1, 1, 1)$. Thus, one possible two-pattern test is $\{(0, 0, 0, 1, 1), (0, 0, 1, 1, 1)\}$, which reduces the number of bits in which the two vectors differ to just one.

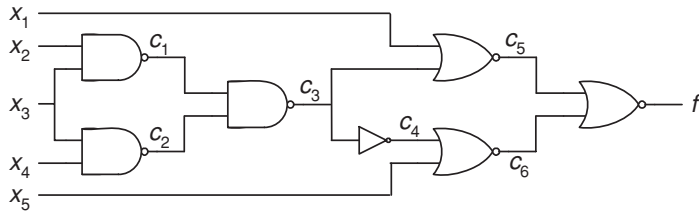


Fig. 8.22 Transition fault testing.

At-speed test generation

All the test generation methods presented so far have been under the variable clock scheme. We now consider test generation under the rated clock scheme. Rated clock tests are also called *at-speed* tests.

Suppose we make the assumption that the delay fault does not cause the delay of the path it is on to exceed two clock cycles. Then a trivial way of obtaining an at-speed test from a two-pattern test (P_1, P_2) derived under a variable clock scheme, is to use the three-pattern test (P_1, P_1, P_2) . By doing this, the signal values in the circuit are guaranteed to stabilize when the first vector is left unchanged for two clock cycles. This method is applicable to two-pattern variable clock tests derived for either fault model, path delay or transition. If we relax our assumption and say that the delay fault does not cause the delay of the path it is on to exceed n clock cycles then we can simply derive an $(n + 1)$ -pattern test where the first vector P_1 is replicated n times.

8.5 Synthesis for testability

Synthesis-for-testability techniques incorporate testability considerations during the synthesis process itself. There are two major sub-areas: synthesis for full

testability and synthesis for easy testability. In the former, one tries to remove all redundancies from the circuit so that it becomes completely testable. In the latter, one tries to synthesize the circuit in order to achieve one or more of the following aims: small test generation time, small test application time, high fault coverage. Of course, one would ideally like to achieve both full and easy testability.

In this section, we consider both the stuck-at and delay fault models. Under the stuck-at fault model, we look at single as well as multiple faults. Under the delay fault model, we consider both path delay and transition faults.

Synthesis for stuck-at fault testability

In this subsection, we limit ourselves to the stuck-at fault model. We will discuss the testability of two-level circuits, logic transformations for preserving single or multiple stuck-at fault testability, and redundancy identification and removal.

Two-level circuits

Two-level circuits are frequently the starting point for further logic optimization. Hence, it is important to consider the testability of such circuits. Suppose an irredundant sum of products is implemented as an AND–OR two-level circuit. Such a circuit is fully testable for all single stuck-at faults. In fact, a single stuck-at fault test set also detects all multiple stuck-at faults in the two-level circuit. The same result holds for a NAND–NAND two-level circuit, which can also be derived from an irredundant sum of products, or for an OR–AND or NOR–NOR two-level circuit derived from an irredundant product of sums.

Example Consider the two-level circuit shown in Fig. 8.23, which implements the irredundant sum of products $f = x_1x_2 + x_2x_3$. It is easy to check that the test set $\{(0, 1, 0), (0, 1, 1), (1, 0, 1), (1, 1, 0)\}$ detects all single stuck-at faults in this circuit.

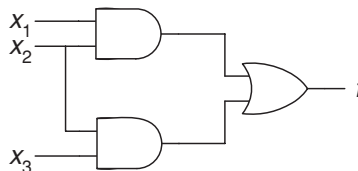


Fig. 8.23 An AND–OR two-level circuit.

Transformations to preserve single stuck-at fault testability

Given an initial circuit that implements the desired functions, one can apply various *transformations* to it to obtain another circuit that meets certain desired

area, delay, testability, and power constraints. In this section, we shall look at transformations which can be applied to initial circuits that are testable for all single stuck-at faults to produce final circuits that are also completely single stuck-at fault testable. We first recapitulate some background material from Chapter 6.

A *cube* is a product of a set C of literals such that if a literal $x \in C$ then $x' \notin C$. Suppose that a function f is expressed as $f_d f_q + f_r$. If f_d and f_q have no inputs in common then both f_d and f_q are said to be *algebraic divisors* of f , where f_r is the remainder. If an algebraic divisor has exactly one cube in it, it is called a *single-cube divisor*. If it has more than one cube, it is called a *multiple-cube divisor*. For example, if $f = x_1 x_2 x_3 + x_1 x_2 x_4 + x_5$ then $g_1 = x_1 x_2$ is a single-cube divisor of f whereas $g_2 = x_2 x_3 + x_2 x_4$ is a multiple-cube divisor of f . If we express f as $x_1 g_2 + x_5$ in this example, g_2 is said to be *algebraically resubstituted* in f . By identifying algebraic divisors common to two or more expressions and resubstituting them, one can convert a two-level circuit into a multi-level circuit. This process is referred to as *algebraic factorization*. If the complement of the algebraic divisor is not used in this factorization, the process is said to be algebraic factorization without the use of the complement. A Boolean expression f is said to be *cube-free* if the only cube dividing f without remainder is 1. A cube-free expression must have more than one cube. For example, $x_1 x_2 + x_3$ is cube-free but $x_1 x_2 + x_1 x_3$ and $x_1 x_2 x_3$ are not. A *double-cube divisor* of a Boolean expression is a cube-free multiple-cube divisor having exactly two cubes. For example, if $f = x_1 x_4 + x_2 x_4 + x_3 x_4$ then the double-cube divisors of f are $\{x_1 + x_2, x_1 + x_3, x_2 + x_3\}$.

We next consider a method for obtaining multi-level circuits that uses only single-cube divisors, double-cube divisors, and their complements. These divisors are extracted from functions that are given in irredundant sum-of-products form. The complements are obtained by using only De Morgan's theorem. Boolean reductions such as $a + a = a$, $a + a' = 1$, $a \cdot a = a$, and $a \cdot a' = 0$ are not used. Furthermore, for simplicity, only two-literal single-cube divisors are used, and the double-cube divisors are assumed to have at most two literals in each of the two cubes and at most three variables as inputs.

In multi-level circuits, the first gate level processes primary inputs and produces intermediate nodes. Then successive logic levels use both primary inputs and intermediate nodes to produce new high-level intermediate nodes and circuit outputs. *Single-cube extraction* is the process of extracting cubes that are common to two or more cubes. The common part is then created as an intermediate node. The transformation is as follows. From the expression $f = x_1 x_2 A_1 + x_1 x_2 A_2 + \cdots + x_1 x_2 A_n$, the cube $C = x_1 x_2$ is extracted and substituted to obtain $CA_1 + CA_2 + \cdots + CA_n$. The *double-cube extraction* transformation consists of extracting a double-cube from a single-output sum-of-products expression $AC + BC$ to obtain $C(A + B)$. *Dual expression extraction* transforms a sum-of-product expression f in the following ways:

1. $f = x_1 A_1 + x_2 A_1 + x'_1 x'_2 A_2$ is transformed to $M = x_1 + x_2$ and $f = M A_1 + M' A_2$;
2. $f = x_1 x'_2 A_1 + x'_1 x_2 A_1 + x'_1 x'_2 A_2 + x_1 x_2 A_2$ is transformed to $M = x_1 x'_2 + x'_1 x_2$ and $f = M A_1 + M' A_2$.
3. $f = x_1 x_2 A_1 + x'_2 x_3 A_1 + x'_1 x_2 A_2 + x'_2 x'_3 A_2$ is transformed to $M = x_1 x_2 + x'_2 x_3$ and $f = M A_1 + M' A_2$.

At each step of the synthesis process the method selects and extracts a double-cube divisor jointly with its dual expression or a single-cube divisor that results in the greatest cost reduction in terms of the total literal-count. If the above transformations are applied to a single-output sum of products then single stuck-at fault testability is preserved.

In order to apply this method to a multiple-output two-level circuit, one implements each output in an irredundant sum-of-products form (such circuits are sometimes called *single-output minimized multiple-output two-level circuits*). Also, care must be taken during resubstitution. In a multiple-output circuit, many nodes $y_1, y_2, \dots, y_k$ may be represented by the same expression. Resubstitution is a transformation that replaces each copy of $y_1, y_2, \dots, y_k$ with a single node. Resubstitution of common subexpressions in a multiple-output function preserves single stuck-at fault testability if no two subexpressions control the same output.

Suppose that some single stuck-at fault testable circuit C_1 is transformed to another circuit C_2 using the above method; then not only is C_2 guaranteed to be single fault testable but also the single stuck-at fault test set of C_1 is guaranteed to detect all single stuck-at faults in C_2 . Such transformations are called *test set preserving*.

Transformations to preserve multiple stuck-at fault testability

If algebraic factorization without complement (see above) is applied to a single-output two-level circuit based on an irredundant sum of products, then the resultant multi-level circuit is testable for all multiple stuck-at faults using the single stuck-at fault test set of the two-level circuit. Unlike the method in the previous section, the algebraic divisors in this case need not be limited to single-cube and double-cube divisors.

The proof that algebraic factorization without complement preserves multiple stuck-at fault testability and test sets is intuitively quite simple. If we collapse the algebraically factored multi-level circuit to a two-level circuit, we arrive at the original sum-of-products expression from which we began the synthesis process. Therefore, for every multiple stuck-at fault in the multi-level circuit, we can obtain a corresponding multiple stuck-at fault in the two-level circuit. Since the test set for the two-level circuit detects all multiple stuck-at faults in it, it also detects all multiple stuck-at faults in the multi-level circuit. However, frequently the size of the test set for two-level circuits is about two to 10 times larger than the size of the single stuck-at fault test set for the

multi-level circuit. Therefore, an increase in the test set size is the price paid for multiple stuck-at fault testability.

Surprisingly, even though general algebraic factorization without complement preserves multiple stuck-at fault testability, it does not preserve single stuck-at fault testability. This is owing to the fact that, in a single stuck-at fault testable circuit, a multiple stuck-at fault may be redundant and after algebraic factorization can become a single redundant stuck-at fault.

Example Consider the circuit in Fig. 8.24a, which can be verified to be completely single stuck-at fault testable. If we replace gates G_1 and G_2 with a single gate, corresponding to factoring out a single cube, we get the circuit in Fig. 8.24b. In this circuit, an $s-a-0$ or $s-a-1$ fault at the output of gate H is not testable. These single stuck-at fault redundancies are a result of the double $s-a-0$ (or $s-a-1$) redundant fault at the outputs of gates G_1 and G_2 in the circuit in Fig. 8.24a.

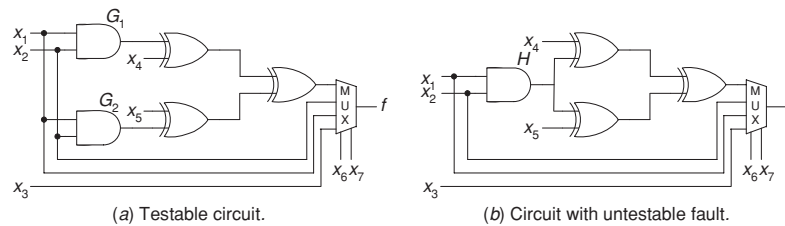


Fig. 8.24 Activation of a latent redundant multiple stuck-at fault [12] © 1992, IEEE.

One can also derive a multiple stuck-at or stuck-open fault testable and delay fault testable multi-level circuit using Shannon's decomposition. This will be discussed later.

Redundancy identification and removal

Owing to suboptimal logic synthesis, unintentional redundancies can be introduced into a circuit, and this can lead to a larger chip area and increase in its propagation delay. However, the identification of redundant faults is computationally expensive since, typically, test generation algorithms declare a fault to be redundant only if they fail to generate a test vector for it after implicit exhaustive enumeration of all the vectors. Furthermore, the presence of a redundant fault may invalidate the test for another fault, make a detectable fault redundant, or make a redundant fault detectable. Therefore, the removal of such redundant faults from a circuit can, in general, help reduce area and delay while at the same time improving its testability.

One can categorize the redundancy identification and removal methods as either indirect or direct. If redundancy identification is a byproduct of test generation, it is called *indirect*. A *direct* method can identify redundancies

without the search process involved in test generation. Such a method can be further subdivided into three categories: *static*, *dynamic*, and *don't-care based*. Static methods analyze the circuit structure and perform value implications to identify and remove redundancies; they usually work as a preprocessing step to an indirect method. Dynamic methods work in concert with an indirect method. However, they do not require an exhaustive search. Don't-care-based methods involve functional extraction, logic minimization, and logic modification.

Indirect methods If a complete test generation algorithm (i.e., one that can guarantee the detection of a fault, given enough time) fails to generate a test for an s - a -0 (s - a -1) fault at l which we abbreviate to " l s - a -0 (l s - a -1)," then l can be connected to the value 0 (1) without changing the function of the circuit. The circuit can then be reduced by simplifying gates connected to constant values, replacing a single-input AND or OR (NAND or NOR) gate obtained as a result of simplification with a direct connection (inverter), and deleting all gates that do not fan out to any circuit output. The simplification rules are as follows.

1. If the input s - a -0 fault of an AND (NAND) gate is redundant, remove the gate and replace it with a 0 (1).
2. If the input s - a -1 fault of an OR (NOR) gate is redundant, remove the gate and replace it with a 1 (0).
3. If the input s - a -1 fault of an AND (NAND) gate is redundant, remove the input.
4. If the input s - a -0 fault of an OR (NOR) gate is redundant, remove the input.

Since the removal of a redundancy can make detectable faults undetectable or undetectable faults detectable, it is not possible to remove all redundancies in a single pass using these methods, as illustrated by the following example.

Example Consider the circuit given in Fig. 8.25a. The following faults in it are redundant: x_1 s - a -0, x_1 s - a -1, x_3 s - a -0, x_3 s - a -1, c_1 s - a -0, c_1 s - a -1, c_2 s - a -1, and c_3 s - a -1. If none of these faults is present we can detect c_4 s - a -1 by the vector (1, 0, 1, 1). However, if the redundant fault x_1 s - a -0 is present, it makes the former fault redundant too.

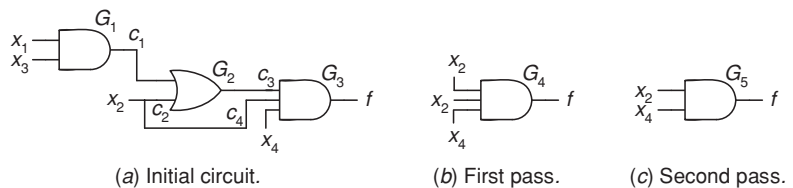


Fig. 8.25 Redundancy identification and removal using the indirect method.

Suppose we target x_1 $s-a-0$ for removal. Using the above simplification rules, we obtain the circuit in Fig. 8.25b in the first pass. We perform test generation for all the faults in this circuit again and find that both the x_2 $s-a-1$ faults seen in this figure are redundant. However, if either of these redundant faults is present then the other becomes detectable. Targeting either fault for redundancy removal, we get the irredundant circuit in Fig. 8.25c in the second pass.

The need to target each fault in each test generation pass makes this method computationally very expensive.

An interesting use of the indirect method is based on deliberately adding redundancies to an irredundant circuit in order to create yet more redundancies, which, upon removal, yield a better optimized circuit. This is done using the concept of *mandatory assignments*. These are value assignments to some lines in the circuit that must be satisfied by any test vector for the given fault. These consist of *control and observation assignments*, which make it possible to control and observe the fault, respectively. If these assignments cannot be simultaneously justified then the fault is redundant. Using this approach, we can add redundant connections (with or without inversions) to the circuit in such a way that the number of connections that become redundant elsewhere in the circuit is maximized. Then after redundancy removal targeted first towards these additional redundancies, we obtain a better irredundant circuit realizing the same functions.

Example Consider the c_1 $s-a-0$ fault in the irredundant circuit shown in Fig. 8.26a (ignore the broken-line connection for the time being). The mandatory control assignment for detecting this fault is $c_1 = 1$ and the mandatory observation assignment is $c_2 = 0$. These assignments imply $x'_1 = 1$, $x_3 = 1$, and $x_2 = 0$, which in turn imply $c_3 = 1$, $c_4 = 0$, and $f_2 = 1$.

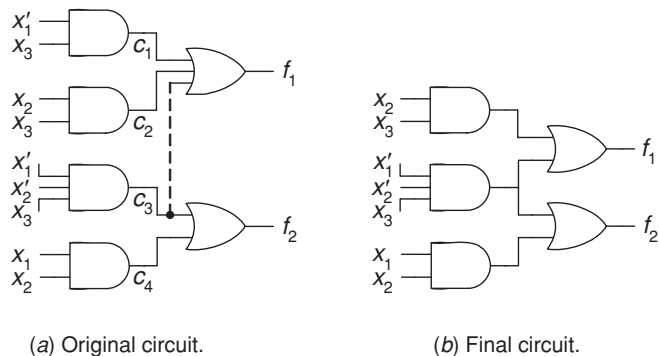


Fig. 8.26 Redundancy addition and removal.

Since $c_3 = 1$, if we were to add the broken-line connection, the effect of the c_1 s - a -0 fault would be no longer visible at f_1 . Thus this fault would become redundant. However, we still have to verify that adding the broken-line connection does not change the input–output behavior of the circuit. In order to test for an s - a -0 fault on this connection, the mandatory control assignment is $c_3 = 1$ and the mandatory observation assignments are $c_1 = 0$ and $c_2 = 0$. However, these three assignments are not simultaneously satisfiable at the primary inputs. Thus, the broken-line connection is indeed redundant. After adding this connection, we can use the simplification rules to remove some logic circuitry, since c_1 s - a -0 is now redundant. Finally, we obtain the circuit in Fig. 8.26b, which implements the same input–output behavior as the circuit in Fig. 8.26a yet requires less area. In the modified circuit, of course, the broken-line connection is no longer redundant.

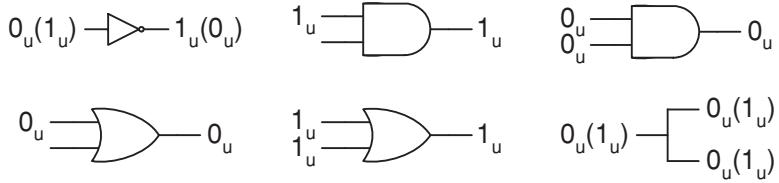
We now consider the three categories of direct methods.

Static direct methods Static methods for redundancy identification are very fast since they do not need an exhaustive search. However, they are usually not able to identify all redundancies; hence, they can be used as a preprocessing step to an indirect method.

One can use an “illegal” combination of values to identify redundancies. Suppose that the values v_1 , v_2 , and v_3 cannot simultaneously occur on, respectively, lines c_1 , c_2 , and c_3 in a circuit, i.e., this combination is illegal. Then faults for which this combination of values is mandatory are redundant. The problem of finding such faults is decomposed into first finding faults for which each condition is individually mandatory. If $S_{c_i}^{v_j}$ denotes the set of faults that must have value v_j on line c_i for detection then the faults that require the above combination for detection are in the set $S_{c_1}^{v_1} \cap S_{c_2}^{v_2} \cap S_{c_3}^{v_3}$. To find these faults, the concept of uncontrollability and unobservability analysis is used.

Recall that the *controlling value* of a gate is the value that determines its output irrespective of its other input values. Thus, the value 0 (1) is the controlling value for AND and NAND (OR and NOR) gates. Let 0_u (1_u) denote the *uncontrollability status* of a line that cannot be controlled to the value 0 (1). The propagation rules of the uncontrollability status indicators are given in Fig. 8.27. Similar rules can be obtained for gates with more inputs. The uncontrollability status indicators are propagated forward and backward. The forward propagation of uncontrollability may make some lines unobservable. In general, if a gate input cannot be set to the noncontrolling value of the gate then all its other inputs become unobservable. The *unobservability status* can be propagated backward from a gate output to all its inputs. When all fanout branches of a stem s are marked unobservable, the stem is also marked unobservable if for each fanout branch b of s , there exists at least one set of lines $\{l_b\}$ such that the following conditions are met:

Fig. 8.27 Uncontrollability propagation rules [14] © 1996, IEEE.



1. b is unobservable because there are uncontrollability indicators on every line in $\{l_b\}$; and
2. every line in $\{l_b\}$ is unreachable from s .

These conditions ensure that stem faults that can be detected by multiple-path sensitization are not marked as unobservable. The redundant faults are identified as those which cannot be activated (an s - a -0 fault on lines with 1_u and an s - a -1 fault on lines with 0_u) and those that cannot be propagated; both stuck-at faults on unobservable lines. The process of propagating uncontrollability and unobservability indicators is called *implication*.

A simple extension of this method based on an arbitrary illegal combination of values is as follows. We first form a list L of all fanout stems and reconvergent inputs of reconvergent gates in the circuit. For each line $c \in L$, we find the implications of $c = 0_u$ to determine all uncontrollable and unobservable lines. Let F_0 be the set of corresponding faults. Similarly, we find the implications of $c = 1_u$ to get the set F_1 . The redundant faults are in the set $F_0 \cap F_1$. The reason is that such faults simultaneously require c to have the values 0 and 1, which is not possible.

Example Consider the circuit in Fig. 8.28a. For this circuit, $L = \{x_1, x_2, c_6, c_7\}$, where x_1 and x_2 are fanout stems and c_6 and c_7 are reconvergent inputs (note that the fanouts from x_1 and x_2 reconverge at these inputs). Suppose that we target c_6 . The status $c_6 = 0_u$ does not imply the uncontrollability or unobservability of any other specific line. Hence, $F_0 = \{c_6 \text{ } s\text{-}a\text{-}1\}$. The status $c_6 = 1_u$ implies $x_3 = c_5 = c_1 = c_3 = x_1 = x_2 = c_2 = c_4 = c_7 = f = 1_u$. Since $c_6 = 1_u$ ($c_7 = 1_u$), the error propagating to c_7 (c_6) owing to any fault cannot propagate further to f . Hence,

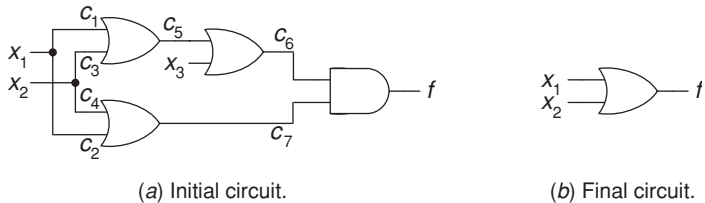


Fig. 8.28 An example circuit to illustrate the static method.

from uncontrollability and unobservability analysis, $F_1 = \{c_6 s-a-0, x_3 s-a-0, c_5 s-a-0, c_1 s-a-0, c_3 s-a-0, x_1 s-a-0, x_2 s-a-0, c_2 s-a-0, c_4 s-a-0, c_7 s-a-0, f s-a-0, c_6 s-a-1, x_3 s-a-1, c_5 s-a-1, c_1 s-a-1, c_3 s-a-1, c_7 s-a-1, c_2 s-a-1, c_4 s-a-1\}$. Since $F_0 \cap F_1 = \{c_6 s-a-1\}$, $c_6 s-a-1$ is redundant. Removing this fault using the simplification rules yields the circuit in Fig. 8.28b.

Dynamic direct methods Dynamic direct methods, like static direct methods, do not require an exhaustive search. However, they use a test generator to first identify a redundant fault. Thereafter, they identify additional redundant faults. They can remove identified redundancies in just one pass of test generation (but cannot guarantee a single stuck-at fault testable circuit at the end), in contrast with the multiple passes required for indirect methods. They also take advantage of the uncontrollability and unobservability analysis method introduced above.

We define the *region* of a redundant fault to be the subcircuit that can be removed because of it, using the simplification rules mentioned earlier. Also, we define the level of a gate in the circuit to be one more than the maximum level of any fanin of the gate, assuming that all primary inputs are at level 0. When the region of one redundant fault r_1 is contained within the region of another redundant fault r_2 , then it makes sense to target r_2 first. In general, with only a few exceptions this can be accomplished by targeting the faults at higher levels first for test generation. Once a redundant fault has been removed, we need to identify newly created redundancies (these are faults that would have been detectable had the removal not occurred). This can be done using the following theorem.

Theorem 8.2 *Let A be an output of a redundant region R and let G be the gate fed by A . Let c be the controlling value and i the inversion of G ($i = 0$ for a noninverting gate and $i = 1$ for an inverting gate). Assume that the combination consisting of the c' values on the remaining inputs of G and the $c \oplus i$ value on its output was feasible (legal) in the old circuit. Then this combination becomes illegal as a result of removal.*

The proof of this theorem is left to the reader as an exercise.

Once an illegal combination of values is identified, uncontrollability and unobservability analysis can identify the newly created redundancies. In doing so, we need to keep in mind that uncontrollability status indicators can be propagated forward and backward everywhere except through gate G . This allows us to identify newly created redundancies, as opposed to redundancies that would be present independently of whether the redundancy removal on the input of G occurred. Of all the newly created redundancies, only the highest level fault is removed and the above process repeated until no more newly created redundancies are found.

Example Consider the circuit in Fig. 8.25a again. Suppose that the test generator has identified x_1 *s-a-0* as redundant. The region R for this fault consists just of gate G_1 . This region feeds gate G_2 , whose controlling value is 1 and inversion 0. The combination $c_2 = 0, c_3 = 1$ was legal in the old circuit. However, once region R is removed, according to the above theorem this combination becomes illegal. This illegal combination can be translated in terms of uncontrollability status indicators as $c_2 = 0_u$ and $c_3 = 1_u$. The status 0_u on c_2 can be propagated backward. Using the notation and analysis introduced in the previous subsection, we obtain $S_{c_2}^0 = \{c_2$ *s-a-1*, x_2 *s-a-1*, c_4 *s-a-1* $\}$. Similarly, by propagating 1_u on c_3 forward and recognizing that the side-inputs of G_3 become unobservable, we obtain $S_{c_3}^1 = \{c_3$ *s-a-0*, f *s-a-0*, c_4 *s-a-0*, c_4 *s-a-1*, x_4 *s-a-0*, x_4 *s-a-1* $\}$. Since $S_{c_2}^0 \cap S_{c_3}^1 = \{c_4$ *s-a-1* $\}$, it follows that c_4 *s-a-1* is the newly redundant fault. After removing this fault as well, we obtain the circuit in Fig. 8.25c in just one pass of test generation. Note that earlier the indirect method required two passes to obtain this final circuit.

Don't-care-based direct methods A multi-level circuit consists of an interconnection of various logic blocks. Even if each logic block is individually irredundant, the multi-level circuit can still contain redundancies. These redundancies may stem from the fact that it may not be possible to feed certain input vectors to some embedded blocks in the circuit. These vectors constitute the *satisfiability don't-care set* (also called the *intermediate variable* or *fanin don't-care set*). Also, for certain input vectors, the output of the block may not be observable at a circuit output. These vectors constitute the *observability don't-care set* (also called the *transitive fanout don't-care set*). These don't-cares can be exploited to resynthesize the logic blocks in such a way that the multi-level circuit has fewer redundancies. Even if the original multi-level circuit is irredundant, this approach can frequently yield another irredundant circuit implementing the same functions with less area and delay.

Let the Boolean variable corresponding to node j , for $j = 1, 2, \dots, p$, of the multi-level circuit be f_j and the logic representation of f_j be F_j ; here “node” refers to the output of the logic blocks. The satisfiability don't-care set, $DSAT$, is common to all nodes and is defined as

$$DSAT = \sum_{j=1}^p DSAT_j,$$

where

$$DSAT_j = f_j \oplus F_j.$$

The expression $DSAT_j$ can be interpreted to mean that, since $f_j = F_j$, the condition $f_j \neq F_j$ is a don't-care.

Next, define the *cofactor* of a function g with respect to a literal l , denoted by g_l , as the function g with $l = 1$. Let the set of circuit outputs be PO . Then the observability don't-care set, $DOBS_j$, for each node j is defined as

$$DOBS_j = \prod_{i \in PO} DOBS_{ij},$$

where

$$DOBS_{ij} = [(F_i)_{f_j} \oplus (F_i)_{f_j'}]'$$

The expression $DOBS_j$ corresponds to a set of values at the primary inputs under which all the circuit outputs are insensitive to the value f_j that node j takes on.

Example Consider the circuit in Fig. 8.29a. Suppose that this circuit is partitioned into three logic blocks, as shown by the dotted boxes. Even though each logic block is individually irredundant, the circuit can be easily checked to be redundant. Since $f_3 = f_1 f_2 x'_1 x'_4 + x_1 x_3$, $DOBS_1 = [(f_2 x'_1 x'_4 + x_1 x_3) \oplus (x_1 x_3)]' = f'_2 + x_1 + x_4$. Therefore, $f_1 = x_1 x_2 + x_3$ can be simplified to just x_3 since x_1 is in $DOBS_1$, which includes $x_1 x_2$ in f_1 . Here, the interpretation is that the $x_1 x_2$ term in f_1 is not observable at circuit output f_3 . One can think of the don't-care minterms in $DOBS_1$ as having been superimposed on f_1 , resulting in a new, incompletely specified, function that needs to be synthesized, as shown in Fig. 8.30a. Similarly, one

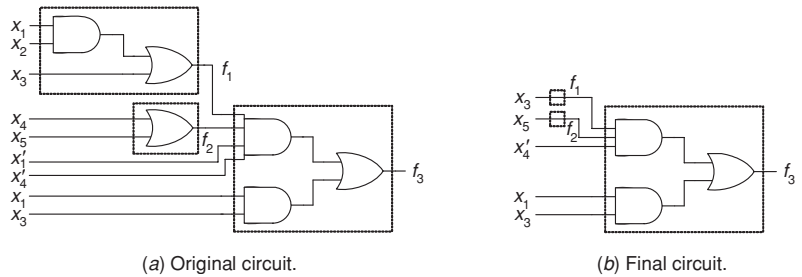


Fig. 8.29 Don't-care-based redundancy removal.

$x_3 \backslash x_1 x_2$	00	01	11	10
0			ϕ	ϕ
1	1	1	ϕ	ϕ

$x_5 \backslash x_4$	0	1
0		ϕ
1	1	ϕ

(a) Simplified $f_1 = x_3$.
(b) Simplified $f_2 = x_5$.

Fig. 8.30 Utilizing satisfiability and observability don't-cares.

can show that $DOBS_2 = f'_1 + x_1 + x_4$. Hence, $f_2 = x_4 + x_5$ can be simplified to just x_5 since x_4 is present in $DOBS_2$, as shown in Fig. 8.30b. Using the simplified equations $f_1 = x_3$ and $f_2 = x_5$, we can conclude that $DSAT_1 = f_1x'_3 + f'_1x_3$ and $DSAT_2 = f_2x'_5 + f'_2x_5$. Therefore f_3 can be simplified with respect to the don't-cares in $DSAT_1 + DSAT_2$. In other words, the don't-care minterms in $DSAT_1 + DSAT_2$ can be superimposed on f_3 , which gives us the simplified expression $f_3 = f_1f_2x'_4 + x_1x_3$; this follows since the consensus (see Section 3.1) of x_1x_3 and $f_1x'_3$ is x_1f_1 , which simplifies the term $f_1f_2x'_1x'_4$ to $f_1f_2x'_4$. The resultant irredundant circuit is shown in Fig. 8.29b.

Synthesis for delay fault testability

In this subsection, we concentrate on path delay and transition fault models, with primary emphasis on the former. We discuss the testability of two-level circuits and also transformations to preserve or enhance delay fault testability.

A circuit is said to be *robustly path delay fault testable* if robust two-pattern tests exist for every path delay fault in it.

Two-level circuits

A simple way to check whether a single-output two-level AND–OR circuit is robustly path delay fault testable is to use *tautology checking*.⁵ Suppose that we want to test a path starting with the literal l and going through AND gate G and the OR gate. Both the faults along this path, i.e., for the rising and falling transitions, are robustly testable if and only if, after making the side-input values of G equal to 1, the output values of the remaining AND gates can be made 0 using some input combination without using l or l' . Thus, we can first make the side-input values of G equal to 1, delete l and l' from the remaining products, and then delete G from the corresponding sum of products. If the remaining switching expression becomes a tautology then the path is not robustly testable; otherwise, it is.

Example Consider the two-level circuit in Fig. 8.31a, for which the corresponding sum of products is $f = x_1x_2 + x_1x'_3 + x'_1x_3$. Suppose that we want to robustly test the rising transition on the path through the literal x_1 in G_1 , as shown in bold. In order to do this, we first need to enable the side-input of G_1 by making $x_2 = 1$. Thereafter we delete G_1 , the literal x_1 from G_2 , and the literal x'_1 from G_3 , obtaining the reduced expression $f_{\text{red}} = x'_3 + x_3$. Since f_{red} reduces to 1 (i.e., it is a tautology), the literal, and hence the path, in question is not robustly testable. The reason is that in the transition from the initialization vector to the test vector in the two-pattern

⁵ A function is a tautology if it is 1 for all input vectors.

test for this path, the outputs of G_2 and G_3 could have a static-0 hazard (i.e., a spurious transition from 0 to 1 and back to 0), thus invalidating the two-pattern test. This can be seen from the partial value assignment shown in Fig. 8.31a (made using the table in Fig. 8.18). One can see that no assignment is possible for x_3 that will result in $S0$ at the outputs of both G_2 and G_3 .

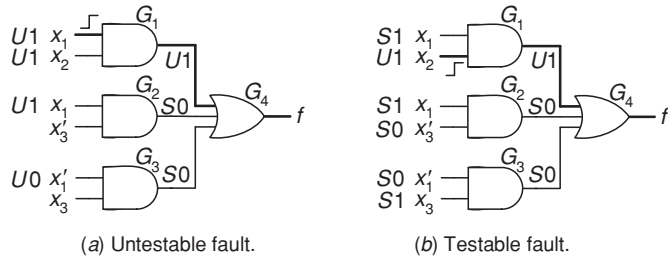


Fig. 8.31 Robust path delay fault testability of two-level circuits.

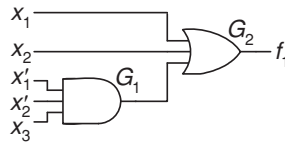
Next, consider the path through the literal x_2 in G_1 , as highlighted in bold in Fig. 8.31b. To test this path, after making $x_1 = 1$ we obtain $f_{\text{red}} = x'_3$, which can be made 0 by setting $x_3 = 1$. Figure 8.31b shows the possible value assignments. Note that since $S1$ is a more stringent condition than $U1$, assigning $S1$ instead of $U1$ to x_1 is perfectly valid. Therefore, a robust test for a rising transition on this path is $\{(1, 0, 1), (1, 1, 1)\}$. Reversing the two vectors, we get a robust test for the falling transition. Readers can check that all the other paths in this circuit are also robustly testable.

An interesting point to note here is that implementing a circuit based on an irredundant sum of products is a necessary condition for robust testability but not a sufficient one. This stems from the fact that if the circuit is not based on an irredundant sum of products then a stuck-at fault in it will be untestable and, hence, the path going through the fault will not be robustly testable. However, the above example shows that, even when the circuit is based on an irredundant sum of products, the robust testability property is not guaranteed.

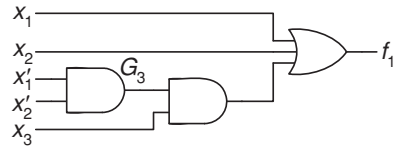
To check whether a multiple-output two-level circuit is robustly testable, one can simply check whether the above conditions are satisfied for paths starting from each literal to each circuit output it feeds.

In order to verify that a two-level circuit is robustly testable for all transition faults, one just needs to verify that at least one path going through each gate in it is robustly testable for rising and falling transitions. Using an irredundant sum of products is neither necessary nor sufficient for the robust testability of all transition faults in a two-level circuit, as the following example shows.

Example Consider the two-level circuit based on the expression $f_1 = x_1 + x_2 + x'_1 x'_2 x_3$, as shown in Fig. 8.32a. This is not an irredundant sum-of-products expression, yet the slow-to-rise and slow-to-fall transition faults (see the end of Section 8.1) at the outputs of both G_1 and G_2 are robustly testable. However, consider the irredundant sum-of-products expression $f_2 = x_1 x_3 + x_1 x_2 + x'_1 x'_2 + x_3 x_4 + x'_3 x'_4$. Even though it is irredundant, the transition faults at the output of the first AND gate with inputs x_1 and x_3 are not robustly testable since neither of the two paths starting from these two literals is robustly testable.



(a) Robustly testable.



(b) Non-robustly testable.

Fig. 8.32 Transition fault testability.

Multi-level circuits

Various methods have been presented for obtaining nearly 100% or fully robustly testable multi-level circuits. We consider some of them here.

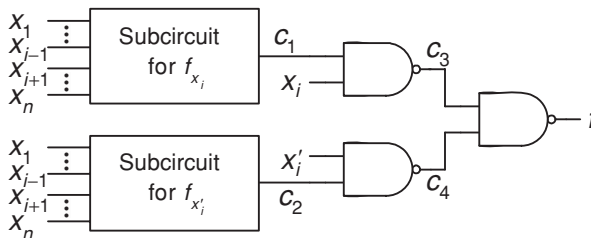
Shannon's decomposition Shannon's decomposition can be used for obtaining completely robustly testable multi-level circuits. This method results in a circuit that is testable for all multiple stuck-at faults, multiple stuck-open faults, and a combination of these faults, using a particular path delay fault test set.

Shannon's decomposition theorem states that

$$f(x_1, x_2, \dots, x_n) = x_i f_{x_i} + x'_i f_{x'_i},$$

where f_{x_i} and $f_{x'_i}$ are cofactors of the function f with respect to the variable x_i and are obtained by making $x_i = 1$ and $x_i = 0$, respectively, in f . The corresponding decomposed circuit is shown in Fig. 8.33. The importance of this theorem in the present context is that one can show that if f is *binate* in x_i (i.e., f depends on both x_i and x'_i) then the decomposed circuit for f is robustly

Fig. 8.33 Circuit based on Shannon's decomposition [17]
© 1988, IEEE.



testable if the subcircuits for the two cofactors are robustly testable. The reason is that for such a decomposition one can always find at least one vector which, when applied to $x_1, x_2, \dots, x_{i-1}, x_{i+1}, \dots, x_n$, results in $f_{x_i} = 1$ and $f_{x'_i} = 0$ ($f_{x_i} = 0$ and $f_{x'_i} = 1$), which allows the path $x_i c_3 f (x'_i c_4 f)$ to be robustly tested. In addition, making $x_i = 1$ ($x_i = 0$) allows us to test the subcircuit for f_{x_i} ($f_{x'_i}$) fully by feeding the robust tests to the corresponding subcircuits. It is possible that the subcircuits will not be robustly testable after one decomposition. Then the method can be applied recursively to the cofactors until a robustly testable circuit is obtained. This method is guaranteed to terminate in a robustly testable circuit since after at most $n - 2$ Shannon decompositions we shall get a two-variable cofactor which is guaranteed to be robustly testable. In fact, one can stop further decomposition if the cofactor is unate in all its variables since robust tests can be found for each path in such a subcircuit in which the initialization and test vectors differ in just the literal being tested. Of course, even if the cofactor is binate in some of its variables, further decomposition can be stopped if the corresponding subcircuit is already robustly testable. Furthermore, the sharing of logic among the cofactor subcircuits does not compromise the robust testability property.

As a useful heuristic for determining which binate variable to target first for decomposition, one can simply choose the variable that appears the most times in complemented or uncomplemented form in the given sum-of-products; alternatively, one can choose a variable that leads to robust untestability in a maximum number of gates.

Example Consider the two-level circuit based on the expression $f_2 = x_1 x_3 + x_1 x_2 + x'_1 x'_2 + x_3 x_4 + x'_3 x'_4$, which we considered in the previous example. The only robustly untestable literals are x_1 and x_3 in the first AND gate. Using one of the above two heuristics, we choose either x_1 or x_3 . If we choose x_1 , we obtain the decomposition

$$f_2 = x_1(x_2 + x_3 + x'_4) + x'_1(x'_2 + x_3 x_4 + x'_3 x'_4).$$

Since the two cofactors are robustly testable, so is the decomposed circuit for f_2 .

Algebraic factorization Readers may not be surprised, having been presented various examples of testability preservation based on algebraic factorization for other fault models, that it plays an important role in delay fault testability as well. Many variations on this factorization technique have been shown to be useful, as we now discuss.

If the given circuit is completely robustly testable for path delay faults then algebraic factorization with a constrained use of the complement maintains its robust testability. Furthermore, the robust test set is also preserved after factorization. The only problem that limits the usefulness of this approach is

that, frequently, two-level circuits based on an irredundant sum of products, which often form the starting point for multi-level logic synthesis are not themselves completely robustly testable. In fact, simple functions exist for which none of the irredundant sum-of-products expressions are robustly testable. We have already seen an example of such a function: $f = x_1x_2 + x_1x'_3 + x'_1x_3$. The only other irredundant sum-of-products expression for this function is $f = x_2x_3 + x_1x'_3 + x'_1x_3$, which also is not robustly testable. In fact it may happen that one such expression of a given function is robustly testable, but another is not.

Example The function $f = x_1x'_2 + x'_1x_2 + x_1x'_3 + x'_1x_3$ is not robustly testable in the literal x_1 in $x_1x'_2$. However, another implementation of this function, $f = x_1x'_2 + x'_1x_3 + x_2x'_3$, is robustly testable.

One can use a heuristic to bias the two-level logic synthesizer towards robustly testable implementations, whenever it is possible to do so. Define a *relatively essential vertex* of a prime implicant in a sum of products to be a minterm that is not contained in any other prime implicant of the sum of products. Also, define the *ON-set (OFF-set)* of a function to be the set of vertices for which the function is 1 (0). Then the above-mentioned heuristic tries to maximize the number of relatively essential vertices in the prime implicants that are just one bit different from some vertex in the OFF-set of the function. This increases the probability that the necessary and sufficient conditions for robust testability presented earlier will be met. After that, algebraic factorization can be used to obtain a highly robustly testable multi-level circuit.

Surprisingly, algebraic factorization does not preserve the robust transition fault testability property.

Example Consider the function $f_1 = x_1 + x_2 + x'_1x'_2x_3$. Its robustly transition fault testable implementation is shown in Fig. 8.32a. However, after one possible algebraic factorization, we obtain the circuit in Fig. 8.32b, which is not robustly transition fault testable since no path through gate G_3 is robustly testable. To preserve robust testability for transition faults, we need to use a constrained form of algebraic factorization in which each cube in each factor should have at least one path through it that is robustly testable. Thus, in the above example, if we had used x'_1x_3 or x'_2x_3 as a factor instead of $x'_1x'_2$, the robust testability property would have been maintained.

Since many implementations based on irredundant sum-of-products expressions are not robustly testable for path delay faults, another method based on *targeted algebraic factorization* can be used; this results in a robustly testable multi-level circuit in the vast majority of cases in which the original two-level circuit is not robustly testable.

The main idea here is first to convert the two-level circuit, which is not robustly testable, into an intermediate circuit (generally with three or four levels) that is robustly testable, making targeted use of the distributive law from switching algebra. Then algebraic factorization with a constrained use of the complement can be employed, as before, to obtain a circuit with more levels but which is robustly testable. Consider the irredundant sum-of-products expression $f = \sum_{j=1}^{q_1} x_1 x_2 \cdots x_n M_j + \sum_{j=1}^{q_2} N_j$, where M_j and N_j are products of literals. Suppose that (i) in each product term in f , all literals in M_j are robustly testable, (ii) each literal in the set $\{x_1, x_2, \dots, x_n\}$ is robustly testable in at least one product term in f , and (iii) the other literals in f are not necessarily robustly testable. Then the literals $x_1, x_2, \dots, x_n$ are robustly testable when factored out from the first set of product terms, resulting in the following modified expression, $f = x_1 x_2 \cdots x_n (\sum_{j=1}^{q_1} M_j) + \sum_{j=1}^{q_2} N_j$. Furthermore, all literals in $\sum_{j=1}^{q_1} M_j$ remain robustly testable, and the robust testability (or lack thereof) of the literals in $\sum_{j=1}^{q_2} N_j$ is not affected. This synthesis rule may have to be applied more than once to obtain a robustly testable circuit.

Example Consider the irredundant sum-of-products expression $f_1 = x_1 x_2 x_3^* + x_1^* x_3 x_4 + x_1' x_2' x_4 + x_3' x_4'$, where the starred literals are not robustly testable. Using the above synthesis rule, we obtain the modified expression $f_1 = x_1 x_3 (x_2 + x_4) + x_1' x_2' x_4 + x_3' x_4'$, which is completely robustly testable. Consider another expression, $f_2 = x_1 x_2^* M_1 + x_1^* x_2 M_2 + x_1^* x_2^* M_3 + x_1^* M_4 + N_1$. Applying the synthesis rule once we get $f_2 = x_1 x_2 (M_1 + M_2 + M_3) + x_1^* M_4 + N_1$. After applying it again we get $f_2 = x_1 [x_2 (M_1 + M_2 + M_3) + M_4] + N_1$, where both x_1 and x_2 are now robustly testable.

Even when the synthesis rule is not successful with the sum-of-products for f , it is frequently successful with the sum-of-products for f' , which can then be followed by an inverter to get f .

It is worth recalling that both simple and targeted algebraic factorizations also result in a multi-level circuit that is completely multiple stuck-at fault testable, using the results described earlier.

Repeated Shannon decomposition, although it guarantees the robust testability property, is not very area efficient. However, area-efficient methods based on the different variations of algebraic factorization cannot guarantee the robust testability property. Hence a possible compromise to obtain both area efficiency and guaranteed robust testability is to marry these two techniques. For the sake of area efficiency, we should generally do this only if targeted algebraic factorization has failed with the switching expression and its complement. After applying Shannon decomposition once to such an expression, we first determine whether the cofactors are already robustly testable or else amenable to targeted algebraic factorization. Only when the answer is negative do we consider applying Shannon decomposition again. Cases where Shannon decomposition needs

to be applied more than once are extremely rare, however. Since multiple stuck-at fault testability is guaranteed by both Shannon decomposition and targeted algebraic factorization, it is also guaranteed by their combination.

8.6 Testing for nanotechnologies

In Chapter 7, we saw that various nanotechnologies implement threshold and majority gates. Since majority gates are also a specific type of threshold gate, it is useful to see how test generation can be achieved for threshold networks and how redundancies can be removed.

Test generation

Let us first see how we can derive a test vector for a single stuck-at fault in a threshold gate. Let $f(x_1, x_2, \dots, x_n)$ be the corresponding threshold function. The s -a-0 fault at input x_i can be activated by $x_i = 1$. In the fault-free case, we have

$$\sum_{j=1, j \neq i}^n w_j x_j + w_i \geq T \quad \Rightarrow \quad f = 1 \quad (8.1)$$

or

$$\sum_{j=1, j \neq i}^n w_j x_j + w_i < T \quad \Rightarrow \quad f = 0. \quad (8.2)$$

Moving w_i to the right-hand side of the inequalities results in

$$\sum_{j=1, j \neq i}^n w_j x_j \geq T - w_i \quad \Rightarrow \quad f = 1 \quad (8.3)$$

or

$$\sum_{j=1, j \neq i}^n w_j x_j < T - w_i \quad \Rightarrow \quad f = 0. \quad (8.4)$$

When the fault is present, we have

$$\sum_{j=1, j \neq i}^n w_j x_j \geq T \quad \Rightarrow \quad f = 1 \quad (8.5)$$

or

$$\sum_{j=1, j \neq i}^n w_j x_j < T \quad \Rightarrow \quad f = 0. \quad (8.6)$$

A test vector can be found for the x_i s -a-0 fault by finding an assignment on the input variables with $x_i = 1$ such that either Eqs. (8.3) and (8.6) or Eqs. (8.4) and (8.5) are satisfied.

A similar analysis for deriving a test vector for x_i s - a -1 shows that the constraints are the same except that now Eqs. (8.3) and (8.4) (Eqs. (8.5) and 8.6)) refer to the faulty (fault-free) cases. This leads to the following general result.

Theorem 8.3 *Given a threshold gate implementing the threshold function $f(x_1, x_2, \dots, x_n)$, to find test vectors for x_i s - a -0 and x_i s - a -1 faults we must find an assignment of values to the remaining input variables such that one of the following inequalities is satisfied:*

$$T - w_i \leq \sum_{j=1, j \neq i}^n w_j x_j < T \quad (8.7)$$

or

$$T \leq \sum_{j=1, j \neq i}^n w_j x_j < T - w_i. \quad (8.8)$$

If an assignment exists then it, along with $x_i = 1$ ($x_i = 0$) is a test vector for x_i s - a -0 (s - a -1). If no assignment exists then both faults are untestable and, therefore, redundant.

Proof This is obvious from the above discussion. $\diamond$

Example Consider a threshold gate that realizes the threshold function $f(x_1, x_2, x_3) = x_1x_2 + x_1x_3$, with weight–threshold vector $\langle 2, 1, 1; 3 \rangle$. Table 8.2 shows an exhaustive list of the test vectors (in bold) for each fault. For example, to test for x_1 s - a -0, the inequalities to be satisfied are $1 \leq \sum_{j=2}^3 w_j x_j < 3$ or $3 \leq \sum_{j=2}^3 w_j x_j < 1$. This leads to three test vectors: 101, 110, and 111. The test vectors for x_1 s - a -1 can be obtained just by replacing $x_1 = 1$ with $x_1 = 0$ in the original test vectors. The new vectors are 001, 010, and 011.

Table 8.2 Test vectors for stuck-at faults in a threshold gate implementing $f = x_1x_2 + x_1x_3$

x_1	x_2	x_3	f	x_1 s - a -0	x_1 s - a -1	x_2 s - a -0	x_2 s - a -1	x_3 s - a -0	x_3 s - a -1	f s - a -0	f s - a -1
0	0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	1	0	0	0	0	0	1
0	1	0	0	0	1	0	0	0	0	0	1
0	1	1	0	0	1	0	0	0	0	0	1
1	0	0	0	0	0	0	1	0	1	0	1
1	0	1	1	0	1	1	1	0	1	0	1
1	1	0	1	0	1	0	1	1	1	0	1
1	1	1	1	0	1	1	1	1	1	0	1

To derive a test set for detecting all single stuck-at faults in a threshold network using the D -algorithm, we need to derive the PDCF, singular covers, and propagation D -cubes. A PDCF for the faulty threshold gate can be obtained on the basis of the above discussion. For example, the three vectors that detect x_1 s-a-0 in the above example give rise to the following PDCFs: $101D$, $110D$, and $111D$. The singular cover of a threshold gate can be derived in a straightforward fashion using its truth table. We next discuss how propagation D -cubes can be obtained.

Propagation D -cubes are used in the D -algorithm to sensitize a path from the fault site to one or more of the circuit outputs. Knowing the threshold function that is implemented by a threshold gate, we can use algebraic substitution to obtain the propagation D -cubes.

Example To obtain the propagation D -cubes from x_1 in $f(x_1, x_2, x_3) = x_1x_2 + x_1x_3$, substituting D for x_1 in f we get $Dx_2 + Dx_3$. For the fault to propagate, only the cubes containing D (or D') should be activated in f . In this case, since both cubes contain D , activating either or both cubes will result in a propagation D -cube. Whether a vector propagates D or D' to the output can be determined in a straightforward manner: this depends upon whether it satisfies Eq. (8.7) or (8.8), respectively. Table 8.3 shows this. Hence, the propagation D -cubes from x_1 are $\{D10D, D01D, D11D\}$. Of course, $\{D'10D', D'01D', D'11D'\}$ are also propagation D -cubes.

Table 8.3 Error propagation

Input error signal	Eq. (8.7)	Eq. (8.8)
D	D	D'
D'	D'	D

Example To propagate a D on x_1 in $f(x_1, x_2, x_3) = x_1x_2 + x_1x_3$, we observe that if $x_2 = 1$ and $x_3 = 0$ then Eq. (8.7) is satisfied. Thus, $f = D$. However, to propagate a D on x_1 in $g(x_1, x_2, x_3) = x'_1x_2 + x'_1x_3$ (a threshold function with weight-threshold vector $\langle -2, 1, 1; 1 \rangle$), we observe that if $x_2 = 1$ and $x_3 = 0$ then Eq. (8.8) is satisfied. In this case, $f = D'$.

Using the above methods to obtain PDCFs, singular covers, and propagation D -cubes, the D -algorithm can be applied directly to threshold networks. The fault collapsing theorem (Theorem 8.1) applicable to Boolean networks is also applicable to threshold networks. Demonstrating this is left as an exercise for the reader (see Problem 8.25).

Example Consider the threshold network in Fig. 8.34. Suppose that we want to derive a test vector for x_1 s-a-1. The PDCF for the fault shown in gate G_1 is 0000 D' . Using the propagation D -cube of gate G_2 as shown, the fault effect can be propagated to circuit output f_1 . This requires 1 to be justified on line c_2 through the application of the relevant singular cube to gate G_3 , as shown. Thus, a test vector for the above fault is $(0, 0, 0, 0, \phi, 1, 0, 0, \phi, \phi)$.

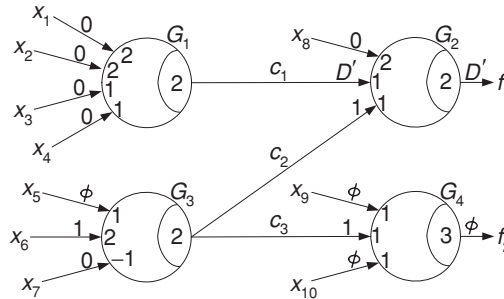


Fig. 8.34 Testing for x_1 s-a-1 [11] © 2008, IEEE.

Redundancy removal

If a stuck-at fault in a threshold network is redundant then corresponding lines and gates can be removed without affecting the functionality of the circuit.

Figure 8.35 shows a fault-free threshold gate and its faulty representations for two faults. For an input s-a-0 fault, the weight of that input becomes 0. Hence, the input can be simply removed. For an input x_i s-a-1 fault, the weight of that input becomes w_i . Hence, the input can be removed and the threshold of the gate lowered by w_i .

When a stuck-at fault in a threshold network is redundant, all gates and lines in the subnetwork that do not fan out to other parts of the circuit and that feed the removed line can be removed from the network.

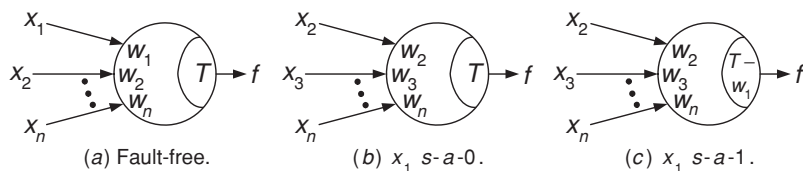


Fig. 8.35 Representations for fault-free and faulty threshold gates [11] © 2008, IEEE.

Example Consider the threshold network shown in Fig. 8.36a. Suppose that c_1 s -a-0 (s -a-1) is redundant; then the simplified network shown in Fig. 8.36b (Fig. 8.36c) can be obtained.

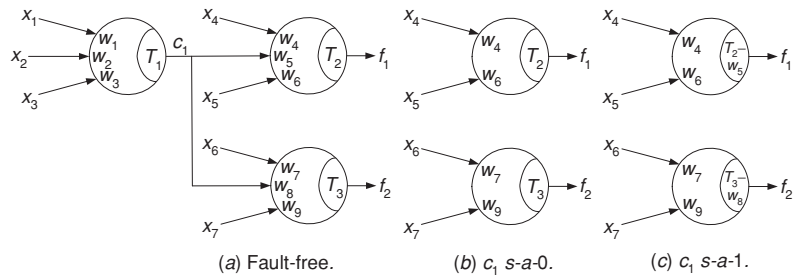


Fig. 8.36 Redundancy removal in a threshold network [11] © 2008, IEEE.

Notes and references

The stuck-open and stuck-on fault models were discussed by Wadsack [37]. Monitoring the current drawn by a CMOS circuit to detect stuck-on faults in it was first studied by Malaiya and Su [23]. Using current monitoring to detect bridging faults was discussed by Levi [20]. The transition fault model was investigated by Hsieh *et al.* [13] and Storey and Barry [35]. The path delay fault model was investigated by Lesser and Shedletsky [19] and Smith [34]. The nonrobust testing of delay faults was discussed by Smith [34] and Lin and Reddy [21] and robust testing by Savir and McAnney [32], Pramanick and Reddy [27] and Devadas and Keutzer [8]. The concept of a validatable nonrobust test was presented by Reddy *et al.* [29].

Path sensitization was studied by Armstrong [2]. Fault equivalence methods were discussed by McCluskey and Clegg [24] and fault dominance by Poage and McCluskey [26]. The D -algorithm was presented by Roth [31].

Techniques for bridging fault collapsing were presented by Reddy *et al.* [30] and efficient testing methodology by Thadikaran and Chakravarty [36].

The variable and rated clock schemes were discussed in [4] by Bose *et al.* The five-valued logic system was used by Lin and Reddy [21] and Cheng *et al.* [6] for path delay fault testing. These works form the basis for the discussion in Section 8.4. Transition and gate delay fault detection methods were discussed by Park and Mercer [25] and Mahlstedt [22]. The at-speed test generation method is due to Bose *et al.* [5]. Gharaybeh *et al.* presented a classification of path delay faults [10].

It was shown by Kohavi and Kohavi [16] and Schertz and Metze [33] that a test set for all single stuck-at faults in a two-level circuit based on an irredundant sum of products also detects all multiple stuck-at faults in it. Transformations for preserving single stuck-at fault testability were presented by Rajsiki and Vasudevamurthy [28]. Transformations for preserving multiple stuck-at fault testability were presented by Hachtel *et al.* [12]. Synthesis-for-full-testability methods involving the deliberate introduction of redundancies were presented by Entrena and Cheng [7]. An efficient static redundancy identification method was introduced by Iyer and Abramovici [14]. A dynamic

redundancy identification method was given by Abramovici and Iyer [1]. The concepts of satisfiability and observability don't-care sets were first presented by Bartlett *et al.* [3]. Necessary and sufficient conditions for robust delay fault testability were discussed by Lin and Reddy [21] and Devadas and Keutzer [8]. The first method, based on Shannon's decomposition, for guaranteeing completely robustly testable multi-level circuits was presented by Kundu and Reddy [17]. This method was later shown to guarantee testability of all multiple stuck-at faults, multiple stuck-open faults, and their combinations by Kundu *et al.* [18]. Heuristics for robust path delay fault testability based on simple and targeted algebraic factorizations were presented by Devadas and Keutzer [9] and by Jha *et al.* [15], respectively. Testing of threshold networks was considered by Gupta *et al.* [11].

- [1] Abramovici, M., and M. A. Iyer: "One-pass redundancy identification and removal," in *Proc. Int. Test Conf.*, pp. 807–815, October 1992.
- [2] Armstrong, D. B.: "On finding a nearly minimal test set of fault detection tests for combinational logic nets," *IEEE Trans. Electronic Computers*, vol. EC-15, pp. 66–73, February 1966.
- [3] Bartlett, K. A., R. K. Brayton, G. D. Hachtel, *et al.*: "Multilevel logic minimization using implicit don't-cares," *IEEE Trans. Computer-Aided Design*, vol. 7, no. 6, pp. 723–740, June 1988.
- [4] Bose, S., P. Agrawal, and V. D. Agrawal: "A rated-clock test method for path delay faults," *IEEE Trans. Very Large Scale Integration Systems*, vol. 6, no. 2, pp. 323–342, June 1998.
- [5] Bose, S., P. Agrawal, and V. D. Agrawal: "Deriving logic systems for path delay test generation," *IEEE Trans. Computers*, vol. 47, no. 8, pp. 829–846, August 1998.
- [6] Cheng, K.-T., A. Krstic, and H.-C. Chen: "Generation of high quality tests for robustly untestable path delay faults," *IEEE Trans. Computers*, vol. 45, no. 12, pp. 1379–1392, December 1996.
- [7] Entrena, L., and K.-T. Cheng: "Combinational and sequential logic optimization by redundancy addition and removal," *IEEE Trans. Computer-Aided Design*, vol. 14, no. 7, pp. 909–916, July 1995.
- [8] Devadas, S., and K. Keutzer: "Synthesis of robust delay fault testable circuits: theory," *IEEE Trans. Computer-Aided Design*, vol. 11, no. 1, pp. 87–101, January 1992.
- [9] Devadas, S., and K. Keutzer: "Synthesis of robust delay fault testable circuits: practice," *IEEE Trans. Computer-Aided Design*, vol. 11, no. 3, pp. 277–300, March 1992.
- [10] Gharaybeh, M. A., M. L. Bushnell, and V. D. Agrawal: "Classification and test generation for path-delay faults using single stuck-fault test," in *Proc. Int. Test Conf.*, pp. 139–148, Oct. 1995.
- [11] Gupta, P., R. Zhang, and N. K. Jha: "Automatic test pattern generation for combinational threshold logic networks," *IEEE Trans. VLSI Systems*, vol. 16, no. 8, pp. 1035–1045, Aug. 2008.
- [12] Hachtel, G. D., R. M. Jacoby, K. Keutzer, and C. R. Morrison: "On properties of algebraic transformations and the synthesis of multifault-irredundant circuits," *IEEE Trans. Computer-Aided Design*, vol. 11, no. 3, pp. 313–321, March 1992.

- [13] Hsieh, E. P., R. A. Rasmussen, L. J. Vidunas, and W. T. Davis: "Delay test generation," in *Proc. Design Automation Conf.*, pp. 486–491, June 1977.
- [14] Iyer, M. A., and M. Abramovici: "FIRE: a fault-independent combinational redundancy identification algorithm," *IEEE Trans. Very Large Scale Integration Systems*, vol. 4, no. 2, pp. 295–301, June 1996.
- [15] Jha, N. K., I. Pomeranz, S. M. Reddy, and R. J. Miller: "Synthesis of multi-level combinational circuits for complete robust path delay fault testability," in *Proc. Int. Symp. Fault-Tolerant Computing*, pp. 280–287, June 1992.
- [16] Kohavi, I., and Z. Kohavi: "Detection of multiple faults in combinational networks," *IEEE Trans. Computers*, vol. C-21, no. 6, pp. 556–568, June 1972.
- [17] Kundu, S., and S. M. Reddy: "On the design of robust testable combinational logic circuits," in *Proc. Int. Symp. Fault-Tolerant Computing*, pp. 220–225, June 1988.
- [18] Kundu, S., S. M. Reddy, and N. K. Jha: "Design of robustly testable combinational logic circuits," *IEEE Trans. Computer-Aided Design*, vol. 10, no. 8, pp. 1036–1048, August 1991.
- [19] Lesser, J. P., and J. J. Shedletsky: "An experimental delay test generator for LSI logic," *IEEE Trans. Computers*, vol. C-29, no. 3, pp. 235–248, March 1980.
- [20] Levi, M. W.: "CMOS is most testable," in *Proc. Int. Test Conf.*, pp. 217–220, October 1981.
- [21] Lin, C. J., and S. M. Reddy: "On delay fault testing in logic circuits," *IEEE Trans. Computer-Aided Design*, vol. 6, no. 9, pp. 694–703, September 1987.
- [22] Mahlstedt, U.: "DELTEST: deterministic test generation for gate delay faults," in *Proc. Int. Test Conf.*, pp. 972–980, October 1993.
- [23] Malaiya, Y. K., and S. Y. H. Su: "A new fault model and testing technique for CMOS devices," in *Proc. Int. Test Conf.*, pp. 25–34, October 1982.
- [24] McCluskey, E. J., and F. W. Clegg: "Fault equivalence in combinational circuits," *IEEE Trans. Computers*, vol. 20, no. 11, pp. 1286–1293, November 1971.
- [25] Park, E. S., and M. R. Mercer: "An efficient delay test generation system for combinational logic circuits," *IEEE Trans. Computer-Aided Design*, vol. 11, no. 7, pp. 926–938, July 1992.
- [26] Poage, J. F., and E. J. McCluskey: "Derivation of optimal test sequences for sequential machines," in *Proc. Symp. Switching Theory and Logic Design*, pp. 121–132, 1964.
- [27] Pramanick, A. K., and S. M. Reddy: "On the design of path delay fault testable combinational circuits," in *Proc. Int. Symp. Fault-Tolerant Computing*, pp. 374–381, June 1990.
- [28] Rajski, J., and J. Vasudevamurthy: "The testability-preserving concurrent decomposition and factorization of Boolean expressions," *IEEE Trans. Computer-Aided Design*, vol. 11, no. 6, pp. 778–793, June 1992.
- [29] Reddy, S. M., C. J. Lin, and S. Patil: "An automatic test pattern generator for the detection of path delay faults," in *Proc. Int. Conf. Computer-Aided Design*, pp. 284–287, November 1987.
- [30] Reddy, R. S., I. Pomeranz, S. M. Reddy, and S. Kajihara: "Compact test generation for bridging faults under I_{DDQ} testing," in *Proc. VLSI Test Symp.*, pp. 310–316, April 1995.
- [31] Roth, J. P.: "Diagnosis of automata failures: a calculus and a method," *IBM J. Research & Development*, vol. 10, pp. 278–291, July 1966.

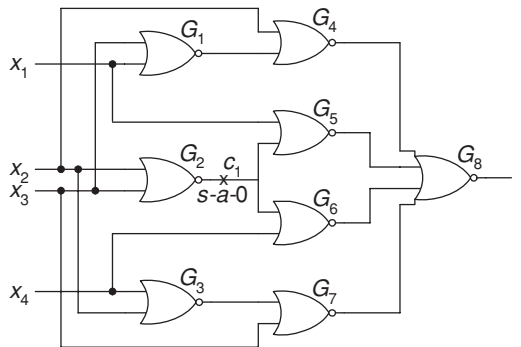
- [32] Savir, J., and W. H. McAnney: "Random pattern testability of delay faults," in *Proc. Int. Test Conf.*, pp. 263–273, October 1986.
- [33] Schertz, D., and G. Metze: "A new representation for faults in combinational digital circuits," *IEEE Trans. Computers*, vol. C-21, no. 8, pp. 858–866, August 1972.
- [34] Smith, G. L.: "A model for delay faults based on paths," in *Proc. Int. Test Conf.*, pp. 342–349, October 1985.
- [35] Storey, T. M., and J. W. Barry: "Delay test simulation," in *Proc. Design Automation Conf.*, pp. 492–494, June 1977.
- [36] Thadikaran, P., and S. Chakravarty: "Fast algorithms for computing I_{DDQ} tests for combinational circuits," in *Proc. Int. Conf. VLSI Design*, pp. 103–106, January 1996.
- [37] Wadsack, R. L.: "Fault modeling and logic simulation of CMOS and MOS integrated circuits," *Bell System Tech. J.*, vol. 57, no. 5, pp. 1449–1474, 1978.

Problems

Problem 8.1. In the circuit in Fig. P8.1, suppose that we want to obtain a test vector for the c_1 s - a -0 fault.

- (a) Show that one-dimensional path sensitization through gates G_5 and G_8 or G_6 and G_8 does not yield such a test vector.
- (b) Obtain a test vector by sensitizing both the above paths simultaneously.

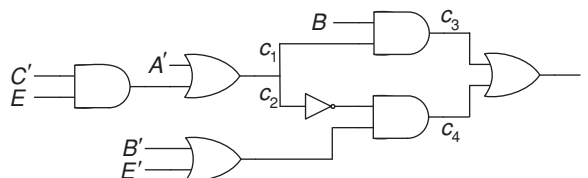
Fig. P8.1



Problem 8.2. For the circuit in Fig. P8.2:

- (a) Find all the test vectors that detect input A' s - a -0 by using the D -algorithm.
- (b) Show all the single stuck-at faults that can be detected by the test vector $(A, B, C, E) = (1, 1, 1, 1)$.

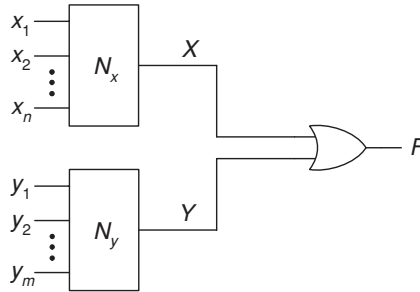
Fig. P8.2



Problem 8.3. Let N_x and N_y in Fig. P8.3 be combinational networks. To test N_x , we need n_x^0 test vectors that result in $X = 0$ and n_x^1 test vectors that result in $X = 1$. Similarly, to test N_y , we need n_y^0 and n_y^1 test vectors.

- Define n_f^0 and n_f^1 in a similar manner and find minimal values for them in terms of $n_x^0, n_x^1, n_y^0, n_y^1$.
- Repeat (a) when the OR gate in Fig. P8.3 is replaced by a NAND gate.

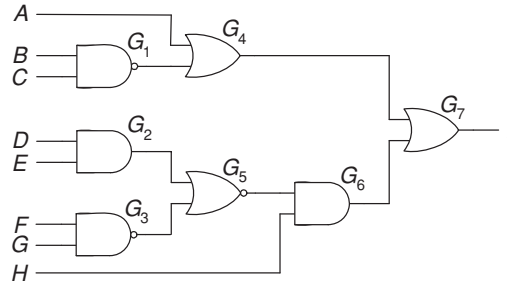
Fig. P8.3



Problem 8.4. The test vector $(A, B, C, D, E, F, G, H) = (0, 1, 1, 1, 1, 1, 1, 1)$ was applied to the circuit shown in Fig. P8.4 and output f indicated an error.

- What are the single stuck-at faults in this network that could cause the output to be erroneous?
- Which of the faults in (a) are equivalent?

Fig. P8.4



Problem 8.5. Derive all test vectors that will detect the multiple stuck-at fault consisting of x_3 $s-a-1$, c_1 $s-a-0$, and c_2 $s-a-0$ in the circuit shown in Fig. 8.1.

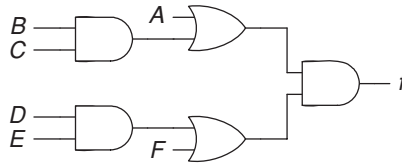
Problem 8.6. The following will demonstrate that a test set which detects all single stuck-at faults in a fanout-free network does not *necessarily* detect all multiple stuck-at faults in it as well.

- Show that the following test set detects all single stuck-at faults in the network of Fig. P8.6:

$$(A, B, C, D, E, F) = \{(1, 1, 1, 0, 1, 0), (0, 0, 1, 0, 0, 1), (0, 1, 1, 1, 1, 0), (1, 0, 0, 1, 0, 0), (1, 0, 1, 1, 0, 1), (0, 1, 0, 1, 1, 1)\}$$

- Prove that the multiple fault consisting of the four faults A and F $s-a-0$ and B and E $s-a-1$ is not detected by the test set in (a).

Fig. P8.6



Problem 8.7. Derive a minimal test set to detect all single stuck-open faults in the two-input NAND gate shown in Fig. 8.3b.

Problem 8.8. Assuming that I_{DDQ} testing is used, derive a minimal test set for all single stuck-on faults in the two-input NOR gate shown in Fig. 8.3a.

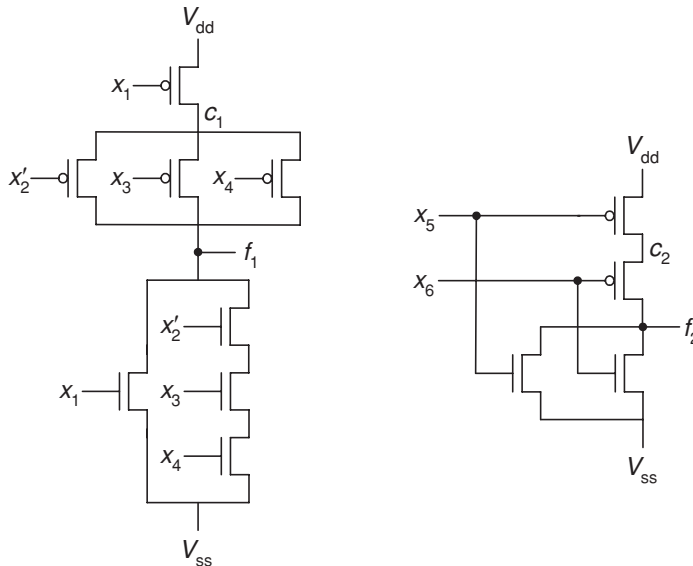
Problem 8.9. Assuming that I_{DDQ} testing is used, derive all test vectors that will detect the bridging fault shown in Fig. 8.4.

Problem 8.10. In the circuit in Fig. 8.11:

- How many gate-level two-node feedback and nonfeedback bridging faults are there?
- How many of the gate-level two-node bridging faults remain after fault collapsing?
- Of the collapsed set of bridging faults, how many are detected by following test set T applied to $(x_1, x_2, x_3, x_4, x_5)$: $\{(1, 0, 0, 1, 0), (1, 1, 0, 0, 0), (0, 1, 1, 1, 1)\}$?

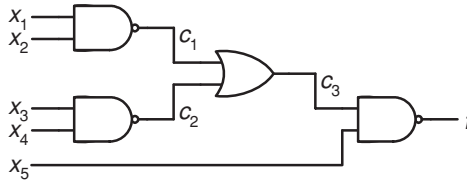
Problem 8.11. For the circuit shown in Fig. P8.11, derive a gate-level model for detecting the bridging fault $\langle c_1, c_2 \rangle$. Obtain all possible test vectors for this bridging fault by targeting the appropriate stuck-at fault in the gate-level model.

Fig. P8.11



Problem 8.12. For the circuit shown in Fig. P8.12, a test set that detects all gate-level two-node bridging faults is $\{(0, 0, 0, 0, 1), (1, 1, 0, 0, 1), (1, 0, 1, 0, 0), (1, 1, 1, 1, 0), (0, 0, 0, 0, 0), (0, 1, 1, 1, 0)\}$. Obtain a minimum subset of this test set that also detects all such bridging faults.

Fig. P8.12

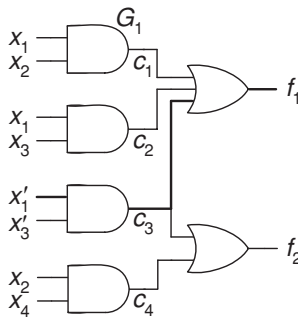


Problem 8.13. The EXCLUSIVE-OR gate implementation shown in Fig. 8.15 has six physical paths, hence 12 logical paths. Six of the logical paths are robustly testable. Identify which these are and derive two-pattern tests for them.

Problem 8.14. For the circuit shown in Fig. P8.14, derive the following tests.

- A robust two-pattern test for the path delay fault shown by the bold path for the falling transition at input x'_1 .
- A robust test for a slow-to-rise transition fault at the output of gate G_1 .

Fig. P8.14



Problem 8.15. Derive a two-pattern test for a slow-to-fall transition fault on line c_1 in the circuit in Fig. 8.22.

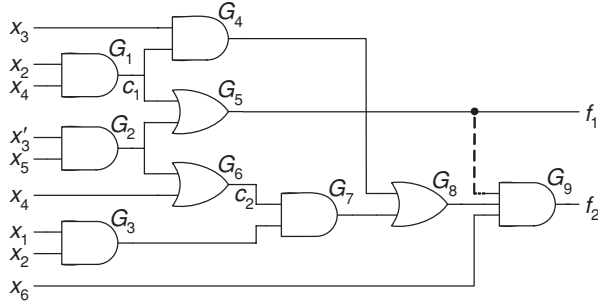
Problem 8.16. Show that a single-output two-level circuit based on an irredundant sum of products is fully testable for all single stuck-at faults.

Problem 8.17. Give an example of a multiple-output two-level circuit in which no output is based on an irredundant sum of products, yet the circuit is single stuck-at fault testable.

Problem 8.18. Obtain a multi-level circuit with as few literals as possible using single-cube, double-cube, and dual expression extraction, starting with a prime and irredundant two-level single-output circuit represented by the expression $f = x_1x_2x_4 + x_1x_2x_5 + x_3x_4 + x_3x_5 + x'_1x'_3x_6 + x'_2x'_3x_6$. Obtain a single stuck-at fault test set for the two-level circuit and show that it also detects all single stuck-at faults in the multi-level circuit.

Problem 8.19. Consider the irredundant circuit shown in Fig. P8.19, to which a broken-line connection is added as shown. Show that this connection is redundant. What are the other faults in the circuit that now become redundant because of the presence of the broken-line connection? Simplify the circuit by removing these redundancies and obtain another irredundant circuit which implements the same input-output behavior.

Fig. P8.19



Problem 8.20. Obtain a redundant circuit in which the region of a redundant fault at level i also contains the region of a redundant fault at a level greater than i .

Problem 8.21. Simplify an AND–OR circuit based on the redundant sum-of-products expression $f = x_1x_2 + x_1x_2x_3 + x_1x'_2$ using observability and satisfiability don't-cares.

Problem 8.22. Derive a robustly path-delay-fault-testable circuit using Shannon decomposition for the function $f = x_1x_2 + x'_1x'_2 + x'_3x_4 + x_3x'_4 + x_1x'_3$. Obtain a robust test set for this multi-level circuit.

Problem 8.23. Find the literals in the following expression, paths starting from which do not have robust tests for either rising or falling transitions: $f = x_1x_2x_3x_5 + x_1x'_3x_4x_5 + x_1x_3x'_4x_5x_6 + x_1x'_2x_4x'_5 + x'_1x_2x_4x'_5 + x'_1x_2x'_3x'_4 + x_2x'_3x_5x'_6 + x_2x_3x_4x_5 + x_2x'_3x'_4x'_5$. Use targeted algebraic factorization to obtain a three-level robustly testable circuit.

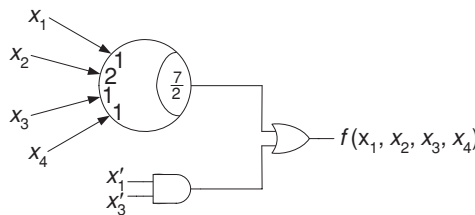
Problem 8.24. Given a threshold gate implementing the function $f(x_1, x_2, \dots, x_n)$, show that

- (a) an output f s - a -0 (s - a -1) dominates an x_i s - a -0 (s - a -1) if Eq. (8.7) is satisfied;
- (b) an output f s - a -1 (s - a -0) dominates an x_i s - a -0 (s - a -1) if Eq. (8.8) is satisfied.

Problem 8.25. Using the proof method from Problem 8.24, prove that any test set that detects all single stuck-at faults on all the primary inputs and fanout branches of an irredundant threshold network detects all single stuck-at faults in the network.

Problem 8.26. For the network shown in Fig. P8.26, obtain all test vectors that detect an s - a -0 fault at the x_2 input of the threshold gate.

Fig. P8.26



Problem 8.27. Given a threshold gate that implements the function $f(x_1, x_2, \dots, x_n)$, prove that if there exist two (or more) inputs x_j and x_k such that $w_j = w_k$ then test vectors to detect x_k s - a -0 and x_k s - a -1 can, respectively, be obtained simply by interchanging

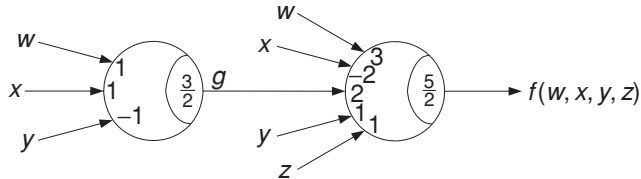
the bit positions of x_j and x_k in the s - a -0 and s - a -1 test vectors for x_j , assuming that they exist.

Verify this result by deriving test vectors for the s - a -0 and s - a -1 faults on inputs x_1 and x_2 of a three-input threshold gate that has a weight–threshold vector $\langle 1, 1, -1; 2 \rangle$.

Problem 8.28. For the network shown in Fig. P8.28:

- show a map for $f(w, x, y, z)$;
- realize f with a single threshold element;
- derive a test vector for an s - a -0 fault on the w input of the threshold gate on the left.

Fig. P8.28



Part 3 **Finite-state machines**

9

Introduction to synchronous sequential circuits and iterative networks

In Part 2 we considered combinational switching circuits in which the output values are functions of only the current circuit input values. In most digital systems, however, additional circuits are necessary that are capable of storing information and data and also of performing some logical or mathematical operations upon this data. The output values of these circuits at any given time are functions of external input values as well as of the stored information at that time. Such circuits are called *sequential circuits*.¹

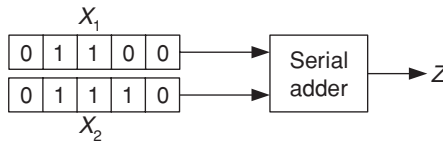
A *finite-state machine* (or *finite automaton*) is an abstract model describing the synchronous sequential machine and its spatial counterpart, the iterative network. It is the basis for the understanding and development of the various computation structures discussed in Part 3 of this book. The behavior, capabilities, limitations, and structure of finite-state machines are studied in Chapters 12 through 16, while Chapters 9 and 10 are devoted to the synthesis of these machines. Chapter 11 is concerned with asynchronous sequential circuits.

9.1 Sequential circuits – introductory example

In our daily activities, we all encounter the use of various sequential circuits. The elevator control which “remembers” to let us out before it picks up people coming into elevator; traffic-light systems on our roads, trains, and subways; the lock on a safe that not only remembers the combination numbers but also their sequence; all these are examples of sequential circuits in action. Before deriving the basic model and general synthesis procedures, we shall investigate the properties of a simple sequential circuit.

¹ Conventionally, the term *sequential machine* refers to the abstract model that represents the actual *sequential circuit*. In many cases, however, these terms are used interchangeably.

Fig. 9.1 Block diagram of a serial binary adder.



The state table

Consider the *serial binary adder* whose block diagram is shown in Fig. 9.1. It is a synchronous circuit with two inputs, X_1 and X_2 , carrying the two binary numbers to be added and one output, Z , which represents the sum. Fixed-length sequences of 0's and 1's are fed to the inputs and obtained at the outputs. The addition is to be performed serially: the least significant digits of numbers X_1 and X_2 arrive at the corresponding input terminals at time t_1 ; a unit time later, the next-to-least significant digits arrive at the input terminals; and so on. The time interval between the arrival of two consecutive input digits is determined by the frequency of the circuit's clock. We shall assume that the delay within the combinational circuit is small with respect to the clock period (which is the inverse of the clock frequency) and, as a consequence, the sum digit arrives at the Z terminal soon after the arrival of the corresponding input digits at the input terminals.

We shall denote by X and Z the input and output sequences, respectively, and by x and z the input and output symbols at a specified point in time. We may often want to emphasize the precise time at which the input or output value occurs. In such cases, the notation $x(t_i)$, $z(t_i)$ will be used.

Consider the following addition of two binary numbers:

$$\begin{array}{rcccccc}
 & t_5 & t_4 & t_3 & t_2 & t_1 & \\
 0 & 1 & 1 & 0 & 0 & = X_1 \\
 + & 0 & 1 & 1 & 1 & 0 = X_2 \\
 \hline
 1 & 1 & 0 & 1 & 0 & = Z
 \end{array}$$

An examination of the correlation between the input values and the required output value reveals the basic difference between a combinational circuit and the serial binary adder. While in a combinational circuit the output value at time t_i is defined uniquely by the input values at t_i , in the serial adder different output values are required for identical input conditions. For example, at t_1 and t_5 the input values are $x_1x_2 = 00$, but the required output values are $z = 0$ and $z = 1$, respectively. Similarly, at t_3 and t_4 the input values are $x_1x_2 = 11$ while the desired output values are 0 and 1, respectively. It is, therefore, evident that *the output value of the serial adder cannot be specified merely in terms of the external input values*, and so different design procedures must be employed.

Following the rules of elementary binary arithmetic, it is evident that the output value at time t_i is a function of the input values x_1 and x_2 at that time and of the carry that was generated at t_{i-1} . This carry (which may have either

Table 9.1 State table for a serial binary adder

PS	NS, z			
	$x_1x_2 = 00$	01	11	10
A	$A, 0$	$A, 1$	$B, 0$	$A, 1$
B	$A, 1$	$B, 0$	$B, 1$	$B, 0$

of the two values 0 or 1) in turn depends on the input values at t_{i-1} and on the carry generated at t_{i-2} , and so on. Hence the adder must be able to preserve information regarding its input values from the time it is set into operation up to time t_i . However, since the starting time may be long past, it is impossible to preserve the whole history of input values. We therefore seek a different relation between the input values $x_1(t_i)$ and $x_2(t_i)$ and the output value $z(t_i)$, as follows.

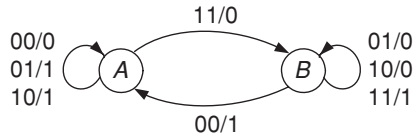
In the case of the serial adder, we can distinguish two classes of past input histories, one resulting in the production of a carry 0 and the other in producing a carry 1. These classes will be called the *internal states* (or simply *states*) of the adder. By “memorizing” the value of the carry, the adder actually shows some “trace” of its past input values, at least to the extent of their influence on the response to current input values.

Let A designate the state of the adder at t_i if a carry 0 is generated at t_{i-1} , and let B designate the state of the adder at t_i if a carry 1 is generated at t_{i-1} . We refer to the state of the adder at the time when the current input values are applied to it as its *present state* (PS) and the state to which the adder goes, as a result of the new (not necessarily different) carry value, as the *next state* (NS). The output value $z(t_i)$ is a function of the input values $x_1(t_i)$ and $x_2(t_i)$ and the state of the adder at time t_i . The next state of the adder depends only on the current input values and on the present state. A convenient way of describing the behavior of the serial adder is by means of a *state table*, as shown in Table 9.1.

Each row of the state table corresponds to a state of the adder, and each column to a particular combination of the external input values x_1 and x_2 . Each entry of the table denotes the state to which a transition is made and the output value associated with this transition. For example, if the adder is in state A , i.e., the current carry is 0, and it receives the input combination $x_1x_2 = 11$ then it will go to state B , which corresponds to carry 1, and produce an output value $z = 0$. The remaining entries of the table can be verified in a straightforward manner and, since the table contains eight entries, corresponding to the eight combinations of states and input values, it completely specifies the serial adder.

It is often convenient to use a directed graph as a counterpart to the state table. Such a graph, shown in Fig. 9.2, is known as the *state diagram* (or *state graph*). The vertices and directed arcs of the graph correspond to the states of the adder and to its state transitions, respectively. The labels of the directed

Fig. 9.2 State diagram for a serial adder.



arcs specify the input values and the corresponding output values; e.g., 10/0 represents the condition $x_1 = 1$, $x_2 = 0$, and $z = 0$. Clearly, both the state diagram and state table provide the same information regarding the operation of the adder, and one can be obtained directly from the other. While in many cases these representations are equally suitable, in some applications one may be more convenient than the other.

The state assignment

In order to implement the serial adder, it is necessary to use some device capable of storing the information regarding the presence or absence of a carry. Such a device must have two distinct states, such that each can be assigned to represent a state of the adder. A number of such devices exist, among which is the *delay element*, which may simply consist of a D flip-flop, to be described subsequently. The capability of the delay element to store information is a result of the fact that it takes a finite amount of time for input signal Y to reach its output y . The length of the delay is usually equal to the interval between two successive clock pulses. For convenience, we will assume that this delay is one time unit long.

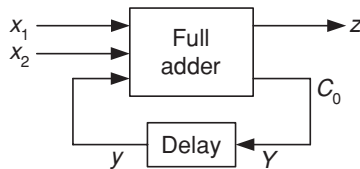
The state of the delay element is specified by the value of its output y , which may assume either of two values, namely, $y = 0$ or $y = 1$. Since the current input value Y of the delay is equal to its next output value, the input value is referred to as the *next state* of the delay, that is, $Y(t) = y(t + 1)$.

If we assign the states of the delay to those of the adder in such a way that $y = 0$ is assigned to A and $y = 1$ to B , the value of y at t_i will correspond to the value of the carry generated at t_{i-1} . The process of assigning the states of a physical device to the states of the serial adder is known as *state assignment* (or *secondary state assignment*). The output value y is referred to as the *state variable* (or *secondary variable*, to distinguish it from the external primary input variables).

The state assignment is completed by modifying the entries of the state table to correspond to the states of y , in accordance with the selected state assignment. The resulting table is given in Table 9.2, where the next-state and output entries have been separated into two sections. The entries of the next-state table define the necessary state transitions of the adder and thus specify the next value of the output, $y(t + 1)$, of the delay. In addition, since $Y(t) = y(t + 1)$, these entries also specify the input values to the delay at time t required to achieve the

Table 9.2 The transition and output tables for a serial binary adder

	Next state Y				Output z			
	x_1x_2				x_1x_2			
y	00	01	11	10	00	01	11	10
0	0	0	1	0	0	1	0	1
1	0	1	1	1	1	0	1	0

Fig. 9.3 Serial binary adder.

desired state transitions. Thus, the next-state part of Table 9.2, which is called the *transition table*, serves also to specify the required *excitation* of the delay.

The output part of Table 9.2, which is identical to that of Table 9.1, specifies the output value z for every combination of x_1 , x_2 , and y . Consequently, using the map method the following logic equations result:

$$Y = x_1x_2 + x_1y + x_2y,$$

$$z = x_1'x_2'y + x_1'x_2y' + x_1x_2'y' + x_1x_2y.$$

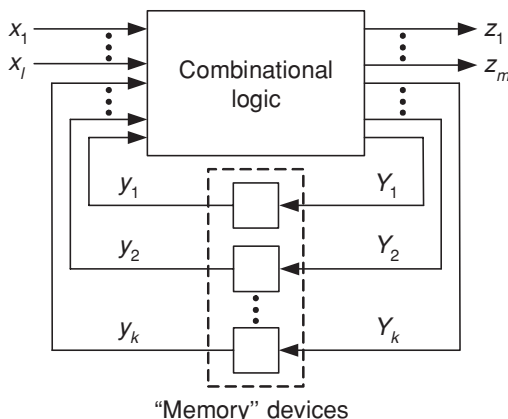
These equations are clearly identical to those obtained in Section 5.4 for the carry and sum functions of the full adder. The addition is accomplished by retransmitting the carry C_0 of the full adder through the delay Y into the full adder's input, as shown in Fig. 9.3. (Note that a delay whose input is Y is generally referred to as “delay Y .”)

9.2 The finite-state model – basic definitions

The behavior of a finite-state machine is described as a sequence of events that occur at discrete instants designated $t = 1, 2, 3$, etc. Suppose that a machine M has been receiving input signals and has been responding by producing output signals. If now, at time t , we were to apply an input signal $x(t)$ to M then its response $z(t)$ would depend on $x(t)$ as well as on the past input signals to M . Also, since a given machine M might have an infinite variety of possible histories, it would need an infinite capacity for storing them.

Since it is impossible to implement machines that have infinite storage capabilities, *we shall concentrate on those machines whose past histories can affect their future behavior in only a finite number of ways*. For example, suppose

Fig. 9.4 Circuit representation of a synchronous sequential machine.



that the serial binary adder of the previous section has been receiving input signals; its response to the signals at t is a function only of these signals and the value of the carry generated at $t - 1$. Thus, although the adder may have a large number of possible input histories, they may be grouped into two classes, those resulting in a carry 1 and those resulting in a carry 0 at t .

We shall study machines that can distinguish among a *finite* number of classes of input histories and shall refer to these classes as the *internal states* of the machine. Every finite-state machine, therefore, contains a finite number of memory devices, which store the information regarding the past input history. Note that, although we are restricting our attention to machines that have finite storage capacity, no bound has been set on the duration for which a particular input value may affect the future behavior of the machine. A discussion of this subject is deferred to Chapter 14.

Synchronous sequential machines

In general, a synchronous sequential machine is represented schematically by the circuit of Fig. 9.4. The circuit has a finite number l of input terminals. The signals entering the circuit via these terminals constitute a set $\{x_1, x_2, \dots, x_l\}$ of *input variables*, where each x_j , for all j , may take on one of the two possible values 0 or 1. An ordered l -tuple of 0's and 1's is an *input configuration* (alternatively, *input symbol*, *pattern*, or *vector*). The set I of $p = 2^l$ distinct input patterns is called the *input alphabet*, and each configuration is referred to as a symbol of the alphabet. Thus, the input alphabet is given by

$$I = \{I_1, I_2, \dots, I_p\}.$$

For example, if a machine has two input variables x_1 and x_2 then its input alphabet I consists of four symbols (or configurations), that is, $I = \{00, 01, 11, 10\}$.

Similarly, the circuit has a finite number m of output terminals which define the set $\{z_1, z_2, \dots, z_m\}$ of *output variables*, where each z_j , for all j , is a

binary variable. An ordered m -tuple of 0's and 1's is an *output configuration* (alternatively, *output symbol*, *pattern*, or *vector*). The set O of $q = 2^m$ ordered m -tuples is called the *output alphabet* and is given by

$$O = \{O_1, O_2, \dots, O_q\}$$

where each output configuration is a *symbol* of the output alphabet.

The signal value at the output of each memory element is referred to as the *state variable*, and $\{y_1, y_2, \dots, y_k\}$ constitutes the set of state variables. The combination of values at the outputs of the k memory elements $y_1, y_2, \dots, y_k$ defines the *present internal state* (or *state*) of the machine. The set S of $n = 2^k$ k -tuples constitutes the entire set of states of the machine, where

$$S = \{S_1, S_2, \dots, S_n\}$$

The external input values $x_1, x_2, \dots, x_l$ and the values of the state variables $y_1, y_2, \dots, y_k$ are supplied to the combinational circuit, which in turn produces the output values $z_1, z_2, \dots, z_m$ and the $Y_1, Y_2, \dots, Y_k$ values. The values of the Y 's, which appear at the outputs of the combinational circuit at time t , are identical to the values of the state variables at $t + 1$ and, therefore, they define the *next state* of the machine, i.e., the state that the machine will assume next.

Synchronization is achieved by means of clock pulses feeding the memory devices.

Specification of machine behavior

The relationships between the input symbol, present state, output symbol, and next state are described by either a *state table* or *state diagram*. A state table has p columns, one for each input symbol, and n rows, one for each state. For each combination of input symbol and present state, the corresponding entry specifies the output symbol that will be generated and the next state to which the machine will go. Although in practice every machine of the type shown in Fig. 9.4 has 2^l input symbols and 2^k states, some of them may be theoretically unnecessary. In other words, theoretically a machine may have any number p of input symbols and n of states. However, in practice, when realizing such a machine the actual circuit will have $l = \lceil \log_2 p \rceil$ input terminals and $k = \lceil \log_2 n \rceil$ memory elements, where $\lceil g \rceil$ is the smallest integer larger than or equal to g .

To each state of the machine there corresponds a vertex in the state diagram (cf. Fig. 9.2). From each vertex emanate p directed arcs, corresponding to the *state transitions* caused by the various input symbols. Each directed arc is labeled by the input symbol that causes the transition and by the output symbol that is to be generated. Since both the state table and state diagram contain the same information, the choice between the two representations is a matter of convenience, as mentioned above. Both have the advantage of being

precise, unambiguous, and thus more suitable for describing the operation of a sequential machine than any verbal description.

The succession of states through which a sequential machine passes, and the output sequence which it produces in response to a known input sequence, are specified uniquely by the state diagram (or table) and the initial state, where by the *initial state* we refer to the state of the machine prior to the application of the input sequence. The state of the machine after the application of the input sequence is called the *final state*.

9.3 Memory elements and their excitation functions

In discussing the basic model for synchronous sequential machines, we showed that a state table (or diagram) completely specifies the behavior of the machine. In order to design a circuit that operates according to the specifications of a given table, it is necessary first to select a number of memory elements, each of which is a device with two distinct states and is capable of storing a binary digit. The states of these elements are next assigned to the states of the machine, a process known as *state assignment*.

A *transition table* is derived from a state table by the replacement of each next-state entry with the corresponding state of memory elements. A transition table thus specifies for every combination of input values and state variables the next state of the memory elements, which is given by $Y_1, Y_2, \dots, Y_k$. To generate these Y 's, the memory elements must be supplied with appropriate input values. The switching functions, which describe the effect of the circuit inputs $x_1, x_2, \dots, x_l$ and state variables $y_1, y_2, \dots, y_k$ on the memory-element inputs, are called *excitation functions*. These functions are derived from an *excitation table*, whose entries are the values of the memory-element inputs.

In Section 9.1, we described the delay element as a memory device. Its storage capability is due to the fact that it takes a finite time for the signal to propagate through it. In practice, the most widely used memory element is the *flip-flop*, which is made up of latches.

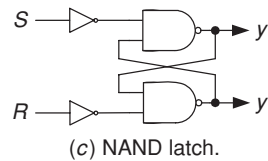
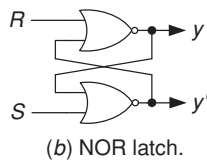
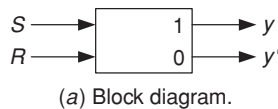
Set-reset or *SR* latch

The set-reset (*SR*) latch has two inputs, S and R , and two outputs, y and y' (often denoted as the 1 and 0 outputs or Q and Q' outputs, respectively). A block diagram representing an *SR* latch is shown in Fig. 9.5a. Such latches are easily implemented with cross-coupled NOR or NAND gates, as shown in Figs. 9.5b, 9.5c, respectively.

The *SR* latch has two states, defined by $y = 1$ and $y = 0$. The output y' is the complement of y . The latch possesses the property that it remains in one state indefinitely until it is directed by an input signal to do otherwise. A signal

Table 9.3 Excitation characteristics of the *SR* latch^a

$y(t)$	$S(t)$	$R(t)$	$y(t + 1)^b$
0	0	0	0
0	0	1	0
0	1	1	?
0	1	0	1
1	1	0	1
1	1	1	?
1	0	1	0
1	0	0	1

^a $RS = 0$.^b $y(t + 1) = R'y(t) + S$.**Fig. 9.5** The *SR* latch.

at the input S sets the latch to the 1 state, i.e., it sets $y = 1$; a signal at the input R resets it to the 0 state. The excitation characteristics of the *SR* latch are given in Table 9.3. If both R and S are excited simultaneously, the operation of the latch becomes unpredictable. Consequently, the requirement that the product $RS = 0$ must be imposed to ensure that the two invalid combinations in Table 9.3 will never occur. The excitation requirements of the *SR* latch are summarized in Table 9.4, in which a dash denotes a situation where the value of the input is a don't-care, since it does not affect the output value.

In practice, a clocked, or synchronous, version of the *SR* latch is generally used. In this version, shown in Fig. 9.6, state changes can occur only in synchronization with the pulses from an electronic clock. To ensure proper operation, restrictions must be placed on the length of the clock pulses and on the frequency of the input changes so that the circuit will change state no more than once for each clock pulse. The synchronization of the S and R inputs with the clock is accomplished in Fig. 9.6*b* by AND-gating them before they enter the latch inputs.

Table 9.4 Excitation requirements for the *SR* latch

Change in <i>y</i> from	to	Required value	
		<i>S</i>	<i>R</i>
0	0	0	—
0	1	1	0
1	0	0	1
1	1	—	0

Fig. 9.6 Clocked *SR* latch.

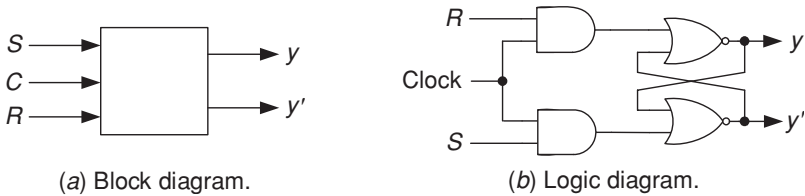
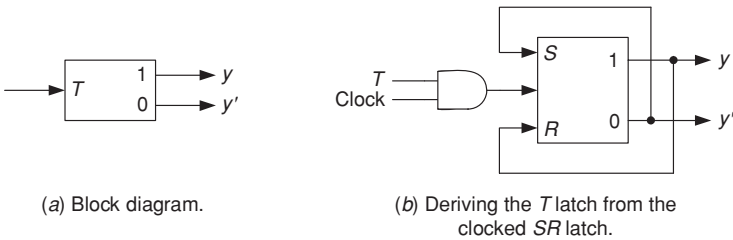


Fig. 9.7 Trigger (or *T*) latch.



To simplify the logic diagrams in subsequent sections we will often ignore the clock, but it is important to note that *in all synchronous circuits, the clock is implicit* whether shown or not.

Trigger or *T* latch

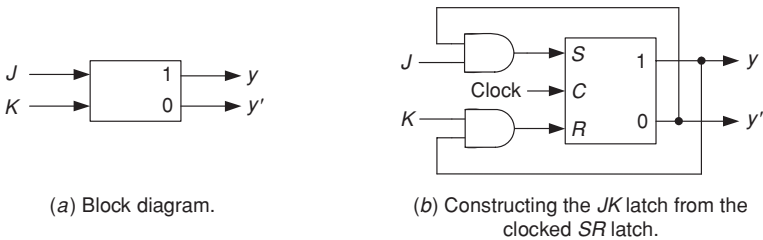
The block diagram of the trigger (*T*) latch is shown in Fig. 9.7*a*. The *T* latch has one input denoted *T* and two outputs denoted *y* and *y'*. It has two distinct states, defined by the logic value of *y*; namely, the latch is in the 1 state when *y* = 1 and in the 0 state when *y* = 0. The output *y'* is the complement of *y*. As in the case of the *SR* latch, the *T* latch remains in one state indefinitely until it is directed by an input signal to do otherwise. A value 1 applied to its input triggers the latch and it changes state.

The terminal characteristics of the *T* latch are summarized in Table 9.5. The next-state function $y(t + 1)$ can be expressed in terms of the present state and

Table 9.5 Excitation requirements for the T latch

Change in y from	to	Required value T
0	0	0
0	1	1
1	0	1
1	1	0

Fig. 9.8 The JK latch.



input as follows:

$$\begin{aligned} y(t + 1) &= T y'(t) + T' y(t) \\ &= T \oplus y(t). \end{aligned}$$

A clocked T latch can be realized by cross-coupling a clocked SR latch, as shown in Fig. 9.7*b*. (The clock in Fig. 9.6*b* is replaced by an AND combination of the input T and a clock.) If nonclocked operation is desired, the clock and AND gate in Fig. 9.7*b* may be removed and T applied directly to the latch. In the clocked realization, if the value of the output y is 1 then the reset input value is 1. The latch will now change state (to $y = 0$) when $TC = 1$, that is, when the values of T and the clock are both 1. Similarly, when $y = 0$ the set input value is 1, and the latch will change state (to $y = 1$) when $TC = 1$.

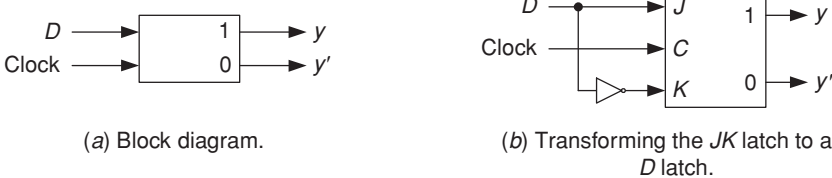
The JK latch

The JK latch has the characteristics of both the SR and T latches. Inputs J and K , like S and R , set and reset the latch, respectively. The combination $J = K = 1$ is permitted. When it occurs, the latch acts like a trigger and switches to its complement state; that is, if $y = 1$ it switches to $y = 0$ and vice versa. The block diagram and excitation requirements for the JK latch are shown in Fig. 9.8*a* and Table 9.6, respectively.

Table 9.6 Excitation requirements for the JK latch

Change in y		Required value	
from	to	J	K
0	0	0	—
0	1	1	—
1	0	—	1
1	1	—	0

Fig. 9.9 The D latch.



One possible realization of a clocked JK latch can be obtained by generalizing the clocked SR latch in the way shown in Fig. 9.8*b*.

The D latch

The block diagram and a possible realization of the D latch are shown in Fig. 9.9. The next state of this device is equal to its present excitation. Hence, it is characterized by the equation

$$y(t + 1) = D(t).$$

This latch clearly behaves like the delay element discussed in the preceding sections and, consequently, its excitation requirements are specified by the transition table.

Clock timing and the master–slave flip-flop

A clocked latch is characterized by the fact that it changes states only in synchronization with the clock pulse. Moreover, it changes state only once during each occurrence of a clock pulse. A sequential circuit operating under these restrictions is said to be a *synchronous sequential circuit*. The duration of the clock pulse is usually determined by the circuit delays and signal propagation time through the latches. In fact, *the clock pulse must be long enough to allow the latch to change state and, at the same time, it must be short enough that the latch will not change state twice due to the same excitation.*

Fig. 9.10 Excitation of a JK latch within a sequential circuit.

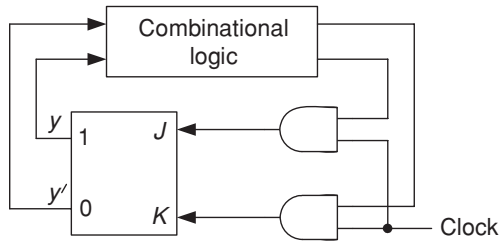
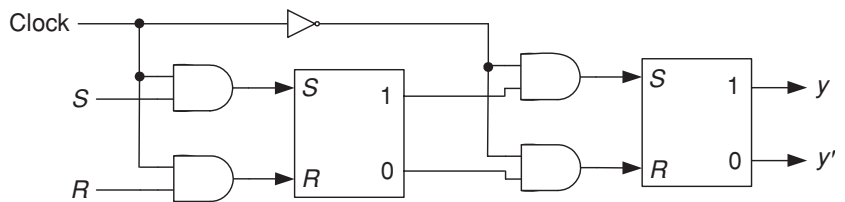


Fig. 9.11 Master-slave SR flip-flop.



In general, referring to the sequential circuit model of Fig. 9.4, the outputs of a latch (which serves as a memory element) are inserted into a combinational circuit, which, in turn, generates the excitation functions for that latch, as illustrated in Fig. 9.10. The length of the clock pulse must be such that it will allow the latch to generate the y 's but will not be present when the values of the y 's have propagated through the combinational circuit. This fine tuning of the length of the clock pulse is difficult to accomplish. To overcome this, another type of synchronous memory element, called a *master-slave flip-flop*, can be used. This flip-flop eliminates the timing problems associated with the feedback loop by essentially isolating the inputs of the flip-flop from its outputs.

A master-slave SR flip-flop, shown in Fig. 9.11, is constructed of two set-reset latches connected in series, with their clock inputs driven in a complementary manner. The first latch, called the *master*, can change state only when the clock is at 1, while the second latch, called the *slave*, can change state only when the clock is at 0. A change in excitation causes a change of state in the master latch. During that period, the slave latch maintains its previous state and serves as a buffer between the master and the next stage. When the clock changes from 1 to 0, the state of the master latch is frozen while the slave latch is enabled and changes its state to that of the master latch. The new state of the slave then determines the state of the entire master-slave flip-flop.

Since the master-slave SR flip-flop still suffers from the drawback that both its inputs cannot simultaneously be 1, it can be converted to a master-slave JK flip-flop to avoid this problem, as shown in Fig. 9.12a. Note the similarity to the JK latch shown in Fig. 9.8b. The only difference is that the SR latch has been replaced by a master-slave SR flip-flop. Thus, when a master-slave JK flip-flop is substituted for the JK latch in Fig. 9.10, the inputs of the combinational circuit do not change when the clock is at 1. When the clock is at 0, the y 's

Fig. 9.12 Master–slave flip-flops.

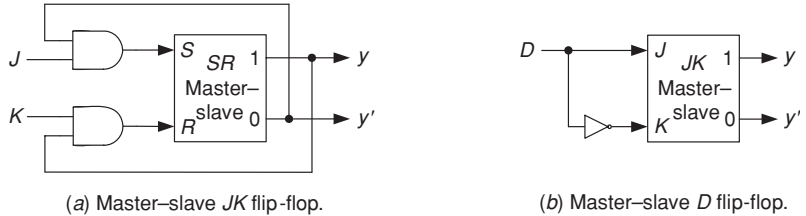
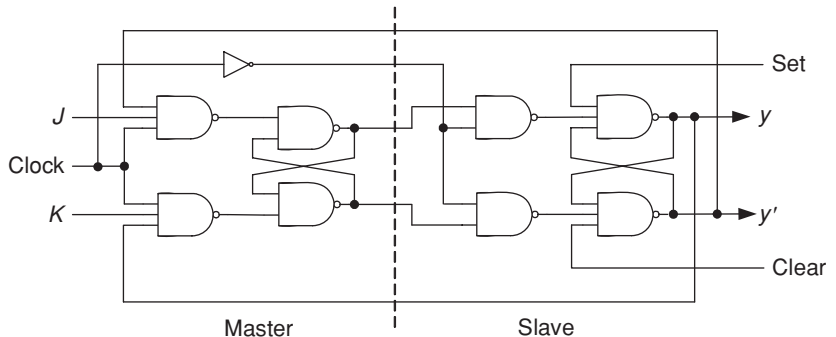


Fig. 9.13 Master–slave JK flip-flop with set and clear inputs.



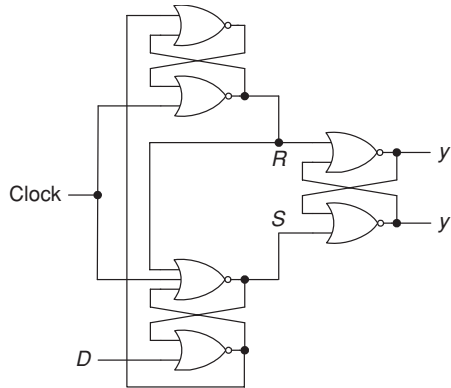
change and, consequently, the output of the combinational circuit changes, but this cannot affect the state of the master latch.

In practice, a master–slave flip-flop has three regular inputs, namely J , K (or S , R) and the clock, and two additional inputs, called (*direct*) *set* and (*direct*) *clear*, as shown in Fig. 9.13. These latter inputs are added to the slave flip-flop and they override the regular input signals and clock. They are used either to set the slave output to 1, by applying 0 to the set input and 1 to the clear input, or to clear the slave output to 0 by applying complementary values to the set and clear inputs. It is not allowable to assign 0's to both the set and clear inputs simultaneously. If we assign both of them 1's, however, the circuit returns to the normal clocked master–slave operation. Such external inputs are very useful, for example, in the design of counters, where it may be necessary to reset a counter to a prespecified count, or in the design of shift registers,² which must be cleared before the start of certain computations.

Both master–slave SR and JK flip-flops suffer from the problem of “1's catching” and “0's catching.” This arises from the fact that the master latch is transparent when the clock is high. Consider the JK flip-flop shown in Fig. 9.12a. When the output of the slave latch is at 0 and the J input has a static-0 hazard (a transient glitch to 1) after the clock has gone high, then the master latch catches this set condition and its output attains the value 1. It then

² A shift register consists of a number of cascaded flip-flops.

Fig. 9.14 A negative edge-triggered D flip-flop.



passes this 1 to the slave latch when the clock goes low. This leads to “1’s catching.” Similarly, when the output of the slave latch is at 1 and the K input has a static-0 hazard after the clock has gone high, then the master latch catches this reset condition and its output attains the value 0, which is then passed on to the slave latch when the clock goes low. This leads to “0’s catching.”

To avoid the above problems, a popular solution is to use a master–slave D flip-flop, as shown in Fig. 9.12b (note again the similarity to the D latch shown in Fig. 9.9b). Now, even if a static hazard were to occur at the D input when the clock is high, the output of the master latch would revert to its old value when the glitch goes away.

The master–slave T flip-flop can be obtained analogously by replacing the SR latch in Fig. 9.7b with master and slave SR latches.

Another type of flip-flop called an *edge-triggered flip-flop* yields a more efficient implementation, in terms of the number of gates, than master–slave flip-flops and hence is popular. This is discussed next.

Edge-triggered flip-flop

A positive (negative) edge-triggered D flip-flop stores the value available on the D input when the clock makes a $0 \rightarrow 1$ ($1 \rightarrow 0$) transition. Any change at the D input after the clock has made a transition does not have any effect on the value stored in the flip-flop.

Consider the negative edge-triggered D flip-flop shown in Fig. 9.14 (a positive edge-triggered flip-flop can be obtained simply by using the complement of the clock). It consists of three latches. When the clock is high, the output of the bottommost (topmost) NOR gate is at D' (D), whereas the S and R inputs of the output latch are both at 0, causing it to hold the previous value. When the clock goes low, the value from the bottommost (topmost) NOR gate gets transferred as D (D') to the S (R) input of the output latch. Thus, the output latch stores the value of D . If there is a change in the value of the D input of the

flip-flop after the clock has made its transition, the output of the bottommost NOR gate attains the value 0 (since its two inputs must have complementary values). However, it can be seen that this cannot change the SR inputs of the output latch.

The excitation characteristics and requirements presented earlier for the various types of latch are also applicable to the corresponding flip-flops. In the subsequent discussion, we shall synthesize sequential circuits using flip-flops. To simplify the resulting tables and circuits, the clock is generally not shown. However, as mentioned before, it is implicit in all synchronous circuits.

9.4 Synthesis of synchronous sequential circuits

We have seen a synthesis procedure in Section 9.1 for a serial binary adder using a delay as the memory element. In this section, we shall develop a general method for designing sequential circuits, using various types of memory elements, and apply this method to the design of some commonly used circuits. The main steps in the method are summarized as follows.

1. From a word description of the problem, form a state table (or a state diagram) that specifies the circuit behavior.
2. Check this table to determine whether it contains any redundant states. (The notion of a redundant state will be defined in Chapter 10, where we shall also present methods for detecting and eliminating such states. The state tables in this section do not contain any redundant states.)
3. Select a state assignment and determine the type of memory elements to be used.
4. Derive the transition and output tables.
5. Derive an excitation table and obtain the excitation and output functions from their respective tables.
6. Draw a circuit diagram.

In effect, in step 5 we are converting a less familiar problem, that of sequential circuit synthesis, into a more familiar problem, that of combinational circuit synthesis, since the construction of the excitation table is actually equivalent to the construction of a set of maps, from which the derivation of the excitation functions is straightforward.

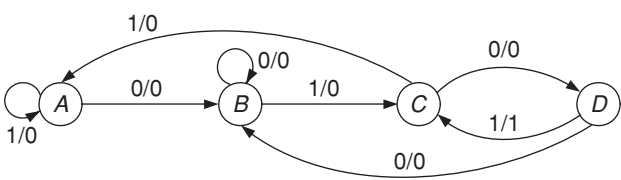
The sequence detector

We wish to design a one-input one-output sequence detector that produces an output value 1 every time the sequence 0101 is detected and an output value 0 at all other times. For example, if the input sequence is 010101 then the corresponding output sequence is 000101. In designing the sequence detector,

Table 9.7 State table for a sequence detector

<i>PS</i>	<i>NS, z</i>	
	<i>x</i> = 0	<i>x</i> = 1
<i>A</i>	<i>B</i> , 0	<i>A</i> , 0
<i>B</i>	<i>B</i> , 0	<i>C</i> , 0
<i>C</i>	<i>D</i> , 0	<i>A</i> , 0
<i>D</i>	<i>B</i> , 0	<i>C</i> , 1

Fig. 9.15 State diagram for a sequence (0101) detector.



we may find it more convenient to start the synthesis procedure by constructing the state diagram of the machine.

At time t_1 the machine is assumed to be in the initial state, designated (arbitrarily) as *A*. While in this state, the machine can receive input values 0 or 1. For each of these input values, an arc is drawn originating in state *A* and terminating in the appropriate next state, as shown in Fig. 9.15. The arc labeled 1/0 forms a self-loop around state *A*, since the machine does not initiate the sequence detection process until it receives a 0 input value. The input value 0 indicates a possible start of the sequence to be detected and, therefore, an arc labeled 0/0 leads from state *A* to *B*. When the machine is in state *B*, a 1 input value takes it to state *C*, while a 0 input value leaves it in the same state. If, when the machine is in state *C*, it receives a 1 input value, its last two input values are 11 and, since this input sequence cannot be completed in any way to yield 0101, the machine is directed back to its initial state. The machine arrives at state *D* after having received an input sequence whose last three symbols are 010. An additional 1 input value produces a 1 output value and causes a transition from state *D* to *C*, which is the state corresponding to input sequences whose last two symbols are 01. A 0 input value, applied to the machine when in state *D*, causes a transition to *B* because the last 0 symbol may be the prefix of 0101.

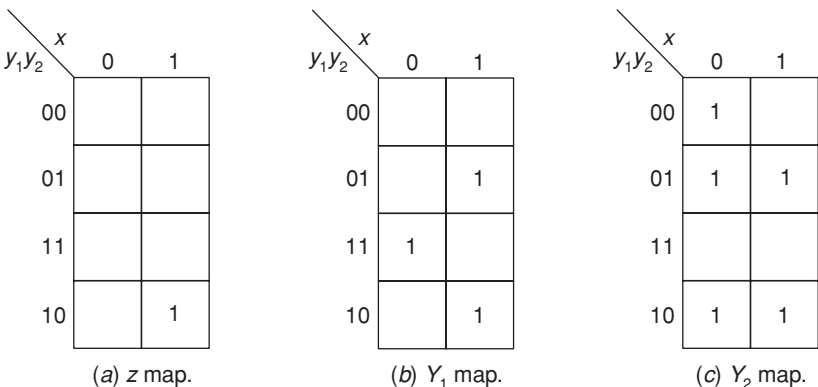
The state table corresponding to the diagram of Fig. 9.15 is given in Table 9.7. The input and output symbols are denoted by x and z , respectively.

Two state variables with $2^2 = 4$ states are needed for the representation of the four states of the sequence detector. If we select two delay elements, Y_1 and Y_2 , as memory devices and choose the state assignment shown in the left-hand block of Table 9.8, we obtain the transition and output tables in the center and

Table 9.8 Transition and output tables

		$Y_1 Y_2$		z	
		$x = 0$	$x = 1$	$x = 0$	$x = 1$
<i>A</i>	00	01	00	0	0
<i>B</i>	01	01	11	0	0
<i>C</i>	11	10	00	0	0
<i>D</i>	10	01	11	0	1

Fig. 9.16 Output and excitation maps.



right-hand blocks of Table 9.8. The entries of the transition table specify, for each combination of present state and input symbol, the values that the outputs of the delays should assume next. However, since the next values of the delays are equal to their present excitation, the transition table entries in effect specify the required excitation of the delay elements. Consequently, whenever delay elements are used as memory devices the transition and excitation tables are identical.

The output table is, actually, a three-variable map in which the value of z is specified for every combination of x , y_1 , and y_2 , as shown in Fig. 9.16*a*. The excitation table consists of two distinct three-variable maps, corresponding to the excitation functions for Y_1 and Y_2 . Entries for the map of Y_1 (Y_2) are given by the left-hand (right-hand) entries of the second block of Table 9.8. The logic equations, derived from the maps of Fig. 9.16, for the output and excitation functions are

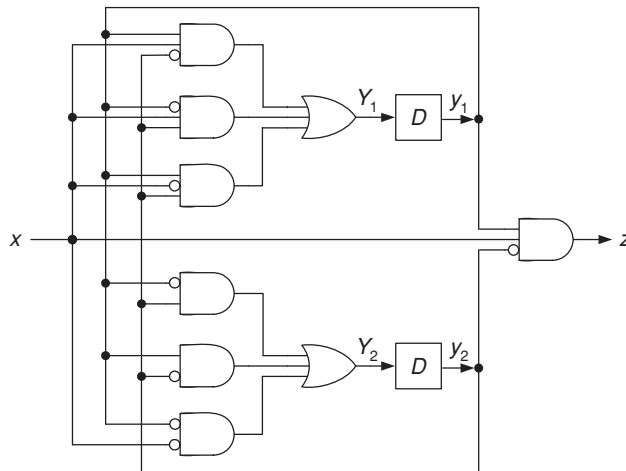
$$\begin{aligned}
 z &= xy_1y_2', \\
 Y_1 &= x'y_1y_2 + xy_1'y_2 + xy_1y_2', \\
 Y_2 &= y_1y_2' + x'y_1' + y_1'y_2.
 \end{aligned}$$

The implementation of these equations yields the sequence detector shown in Fig. 9.17.

The reader may have observed that the state assignment employed in Table 9.8 is not the only possible one. In general, different state assignments

Table 9.9 A second assignment

		$Y_1 Y_2$		z	
		$x = 0$	$x = 1$	$x = 0$	$x = 1$
<i>A</i>	00	01	00	0	0
<i>B</i>	01	01	10	0	0
<i>C</i>	10	11	00	0	0
<i>D</i>	11	01	10	0	1

Fig. 9.17 Logic diagram of a sequence detector.

yield different logic equations, which can affect to a considerable degree the area and structure of the resulting circuit. For example, if we interchange the codes assigned to states *C* and *D* then we obtain Table 9.9 and the following logic equations:

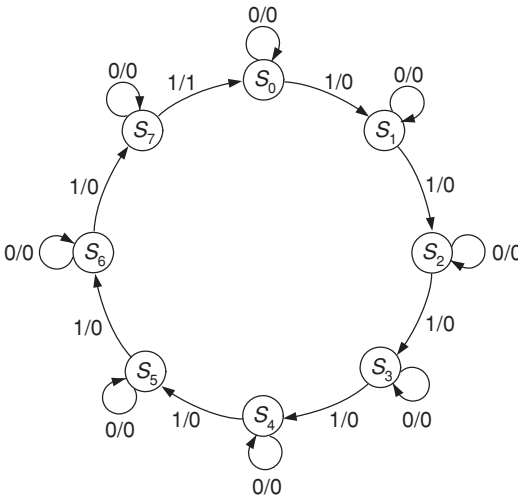
$$\begin{aligned} Y_1 &= x'y_1y_2' + xy_2, \\ Y_2 &= x', \\ z &= xy_1y_2. \end{aligned}$$

The implementation of the equations derived from this second state assignment requires less than half the number of gates required for the circuit of Fig. 9.17. Also, the second excitation function for Y_2 is independent of the state variables y_1 and y_2 ; it depends only on the input. Unfortunately, there is no simple procedure that can be used to arrive at an assignment yielding a minimal circuit under some well-defined cost criterion. Some trial and error is consequently necessary until an acceptable assignment is achieved. The state-assignment problem and, in particular, its effect on the machine structure will be discussed extensively in Chapter 12.

Table 9.10 State table for a modulo-8 binary counter

<i>PS</i>	<i>NS</i>		Output	
	<i>x</i> = 0	<i>x</i> = 1	<i>x</i> = 0	<i>x</i> = 1
<i>S</i> ₀	<i>S</i> ₀	<i>S</i> ₁	0	0
<i>S</i> ₁	<i>S</i> ₁	<i>S</i> ₂	0	0
<i>S</i> ₂	<i>S</i> ₂	<i>S</i> ₃	0	0
<i>S</i> ₃	<i>S</i> ₃	<i>S</i> ₄	0	0
<i>S</i> ₄	<i>S</i> ₄	<i>S</i> ₅	0	0
<i>S</i> ₅	<i>S</i> ₅	<i>S</i> ₆	0	0
<i>S</i> ₆	<i>S</i> ₆	<i>S</i> ₇	0	0
<i>S</i> ₇	<i>S</i> ₇	<i>S</i> ₀	0	1

Fig. 9.18 State diagram for a modulo-8 binary counter.



A binary counter

A modulo-8 binary counter is to be designed with one input terminal and one output terminal. It should be capable of counting in the binary number system up to 7 and producing an output value 1 for every eight input 1 values. After a count of seven is reached, the next input value 1 will reset the counter to its initial state, i.e., to a count of zero.

Let $S_0, S_1, \dots, S_7$ respectively be the states of the counter after it has received 0, 1, $\dots$, 7 input values equal to 1. The state S_0 that designates the zero count is the initial state. Transitions occur between successive states only when the counter receives the input value 1. The state diagram and state table of the counter are shown in Fig. 9.18 and Table 9.10.

Table 9.11 Transition and output tables for a modulo-8 binary counter

PS $y_3y_2y_1$	NS		z	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
000	000	001	0	0
001	001	010	0	0
010	010	011	0	0
011	011	100	0	0
100	100	101	0	0
101	101	110	0	0
110	110	111	0	0
111	111	000	0	1

From the correspondence between the states and the count, it is evident that no state in Table 9.10 is redundant. Also, since the counter has eight states, a state assignment requires three state variables (having $2^3 = 8$ states). The states of these variables, starting from the all-zero position, are 000, 001, $\dots$, 111. The choice of assignment in this example should not be made arbitrarily since it determines the characteristics of the circuits and, in particular, specifies the code and number system in which the counter actually counts. Our objective is to design a counter that counts in the binary number system. Accordingly, the code assigned to each state must be a binary representation of the actual count associated with that state, that is, $S_0 \rightarrow 000$, $S_1 \rightarrow 001$, $\dots$, $S_7 \rightarrow 111$. The transition and output tables corresponding to the foregoing assignment are shown in Table 9.11.

Implementing the counter with T flip-flops

To complete the synthesis, we need to choose an appropriate set of memory elements and derive their excitation functions. Let us select T flip-flops whose excitation requirements are specified by Table 9.5.

Up to now we have used a delay element whose output $y(t)$ equals its excitation at time $t - 1$ and, consequently, the transition table that specifies the required changes in the values of the y 's yields the necessary current excitations as well. Table 9.11, however, does not yield the necessary excitations for the T flip-flops. Consider, for example, entries 000 at the top of the $x = 0$ column and the bottom of the $x = 1$ column. In the first case the flip-flops remain unchanged, since the transitions are from $S_0 = 000$ to $S_0 = 000$. In the second case, however, the transitions are from $S_7 = 111$ to $S_0 = 000$ and, therefore, all three flip-flops must change state. Hence, while in the first case no excitations are needed, in the second case all three flip-flops must be triggered, i.e., $T_1 = T_2 = T_3 = 1$. Similarly, the transition from $S_5 = 101$ to $S_6 = 110$,

Table 9.12 Excitation table for T flip-flops

$y_3 y_2 y_1$	$T_3 T_2 T_1$	
	$x = 0$	$x = 1$
000	000	001
001	000	011
010	000	001
011	000	111
100	000	001
101	000	011
110	000	001
111	000	111

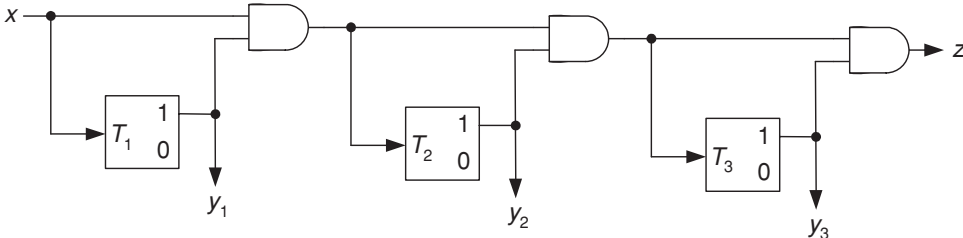


Fig. 9.19 Schematic diagram of a modulo-8 binary counter with T flip-flops.

under $x = 1$, requires y_3 to remain unchanged while y_1 and y_2 change state. Thus, from Table 9.5 it is evident that the required excitation is 011. In the same manner we can specify the required excitations for each transition, and the excitation table shown in Table 9.12 results.

This excitation table consists of three distinct maps specifying T_1 , T_2 , and T_3 as functions of x , y_1 , y_2 , and y_3 . The logic equations for the output and excitation functions are derived from Tables 9.11 and 9.12, respectively, and are as follows (note that the code resulting from the binary state assignment is not cyclic and thus the reader must be careful when “reading” the equations from the corresponding tables; alternatively, it is possible to transform the tables into three maps and to determine the equations directly from these maps):

$$\begin{aligned}
 T_1 &= x, \\
 T_2 &= x y_1, \\
 T_3 &= x y_1 y_2, \\
 z &= x y_1 y_2 y_3.
 \end{aligned}$$

A schematic diagram for a modulo-8 counter is shown in Fig. 9.19. The clock has not been shown but is implicit in this and subsequent figures. A 1 appears on terminal z whenever the total number of 1’s received at input line x is a multiple of 8. The actual count (modulo 8) of the number of incoming 1’s is given by the values of the state variables y_1 , y_2 , and y_3 , which have binary

Table 9.13 Excitation table for *SR* flip-flops

$y_3 y_2 y_1$	$x = 0$			$x = 1$		
	$S_3 R_3$	$S_2 R_2$	$S_1 R_1$	$S_3 R_3$	$S_2 R_2$	$S_1 R_1$
000	0—	0—	0—	0—	0—	10
001	0—	0—	—0	0—	10	01
010	0—	—0	0—	0—	—0	10
011	0—	—0	—0	10	01	01
100	—0	0—	0—	—0	0—	10
101	—0	0—	—0	—0	10	01
110	—0	—0	0—	—0	—0	10
111	—0	—0	—0	01	01	01

weights 1, 2, and 4, respectively. For example, if $y_1 = 1$, $y_2 = 0$, and $y_3 = 1$, the number of incoming 1's has been 5 modulo 8, i.e. 5, 13, 21, ...

Implementing the counter with *SR* flip-flops

The modulo-8 binary counter can also be implemented using *SR* flip-flops. The excitation table (Table 9.13) is derived from the transition table (Table 9.11) and from the excitation requirements in Table 9.4. As an example, consider the specification of the transition from $S_5 = 101$, under $x = 1$, to $S_6 = 110$. The value of y_1 will change from 1 to 0 and, consequently, the flip-flop must be reset. From Table 9.4, it is evident that this is accomplished by setting $S_1 = 0$ and $R_1 = 1$, and thus the value 01 is entered in row 101, column $S_1 R_1$, of Table 9.13. Similarly, y_2 must change from 0 to 1, and the value 10 is entered in column $S_2 R_2$, row 101. The value of y_3 , however, is to remain unchanged; hence R_3 must not be 1 while S_3 may be either 0 or 1, which means that the appropriate entry in row 101, column $S_3 R_3$, is —0. The entire excitation table is specified in a similar way.

Table 9.13 consists of six distinct maps for S_1 , R_1 , S_2 , R_2 , S_3 , and R_3 as functions of the variables x , y_1 , y_2 , and y_3 . The logic equations for the excitation functions are

$$\begin{aligned} S_1 &= xy_1', & S_2 &= xy_1y_2', & S_3 &= xy_1y_2y_3', \\ R_1 &= xy_1, & R_2 &= xy_1y_2, & R_3 &= xy_1y_2y_3. \end{aligned}$$

The schematic diagram³ corresponding to these equations is shown in Fig. 9.20.

³ It is interesting to observe that the binary counter is an iterative network, in the sense that, from the terminal viewpoint, each cell, containing a flip-flop and its associated logic, is indistinguishable from the others. Consequently, in order to design a modulo-16 counter, all that is necessary is to add a fourth identical cell in cascade with the three cells shown in Fig. 9.20.

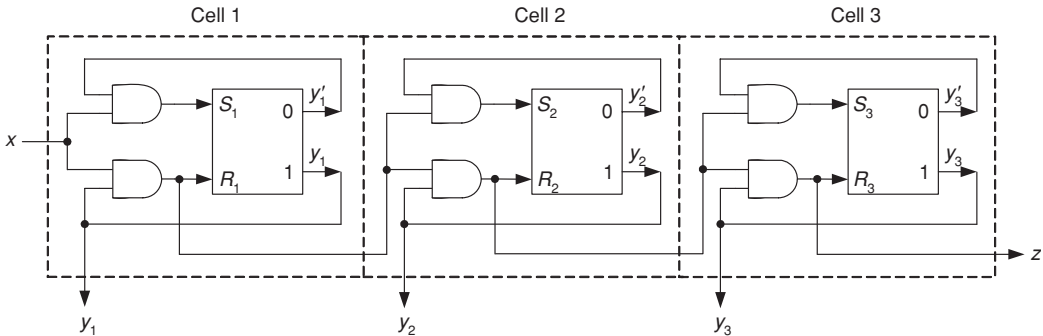


Fig. 9.20 Schematic diagram of a modulo-8 binary counter with SR flip-flops.

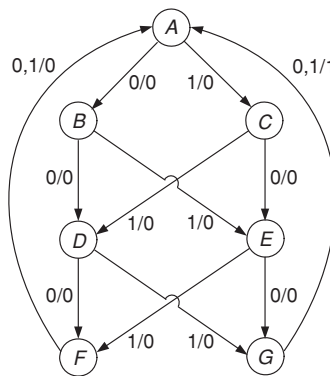


Fig. 9.21 State diagram for a parity-bit generator.

A parity-bit generator

A serial parity-bit generator is a one-terminal circuit that receives coded messages and adds a parity bit to every m -bit message, so that the resulting outcome is an error-detecting coded message. The input values in our example are assumed to arrive in strings of three symbols, i.e., $m = 3$, the strings being spaced apart by single time units. The parity bits are to be inserted in the appropriate spaces, and the resulting outcome is a continuous string of symbols without spaces. Even parity will be used; that is, a parity bit 1 is to be inserted if and only if the number of 1's in the preceding string of three symbols is odd.

The state diagram for the parity-bit generator is shown in Fig. 9.21. States B , D , and F correspond to even numbers of 1's out of one, two, and three incoming input symbols, respectively. Similarly, states C , E , and G correspond to odd numbers of 1's out of one, two, and three incoming input symbols, respectively. From either state F or state G the machine goes to state A , regardless of the input symbol. (Note that, in fact, the fourth input symbol is a blank, i.e., 0.)

Since the state diagram of Fig. 9.21 contains seven states, three state variables are needed for an assignment. However, since three state variables

Table 9.14 State table for a parity-bit generator

	PS $y_1y_2y_3$	NS		z	
		$x = 0$	$x = 1$	$x = 0$	$x = 1$
A	000	B	C	0	0
B	010	D	E	0	0
C	011	E	D	0	0
D	110	F	G	0	0
E	111	G	F	0	0
F	100	A	A	0	0
G	101	A	A	1	1

have a total of eight states, one of the states will not be assigned and so its entries in the corresponding state table may be considered as don't-cares. We shall defer the study of the properties of incompletely specified machines to Chapter 10, however. The state table and a possible state assignment are shown in Table 9.14. The reader can verify that the following logic equations result if JK flip-flops are used as memory elements:

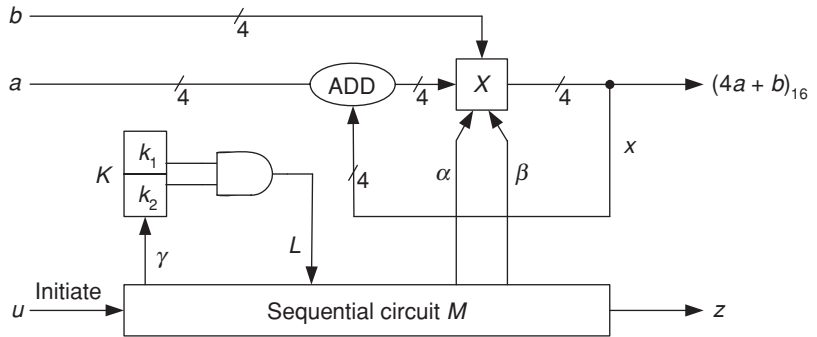
$$J_1 = y_2, \quad J_2 = y_1', \quad J_3 = xy_1' + xy_2, \quad z = y_2'y_3, \\ K_1 = y_2', \quad K_2 = y_1, \quad K_3 = y_2' + x.$$

Since the specification of the problem does not offer any clue as to which assignment to select, it may be chosen arbitrarily. The assignment shown in Table 9.14 has been selected so as to yield “reduced dependency” among the state variables; that is, J_1 and K_1 depend only on the second flip-flop while J_2 and K_2 depend only on the first flip-flop. The method of selecting assignments that result in such circuit properties will be presented in Chapter 12.

A sequential circuit as a control element in a computation

In the preceding examples, each sequential circuit received an input sequence and, in turn, produced an output sequence. This output sequence was the objective of the computation. However, many sequential circuits are used to control more complex computations. Indeed, the data for such computations do not even pass through the controlling circuit and are, therefore, not processed by it. The main role of a sequential circuit in the capacity of a control element is to streamline the computation by providing the appropriate control signals. Such circuits usually have a large number of inputs and outputs and, consequently, more informal design techniques simplify the design process considerably. The following example illustrates a simple computation in which a sequential circuit is the control element.

Fig. 9.22 A system to compute $(4a + b)$ modulo 16.



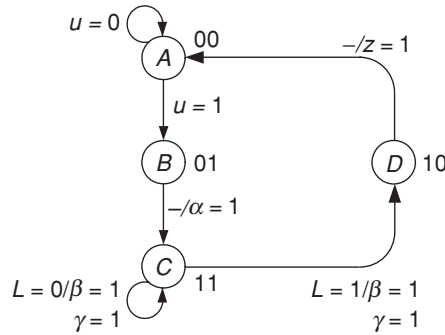
The schematic diagram in Fig. 9.22 describes a digital system that computes the value of $(4a + b)$ modulo 16, where a and b are each a four-bit binary number. In this figure, X is a register⁴ containing four flip-flops while x is the number stored in X . The register can be loaded with either b or $a + x$. The addition of a and x is performed by the four-bit parallel adder, denoted ADD. Input b to X is the channel through which the four-bit binary number b is loaded into the register in such a way that each bit enters the corresponding flip-flop. In general, if a number is loaded into the register then it replaces the number presently stored in it. The slash followed by the number 4 across several lines in Fig. 9.22 indicates that each such line actually consists of four wires. The output L of the modulo-4 binary counter K is equal to 1 whenever the count is 3 modulo 4.

The sequential circuit M has two inputs – an input u which initiates the computation and an input L that gives the count of K . It has four outputs, α , β , γ , z , whose tasks are as follows. The outputs α and β are control lines for loading the register X . Whenever $\alpha = 1$, the contents of b are transferred into X . Whenever $\beta = 1$, the values of x and a are added and transferred back into X . The input of the counter is γ . Hence, whenever $\gamma = 1$ the count of K increases by 1. Output z assumes the value 1 whenever the final result is available in X , that is, whenever $x = (4a + b)$ modulo 16. Output z can itself be a control input of another register that is to receive the final result of the computation. However, to simplify the design, this register is not shown.

Initially the count of K is zero, as are the values of u and z . When the value of u becomes 1 the computation starts by setting $\alpha = 1$, which causes b to be loaded into X . Next, a is added to x . This is accomplished by setting β to 1 and, simultaneously, γ to 1, so that the count in K will keep track of the number of times that a has been added to x . After four such additions, z assumes the value 1 and the computation is complete. At this point, the count in K is again zero and, hence, K is ready for the start of the next computation.

⁴ A k -bit register is a group of k flip-flops such that each flip-flop can store one binary digit and the entire register thus stores a k -bit binary word.

Fig. 9.23 State diagram for circuit M .



A compact state diagram for M is shown in Fig. 9.23. In this diagram, only some of the input and output symbols are shown, in particular, only those that change during the transition and are relevant for the transition in question. The clock is as usual omitted, although it is implicit. Initially, M is in state A . When $u = 1$, M goes to state B without changing the output values. The next clock pulse causes M to go to state C and to produce the output symbol $\alpha = 1$, regardless of the other input symbols. This is indicated by the symbol $-/\alpha = 1$ on the line going from B to C . Register X contains the value of b now. If u is at 1, its value may change to 0 without affecting the computation; u was only needed to cause the transition from A to B and thus initiate the computation. Since $L = 0$, the machine remains in state C and for each clock pulse it produces two output values, $\beta = 1$ and $\gamma = 1$. These output values add a to x while advancing the count in K by one unit. After three such advances, L 's value becomes 1 and M goes to state D . During this transition, a is added to x for the fourth time and K is set to zero. At this point, $x = (4a + b)$ modulo 16 and, consequently, z 's value becomes 1. The system is now back in state A , ready to start a new computation.

Let the state variables $y_1 y_2$ be assigned to the states of M as follows: $A \rightarrow 00$, $B \rightarrow 01$, $C \rightarrow 11$, $D \rightarrow 10$. This assignment is indicated in Fig. 9.23. The output functions can now be derived directly from the state diagram without any tables or maps. For example, α 's value must become 1 whenever the state variable values are $y_1 y_2 = 01$. Thus

$$\alpha = y_1' y_2.$$

Expressions for the other outputs are obtained in a similar manner:

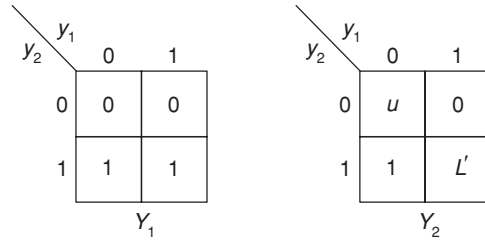
$$\begin{aligned}\beta &= \gamma = y_1 y_2, \\ z &= y_1 y_2' .\end{aligned}$$

The next-state variables can be obtained with the aid of the transition table shown in Fig. 9.24a and the corresponding maps shown in Fig. 9.24b, assuming a realization of M by two D flip-flops. In the transition table, some next-state entries are variables, and the treatment of such variables is analogous to the

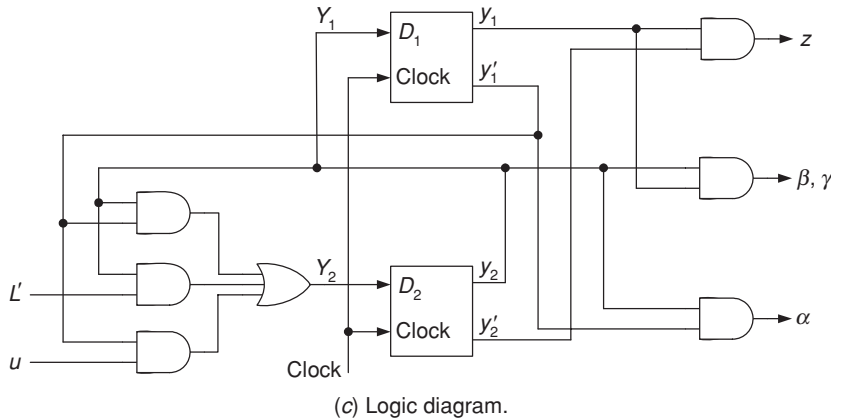
Fig. 9.24 Implementing the sequential circuit M with D flip-flops.

PS $y_1 y_2$	NS $Y_1 Y_2$
00	$0u$
01	11
11	$1L'$
10	00

(a) Transition table.



(b) Maps for Y_1 and Y_2 .



(c) Logic diagram.

treatment of the map-entered variables discussed in Section 4.6. When the present state of M is $y_1 y_2 = 00$, the next state depends on u ; that is, the next state is 00 if $u = 0$ and 01 if $u = 1$. Consequently, the next-state entry in row 00 is $0u$. However, if the present state is 01 then the next state is 11, regardless of the input values; hence, the next-state entry in row 01 is 11. In a similar manner we derive the entire transition table of Fig. 9.24a. The maps in Fig. 9.24b are obtained directly from the transition table. For example, the entry in row 11 of the transition table is $Y_1 Y_2 = 1L'$. Consequently, a 1 is entered in the Y_1 map in cell 11 while an L' is entered in the same cell in the Y_2 map. Following the procedure for covering maps with map-entered variables, we obtain the following next-state equations:

$$\begin{aligned} Y_1 &= y_2, \\ Y_2 &= y_1' y_2 + u y_1' + L' y_2. \end{aligned}$$

It is useful to note that the next-state equations can also be derived directly from the state diagram: Y_1 is 1 in states C and D , hence it must change to 1 whenever the circuit is in either state B or C . Thus, from the state assignments

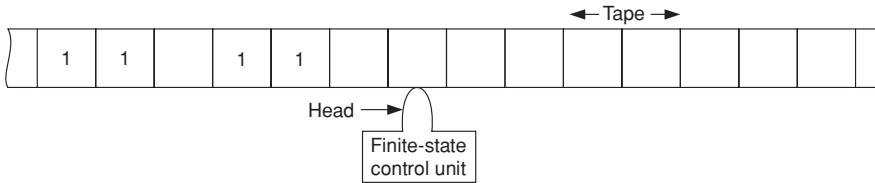


Fig. 9.25 An example of a writing machine.

of these states we obtain

$$Y_1 = y'_1 y_2 + y_1 y_2 = y_2.$$

This equation is clearly identical to the one obtained above. Similarly, we can obtain the foregoing equation for Y_2 just by inspecting the state diagram. A logic diagram for M is shown in Fig. 9.24c.

9.5 An example of a computing machine

We have been considering sequential machines as independent units possessing finite and limited memory capabilities, whose task is to produce prespecified output sequences in response to the application of external input sequences. Such finite-state machines are known as *nonwriting*, since they have no control on the external input and, in particular, cannot “write” or change their own input symbols. We shall subsequently consider a simple example of a *writing machine*, that is, a finite-state machine that is capable of modifying its own input symbols.

The machine

Consider a system consisting of a finite-state machine M that is coupled through a *head* to an arbitrarily long storage register, called the *tape* (Fig. 9.25). The tape is divided into squares, and each square stores a single symbol at any moment. (Blank squares will be said to store the symbol “blank,” denoted 0.) The head is capable of performing three operations, *reading* the symbol contained in the square being scanned, *writing* a new, not necessarily distinct, symbol in the scanned square, and *shifting* the tape one square in either direction. When a new symbol is written on the tape, it replaces the symbol previously there. The finite-state machine acts as the control unit, specifying the operations to be executed by the head. In what is termed a *cycle of computation*, the machine starts in some state S_i , reads the symbol currently being scanned by the head, writes a new symbol there, shifts right or left according to its state table, and then enters state S_j . For convenience, we shall assume that the tape is stationary and the head is moving. Such a machine is usually called a *Turing machine*, after A. M. Turing.

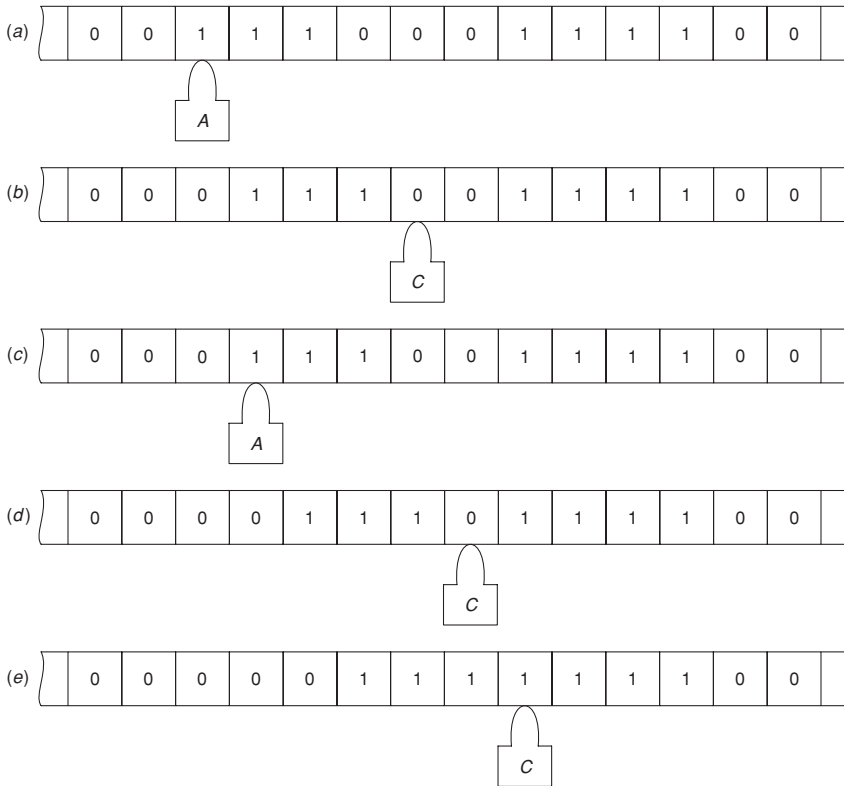


Fig. 9.26 Cycles of computation.

The machine receives its input symbols by reading the pattern of symbols written on the tape. Its output has the dual function of providing the head with the new symbols to be written on the tape and shifting the head in either direction. At the end of the computation, a new pattern of symbols is written on the tape. This pattern is the final objective of the entire computation.

The computation

As an example, let us design a finite-state machine that executes the following computation. The initial pattern of symbols on the tape consists of two finite blocks of 1's separated by a finite block of blanks. The machine is to shift the left-hand block of 1's to the right until it touches the right-hand block, and then halt. The machine is initially in state A, and its head is placed under the leftmost square containing a 1. Let the initial tape consist, for example, of the pattern $\dots 0011100011100\dots$, as shown in Fig. 9.26a, where the 0's designate blank squares. The desired final pattern is shown in Fig. 9.26e.

A simple way of performing the above computation is to erase, at each step, the leftmost 1 and write a new 1 in the first blank square to the right of the left-hand block of 1's, as shown in Fig. 9.26b. This computation is described in

Table 9.15 State table

<i>PS</i>	<i>NS</i> , write shift	
	0	1
<i>A</i>	—	<i>B</i> , 0 <i>R</i>
<i>B</i>	<i>C</i> , 1 <i>R</i>	<i>B</i> , 1 <i>R</i>
<i>C</i>	<i>D</i> , 0 <i>L</i>	Halt
<i>D</i>	<i>A</i> , 0 <i>R</i>	<i>D</i> , 1 <i>L</i>
Halt	Halt	Halt

Table 9.15, where the letters *R* and *L* designate right and left shifts, respectively, while 1 and 0 designate the symbols to be written on the tape in each cycle. Thus, for example, the entry *B*, 0*R* in row *A*, column 1, means that the machine is to write symbol 0 in the currently scanned square, shift its head one square to the right, and go to state *B*.

The computation starts when the machine erases the leftmost 1, currently under the head, shifts one square to the right, and enters state *B*. As long as it scans squares containing 1 symbols, it leaves them unchanged, shifts to the right, and stays in state *B*, in accordance with the specification *B*, 1*R* in row *B*, column 1, of the state table. After the third right shift, the head scans a square containing a 0 and, consequently, it must replace it by a 1, shift right, and go to state *C*. This situation is illustrated in Fig. 9.26*b*.

At this point, the machine is in state *C*, scanning a 0. The entry in row *C*, column 0, indicates that the machine is to leave that symbol unchanged, shift left, and enter state *D*. The machine now moves to the left, leaving all 1's unchanged and remaining in state *D* until it reaches the first 0 symbol, where it changes direction, shifts right, and enters state *A*. (See Fig. 9.26*c*.)

The machine is now in a similar situation to that illustrated in Fig. 9.26*a*. Hence, the foregoing sequence of operations will be repeated; that is, the 1 symbol under the head will be replaced by a 0, the machine will move right until it scans the first 0, which it replaces by a 1, shifts right once again, and enters state *C*. It is now in the position illustrated in Fig. 9.26*d*. The direction of shifts is now to the left until it scans the first 0 symbol, which once again causes a change in the shift direction and sends the machine to state *A*, with its head scanning the leftmost 1 symbol. After an additional cycle the machine will be in the position shown in Fig. 9.26*e*, in state *C* and scanning a 1. This terminates the computation, and the machine halts. Clearly, the computation described by Table 9.15 is independent of the precise size of the blocks of 1's and blocks of 0's separating the 1's as long as each block is finite.

The unspecified entry in row *A*, column 0, is a result of our initial assumption that at the start the head is placed on the leftmost square containing a 1 and, similarly, in all other cases when *M* enters *A* it is scanning a 1. This entry may be considered as a don't-care, or alternatively, one may specify that the

machine is to halt, or to cycle in a self-loop, etc. If the initial pattern on the tape contained two or more blocks of 1's, separated by blocks of 0's, the machine will execute the above computation on the two leftmost blocks and will always halt. If, however, it is presented with a tape containing just a single block of 1's then it will shift this block continuously to the right, looking for a second block of 1's, until the entire tape is exhausted. If we assume that the tape is infinite in length, the machine will never halt.

It can be shown that a Turing machine is more powerful than a finite-state machine, in the sense that it can execute computations that cannot be accomplished by any finite-state machine. In the next chapter, we shall show that the preceding computation, for arbitrarily large blocks of 1's, cannot be performed by any finite-state machine. This is clearly a result of the ability of the writing machines to change and write their own input symbols. From a theoretical viewpoint, each finite-state control unit is given access to an arbitrarily large external memory, in which it executes the computations, stores partial results, modifies and replaces input information, and finally stores the output pattern and halts. (We shall keep in mind, however, that there exist computations that never halt, as shown above, but will not refer to them further.)

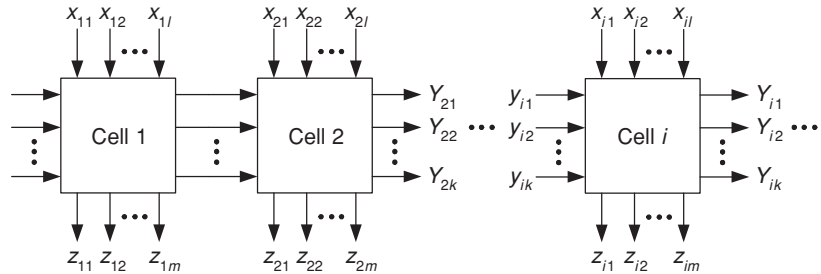
From the nature of the computations that can be performed by a Turing machine, we may suspect that it can serve as a theoretical model for digital computers. Clearly, no physical computing machine operates as inefficiently as the preceding model, nor does it have an arbitrarily large memory. The model, however, can serve as a tool for studying the capabilities and limitations of physical computing machines, the nature of computations, and the types of function that are not computable by any realizable machine. The study of these important problems is, however, beyond the scope of this book.

Our main objectives in this section have been the introduction of a finite-state machine as the control unit of a larger computing system and the development of a simple model for studying the computation power of digital computers. There is no point in implementing Table 9.15, although this could be accomplished in the usual manner.

9.6 Iterative networks

An *iterative network* is a digital structure composed of a cascade of identical circuits or *cells*. An iterative network may be sequential in nature, where each cell is a sequential circuit, e.g., the counter in Fig. 9.20 or a shift register, or it may be a combinational network where each cell is itself a combinational network. The description and synthesis of combinational iterative networks are similar to those of synchronous sequential circuits. Moreover, it will be shown that every finite output sequence that can be produced *sequentially* by a sequential machine can also be produced *spatially* (or simultaneously) by a combinational iterative network.

Fig. 9.27 General structure of an iterative network.



Because an iterative network consists of identical cells, we shall restrict our attention to the design of any arbitrary cell, which will be referred to as a *typical cell*.

The analogy between iterative networks and sequential machines

The general structure of an iterative network is shown in Fig. 9.27. The external *cell inputs* applied to the i th cell are designated $x_{i1}, x_{i2}, \dots, x_{il}$, where the i th (typical) cell is counted from the left. The *cell outputs* are designated $z_{i1}, z_{i2}, \dots, z_{im}$. In addition, each cell receives information from the preceding cell via the intercell carry wires $y_{i1}, y_{i2}, \dots, y_{ik}$, which are called *input carries*, and transmits information to the next cell via the intercell carry wires $Y_{i1}, Y_{i2}, \dots, Y_{ik}$, called *output carries*. Often, we are interested only in the output values from the rightmost cell. In this case the cell outputs are eliminated and the output is taken from the output carries of the last cell.

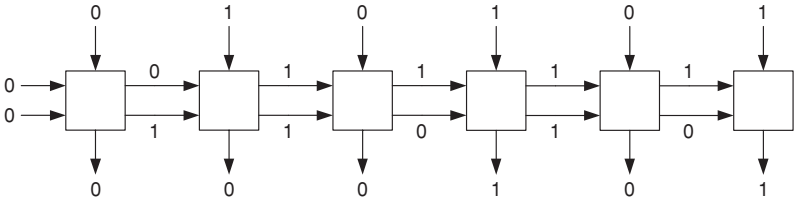
The operation of a cell can be described by means of a *cell table*, which specifies, for each combination of cell inputs and input carries, the values of the cell outputs and output carries. For example, let us construct the iterative network analogous to the sequence detector of Section 9.4. That is, we want to design an iterative network that consists of an arbitrarily large number of cells and whose typical cell contains a single cell input x_i and a single cell output z_i . The input symbols are applied to all cells *simultaneously* and the output symbols are assumed to be generated *instantaneously* in such a way that the output z_i is 1 if and only if the input pattern of the four cells $i-3, i-2, i-1$, and i is 0101, i.e., $x_{i-3} = x_{i-1} = 0$, and $x_{i-2} = x_i = 1$.

The technique of specifying the cell table for the i th cell is similar to that used in forming Table 9.7. The table must have four rows (or states), corresponding to the four possible distinct signals delivered by the intercell input carries. The resulting table, which is identical to Table 9.7, is repeated in Table 9.16. Row *D* designates the signals received by the i th cell when the input pattern in the three preceding cells is 010. Similarly, row *C* designates the signal when the input pattern in the two preceding cells is 01, and so on. From these incoming intercell signals and from cell input x_i , the i th cell can compute the necessary

Table 9.16 Cell table for an iterative pattern detector

PS	NS, z_i	
	$x_i = 0$	$x_i = 1$
A	$B, 0$	$A, 0$
B	$B, 0$	$C, 0$
C	$D, 0$	$A, 0$
D	$B, 0$	$C, 1$

Fig. 9.28 Pattern detection.



cell output value and the signals to be transmitted to the next cell via the output carry wires.

If we specify the intercell signals in such a way that A is represented by $y_{i1}y_{i2} = 00$, B by 01 , C by 11 , and D by 10 , the transition table shown in Table 9.8 results and, as a consequence, the logic equations derived in Section 9.4 are obtained. In general, *if the same assignment is selected for the iterative network as for the sequential circuit, the logic circuit of the i th cell and the combinational logic of the sequential circuit are identical*. While in the sequential case information is fed back through delays, in the iterative network, the entire computation is executed by using many identical cells. Clearly, the number of cells in an iterative network must equal the length of the input patterns applied to it. For example, if the input patterns are limited to length 6, and the specific input pattern applied to the above pattern detector has the form 010101, then the resulting output pattern will be 000101, as shown in Fig. 9.28. (The symbols along the intercell carry leads denote the transmitted signals.)

The reader is encouraged to apply the foregoing procedure and to design a parallel parity-bit generator as a counterpart to the sequential parity-bit generator specified by Table 9.14.

Synthesis

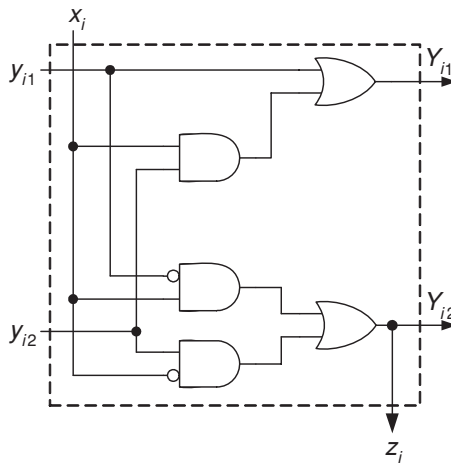
The synthesis procedure for iterative networks is best illustrated by an example. We wish to design an n -cell network where each cell has one cell input x_i and

Table 9.17 Cell table

PS	NS, z_i	
	$x_i = 0$	$x_i = 1$
A	$A, 0$	$B, 1$
B	$B, 1$	$C, 1$
C	$C, 1$	$D, 0$
D	$D, 0$	$D, 0$

Table 9.18 Output carries and cell output table

$y_{i1}y_{i2}$	$Y_{i1}Y_{i2}, z_i$	
	$x_i = 0$	$x_i = 1$
00	00, 0	01, 1
01	01, 1	11, 1
11	11, 1	10, 0
10	10, 0	10, 0

Fig. 9.29 Iterative network cell derived from Table 9.18.

one cell output z_i , such that $z_i = 1$ if and only if either one or two of the cell inputs $x_1, x_2, \dots, x_i$ have the value 1.

The cell table of the i th cell must have at least four rows to distinguish the following four distinct states. Row A designates the state where none of the cell inputs to preceding cells has the value 1. Similarly, rows B , C , and D designate, respectively, the states where one, two, three or more of the cell inputs to preceding cells have the value 1. The resulting cell table is given as Table 9.17. The state assignment and output tables are shown in Table 9.18, and the typical cell is shown in Fig. 9.29.

The logic equations corresponding to the output carries and the i th cell output are

$$\begin{aligned} Y_{i1} &= y_{i1} + x_i y_{i2}, \\ Y_{i2} &= x'_i y_{i2} + x_i y'_{i1}, \\ z_i &= Y_{i2}. \end{aligned}$$

As a consequence of their iterative structure, such networks are easier to design and construct. The time of operation may be substantially longer

than for other possible realizations, however. When realizing combinational circuits, for which the speed of operation is not crucial and which can be composed of identical cells, iterative networks prove to be very useful and economical.

Notes and references

The finite-state model described in this chapter was proposed by Mealy [7] in 1955, on the basis of earlier models by Huffman [3] and Moore [8]. The applicability of the model to iterative combinational circuits was pointed out by McCluskey [6]. Recently, there have been several texts devoted to finite-state machines, among which are Hill and Peterson [2], Katz [4], Mano and Ciletti [5], and Wakerly [10]. A collection of original basic papers dealing with various aspects of finite automata is available in a book edited by Moore [9]. A comprehensive presentation of iterative networks is available in Hennie [1].

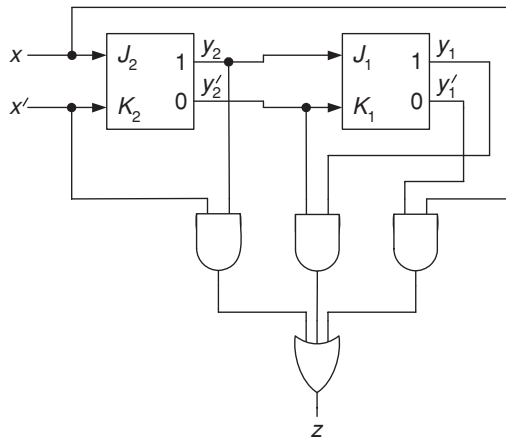
- [1] Hennie, F. C.: *Iterative Arrays of Logical Circuits*, MIT Press, Cambridge MA, 1961.
- [2] Hill, F. J., and G. R. Peterson: *Computer Aided Logical Design With Emphasis on VLSI*, fourth edition, John Wiley & Sons, New York, 1993.
- [3] Huffman, D. A.: "The synthesis of sequential switching circuits," *J. Franklin Inst.*, vol. 257, pp. 161–190, March 1954; pp. 275–303, April 1954. Reprinted in Moore [9].
- [4] Katz, R. H., and G. Borriello: *Contemporary Logic Design*, second edition, Pearson Prentice Hall, Upper Saddle River NJ, 2005.
- [5] Mano, M. M., and M. D. Ciletti: *Digital Design*, fourth edition, Prentice Hall, Upper Saddle River, NJ, 2007.
- [6] McCluskey, E. J.: "Iterative combinational switching networks: general design considerations," *IRE Trans. Electron. Computers*, vol. EC-7, pp. 285–291, December 1958.
- [7] Mealy, G. H.: "A method for synthesizing sequential circuits," *Bell System Tech. J.*, vol. 34, pp. 1045–1079, September 1955.
- [8] Moore, E. F.: Gedanken-experiments on sequential machines, pp. 129–153, *Automata Studies*, Princeton University Press, 1956.
- [9] Moore, E. F. (ed.): *Sequential Machines: Selected Papers*, Addison Wesley, Reading, Mass., 1964.
- [10] Wakerly, J. F.: *Digital Design Principles and Practices*, Prentice Hall, Englewood Cliffs NJ, 1990.

Problems

Problem 9.1. Analyze the synchronous circuit of Fig. P9.1 (the clock is not shown, but is implicit).

- Write down the excitation and output functions.
- Form the excitation and state tables.
- Give a word description of the circuit operation.

Fig. P9.1



Problem 9.2. A long input sequence enters a one-input one-output synchronous sequential circuit, that is required to produce an output symbol $z = 1$ whenever the sequence 1111 occurs. Overlapping sequences are accepted; for example, if the input sequence is 01011111..., the required output sequence is 00000011....

- Draw a state diagram.
- Select an assignment and show the excitation and output tables.
- Write down the excitation functions for SR flip-flops, and draw the corresponding logic diagram.

Problem 9.3. Repeat Problem 9.2 for the sequence 01101, and implement the circuit with T flip-flops as memory elements.

Problem 9.4. Construct the state diagram for a one-input eight-state machine that is to produce an output symbol $z = 1$ whenever the last string of five input symbols contains exactly three 1's and starts with two 1's. After each string that starts with two 1's, analysis of the next string does not start until the end of this string of five symbols, whether it produces an output value 1 or not. For example, if the input sequence is 11011010 then the output sequence is 00000000, while an input sequence 10011010 produces an output sequence 00000001.

Problem 9.5. For each of the following cases, show the state table that describes a one-input one-output machine having the following specifications.

- An output symbol $z = 1$ is to be produced to coincide with every occurrence of the input symbol 1 following a string of two or three consecutive 0's at the input. At all other times, the output symbol is to be 0.
- Regardless of the input symbols, the first two output symbols are 0's. Thereafter, output symbol z is a replica of input symbol x but delayed by two time units, that is, $z(t) = x(t - 2)$ for $t \geq 3$.
- The output $z(t)$ is 1 if and only if $x(t) = x(t - 2)$. At all other times, z is to be 0.

- (d) The output z has the value 1 whenever the last four input symbols correspond to a BCD number that is a multiple of 3, i.e., 0, 3, 6, ...

Problem 9.6. Design a one-input one-output synchronous sequential circuit that produces an output symbol $z = 1$ whenever any of the following input sequences occurs: 1100, 1010, or 1001. The circuit resets to its initial state after an output symbol 1 has been generated.

- (a) Form the state diagram or table. (Seven states are sufficient.)
 (b) Choose an assignment, and show the excitation functions for JK flip-flops.

Problem 9.7. Design a one-input one-output synchronous sequential circuit that examines the input sequence in nonoverlapping strings having three input symbols each and produces an output symbol 1 that is coincident with the last input symbol of the string if and only if the string consisted of either two or three 1's. For example, if the input sequence is 010101110, the required output sequence is 000001001. Use SR flip-flops in your realization.

Problem 9.8. Design a modulo-8 counter that counts in the way specified in Table P9.8. Use JK flip-flops in your realization.

Table P9.8

Decimal	Gray code		
0	0	0	0
1	0	0	1
2	0	1	1
3	0	1	0
4	1	1	0
5	1	1	1
6	1	0	1
7	1	0	0

Problem 9.9. Construct the state diagram for a synchronous sequential machine that can be used to detect faults in coded messages of the 2-out-of-5 type. That is, the machine examines the messages serially and produces an output symbol 1 whenever an illegal message of five binary digits is detected.

Problem 9.10. When a certain serial binary communication channel is operating correctly, all blocks of 0's are of even length and all blocks of 1's are of odd length. Show the state diagram or table of a machine that will produce an output symbol $z = 1$ whenever a discrepancy from the above pattern is detected. The following is an example.

$X: 0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 0 \ 1 \ 1 \ 0 \ 0 \ \dots$
 $Z: 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1 \ 0 \ 1 \ 0 \ \dots$

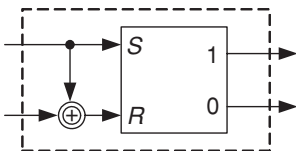
Problem 9.11. A new kind of flip-flop has been designed. It is equivalent to an SR flip-flop with gated inputs, as shown in Fig. P9.11.

A synchronous sequential circuit that generates an output symbol $z = 1$ whenever the string 0101 is scanned in the input sequence is to be designed. Overlapping strings

are accepted; for example, corresponding to the input sequence 0010101, the required output sequence is 0000101.

- (a) Construct the state diagram and table for the circuit, using the letters A, B, C , etc.
- (b) Make a state assignment (use a Gray code, starting with an all-0 assignment for the initial state).
- (c) Realize the sequential circuit using the new flip-flops as memory elements. Give the logic equations for the memory elements and the output.

Fig. P9.11



Problem 9.12. The clocked memory device shown in Fig. P9.12 has one binary input Y and one binary output y . If $Y(t) = 0$ then $y(t + 1) = 0$; if $Y(t) = 1$ then $y(t + 1) = y'(t)$.

- (a) The state table given in Table P9.12 is to be realized using two such memory devices. Choose an appropriate state assignment and give the corresponding excitation and output equations.
- (b) Briefly discuss the possibility and practicality of using such memory devices to realize an arbitrary state table.

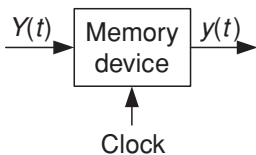


Fig. P9.12

Table P9.12

PS	NS, z	
	$x = 0$	$x = 1$
A	$B, 0$	$B, 0$
B	$C, 0$	$A, 1$
C	$B, 0$	$D, 0$
D	$C, 0$	$B, 1$

Problem 9.13. Write the state table for a synchronous circuit, with one input x and one output z , that operates according to the following specifications. At time $t = 0$, the initial state is A , and $x(t) = 0$ for $t < 0$. The output function is given by either (a) or (b) as follows:

- (a) $z(t) = x(t) + x(t - 1)$,
- (b) $z(t) = x(t) \cdot x(t - 1)$

where the change from (a) to (b) occurs at times τ such that

$$x(\tau) = x(\tau - 1) = x(\tau - 2) = 1$$

and the change from (b) to (a) occurs at times T such that

$$x(T) = x(T - 1) = x(T - 2) = 0.$$

An example is shown in Fig. P9.13.

Fig. P9.13

$t =$	0	1	2	3	4	5	6	7	8	9	10	11	12	...
$x(t) =$	0	1	1	1	0	0	1	1	0	0	0	1	0	...
$z(t) =$	0	1	1	1	0	0	0	1	0	0	0	1	1	...
	(a)				(b)						(a)			

Problem 9.14. The synchronous circuit shown in Fig. P9.14, where D denotes a unit delay, produces a periodic binary output sequence. Assume that initially $x_1 = 1$, $x_2 = 1$, $x_3 = 0$, $x_4 = 0$ and that the initial output sequence is 1100101000. Thereafter, this sequence repeats itself. Find a minimal expression for the combinational circuit $f(x_1, x_2, x_3, x_4)$.

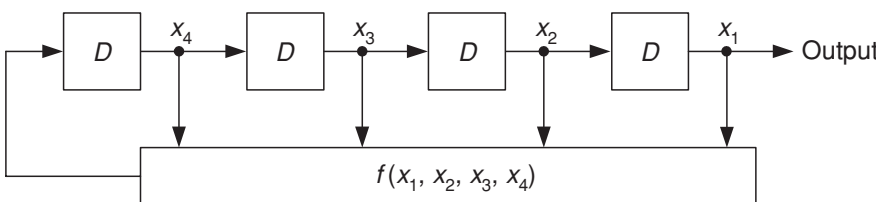


Fig. P9.14

Problem 9.15. A synchronous machine N is part of a transmitter and is used to encode binary serial messages. The coded messages are then transmitted to a receiver, as shown in Fig. P9.15. The receiver contains a synchronous machine M that is used to decode the received messages.

- (a) Given that the initial state of N is A , find the state diagram of machine M .
- (b) Suppose the initial state of N is unknown and machine M received a 10-bit message; which of the 10 bits can be uniquely decoded without an error? Explain.

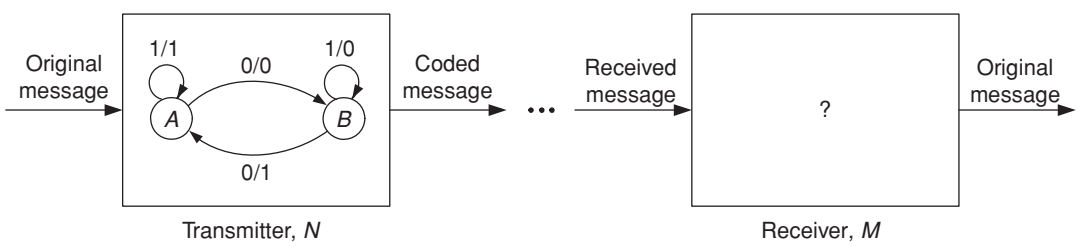


Fig. P9.15

Problem 9.16. A *palindrome* is a sequence which reads the same backward as forward, e.g., 11011 or 01010. Show the finite-state control of a Turing machine that is capable of detecting arbitrarily long palindromes. Assume that you are given a tape initially marked only with symbols #, 0, 1, where the blanks (#) separate blocks of intermixed 0's and 1's. The machine will be started on a # and then checks whether the sequence to its right is a palindrome. If not, the machine should proceed to the next block. If the sequence is a palindrome, the machine should stop at the # to the right of the block. An example is shown in Fig. P9.16.

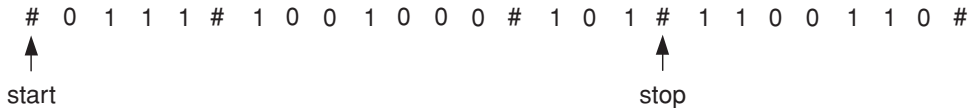


Fig. P9.16

Hint: It is often useful in the course of computation to mark certain digits. This can be accomplished by replacing those digits with different symbols; for example, 0's may be replaced by 2's, while 1's may be replaced by 3's, etc. When these markers are no longer necessary, they are replaced with the old symbols. Use as many new symbols as necessary.

Problem 9.17. Assume that you have a Turing machine that is started at the leftmost 1 in a block of n 1's on a tape that otherwise contains only #'s (blanks), as shown in Fig. P9.17. Using as many symbols as you like:

- Show a finite-state control that will duplicate the block of 1's immediately to the right of the original block, leaving the original block and the rest of the tape intact when the machine stops (viz., the block is simply doubled in size – it now contains $2n$ 1's). The machine should stop at the leftmost 1.
- Show a finite-state control that will produce a number of replicas equal to the original number of 1's (it stops with a block of n^2 1's).
- Show a finite-state control that will increase the number of 1's to 2^n and will then stop.

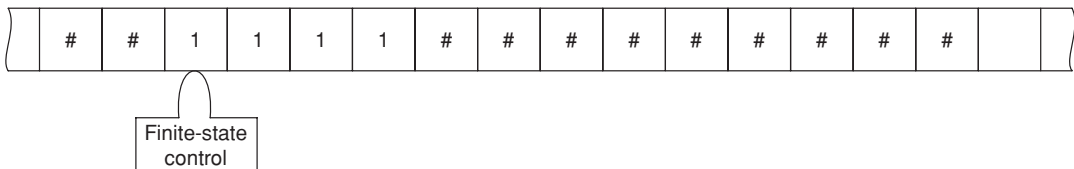


Fig. P9.17

Problem 9.18. An iterative network to be used for detecting faults in Ringtail-coded messages is to be designed. The network consists of five cells, each receiving a digit of the coded message, and is to produce an output symbol 1 when and only when an illegal message is detected. (The Ringtail code is defined in Problem 5.2.)

- Construct a cell table.
- Select an assignment and derive the logic equations for the output carries and the cell output.
- Construct a typical cell using AND, OR, and NOT gates.

Problem 9.19. The cell output of a typical cell of an iterative network has the value 1 if and only if the input pattern of the preceding cells consists of groups of 0's and 1's such that each group contains an odd number of members.

- Construct a cell table.
- Realize the typical cell using AND, OR, and NOT gates.

Problem 9.20. The typical cell of an iterative network has one binary input x_i and one binary output z_i . The output $z_i = 1$ if and only if $x_i \neq x_{i-2}$. For the first two cells (i.e., $i = 1, 2$), assume that $x_{-1} = x_0 = 0$.

- (a) Construct a cell table.
- (b) Make a Gray-code state assignment and give the output and carry functions.

10

Capabilities, minimization, and transformation of sequential machines

This chapter extends some of the concepts introduced in Chapter 9 and presents important techniques for the synthesis of sequential machines and for other problems considered in later chapters. The first two sections are concerned with the general finite-state model, its definition, capabilities, and limitations. The last two sections are concerned with the minimization of completely, as well as incompletely, specified machines.

10.1 The finite-state model – further definitions

Our attention will be focused primarily on *deterministic machines*, which possess the property that the next state $S(t + 1)$ is determined uniquely by the present state $S(t)$ and the present input $x(t)$. Thus,

$$S(t + 1) = \delta\{S(t), x(t)\}, \quad (10.1)$$

where δ is called the *state transition function*. The value of the output $z(t)$ is, in the most general case, a function of the present state $S(t)$ and the inputs $x(t)$, i.e.,

$$z(t) = \lambda\{S(t), x(t)\}, \quad (10.2)$$

where λ is called the *output function*. A machine possessing properties in Eqs. (10.1) and (10.2) is generally known as a *Mealy machine*. Another machine, known as a *Moore machine*, results when the output is a function of only the present state and is independent of the external input. In this case,

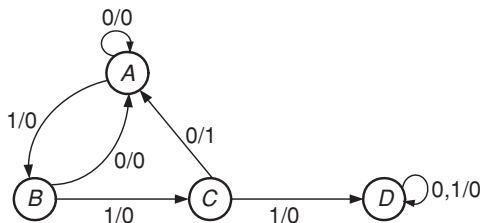
$$z(t) = \lambda\{S(t)\}. \quad (10.3)$$

Thus, we arrive at the following formal definition of a sequential machine.

Definition 10.1 A *synchronous sequential machine* M is a quintuple

$$M = (I, O, S, \delta, \lambda),$$

Fig. 10.1 State diagram for machine M .



where I , O , and S are finite nonempty sets of inputs, outputs, and states, respectively:

$\delta: I \times S \rightarrow S$ is the state transition function;

λ is the output function such that

$\lambda: I \times S \rightarrow O$ for Mealy machines;

$\lambda: S \rightarrow O$ for Moore machines.

The Cartesian product $I \times S$ is the set containing all pairs of elements (I_i, S_j) . The state transition function δ associates with each pair (I_i, S_j) an element S_k from S called the *next state*. In a Mealy machine the output function λ associates with each pair (I_i, S_j) an element O_k from O , while in a Moore machine a correspondence exists between the states and outputs.

Input-output transformations

Consider the machine M whose state diagram is given in Fig. 10.1. It is a four-state machine, with one input variable and one output variable, for which

$$S = \{A, B, C, D\}, \quad I = \{0, 1\}, \quad O = \{0, 1\}.$$

Suppose that the initial state of M is A and the input sequence is 110. Then the machine will proceed through states B and C and return to state A , while producing the output sequence 001. Thus, for an initial state A , the machine M *transforms* the input sequence 110 into 001. Similarly, for the same initial state, the input sequence 01100 is transformed into 00010. Since every computation involves some transformation of input-to-output sequences, a finite-state machine is capable of performing a variety of computations and solving a number of problems that can be expressed as a transformation of sequences.

An important function of a sequential machine is to determine whether a given input sequence is a member of some prespecified set of sequences. The machine accomplishes this function by accepting those sequences that are members of the set and rejecting those that are not. A machine, when started in its initial state, *accepts* an input sequence by producing an output value 1 as it receives the last symbol of that sequence. Thus machine M accepts the

sequences 110 and 0110 and rejects the sequence 01100, since its corresponding last output symbol is 0. The sequence detector of Fig. 9.15 can also be described as a machine that accepts those input sequences that are members of the set {all sequences whose last four symbols are 0101}.

The general problem of characterizing a machine's behavior by observing its input-output transformations is quite complex. Clearly, it is impractical to feed a machine with all possible input sequences in order to decide which it accepts. The problem increases in complexity if we wish to determine whether two arbitrary machines are related, in the sense that one machine accepts all the sequences accepted by the other. In this chapter, we shall present finite experiments to determine the characteristics, capabilities, and limitations of a machine and the relations between machines. These subjects are further developed in Chapters 13, 14, and 16.

Returning to the state diagram for M , we note that the application of input symbol 1 to M , when initially in state A , causes a transition to state B . We thus say that B is the 1-successor of A . In general, if an input sequence X takes a machine from state S_i to S_j then S_j is said to be the X -successor of S_i . For example, state D is the 111-successor of A . If M is known to be initially in either state B or C , the 10-successor will be either state A or D . We say that (AD) is the 10-successor of (BC) if A is the 10-successor of B and D that of C .

It is evident that no input sequence exists that can take M out of state D , and thus D is said to be a terminal state. Generally, a state is called *terminal* if either of the following is true: (i) the corresponding vertex in the state diagram is a *sink* vertex, i.e., no outgoing arcs that emanate from it terminate in other vertices; (ii) the corresponding vertex is a *source*, i.e., no arcs that emanate from other vertices terminate in it.

A source state is clearly not accessible from any other state and, similarly, no state is accessible from a sink state. These are extreme examples of situations that limit the state transitions in a sequential machine. In other cases, certain subsets of states may not be reachable from other subsets of states, even if the machine does not contain any terminal state. If, for every pair of states S_i, S_j of a machine M , there exists an input sequence that takes M from S_i to S_j then M is said to be *strongly connected*. Clearly, any nontrivial machine that has terminal states is not strongly connected.

10.2 Capabilities and limitations of finite-state machines

At this point, having established several behavioral properties and synthesis procedures for finite-state machines, we turn our attention to some basic questions regarding the capabilities of these machines. What can a machine do? Are there any limitations on the type of input-output transformations that can be performed by a machine? What restrictions are imposed on the capabilities of

the machine by the finiteness of the number of its states? Although a precise answer to these questions will be deferred to Chapter 16, we will point out the existence of problems not solvable by any finite-state machine and determine a characteristic of the transformations that are realizable by such machines.

Let the input to an n -state machine be an arbitrarily long sequence of 1's. In response, the machine will progress, starting from some initial state, through a succession of states, in accordance with its specified state transitions. Now, if we let the sequence be longer than n , the machine must eventually arrive at a state in which it has previously been. Consequently, from this point on, and because the input symbol remains the same, the machine must continue in a periodically repeating fashion. Clearly, for an n -state machine the period cannot exceed n and could be smaller. Moreover, the transient time until the output reaches its periodic pattern cannot exceed the number of states n . The preceding result can easily be generalized to any arbitrary input sequence consisting of a string of repeated symbols. In every such case, the output will become periodic after a transient time no longer than n .

This conclusion leads to many interesting results that exhibit the limitations of finite-state machines. For example, suppose that we want to design a machine which receives a long sequence of 1's and is to produce output symbol 1 when and only when the number of input symbols that it has received so far is equal to $k(k+1)/2$, for $k = 1, 2, 3, \dots$. That is, the desired input-output transformation has the following form:

input	=	1	1	1	1	1	1	1	1	1	1	1	1	1	1	...	
output	=	1	0	1	0	0	1	0	0	0	1	0	0	0	0	1	...

Clearly, since the output sequence does not eventually become periodic, no finite-state machine can produce such an infinite sequence.

In Section 9.1 we designed a serial adder capable of serially adding two binary numbers of arbitrary length. As another example demonstrating the limitations on the capabilities of finite-state machines, we shall show that the serial-multiplication problem is not solvable by a fixed finite-state machine, i.e., *no finite-state machine with a fixed number of states can multiply two arbitrarily large binary numbers*.

To prove the foregoing assertion, suppose that there does exist an n -state machine capable of serially multiplying any two binary numbers. Let us select 2^p as each of the two numbers to be multiplied, so that $2^p \times 2^p = 2^{2p}$, where $p > n$. The inputs are fed serially into the machine, least significant digits first: 2^p is represented by a 1 followed by p 0's, and 2^{2p} by a 1 followed by $2p$ 0's. The input symbols are fed into the machine during the first $p+1$ time units, i.e., between t_1 and t_{p+1} , as shown in the table below. During this period, the machine produces 0's. At t_{p+1} the input stops, while the machine must go on producing p additional 0's followed by a 1.

t_{2p+1}	t_{2p}	$\cdots$	t_{p+1}	t_p	$\cdots$	t_2	t_1	time
			1	0	$\cdots$	0	0	first number
			1	0	$\cdots$	0	0	second number
1	0	$\cdots$	0	0	$\cdots$	0	0	product

During the time period between t_{p+1} and t_{2p} the machine receives no input but, since $p > n$, it must have been at one of the states twice during that time. Following the same line of argument as that pursued earlier, we are led to the conclusion that its output must be periodic and the period is smaller than p . Therefore, the machine will never produce the required output symbol 1.

Note that, for any two finite numbers, we can find a machine that is capable of multiplying them. However, the preceding result demonstrates that, for every finite-state machine capable of performing serial multiplication, we can find finite numbers that it cannot multiply. The reason for this limitation stems from the limited “memory” available to the machine. While in performing addition it only had to store information regarding a single-digit carry, in the multiplication problem it must be able to store arbitrarily large partial products.

In a similar manner, we can show that no finite-state machine with a fixed number of states can perform, for arbitrarily large size blocks, the computation executed by the Turing machine of Section 9.5.

As mentioned earlier, a more general and precise study of the capabilities and limitations of finite-state machines is deferred to Chapter 16, where they will be defined in terms of regular expressions.

10.3 State equivalence and machine minimization

In constructing the state diagram (or table) for a finite-state machine, it often happens that the diagram contains redundant states, i.e., states whose functions can be accomplished by other states. We note that the number of memory elements required for the realization of a machine is directly related to the number of states. (Recall that, for an n -state machine, $k = \lceil \log_2 n \rceil$ state variables are needed for an assignment.) Consequently, the minimization of the number of states does reduce the complexity and cost of the realization in many cases. Moreover, the testing of sequential machines, which is studied in Chapter 13, is considerably simpler when the machine does not contain redundant states. It is, therefore, desirable to develop techniques for transforming a given machine into another machine that has no redundant states, such that both have the same terminal behavior.

Table 10.1 (a) Machine M_1 and (b) its state partitions

PS	NS, z		Symbol	Partition
	$x = 0$	$x = 1$		
A	$E, 0$	$D, 1$	P_0	(ABCDEF)
B	$F, 0$	$D, 0$	P_1	(ACE)(BDF)
C	$E, 0$	$B, 1$	P_2	(ACE)(BD)(F)
D	$F, 0$	$B, 0$	P_3	(AC)(E)(BD)(F)
E	$C, 0$	$F, 1$	P_4	(AC)(E)(BD)(F)
F	$B, 0$	$C, 0$		

(a)

(b)

The k -equivalence of states

Two states, S_i and S_j , of a machine M are *distinguishable* if and only if there exists at least one finite input sequence that, when applied to M , causes different output sequences depending on whether S_i or S_j is the initial state. The sequence that distinguishes these states is called a *distinguishing sequence for the pair* (S_i, S_j) . If there is any uncertainty as to whether the state of M is S_i or S_j then an application of the corresponding distinguishing sequence yields an output sequence that is sufficient to determine the unknown state uniquely. If there exists a distinguishing sequence of length k for the pair (S_i, S_j) , the states S_i, S_j are said to be *k -distinguishable*.

As an example, consider pair (A, B) of the machine M_1 whose state table is shown in Table 10.1a. The pair (A, B) is 1-distinguishable, since the input symbol 1 applied to M_1 when initially in state A yields the output symbol 1 and when initially in state B yields the output symbol 0. However, the pair (A, E) is 3-distinguishable since there is no input sequence of length less than 3 that distinguishes A from E . Furthermore, the only sequence of length 3 that is a distinguishing sequence for the pair (A, E) is $X = 111$, and the output sequences corresponding to the initial states A and E are 100 and 101, respectively. Note that 1101 is also a sequence that distinguishes A from E , although it is not the shortest such sequence. An all-zero sequence will produce identical output sequences independently of whether the initial state is A or E .

The concept of k -distinguishability leads directly to the definition of k -equivalence and equivalence. States that are not k -distinguishable are said to be *k -equivalent*. For example, states A and E of M_1 are 2-equivalent. States that are k -equivalent are also r -equivalent, for all $r < k$. States that are k -equivalent for all k are said to be *equivalent*. Thus, we arrive at the following definition.

Definition 10.2 The states S_i and S_j of machine M are said to be *equivalent* if and only if, for every possible input sequence, the same output sequence is produced regardless of whether S_i or S_j is the initial state.

Thus, S_i and S_j are equivalent (indicated by $S_i = S_j$) if there is no input sequence that distinguishes them. It will be subsequently shown (see Theorem 10.2) that states which are k -equivalent for all $k \leq n - 1$ are equivalent. Clearly, if $S_i = S_j$ and $S_j = S_k$ then $S_i = S_k$. It therefore follows (see Section 2.2) that state equivalence is an equivalence relation. In consequence of this characteristic, the set of states of the machine can be partitioned into disjoint subsets, known as *equivalence classes*, such that two states are in the same equivalence class if and only if they are equivalent and are in different classes if and only if they are distinguishable. Definition 10.2 can be generalized to the case where S_i is a possible initial state in machine M_1 while S_j is an initial state in machine M_2 , where both M_1 and M_2 have the same input alphabet.

The procedure for determining the sets of equivalent states in a machine, i.e., the equivalence classes, ensues from the following property. *If S_i and S_j are equivalent states then their corresponding X -successors, for all X , are also equivalent.* This follows since otherwise it would be trivial to construct a distinguishing sequence for (S_i, S_j) by first applying an input sequence that transfers the machine to the distinguishable successors of S_i and S_j .

The minimization procedure

The object of this section is to describe a procedure for determining the sets of equivalent states of a specified machine M . The result sought is a partition on the states of M such that two states are in the same block if and only if they are equivalent.

The first step is to partition the states of M into subsets such that all states in the same subset are 1-equivalent. This is accomplished by placing states having identical output symbols under all possible input symbols in the same subset. Clearly, two states that are in different subsets are 1-distinguishable. As an example, consider the partitions of the states of machine M_1 given in Table 10.1*b*. The first partition P_0 corresponds to 0-distinguishability and defines our initial “ignorance,” regarding the response of the various states, prior to the application of any input symbol. The partition P_1 is obtained simply by inspecting the table and placing in the same block states having the same output symbols for all input symbols. Thus A , C , and E are in the same block since their output symbols, for input symbols 0 and 1, are 0 and 1, respectively. A similar argument places B , D , and F in the other block. Clearly, P_1 establishes the sets of states that are 1-equivalent.

The next step is to obtain the partition P_2 whose blocks consist of the sets of states which are 2-equivalent, that is, equivalent under any input sequence of length 2. This is accomplished by observing that two states are 2-equivalent if and only if they are 1-equivalent and their I_i -successors, for all possible I_i , are also 1-equivalent. Consequently, two states are placed in the same block of P_2 if and only if they are in the same block of P_1 and, for each possible I_i , their I_i -successors are also contained in a block of P_1 . This step is carried out

by splitting the blocks of P_1 whenever their successors are not contained in a common block of P_1 . The 0- and 1-successors of (ACE) are (CE) and (BDF) , respectively, and, since both are contained in common blocks of P_1 , the states in (ACE) are 2-equivalent and therefore (ACE) constitutes a block in P_2 . The 1-successor of (BDF) is (DBC) but, since (DB) and (C) are not contained in a single block of P_1 , block (BDF) must be split into (BD) and (F) in such a way that the successors of the blocks in the refined¹ partition are 1-equivalent. In a similar manner P_3 is obtained by splitting block (ACE) of P_2 into (AC) and (E) , since the 1-successors of A , C , and E are D , B , and F , which are not 2-equivalent.

In general, the partition P_{k+1} is obtained from P_k by placing in the same block of P_{k+1} those states that are in the same block of P_k and whose I_i -successors for every possible I_i are also in a common block of P_k . This process places the states that are $(k + 1)$ -equivalent in the same block and states that are $(k + 1)$ -distinguishable in different blocks. Note that no state can belong to more than one block since this would make it distinguishable with respect to itself.

If, for some k , $P_{k+1} = P_k$ then the process terminates and P_k defines the sets of equivalent states of the machine; that is, all states contained in the same block of P_k are equivalent while states belonging to different blocks are distinguishable. The partition P_k is thus called the *equivalence partition*, and the foregoing procedure is referred to as the *Moore reduction procedure*. For the machine M_1 , the equivalence partition is P_3 and therefore states A and C are equivalent and so are B and D . Before proceeding with the minimization procedure, we shall prove two theorems to establish its validity and determine its length.

Theorem 10.1 *The equivalence partition is unique.*

Proof Suppose that there exist two equivalence partitions P_a and P_b and that $P_a \neq P_b$. Then there exist two states S_i and S_j that are in the same block of one partition and are not in the same block of the other. Since S_i and S_j are in different blocks of (say) P_b , there exists at least one input sequence that distinguishes S_i from S_j and, therefore, they cannot be in the same block of P_a . $\diamond$

Theorem 10.2 *If two states S_i and S_j of machine M are distinguishable then they are distinguishable by a sequence of length $n - 1$ or less, where n is the number of states in M .*

Proof The partition P_1 contains at least two blocks; otherwise M would be reducible to a combinational circuit that has only a single state. At each step, the partition P_{k+1} is smaller than or equal to P_k . (Recall that a partition $P_i \leq P_j$ if every block of P_i is contained in a block of P_j ; e.g., P_2 of M_1 is smaller

¹ A partition P is said to be a *refinement* of a partition Q if P is smaller than Q .

Table 10.2 Machine M_1^*

PS	NS, z	
	$x = 0$	$x = 1$
α	$\beta, 0$	$\gamma, 1$
β	$\alpha, 0$	$\delta, 1$
γ	$\delta, 0$	$\gamma, 0$
δ	$\gamma, 0$	$\alpha, 0$

than P_1 .) If P_{k+1} is smaller than P_k then it contains at least one more block than P_k . However, since the number of blocks is limited to n , at most $n - 1$ partitions can be generated in the reduction procedure and, thus, if S_i and S_j are distinguishable then they are distinguishable by a sequence of length $n - 1$ or smaller. $\diamond$

It can be shown (see Problem 10.15) that the above is indeed the least upper bound.

Machine equivalence

Before proceeding with the determination of the minimal machine that is equivalent to M_1 , we shall define precisely what we mean by equivalent and minimal machines.

Definition 10.3 Two machines M_1 and M_2 are said to be *equivalent* if and only if for every state in M_1 there is a corresponding equivalent state in M_2 and vice versa.

The equivalence partition has been shown to be unique. Thus, the number of blocks in the equivalence partition of a machine M defines the *minimum* number of states that any machine equivalent to M must have. The machine that contains no equivalent states and is equivalent to M is called the *minimal*, or *reduced*, form of M .

If we denote the blocks of the equivalence partition P_3 of M_1 by α , β , γ , and δ , corresponding respectively to (AC) , (E) , (BD) , and (F) , we obtain the machine M_1^* (Table 10.2). In constructing M_1^* , we specify the 1-successor of α to be γ , since the 1-successor of (AC) is (BD) , and so on. In this manner, M_1^* is specified to duplicate the state transitions and response of M_1 and, therefore, is equivalent to it. In addition, since it has been generated by the equivalence partition of M_1 , it is its minimal form.

Example We shall illustrate the reduction procedure further by applying it to a machine M_2 (Table 10.3) and finding its minimal form. The blocks

of the equivalence partition P_4 are denoted $\alpha, \beta, \dots, \epsilon$, and the reduced machine M_2^* (Table 10.4) results.

Table 10.3 (a) Machine M_2 and (b) its state partition

NS, z			Symbol	Partition
PS	$x = 0$	$x = 1$		
A	$E, 0$	$C, 0$	P_0	(ABCDEFGG)
B	$C, 0$	$A, 0$	P_1	(ABCDG)(E)
C	$B, 0$	$G, 0$	P_2	(AF)(BCDG)(E)
D	$G, 0$	$A, 0$	P_3	(AF)(BD)(CG)(E)
E	$F, 1$	$B, 0$	P_4	(A)(F)(BD)(CG)(E)
F	$E, 0$	$D, 0$	P_5	(A)(F)(BD)(CG)(E)
G	$D, 0$	$G, 0$		

(a)

(b)

Table 10.4 Machine M_2^*

NS, z			
PS		$x = 0$	$x = 1$
(A)	α	$\epsilon, 0$	$\delta, 0$
(F)	β	$\epsilon, 0$	$\gamma, 0$
(BD)	γ	$\delta, 0$	$\alpha, 0$
(CG)	δ	$\gamma, 0$	$\delta, 0$
(E)	ϵ	$\beta, 1$	$\gamma, 0$

The selection of labels $\alpha, \beta, \dots$ assigned to the blocks of P_4 is obviously arbitrary. A different assignment of labels would have described a machine with the same behavioral properties. In general, if one machine can be obtained from the other by relabeling its states then they are said to be *isomorphic* to each other. The foregoing results lead to the following basic conclusion:

- To every machine M there corresponds a minimal machine M^* that is equivalent to M and is unique up to isomorphism.

The detection of isomorphism is not always easy and is best accomplished by using a canonical representation for a machine. Such a representation is obtained by selecting a state (preferably the starting state if specified) and labeling it A . The next labels are selected in such a way that when successive rows of the table, starting in A and going down through B, C , etc., are read from left to right, the first occurrence of each new label will be in alphabetical order. Whenever a machine is given in this canonical representation, it is said to be in *standard form*. Clearly, when the starting state of a reduced machine is specified, its standard form is unique.

Table 10.5 Standard form
for M_2^*

		NS, z	
		$x = 0$	$x = 1$
α	A	$B, 0$	$C, 0$
ϵ	B	$D, 1$	$E, 0$
δ	C	$E, 0$	$C, 0$
β	D	$B, 0$	$E, 0$
γ	E	$C, 0$	$A, 0$

The transformation of a machine into its standard form will be illustrated by means of M_2^* . Denoting α by A implies that its 0-successor ϵ must be denoted B , because it is the first occurrence of a new label. Similarly, its 1-successor δ must be denoted C . Row B (i.e., ϵ) must be relabeled next; its first entry is β and, since it is a new label, it is denoted D . Similarly, γ is denoted E , and the standard form of Table 10.5 results.

When the starting states are not specified the detection of isomorphism is, in general, not as simple. If the number of states is not too large, however, isomorphism can be detected by inspecting the state diagrams of the machines. The necessary and sufficient condition for two machines to be isomorphic to each other is that their state diagrams are identical except for the labeling of their vertices.

10.4 Simplification of incompletely specified machines

In practice, it often happens that various combinations of states and input symbols are not possible. For example, the machine of Table 9.15, when in state A , will never receive input symbol 0 and, consequently, the corresponding transition and its associated output symbol may be left unspecified. In other situations the state transitions are completely defined but, for some combinations of states and input symbols, the output values may not be critical and thus are left unspecified. Such machines are said to be *incompletely specified*; the determination of their properties and methods for simplifying them are the subject of this section.

Whenever a state transition is unspecified the future behavior of the machine may become unpredictable. In order to avoid such a situation, we shall assume that the input sequences applied to the machine, when in any of its possible starting states, are such that no unspecified next state is encountered except possibly at the final step. Such an input sequence is said to be *applicable* to the starting state S_i of M . Note that the output symbols encountered need not

Table 10.6 Machine M_3 with unspecified transitions

PS	NS, z	
	$x = 0$	$x = 1$
A	$B, 1$	—
B	$-, 0$	$C, 0$
C	$A, 1$	$B, 0$

Table 10.7 An equivalent description where all transitions are specified

PS	NS, z	
	$x = 0$	$x = 1$
A	$B, 1$	$T, -$
B	$T, 0$	$C, 0$
C	$A, 1$	$B, 0$
T	$T, -$	$T, -$

all be specified for a sequence to be applicable to S_i . The next states, however, must be specified except possibly for the last symbol of the sequence.

Actually, the specified behavior of a machine with partially specified transitions can be described by another machine whose state transitions are completely specified. This transformation is accomplished by adding a terminal state T whose output symbols are unspecified and replacing all the dashes in the next-state entries by T . As an illustration, consider the machine M_3 shown in Table 10.6. The specified behavior of M_3 can be described by Table 10.7, in which all state transitions are specified and only the output symbols are partially defined.

Compatible states

In Section 10.3 we defined state and machine equivalence. We shall find it useful to generalize these concepts as follows.

Definition 10.4 State S_i of M_1 is said to *cover*, or *contain*, state S_j of M_2 if and only if every input sequence applicable to S_j is also applicable to S_i and its application to both M_1 and M_2 when they are initially in S_i and S_j , respectively, results in identical output sequences whenever the output symbols of M_2 are specified.

This covering concept can be extended to machines as follows. *Machine M_1 is said to cover machine M_2 if and only if, for every state S_j in M_2 , there is a corresponding state S_i in M_1 such that S_i covers S_j .* Clearly the machine specified by Table 10.6 is covered by that of Table 10.7. If state S_i of machine M covers another state S_j of the same machine then only S_i must be retained; S_j may be deleted.

Definition 10.5 Two states S_i and S_j of a machine M are *compatible* if and only if, for every input sequence applicable to both S_i and S_j , the same output sequence will be produced *whenever both output symbols are specified* and regardless of whether S_i or S_j is the initial state.

Hence S_i and S_j are compatible if and only if their output symbols are not conflicting (i.e., identical when specified) and their I_i -successors, for every I_i for which both are specified, are either the same or also compatible. In general, three or more states, $S_i, S_j, S_k, \dots$, are compatible if and only if, for every applicable input sequence, no two conflicting output sequences will be produced, without regard as to which of the above states is the initial state. Thus, a set of states ($S_i, S_j, S_k, \dots$) is called a *compatible* if all its members are compatible.

A compatible C_i is said to be *larger* than, or to *cover*, another compatible C_j if and only if every state contained in C_j is also contained in C_i . A compatible is *maximal* if it is not covered by any other compatible. (Note that a single state that is not compatible with any other state is a maximal compatible.) Thus, if we find the set of all the maximal compatibles, this in effect is equivalent to finding all compatibles since every subset of a compatible is also a compatible.

Generalizing slightly, we find that, in the case of incompletely specified machines, the analog to the equivalence relation studied earlier is the compatibility relation. The similarities and differences between these two relations will be pointed out subsequently.

The nonuniqueness of the reduced and minimal machines

Before developing the simplification procedure for incompletely specified machines, we shall illustrate some difficulties encountered in applying the minimization procedure of Section 10.3 to the machine M_4 shown in Table 10.8.

The dashes in row A , column 1, and in row B , column 0, mean that the output symbols associated with these transitions will be ignored and thus may be specified according to our convenience. If we replace both dashes by 1's, we find that states A and B become equivalent since their output symbols and corresponding successors are identical. Consequently, we may combine these states by redirecting to A all the transitions presently leading to B . The resulting simplified machine, shown in Table 10.9, is in reduced form and thus cannot be further simplified. If, however, we choose to specify the dashes as 0's then it is easy to verify that states A and E are equivalent, and in addition states B, C , and D become equivalent. Thus, we may relabel blocks (AE) and (BCD) by α and β , respectively, and the minimal machine of Table 10.10 results.

From the foregoing example, the following observations can be made. States A and B of M_4 are compatible and, if C and D are also compatible, so are A and E . However, states B and E are 1-distinguishable and, therefore, incompatible. Consequently, since it is not transitive the compatibility relation is not an equivalence relation. It thus follows that *a set of states is a compatible if and only if every pair of states in that set is compatible*. For example, states B, C ,

Table 10.8 Machine M_4

PS	NS, z	
	$x = 0$	$x = 1$
A	$C, 1$	$E, -$
B	$C, -$	$E, 1$
C	$B, 0$	$A, 1$
D	$D, 0$	$E, 1$
E	$D, 1$	$A, 0$

Table 10.9 A simplified reduced machine, M_4^*

PS	NS, z	
	$x = 0$	$x = 1$
A	$C, 1$	$E, 1$
C	$A, 0$	$A, 1$
D	$D, 0$	$E, 1$
E	$D, 1$	$A, 0$

Table 10.10 A minimal machine, $M_4^\#$

PS	NS, z		
	$x = 0$	$x = 1$	
(AE)	α	$\beta, 1$	$\alpha, 0$
(BCD)	β	$\beta, 0$	$\alpha, 1$

and D of M_4 form the compatible (BCD) , since (BC) , (BD) , and (CD) are compatibles.

The machines M_4^* and $M_4^\#$ both cover M_4 , and their numbers of states are each smaller than the number of states of M_4 . Both are in reduced form; i.e., they contain no redundant states. This situation, in which two different reduced machines cover a third one, is evidently in contrast with Theorem 10.1. This poses a serious difficulty in applying the previously derived minimization procedure, since we can no longer be content with finding a reduced machine covering the original one; our aim must be to find a reduced machine that not only covers the original machine but also has a minimal number of states.

A further and crucial difference between completely and incompletely specified machines is demonstrated by means of machine M_5 (Table 10.11). Because of the output entries, the only candidates for equivalence are the states A and B or B and C . Also, because of the next-state entries, A is equivalent to B only if B is equivalent to C . However, for A and B to be equivalent the dash must be replaced by a 0 while for B and C to be equivalent the dash must be replaced by a 1. Evidently, there is no way of specifying the unspecified entry so as to achieve any state equivalence. However, a hasty conclusion that M_5 is in reduced form would be false, as is shown subsequently.

The augmented machine of Table 10.12 is obtained by a process known as *state splitting*. This process involves the replacement of a state S_i by two or more states $S_i', S_i'', \dots$ such that each new state covers S_i . To ensure that the

Table 10.11 Machine M_5

PS	NS, z	
	$x = 0$	$x = 1$
A	$A, 0$	$C, 0$
B	$B, 0$	$B, -$
C	$B, 0$	$A, 1$

Table 10.12 Augmented machine

PS	NS, z	
	$x = 0$	$x = 1$
A	$A, 0$	$C, 0$
B'	$B', 0$	$B'', -$
B''	$B^+, 0$	$B', -$
C	$B^+, 0$	$A, 1$

Table 10.13 Two minimal machines corresponding to M_5

PS	NS, z		
		$x = 0$	$x = 1$
(AB')	α	$\alpha, 0$	$\beta, 0$
$(B''C)$	β	$\alpha, 0$	$\alpha, 1$

(a) Setting $B^+ = B'$

PS	NS, z		
		$x = 0$	$x = 1$
(AB')	α	$\alpha, 0$	$\beta, 0$
$(B''C)$	β	$\beta, 0$	$\alpha, 1$

(b) Setting $B^+ = B''$

augmented machine covers the original one, it is necessary to modify the next-state entries in such a way that each transition to S_i is replaced by a transition to either S'_i or S''_i , etc. In our case, state B has been split into B' and B'' and the next-state entries modified as shown in Table 10.12, where the symbol B^+ means that the transition may be either B' or B'' . Clearly, the augmented machine covers M_5 and is reducible to it by letting $B' = B'' = B$.

In general, since B' and B'' both cover B , we may specify the next-state entries B arbitrarily as B' or B'' . If, however, we select the specification shown in Table 10.12 then a simplification of M_5 becomes possible. States A and B' are compatible if their 1-successors C and B'' are compatible. Similarly, states B'' and C are compatible if their 1-successors B' and A are compatible. Thus, if we designate the compatibles (AB') and $(B''C)$ by α and β , respectively, we obtain the minimal machines of Table 10.13. The result is Table 10.13a or 10.13b, depending on whether B^+ is specified as B' or B'' .

The foregoing example demonstrates the nonuniqueness of the minimal machine in the case of incompletely specified machines. The minimal machines of Table 10.13 were obtained by allowing state B to be split in such a way that it can be made equivalent to both A and C (by specifying the unspecified output symbol differently). This points out the main difference between completely and incompletely specified machines. *While the equivalence partition consists of disjoint blocks, the subsets of compatibles may be overlapping.*

Table 10.14 Machine M_6

PS	NS, z			
	I_1	I_2	I_3	I_4
A	—	$C, 1$	$E, 1$	$B, 1$
B	$E, 0$	—	—	—
C	$F, 0$	$F, 1$	—	—
D	—	—	$B, 1$	—
E	—	$F, 0$	$A, 0$	$D, 1$
F	$C, 0$	—	$B, 0$	$C, 1$

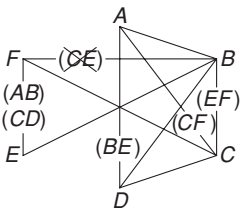
The merger graph

In reducing the machine M_4 , we actually specified the don't-care entries and thus transformed the incompletely specified machine into a completely specified one. Such a specification may not be optimal and then would drastically reduce our freedom in simplifying the machine. It is, therefore, desirable first to generate the entire set of compatibles and then to select an appropriate subset, which will form the basis for a state reduction leading to a minimal machine.

Since a set of states is compatible if and only if every pair of states in that set is compatible, it is sufficient to consider only pairs of states and to use them to generate the entire set. We shall refer to a compatible pair of states as a *compatible pair*. Let the I_k -successors of S_i and S_j be S_p and S_q , respectively; then $(S_p S_q)$ is said to be implied by $(S_i S_j)$. For example, the compatible (CF) of machine M_6 (Table 10.14) is implied by (AC) , and so on. Thus, if $(S_i S_j)$ is a compatible pair then $(S_p S_q)$ is referred to as its *implied pair*. In general, a set of states P is *implied* by a set of states Q if, for some input symbol I_k , P is the set of all I_k -successors of the states in Q . The merger graph, presented below, serves as a major tool in the determination of the set of all compatibles.

The *merger graph* of an n -state machine M is an undirected graph defined as follows.

1. It consists of n vertices, each of which corresponds to a state of M .
2. For each pair of states $(S_i S_j)$ in M , whose next-state and output entries are not conflicting, an undirected arc is drawn between the vertices S_i and S_j .
3. If, for a pair of states $(S_i S_j)$, the corresponding output symbols under all input symbols are not conflicting but the successors are not the same, an interrupted arc is drawn between S_i and S_j and the implied pairs are entered in the space.

**Fig. 10.2** Merger graph for M_6 .

Consider the machine M_6 (Table 10.14) and its merger graph, shown in Fig. 10.2. Since the next-state and output entries of states A and B are not conflicting, an arc is drawn between vertices A and B . States A and C , however, have nonconflicting output symbols but their successors under the input symbol I_2 are C and F . Therefore, (AC) is a compatible only if (CF) is; consequently,

an interrupted arc is drawn between the vertices A and C and (CF) is entered in the space. Similarly, (AD) is a compatible only if (BE) is, and thus (BE) is entered in the space of the interrupted arc drawn between A and D . No arc is drawn between A and E since these states are incompatible, their output symbols under I_2 and I_3 being conflicting. In a similar manner, every possible pair of states is checked, and the entire merger graph obtained.

A merger graph displays all possible pairs of states and their implied pairs, and since a pair of states is compatible only if its implied pair is, one must now check to determine whether the implied pairs are indeed compatibles. A pair $(S_p S_q)$ is incompatible if no arc is drawn between vertices S_p and S_q . In such a case, if $(S_p S_q)$ is written in the space of an interrupted arc, entry $(S_p S_q)$ is crossed off and the corresponding arc ignored. For example, in Fig. 10.2 the condition for (BF) to be compatible is that (CE) be compatible but, since there is no arc drawn between C and E , (CE) is incompatible and the arc between B and F is ignored. Thus, states B and F are incompatible. Next it is necessary to check whether the incompatibility of (BF) invalidates any other implied pair, that is, whether (BF) is written in the space of another interrupted arc, and so on. The interrupted arcs that remain in the graph, after all the implied pairs have been verified to be compatible, are regarded as solid ones.

For the machine M_6 , the merger graph reveals the existence of nine compatible pairs:

$$(AB), (AC), (AD), (BC), (BD), (BE), (CD), (CF), (EF)$$

Moreover, since (AB) , (AC) , and (BC) are compatibles then (ABC) is also a compatible, and so on. In this manner, the entire set of compatibles of M_6 can be generated from its compatible pairs.

In order to find a minimal set of compatibles, which covers the original machine and can be used as a basis for the construction of a minimal machine, it is often useful to find the set of maximal compatibles. Recall that a compatible is maximal if it is not contained in any other compatible. In terms of the merger graph, we are looking for complete polygons that are not contained within any higher-order complete polygons. (A *complete polygon* is one in which all possible $(n - 3)n/2$ diagonals exist, where n is the number of sides in the polygon.) Since the states covered by a complete polygon are all pairwise compatible, they constitute a compatible; and, if the polygon is not contained in any higher-order complete polygon, they constitute a maximal compatible.

In Fig. 10.2 the set of highest-order polygons are the tetragon $(ABCD)$ and the arcs (CF) , (BE) , and (EF) . Generally, after a complete polygon of order n has been found, all polygons of order $n - 1$ contained in it can be ignored. Consequently, the triangles (ABC) , (ACD) , etc., are not considered. Thus, the following set of maximal compatibles for machine M_6 results:

$$\{(ABCD), (BE), (CF), (EF)\}$$

The closed sets of compatibles

Consider the set of compatibles $\{(ABCD), (EF)\}$ of machine M_6 . Since this is the minimal number of compatibles covering all the states of M_6 , it defines a *lower bound* on the number of states in the minimal machine that covers M_6 . However, if we select the maximal compatible $(ABCD)$ to be a state in the reduced machine, its I_2 - and I_3 -successors, (CF) and (BE) , respectively, must also be selected. Since none of these compatible pairs is contained in the above set the lower bound cannot be achieved, and the set of maximal compatibles $\{(ABCD), (EF)\}$ cannot be used to define the states of a minimal machine that covers M_6 .

Definition 10.6 A set of compatibles (for a machine M) is said to be *closed* if, for every compatible contained in the set, all its implied compatibles are also contained in the set. A closed set of compatibles that contains all the states of M is called a *closed covering*.

Example For M_6 , the set $\{(AD), (BE), (CD)\}$ is closed. The set $\{(AB), (CD), (EF)\}$ is a closed covering.

For incompletely specified machines, the closed covering serves the same function as that served by the equivalence partition for completely specified machines. It specifies the states that are compatible and may be covered by a single state of a reduced machine. However, as demonstrated by the preceding examples, the closed covering is not unique and so our task is to select the one which has a minimum number of compatibles and thus defines a minimal-state machine that covers the original one.

The set containing all the maximal compatibles is, clearly, a closed covering since it covers all the states of the machine and every implied compatible is contained in the set. Consequently, the set of maximal compatibles places an *upper bound* on the number of states in the machine that cover the original state. For machine M_6 , this upper bound is four. It must be noted at this point that the concept of an upper bound is meaningless when the number of maximal compatibles is larger than the number of states in the original machine.

In the preceding discussion, we showed that the bounds on the number of states in the minimal machine can be derived from the set of all the maximal compatibles. For machine M_6 , these bounds were found to be two and four. However, since the lower bound cannot be achieved it becomes necessary to determine whether a closed covering containing three compatibles can be found. These compatibles need not necessarily be maximal; in fact, the maximal compatible $(ABCD)$ cannot be included in that set since it implies the entire set of maximal compatibles.

An inspection of the merger graph of Fig. 10.2 reveals that states A and B can be covered by the compatible pair (AB) and, similarly, states C and D

Table 10.15 A minimal machine covering M_6

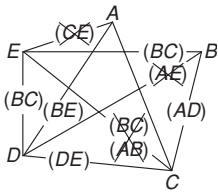
PS		NS, z			
		I_1	I_2	I_3	I_4
(AB)	α	$\gamma, 0$	$\beta, 1$	$\gamma, 1$	$\alpha, 1$
(CD)	β	$\gamma, 0$	$\gamma, 1$	$\alpha, 1$	—
(EF)	γ	$\beta, 0$	$\gamma, 0$	$\alpha, 0$	$\beta, 1$

Table 10.16 Machine M_7

PS		NS, z			
		I_1	I_2	I_3	I_4
A	—	—	—	$E, 1$	—
B	$C, 0$	$A, 1$	$B, 0$	—	—
C	$C, 0$	$D, 1$	—	—	$A, 0$
D	—	$E, 1$	$B, -$	—	—
E	$B, 0$	—	$C, -$	$B, 0$	—

can be covered by (CD) ; no pairs are implied by these compatibles, which thus form a closed set. In order to obtain the desired covering, all we need is a single compatible that covers states E and F . Fortunately, the pair (EF) is compatible and implies the pairs (AB) and (CD) , which are contained in the above set. Consequently, the set $\{(AB), (CD), (EF)\}$ is a closed covering containing three compatibles, and it thus yields a minimal three-state machine that covers M_6 . This machine is shown in Table 10.15. In a similar manner, we can show that the set $\{(AD), (BE), (CF)\}$ is also a closed covering that corresponds to a minimal machine containing M_6 .

The preceding closed coverings have been obtained by inspecting the merger graph and employing a “trial-and-error” procedure. In the following section, we shall discuss in detail a more systematic procedure for obtaining minimal closed coverings.

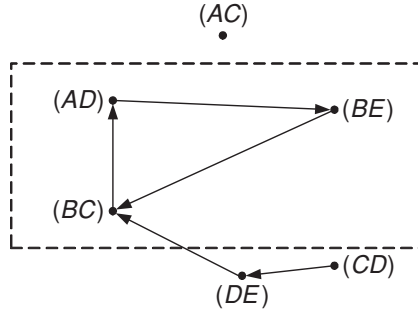
**Fig. 10.3** Merger graph for M_7 .

The compatibility graph

Consider the machine M_7 and its merger graph, shown in Table 10.16 and Fig. 10.3, respectively. The merger graph is constructed in the usual manner; since states A and B are incompatible, the arc between C and E is crossed off and, as a result, (AE) and (BD) are also found to be incompatible. The set of maximal compatibles derived from the merger graph contains four members and is given by

$$\{(ACD), (BC), (BE), (DE)\}.$$

Fig. 10.4 Compatibility graph for M_7 .



The *compatibility graph* is a directed graph whose vertices correspond to all compatible pairs and for which an arc leads from vertex $(S_i S_j)$ to vertex $(S_p S_q)$ if and only if $(S_i S_j)$ implies $(S_p S_q)$. It is a tool that aids our search for a minimal closed covering.

The compatible pairs and their implied pairs are usually obtained from the merger graph and, since a set of states is a compatible if and only if every pair of states in that set is compatible, then for a given machine the set of compatible pairs uniquely defines the entire set of compatibles.² In the compatibility graph of machine M_7 (Fig. 10.4), an arc leads from vertex (AD) to vertex (BE) because (AD) implies (BE) . No arcs emanate from (AC) since no other compatible is implied by it.

A subgraph of a compatibility graph is said to be closed if, for every vertex in the subgraph, all outgoing arcs and their terminating vertices also belong to the subgraph. In addition, if every state of the machine is covered by at least one vertex of the subgraph then the subgraph forms a closed covering for that machine.

Example The compatibility graph of Fig. 10.4 contains seven closed subgraphs (including (AC) alone and the graph itself), six of which form closed coverings for M_7 ; among them, we find the subgraphs corresponding to the following coverings:

$$\{(BC), (AD), (BE)\}$$

$$\{(AC), (BC), (AD), (BE)\}$$

$$\{(DE), (BC), (AD), (BE)\}$$

The compatibility graph itself forms a closed covering. However, it is often desirable to look for a closed subgraph that yields a simpler machine. If a closed subgraph containing the compatible pairs $(S_i S_j)$, $(S_j S_k)$, and $(S_i S_k)$ has been found, the compatible $(S_i S_j S_k)$ can be formed, and so on. Although the number of states in the minimal machine is not necessarily proportional to the number

² In order to take into account states that are incompatible with all other states, the definition of the set of compatible pairs must be generalized to include the pairs corresponding to self-compatibility, i.e., (AA) , (BB) , etc.

Table 10.17 A minimal machine that covers M_7

PS		NS, z			
		I_1	I_2	I_3	I_4
(AD)	α	—	$\gamma, 1$	$\gamma, 1$	—
(BC)	β	$\beta, 0$	$\alpha, 1$	$\beta/\gamma, 0$	$\alpha, 0$
(BE)	γ	$\beta, 0$	$\alpha, 1$	$\beta, 0$	$\beta/\gamma, 0$

Fig. 10.5 Merger table for the machine M_8 .

B	EF				
C	BC	AC, EF			
D	$\times$	$\times$	EF		
E	$\times$	$\times$	$\checkmark$	CD, CF	
F	DE	AB, DF	BC, DE	BD	BC, CD
	A	B	C	D	E

of vertices in the closed graph, the inclusion of many redundant vertices in it does tend to increase the size of the machine. A trial-and-error technique can be employed for this step. The compatibility graph thus serves to display the various possible reduced machines that correspond to the different closed coverings.

In the compatibility graph of the machine M_7 , state B is covered by the vertices (BE) and (BC) and, since at least one of them must be included in any closed covering, the entire triangle $\{(BC), (AD), (BE)\}$ must also be included. This triangle, being a closed graph that covers every state of M_7 , implies that the corresponding set of compatibles yields the desired minimal machine. Its state table is shown in Table 10.17, where the entry β/γ means that the next state may be either β or γ .

The merger table

When dealing with machines with a large number of states, it may be more convenient to record the compatible pairs and their implications in a merger table of the form illustrated in Fig. 10.5, instead of using a merger graph. Each cell of the table corresponds to the compatible pair defined by the intersection of the row and column headings. The incompatibility of two states is recorded by placing an $\times$ in the corresponding cell, while their compatibility is recorded

Table 10.18 Machine M_8

PS	NS, z	
	I_1	I_2
A	$E, 0$	$B, 0$
B	$F, 0$	$A, 0$
C	$E, -$	$C, 0$
D	$F, 1$	$D, 0$
E	$C, 1$	$C, 0$
F	$D, -$	$B, 0$

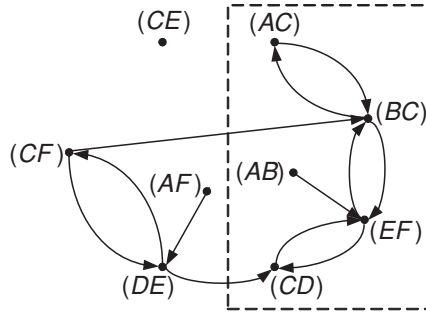
by a check mark ($\checkmark$). The entries in the cell S_i, S_j are the pairs implied by $(S_i S_j)$.

As an example, let us consider the machine M_8 , whose state table is given in Table 10.18. Its merger table is shown in Fig. 10.5. An $\times$ is inserted in cell (AD) since states A and D have conflicting output symbols; a check mark is inserted in cell (CE) because state E contains state C . In a similar way the entire table is completed and the implied compatibles entered in the appropriate cells. Now it becomes necessary to check whether these entries indeed correspond to compatible pairs. Starting from the rightmost cell, we find no contradiction until we arrive at the entry (BD) in cell (DF) . Since there is an $\times$ in cell (BD) , the pair (DF) is incompatible and is, therefore, “crossed off.” As a consequence of the incompatibility of (DF) , the pair (BF) is also incompatible and the corresponding cell is crossed off.

Once the merger table has been completed, we continue to construct the corresponding compatibility graph and to find a closed subgraph, in order to obtain the smallest closed set of compatibles. Before continuing in the above-outlined direction, we shall pause and describe a procedure for finding the set of all maximal compatibles. This procedure is the tabular counterpart to that of finding complete polygons in the merger graph. It is executed in the following manner.

1. Start in the rightmost column of the merger table for the machine and proceed left until a column containing a compatible pair is encountered. List all the compatible pairs in that column. In our example, this step yields the pair (EF) .
2. Proceed left to the next column containing at least one compatible pair. If the state to which this column corresponds is compatible with all members of some previously determined compatible, add this state to that compatible to form a larger compatible. If the state is not compatible with all members of a previously determined compatible but is compatible with some members of such a compatible, form a new compatible that includes those members and the state in question. Next, list all compatible pairs that are not included in any previously derived compatible.
3. Repeat step 2 until all columns have been considered. The final set of compatibles constitutes the set of maximal compatibles.

Fig. 10.6 Compatibility graph for M_8 .



Applying this procedure to the merger table for machine M_8 yields the following sequence of compatibility classes:

column E , (EF) ;
column D , (EF) , (DE) ;
column C , (CEF) , (CDE) ;
column B , (CEF) , (CDE) , (BC) ;
column A , (CEF) , (CDE) , (ABC) , (ACF) .

From column C , it is evident that state C is compatible with states D , E , and F and consequently the compatibles generated previously are enlarged to include state C . Column B , however, consists of a single compatible pair, which is added to the previously generated list. From column A and rows B and C we obtain the compatible (ABC) , while rows C and F , together with previously available compatibility relations, yield the compatible (ACF) . The final list is the set of maximal compatibles of M_8 .

The set of maximal compatibles clearly indicates that M_8 can be covered by a four-state machine and cannot be covered by any two-state machine. To determine whether a three-state machine that covers M_8 exists, we construct the compatibility graph shown in Fig. 10.6. It must be emphasized at this point that in many simple cases a shortcut can be taken, and the compatibility graph can be constructed directly from the state table, without the need to first find the merger graph or table.

An initial inspection of the compatibility graph does not reveal any subgraph that covers every state of M_8 and consists of just three vertices. In fact, any such graph must contain the subgraph whose vertices are (AC) , (BC) , (EF) , and (CD) . Also, since this subgraph is closed, it may seem that there exists no three-state machine that covers M_8 . However, it was pointed out earlier that it may be desirable to find a larger closed subgraph if the added vertices can be used to merge compatible pairs to yield larger compatibles. In the above example, if we add the vertex (AB) to the preceding subgraph, we obtain a set that consists of five compatible pairs, $\{(AB), (AC), (BC), (EF), (CD)\}$, and is reducible to the following closed covering:

$$\{(ABC), (CD), (EF)\}.$$

Table 10.19 A minimal machine that covers M_8

	PS	NS, z	
		I_1	I_2
(ABC)	α	$\gamma, 0$	$\alpha, 0$
(CD)	β	$\gamma, 1$	$\beta, 0$
(EF)	γ	$\beta, 1$	$\alpha, 0$

Thus, the minimum-state machine that covers M_8 consists of three states and is given in Table 10.19.

Notes and references

The minimization of completely specified machines was first studied by Moore [7] and Huffman [4] and later extended to synchronous machines by Mealy [6]. The reduction procedure for incompletely specified machines is due to Ginsburg [1, 2], Paull and Unger [8], and Kohavi [5]. Other techniques for obtaining minimal machines are available in Grasselli and Luccio [3].

- [1] Ginsburg, S.: "A synthesis technique for minimal state sequential machines," *IRE Trans. Electron. Computers*, vol. EC-8, no. 1, pp. 13–24, March 1959.
- [2] Ginsburg, S.: "On the reduction of superfluous states in a sequential machine," *J. Assoc. Computing Machinery*, vol. 6, pp. 259–282, April 1959.
- [3] Grasselli, A., and F. Luccio: "A method for combined row–column reduction of flow tables," in *Proc. Seventh Symp. Switching and Automata Theory*, Oct. 26–28, pp. 136–147, 1966.
- [4] Huffman, D. A.: "The synthesis of sequential switching circuits," *J. Franklin Inst.*, vol. 257, no. 3, pp. 161–190, 1954; no. 4, pp. 275–303, 1954.
- [5] Kohavi, Z.: "Minimization of incompletely specified sequential switching circuits," Research Report of the Polytechnic Institute of Brooklyn, PIBMRI, May 1962, New York.
- [6] Mealy, G. H.: "A method for synthesizing sequential circuits," *Bell System Tech. J.*, vol. 34, pp. 1045–1079, September 1955.
- [7] Moore, E. F.: "Gedanken-experiments on sequential machines," pp. 129–153, in *Automata Studies*, Princeton University Press, 1956.
- [8] Paull, M. C., and S. H. Unger: "Minimizing the number of states in incompletely specified sequential switching functions," *IRE Trans. Electron. Computers*, vol. EC-8, pp. 356–366, September 1959.

Problems

Problem 10.1

- (a) Prove that $n(n - 1)/2$ is an upper bound on the length of the shortest input sequence that will take a strongly connected n -state machine through each of its states at least once, regardless of the initial state. Is this the least upper bound?

- (b) Find a one-input 12-state machine for which the length of an input sequence such as that in (a) is as large as possible. (A machine for which the length is 26 can be obtained after a number of trials.)

Problem 10.2. An n -state machine is supplied with a periodic input sequence whose period is p .

- (a) Prove that the output sequence must eventually become periodic, and find a bound for the period.
- (b) Show the response of the machine M_1^* (Table 10.2) to the input sequence 010010010... In particular, find the period of the output sequence and the amount of time required for periodic behavior to start.

Problem 10.3. Prove that there exists no finite-state machine that accepts precisely *all* those sequences that read the same forward as backward, i.e., sequences that are their own reverses. (Such sequences are called *palindromes*.)

Hint: Suppose that there exists an n -state machine that accepts all palindromes; then it accepts the sequence $\underbrace{00 \cdots 00}_{n+1} 1 \underbrace{00 \cdots 00}_{n+1}$. However, this implies that it also accepts a sequence that is not a palindrome.

Problem 10.4. Determine which of the machines with the following specifications is realizable with a finite number of states. If any machine is not realizable, explain why.

- (a) A machine is to produce an output symbol 1 whenever the number of 1's in the input sequence, starting at $t = 1$, exceeds the number of 0's. For example, if the input sequence is 01100111, the required output sequence is 00100011.
- (b) A machine with a single input line and 10 output lines numbered 0 through 9 is to be designed such that, following the n th input symbol, only one output symbol 1 will be produced on the line whose corresponding number is equal to the n th digit of π (i.e., 3.14...).

Problem 10.5

- (a) Find the equivalence partition for the machine shown in Table P10.5.
- (b) Show the *standard form* of the corresponding reduced machine.
- (c) Find a minimum-length sequence that distinguishes state A from state B .

Table P10.5

PS	NS, z	
	$x = 0$	$x = 1$
A	$B, 1$	$H, 1$
B	$F, 1$	$D, 1$
C	$D, 0$	$E, 1$
D	$C, 0$	$F, 1$
E	$D, 1$	$C, 1$
F	$C, 1$	$C, 1$
G	$C, 1$	$D, 1$
H	$C, 0$	$A, 1$

Problem 10.6. For each machine in Table P10.6, find the equivalence partition and the corresponding reduced machine in standard form.

Table P10.6

NS, z			NS, z			NS, z		
PS	$x = 0$	$x = 1$	PS	$x = 0$	$x = 1$	PS	$x = 0$	$x = 1$
A	$B, 0$	$E, 0$	A	$F, 0$	$B, 1$	A	$D, 0$	$H, 1$
B	$E, 0$	$D, 0$	B	$G, 0$	$A, 1$	B	$F, 1$	$C, 1$
C	$D, 1$	$A, 0$	C	$B, 0$	$C, 1$	C	$D, 0$	$F, 1$
D	$C, 1$	$E, 0$	D	$C, 0$	$B, 1$	D	$C, 0$	$E, 1$
E	$B, 0$	$D, 0$	E	$D, 0$	$A, 1$	E	$C, 1$	$D, 1$
(a)			F	$E, 1$	$F, 1$	F	$D, 1$	$D, 1$
			G	$E, 1$	$G, 1$	G	$D, 1$	$C, 1$
			(b)			H	$B, 1$	$A, 1$
						(c)		

Problem 10.7. Two columns of the state table of an eight-state p -input symbol finite-state machine are shown in Table P10.7. Prove that this machine has either no equivalent states or no distinguishable states.

Table P10.7

NS, z			
PS	$\dots$	I_i	I_j
A		$A, 1$	$H, 0$
B		$C, 1$	$A, 0$
C		$D, 1$	$B, 0$
D		$E, 1$	$C, 0$
E		$F, 1$	$D, 0$
F		$G, 1$	$E, 0$
G		$H, 1$	$F, 0$
H		$B, 1$	$G, 0$

Problem 10.8. A transfer sequence $T(S_i, S_j)$ is defined as the shortest input sequence that takes a machine from state S_i to state S_j .

Table P10.8

NS, z		
PS	$x = 0$	$x = 1$
A	$A, 0$	$B, 0$
B	$C, 0$	$D, 1$
C	$E, 0$	$D, 0$
D	$F, 0$	$E, 1$
E	$G, 0$	$A, 0$
F	$G, 0$	$B, 1$
G	$C, 0$	$F, 0$

- (a) Find a general procedure to determine the transfer sequence for a given machine and two specified states.
- (b) Find a transfer sequence $T(A, G)$ for the machine shown in Table P10.8.

Hint: It is helpful to determine which states can be reached from S_i first by sequences of length 1, then by sequences of length 2, and so on.

Problem 10.9

- (a) Develop a procedure to determine the shortest input sequence that distinguishes a state S_i from another state S_j of a given machine.
- (b) Apply your procedure to determine the shortest input sequence that distinguishes state A from state G in the machine of Table P10.8.

Hint: Start from the first partition P_k in which S_i and S_j appear in separate blocks.

Problem 10.10. The *direct sum* $M_1 + M_2$ of two machines M_1 and M_2 is obtained by combining the tables of the individual machines, as shown in Table P10.10, in such a way that each state of the direct sum is denoted by a distinct symbol.

- (a) Use the direct sum to determine whether state A of machine M_1 is equivalent to state H of machine M_2 .
- (b) Prove that machine M_1 is contained in machine M_2 .
- (c) Under what starting conditions are machines M_1 and M_2 equivalent?

Hint: Find the equivalence partition of the direct sum.

Table P10.10

NS, z			NS, z			NS, z		
PS	$x = 0$	$x = 1$	PS	$x = 0$	$x = 1$	PS	$x = 0$	$x = 1$
A	$B, 0$	$C, 1$	E	$H, 1$	$E, 0$	A	$B, 0$	$C, 1$
B	$D, 1$	$C, 0$	F	$F, 1$	$E, 0$	B	$D, 1$	$C, 0$
C	$A, 1$	$C, 0$	G	$E, 0$	$G, 1$	C	$A, 1$	$C, 0$
D	$B, 1$	$C, 0$	H	$F, 0$	$E, 1$	D	$B, 1$	$C, 0$
M_1			M_2			E	$H, 1$	$E, 0$
						F	$F, 1$	$E, 0$
						G	$E, 0$	$G, 1$
						H	$F, 0$	$E, 1$
						$M_1 + M_2$		

Problem 10.11

- (a) Let M_1 and M_2 be strongly connected and completely specified machines, and suppose that a state S_i of M_1 is equivalent to a state S_j of M_2 . Prove that M_1 is equivalent to M_2 .
- (b) Let M_1 be a strongly connected machine, and let M_2 be completely specified. Prove that if S_i of M_1 is equivalent to S_j of M_2 then M_1 is covered by M_2 .

Problem 10.12. Determine the conditions under which two equivalent machines are isomorphic.

Problem 10.13. An unknown one-input three-state machine produces an output sequence Z in response to an input sequence X , as follows.

$X :$	0	0	0	0	1	0	1	0	0	0	1	0
$Z :$	1	0	1	0	0	1	1	0	0	0	0	1

Assuming that A is the initial state, determine the reduced standard form description of the machine.

Problem 10.14. In this problem, we shall establish a procedure for transforming a Mealy machine into a corresponding Moore machine accepting exactly the same set of sequences. To obtain the Moore machine, it is first necessary to split every state of the Mealy machine if different output values are associated with the transitions into that state. For example, state B of Table P10.14a can be reached from either state A or C . However, since different output symbols are associated with these transitions, state B must be replaced by two equivalent states, B_0 with an output symbol 0 and B_1 with an output symbol 1, as shown in Table P10.14b. Every transition to B with output symbol 0 is directed to B_0 , and every transition to B with output symbol 1 to B_1 . Applying the same procedure to state D yields the state table of Table P10.14b, which can be transformed to the Moore machine of Table P10.14c.

We now observe that the Moore machine in Table P10.14c accepts the sequences accepted by the Mealy machine in Table P10.14a, but, in addition, it produces an output symbol 1 when started in state A without having been presented with any input sequence. This Moore machine in fact accepts a zero-length sequence, called the *null sequence*. To prevent this situation we add a new starting state A' , whose state transitions are identical to those of A but whose output symbol is 0, as shown in Table P10.14d.

- Prove that, to every q -output-symbol n -state Mealy machine, there corresponds a q -output-symbol Moore machine that accepts exactly the same sequences and has no more than $qn + 1$ states.
- If the definition of acceptance by a Moore machine is modified so that acceptance of the null sequence is disregarded, show a procedure for transforming a Moore machine to the corresponding Mealy machine such that both accept the same set of sequences.
- Prove that if the Mealy machine is strongly connected and completely specified, the corresponding Moore machine will also be strongly connected and completely specified.

Table P10.14

PS	NS, z	
	$x = 0$	$x = 1$
A	$C, 0$	$B, 0$
B	$A, 1$	$D, 0$
C	$B, 1$	$A, 1$
D	$D, 1$	$C, 0$
(a)		

PS	NS, z	
	$x = 0$	$x = 1$
A	$C, 0$	$B_0, 0$
B_0	$A, 1$	$D_0, 0$
B_1	$A, 1$	$D_0, 0$
C	$B_1, 1$	$A, 1$
D_0	$D_1, 1$	$C, 0$
D_1	$D_1, 1$	$C, 0$
(b)		

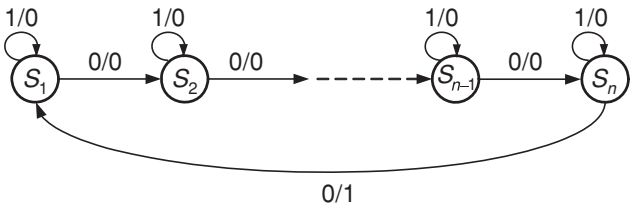
NS				NS			
PS	$x = 0$	$x = 1$	z	PS	$x = 0$	$x = 1$	z
A	C	B_0	1	A'	C	B_0	0
B_0	A	D_0	0	A	C	B_0	1
B_1	A	D_0	1	B_0	A	D_0	0
C	B_1	A	0	B_1	A	D_0	1
D_0	D_1	C	0	C	B_1	A	0
D_1	D_1	C	1	D_0	D_1	C	0
				D_1	D_1	C	1

(c)

(d)

Problem 10.15. By referring to the machine shown in Fig. P10.15, prove that the bound established in Theorem 10.2 is the least upper bound; that is, show that, for every n , the states in the pair $(S_1 S_2)$ cannot be distinguished by a sequence shorter than $n - 1$.

Fig. P10.15



Problem 10.16. A given machine is known to be either a machine M_1 in state S_i or a machine M_2 in state S_j , where S_i is not equivalent to S_j . Suppose that you are given the state tables of M_1 and M_2 , and assume that M_1 has n_1 states and M_2 has n_2 states. Prove that the given machine and its initial state can always be identified by means of an input sequence whose length L is bounded by $L \leq n_1 + n_2 - 1$.

Problem 10.17. Give a procedure that can be used to determine whether two incompletely specified machines M_1 and M_2 are related, in such a way that either M_1 contains M_2 or vice versa.

Problem 10.18

- (a) Find all the state containments present in the machine shown in Table P10.18.
- (b) Find two minimum-state machines that contain the given machine, and prove that these machines are indeed minimal.

Table P10.18

<i>PS</i>	<i>NS, z</i>	
	<i>x</i> = 0	<i>x</i> = 1
<i>A</i>	<i>B</i> , 0	<i>C</i> , 1
<i>B</i>	<i>D</i> , 0	<i>C</i> , 1
<i>C</i>	<i>A</i> , 0	<i>E</i> , 0
<i>D</i>	—	<i>F</i> , 1
<i>E</i>	<i>G</i> , 1	<i>F</i> , 0
<i>F</i>	<i>B</i> , 0	—
<i>G</i>	<i>D</i> , 0	<i>E</i> , 0

Problem 10.19. For the incompletely specified machines shown in Table P10.19, find a minimum-state reduced machine containing the original one.

Table P10.19

<i>PS</i>	<i>NS, z</i>		
	<i>I</i> ₁	<i>I</i> ₂	<i>I</i> ₃
<i>A</i>	<i>C</i> , 0	<i>E</i> , 1	—
<i>B</i>	<i>C</i> , 0	<i>E</i> , —	—
<i>C</i>	<i>B</i> , —	<i>C</i> , 0	<i>A</i> , —
<i>D</i>	<i>B</i> , 0	<i>C</i> , —	<i>E</i> , —
<i>E</i>	—	<i>E</i> , 0	<i>A</i> , —

<i>PS</i>	<i>NS, z</i>	
	<i>I</i> ₁	<i>I</i> ₂
<i>A</i>	—	<i>F</i> , 0
<i>B</i>	<i>B</i> , 0	<i>C</i> , 0
<i>C</i>	<i>E</i> , 0	<i>A</i> , 1
<i>D</i>	<i>B</i> , 0	<i>D</i> , 0
<i>E</i>	<i>F</i> , 1	<i>D</i> , 0
<i>F</i>	<i>A</i> , 0	—

Problem 10.20. Prove that the machine shown in Table P10.20 is minimal.

Table P10.20

<i>PS</i>	<i>NS, z</i>						
	<i>I</i> ₁	<i>I</i> ₂	<i>I</i> ₃	<i>I</i> ₄	<i>I</i> ₅	<i>I</i> ₆	<i>I</i> ₇
<i>A</i>	<i>F</i> , 0	<i>A</i> , —	<i>D</i> , —	<i>C</i> , —	—	—	—
<i>B</i>	—, 1	—	—	—	<i>C</i> , —	<i>D</i> , —	<i>E</i> , —
<i>C</i>	<i>C</i> , —	<i>E</i> , —	—	—	<i>F</i> , 0	<i>B</i> , —	—
<i>D</i>	—	—	<i>F</i> , —	<i>E</i> , —	—, 1	—	<i>A</i> , —
<i>E</i>	<i>A</i> , —	—	<i>A</i> , 1	—	<i>B</i> , —	—	<i>C</i> , —
<i>F</i>	—	<i>D</i> , —	—, 0	<i>B</i> , —	—	<i>E</i> , —	—

Problem 10.21. Find the reduced state table for the machine of Table P10.21. Design the circuit using a single *SR* flip-flop.

Table P10.21

<i>PS</i>	<i>NS, z₁z₂</i>			
	00	01	11	10
<i>A</i>	<i>A, 00</i>	<i>E, 01</i>	—	<i>A, 01</i>
<i>B</i>	—	<i>C, 10</i>	<i>B, 00</i>	<i>D, 11</i>
<i>C</i>	<i>A, 00</i>	<i>C, 10</i>	—	—
<i>D</i>	<i>A, 00</i>	—	—	<i>D, 11</i>
<i>E</i>	—	<i>E, 01</i>	<i>F, 00</i>	—
<i>F</i>	—	<i>G, 10</i>	<i>F, 00</i>	<i>G, 11</i>
<i>G</i>	<i>A, 00</i>	—	—	<i>G, 11</i>

Problem 10.22. Design a serial-to-parallel Excess-3-to-BCD code converter. The circuit has a single input line, receiving messages in Excess-3 code, and four output lines, z_1, z_2, z_4 , and z_8 , which are to reproduce the input messages in BCD code. Input symbols arrive serially, with the least significant digit first. Output symbols are specified only at the occurrence of every fourth input symbol. For example, if the input sequence is 1001 (which is 6 in Excess-3 code), the required output sequence is $z_1 = 0, z_2 = 1, z_4 = 1, z_8 = 0$.

11

Asynchronous sequential circuits

In many practical situations, synchronous circuits lead to more power consumption and delay than asynchronous circuits. Moreover, within large synchronous systems, it is often desirable to allow certain subsystems to operate asynchronously, thereby avoiding some of the problems associated with clocking. In this chapter, we present some of the basic properties of asynchronous sequential circuits and methods for their synthesis.

11.1 Modes of operation

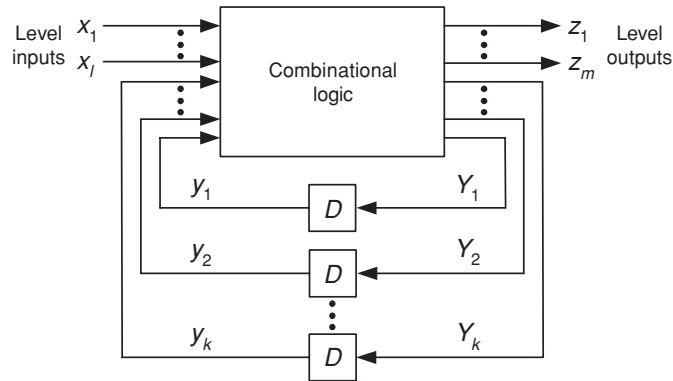
Although there are many forms that an asynchronous sequential circuit might take, the one shown in Fig. 11.1 is the most straightforward for a quick understanding of how such a circuit works. Externally, the circuit is characterized by the fact that its inputs can change at any time. Internally, it is characterized by the use of delay elements as memory devices.¹

The combination of the signals that appear at the primary inputs and delay outputs defines what is called the *total state* of the circuit. The combination of input signals $x_1, x_2, \dots, x_l$ is referred to as the *input state*; the combination of signals at the outputs of the delays, i.e., $y_1, y_2, \dots, y_k$, is referred to as the *secondary* or *internal state* of the circuit. The output values generated by the combinational logic define the output symbol of the entire circuit as well as the secondary state that the circuit will assume next. The variables $y_1, y_2, \dots, y_k$ are referred to as *secondary* or *internal variables*, and the variables $Y_1, Y_2, \dots, Y_k$ are called *excitation variables*.

For a given input state, the circuit is said to be in a *stable state* if and only if $y_i = Y_i$ for $i = 1, 2, \dots, k$. In response to a change in the input state,

¹ In practice, when the inherent delay of the combinational logic is large enough the external delay elements may not be necessary. However, for clarity of presentation, we shall assume they are present.

Fig. 11.1 The basic model for fundamental-mode circuits.



the combinational logic produces a new set of values for the excitation variables. As a result, the circuit enters what is called an *unstable state*. When the secondary variables assume their new values, i.e., the y 's become equal to the corresponding Y 's, the circuit enters its “next” stable state. Thus, a *transition from one stable state to another occurs only in response to a change in the input state*. We shall initially assume that, after a change in input values has occurred, there is no other change in any input value until the circuit enters a stable state. Such a mode of operation is often referred to as the *fundamental mode*. If only a single input value is allowed to change at any given time, it is called a *single-input-change (SIC) fundamental mode*, otherwise a *multiple-input-change (MIC) fundamental mode*. Even though SIC fundamental-mode circuits work under very restrictive assumptions, we will discuss first methodologies applicable to them, for ease of exposition, and then those applicable to MIC fundamental-mode circuits. We will then consider a generalization of MIC fundamental-mode circuits called *burst-mode* circuits. There are many other types of asynchronous circuits as well. However, they are beyond the scope of this book.

11.2 Hazards

Hazards refer to glitches. They are of two types: logic hazards and function hazards. *Logic hazards* are caused by noninstantaneous changes in circuit signals. *Function hazards* are inherent in the functional specification. The presence of hazards poses a fundamental challenge to the design of asynchronous circuits since a glitch may be misunderstood by another part of the circuit as a valid transition and cause incorrect behavior. Since we are interested in both SIC and MIC fundamental modes of operation, we will see how hazards can form under each mode and how to design circuits to be free of hazards whenever possible.

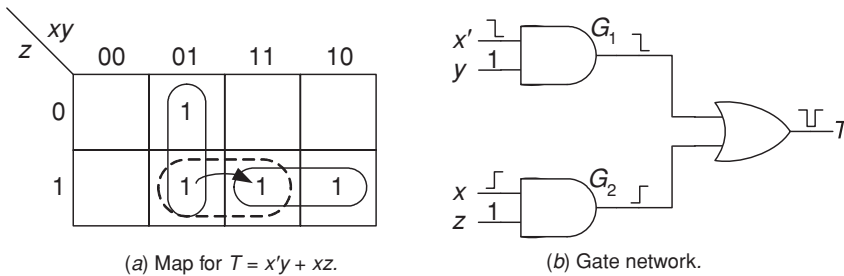


Fig. 11.2 Single-input-change static hazard example.

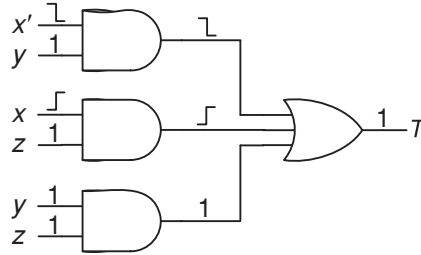
Design of SIC hazard-free circuits

Consider the function $T(x, y, z) = \sum(2, 3, 5, 7)$, whose map is shown in Fig. 11.2a, and its minimal sum-of-products implementation in Fig. 11.2b. Suppose that the value of inputs y and z is 1 and that the value of input x is changed from 0 to 1. Clearly, the value of T must remain at 1 regardless of the value of x . As the value of x changes, the transmission path through the network of Fig. 11.2b changes from gate G_1 to G_2 . In an ideal situation this change would be instantaneous, and the value of T would remain constant at 1. In practice, however, different delays are associated with the gates G_1 and G_2 . As a consequence, if, for example, the delay of gate G_1 is smaller than that of gate G_2 , and if x changes from 0 to 1 (while $y = z = 1$), then the transmission $x'y$ through gate G_1 will become 0 shortly before the transmission xz through gate G_2 becomes 1. During this period, T will be 0. This phenomenon is known as a *static logic hazard* and is indicated by the arrow in the map of Fig. 11.2a. More specifically, since only a single bit changes in the transition, it is called an *SIC static logic hazard*. In general, an SIC static logic hazard is a scenario in which a single input-variable change might cause a momentarily incorrect output value when, in fact, the output value should remain constant. Whether such an incorrect output value actually occurs depends on the exact amounts of delay associated with the various circuit elements.

Two input combinations are said to be *adjacent* if they differ by the value of a single input variable. For example, $x'yz$ and xyz are adjacent. A transition between a pair of adjacent input combinations that correspond to identical output values contains an SIC static logic hazard if it makes possible the generation of a momentary spurious output value. Such hazards may occur whenever there exists a pair of adjacent input combinations that produce the same output value and there is no cube (in the map) containing both combinations.

On the basis of the above discussion, in the above example the static logic hazard can be removed by including the prime implicant yz in the expression for T , as indicated by the dotted cube in the map of Fig. 11.2a, that is, writing $T = x'y + xz + yz$. The resulting circuit is shown in Fig. 11.3. Clearly, when

Fig. 11.3 Single-input change hazard-free network.



$y = z = 1$ the output value will be 1 regardless of the delays associated with x' and x .

When the hazard occurs during a static $0 \rightarrow 0$ transition at the output it is called a *static-0 logic hazard*, and for a $1 \rightarrow 1$ transition a *static-1 logic hazard*.

A *transition cube* $[m_1, m_2]$ is a set of all minterms that can be reached starting from minterm m_1 and ending at minterm m_2 . For example, the transition cube $[010, 100]$ contains the following minterms: 000, 010, 100, 110. In the example in Fig. 11.2, we saw that transition cube $[011, 111]$ must be included in some product of the sum-of-products realization in order to get rid of the static-1 logic hazard. Such a cube is called a *required cube*.

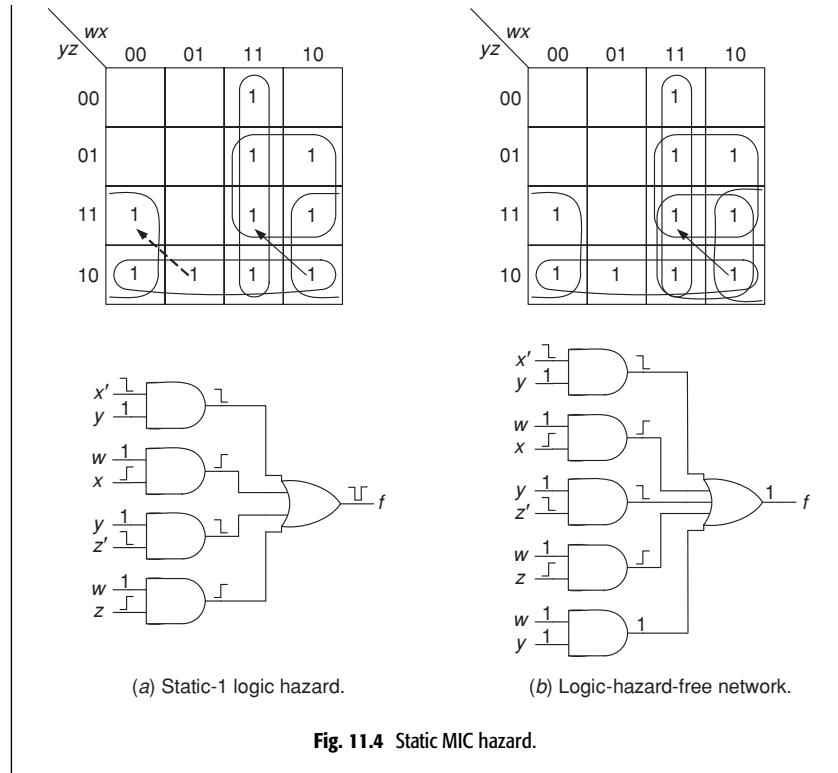
In the sum-of-products realization of a function, no cube for any product term can contain either of the two input combinations involved in a $0 \rightarrow 0$ output transition since a cube only includes the 1's of a function. Thus, the only way in which a static-0 logic hazard can occur is if a product term has both x_i and x'_i as input literals. Since there is no reason to include such product terms in the expression for the function, such hazards can be trivially avoided.

If the two input combinations are such that they correspond to a $0 \rightarrow 1$ output transition but during the transition the 0 may change to 1 and then 0 and finally stabilize at 1 then the sum-of-products realization is said to have a *dynamic $0 \rightarrow 1$ logic hazard*. A dynamic $1 \rightarrow 0$ logic hazard can be similarly defined. Using reasoning similar to that above for static-0 logic hazards, a dynamic $0 \rightarrow 1$ or $1 \rightarrow 0$ logic hazard is not possible in the SIC scenario unless some product term has both x_i and x'_i as input literals.

Design of MIC hazard-free circuits

In an MIC scenario, several inputs change values *monotonically*, i.e., at most once, from one input combination to another. In this transition, if the function changes values more than once then the transition is said to have a *function hazard*.

Example Consider the MIC transition, denoted by the broken arrow in the map shown in Fig. 11.4a, from $wxyz = 0110$ to $wxyz = 0011$. If z changes before x does then the function will go from 1 to 0 and then back to 1. Hence, the function changes values more than once and thus this transition has a function hazard.



If a transition has a function hazard then no implementation can be guaranteed to be hazard-free for this transition, assuming that the gates and wires have arbitrary delays, because the glitch is present in the functional specification itself. Fortunately synthesis approaches, such as those based on the burst mode, only need to deal with transitions that are free of function hazards. Thus, we shall focus only on MIC transitions that are free of function hazards.

Example Consider the MIC transition, denoted by the solid arrow in the map shown in Fig. 11.4a, from $wxyz = 1010$ to $wxyz = 1111$. This transition does not have a function hazard. However, it may lead to a static-1 logic hazard, as shown in the AND-OR circuit in Fig. 11.4a. Such a hazard could occur in a situation in which the falling transitions at the outputs of two AND gates are faster than the rising transitions at the outputs of the other two AND gates. Such hazards can be tackled in the same manner as those caused by an SIC transition, as shown in Fig. 11.4b. The AND gate that realizes wy has a steady 1 at its output during the above transition. The reason is that it covers the entire required cube $[1010, 1111]$ in the map. Such a cube includes all the minterms that can be encountered during such a monotonic transition. This eliminates the hazard at f .

Just as in the SIC case, avoiding a static-0 logic hazard is straightforward (simply avoid any product term with both x_i and x'_i as literals). Thus, we will look at MIC dynamic hazards next.

Example Consider the MIC transition, denoted by the solid arrow in the map shown in Fig. 11.5a, from $wxyz = 1110$ to $wxyz = 0111$. This dynamic transition does not have a function hazard. However, the transition does have a dynamic logic hazard, as can be seen from the AND–OR circuit in Fig. 11.5a. This dynamic hazard may be created by a combination of the static-0 hazard at the output of the AND gate G_1 and the falling transition at the outputs of several other AND gates.

A necessary condition for a dynamic transition to be hazard-free is that each of its $1 \rightarrow 1$ subtransitions are also hazard-free. This can be ensured by including these subtransitions in some product of the sum-of-products realization. For the above dynamic transition, these subtransitions are $[1110, 1111]$ and $[1110, 0110]$. They are called the required cubes of this dynamic transition. The set of required cubes includes all minterms that can be encountered in the dynamic transition. Since $[1110, 1111]$ and $[1110, 0110]$ are included in the products wx and yz' , respectively, the above necessary condition is already met in this case.

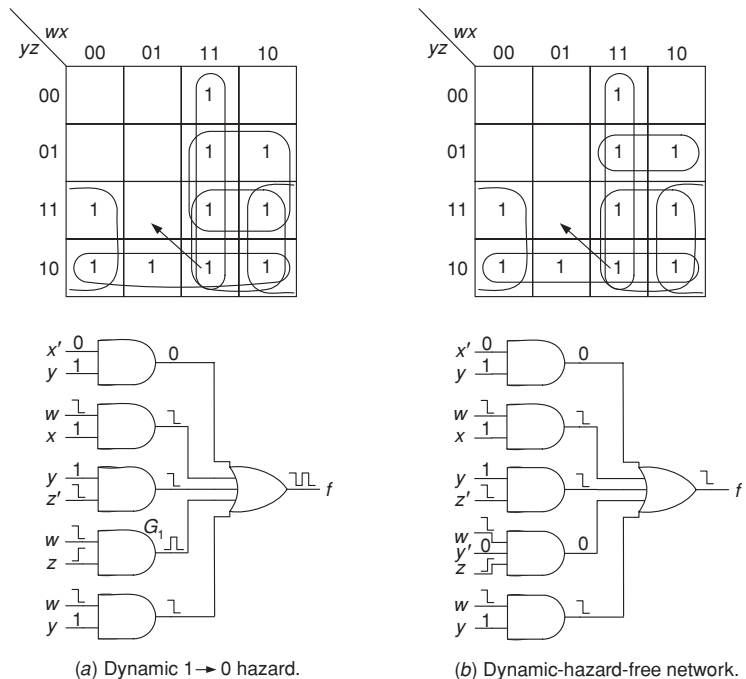


Fig. 11.5 Dynamic MIC hazard.

In order to prevent the dynamic hazard at f , we also need to make sure that no AND gate temporarily turns on during the MIC transition. For example, the static-0 hazard at the output of G_1 needs to be avoided. This hazard is caused when G_1 temporarily turns on. This happens because the corresponding product term wz intersects the dynamic MIC transition $1110 \rightarrow 0111$. This is called an *illegal intersection* and the dynamic transition is called a *privileged cube*. One can see that, during this dynamic transition, the inputs could be momentarily at 1111 (if z changes before w), which is a minterm of wz . To avoid this situation, illegal intersections of privileged cubes are disallowed by reducing the product term wz to $wy'z$, as shown in the map in Fig. 11.5b, thus eliminating the hazards as can be seen from the corresponding circuit.

The above discussions show how to eliminate hazards for an MIC transition. An MIC transition that results in a $1 \rightarrow 1$ transition at the output must be completely covered by a product term. The $0 \rightarrow 0$ MIC transition does not lead to a hazard. For the $1 \rightarrow 0$ and $0 \rightarrow 1$ cases, we have to make sure that every product term that intersects the MIC transition also contains its starting or end point, respectively.

To obtain a hazard-free sum-of-products implementation H of function f for a specified set of input transitions, we need to make sure that (i) each required cube is contained in some implicant of H and (ii) no implicant of H *illegally intersects* any specified dynamic transition. Such an implicant is called a dynamic-hazard-free implicant (dhf-implicant).

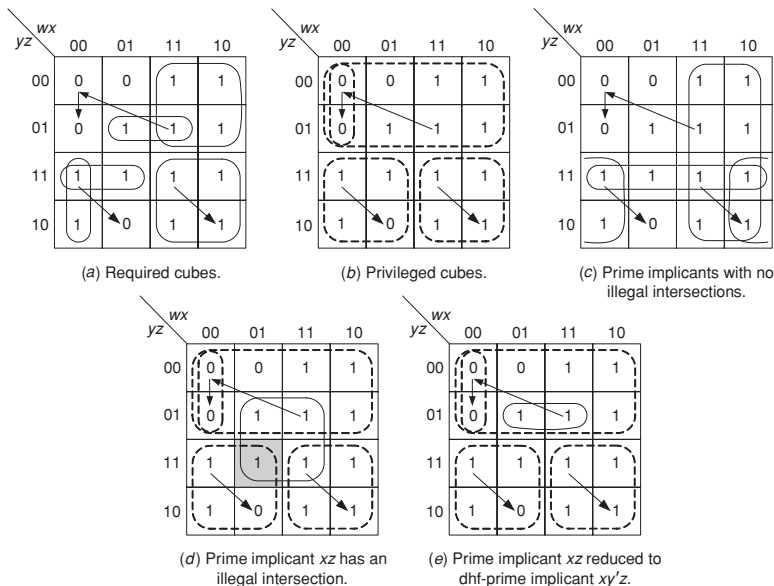
The above problem requires that we make use only of dhf-prime implicants² while covering every required cube in the sum-of-products minimization. This is similar to the Quine–McCluskey minimization method we discussed in Chapter 4.

Example Consider the map shown in Fig. 11.6a. It contains four function-hazard-free transitions, depicted by the four arrows. The set of required cubes is also shown in this map. The map in Fig. 11.6b shows the set of privileged cubes. The prime implicants that do not have any illegal intersections with the two dynamic transitions ($1101 \rightarrow 0000$ and $0011 \rightarrow 0110$), i.e., the dhf-prime implicants, are shown in Fig. 11.6c. However, the prime implicant xz does have an illegal intersection with the transition $0011 \rightarrow 0110$, as shown by the shaded region in Fig. 11.6d. This intersection can be avoided by reducing xz to the dhf-prime implicant $xy'z$, as shown in Fig. 11.6e.

² A *dhf-prime implicant* is a dhf-implicant that is not contained in any other dhf-implicant.

Table 11.1 Chart for dhf-prime implicants

Dhf-prime implicants	Required cubes				
	wy'	wy	$xy'z$	$w'yz$	$w'x'y$
w	×	×			
yz				×	
$x'y$					×
$xy'z$			×		

**Fig. 11.6** Derivation of a hazard-free sum-of-products expression.

A minimal hazard-free sum-of-products realization can now be obtained using a concept similar to the prime implicant chart (see Chapter 4). This is shown in Table 11.1, in which the rows correspond to dhf-prime implicants and the columns to required cubes. The aim is to find a minimal set of dhf-prime implicants that contains all required cubes. This can be done using the analogous concepts of essential rows, dominated rows, and dominating columns used earlier for prime implicant charts. From Table 11.1, we see that all rows are essential. Thus, $w + yz + x'y + xy'z$ is a hazard-free sum-of-products expression.

Since the hazard-free AND–OR implementation may be too large, it may be necessary to obtain a hazard-free multi-level implementation from it. In order to do so, we have to apply *hazard-nonincreasing* logic transformations. These transformations ensure that if the initial circuit is hazard-free, so is the final circuit. The following laws from Boolean algebra constitute some hazard-nonincreasing transformations:

- the associative law, $(x + y) + z \Leftrightarrow x + (y + z)$, and its dual, $(xy)z \Leftrightarrow x(yz)$,
- De Morgan's theorem, $(x + y)' \Leftrightarrow x'y'$, and its dual, $(xy)' \Leftrightarrow x' + y'$,
- the distributive law, $xy + xz \Rightarrow x(y + z)$,
- the absorption law, $x + xy \Rightarrow x$, and
- the $x + x'y \Rightarrow x + y$ law.

The directions of the implication arrows indicate in which directions the transformations are applicable. Similarly, the insertion of inverters at the primary inputs and the circuit output is also hazard-nonincreasing.

Example Consider the AND–OR realization in Fig. 11.5b, which is dynamic-hazard-free for the MIC transition $1110 \rightarrow 0111$. A multi-level realization can be obtained from it using the distributive law: $x'y + wx + yz' + wy'z + wy = (x' + z' + w)y + wx + wy'z$, as shown in Fig. 11.7. As can be seen, this multi-level realization is also dynamic-hazard-free.

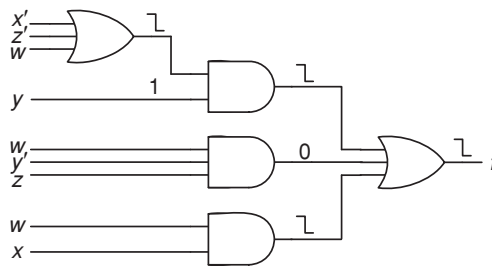


Fig. 11.7 Multi-level hazard-nonincreasing realization.

11.3 Synthesis of SIC fundamental-mode circuits

The purpose of this section is to develop systematic techniques for the design of SIC fundamental-mode asynchronous sequential circuits. The approach to be followed is to construct a flow table which describes the circuit behavior, to simplify the table, whenever possible, and finally, to realize it at the gate level.

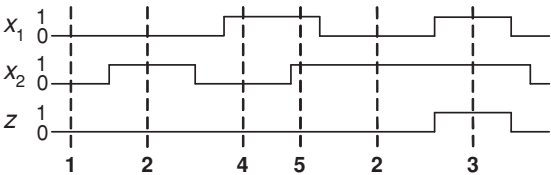
The flow table

As in the case of synchronous circuits, the least systematic step in the synthesis procedure is that of transforming a verbal statement of the desired circuit behavior into a precise description that specifies the circuit operation for every applicable input sequence. A convenient method for describing the behavior of an asynchronous circuit is by means of a *flow table*. As an example, consider a sequential circuit with two inputs, x_1 and x_2 , and one output, z . The initial

Table 11.2 Partial flow table

State, output			
x_1x_2			
00	01	11	10
1 , 0	→ 2		
	↓		
	2 , 0	→ 3	
		↓	
		3 , 1	

Fig. 11.8 Input-output sequences.



input state is $x_1 = x_2 = 0$. The output value is to be 1 if and only if the input state is $x_1 = x_2 = 1$ and the preceding input state is $x_1 = 0, x_2 = 1$. A possible pair of input sequences and the corresponding output sequence are illustrated in Fig. 11.8.

We now show how to construct the flow table for the given circuit. The column headings of Table 11.2 are the input combinations. The table entries give the states and output values. The arrows indicate state transitions between the table entries. Initially, the input values are $x_1 = x_2 = 0$, and the circuit is in a state designated **1**; the use of boldface indicates that the state in question is stable. This is recorded in the table by entering a **1** in the first row of column $x_1x_2 = 00$. To the right of the **1**, output entry 0 is entered, since the output value is 0 when the circuit is in state **1**. Now x_2 becomes 1 while x_1 remains 0, as illustrated in Fig. 11.8; the circuit enters a different state, designated **2**, while its output value is still 0. This is recorded in Table 11.2 by entering a **2** in the second row, column $x_1x_2 = 01$, and a 0 in the corresponding output location. In the first row of the 01 column, we enter a 2 to indicate that, as a result of the change in the value of the input variables, a transition to state **2** will occur. Thus, while the lightface entry 2 designates an *unstable* transient condition, the boldface entry **2** designates the *stable* state assumed by the circuit as a result of the above input change. If input x_1 changes from 0 to 1 while the circuit is in state **2**, the circuit enters another stable state, designated **3**, which is associated with the output value $z = 1$. This is indicated by entering a lightface 3 in the second row, column 11. In the same column and immediately below the lightface 3, a boldface **3** is entered to identify the stable state to which the circuit goes as a result of the last change of input values. The output value 1 is associated with the stable state **3**.

Table 11.3 Primitive flow table

State, output			
x_1x_2			
00	01	11	10
1 , 0	2	—	4
1	2 , 0	3	—
—	2	3 , 1	4
1	—	5	4 , 0
—	2	5 , 0	4

Thus a change in the value of the circuit inputs causes a *horizontal* move in the flow table to the column whose heading corresponds to the new input value. A change in the internal state of the circuit is reflected by a *vertical* move, as shown by the arrows in Table 11.2. (Note that, since a change in the inputs can occur only when the circuit is in a stable state, a horizontal move can emanate only from a boldface entry.) For the time being, we shall specify only the output symbols of stable states, leaving the output symbols of unstable states for later consideration.

So far, we have specified the state transitions leading from the initial state to a state that generates an output value 1. Clearly, we must also specify what is to happen if an input sequence other than the one considered occurs. Suppose, for example, that initially x_1 changes before x_2 . As a result, the circuit will go through unstable state 4 to stable state **4** (see Table 11.3), for which the output symbol is 0. Since the two inputs are not allowed to change simultaneously, a dash is entered in the first row, column 11, and in the second row, column 10, of Table 11.3 and so on. In general, to specify the operation of a circuit, we use a partly developed table similar to Table 11.2 and specify the transitions for each allowable input change, starting from every stable state. If a new stable state is to be added, a *new* row is created in the column corresponding to the present values of input variables. Any move from a stable state can be caused only by a change in the input variables.

The table thus constructed is called a *primitive flow table*. Its main characteristics are that *only one stable state appears in each row and the output symbols are specified only for stable states*.

We will now complete the flow table. Starting from entry **2** in column 01, if the inputs change to 00, it is necessary to send the circuit into the state that corresponds to the input conditions $x_1 = x_2 = 0$ and output $z = 0$, i.e., the state **1**. Therefore, a lightface 1 is entered in column 00 of the row containing **2**. The circuit can leave state **3** by a change of inputs from $x_1x_2 = 11$ to either $x_1x_2 = 01$ or $x_1x_2 = 10$. In the first case the value of input x_1 has changed from 1 to 0, while x_2 remains equal to 1; if x_1 changes again (to 11, we want the circuit to go back to state **3** and to produce a 1 output value.

This transition can be accomplished if we enter a lightface 2 in column 01 in the third row. If, however, x_2 changes from 1 to 0 while x_1 remains at 1 then the circuit goes to state **4**, which satisfies these conditions. Starting from state **4**, we observe that if the value of x_2 changes from 0 to 1 then the two circuit input values are 1's. However, since the last input to change was x_2 , not x_1 , the output value should be 0. Consequently, a new state, designated **5**, for which the output value is 0 must be added in column 11.

At this point, we have obtained all the stable states shown in Table 11.3. The table is completed by entering the unstable states corresponding to the various possible changes of input variables. A dash has been entered wherever a change of input variables is not allowed.

Reduction of flow tables

The primitive flow table developed in Table 11.3 has five *distinct* states. Thus, it appears that at least three variables are needed to represent these states. However, as we shall see, this does not necessarily mean that three secondary variables must be employed, since the input variables may be used to distinguish some of the states. This problem can be better understood if we think of each *stable state* as representing a *total state* of the circuit, i.e., a state defined by the state of the internal (i.e., secondary) variables as well as by the state of the primary input variables. Accordingly, an asynchronous circuit can go from one stable state to another stable state without necessarily changing the values of any of its internal variables. Such a situation simply means that these two states are distinguished only by the states of the input variables. (Note that in the case of synchronous circuits the input variables cannot be used to specify the total state of the circuit since, although a synchronous circuit is stable when the clock pulses are absent, the input values are not available to it.)

In general, when setting up a primitive flow table, one is not concerned about adding states that may turn out to be redundant. All that is necessary is that a sufficient number of states be included, such that the circuit behavior is completely specified for every allowable input sequence. The reduction of a primitive flow table thus has two functions, namely, eliminating redundant stable states and merging those stable states that are distinguishable by the input states. Since there is only one stable state in each row of the primitive flow table, we may think of it as the “present state (PS)” and rewrite Table 11.3 in the form shown in Table 11.4, where the boldface entries again serve to identify stable states. The flow table in the form of Table 11.4 is now indistinguishable from the state table of an incompletely specified synchronous circuit, possibly with the exception that every row of the flow table contains one “next-state” entry which is identical to the “present state.”

The analogy between the minimization problem of synchronous circuits and the reduction of primitive flow tables of asynchronous circuits is now apparent.

Table 11.4 Primitive flow table

<i>PS</i>	State, output			
	<i>x₁x₂</i>			
	00	01	11	10
1	1, 0	2	—	4
2	1	2, 0	3	—
3	—	2	3, 1	4
4	1	—	5	4, 0
5	—	2	5, 0	4

Table 11.5 Reduced flow tables

State, output				State, output			
<i>x₁x₂</i>				<i>x₁x₂</i>			
00	01	11	10	00	01	11	10
1, 0	2, 0	3, 1	4, 0	1, 0	2, 0	5, 0	4, 0
1, 0	2, 0	5, 0	4, 0	1, 0	2, 0	3, 1	4, 0

(a) The closed covering
{(123), (45)}

(b) The closed covering
{(145), (23)}

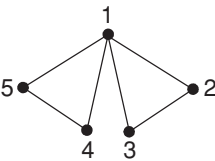


Fig. 11.9 Merger graph for Table 11.4.

We may, therefore, utilize the techniques of Section 10.4 to reduce the number of rows in primitive flow tables. The merger graph for the flow table of Table 11.4 is shown in Fig. 11.9, where the maximal compatibles are {(123), (145)}. Whenever bold and lightface entries are to be combined, the resulting entry is bold since the corresponding state must be stable. Thus, for example, the row of Table 11.4 that corresponds to the maximal compatible (123) is

$$\mathbf{1, 0} \quad \mathbf{2, 0} \quad \mathbf{3, 1} \quad 4, -$$

Two minimum-row flow tables corresponding to Table 11.3 are shown in Table 11.5. Table 11.5a corresponds to the closed covering {(123), (45)} while Table 11.5b corresponds to the closed covering {(145), (23)}. The output symbols associated with unstable states have been specified to correspond to their respective stable states, e.g., the output symbol associated with the unstable state 2 is 0 since the output symbol of the stable state 2 is 0, and so on.

Specifying the output symbols

Our next step is to consider the assignment of output values to the unstable states in the reduced flow table. This assignment depends on the required output value changes, as well as on a number of design objectives that will be discussed subsequently. Suppose that the circuit is to go from one stable state to another stable state associated with the same output value, as is the case, for example,

Table 11.6 Specification of output symbols

State, output				State, output			
x_1x_2				x_1x_2			
00	01	11	10	00	01	11	10
1 , 0	2	3 , 0	4	1 , 0	2, 1	3 , 0	4, 0
1	2 , 1	3	4 , 0	1, 0	2 , 1	3, 0	4 , 0
5 , 1	6	7 , 1	8	5 , 1	6, 0	7 , 1	8, 0
5	6 , 0	7	8 , 0	5, 1	6 , 0	7, 1	8 , 0

(a) Reduced flow table

(b) Reduced flow table with output values specified

in Table 11.5a in the transition from state **1** to state **4**. In such a case there must be no momentary complementary output value. Consequently, unstable state 4 must be assigned a 0 output value. Similarly, the output value associated with unstable state 2 is specified as 0.

When a circuit changes from one stable state with a given output value to another stable state with a different output value, the transition may be associated with either output value. The choice of output value can be made according to whether it is desired that the output-value change will occur as soon as possible or as late as possible. When the relative timing of the output-value change is of no importance, the choice of output value is made in such a way as to minimize the output logic. Consider, for example, the flow table in Table 11.6a. To determine the output value associated with unstable state 2, note that state **2** can be reached from either state **1** or state **3**. Since both are associated with a 0 output value, while the output value of state **2** is 1, then if a fast output value change is desired the output value associated with unstable state 2 must be a 1 but if a slow output value change is desired then the output of 2 should be set to 0. However, the output of unstable state 1 must be set to 0, since the output values of states **1** and **4** are both 0's.

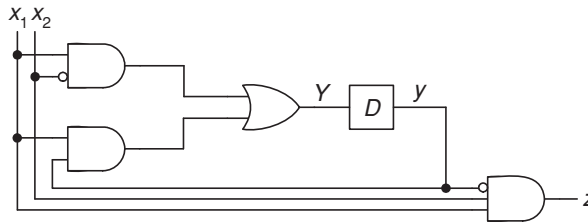
The output value associated with unstable state 4 must be a 0, as must the output value associated with 3, since in each case the transition is between stable states associated with 0 output values. Note that this output assignment means that the output value associated with the transition from **2** to **3** cannot be made in such a way that the change is as late as possible. An examination of the output values associated with the unstable states in the last two rows shows that they are all optional. The output assignment shown in Table 11.6b has been made in such a way as to obtain fast output value changes.

Excitation and output tables

To realize a reduced flow table, it is necessary to assign distinct combinations of the secondary-variable values to the rows of the flow table and derive the corresponding excitation and output functions. For a state to be stable, the

Table 11.7 Excitation and output table

y	Y, z			
	x_1x_2	01	11	10
0	0, 0	0, 0	0, 1	1, 0
1	0, 0	0, 0	1, 0	1, 0

Fig. 11.10 A realization of Table 11.7.

values of the Y 's must be the same as those of the y 's. Therefore, the excitation required for any stable state is determined from the value of the secondary variables assigned to the row in which the stable state is contained. A lightface entry represents an unstable state, which must eventually assume the value of the secondary state assigned to the boldface entry having the same number. There are several difficulties associated with the state-assignment problem and with the transitions assigned to the unstable states. These problems are discussed in detail later.

To realize the reduced flow table of Table 11.5a, we assign a 0 to the first row and a 1 to the second row, as shown in Table 11.7. Every boldface entry in the first row is now replaced by a 0, and in the second row by a 1. The lightface entry 2 is assigned a 0, since the circuit must go into stable state **2**; this assignment thus requires the variable y to change its state from 1 to 0 upon receiving input symbol 01. Similarly, the lightface entries 1 and 4 are assigned 0 and 1, corresponding respectively to the assignments of the boldface entries **1** and **4**. The excitation and output functions derived from Table 11.7 are

$$Y = x_1x_2' + x_1y,$$

$$z = x_1x_2y'.$$

A corresponding realization is shown in Fig. 11.10.

A synthesis example

The synthesis procedure for SIC fundamental-mode asynchronous circuits developed in the foregoing section consists of several steps, which can be summarized as follows.

1. A primitive flow table is constructed from the verbal description of circuit operation. In most cases, we specify only those output values that are associated with stable states.
2. A minimum-row reduced flow table is obtained by merging the rows in the primitive flow table. Either the merger graph or the merger table may be used to perform the reduction.
3. Secondary variables are assigned to the rows of the reduced flow table, from which excitation and output tables are constructed. The output values associated with unstable states are specified according to various design requirements.
4. The excitation and output functions are derived, and the corresponding hazard-free circuit constructed.

We shall now illustrate the above procedure by designing an asynchronous sequential circuit with two inputs, x_1 and x_2 , and two outputs, G and R , which is to behave in the following manner. Initially, both input values and both output values are equal to 0. Whenever $G = 0$ and either the value of x_1 or x_2 becomes 1, G turns “on” (i.e., attains the value 1). When the value of the second input becomes 1, R turns on. The first input value that changes from 1 to 0 turns G “off” (i.e., sets G equal to 0). The output R turns off when G is off and either input value changes from 1 to 0.

From the specification of the problem, it is evident that whenever $x_1 = x_2 = 0$ then $G = R = 0$, and whenever $x_1 = x_2 = 1$ then $G = R = 1$. Consequently, columns 00 and 11 of the primitive flow table must each contain a single stable state. When the input combination x_1x_2 is 01, the output symbol GR may be either 10 or 01, depending on the preceding input combination. Since a different stable state must be included in each column of the flow table for every possible output condition, column 01 must contain at least two stable states. Similar arguments show that column 10 must also contain at least two stable states, which will be associated with the output combinations 01 and 10. We thus conclude that the primitive flow table for the circuit in question must contain six stable states, as illustrated in Table 11.8a. The primitive flow table can now be completed by inserting the dashes, whenever a multiple change of input values is implied, and by specifying the unstable states.

When the circuit is in state **1**, any allowed change of input symbols causes a change in output symbols from 00 to 10. Hence, the circuit must be directed to either state **2** or **5**, depending on whether the change in input symbols is from 00 to 01 or 10, respectively. This is accomplished by entering a 2 in column 01 and a 5 in column 10 in the first row of Table 11.8b. It is a simple matter to complete the unstable entries in columns 00 and 11, since each of these columns contains just a single stable state. Therefore, 1's and 4's are entered in the appropriate locations in Table 11.8b. The only as yet unspecified entries are those in the row containing **4** in columns 01 and 10. If we start from state **4** and change the input symbols to 01 or 10, G must be turned off. Hence, we

Table 11.8 Primitive flow table

State, GR				State, GR			
x_1x_2				x_1x_2			
00	01	11	10	00	01	11	10
1 , 00				1 , 00	2	—	5
	2 , 10			1	2 , 10	4	—
	3 , 01			1	3 , 01	4	—
		4 , 11		—	3	4 , 11	6
			5 , 10	1	—	4	5 , 10
			6 , 01	1	—	4	6 , 01

(a) Table containing only stable states

(b) Completed primitive flow table

Table 11.9 Reduced flow table

State, GR			
x_1x_2			
00	01	11	10
1 , 00	2 , 10	4, 11	5 , 10
1, 01	3 , 01	4 , 11	6 , 01

Table 11.10 Excitation and output table

Y, GR				
y	x_1x_2			
	00	01	11	10
0	0, 00	0, 10	1, 11	0, 10
1	0, 01	1, 01	1, 11	1, 01

direct the transitions to states **3** and **6**, which correspond to the output condition $GR = 01$.

The merger graph for the primitive flow table is shown in Fig. 11.11. It contains two triangles leading to the closed covering $\{(125), (346)\}$. The reduced flow table, which consists of two rows, is given in Table 11.9. The optional output symbols associated with the unstable states have been specified in such a way that R will be fast in turning on and slow in turning off.

The assignment of $y = 0$ to the first row and $y = 1$ to the second row of the reduced flow table leads to the excitation and output tables of Table 11.10. The excitation and output functions are

$$\begin{aligned} Y &= (x_1 + x_2)y + x_1x_2, \\ G &= (x_1 + x_2)y' + x_1x_2, \\ R &= y + x_1x_2. \end{aligned}$$

Races and cycles

In Section 11.1, we discussed the difficulties that may arise as a result of the different delays associated with the various gates if multiple input changes are allowed. The same difficulties may arise if two or more secondary variables are required to change their values simultaneously. For practical reasons, it

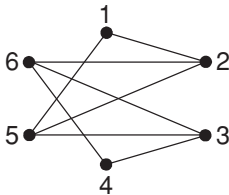


Fig. 11.11 Merger graph for the flow table of Table 11.8b.

Table 11.11 Illustration of races and cycles

y_1y_2	Y_1Y_2			
	x_1x_2			
	00	01	11	10
00	11	00	10	01
01	11	00	11	01
11	11	00	10	11
10	11	10	10	11

is clearly impossible to guarantee that all secondary elements indeed have precisely the same delays. As a result, the assignment of secondary variables to the rows of a reduced flow table must be such that the circuit will operate correctly even if different delays are associated with the various secondary elements.

A reduced excitation table is shown in Table 11.11. When both input values are 0 and $y_1y_2 = 00$, the required transition to the state $y_1y_2 = 11$ involves a change in the values of two secondary variables. If these two changes occur simultaneously, the transition specified in the table will actually take place. However, if either y_1 or y_2 changes first then, instead of going directly to the secondary state 11, the circuit will go to either state 01 or state 10. Fortunately, since in either case the required transition is to state 11, as indicated by the entries 11 in rows 01 and 10, column 00, the circuit will finally reach its destination. Such a situation, where a change in more than one secondary variable is required, is called a *race*. If the final state reached by the circuit does not depend on the order in which the variables change, as is the case discussed above, then the race is said to be a *noncritical race*.

Now suppose that the circuit is in the state $y_1y_2 = 11$ and that the input state is $x_1x_2 = 01$. The required transition is to the state $y_1y_2 = 00$. If y_1 changes faster than y_2 then the circuit will go to state 01, from which it will reach state 00, as indicated by entry 00 in row 01, column 01. However, if y_2 changes faster than y_1 then the circuit will go to the state $y_1y_2 = 10$ and remain there, since the total state $x_1x_2 = 01$, $y_1y_2 = 10$ is a stable state. Thus, the circuit operation will be incorrect. Such a situation, where the final stable state reached by the circuit depends on the order in which the internal variables change, is referred to as a *critical race* and must always be avoided.

Races can sometimes be avoided by directing the circuit through intermediate unstable states, before it reaches its final destination. When the circuit of Table 11.11 is in the secondary state $y_1y_2 = 01$ and the input state $x_1x_2 = 11$, the required transition is to state 10. However, since such a transition, from 01 to 10, involves two simultaneous changes in the y s, the unstable state 11 is entered in row 01, column 11, thereby directing the circuit to row 11, from

Table 11.12 A valid assignment for the flow table of Table 11.11

$y_1 y_2$	$Y_1 Y_2$			
	$x_1 x_2$	01	11	10
00	10	00	10	01
01	10	00	11	01
10	10	00	11	10
11	10	11	11	10

which it is directed to go to **10**. Such a situation, where a circuit goes through a *unique* sequence of unstable states, is called a *cycle*. When a state assignment is made such that it introduces cycles, care must be taken to ensure that each cycle terminates on a stable state. If a cycle does not contain a stable state then the circuit will go from one unstable state to another, until the inputs are changed. Obviously, such a situation must always be avoided when designing asynchronous circuits.

To eliminate the critical race in column 01, it is necessary to select another secondary assignment such that all critical transitions involve single variable changes. This can be accomplished by the assignment shown in Table 11.12. It is, of course, necessary to check that no new critical races have been introduced by this assignment. Having verified this, we can proceed to realize the flow table.

An assignment that contains no critical races or undesired cycles is referred to as a *valid* assignment. As we shall subsequently see, in many situations a valid assignment cannot be obtained merely by interchanging the assignments of several states in an invalid assignment; more sophisticated methods must be used.

Methods of secondary assignment

We now propose methods for obtaining secondary-state assignments such that each transition is accomplished either by a change of secondary state in which only one secondary variable changes or by a change of secondary state in which a multiple change of secondary variables does not result in a critical race. One way of arriving at the desired result is to test each transition and to ensure that the assignment of rows containing a lightface entry i will be adjacent to the assignment of the row containing the boldface entry $\mathbf{i}$. Subsequently, we shall refer to states that differ in only one variable as *adjacent* states.

The flow table of Table 11.13 contains three rows, denoted a , b , and c . Inspection of column 00 in the table reveals that the assignment of row a must be adjacent to that of row b , such that the transition from unstable state 1 to the

Table 11.13 A flow table

PS	State			
	x_1x_2			
	00	01	11	10
<i>a</i>	1	3	4	6
<i>b</i>	1	3	5	7
<i>c</i>	2	3	5	6

stable state **1** will involve just a single variable change. In a similar fashion, we arrive at the following required adjacencies for race-free operation:³

- column 00 : row *b* must be adjacent to row *a*
- column 01 : rows *a* and *b* must be adjacent to row *c*
- column 11 : row *c* must be adjacent to row *b*
- column 10 : row *c* must be adjacent to row *a*

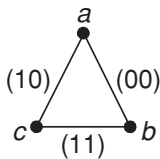


Fig. 11.12 Transition diagram for the flow table of Table 11.13.

These required adjacencies can be demonstrated by the diagram shown in Fig. 11.12, where each row is represented by a vertex and, for each pair of adjacent rows, an arc is drawn between the corresponding vertices. The arc labels (in parentheses) indicate the columns of the flow table in which the transitions are required. Such a diagram is known as a *transition diagram*. The problem now is to assign secondary states to the vertices of the transition diagram, such that each pair of adjacent vertices is assigned a pair of adjacent secondary states.

If row *a* of Table 11.13 is assigned a combination of values of state variables with an even number of 1's, say 00, row *b* must contain an odd number of 1's, say 01. Now, for row *c* to be adjacent to both rows *a* and *b*, it must contain an odd number of 1's *and* an even number of 1's, which obviously cannot be achieved. To overcome this difficulty, it is necessary to augment the flow table either by assigning two secondary states to row *c* or by introducing cycles that lead the circuit to the desired stable states. These possibilities are illustrated in Tables 11.14*a, b*. In the first case, each transition to state *c* (see below) is directed to the adjacent one, as illustrated in column 01. In the second case, an entry in row 10 is used as an intermediate unstable state to direct the circuit to the desired stable state.

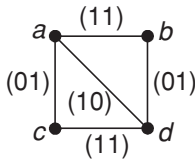


Fig. 11.13 Transition diagram for the flow table of Table 11.15.

Here, the use of a fourth row does not increase the number of secondary variables. In other situations, however, the augmentation of a flow table may involve such an increase. To examine this problem in terms of a specific situation, consider the flow table in Table 11.15 and its transition diagram shown in Fig. 11.13. We observe that row *a* must be adjacent to three other rows, as must row *d*. Clearly there is no way of assigning four secondary states such that the

³ If noncritical races are permitted, as is usually the case, then column 01 requirement may be eliminated, since column 01 contains only one stable state.

Table 11.14 Augmented flow tables

	y_1y_2	Y_1Y_2			
		00	01	11	10
a	00	00	10	00	00
b	01	00	11	01	01
c	11	11	11	01	10
c	10	10	10	11	00

(a) Two assignments to row c

	y_1y_2	Y_1Y_2			
		00	01	11	10
a	00	00	01	00	00
b	01	00	11	01	01
c	11	11	11	01	10
	10	—	—	—	00

(b) Utilizing an unspecified entry as an unstable state

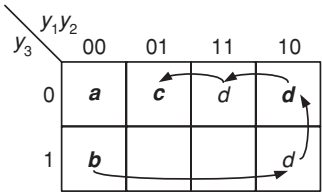


Fig. 11.14 Transition diagram.

Table 11.15 A flow table that requires three secondary variables

	State			
	x_1x_2	01	11	10
PS	00			
a	1	2	4	6
b	1	3	4	7
c	1	2	5	8
d	1	3	5	6

above adjacencies will be satisfied. Hence a third secondary variable must be added.

The eight combinations of three secondary variables are represented by the cells of the map of Fig. 11.14. To find a valid assignment, we start by placing a bold a in cell $y_1y_2y_3 = 000$ to indicate that row a will be assigned the secondary state 000. Similarly, we place b , c , and d in the three cells adjacent to cell a . This, however, means that each of the transitions from rows b to d and d to c requires two changes of secondary variables. These multiple changes can be accomplished by directing the circuit to its final destination through unstable states, as shown by the arrows in Fig. 11.14. The flow table resulting from this assignment is shown in Table 11.16.

11.4 Synthesis of burst-mode circuits

Since SIC fundamental-mode machines are quite restrictive, a straightforward generalization leads to *multiple-input-change (MIC) fundamental-mode* machines, in which several inputs can change values in a narrow time interval and no further inputs change values until the machine has stabilized. However, because of the narrow time interval allowed for all input value changes, MIC

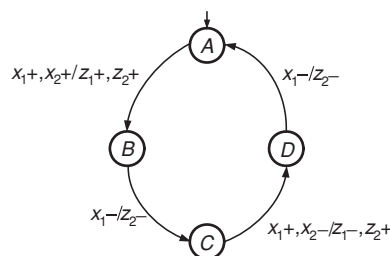


Fig. 11.15 A burst-mode specification.

Table 11.16 A race-free flow table

		State			
		x_1x_2	00	01	11
	$y_1y_2y_3$	00	01	11	10
<i>a</i>	000	1	2	4	6
<i>b</i>	001	1	3	4	7
	011				
<i>c</i>	010	1	2	5	8
	110			5	
	111				
	101		3		
<i>d</i>	100	1	3	5	6

fundamental-mode machines are still quite restrictive. A further generalization of such machines is *burst-mode machines*. Such machines also allow several inputs to change values concurrently. However, all the changes need not occur in a narrow time interval. They can change in any order at any time within a given *input burst* and respond with a set of output value changes called the *output burst*. This eases the timing constraints imposed on the environment in which the machine is placed.

Burst-mode specification

A burst-mode specification with two inputs, x_1 and x_2 , and two outputs, z_1 and z_2 , is shown in Fig. 11.15. The start state is A, as indicated. The initial values of the inputs and outputs can be specified or assumed to have a default value 0. A label is associated with each arc consisting of an input burst and an output burst separated by /. A rising (falling) transition is denoted by + (−).

The machine is initially stable in any given state. The rising or falling transitions associated with an input burst of an outgoing arc can arrive in any order and at any time. However, their change is monotonic. When the last input transition arrives, the burst is deemed complete. The machine then generates the corresponding output burst, if any, and moves to the specified next state. After the machine stabilizes, this process can begin anew.

There are three restrictions that a burst-mode specification must obey.

- *Nonempty input bursts* If no input undergoes a transition, the machine remains in its current state.
- *Maximal set property* No input burst on an outgoing arc from any state must be a subset of an input burst on another outgoing arc from the same state. Note that if such a subset were allowed, the machine would not know whether it should wait for another input transition.

Table 11.17 A flow table

	State, $z_1 z_2$			
	$x_1 x_2$			
PS	00	01	11	10
A	$A, 00$	$A, 00$	$B, 11$	$A, 00$
B	—	$C, 10$	$B, 11$	—
C	$C, 10$	$C, 10$	$C, 10$	$D, 01$
D	$A, 00$	—	—	$D, 01$

- *Unique entry point* Each state should have a unique set of input and output values through which it is entered. For example, in the specification shown in Fig. 11.15, let us assume that in starting state A , $x_1 x_2 = 00$ and $z_1 z_2 = 00$. Then we can check that the input/output values for states B , C , and D are 11/11, 01/10, 10/01, respectively. The arc from D to A takes these values back to 00/00, which is the unique entry point for A .

Flow table

In order to synthesize a circuit from a burst-mode specification, first it has to be translated into a flow table. For the specification shown in Fig. 11.15, the flow table is shown in Table 11.17. Each state in the specification is represented by a row in the flow table and each input combination by a column. Each entry in the table represents the *complete state* of the machine, which includes the state the machine goes to and the corresponding output values. Consider initial state A , which is mapped to row A where the complete state $A, 00$ is stable. The input burst $x_1 +, x_2 +$ on the outgoing arc from state A is also mapped to this row. This input burst leads to four possible temporary input combinations: no change, $x_1 +, x_2 +$, and $(x_1 +, x_2 +)$. The complete state remains the same until the input burst is complete, after which the state is specified as are the output values based on the output burst $z_1 +, z_2 +$, thus leading to the complete state $B, 11$. On the outgoing arc from state B to C the input burst is simply $x_1 -$. Thus, there are only two temporary input combinations in this case: no change and $x_1 -$. The latter yields the entry $C, 10$ in this row. This complete state incorporates the effect of the output burst $z_2 -$. The remaining two entries in this row cannot be reached and are hence left unspecified. A similar analysis applies to the other rows.

Flow table reduction and state assignment

The flow table for a burst-mode specification has no function hazards; this stems from the requirement that the complete state must not change until the full input burst has arrived. Also, it is always possible to obtain a hazard-free

sum-of-products realization H for each secondary variable and output. This follows from the fact that, for each such variable, the required cube can be included in some product of H and no product of H illegally intersects any privileged cube. The latter is true because all transitions in any row of the flow table have the same complete start state, which will be included in the required cubes for these transitions.

It is possible to minimize the number of states in a flow table through state merging. However, even when two states are compatible it may sometimes be incorrect to merge them since it may no longer be possible to guarantee a hazard-free realization of all secondary and output variables. The conditions under which state merging is possible are given in [13]. However, for the rest of the discussion, we will assume that no state merging is done.

Various methods are available for obtaining a critical race-free secondary state assignment for the flow table. One way is use the transition diagrams discussed earlier.

Example Consider the burst-mode specification in Fig. 11.15. Its transition diagram and a possible state assignment are shown in Fig. 11.16.

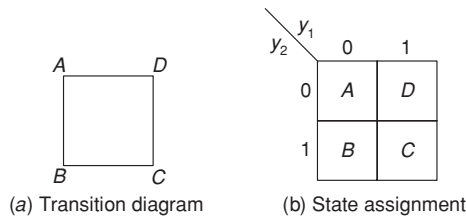
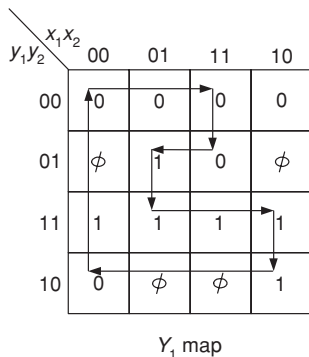


Fig. 11.16 A critical race-free state assignment.

A synthesis example

The excitation and output table is the starting point for further synthesis. As discussed earlier, we need to identify next the required cubes and dhf-prime implicants for each next-state and output variable and obtain the minimal sum-of-products expressions based on the subset of the dhf-prime implicants that covers all the required cubes.

Continuing with the state assignment in Fig. 11.16, consider its excitation and output table, shown in Table 11.18. For Y_1 , Y_2 , z_1 , and z_2 , the maps with the relevant transitions as well as the dhf-prime implicant charts are shown in Fig. 11.17. The horizontal transitions shown in the maps correspond to the input burst and the vertical transitions to the change in state. For example, the input burst $x_1 + x_2 +$ in the specification shown in Fig. 11.15 takes the machine from

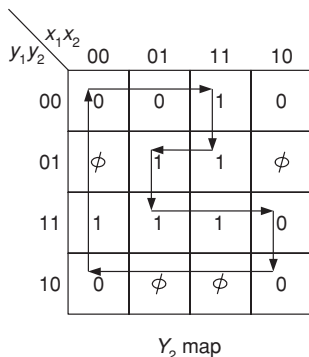


$\Rightarrow$

dhf-prime implicants	Required cubes		
	$x'_1x_2y_2$	y_1y_2	$x_1x'_2y_1$
x'_1y_2	×		
x'_2y_2			
y_1y_2		×	
x_1y_1			×
x_2y_1			

Minimal hazard-free sum-of-products

$$Y_1 = x'_1y_2 + y_1y_2 + x_1y_1$$

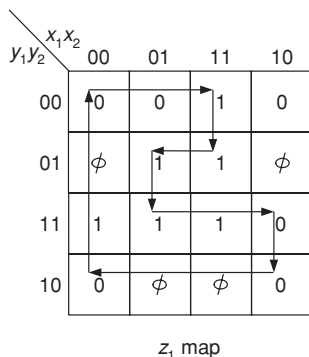


$\Rightarrow$

dhf-prime implicants	Required cubes				
	$x_1x_2y'_1$	$x_2y'_1y_2$	$x'_1x_2y_2$	$x_2y_1y_2$	$x'_1y_1y_2$
$x_1x_2y'_1$	×				
y'_1y_2		×			
x'_1y_2			×		×
x_2y_2		×	×	×	
x_2y_1				×	

Minimal hazard-free sum-of-products

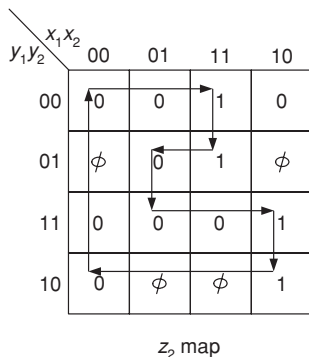
$$Y_2 = x_1x_2y'_1 + x'_1y_2 + x_2y_2$$



$\Rightarrow$

Minimal hazard-free sum-of-products

$$z_1 = Y_2 = x_1x_2y'_1 + x'_1y_2 + x_2y_2$$



$\Rightarrow$

dhf-prime implicants	Required cubes	
	$x_1x_2y'_1$	$x_1x'_2y_1$
$x_1x_2y'_1$	×	
$x_1y'_1y_2$		
$x_1x'_2y_1$		×
$x_1y_1y'_2$		
$x_1x_2y'_2$		

Minimal hazard-free sum-of-products

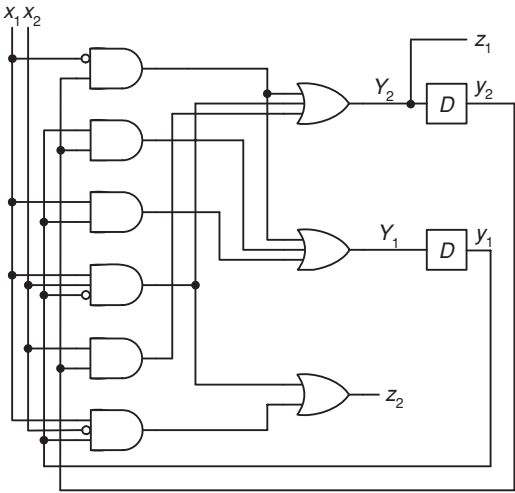
$$z_2 = x_1x_2y'_1 + x_1x'_2y_1$$

Fig. 11.17 Synthesis from a burst-mode specification.

Table 11.18 Excitation and output table

y_1y_2	Y_1Y_2, z_1z_2			
	x_1x_2 00	01	11	10
00	00, 00	00, 00	01, 11	00, 00
01	—	11, 10	01, 11	—
11	11, 10	11, 10	11, 10	10, 01
10	00, 00	—	—	10, 01

Fig. 11.18 Synthesized circuit.



state A to B . This corresponds to a horizontal transition from $(x_1, x_2, y_1, y_2) = 0000$ to 1100 , followed by a vertical transition from 1100 to 1101 (note that A 's assignment is 00 whereas B 's assignment is 01). Some dhf-prime implicants are not needed for any required cube, with the result that the corresponding row is blank in the dhf-prime implicant chart. The minimal hazard-free sum-of-products expressions are also shown in Fig. 11.17. The corresponding circuit is shown in Fig. 11.18.

Notes and references

The first systematic treatment of asynchronous sequential circuits was due to Huffman [7], whose model for fundamental-mode circuits was presented in this chapter. McCluskey [10] also studied fundamental-mode circuits. Huffman [6] and McCluskey [9] were also the main initial contributors to hazard analysis and hazard-free circuit

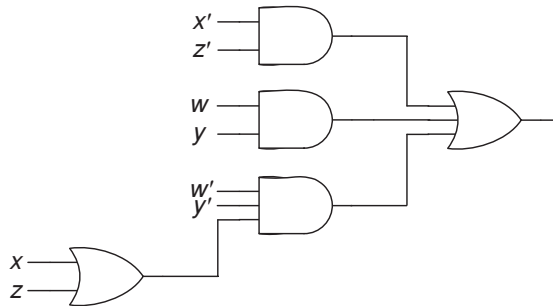
design. Eichelberger [4] dealt with MIC logic hazards. Beister [1] showed how to get rid of MIC dynamic logic hazards. Nowick and Dill [13] presented an exact two-level minimization algorithm for obtaining hazard-free circuits. Unger [15], Bredeson [2], and Kung [8] presented hazard-nonincreasing logic transformations. Huffman [5] studied the secondary-assignment problem for asynchronous circuits and proposed several race-free universal assignments. Unger [14] pointed out the existence of inherent hazards within fundamental-mode circuits and showed how to eliminate such hazards by inserting delays. Good presentations of asynchronous circuits are available in Miller [11] and Unger [15]. The survey article by Davis and Nowick [3] and the book by Myers [12] provide excellent further reading material for interested readers.

- [1] Beister, J.: "A unified approach to combinational hazards," *IEEE Trans. Computers*, vol. C-23, no. 6, pp. 566–575, June 1974.
- [2] Bredeson, J. G.: "Synthesis of multiple-input change hazard-free combinational switching circuits without feedback," *Int. J. Electronics (GB)*, vol. 39, no. 6, pp. 615–624, December 1975.
- [3] Davis, A., and S. M. Nowick: "An introduction to asynchronous circuit design," University of Utah Technical Report, Department of Computer Science, UUCS-97-013, September 1997.
- [4] Eichelberger, E. B.: "Hazard detection in combinational and sequential switching circuits," *IBM J. Research & Development*, vol. 9, pp. 90–99, 1965.
- [5] Huffman, D. A.: "A study of the memory requirements of sequential switching circuits," *MIT Res. Lab. Electron. Technical Report 293*, April 1955.
- [6] Huffman, D. A.: "The design and use of hazard-free switching networks," *J. Assoc. Computing Machinery*, vol. 4, pp. 47–62, January 1957.
- [7] Huffman, D. A.: "The synthesis of sequential switching circuits," *J. Franklin Inst.*, vol. 257, pp. 275–303, March–April 1954.
- [8] Kung, D. S.: "Hazard-nonincreasing gate-level optimization algorithms," in *Proc. In. Conf. Computer-Aided Design*, pp. 631–634, November 1992.
- [9] McCluskey, E. J.: "Transient in combinational logic circuits," in *Redundancy Techniques for Computing Systems*, pp. 9–46, Spartan, Washington, DC, 1962.
- [10] McCluskey, E. J.: "Fundamental and pulse mode sequential circuits," in *Proc. IFIP Congress 1962*, North Holland, Amsterdam, 1963.
- [11] Miller, R. E.: *Switching Theory*, vol. 2, John Wiley & Sons, New York, 1965.
- [12] Myers, C. J.: *Asynchronous Circuit Design*, John Wiley & Sons, New York, July 2001.
- [13] Nowick, S. M., and D. L. Dill: "Exact two-level minimization of hazard-free logic with multiple-input changes," *IEEE Trans. Computer-Aided Design*, vol. 14, no. 8, pp. 986–997, August 1995.
- [14] Unger, S. H.: "Hazards and delays in asynchronous sequential switching circuits," *IRE Trans. Circuit Theory*, vol. CT-6, no. 12, 1959.
- [15] Unger, S. H.: *Asynchronous Sequential Switching Circuits*, John Wiley & Sons, New York, 1969.

Problems

Problem 11.1. Analyze the circuit in Fig. P11.1 for SIC static hazards. Redesign it to make it SIC hazard-free.

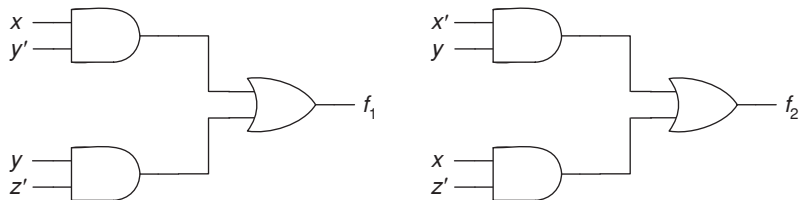
Fig. P11.1



Problem 11.2. Consider the two-output circuit shown in Fig. P11.2. Without inserting any extra gates in it, make both outputs SIC hazard-free.

Hint: You are allowed to add connections to the circuit.

Fig. P11.2

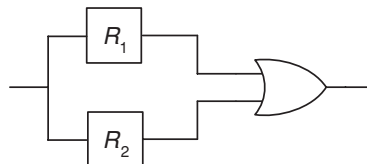


Problem 11.3

- If two AND–OR two-level circuits are SIC hazard-free, is the single-output circuit obtained by performing an OR of the two outputs guaranteed to be SIC hazard-free? Either prove this or provide a counter-example.
- Conversely, if two AND–OR two-level circuits each have an SIC hazard, is the single-output circuit obtained by, performing an OR of the two outputs guaranteed to have an SIC hazard? Either prove this or provide a counter-example.

Problem 11.4. Two different realizations, R_1 and R_2 , of a function F are fed to an OR gate, as shown in Fig. P11.4. If both R_1 and R_2 are SIC hazard-free, is the overall circuit guaranteed to be SIC hazard-free? Explain your reasoning.

Fig. P11.4

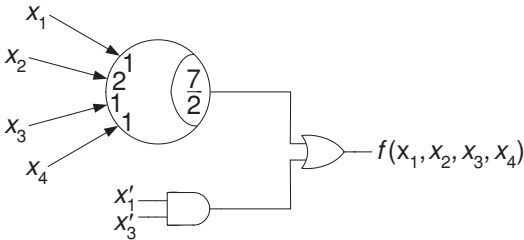


Problem 11.5

- Find all SIC static hazards in the circuit shown in Fig. P11.5. (Assume the individual elements to be hazard-free.)

- (b) Changing *only* the parameters of the threshold element, redesign the circuit in such a way that all SIC static hazards are eliminated.

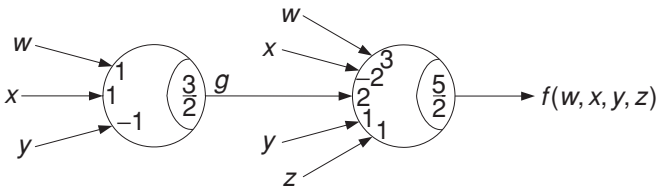
Fig. P11.5



Problem 11.6. For the network shown in Fig. P11.6:

- (a) show a map for $f(w, x, y, z)$;
(b) find all SIC hazards of the network;
(c) realize f with a single threshold element.

Fig. P11.6



Problem 11.7. In the function $f(x, y, z) = \sum(1, 3, 4, 5, 6, 7)$:

- (a) find all MIC transitions that have a function hazard;
(b) find the required cubes for the MIC transition $111 \rightarrow 010$. What is the privileged cube for this transition?
(c) find the required cubes for MIC transition $111 \rightarrow 000$.

Problem 11.8. For the function $f(w, x, y, z) = \sum(0, 1, 2, 3, 5, 6, 7, 8, 9, 12, 13, 15)$ and the transitions $0001 \rightarrow 0100$, $0110 \rightarrow 0011$, $1101 \rightarrow 1010$, and $1011 \rightarrow 1010$:

- (a) find all the dhf-prime implicants;
(b) find a hazard-free sum-of-products expression.

Problem 11.9. From the excitation and output tables, in Table P11.9, for an SIC fundamental-mode asynchronous sequential circuit, determine which input sequences result in a 1 output value.

Table P11.9

$y_1 y_2$	$Y_1 Y_2, z$			
	$x_1 x_2$			
	00	01	11	10
00	00 , 0	10, 0	01, 0	00 , 0
01	00, 0	11, 0	01 , 1	11, 0
11	00, 0	11 , 0	10, 0	11 , 0
10	00, 0	10 , 0	10 , 0	11, 0

Problem 11.10. Each of the following specifications describes an SIC fundamental-mode sequential circuit with two inputs, x_1 and x_2 , and one output, z . Show a primitive and a reduced flow table for each circuit.

- The output $z = 1$ if both x_1 and x_2 are at 1 and the value of x_1 becomes 1 before that of x_2 .
- When $x_2 = 1$, the value of the output z is equal to the value of x_1 ; when $x_2 = 0$, the output remains fixed at its last value prior to when the value of x_2 became 0.
- The value of the output z is equal to 0 whenever $x_1 = 0$. The first change in the value of input x_2 occurring while $x_1 = 1$ causes the value of z to become 1. Thereafter, the value of z remains at 1 until the value of x_1 returns to 0.

Problem 11.11. Give a minimum-row reduced-flow-table description of an SIC fundamental-mode two-input (x_1, x_2), one-output (z) sequential circuit that operates in the following manner: the output $z = 1$ if and only if the input state $x_1 = x_2 = 1$ and the next-to-last input variable change was a change in the value of x_1 . Assume that the circuit is initially in the input state $x_1 = x_2 = 0$. Is the reduced flow table unique?

Problem 11.12. The value of the output z of an SIC fundamental-mode two-input sequential circuit is to change from 0 to 1 only when the value of x_2 changes from 0 to 1 while $x_1 = 1$. The output value is to change from 1 to 0 only when the value of x_1 changes from 1 to 0 while $x_2 = 1$.

- Find a minimum-row reduced flow table. The output should be fast and flicker-free.
- Show a valid assignment and write a set of (*static*) *hazard-free* excitation and output equations.

Problem 11.13. An SIC fundamental-mode sequential circuit with two inputs, x_1 and x_2 , and two outputs, z_1 and z_2 , is to be designed so that z_i (for $i = 1, 2$) takes on the value 1 if and only if x_i was the input whose value changed last.

- Find a minimum-row reduced flow table and a valid assignment.
- Assuming that all inputs are available in an uncomplemented as well as a complemented form, show a realization using NAND gates. (fourteen gates are sufficient.)

Problem 11.14. Design an SIC fundamental-mode asynchronous sequential circuit with two inputs, x_1 and x_2 , and two outputs, G and R , which is to operate in the following manner. Initially, both input values and both output values are equal to 0. The first input to assume the value 1, either x_1 or x_2 , turns G “on” (i.e., sets G to 1). With the first input value equal to 1, if the second input value becomes equal to 1 then R turns on. Thereafter, as long as either input value remains equal to 1, the input that first caused G to turn on controls the operation of G , i.e., it causes G to turn off when it assumes the value 0 and to turn on again when it assumes the value 1. The second input controls the operation of R in the same manner.

- Show a minimum-row reduced flow table and find a valid assignment.
- Find the excitation and output equations.

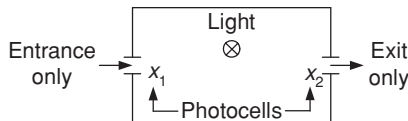
Problem 11.15. At a junction of a single-track railroad and a road, traffic lights are to be installed. The lights are to be controlled by switches that are pressed or released by the trains. When a train approaches the junction from either direction and is within 1500 feet from it, the lights are to change from green to red and remain red until the train is 1500 feet past the junction.

- Write a primitive flow table and reduce it. You may assume that the length of a train is smaller than 3000 feet.
- Show a circuit realization of the light-control network.
- Repeat the design if it is known that the trains may be longer than 3000 feet.

Problem 11.16. Figure P11.16 illustrates an office for two students. Instead of light switches the room has two photocells, one at each door. If either or both students are in the office, the light is to be on. The students can enter or exit only as shown; entrances and exits never occur simultaneously. The photocells indicate a 1 when their beam is interrupted by a student entering or exiting and a 0 at all other times.

- Find a primitive and a minimum-row reduced flow table that describe the light-control operation.
- Show a valid assignment and find the excitation and output equations.
- Repeat (a) if entering and exiting the room simultaneously is allowed.

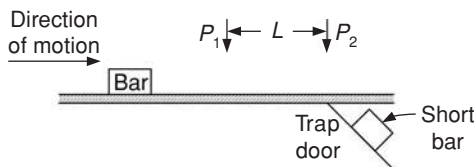
Fig. P11.16



Problem 11.17. A factory produces steel bars of length $L + \delta$ and $L - \delta$. It is required that the bars are to be sorted by placing them on a conveyor belt passing under two photocells, as shown in Fig. P11.17. The spacing between the bars on the belt is greater than δ . To the right of P_2 is a trap door through which short bars can drop. The trap door should not be open when the beam of P_2 is interrupted and should be open immediately after a short bar, of length $L - \delta$, has completely passed P_2 . Let the value of output x_i of P_i be 1 when the beam of P_i is interrupted. Let the value of the trap-door control z be 1 when the door is open.

- Find a minimum-row reduced flow table, with eight stable states, that describes the trap-door control operation.
- Show a valid assignment and find the logic equations for the memory elements and the trap-door control.

Fig. P11.17



Problem 11.18. A completely automatic and independent traffic-light system for the intersection of roads x and y consists of two sensors, some processing circuitry, and the lights. The sensors and circuitry generate two outputs, z and w . Output z attains the value 1 if and only if $m(x) - m(y) \geq 6$, where $m(x)$ indicates the number of cars waiting to cross a road y . Output w attains the value 1 if and only if $m(y) - m(x) \geq 6$. We wish to design an SIC fundamental-mode sequential circuit with inputs (z, w, z', w') and outputs (G_x, R_x, G_y, R_y) , where G and R refer to green and red lights, respectively, and the subscripts indicate the street from which the light is visible. The objective is to minimize intersection load by unloading whichever street is overloaded, i.e., has at

least six cars more than the other. The lights of the street being unloaded should remain green until the other street becomes overloaded.

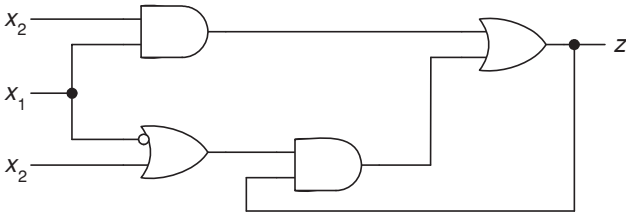
- (a) Show a primitive flow table.
- (b) Give a reduced flow table.
- (c) Show a circuit realization. The outputs are to be fast and flicker-free.

Problem 11.19. In the circuit of Fig. P11.19, the values of input variables x_1 and x_2 never change simultaneously.

- (a) Describe in words the terminal behavior of the circuit.
- (b) Derive the flow table for the circuit.
- (c) Show how one of the gates can be eliminated without changing the flow table. What physical problems might this cause, and how can they be prevented?

Hint: To derive the flow table, open the feedback loop.

Fig. P11.19



Problem 11.20. The reduced flow table of Table P11.20a is to be assigned three secondary variables, as shown in Table P11.20b. Note that several combinations of $y_1y_2y_3$ values have been assigned to the first two rows of the reduced table. Consequently the circuit will be stable when $x_1x_2 = 00$ in any of the $y_1y_2y_3$ combinations 000, 001, 011, for example, and each of these stable configurations must be equivalent to **1**. Complete an excitation table for the situation when each transition takes as short a time as possible. Is the excitation table unique?

Table P11.20

PS	State				$y_1y_2y_3$	$Y_1Y_2Y_3$			
	x_1x_2					x_1x_2			
	00	01	11	10		00	01	11	10
a	1	5	6	9	a	000			
b	1	4	7	8	a	001			
c	2	5	7	9	a	011			
d	3	4	6	9	b	010			
					b	100			
					b	101			
					c	111			
					d	110			

(a) Reduced flow table

(a) Reduced flow table

(b) Excitation table

Problem 11.21

- (a) Find all the races in the flow table of Table P11.21 and indicate those that are critical and those that are not.
- (b) Find another assignment that contains no critical races.

Table P11.21

		State			
		x_1x_2			
y_1y_2		00	01	11	10
00	00		11	00	11
01	11	01		11	11
10	00	10		11	11
11	11	11	00		11

Problem 11.22. For each of the reduced flow tables in Table P11.22, find an assignment that contains no critical races and requires a minimum of secondary variables.

Table P11.22

		State			
		x_1x_2			
		00	01	11	10
1	3	5		7	
2	3	6		7	
1	4	6		7	

(a)

		State			
		x_1x_2			
		00	01	11	10
1	3	6	7		
1	3	5	7		
2	4	5	7		
2	4	6	7		

(b)

		State			
		x_1x_2			
		00	01	11	10
1	3	5		7	
1	4	6		8	
2	3	6		7	
2	4	5		8	

(c)

		State			
		x_1x_2			
		00	01	11	10
1	5	7	10		
2	4	8	10		
3	5	9	12		
2	5	9	11		
3	4	7	11		
1	6	8	12		

(d)

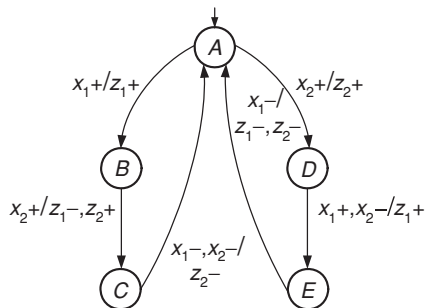
		State			
		x_1x_2			
		00	01	11	10
1	4	7	10		
1	5	8	11		
2	6	8	10		
2	4	9	11		
3	6	9	10		
3	5	7	11		

(e)

Problem 11.23. Consider the burst-mode specification shown in Fig. P11.23.

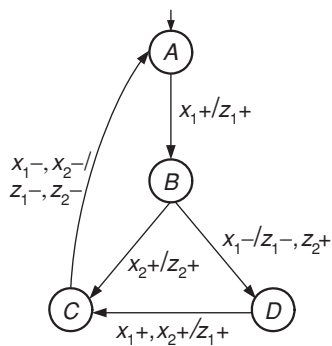
- Assuming that the unique entry point for state A is $00/00$, what are the entry points for each of the other four states?
- Obtain a flow table from the specification.
- Find a secondary state assignment that is free of critical races.
- Obtain an excitation and output table based on the above state assignment.
- Synthesize a minimal two-level hazard-free circuit.

Fig. P11.23



Problem 11.24. Repeat Problem 11.23 for the burst-mode specification shown in Fig. P11.24.

Fig. P11.24



12

Structure of sequential machines

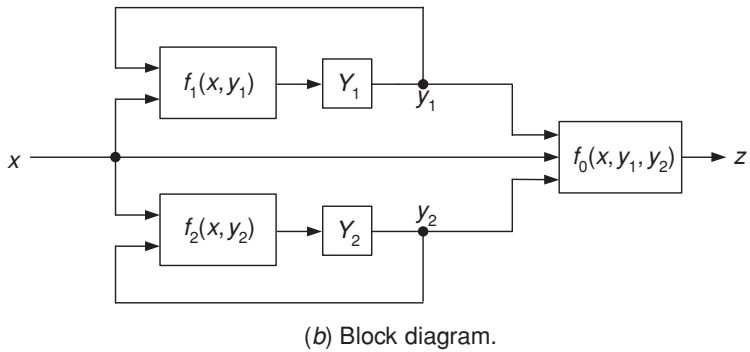
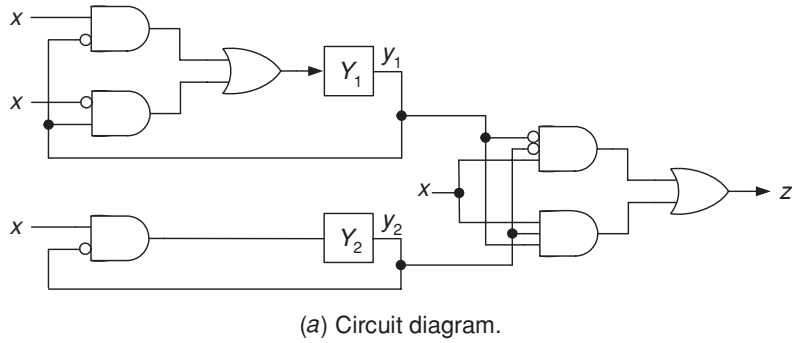
One of the main problems in the synthesis of sequential machines is that of assigning combinations of state-variable values to the states of the machine. This assignment determines the complexity and structure of the circuit which realizes the machine. Various restrictions and requirements may be imposed on the state assignment, depending on the design objectives and intended use of the circuit. It may be desirable, for example, to construct it using a minimum amount of logic, or to build it from an interconnection of smaller circuits, and so on. The *structure* of a sequential machine includes the manner in which a machine can be realized from a set of smaller component machines as well as the functional dependencies of its state and output variables. It is our aim in this chapter to study the state-assignment problem and how it affects the structure and complexity of sequential machines.

12.1 Introductory example

The close relationship between the state-assignment problem and the structure of sequential machines will be demonstrated by means of the machine M_1 shown in Table 12.1. Two possible state assignments for M_1 are shown in Table 12.2. The logic equations corresponding to assignment α , which are derived from the excitation and output tables, are

$$\begin{aligned}Y_1 &= x'y_1 + xy_1' = f_1(x, y_1), \\Y_2 &= x'y_1 + xy_2 = f_2(x, y_1, y_2), \\z &= xy_2' = f_0(x, y_2).\end{aligned}$$

From these equations, it is evident that Y_1 is a function of y_1 and of the external input and is independent of y_2 . However, Y_2 depends on the external input as well as y_1 and y_2 . The output z is a function of x and y_2 only. The circuit diagram of M_1 is shown in Fig. 12.1a. The dependency of the next-state variables and the output is illustrated by the block diagram of Fig. 12.1b, where,

Fig. 12.2 Second realization of M_1 .

for example, the block labeled $f_1(x, y_1)$ corresponds to the combinational logic associated with memory element Y_1 , and so on.

The logic equations corresponding to assignment β , shown in Table 12.2b, are

$$\begin{aligned} Y_1 &= x'y_1 + xy_1' = f_1(x, y_1), \\ Y_2 &= xy_2' = f_2(x, y_2), \\ z &= xy_1'y_2' + xy_1y_2 = f_0(x, y_1, y_2). \end{aligned}$$

In this case Y_1 is independent of y_2 and Y_2 is independent of y_1 . In other words, the next value of each state variable can be computed from its present value and the value of the present input, regardless of the value of the other state variable. The dependency of the output function, however, has increased in comparison with its dependency in assignment α , shown in Table 12.2a. The circuit and block diagrams corresponding to assignment β are shown in Fig. 12.2.

The preceding two realizations of machine M_1 clearly demonstrate that the choice of assignment affects the complexity of the circuit and determines the dependency of the next-state variables and the overall structure of the machine. Our objective in this chapter is to investigate the relationship of the

state assignment and the reduction in dependency of the state variables to the structure of a sequential machine. These factors will be shown to affect the complexity and cost of the final circuits as well.

12.2 State assignments using partitions

In this section we shall derive necessary and sufficient conditions for a sequential machine M to have assignments that result in reduced dependencies among the state variables. Such assignments generally yield simpler logic equations and circuits; they are also the fundamental means by which machine decompositions are obtained.

Closed partitions

Let machine M have a set of n states $S = \{S_1, S_2, \dots, S_n\}$ and a set of p input symbols $I = \{I_1, I_2, \dots, I_p\}$; then $k = \lceil \log_2 n \rceil$ state variables and $l = \lceil \log_2 p \rceil$ input variables are needed for a complete assignment, where $\lceil g \rceil$ is defined as the smallest integer equal to or greater than g . Each of the k next-state variables depends, in general, on the external inputs $x_1, x_2, \dots, x_l$ and the k state variables, i.e.,

$$Y_i = f_i(y_1, y_2, \dots, y_k, x_1, x_2, \dots, x_l), \quad i = 1, 2, \dots, k.$$

Our objective is to obtain assignments in which the values of one or more subsets of the next-state variables can be determined independently of the values of the remaining variables, that is, assignments which yield logic equations for the variables $Y_1, Y_2, \dots, Y_r$, where $1 \leq r < k$, that are independent of the remaining $k - r$ variables. Thus,

$$Y_i = f_i(y_1, y_2, \dots, y_r, x_1, x_2, \dots, x_l), \quad i = 1, 2, \dots, r.$$

The subset $\{Y_1, Y_2, \dots, Y_r\}$ of state variables, whose values are independent of the values of $y_{r+1}, y_{r+2}, \dots, y_k$, is said to be a *self-dependent* subset, and an assignment that yields such a subset is said to possess self-dependent subsets. Assignments α and β of machine M_1 both have this property.

The state-assignment problem may be viewed as either a coding problem or a partitioning problem. In viewing the state assignment as a coding problem, a distinct code is assigned to each row (state) of the state table. From the partitioning point of view, which we shall adopt in this chapter, each state variable y_i induces a partition τ_i on the set of states of the machine, such that two states are in the same block of τ_i if and only if they are assigned the same value of y_i . For example, in assignment α for machine M_1 , $y_1 = 0$ for states A and B and $y_1 = 1$ for states C and D . Hence y_1 induces the partition $\tau_1 = \{\overline{A, B}; \overline{C, D}\}$ (see Definition 2.1) on the states of M_1 . Similarly, y_2 induces the partition $\tau_2 = \{\overline{A, D}; \overline{B, C}\}$. Clearly, if the assignment is such that each

state has a unique code then the product of the k partitions $\tau_1, \tau_2, \dots, \tau_k$ corresponding to $y_1, y_2, \dots, y_k$ is equal to zero, that is,

$$\tau_1 \cdot \tau_2 \cdot \dots \cdot \tau_k = \pi(0).$$

We have shown how an assignment induces a set of partitions whose product is the zero partition $\pi(0)$. The inverse process, that of assigning the values of the state variables to distinguish the blocks of a set of partitions, is the process of significance in the synthesis procedure. Given a partition τ with $\#(\tau)$ blocks on the set of states of M , to distinguish between these blocks it is necessary to select $r = \lceil \log_2 \#(\tau) \rceil$ state variables and assign a distinct combination of these variables to each block of τ ; that is, all the states in each block are assigned the same values of $y_1, y_2, \dots, y_r$. Each partition on the states of M provides some information regarding M 's state. If M possesses two partitions τ_1 and τ_2 such that $\tau_1 > \tau_2$ then τ_2 provides more information than τ_1 . Clearly, the zero partition provides all the necessary information, since knowledge of which block of $\pi(0)$ the machine is in is sufficient to determine the state of M uniquely. Thus, to obtain an assignment for M such that each state has a distinct code, it is necessary to assign the values of the state variables in such a way that they distinguish between the blocks of a set of partitions whose product is the zero partition.

Example For machine M_1 , the product of the partitions $\tau_1 = \{\overline{A}, \overline{B}; \overline{C}, \overline{D}\}$ and $\tau_2 = \{\overline{A}, \overline{C}; \overline{B}, \overline{D}\}$ is zero, i.e., $\tau_1 \cdot \tau_2 = \pi(0)$. Hence, if we assign y_1 in such a way as to distinguish block (A, B) from block (C, D) , and y_2 in such a way as to distinguish the blocks of τ_2 , then each state of M_1 will have a distinct code. One such assignment is β , shown in Table 12.2b.

Definition 12.1 A partition π on the set of states of a sequential machine M is said to be *closed* if, for every two states S_i and S_j which are in the same block of π and any input symbol I_k in I , the states $I_k S_i$ and $I_k S_j$ are in a common block of π ; $I_k S_i$ denotes the I_k -successor of S_i .

Example For machine M_1 , Table 12.1, the partitions $\pi_1 = \{\overline{A}, \overline{B}; \overline{C}, \overline{D}\}$ and $\pi_2 = \{\overline{A}, \overline{C}; \overline{B}, \overline{D}\}$ are closed.¹ The 0- and 1-successors of (A, C) are (A, C) and (B, D) , respectively, while the only successor of (B, D) is (A, C) . If we denote the blocks of π_2 , (A, C) and (B, D) , by P and Q respectively then we may describe the successor relationships of these blocks by means of the graph of Fig. 12.3. Clearly, knowledge of the present block of M_1 and the input symbol is sufficient to determine the next block

¹ In general, we shall reserve π to denote closed partitions while τ, θ , etc., will denote arbitrary partitions.

uniquely. (We shall subsequently say that a machine is in a block when we mean that it is in one of the states contained in the block.)

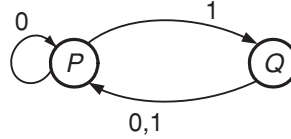


Fig. 12.3 Successor relationships of the blocks of the partition $\pi_2 = \{\overline{A}, \overline{C}; \overline{B}, \overline{D}\} = \{\overline{P}; \overline{Q}\}$.

Reduction of the functional dependency of the state variables

We shall now establish the relationship between closed partitions and the reduction in functional dependency of state variables.

Theorem 12.1 *Let M be a sequential machine with k state variables, $y_1, y_2, \dots, y_k$. If there exists a closed partition π on the states of M and if r state variables, where $r = \lceil \log_2 \#(\pi) \rceil$, are assigned to the blocks of π , such that all the states contained in each block are assigned the same values of $y_1, y_2, \dots, y_r$, then the next-state variables $Y_1, Y_2, \dots, Y_r$ are independent of the remaining $k - r$ variables. Conversely, if the first r next-state variables, $Y_1, Y_2, \dots, Y_r$ ($1 \leq r < k$), can be determined from the values of the inputs and the first r state variables, independently of the values of the remaining $k - r$ variables, then there exists a closed partition π on the states of M such that two states, S_i and S_j , are in the same block of π if and only if they are assigned the same values of the first r variables.*

Proof Since each block of π is assigned the same values of the variables $y_1, y_2, \dots, y_r$, and since π is closed, knowledge of the present block of π and the present input values is sufficient to determine the next block of π . In other words, knowledge of the present values of $y_1, y_2, \dots, y_r$ and of the present input values is sufficient to determine the values of $Y_1, Y_2, \dots, Y_r$, regardless of the values of the remaining variables. To prove the converse, form a partition π on the states of M such that all the states with the same assigned values of $y_1, y_2, \dots, y_r$ are in the same block of π . To prove that π is closed, consider two states S_i and S_j that belong to the same block of π . Each of these states has the same assigned values of the first r variables and, since these variables are independent of the values of the remaining ones, an application of the same input sequence to both S_i and S_j causes the same change in the values of the first r variables for these two states. Therefore, for each value of I_k , the successors $I_k S_i$ and $I_k S_j$ have the same assignment of values for the first r variables and, consequently, are contained in the same block of π . Thus, π is closed. $\diamond$

Example For machine M_1 , the partitions $\pi_1 = \{\overline{A, B}; \overline{C, D}\}$ and $\pi_2 = \{\overline{A, C}; \overline{B, D}\}$ are closed. Since y_1 in assignment β has been assigned to distinguish the blocks of π_1 , it is independent of y_2 . Similarly, since y_2 has been assigned to distinguish the blocks of π_2 , it is independent of y_1 .

Theorem 12.1 actually states a necessary and sufficient condition for the decomposition of sequential machines. The existence of a partition τ and a closed partition π on the set of states of a machine M , such that $\pi \cdot \tau = \pi(0)$ guarantees that M can be composed of two component machines connected in *series*. The first component in the connection consists of $\lceil \log_2 \#(\pi) \rceil$ memory elements (and their excitation circuitry), corresponding to the state variables assigned to distinguish the blocks of π . Since these variables are independent of the remaining variables, the first component is often referred to as the *independent* component. The second component in the serial connection, also referred to as the *dependent* component, contains $\lceil \log_2 \#(\tau) \rceil$ memory elements, corresponding to the state variables assigned to distinguish the blocks of τ . We shall refer to the independent component as the *predecessor* machine and the dependent component as the *successor* machine. It is often convenient to view the predecessor machine as the component that distinguishes between the blocks of π , and the successor machine as the component that distinguishes between the states within the blocks of π .

The existence of two closed partitions on the states of M such that their product is zero, i.e., $\pi_1 \cdot \pi_2 = \pi(0)$, implies that M can be composed of two components operating in *parallel*, independently of each other. One component consists of $\lceil \log_2 \#(\pi_1) \rceil$ memory elements, corresponding to the variables assigned to distinguish the blocks of π_1 . The second component consists of $\lceil \log_2 \#(\pi_2) \rceil$ memory elements, corresponding to the variables assigned to distinguish the blocks of π_2 .

The preceding arguments can thus be summarized as follows.

- An n -state machine M can be decomposed into two independent components operating in parallel if and only if there exist two nontrivial closed partitions π_1 and π_2 on the states of M such that $\pi_1 \cdot \pi_2 = \pi(0)$. This decomposition requires a minimal number (i.e., $\lceil \log_2 n \rceil$) of state variables if and only if

$$\lceil \log_2 \#(\pi_1) \rceil + \lceil \log_2 \#(\pi_2) \rceil = \lceil \log_2 n \rceil.$$

Example Consider the machine M_2 given in Table 12.3. It can be shown that M_2 has seven closed partitions, which are listed in Fig. 12.4. Since M_2 has eight states, three state variables are needed for an assignment. The existence of the closed partition π_5 suggests that M_2 can be realized as two component machines connected in series. The predecessor component has two state variables, y_1 and y_2 , which are assigned to the blocks of π_5 and,

Table 12.3 Machine M_2

<i>PS</i>	<i>NS</i>		<i>z</i>
	<i>x</i> = 0	<i>x</i> = 1	
<i>A</i>	<i>H</i>	<i>B</i>	0
<i>B</i>	<i>F</i>	<i>A</i>	0
<i>C</i>	<i>G</i>	<i>D</i>	0
<i>D</i>	<i>E</i>	<i>C</i>	1
<i>E</i>	<i>A</i>	<i>C</i>	0
<i>F</i>	<i>C</i>	<i>D</i>	0
<i>G</i>	<i>B</i>	<i>A</i>	0
<i>H</i>	<i>D</i>	<i>B</i>	0

$$\pi_0 = \{\overline{A}; \overline{B}; \overline{C}; \overline{D}; \overline{E}; \overline{F}; \overline{G}; \overline{H}\} = \pi(0),$$

$$\pi_1 = \{\overline{A}, \overline{B}, \overline{C}, \overline{D}; \overline{E}, \overline{F}, \overline{G}, \overline{H}\},$$

$$\pi_2 = \{\overline{A}, \overline{D}, \overline{E}, \overline{H}; \overline{B}, \overline{C}, \overline{F}, \overline{G}\},$$

$$\pi_3 = \{\overline{A}, \overline{D}; \overline{B}, \overline{C}, \overline{F}, \overline{G}; \overline{E}, \overline{H}\},$$

$$\pi_4 = \{\overline{A}, \overline{D}, \overline{E}, \overline{H}; \overline{B}, \overline{C}; \overline{F}, \overline{G}\},$$

$$\pi_5 = \{\overline{A}, \overline{D}; \overline{B}, \overline{C}; \overline{E}, \overline{H}; \overline{F}, \overline{G}\},$$

$$\pi_6 = \{\overline{A}, \overline{B}, \overline{C}, \overline{D}, \overline{E}, \overline{F}, \overline{G}, \overline{H}\} = \pi(I).$$

Fig. 12.4 Closed partitions for M_2 .

consequently, are independent of y_3 , while the successor component has a single variable, y_3 , which distinguishes the states in the blocks of π_5 .

Maximal reduction in the dependency of state variables would be achieved if we could find three two-block closed partitions whose product is zero. In such a case, each state variable would be independent of the remaining two variables and the machine would be realized as a parallel connection of three component machines. It is evident, however, from the list of nontrivial closed partitions of M_2 that only two two-block partitions can be found, namely, π_1 and π_2 . In fact, since each nontrivial closed partition is greater than π_5 , no combination of closed partitions can be found whose product is zero. Therefore, we must select a partition τ such that

$$\pi_1 \cdot \pi_2 \cdot \tau = \pi(0).$$

One possible such partition is

$$\tau = \{\overline{A}, \overline{D}, \overline{G}, \overline{H}; \overline{C}, \overline{D}, \overline{E}, \overline{F}\}.$$

Assigning y_1 to distinguish the blocks of π_1 , y_2 to distinguish the blocks of π_2 , and y_3 to distinguish the blocks of τ results in the assignment given in Table 12.4. Clearly, y_1 and y_2 , which are assigned to the blocks of closed partitions, will be self-dependent, while y_3 , which is assigned to the blocks

Table 12.4 Excitation and output table for M_2

	$y_1 y_2 y_3$	$Y_1 Y_2 Y_3$		z
		$x = 0$	$x = 1$	
A	000	100	010	0
B	010	111	000	0
C	011	110	001	0
D	001	101	011	1
E	101	000	011	0
F	111	011	001	0
G	110	010	000	0
H	100	001	010	0

of τ , will be a function of the external input and all three state variables. The logic equations derived from Table 12.4 are

$$Y_1 = x' y_1',$$

$$Y_2 = x' y_2 + x y_2',$$

$$Y_3 = x y_3 + x' y_1' y_2 y_3' + y_1' y_2' y_3 + y_1 y_2 y_3 + x' y_1 y_2' y_3',$$

$$z = y_1' y_2' y_3.$$

The corresponding schematic diagram is shown in Fig. 12.5.

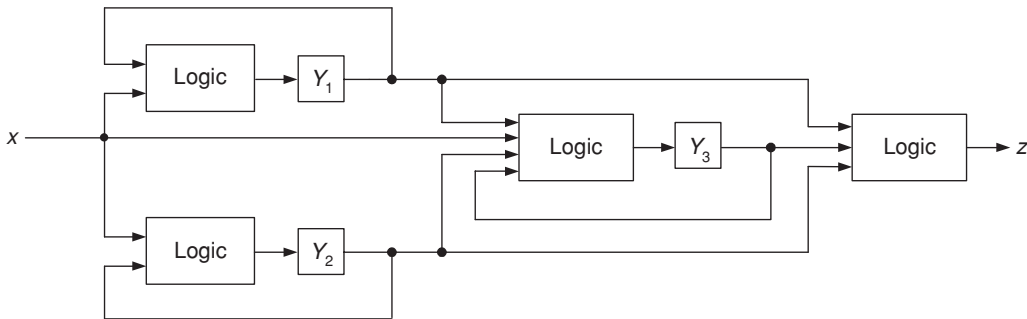


Fig. 12.5 Schematic diagram for M_2 .

12.3 The lattice of closed partitions

Closed partitions have been shown to play a significant role in the state-assignment problem and in determining the dependency of the state variables. Therefore we will present a method for generating these partitions and will investigate their properties.

Theorem 12.2 *The product $\pi_1 \cdot \pi_2$ and the sum $\pi_1 + \pi_2$ of two closed partitions on the set of states of M are also closed.*

Proof Let π_1 and π_2 be two closed partitions on the states of M . We will show that the partition $\pi_1 \cdot \pi_2$ is also closed, leaving the proof that $\pi_1 + \pi_2$ is closed as an exercise to the reader.

Let B be an arbitrary block of $\pi_1 \cdot \pi_2$. Since B is the intersection of some blocks B_1 of π_1 and B_2 of π_2 , B is contained in both B_1 and B_2 . Since π_1 and π_2 are closed, the I_k -successor of B is also contained within some block $I_k B_1$ of π_1 and some block $I_k B_2$ of π_2 , where $I_k B_i$ is the I_k -successor of B_i . Therefore $I_k B$ is contained within the intersection $I_k B_1 \cdot I_k B_2$. However, the intersection $I_k B_1 \cdot I_k B_2$ is contained in a block of $\pi_1 \cdot \pi_2$ and, consequently, $I_k B$ is contained in a block of $\pi_1 \cdot \pi_2$. Therefore, $\pi_1 \cdot \pi_2$ is closed. $\diamond$

From this theorem, it follows that to each pair of closed partitions π_1 and π_2 there corresponds a *least upper bound* (*lub*) $\pi_1 + \pi_2$ and a *greatest lower bound* (*glb*) $\pi_1 \cdot \pi_2$. Consequently, the set of closed partitions on the states of a machine is closed under the $+$ and $\cdot$ binary operations and, therefore, forms a lattice (by Definition 2.2 in Section 2.4). This lattice is referred to as the π -lattice.

Let $\pi_{S_i S_j}$ be the *smallest* closed partition containing S_i and S_j in one block. We shall subsequently refer to the placing of S_i and S_j in one block as *identifying* them. To determine $\pi_{S_i S_j}$, we first identify S_i and S_j . This identification implies that we must also identify the successors $I_k S_i$ and $I_k S_j$ for every input symbol I_k in I . States $I_k S_i$ and $I_k S_j$ are said to be *implied* by S_i and S_j . Whenever a state S_i is identified with S_j and S_k , the transitive law must be applied in such a way that (S_i, S_j, S_k) are placed in the same block of π . If we repeat the above procedure and find the smallest closed partition $\pi_{S_i S_j}$ for every pair of states $S_i S_j$, we obtain a set of partitions known as the *basic* partitions. The π -lattice can now be obtained in two steps:

1. for every pair of states $S_i S_j$, obtain $\pi_{S_i S_j}$;
2. obtain all possible sums of basic partitions.

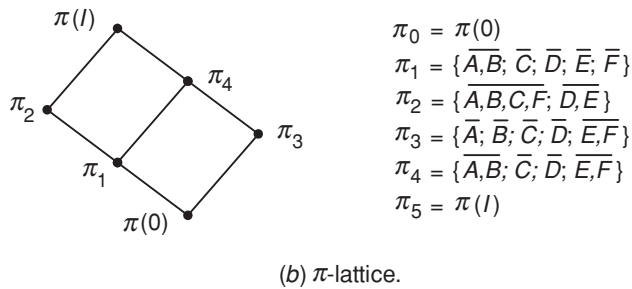
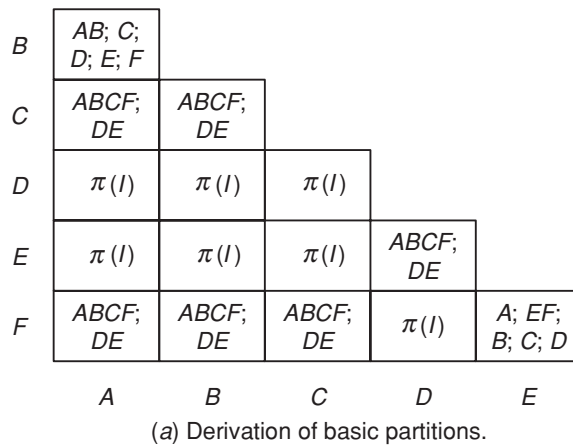
Since every closed partition can be shown (see Problem 12.5) to be the sum of one or more basic partitions, the above procedure indeed generates the set of all closed partitions.

As an illustration, we shall determine the π -lattice of the machine M_3 shown in Table 12.5. The table in Fig. 12.6a shows the possible initial identifications and their implications. Within the cell in row S_i , column S_j , we write the identifications implied by the initial identification of S_i and S_j . For example, if we start by identifying the states A, B , we find that no other pair of states is implied. Consequently, the partition $\{\overline{A}, \overline{B}; \overline{C}; \overline{D}; \overline{E}; \overline{F}\}$ is closed. We continue by identifying A, C , which, in turn, implies A, B and D, E . These implications may be described as

$$A, C \rightarrow A, B; D, E.$$

Table 12.5 Machine M_3

PS	NS	
	$x = 0$	$x = 1$
A	E	B
B	E	A
C	D	A
D	C	F
E	F	C
F	E	C

Fig. 12.6 Construction of the π -lattice of M_3 .

It is already known that the identification of A, B does not imply any other pair. Hence, we need to check only the implications due to D, E . From the state table we find that D, E implies C, F . Since A, C and C, F are identified, the transitive law must be applied to yield A, C, F . This process is thus summarized as follows:

$$A, C \rightarrow A, B; D, E \rightarrow A, C, F; A, B; D, E \rightarrow A, B, C, F; D, E.$$

The entire table is completed in a similar manner. Many shortcuts are possible. For example, while identifying B, D , the pair A, F is implied. However, since the implications which result from the identification of A, F have already been determined, it becomes immediately evident that the identification of B, D implies the identity partition, i.e.,

$$B, D \rightarrow C, E; A, F \rightarrow A, B, C, F; D, E; C, E \rightarrow \pi(I).$$

The next step in the procedure is to determine the remaining (nonbasic) closed partitions. This is done by computing the sums of pairs of basic partitions to obtain “second-level” partitions and then using only pairs of “second-level” partitions to obtain “third-level” partitions, and so on. For the machine M_3 , the basic partitions are

$$\begin{aligned}\pi_1 &= \{\overline{A}, \overline{B}; \overline{C}; \overline{D}; \overline{E}; \overline{F}\}, \\ \pi_2 &= \{\overline{A}, \overline{B}, \overline{C}, \overline{F}; \overline{D}, \overline{E}\}, \\ \pi_3 &= \{\overline{A}; \overline{B}; \overline{C}; \overline{D}; \overline{E}, \overline{F}\}.\end{aligned}$$

The only sum that yields a nontrivial closed partition is

$$\pi_4 = \pi_1 + \pi_3 = \{\overline{A}, \overline{B}; \overline{C}; \overline{D}; \overline{E}, \overline{F}\}.$$

The π -lattice for the machine M_3 is shown in Fig. 12.6b.

12.4 Reduction of the output dependency

So far, attention has been focused on reducing the dependency of state variables. In assigning the states of these variables to the blocks of a closed partition, we have a considerable amount of freedom. It is our aim in the following discussion to show how this freedom can be used to obtain simpler output circuits with reduced dependencies. The problem is illustrated by considering two possible assignments for the machine M_4 shown in Table 12.6.

Machine M_4 possesses the closed partition $\pi = \{\overline{A}, \overline{B}; \overline{C}, \overline{D}\}$. To obtain a state assignment, we are looking for a partition τ such that $\pi \cdot \tau = \pi(0)$. The assignments α and β shown in Table 12.7 correspond, respectively, to the partitions $\tau_a = \{\overline{A}, \overline{C}; \overline{B}, \overline{D}\}$ and $\tau_b = \{\overline{A}, \overline{D}; \overline{B}, \overline{C}\}$. The state variables and

Table 12.6 Machine M_4

PS	NS		z	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
A	B	D	1	0
B	A	C	0	1
C	D	A	0	1
D	C	B	1	0

Table 12.7 Two possible assignments for machine M_4

	$y_1 y_2$		$y_1 y_2$
A	00	A	00
B	01	B	01
C	10	C	11
D	11	D	10

(a) Assignment α (b) Assignment β

output function corresponding to assignment α are as follows:

$$\begin{aligned}
 Y_1 &= x'y_1 + xy'_1, \\
 Y_2 &= x'y'_2 + y'_1 y'_2 + xy_1 y_2, \\
 z &= x'y'_1 y'_2 + x'y_1 y_2 + xy'_1 y_2 + xy_1 y'_2.
 \end{aligned}$$

The number of transistors required for a two-level NAND–NAND CMOS realization of these functions is 64.

For assignment β , we obtain

$$\begin{aligned}
 Y_1 &= x'y_1 + xy'_1, \\
 Y_2 &= x'y'_2 + xy'_1 y_2 + y_1 y'_2, \\
 z &= x'y'_2 + xy_2.
 \end{aligned}$$

The realization of these functions requires only 40 transistors.

Evidently, the reduction in circuit complexity is the outcome of the decrease in the dependency of the output function. While in assignment α the output depends on x , y_1 , and y_2 , in assignment β it is independent of y_1 . Although such a reduction in the dependency of the output does not always ensure simpler output circuits, in most cases it does tend to decrease the complexity of the circuit. Our aim, therefore, is directed towards obtaining assignments which reduce the dependencies of the output logic.

Definition 12.2 A partition λ_o on the states of a machine M is said to be *output-consistent* if, for every block of λ_o and every input symbol, all the states contained in the block have the same output symbols.

Example The partition $\lambda_o = \tau_b = \{\overline{A, D}; \overline{B, C}\}$ is an output-consistent partition of the machine M_4 .

Let M have n states to which we assign k variables, where $k = \lceil \log n \rceil$. Let $r = \lceil \log_2 \#(\lambda_o) \rceil$ variables be assigned to the blocks of M 's output-consistent partition λ_o . Because λ_o is output-consistent, the output symbols associated with the blocks of λ_o can be computed from these r variables, independently

of the remaining $k - r$ variables assigned to the states in the blocks of λ_o . Consequently, we arrive at the following general result.

- The existence of an output-consistent partition λ_o on the states of a sequential machine M implies that there exists an assignment for M such that the outputs depend, at most, on the external inputs and on the variables assigned to the blocks of λ_o .

This result can be generalized as follows. Let $\Theta = \{\tau_1, \tau_2, \dots, \tau_k\}$ be the set consisting of partitions induced by the state variables $y_1, y_2, \dots, y_k$. Let $\lambda_{o1}, \lambda_{o2}, \dots, \lambda_{om}$ be the output-consistent partitions induced by the outputs $z_1, z_2, \dots, z_m$. If, for some subset Q of Θ ,

$$\lambda_{oi} \geq \prod_{j \in Q} \tau_j$$

then z_i is a function of the external input x and the variables assigned to the partitions contained in Q .

Example In the machine M_4 , $\lambda_o = \lambda_{o1} = \{\overline{A}, \overline{D}; \overline{B}, \overline{C}\}$. Since y_2 has been assigned to λ_o in assignment β , the output z depends only on this variable and is independent of y_1 .

In assignment β we obtained a reduction in the dependency of y_1 and (simultaneously) of the output z . This is possible since $\pi \cdot \lambda_o = \pi(0)$. In general, however, we cannot efficiently obtain a complete assignment on the basis of any arbitrary closed partition π and any output-consistent partition λ_o . For example, if $\pi \cdot \lambda_o = \pi(0)$ but $\lceil \log_2 \#(\pi) \rceil + \lceil \log_2 \#(\lambda_o) \rceil > \lceil \log_2 n \rceil$ then an assignment can be obtained in which the outputs depend on $\lceil \log_2 \#(\lambda_o) \rceil$ variables and these $\lceil \log_2 \#(\pi) \rceil$ variables are independent of the remaining ones. However, such an assignment is not minimal since it requires extra variables. For example, if $\pi = \{\overline{A}, \overline{B}; \overline{C}, \overline{D}; \overline{E}, \overline{F}; \overline{G}, \overline{H}\}$ while $\lambda_o = \{\overline{A}, \overline{C}; \overline{B}, \overline{E}; \overline{D}, \overline{G}; \overline{F}, \overline{H}\}$ then $\pi \cdot \lambda_o = \pi(0)$ but $\log_2 4 + \log_2 4 = 4$. If we use only π or only λ_o , we can obtain an assignment with only three variables. It should be noted that, while λ_o simplifies the output circuit, the additional variables (the fourth one in the above case), which are not assigned to any closed partition, may add a significant amount of logic to the overall circuit. Consequently, we have two different requirements: to make an assignment based on an output-consistent partition λ_o and, at the same time, to reduce the dependencies of the state variables, i.e., to assign the variables to the blocks of a closed partition π . These two requirements often conflict. Various approaches have been tried in attempts to solve this problem (see, for example, [10]). This may require some trial and error.

12.5 Input independency and autonomous clocks

Some machines can be constructed from two components: one input-independent and the other input-dependent. Our aim in this section is to determine necessary and sufficient conditions for the existence of state assignments that result in such a structure.

Definition 12.3 A partition λ_i on the states of a machine M is said to be *input-consistent* if, for every state S_i of M and all input symbols $I_1, I_2, \dots, I_p$, the next states $I_1 S_i, I_2 S_i, \dots, I_p S_i$ are in the same block of λ_i .

Example Consider the machine M_5 shown in Table 12.8. State A implies the identification of states C and D . Similarly, the identification of E and F is implied by state C , while the identification of A and B is implied by state E . Thus, the smallest input-consistent partition for M_5 is $\lambda_i = \{\overline{A, B}, \overline{C, D}, \overline{E, F}\}$. Clearly, any partition that contains λ_i is also input-consistent. Unless otherwise indicated, λ_i will subsequently designate the smallest input-consistent partition.

Table 12.8 Machine M_5

PS	NS		z	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
A	D	C	0	1
B	C	D	0	0
C	E	F	0	1
D	F	F	0	0
E	B	A	0	1
F	A	B	0	0

Since the successor relationships between the blocks of λ_i are independent of the inputs, the $\lceil \log_2 \#(\lambda_i) \rceil$ variables assigned to distinguish the blocks of λ_i are input-independent. If, in addition to λ_i , a machine M possesses a closed partition π such that $\pi \geq \lambda_i$ then, for a given state S_j and every input symbol $I_1, I_2, \dots, I_p$ in I , the next states, $I_1 S_j, I_2 S_j, \dots, I_p S_j$, must be in the same block of λ_i and, therefore, in the same block of π as well. Consequently, for a given initial state, the block of π in which the state of M is contained after any finite input sequence depends only on the initial block and on the length of the sequence. This property may be summarized as follows.

- The existence of a closed partition π and a nontrivial input-consistent partition λ_i on the states of M , where $\pi \geq \lambda_i$, is a necessary and sufficient condition for the existence of an assignment for M such that the $\lceil \log_2 \#(\pi) \rceil$ variables assigned to the blocks of π are independent of the input and of the remaining state variables.

A component machine whose output at any time is independent of the input is called an *autonomous clock*. If M possesses an input-consistent partition λ_i and several closed partitions, each greater than or equal to λ_i , then the autonomous clock corresponding to the smallest such closed partition is referred to as the *maximal autonomous clock*.

Example For M_5 , the input-consistent partition $\lambda_i = \{\overline{A}, \overline{B}; \overline{C}, \overline{D}; \overline{E}, \overline{F}\}$ is closed. The output-consistent partition is $\lambda_o = \{\overline{A}, \overline{C}, \overline{E}; \overline{B}, \overline{D}, \overline{F}\}$. Since $\pi = \lambda_i$ and $\pi \cdot \lambda_o = \pi(0)$ the assignment and logic equations in Table 12.9 result. The schematic diagram corresponding to this assignment is shown in Fig. 12.7. It clearly displays the existence of an autonomous clock as well as the reduction in the dependency of z due to λ_o . The external clock has not been shown but is implicit. In fact, it triggers the autonomous clock and causes it to change states.

Table 12.9 Assignments and equations for M_5

	$y_1 y_2 y_3$	
A	000	$Y_1 = y_2$
B	001	$Y_2 = y_1' y_2'$
C	010	$Y_3 = x y_2 + x y_3 + x' y_2' y_3' + y_2 y_3$
D	011	$z = x y_3'$
E	100	
F	101	

(b) Logical equations

(a) Assignment

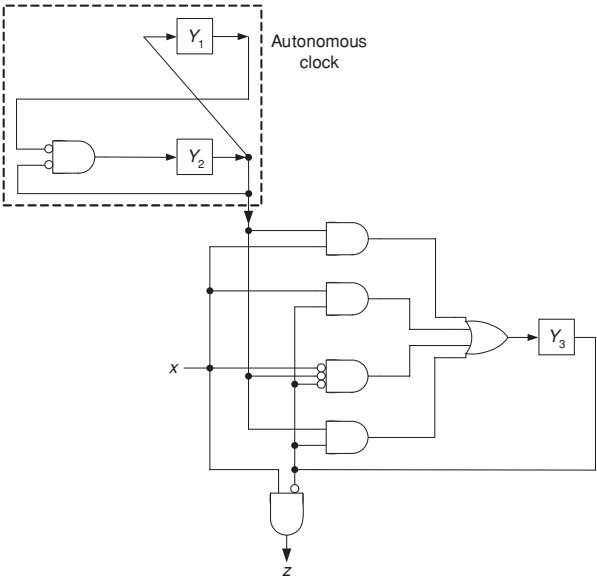


Fig. 12.7 Realization of M_5 .

It is easy to show that if M is a strongly connected machine then any component induced by a closed partition on the states of M is also strongly connected. Hence, the autonomous clock of a strongly connected machine is also strongly connected and, furthermore, it is a periodic machine. To find the period p of the autonomous clock, suppose that the machine M possesses a closed partition π such that $\pi \geq \lambda_i$. The clock has $\#(\pi)$ states and, therefore, during $\#(\pi) + 1$ time units, it must pass at least twice through one of the states. Thus, the period p is less than or equal to $\#(\pi)$.

Example The maximal autonomous clock of machine M_5 is determined from the partition $\pi = \lambda_i$, where

$$\pi = \{A, B; C, D; E, F\} = \{\bar{\alpha}; \bar{\beta}; \bar{\gamma}\}.$$

In the state table of M_5 , let us denote the blocks (A, B) , (C, D) , and (E, F) by α , β , and γ , respectively. The graph describing the block-successor relationships of π yields the state diagram of the maximal autonomous clock, as shown in Fig. 12.8. From the graph it is clear that the period p of the clock is 3.

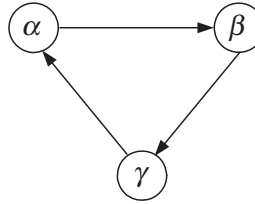


Fig. 12.8 The autonomous clock of machine M_5 .

12.6 Covers, and the generation of closed partitions by state splitting

The correlation between closed partitions and the existence of assignments with self-dependent and autonomous subsets have been established in the preceding sections. These assignments have been shown to yield simpler circuits and affect a circuit's structure. Many machines, however, do not possess such partitions and therefore cannot be implemented with independent components. Our objective in this section is to develop a method that will enable us to generalize the preceding structure theory and, by allowing the classification of the states into nondisjoint subsets, to augment a machine that does not possess any closed partition into an equivalent machine that does possess such partitions. Such an augmentation is achieved by splitting some states of the original machine. The basic tool in this procedure is the implication graph, which will be defined shortly.

Table 12.10 Machine M_6

PS	NS		z	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
A	A	B	0	1
B	C	B	0	0
C	A	C	0	0

Table 12.11 Machine M'_6

PS	NS		z	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
A	A	B	0	1
B	C'	B	0	0
C'	A	C''	0	0
C''	A	C''	0	0

Covers

To illustrate the basic ideas, consider the machine M_6 shown in Table 12.10. It can be verified that no closed partition exists for this machine and therefore it would appear that it cannot be decomposed in any manner. Consider next the machine M'_6 (Table 12.11), which is reducible to machine M_6 since the states C' and C'' are equivalent. Machine M'_6 possesses the closed partition $\pi = \{\overline{A, C'}, \overline{B, C''}\}$. If we choose a partition $\tau = \{\overline{A, B}, \overline{C', C''}\}$ such that $\pi \cdot \tau = \pi(0)$, and if we assign y_1 and y_2 to the blocks of π and τ , respectively, then the following equations result:

$$\begin{aligned} Y_1 &= x, \\ Y_2 &= xy_2 + x'y_1y'_2, \\ z &= xy'_1y'_2. \end{aligned}$$

Clearly, machine M'_6 is realizable as a serial connection of a predecessor component (Y_1) and a successor component (Y_2). Such a decomposition of machine M'_6 is also a valid realization of the equivalent machine M_6 , although the latter machine does not possess any closed partition. If we work backward from machine M'_6 to M_6 , we observe that the closed partition $\pi = \{\overline{A, C'}, \overline{B, C''}\}$ becomes equal to $\{\overline{A, C}, \overline{B, C}\}$ when the two equivalent states C' and C'' are merged. Although this collection of subsets covers all the states and is closed with respect to the states of M_6 , it does not constitute a partition since its blocks are not disjoint. In order to cover such situations it becomes necessary to generalize the structure theory and to define sets consisting of overlapping subsets of states.

Table 12.12 State transitions of the predecessor component in the serial decomposition of M_6

	$x = 0$	$x = 1$
P	P	Q
Q	P	Q

A collection φ of subsets, whose set union is S , such that no subset is included in another subset in the collection, is referred to as a *cover* on set S . The subsets are called the *blocks* of φ . The cover φ on the set of states of a machine M is said to be *closed* if, for every two states S_i and S_j which are in the same block of φ and any input symbol I_k in I , the states $I_k S_i$ and $I_k S_j$ are in a common block of φ . The number of blocks in φ and the number of elements in the largest block of φ are denoted $\#(\varphi)$ and $\rho(\varphi)$, respectively.

Example The covers $\{\overline{A, C}; \overline{B, C}\}$ and $\{\overline{A, B}; \overline{A, C}; \overline{B, C}\}$ on the set of states of M_6 are closed.

If we denote subsets (AC) and (BC) by P and Q , respectively, we obtain the successor relationships given in Table 12.12. Since the predecessor machine in the serial connection of M_6 distinguishes the blocks of φ , the successor relationships of Table 12.12 define uniquely the state transitions of the predecessor component.

In order to be able to decompose machines that do not possess any closed partition, it is necessary either to generalize the results of the previous sections to include covers or develop a method whereby any such machine can be augmented to an equivalent machine that has one or more closed partitions and is, therefore, decomposable. The approach taken in this section is the latter.

The implication graph

The main difference between the machines M_6 and M'_6 is that state C of M_6 has been split into states C' and C'' in M'_6 . In general, state S_i is said to be *split* into states S'_i and S''_i if (i) the output symbols of S'_i and S''_i are exactly the same as those of S_i and (ii) for every I_k in I , states $I_k S'_i$ and $I_k S''_i$ are identical to $I_k S_i$, except where “primes” are necessary, as will be shown later.

An *implication graph* is a directed graph, with vertices representing subsets of the set of states of a machine M . Each subset consists of states to be identified in the state table of M or which are implied by previously identified subsets of states. The arc labeled I_k represents the transition from one subset of states $(S_i, S_j, \dots)$ to the subset consisting of the I_k -successors $(I_k S_i, I_k S_j, \dots)$.

Definition 12.4 A *closed* implication graph is a subgraph of an implication graph such that: (i) for every vertex in the subgraph all outgoing arcs and their terminating vertices also belong to the subgraph; and (ii) every state of M is represented by at least one vertex.

From the definition of the implication graph for a given machine M , it is evident that the collection of subsets associated with the vertices of the closed graph constitutes a closed cover on the set of states of M . From now on, we shall consider implication graphs whose vertices represent only pairs of states. It will be shown later that such graphs provide the necessary information regarding all closed covers.

An implication graph is constructed in the following manner. Identify any pair of states S_i and S_j and assign (S_i, S_j) to some initial vertex. For each input symbol I_k , draw an arc from the vertex (S_i, S_j) to the vertex that represents the successors $(I_k S_i, I_k S_j)$. Repeat this process for all the vertices implied by the initial identification until no new vertex is generated.

If M is strongly connected, an initial identification of any pair of states will result in a closed graph. If, however, M is not strongly connected then the closed graph might have to be constructed from two or more disjoint subgraphs, that is, another pair of states not implied by (S_i, S_j) must be identified, its successors determined, and so on.

Example To construct the implication graph for the machine M_6 , start by identifying the pair of states (A, B) . This identification implies the identification of (A, C) , which in turn implies (B, C) . The graph, which is closed, is shown in Fig. 12.9. It is evident that the subgraph enclosed by the broken lines is also closed, since it satisfies Definition 12.4.

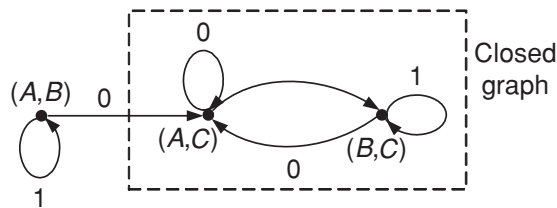


Fig. 12.9 Implication graph for M_6 .

The general procedure for augmenting an arbitrary machine M into an equivalent machine M' that possesses one or more closed partitions can now be summarized as follows.

1. Construct the implication graph of the given machine M .
2. From the implication graph, choose a closed subgraph with a minimal number of vertices. This subgraph yields a closed cover φ on M . If any state S_i

- is represented by more than one vertex, relabel S_i in the first vertex as S'_i , in the second vertex as S''_i , and so on.
3. For each S_i that has been replaced by $S'_i, S''_i, \dots$, split the corresponding state in M 's state table.
 4. Modify the entries of the new state table by inserting the necessary primes. An entry S_p in row S_i , column I_k , is changed to S'_p if S_i is represented by some vertex (S_i, S_j) and the I_k -successor vertex is (S'_p, S_q) .

Example In the implication graph of Fig. 12.9, state C appears in two vertices and thus is split into C' and C'' , as shown in Table 12.11. The partition $\pi = \{A, C'; B, C''\}$, whose blocks correspond to subsets represented by vertices of the implication graph, is clearly closed.

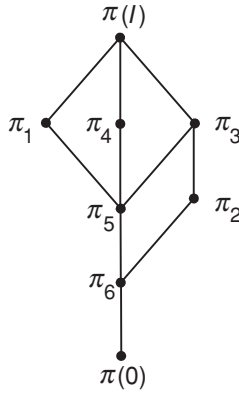
In general, a partition π whose blocks correspond to subsets represented by vertices of the closed implication graph is closed with respect to the set of states of the augmented machine M' . This partition has a finite number of blocks, since $(n - 1)n/2$ is the total number of distinct pairs of states. The closed implication graph actually describes the successor relationship of the blocks of π graphically and, consequently, represents the state diagram of the predecessor component in a possible serial realization of M' . The *implication table*, which is the tabular representation of the implication graph, is therefore the state table of the predecessor component. The implication table that corresponds to the closed graph of Fig. 12.9 was derived earlier and is shown in Table 12.12.

From the foregoing procedure it follows that *corresponding to every finite-state machine M , there exists at least one equivalent finite-state machine M' that possesses a closed partition and is therefore serially decomposable*. It should be emphasized, however, that such decompositions are not necessarily the most economical way of realizing a machine. In fact, for an n -state machine, the closed cover may have up to $(n - 1)n/2$ blocks, which means that the predecessor component will have more states than the original machine. The primary case of practical interest is that in which none of the components in the decomposition is equal to or greater than the original machine. This condition is satisfied whenever the number of vertices in the closed implication graph is smaller than n .

In the foregoing discussion, attention has been focused primarily on uniform closed covers containing two states per block. The remaining covers can be determined from this set of basic covers by obtaining all possible sums in a manner analogous to the method of generating the set of closed partitions. The preceding techniques can be extended easily to blocks of any size and of uniform, as well as nonuniform, covers.

Table 12.13 Machine M_7

PS	NS		z	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
A	B	C	0	0
B	A	F	1	1
C	F	E	1	0
D	F	E	1	1
E	G	D	0	0
F	D	B	0	0
G	E	F	1	0

Fig. 12.10 The π -lattice for M_7 .

$$\begin{aligned}
 \pi(I) &= \{\overline{A, B, C, D, E, F, G}\} \\
 \pi_1 &= \{\overline{A, C, D, E}, \overline{B, F, G}\} \\
 \pi_2 &= \{\overline{A, G}, \overline{B, E}, \overline{C, D, F}\} \\
 \pi_3 &= \{\overline{A, B, E, G}, \overline{C, D, F}\} \\
 \pi_4 &= \{\overline{A, E, F}, \overline{B, C, D, G}\} \\
 \pi_5 &= \{\overline{A, E}, \overline{B, G}, \overline{C, D}, \overline{F}\} \\
 \pi_6 &= \{\overline{A}, \overline{B}, \overline{C, D}, \overline{E}, \overline{F}, \overline{G}\} \\
 \pi(0) &= \{\overline{A}, \overline{B}, \overline{C}, \overline{D}, \overline{E}, \overline{F}, \overline{G}\}
 \end{aligned}$$

An application of state splitting to parallel decomposition

A machine M_7 and its π -lattice are given in Table 12.13 and Fig. 12.10, respectively. In addition to these closed partitions, M_7 possesses an output-consistent partition λ_o and an input-consistent partition λ_i , namely,

$$\begin{aligned}
 \lambda_o &= \{\overline{A, E, F}, \overline{B, D}, \overline{C, G}\}, \\
 \lambda_i &= \{\overline{A, E, F}, \overline{B, C, D, G}\} = \pi_4.
 \end{aligned}$$

Our aim is to obtain a parallel decomposition of M_7 . A brief inspection of the π -lattice reveals that no such decomposition is possible, since no two nontrivial closed partitions exist such that $\pi_i \cdot \pi_j = \pi(0)$ (the subset (C, D) is common to all nontrivial partitions). Consequently, it becomes necessary to check whether there exist any closed covers that yield a parallel decomposition.

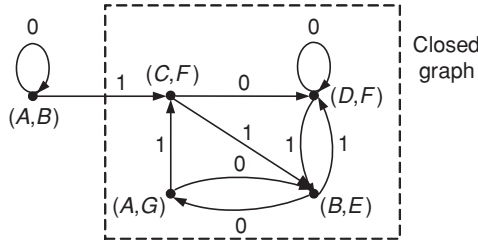
The implication graph, when started by the identification of (A, B) , is given in Fig. 12.11. From the closed graph, we obtain the closed cover

$$\varphi = \{\overline{A, G}, \overline{B, E}, \overline{C, F}, \overline{D, F}\}.$$

The corresponding augmented machine M'_7 is given in Table 12.14.

Table 12.14 Machine M'_7

PS	NS		z	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
A	B	C	0	0
B	A	F''	1	1
C	F''	E	1	0
D	F''	E	1	1
E	G	D	0	0
F'	D	B	0	0
F''	D	B	0	0
G	E	F'	1	0

Fig. 12.11 The implication graph for M_7 .

In general, for every closed partition π on M , a corresponding closed partition π' on M' can be obtained by placing the states S'_i, S''_i , etc., in π' for every split state S_i in π . The closed partitions of machine M'_7 , which may be used to achieve a parallel decomposition, are

$$\begin{aligned}\pi &= \{\overline{A, G}; \overline{B, E}; \overline{C, F'}; \overline{D, F''}\}, \\ \pi'_4 &= \{\overline{A, E, F', F''}; \overline{B, C, D, G}\}, \\ \pi'_3 &= \{\overline{A, B, E, G}; \overline{C, D, F', F''}\}.\end{aligned}$$

In addition, the augmented machine possesses the following output-consistent and input-consistent partitions:

$$\begin{aligned}\lambda'_0 &= \{\overline{A, E, F', F''}; \overline{B, D}; \overline{C, G}\}, \\ \lambda'_1 &= \pi'_4.\end{aligned}$$

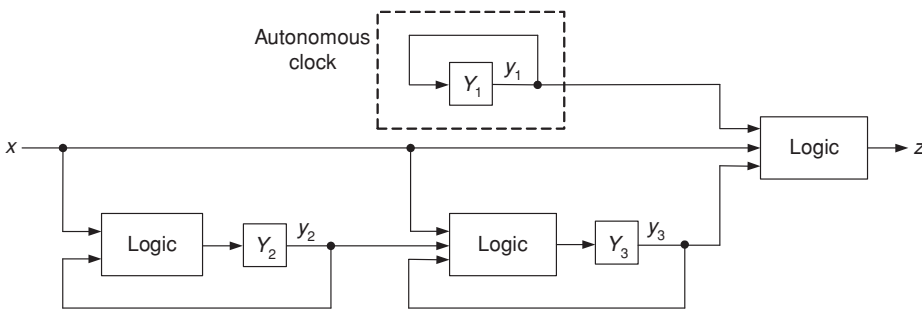
From this set of partitions, the following observations can be made:

1. The product $\pi \cdot \pi'_4 = \pi(0)$, which implies that a parallel decomposition is possible.
2. The component machine corresponding to π'_4 consists of a single variable, y_1 . It is an autonomous clock since $\pi'_4 = \lambda'_1$.
3. Because each block of π'_3 contains exactly two blocks of π , we may assign y_2 to the blocks of π'_3 and thus make it independent of the value of y_3 .

Fig. 12.12 Decomposition of M'_7 .

	$y_1 y_2 y_3$	
<i>A</i>	000	$y_1 = y'_1$
<i>B</i>	101	$y_2 = x' y_2 + x y'_2$
<i>C</i>	110	$y_3 = y_2 + x y_3 + x' y'_3$
<i>D</i>	111	$z = x' y_1 + y_1 y_3$
<i>E</i>	001	
<i>F'</i>	010	
<i>F''</i>	011	
<i>G</i>	100	

(a) Assignment and logic equations.



(b) Schematic diagram.

- The variable y_3 must be assigned to the blocks of a partition τ such that $\pi'_3 \cdot \tau = \pi$. The partition $\tau = \{\overline{A, C, F', G}; \overline{B, D, E, F''}\}$ satisfies this condition.
- The product $\tau \cdot \pi'_4 = \{\overline{A, F'}; \overline{B, D}; \overline{C, G}; \overline{E, F''}\}$ is smaller than λ'_6 ; consequently, the output z will be a function of only y_1 and y_3 .

The assignment and logic equations resulting from the preceding observations are shown in Fig. 12.12a. The schematic diagram is shown in Fig. 12.12b.

12.7 Information flow in sequential machines

In the previous sections we have dealt mainly with serial and parallel decompositions. Of course, there are more complex structures, and our aim in this section is to define them and determine the conditions under which they exist. The main tool for accomplishing this task is the partition pair. It will be shown that the problem of finding state assignments leading to specified machine structures is equivalent to the problem of finding an appropriate set of partition pairs and determining their properties.

Table 12.15 Machine M_8

PS	NS				z
	x_1x_2	00	01	10	
A	A	C	D	F	0
B	C	B	F	E	0
C	A	B	F	D	0
D	E	F	B	C	0
E	E	D	C	B	0
F	D	F	B	A	1

Table 12.16 Two possible assignments for M_8

	$y_1y_2y_3$		$y_1y_2y_3$
A	000	A	000
B	010	B	011
C	011	C	010
D	111	D	110
E	100	E	100
F	110	F	111

(a) Assignment α (b) Assignment β

Introduction

The machine M_8 shown in Table 12.15 possesses two closed partitions: $\pi_1 = \{\overline{A, B, C}; \overline{D, E, F}\}$ and $\pi_2 = \{\overline{A, E}; \overline{B, F}; \overline{C, D}\}$, where $\pi_1 \cdot \pi_2 = \pi(0)$. Consequently, M_8 can be decomposed into two parallel components, as shown by assignment α in Table 12.16a. The corresponding logic equations for the state variables are

$$\begin{aligned} Y_1 &= x'_1y_1 + x_1y'_1 = f_1(x_1, y_1), \\ Y_2 &= x_2 + x_1y'_2 + x_1y_3 + x'_1y_2y'_3 = f_2(x_1, x_2, y_2, y_3), \\ Y_3 &= x'_1x'_2y_2y'_3 + x_2y'_2 + x_1x'_2y_2y_3 = f_3(x_1, x_2, y_2, y_3). \end{aligned}$$

The two-level NAND–NAND CMOS realization of the above equations requires 60 transistors, and the functional dependencies are such that two of the next-state variables (Y_2 and Y_3) each depend on two of the present-state variables (y_2 and y_3).

Next, we examine assignment β in Table 12.16b, which yields the following equations:

$$\begin{aligned} Y_1 &= x'_1y_1 + x_1y'_1 = f_1(x_1, y_1), \\ Y_2 &= x_2 + x'_1y_3 + x_1y'_3 = f_2(x_1, x_2, y_3), \\ Y_3 &= x_2y_2 + x_1x'_2y'_2 = f_3(x_1, x_2, y_2). \end{aligned}$$

The two-level realization of these equations requires only 40 transistors. This reduction in the number of transistors has been accomplished by reducing the functional dependencies of the variables, since each next-state variable now depends on just a single present-state variable. Evidently, this type of reduced dependency (which actually contains “cross dependencies”) cannot be predicted just from the closed partitions. Consequently, a more general tool is needed.

Partition pairs

In order to determine the cause of the cross dependencies obtained by assignment β , we first observe that y_1 induces π_1 while y_2 and y_3 induce the partitions $\tau(y_2) = \{\overline{A, E}; \overline{B, C, D, F}\}$ and $\tau(y_3) = \{\overline{A, C, D, E}; \overline{B, F}\}$, respectively, where $\pi_1 \cdot \tau(y_2) \cdot \tau(y_3) = \pi(0)$. Except for π_1 , neither of these partitions is closed although the product $\tau(y_2) \cdot \tau(y_3) = \pi_2$ is closed. However, knowledge of the block of $\tau(y_2)$ and the input symbols is sufficient to determine uniquely the successor block contained in some block of $\tau(y_3)$; that is, successors of the blocks of $\tau(y_2)$ are contained in the blocks of $\tau(y_3)$. Similarly, it is evident that the blocks of $\tau(y_2)$ are successors of the blocks of $\tau(y_3)$.

Definition 12.5 A *partition pair* (τ, τ') on the states of a sequential machine M is an ordered pair of partitions such that, if S_i and S_j are in the same block of τ then, for every input symbol I_k in I , $I_k S_i$ and $I_k S_j$ are in the same block of τ' .

Thus τ' consists of all the successor blocks implied by τ . If $\tau = \tau'$ then τ is closed, since it contains its own successor blocks. Hence, the set of closed partitions may be viewed as a special case of the (more general) set of partition pairs.

Example The following are partition pairs on the states of M_8 :

$$(\pi_1, \pi'_1) = (\{\overline{A, B, C}; \overline{D, E, F}\}, \{\overline{A, B, C}; \overline{D, E, F}\}),$$

$$(\tau_1, \tau'_1) = (\{\overline{A, C, D, E}; \overline{B, F}\}, \{\overline{A, E}; \overline{B, C, D, F}\}),$$

$$(\tau_2, \tau'_2) = (\{\overline{A, E}; \overline{B, C, D, F}\}, \{\overline{A, C, D, E}; \overline{B, F}\}).$$

In assignment β of Table 12.16, y_1 , y_2 , and y_3 have been assigned to π'_1 , τ'_1 , and τ'_2 , respectively. Note that in this example, (τ'_1, τ_1) and (τ'_2, τ_2) are also partition pairs.

In general, since τ consists of the blocks we want to identify while τ' contains the implied successor blocks, it is evident that any partition τ'_p such that $\tau'_p \geq \tau'$ will also contain the successor blocks of τ . Similarly, the implied successors of any partition τ_q such that $\tau_q \leq \tau$ are smaller than those of τ and, therefore,

will be contained within the blocks of τ' . Thus, the pairs (τ_q, τ') and (τ, τ'_p) are also partition pairs on the states of M .

Example The pair $(\tau_3, \tau'_3) = (\overline{\{A, D; B; C; E; F\}}, \overline{\{A, E; B; D; C; F\}})$ is a partition pair on M_8 . The following are also partition pairs on M_8 :

$$\begin{aligned} &(\overline{\{A, D; B; C; E; F\}}, \overline{\{A, E; B; D; C; F\}}), \\ &(\overline{\{A, D; B; C; E; F\}}, \overline{\{A, E; B; C, D, F\}}). \end{aligned}$$

A partial ordering on partition pairs is defined in the following way. If (τ_1, τ'_1) and (τ_2, τ'_2) are partition pairs then $(\tau_1, \tau'_1) \geq (\tau_2, \tau'_2)$ if and only if $\tau_1 \geq \tau_2$ and $\tau'_1 \geq \tau'_2$. We shall now prove that if (τ_1, τ'_1) and (τ_2, τ'_2) are partition pairs on the states of a machine M then $(\tau_1 \cdot \tau_2, \tau'_1 \cdot \tau'_2)$ and $(\tau_1 + \tau_2, \tau'_1 + \tau'_2)$ are also partition pairs on the states of M and define, respectively, the glb and lub of the given partition pairs. The assertion that $(\tau_1 \cdot \tau_2, \tau'_1 \cdot \tau'_2)$ is the glb of (τ_1, τ'_1) and (τ_2, τ'_2) can be proved by observing that if S_i and S_j are contained in some block of $\tau_1 \cdot \tau_2$, then they are contained in the same block in τ_1 and in τ_2 . Therefore, for every input symbol I_k , the successors $I_k S_i$ and $I_k S_j$ are also contained in the same block of τ'_1 and τ'_2 and, hence, of $\tau'_1 \cdot \tau'_2$. The assertion that $(\tau_1 + \tau_2, \tau'_1 + \tau'_2)$ is the lub of (τ_1, τ'_1) and (τ_2, τ'_2) can be proved in a similar manner. Consequently, the set of all partition pairs forms a lattice under the above partial ordering.

Definition 12.6 Let τ' be a partition on the set of states of M . Define a partition $M(\tau')$ such that $M(\tau') = \sum \tau_i$, where the sum is over all τ_i such that (τ_i, τ') is a partition pair. Similarly, define a partition $m(\tau) = \prod \tau'_i$, where the product is over all τ'_i such that (τ, τ'_i) is a partition pair. A partition pair (τ, τ') is said to be an *Mm pair* if and only if $\tau = M(\tau')$ and $\tau' = m(\tau)$.

Since the lub of two partition pairs is a partition pair it follows that $(M(\tau'), \tau')$ is a partition pair, where $M(\tau')$ is the lub of all τ_i such that (τ_i, τ') is a partition pair. In fact, $M(\tau')$ is the largest partition the successors of whose blocks are contained in the blocks of τ' . Similarly, since the glb of two partition pairs is a partition pair, it follows that $(\tau, m(\tau))$ is a partition pair, where $m(\tau)$ is the glb of all τ'_i such that (τ, τ'_i) is a partition pair. The partition $m(\tau)$ is thus the smallest partition containing all the successors of the blocks of τ . Hence, $m(\tau)$ describes the largest amount of information that can be obtained from τ regarding the next state of the machine M .

It can be shown (see Problem 12.15) that the M and m partitions possess the following properties. If τ is a partition on machine M then

$$\begin{aligned} m[M(\tau)] &\leq \tau, \\ M[m(\tau)] &\geq \tau, \\ M\{m[M(\tau)]\} &= M(\tau), \\ m\{M[m(\tau)]\} &= m(\tau). \end{aligned}$$

Consequently, for every partition τ on the states of M , $\{M(\tau), m[M(\tau)]\}$ and $\{M[m(\tau)], m(\tau)\}$ are Mm pairs on the states of M .

If (λ, λ') is an Mm pair then λ is the largest partition from that we can determine λ' and, at the same time, λ' is the smallest partition that contains the successor blocks implied by λ . Thus, by enlarging λ' or by refining λ , we can obtain other partition pairs. Consequently, corresponding to every partition pair (τ, τ') there exists an Mm pair (λ, λ') such that $\lambda \geq \tau$ and $\lambda' \leq \tau'$. Clearly, the set of all Mm pairs (which is, in general, substantially smaller than the set of all partition pairs) completely characterizes the set of all partition pairs on the states of M , since any partition pair can be generated from the corresponding Mm pair, as shown above.

Information-flow inequalities

In this section we shall derive the main theorem relating the algebraic properties of partitions to the dependencies of state variables and the structure of sequential machines. We shall also show that the existence of assignments with reduced dependencies of state variables can be predicted from the set of Mm pairs associated with the machine.

Theorem 12.3 *Let the variables $y_1, y_2, \dots, y_k$ be assigned to the states of machine M , and let $\tau(y_i)$ be the partition induced by the variable y_i , where $1 \leq i \leq k$. If the next-state variable Y_i can be computed from the external inputs and a subset P_i of variables, then*

$$\prod \tau(y_j) \leq M[\tau(y_i)],$$

where the product is taken over all $\tau(y_j)$ such that y_j is contained in subset P_i . Conversely, a sufficient condition for the existence of an assignment, in which a next-state variable Y_i depends only on the external inputs and the value of a corresponding subset P_i of state variables, is the existence of a partition pair $(\tau, \tau(y_i))$ on M such that, for each τ'_i ,

$$\prod \tau(y_j) \leq M[\tau(y_i)],$$

where the product is taken over all $\tau(y_j)$ such that y_j is in P_i .

Proof The blocks of the partition $\prod \tau(y_j)$ consist of all the states that have the same value of the variables contained in P_i . Recalling that Y_i depends only on variable y_j if y_j is in P_i then, for any two states S_p and S_q that are in the same block of $\prod \tau(y_j)$, and for all input symbols I_k in I , the successor states $I_k S_p$ and $I_k S_q$ are in the same block of $\tau(y_i)$. Consequently,

$$\left(\prod \tau(y_j), \tau(y_i) \right)$$

is a partition pair. However, since $M[\tau(y_i)]$ is the largest partition such that $(M[\tau(y_i)], \tau(y_i))$ is a partition pair,

$$M[\tau(y_i)] \geq \prod \tau(y_j).$$

Hence, if the next-state variable Y_i can be computed from a subset of the state variables then we must have at least as much information about the present state as is contained in $M[\tau(y_i)]$.

To prove the converse note that $(M[\tau(y_i)], \tau(y_i))$ is a partition pair and, since $\prod \tau(y_j) \leq M[\tau(y_i)]$,

$$\left(\prod \tau(y_j), \tau(y_i) \right)$$

is also a partition pair. Knowledge of the values of the variables y_j in P_i is sufficient to determine the present block of $\prod \tau(y_j)$ and, therefore (by the definition of partition pairs), it is also sufficient to determine the successor block in $\tau(y_i)$. This in turn determines the value of the next state of y_i , that is, Y_i . Thus, the theorem is proved. $\diamond$

Returning to machine M_8 we note that $\pi'_1 \cdot \tau'_1 \cdot \tau'_2 = \pi(0)$ and that $\pi_1 = \pi'_1$, $\tau'_1 = \tau_2$, and $\tau'_2 = \tau_1$. Therefore, a three-variable assignment exists such that Y_1 (which is assigned to π'_1) is self-dependent while Y_2 and Y_3 (which are assigned to τ'_1 and τ'_2) can be computed from y_3 and y_2 , respectively. The above arguments lead to assignment β of Table 12.16b.

The partition inequality in Theorem 12.3 is frequently referred to as *information-flow inequality*. It defines the minimal amount of information which we must have in order to compute the value of y_i for the next state. In other words, since $M[\tau(y_i)]$ is the largest partition (the least amount of information regarding the machine's state) from which we can determine the block of $\tau(y_i)$ containing the next state of the machine then, in order to compute the value of y_i for the next state, we must have at least as much information about the present state as is contained in $M[\tau(y_i)]$. Thus, knowledge of the information-flow inequalities is sufficient to specify the dependencies of the state variables and determine the direction of "information flow" in the machine.

Computing the Mm pairs

Having established (in Theorem 12.3) the role of Mm pairs in the determination of assignments with reduced dependencies, we proceed to develop a systematic procedure to generate these pairs. Let a and b be two arbitrary states of machine M , and let τ_{ab} be the partition that includes a block (ab) and leaves all other states in separate blocks. Then $m(\tau_{ab})$ is the smallest partition containing the blocks implied by the identification of (ab) . Clearly, $(\tau_{ab}, m(\tau_{ab}))$ is a partition pair.

Table 12.17 Machine M_9

PS	NS				
	x_1x_2				
	00	01	11	10	z
A	C	A	D	B	0
B	E	C	B	D	0
C	C	D	C	E	0
D	E	A	D	B	0
E	E	D	C	E	1

Any partition τ can be expressed as a sum $\tau = \sum \tau_{ab}$, where the sum is taken over all τ_{ab} such that $\tau_{ab} \leq \tau$. In addition, since the sum of partition pairs is also a partition pair, $(\sum \tau_{ab}, \sum m(\tau_{ab}))$ must be a partition pair. Therefore, if τ is the M -partition then $(\tau, \sum m(\tau_{ab}))$ is an Mm pair.

The preceding result provides us with the basic tool for the computation of Mm pairs. First, we find the set $\{m(\tau_{ab})\}$ for all distinct a and b . This process requires $n(n-1)/2$ computations. Next, we find all possible sums of these partitions. From the preceding results, it is evident that this process generates all the m -partitions. The M -partition $\tau = M(\tau')$ corresponding to every m -partition τ' is given by $\tau = \sum \tau_{ab}$, where the sum is taken over all τ_{ab} such that $m(\tau_{ab}) \leq \tau'$. This procedure actually generates the sum of all τ_{ab} which satisfy the requirement that (τ_{ab}, τ') is a partition pair. As an example, we shall compute the Mm pairs for the machine M_9 given in Table 12.17.

First, we compute the $m(\tau_{ab})$, starting from $m(\tau_{AB})$ and continuing through all possible pairs up to $m(\tau_{DE})$. The m -partition $m(\tau_{AB})$ is found by obtaining the successors implied by the identification of A and B . From the state table we conclude that the identification of (AB) implies the identifications of (CE) , (AC) , and (BD) . The application of the transitive rule yields

$$m(\tau_{AB}) = \{\overline{A, C, E}; \overline{B, D}\} = \tau'_1.$$

Hence, if the uncertainty regarding the present state of M , which is specified by τ_{AB} , is (AB) then the uncertainty regarding the next state of M is given by $m(\tau_{AB}) = \tau'_1$. In a similar fashion, we find the following set of distinct $m(\tau_{ab})$'s for machine M_9 :

$$\begin{aligned} m(\tau_{AC}) &= m(\tau_{DE}) = \{\overline{A, C, D}; \overline{B, E}\} = \tau'_2, \\ m(\tau_{AD}) &= m(\tau_{CE}) = \{\overline{A}; \overline{B, C, E}; \overline{D}\} = \tau'_3, \\ m(\tau_{AE}) &= m(\tau_{CD}) = \pi(I), \\ m(\tau_{BC}) &= m(\tau_{BE}) = \{\overline{A}; \overline{B, C, D, E}\} = \tau'_4, \\ m(\tau_{BD}) &= \{\overline{A, C}; \overline{B, D}; \overline{E}\} = \tau'_5. \end{aligned}$$

The next step in the computation of m -partitions is to form all possible sums of the $m(\tau_{ab})$. This is accomplished by performing all pairwise sums,

then pairwise sums of the new partitions generated, and so on. In the above example, no new nontrivial m -partitions are generated in this step.

Using the above set of m -partitions, we compute next the corresponding set of M -partitions. Recalling that $M(\tau'_i) = \sum \tau_{ab}$, where the sum is taken over all τ_{ab} such that $m(\tau_{ab}) \leq \tau'_i$, we obtain

$$\begin{aligned} M(\tau'_1) &= \tau_{AB} + \tau_{AD} + \tau_{CE} + \tau_{BD} = \{\overline{A, B, D}; \overline{C, E}\} = \tau_1, \\ M(\tau'_2) &= \tau_{AC} + \tau_{DE} = \{\overline{A, C}; \overline{B, D}, \overline{E}\} = \tau_2, \\ M(\tau'_3) &= \tau_{AD} + \tau_{CE} = \{\overline{A, D}; \overline{B, C}, \overline{E}\} = \tau_3, \\ M(\tau'_4) &= \tau_{BC} + \tau_{BE} + \tau_{AD} + \tau_{CE} = \{\overline{A, D}; \overline{B, C, E}\} = \tau_4, \\ M(\tau'_5) &= \tau_{BD} = \{\overline{A}; \overline{B, D}; \overline{C, E}\} = \tau_5. \end{aligned}$$

Thus, the machine M_9 possesses a set of seven Mm pairs (of which two pairs are trivial), namely,

$$\begin{aligned} &(\pi(I), \pi(I)), \\ (\tau_1, \tau'_1) &= (\{\overline{A, B, D}; \overline{C, E}\}, \{\overline{A, C, E}; \overline{B, D}\}), \\ (\tau_2, \tau'_2) &= (\{\overline{A, C}; \overline{B, D}, \overline{E}\}, \{\overline{A, C, D}; \overline{B, E}\}), \\ (\tau_3, \tau'_3) &= (\{\overline{A, D}; \overline{B, C}, \overline{E}\}, \{\overline{A}; \overline{B, C, E}; \overline{D}\}), \\ (\tau_4, \tau'_4) &= (\{\overline{A, D}; \overline{B, C, E}\}, \{\overline{A}; \overline{B, C, D, E}\}), \\ (\tau_5, \tau'_5) &= (\{\overline{A}; \overline{B, D}; \overline{C, E}\}, \{\overline{A, C}; \overline{B, D}; \overline{E}\}), \\ &(\pi(0), \pi(0)). \end{aligned}$$

The Mm -lattice can now be drawn in a straightforward manner.

The above Mm pairs characterize the machine and contain all the information regarding its structure. In addition to numerous partition pairs that can be generated from these Mm pairs, two closed partitions π_1 and π_2 exist, where

$$\begin{aligned} \pi_1 &= \{\overline{A, D}; \overline{B}; \overline{C, E}\}, \\ \pi_2 &= \{\overline{A}; \overline{B}; \overline{C, E}; \overline{D}\}. \end{aligned}$$

The closed partitions are generated by enlarging the m -partition and refining the M -partition of the Mm pair (τ_3, τ'_3) .

State assignments based on partition pairs

We shall now apply the principles developed in this section, and our knowledge about the information flow in the machine M_9 , to obtain an assignment in which the dependencies of the variables will be reduced. For the example at hand our aim is to obtain a three-variable assignment. Consequently, we are seeking three partitions, $\lambda_1, \lambda_2, \lambda_3$, of two blocks each, such that

$$\lambda_1 \cdot \lambda_2 \cdot \lambda_3 = \pi(0).$$

For each λ_i , we shall determine the corresponding $M(\lambda_i)$ and thus obtain three partition pairs, $(M(\lambda_1), \lambda_1), (M(\lambda_2), \lambda_2), (M(\lambda_3), \lambda_3)$, from which the structure of the machine can be determined.

To each partition λ_i we assign one state variable, y_i (in general, there are $\lceil \log_2 \#(\lambda_i) \rceil$ state variables). Then $M(\lambda_i)$ is the partition containing the smallest amount of information from which we can compute the value of y_i assigned to the block of λ_i that contains the next state of the machine. From Theorem 12.3 it is evident that a reduction in the dependency of the variable assigned to a partition λ_i is achieved if $M(\lambda_i)$ is greater than or equal to the product of a small subset of partitions, $\lambda_1, \lambda_2, \lambda_3$. The variables assigned to the partitions in the subset provide y_i with at least that information specified by $M(\lambda_i)$.

In order to select the partitions $\lambda_1, \lambda_2, \lambda_3$, we look for two-block partitions in the set of m -partitions τ_i 's. In particular, if a variable y_i assigned to λ_i is to depend on just one other variable assigned to the blocks of λ_j then $\lambda_j \leq M(\lambda_i)$ and $M(\lambda_i)$ can have at most two blocks. Thus, as our initial selection, let $\lambda_1 = \tau'_1$. Since $M(\tau'_1)$ consists of two blocks, we may select it as the second partition, i.e., $\lambda_2 = M(\tau'_1)$. Hence the variable Y_1 defined by λ_1 will depend only on the information provided by y_2 , which is defined by λ_2 . As λ_1 and λ_2 have already been selected, the selection of λ_3 is simple, since it must satisfy $\lambda_1 \cdot \lambda_2 \cdot \lambda_3 = \pi(0)$. We thus choose $\lambda_3 = \tau'_2$. The partitions $\lambda_1, \lambda_2, \lambda_3$ and their corresponding M -partitions $M(\lambda_1), M(\lambda_2), M(\lambda_3)$ are given as follows:

$$\begin{aligned} (M(\lambda_1), \lambda_1) &= (\overline{\{A, B, D; C, E\}}, \overline{\{A, C, E; B, D\}}), \\ (M(\lambda_2), \lambda_2) &= (\overline{\{A, D; B; C, E\}}, \overline{\{A, B, D; C, E\}}), \\ (M(\lambda_3), \lambda_3) &= (\overline{\{A, C; B; D, E\}}, \overline{\{A, C, D; B, E\}}). \end{aligned}$$

Note that λ_2 is not an m -partition but, since $\lambda_2 > \tau'_3$, we have $M(\lambda_2) \geq M(\tau'_3)$.

From the way in which we selected the above partition pairs it is evident that Y_1 depends only on y_2 , since λ_2 provides all the information that Y_1 requires as specified by $M(\lambda_1)$. In order to determine the dependencies of Y_2 and Y_3 , we check to see whether there exists a partition $\lambda_i \leq M(\lambda_2)$ or $\lambda_j \leq M(\lambda_3)$. Since there are no such partitions, the next step is to check whether we can form a product of two partitions such that $\lambda_i \cdot \lambda_j \leq M(\lambda_2)$ or $\lambda_p \cdot \lambda_q \leq M(\lambda_3)$. Indeed, this can be accomplished, since

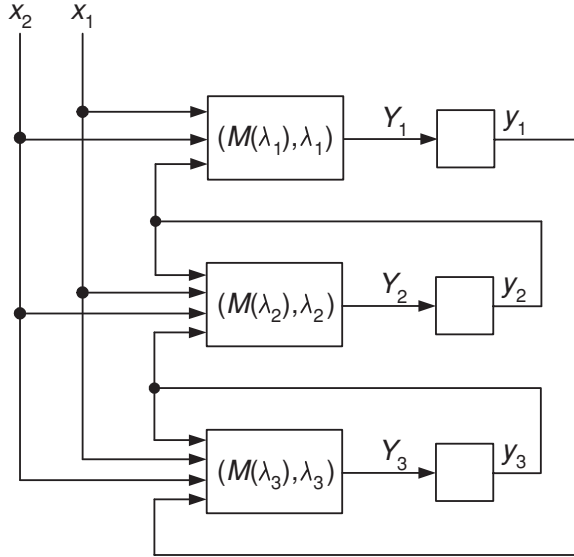
$$\begin{aligned} \lambda_2 \cdot \lambda_3 &< M(\lambda_2), \\ \lambda_1 \cdot \lambda_3 &< M(\lambda_3). \end{aligned}$$

Consequently, Y_2 depends on the information supplied by y_2 and y_3 , while Y_3 receives its inputs from y_1 and y_3 . The functional dependencies of the next-state variables are summarized as follows:

$$\begin{aligned} Y_1 &= f_1(x_1, x_2, y_2), \\ Y_2 &= f_2(x_1, x_2, y_2, y_3), \\ Y_3 &= f_3(x_1, x_2, y_1, y_3). \end{aligned}$$

The schematic diagram of the circuit structure is shown in Fig. 12.13.

Fig. 12.13 Schematic diagram of the structure of M_9 when realized using λ_1 , λ_2 , and λ_3 .



12.8 Decomposition

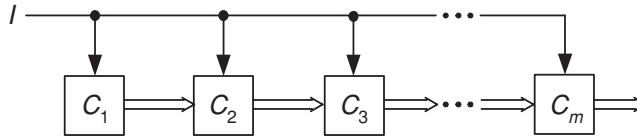
In the preceding sections we have studied the relationship between the state-assignment problem and the structure of sequential machines and have determined necessary and sufficient conditions for a machine to be decomposable. Our objective in this section is to investigate further the properties of decomposable machines and of various component machines.

Serial decomposition

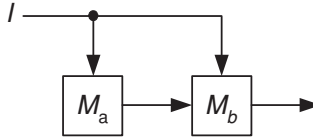
We shall first determine the conditions for a machine M to be decomposable into a serial (cascade) chain of component machines $C_1, C_2, \dots, C_m$ in which the outputs of any component may be used as inputs to other components. If an output of machine C_i is an input of machine C_j then C_i is said to be a *predecessor* of C_j and C_j is said to be a *successor* of C_i . We shall assume that the component machines *operate concurrently*, i.e., that the next state of each component depends on its present state, on the current values of external inputs, and on the present state of its predecessors. We shall assume further that the component machines form a *loop-free interconnection*; i.e., if C_i or any of its successors or successors of successors, etc., is a predecessor of C_j then C_j must not be a predecessor of C_i . A schematic diagram of such a serial decomposition is shown in Fig. 12.14a.

Theorem 12.4 *Let a machine M be realizable as a serial loop-free connection of m components $C_1, C_2, \dots, C_m$; then there exists a set of m closed partitions $\{\pi_1, \pi_2, \dots, \pi_m\}$ such that $\pi_1 \geq \pi_2 \geq \dots \geq \pi_m$ and $\pi_m = \pi(0)$. Conversely,*

Fig. 12.14 Serial decomposition of a machine.



(a) Cascaded chain. (The double arrows indicate a transfer of information from all predecessor stages.)



(b) Block diagram of the cascaded chain.

such a set of closed partitions is a sufficient condition for the existence of a serial decomposition in which C_i is a predecessor of C_j if and only if $\pi_i \geq \pi_j$.

Proof Suppose that the machine M has been realized as a serial connection of m components, as shown in Fig. 12.14a. For the purpose of analysis we may divide these components into two groups, as shown in Fig. 12.14b. The first group, denoted M_a , consists of k components and the second group, denoted M_b , consists of $m - k$ components. If we let k equal 1 then, by Theorem 12.1, there exists a closed partition π_1 on the states of M . Similarly, if we group the machines together as (C_1, C_2) and $(C_3, C_4, \dots, C_m)$, we obtain another serial decomposition, of the type shown in Fig. 12.14b, to which there corresponds another closed partition π_2 on the states of the machine M .

To determine the relation between π_1 and π_2 , note that, since C_1 distinguishes the blocks of π_1 , each block of π_1 in fact corresponds to a state of C_1 . Similarly, each block of π_2 corresponds to a state of the composite machine (C_1, C_2) . However, since (C_1, C_2) can be decomposed into C_1 in series with C_2 , it follows that each state of C_1 represents one or more states of the composite machine (C_1, C_2) . Consequently, each block of π_1 contains one or more blocks of π_2 , i.e., $\pi_1 \geq \pi_2$. There exist m possible ways (one of which is trivial) of arranging the component machines in two groups, $(C_1, \dots, C_k)$ and $(C_{k+1}, \dots, C_m)$. Hence, there exist m closed partitions $\pi_1 \geq \pi_2 \geq \dots \geq \pi_m$. Note that the equality sign in the above relation can be omitted, since it corresponds to a degenerate case. In fact, if $\pi_{k-1} = \pi_k$ then the component C_k is redundant and may be deleted.

The converse can be proved by illustrating the construction of the decomposed machine. Let $\pi_1 > \pi_2 > \dots > \pi_m$ be a set of closed partitions on M . Select another set of partitions, $\tau_1, \tau_2, \dots, \tau_{m-1}$, such that, for each value of i in the range $1 \leq i \leq m - 1$,

$$\pi_i \cdot \tau_i = \pi_{i+1}$$

and

$$\pi_1 \cdot \tau_1 \cdot \tau_2 \cdot \dots \cdot \tau_{m-1} = \pi(0).$$

The first component, C_1 , contains $\lceil \log_2 \#(\pi_1) \rceil$ state variables, which are assigned to distinguish the blocks of π_1 . Thus, C_1 is independent of the remaining components. The second component, C_2 , consists of the $\lceil \log_2 \#(\tau_1) \rceil$ variables assigned to the blocks of τ_1 . Since τ_1 is not necessarily closed, C_2 depends on C_1 . However, since $\pi_1 \cdot \tau_1 = \pi_2$, C_2 is independent of the remaining components $C_3, \dots, C_m$. In a similar manner, the decomposed machine is constructed in such a way that each component C_k is independent of $C_{k+1}, \dots, C_m$ and is a function of $C_1, \dots, C_k$. $\diamond$

Theorem 12.4 establishes the concept of *information flow* in a sequential machine, i.e., a machine realized as a serial connection of smaller components. In fact we have proved that, in the cascaded chain, information flows from component C_i to C_j if and only if $\pi_i \geq \pi_j$.

Example Consider the machine M_{10} given in Table 12.18. It has three closed partitions (including the zero partition) and an output-consistent partition λ_0 . Since $\pi_a > \pi_b > \pi_0$, M_{10} is decomposable into three components connected in series such that each component is a two-state machine:

Table 12.18 Machine M_{10}

PS	NS		z
	$x = 0$	$x = 1$	
A	G	D	1
B	H	C	0
C	F	G	1
D	E	G	0
E	C	B	1
F	C	A	0
G	A	E	1
H	B	F	0

$$\begin{aligned}\pi_0 &= \pi(0), \\ \pi_a &= \{\overline{A, B, G, H}; \overline{C, D, E, F}\}, \\ \pi_b &= \{\overline{A, B}; \overline{C, D}; \overline{E, F}; \overline{G, H}\}, \\ \lambda_0 &= \{\overline{A, C, E, G}; \overline{B, D, F, H}\}.\end{aligned}$$

The machine C_a , which is derived from π_a , consists of $\#(\pi_a) = 2$ states and, therefore, can be realized by a single state variable, y_a . The second component, C_b , is derived from a partition τ_1 such that $\pi_a \cdot \tau_1 = \pi_b$. One

possible such partition, τ_1 , is given by

$$\tau_1 = \{\overline{A, B, C, D}; \overline{E, F, G, H}\}.$$

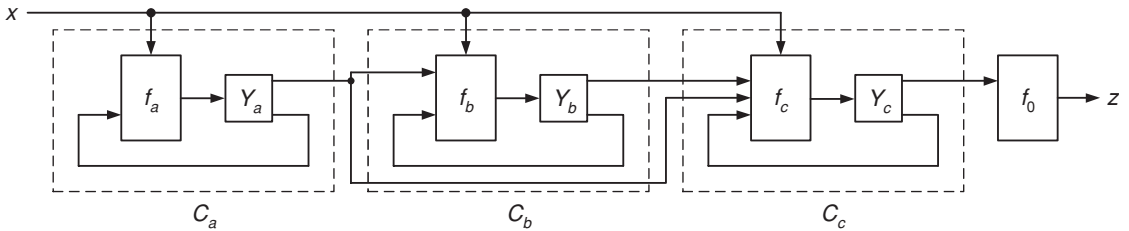
Since $\#(\tau_1) = 2$, the machine C_b will consist of a single variable, y_b . Variables y_a and y_b are actually assigned to the blocks of the closed partition π_b and, therefore, are independent of the remaining variable, which is assigned to the blocks of some partition τ_2 such that $\tau_2 \cdot \pi_b = \pi(0)$. Several partitions satisfy the last requirement. It is desirable, however, to select (whenever possible) a partition yielding simpler output circuits, i.e., for which $\tau_2 \leq \lambda_0$. A choice satisfying this condition is

$$\tau_2 = \lambda_0 = \{\overline{A, C, E, G}; \overline{B, D, F, H}\}.$$

An assignment based on the above partitions will yield the following functional relationships:

$$\begin{aligned} Y_a &= f_a(x, y_a), \\ Y_b &= f_b(x, y_a, y_b), \\ Y_c &= f_c(x, y_a, y_b, y_c), \\ z &= f_0(y_c). \end{aligned}$$

The schematic diagram of this realization and the π -lattice of M_{10} are shown in Fig. 12.15.



(a) Serial decomposition.



(b) π -lattice.

Fig. 12.15 Schematic diagram and π -lattice of M_{10} .

The machine M_{10} has thus been decomposed into three components connected in series. It is often necessary to determine the state table of each of these components, a task accomplished as follows. The state diagram of C_a is obtained by constructing the implication graph of π_a . It consists of two vertices,

Table 12.19 State tables of the component machines realizing M_{10}

x				$y_a x$						$PS \quad i_1 \quad i_2 \quad y_b$			
PS	0	1	y_a	PS	00	01	10	11	y_b	α	β	α	0
P	P	Q	0	α	β	α	β	β	0	β	α	β	1
Q	Q	P	1	β	α	β	α	α	1	(c) C_b – reduced form			
(a) C_a				(b) C_b									

$y_a y_b x$										$PS \quad I_1 \quad I_2 \quad I_3 \quad z$				
PS	000	001	010	011	100	101	110	111	z	γ	δ	γ	δ	1
γ	γ	δ	γ	γ	δ	γ	γ	δ	1	γ	δ	γ	γ	0
δ	δ	γ	δ	δ	γ	γ	γ	γ	0	(e) C_c – reduced form				
(d) C_c														

P and Q , corresponding respectively to the blocks $(ABGH)$ and $(CDEF)$. The state table of C_a , which is identical to the implication table derived from π_a , is given in Table 12.19a. The output of C_a is associated with its state and is identical to the value of y_a .

The inputs to C_b are x and y_a , and its state-dependent output is y_b . It contains two states, α and β , corresponding respectively to the blocks $(ABCD)$ and $(EFGH)$ of τ_1 . The state table of C_b is shown in Table 12.19b; the input symbol 00 means that C_a is in state P , i.e., $y_a = 0$, and that the external input value is $x = 0$. When C_a is in state P and C_b is in state α then M_{10} is in either state A or state B . From these states C_b goes to state β , which corresponds to G and H . When C_a is in state P , C_b is in state β , and the input value $x = 0$ is applied, C_b is to go to state α , which corresponds to states A and B in M_{10} . The entire table is completed in a similar fashion. The composite states of C_a and C_b correspond to the blocks of π_b . Since $\pi_a = \{P; Q\}$ and $\tau_1 = \{\alpha; \beta\}$,

$$\pi_b = \pi_a \cdot \tau_1 = \{P\alpha; P\beta; Q\alpha; Q\beta\} = \{\overline{A}, \overline{B}; \overline{G}, \overline{H}; \overline{C}, \overline{D}; \overline{E}, \overline{F}\}.$$

Finally, we note that C_b can be reduced to a machine with only two input symbols, since the next-state entries in three columns of C_b are identical. If we define i_1 and i_2 as

$$\begin{aligned} i_1 &= x' + y_a, \\ i_2 &= xy'_a \end{aligned}$$

we obtain the reduced form of C_b , as shown in Table 12.19c.

The machine C_c consists of two states, γ and δ , corresponding to the blocks of $\tau_2 = \{\overline{A}, \overline{C}, \overline{E}, \overline{G}; \overline{B}, \overline{D}, \overline{F}, \overline{H}\} = \{\gamma; \delta\}$, as shown in Table 12.19d. It receives three inputs, x , y_a , and y_b , and produces one output, z . The input symbol 000 in this table means that C_a and C_b are in states P and α , respectively, and

Table 12.20 Machine M_{11}

PS	NS		z
	$x = 0$	$x = 1$	
A	D	G	0
B	C	E	0
C	H	F	0
D	F	F	0
E	B	B	0
F	G	D	0
G	A	B	0
H	E	C	1

$x = 0$. This, in turn, implies that M_{10} is in either state A or B , depending on whether C_c is in state γ or δ , respectively. The 0-successors of A and B are G and H , which correspond to $P\beta\gamma$ and $P\beta\delta$, respectively. Therefore, the entries in column 000 of C_c are γ and δ . In a similar fashion, the state table of C_c is derived from Table 12.18 and set of partitions π_a , τ_1 , and τ_2 . By making the appropriate input assignment, Table 12.19d may be reduced to the form shown in Table 12.19e.

Parallel decompositions

We have already shown that a necessary and sufficient condition for a sequential machine M to be decomposable into two independent components operating in parallel is the existence of two closed partitions (or covers), π_1 and π_2 , such that $\pi_1 \cdot \pi_2 = \pi(0)$. This result can be easily generalized to a decomposition into m parallel components, which can be accomplished if and only if there exists a set of m closed partitions (or covers) on M such that $\pi_1 \cdot \pi_2 \cdot \dots \cdot \pi_m = \pi(0)$.

The machine M_{11} given in Table 12.20 has the π -lattice of Fig. 12.16a and the following nontrivial closed partitions:

$$\begin{aligned}\pi_a &= \{\overline{A, B}; \overline{C, D}; \overline{E, G}; \overline{F, H}\}, \\ \pi_b &= \{\overline{A, H}; \overline{B, F}; \overline{C, G}; \overline{D, E}\}, \\ \pi_c &= \{\overline{A, B, F, H}; \overline{C, D, E, G}\}.\end{aligned}$$

Since $\pi_a \cdot \pi_b = \pi(0)$, a parallel decomposition of M_{11} is possible. However, $\lceil \log_2 \#(\pi_a) \rceil + \lceil \log_2 \#(\pi_b) \rceil = 4$ and so such a decomposition requires four state variables. The state tables of the component machines M_a and M_b , which correspond respectively to π_a and π_b , are given in Table 12.21. The schematic diagram of the realization is shown in Fig. 12.16b.

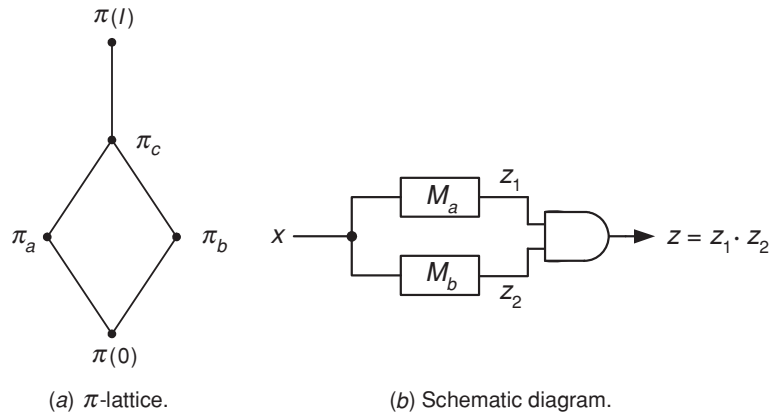
Since this realization requires four state variables, we next seek another decomposition, one which will require only three variables. Ultimately, our

Table 12.21 Parallel decomposition of M_{11}

NS					NS				
PS	$x = 0$		$x = 1$	z_1	PS	$x = 0$		$x = 1$	z_2
A, B	a	b	c	0	A, H	α	δ	γ	1
C, D	b	d	d	0	B, F	β	γ	δ	0
E, G	c	a	a	0	C, G	γ	α	β	0
F, H	d	c	b	1	D, E	δ	β	β	0

(a) M_a (b) M_b

Fig. 12.16 Parallel decomposition of M_{11} .



aim is to determine whether the machines M_a and M_b can each be serially decomposed in such a manner that both have an identical independent component. If such a component can be found, it may be “factored out” to serve as a common predecessor for both M_a and M_b . A necessary condition for the existence of such a common component is that both M_a and M_b can be serially decomposed; i.e., that both M_a and M_b have nontrivial closed partitions on their respective states. Clearly, the largest component machine that can be factored out is given by the smallest closed partition that is greater than π_a and π_b , i.e., $\text{lub } \pi_a + \pi_b$. For the machine M_{11} ,

$$\pi_c = \pi_a + \pi_b = \overline{\{A, B, F, H; C, D, E, G\}}.$$

Since $\text{lub } \pi_c$ is nontrivial, a two-state component can be factored out and thus a decomposition of the form shown in Fig. 12.17 is possible for M_{11} . The common factor M_c in series with M_d realizes M_a , while M_c in series with M_e realizes M_b . The factor M_c and the components M_d and M_e are given in Table 12.22.

Table 12.22 The component machines corresponding to Fig. 12.17

PS		x		y_c
		0	1	
A, B, F, H	P	Q	Q	0
C, D, E, G	Q	P	P	1

(a) M_c

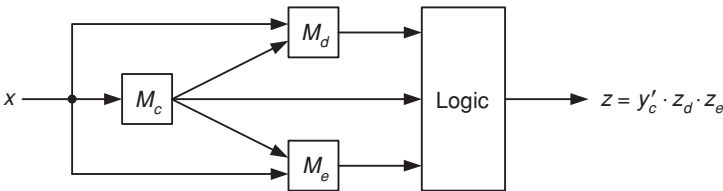
PS		$y_c x$				z_d
		00	01	10	11	
A, B, C, D	r	r	s	s	s	0
E, F, G, H	s	s	r	r	r	1

(b) M_d

PS		$y_c x$				z_d
		00	01	10	11	
A, C, G, H	u	v	u	u	v	1
B, D, E, F	v	u	v	v	v	0

(c) M_e

Fig. 12.17 Another decomposition of M_{11} .



Decompositions with specified components

We have studied several machine structures and determined the conditions for a machine to be decomposable into these structures. Our present objective is to determine whether a machine can be decomposed in such a manner that one (or more) of its components is specified. One possible approach to the solution of this problem is to check all closed partitions and covers and determine whether any of them yields the desired specified component. This approach, however, is long and impractical, and so a new technique to handle this type of decompositions will be developed.

As an example, consider the machines M_{12} and C_1 given in Tables 12.23 and 12.24, respectively. Our objective is to determine whether M_{12} can be serially

Table 12.23 Machine M_{12}

PS	NS		z	
	I_1	I_2	I_1	I_2
A	C	D	0	0
B	D	E	0	1
C	A	C	0	0
D	B	D	0	0
E	F	E	1	1
F	C	D	1	1

Table 12.24 Machine C_1

PS	NS	
	I_1	I_2
P	S	Q
Q	R	Q
R	S	Q
S	P	S

Table 12.25 Composite machine for M_{12} and C_1 and initial states A and P

PS	NS	
	I_1	I_2
AP	CS	DQ
CS	AP	CS
DQ	BR	DQ
BR	DS	EQ
DS	BP	DS
EQ	FR	EQ
BP	DS	EQ
FR	CS	DQ

decomposed in such a way that C_1 is the predecessor component. In order to determine whether such a decomposition is possible, it is necessary to establish what information regarding the states of M_{12} is contained in C_1 . This can be accomplished by constructing a composite machine that contains both M_{12} and C_1 and is defined as follows.

Let the *general composite machine*, corresponding to the two machines M_1 and M_2 , having sets of states R and S , respectively, be the machine that contains the set of states $R \times S$. We shall use $R_i S_j$ to denote the state of the general composite machine which corresponds to R_i in M_1 and (simultaneously) S_j in M_2 . For two machines M_1 and M_2 having simultaneous initial states R_1 and S_1 , the *composite machine* is that having initial state $R_1 S_1$ and subsequent states implied in chain fashion by $R_1 S_1$ and its successors.

The composite machine corresponding to the machines M_{12} and C_1 and to the initial states A and P respectively is given in Table 12.25. Starting with AP , the application of the input symbol I_1 takes M_{12} to state C and C_1 to state S . Therefore, the I_1 -successor of AP is CS . In a similar fashion, we conclude that the I_2 -successor of AP is DQ , and so on. Next, we obtain the successors of states CS and DQ , and this process continues until no new states are generated.

In general, if M_1 has n_1 states and M_2 has n_2 states, the general composite machine has $n_1 \cdot n_2$ states. However, it may have as many as $n_1 \cdot n_2$ states, or as few as the smaller of n_1 or n_2 states. The I_k -successor of state $R_i S_j$ of the composite machine is obtained from the I_k -successors of R_i and S_j in their respective machines, i.e., if $I_k R_i$ is R_p and $I_k S_j$ is S_q then the I_k -successor of $R_i S_j$ is $R_p S_q$.

For the machine M_{12} to be serially decomposable in such a way that C_1 is the predecessor component, it is necessary that M_{12} should have a closed cover whose corresponding implication graph is *equivalent* to the state diagram of C_1 ; i.e., both graphs must be isomorphic and the labels of the arcs connecting corresponding vertices must be identical. This closed cover can be obtained from the composite machine of Table 12.25 in a straightforward manner. From the names of the new states in this table, it can be concluded that when C_1 is in state P the composite machine can be in either state AP or state BP , and M_{12} can only be in A or B . Similarly, when C_1 is in state S , M_{12} can only be in state C or D , and so on. We can thus form a cover φ on the states of M_{12} such that two states (say R_i and R_j) are in the same block of φ if and only if they are associated with the same state of C_1 (say S_k); i.e., the composite machine of M_{12} and C_1 contains the states $R_i S_k$ and $R_j S_k$. Thus, for machine M_{12} , we have

$$\varphi = \{\overline{A, B}; \overline{D, E}; \overline{B, F}; \overline{C, D}\}.$$

Blocks (A, B) and (D, E) of φ correspond respectively to states P and Q in C_1 , while (B, F) and (C, D) correspond respectively to states R and S . Consequently, knowledge of the state of C_1 is always sufficient to obtain the state of M_{12} to within at most two states.

In order to complete the synthesis it is necessary to specify the successor component. A simple way to accomplish this is first to split states B and D of machine M_{12} in such a way that $\pi = \{\overline{A, B'}; \overline{D', E}; \overline{B'', F}; \overline{C, D''}\}$ is a closed partition on the states of the augmented machine. The predecessor component of the augmented machine is isomorphic to C_1 , while the successor component, which consists of two states, distinguishes the blocks of a partition τ given by

$$\tau \cdot \{\overline{A, B'}; \overline{D', E}; \overline{B'', F}; \overline{C, D''}\} = \pi(0).$$

One possibility is

$$\tau = \{\overline{A, D', D'', F}; \overline{B', B'', E, C}\}.$$

The state tables of the augmented machine and the successor component are obtained in the usual manner, as illustrated in the previous sections.

*12.9 Synthesis of multiple machines

We shall now generalize the decomposition problem to include the simultaneous decomposition of two or more machines. More precisely, given two reduced

Fig. 12.18 Two machines having a common predecessor component M_C .

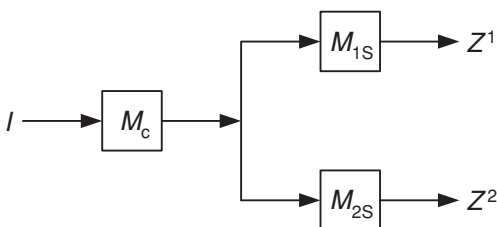


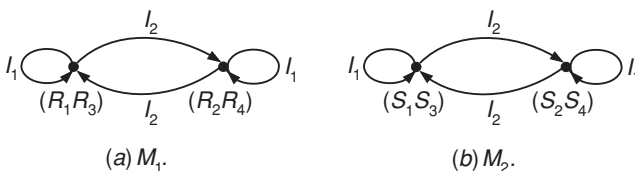
Table 12.26 Two machines, M_1 and M_2 , to be decomposed simultaneously

$N S$				$N S$			
R	I_1	I_2	Z^1	S	I_1	I_2	Z^2
R_1	R_1	R_2	Z_1^1	S_1	S_3	S_2	Z_1^2
R_2	R_2	R_3	Z_2^1	S_2	S_4	S_3	Z_2^2
R_3	R_3	R_4	Z_3^1	S_3	S_1	S_4	Z_3^2
R_4	R_4	R_1	Z_4^1	S_4	S_2	S_1	Z_4^2

(a) M_1

(b) M_2

Fig. 12.19 Implication graphs.



machines M_1 and M_2 having the same input alphabet I , which are initially in states R_1 and S_1 respectively, we wish to find three machines M_C , M_{1S} , and M_{2S} , where M_C is a common predecessor component whose output feeds the successors M_{1S} and M_{2S} in such a way that M_C and M_{1S} form a serial decomposition of M_1 while M_C and M_{2S} form a serial decomposition of M_2 . Figure 12.18 shows the desired structure, in which Z^1 and Z^2 are the outputs of M_{1S} and M_{2S} , respectively. When a maximum common predecessor component exists, the total state variables required for the realization is minimum, while the total output logic circuitry is not more complex than if the two machines were realized separately.

The common predecessor machine

As an example, consider the two reduced Moore-type machines given in Table 12.26. The implication graphs of machines M_1 and M_2 , for the initial identifications of $(R_1 R_3)$ and $(S_1 S_3)$ respectively, are shown in Fig. 12.19. These closed

Table 12.27 Machine M_C

PS	NS	
	I_1	I_2
P	P	Q
Q	Q	P

graphs are equivalent since they are isomorphic and the labels of arcs that connect corresponding vertices are identical. We have already established that the closed implication graph of a sequential machine M is actually equivalent to the state diagram of the predecessor component in a serial decomposition of M . Consequently, each graph in Fig. 12.19 can serve as a state diagram of the predecessor component in the serial decomposition of the respective machine. In addition, since the two graphs are equivalent they correspond to equivalent machines. Because the two predecessor components are equivalent, one may be removed and the other retained as the common predecessor component.

The graphs of Fig. 12.19 correspond respectively to the closed partitions

$$\pi_1 = \{\overline{R_1, R_3}, \overline{R_2, R_4}\} \quad \text{and} \quad \pi_2 = \{\overline{S_1, S_3}, \overline{S_2, S_4}\}.$$

If we denote the first and second blocks of each partition by P and Q respectively then we obtain the implication table in Table 12.27. This is the state table of the common predecessor component M_C . Successor components M_{1S} and M_{2S} can be obtained by using the methods developed in the foregoing section.

From the preceding example, it is evident that *a collection of two (or more) machines contains a common predecessor component M_C if and only if they possess equivalent implication graphs; the vertices and arcs of this common graph are in one-to-one correspondence with the states and state transitions respectively of M_C* . The procedure for finding the equivalent graphs is not entirely systematic, however, since it depends on the selection of the initial state identifications. This limitation can be overcome by using the composite machine, as is shown subsequently.

The composite machine corresponding to M_1 and M_2 and to initial states R_1 and S_1 is given in Table 12.28. It consists of eight states. While the composite machine includes all states of M_1 and M_2 , it does not include all combinations of these states; e.g., R_1S_2 is not encountered when any of the eight states of the composite machine is selected as the initial state. Furthermore, if M_1 is initially in state R_1 , then M_2 can be started only in either S_1 or S_3 , since the only combinations of states included in the composite machine are R_1S_1 and R_1S_3 . Thus, the choice of an initial state, in effect, locks the two machines together, in an operational sense.

Table 12.28 Composite machine for M_1 and M_2 and initial states R_1 and S_1

PS	NS		$Z^1 Z^2$
	I_1	I_2	
$R_1 S_1$	$R_1 S_3$	$R_2 S_2$	$Z_1^1 Z_1^2$
$R_1 S_3$	$R_1 S_1$	$R_2 S_4$	$Z_1^1 Z_3^2$
$R_2 S_2$	$R_2 S_4$	$R_3 S_3$	$Z_2^1 Z_2^2$
$R_2 S_4$	$R_2 S_2$	$R_3 S_1$	$Z_2^1 Z_4^2$
$R_3 S_3$	$R_3 S_1$	$R_4 S_4$	$Z_3^1 Z_3^2$
$R_3 S_1$	$R_3 S_3$	$R_4 S_2$	$Z_3^1 Z_1^2$
$R_4 S_4$	$R_4 S_2$	$R_1 S_1$	$Z_4^1 Z_4^2$
$R_4 S_2$	$R_4 S_4$	$R_1 S_3$	$Z_4^1 Z_2^2$

Using the above procedure, we have transformed the two-machine problem into the well-known single-machine decomposition problem. The methods developed in the preceding sections are now applicable to the composite machine which contains the two machines M_1 and M_2 .

Decomposing the composite machine

Let us now define two partitions, π_R and π_S , on the states of the composite machine such that two states are placed in the same block of π_R if and only if their labels start with the same state R_i in M_1 ; two states are placed in the same block of π_S if and only if their names end with the same state S_j in M_2 . Such partitions are often referred to as *state-consistent* partitions and are derived directly from the composite machine.

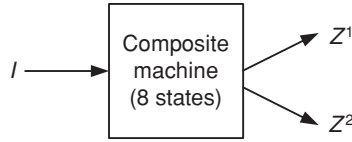
Example The state-consistent partitions for the composite machine of Table 12.28 are

$$\pi_R = \{\overline{R_1 S_1, R_1 S_3}, \overline{R_2 S_2, R_2 S_4}, \overline{R_3 S_3, R_3 S_1}, \overline{R_4 S_4, R_4 S_2}\},$$

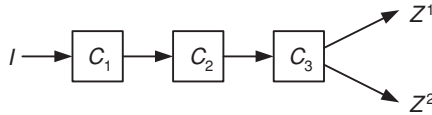
$$\pi_S = \{\overline{R_1 S_1, R_3 S_1}, \overline{R_2 S_2, R_4 S_2}, \overline{R_1 S_3, R_3 S_3}, \overline{R_2 S_4, R_4 S_4}\}.$$

The block $(R_1 S_1, R_1 S_3)$ of π_R corresponds to state R_1 in M_1 , the block $(R_1 S_1, R_3 S_1)$ of π_S corresponds to state S_1 in M_2 , and so on. From the way in which the state-consistent partitions π_R and π_S are constructed, it is evident that they correspond to the zero partitions on the set of states of the machines M_1 and M_2 , respectively. Consequently, the implication graphs corresponding to π_R and π_S are equivalent to the state graphs of M_1 and M_2 respectively; therefore these partitions are closed with respect to the states of the composite machine.

Fig. 12.20 Two possible realizations of the composite machine.



(a) Simple realization.



(b) Decomposition of the composite machine.

From the composite machine of Table 12.28, it is apparent that the required outputs Z^1 and Z^2 can be generated by a machine having three state variables and the appropriate output logic rather than by two separate machines having a total of four state variables. This result is illustrated in Fig. 12.20a. We also observe that $\pi_R \cdot \pi_S = \pi(0)$, which, since both partitions are closed, is the condition for a parallel decomposition of the composite machine. In this case of course the result is simply the original two machines, M_1 and M_2 , realized separately and having four state variables.

The composite machine is next examined for other possible decompositions, following the techniques previously developed. For example, the partitions

$$\pi_1 = \{\overline{R_1 S_1}, R_2 S_2, R_3 S_3, R_4 S_4; \overline{R_1 S_3}, R_2 S_4, R_3 S_1, R_4 S_2\}$$

and

$$\pi_2 = \{\overline{R_1 S_1}, R_3 S_3; \overline{R_2 S_2}, R_4 S_4; \overline{R_1 S_3}, R_3 S_1; \overline{R_2 S_4}, R_4 S_2\}$$

are easily shown to be closed and, since $\pi_1 > \pi_2$, a cascade realization of the type shown in Fig. 12.20b results, where each component, C_1 , C_2 , and C_3 , is a two-state machine.

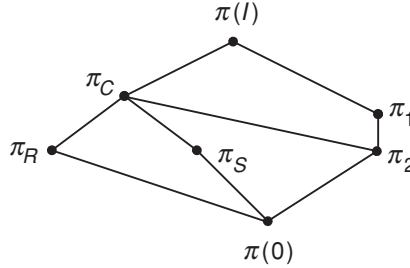
At this point, we turn our attention to the question of determining whether a common predecessor component exists for M_1 and M_2 and, if several such components exist, how to find the largest. From the results of the preceding section and from the properties of the composite machine and the state-consistent partitions π_R and π_S , it is evident that a common component exists if and only if we can find a closed partition π_C such that $\pi_C > \pi_R$ and $\pi_C > \pi_S$. Clearly, the smallest partition that satisfies these inequalities and, thus, yields the largest common component M_C , is

$$\pi_C = \pi_R + \pi_S.$$

For our example, we obtain

$$\pi_C = \pi_R + \pi_S = \{\overline{R_1 S_1}, \overline{R_1 S_3}, \overline{R_3 S_1}, \overline{R_3 S_3}; \overline{R_2 S_2}, \overline{R_2 S_4}, \overline{R_4 S_2}, \overline{R_4 S_4}\}.$$

Fig. 12.21 The π -lattice for the composite machine.



Thus, a common predecessor component consisting of one state variable exists. The resulting decomposition is shown in Fig. 12.18. It is easy to verify that this machine is identical to that obtained using the implication graphs (see Table 12.27). Successor machines M_{1S} and M_{2S} (each consisting of one state variable) are obtained by partitions τ_{1S} and τ_{2S} , respectively, such that

$$\pi_C \cdot \tau_{1S} = \pi_R \quad \text{and} \quad \pi_C \cdot \tau_{2S} = \pi_S.$$

Possible partitions are

$$\begin{aligned} \tau_{1S} &= \{\overline{R_1 S_1}, \overline{R_1 S_3}, \overline{R_2 S_2}, \overline{R_2 S_4}; \overline{R_3 S_1}, \overline{R_3 S_3}, \overline{R_4 S_2}, \overline{R_4 S_4}\}, \\ \tau_{2S} &= \{\overline{R_1 S_1}, \overline{R_3 S_1}, \overline{R_2 S_2}, \overline{R_4 S_2}; \overline{R_1 S_3}, \overline{R_3 S_3}, \overline{R_2 S_4}, \overline{R_4 S_4}\}. \end{aligned}$$

Clearly, Z^1 and Z^2 are each dependent upon only two state variables and the entire machine requires a total of three state variables.

The lattice of all closed partitions on the set of states of the composite machine is shown in Fig. 12.21. However, it is of interest that our two-machine cascade decomposition has been obtained without searching for closed partitions; π_R and π_S were obtained directly by inspection of the composite machine while π_C followed from the addition of the two partitions π_R and π_S . Thus, the process involves a minimum of computation or manipulation.

Notes and references

The structure theory of machines and the study of machine decomposition were originated by Hartmanis [5] in 1960 and further developed in a series of papers by Hartmanis [6], Stearns and Hartmanis [14], Karp [8], Yoeli [15, 16], and Kohavi [9, 10]. The concept of closed covers and the procedure for augmenting a machine by state splitting were introduced by Kohavi [9] and further developed to cover multiple machines by Kohavi and Smith [11] and Smith and Kohavi [13]. Other contributions to general machine-structure theory include Krohn and Rhodes [12], Zeiger [17], and Gill [4]. A comprehensive treatment of structure and decomposition theory can be found in the book by Hartmanis and Stearns [7].

The state-assignment problem has been treated from different points of views by many authors. Of particular interest are the papers by Armstrong [1, 2] and Dolotta and McCluskey [3].

- [1] Armstrong, D. B.: "A programmed algorithm for assigning internal codes to sequential machines," *IRE Trans. Electron. Computers*, vol. EC-11, no. 4, pp. 466–472, August 1962.
- [2] Armstrong, D. B.: "On the efficient assignment of internal codes to sequential machines," *IRE Trans. Electron. Computers*, vol. EC-11, no. 5, pp. 611–622, October 1962.
- [3] Dolotta, T. A., and E. J. McCluskey, Jr.: "The coding of internal states of sequential circuits," *IEEE Trans. Electron. Computers*, vol. EC-13, no. 5, pp. 549–562, October 1964.
- [4] Gill, A.: "Cascaded finite-state machines," *IRE Trans. Electron. Computers*, vol. EC-10, no. 3, pp. 366–370, September 1961.
- [5] Hartmanis, J.: "Symbolic analysis of a decomposition of information processing machines," *Information and Control*, vol. 3, no. 2, pp. 154–178, June 1960.
- [6] Hartmanis, J.: "On the state assignment problem for sequential machines I," *IRE Trans. Electron. Computers*, vol. EC-10, pp. 157–165, June 1961.
- [7] Hartmanis, J., and R. E. Stearns: *Algebraic Structure Theory of Sequential Machines*, Prentice-Hall, Englewood Cliffs NJ, 1966.
- [8] Karp, R. M.: "Some techniques of state assignment for synchronous sequential machines," *IEEE Trans. Electron. Computers*, vol. EC-13, no. 5, pp. 507–518, October 1964.
- [9] Kohavi, Z.: "Secondary state assignment for sequential machines," *IEEE Trans. Electron. Computers*, vol. EC-13, no. 3, pp. 193–203, June 1964.
- [10] Kohavi, Z.: "Reduction of output dependency in sequential machines," *IEEE Trans. Electron. Computers*, vol. EC-14, pp. 932–934, December 1965.
- [11] Kohavi, Z., and E. J. Smith: "Decomposition of sequential machines," in *Proc. Sixth Ann. Symp. Switching Theory and Logical Design*, Ann Arbor, Mich., October 1965.
- [12] Krohn, K. B., and J. L. Rhodes: "Algebraic theory of machines," in *Proc. Symp. Mathematical Theory of Automata*, Polytechnic Press, Brooklyn NY, 1962.
- [13] Smith, E. J., and Z. Kohavi: "Synthesis of multiple sequential machines," in *Proc. Seventh Ann. Symp. Switching and Automata Theory*, Berkeley CA, October 1966.
- [14] Stearns, R. E., and J. Hartmanis: "On the state assignment problem for sequential machines II," *IRE Trans. Electron. Computers*, vol. EC-10, no. 4, pp. 593–603, December 1961.
- [15] Yoeli, M.: "The cascade decomposition of sequential machines," *IRE Trans. Electron. Computers*, vol. EC-10, pp. 587–592, April 1961.
- [16] Yoeli, M.: "Cascade-parallel decompositions of sequential machines," *IEEE Trans. Electron. Computers*, vol. EC-12, no. 3, pp. 322–324, June 1963.
- [17] Zeiger, H. P.: "Loop-free synthesis of finite-state machines," MIT. Ph.D. thesis, Dept of Electrical Engineering, Cambridge MA, September 1964.

Problems

Problem 12.1. Show that every n -state machine has N distinct state assignments, where

$$N = \frac{(2^k - 1)!}{(2^k - n)!k!}, \quad k = \lceil \log_2 n \rceil.$$

Note that two assignments are said to be *distinct* if one cannot be obtained from the other by permuting or complementing the variables or by relabeling them.

Hint: Recall that k binary variables can be permuted in $k!$ ways and that there are 2^k ways of complementing them.

Problem 12.2

- (a) Given the machine shown in Table P12.2 and two assignments α and β , derive in each case the logic equations for the state variables and output function and compare the results.
- (b) Express explicitly in each case the dependency of the output and state variables.

Table P12.2

PS	NS		z		$y_1y_2y_3$		$y_1y_2y_3$	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$				
A	D	C	0	0	A	000	A	110
B	F	C	0	1	B	001	B	101
C	E	B	0	0	C	010	C	100
D	B	E	1	0	D	011	D	000
E	A	D	1	1	E	100	E	001
F	C	D	1	0	F	101	F	010
					Assignment α		Assignment β	

Problem 12.3. A six-state machine is said to have the five closed partitions shown below and *no* other closed partitions. Is this possible?

$$\pi_1 = \{\overline{A}, \overline{C}; \overline{B}; \overline{D}; \overline{E}, \overline{F}\}, \quad \pi_4 = \pi(0),$$

$$\pi_2 = \{\overline{A}, \overline{D}; \overline{B}, \overline{C}; \overline{E}; \overline{F}\}, \quad \pi_5 = \pi(I),$$

$$\pi_3 = \{\overline{A}, \overline{B}; \overline{C}, \overline{D}; \overline{E}, \overline{F}\}.$$

Problem 12.4. The machine shown in Table P12.4 has the following closed partitions:

$$\pi_1 = \{\overline{A}, \overline{C}, \overline{E}; \overline{B}, \overline{D}, \overline{F}\}, \quad \pi_2 = \{\overline{A}, \overline{F}; \overline{B}, \overline{E}; \overline{C}, \overline{D}\}.$$

Table P12.4

PS	NS		z
	$x = 0$	$x = 1$	
A	D	C	1
B	A	D	0
C	B	E	0
D	E	B	0
E	F	C	0
F	C	D	0

- (a) Find a state assignment that reduces the interdependencies of the state variables.
- (b) Derive the logic equations and show the circuit diagram when unit delays are used as memory elements.

Problem 12.5

- (a) Show that every closed partition is the sum of some *basic* partitions. (Recall that a *basic* partition $\pi_{S_i S_j}$ is the smallest closed partition containing $S_i S_j$ in one block.)
- (b) Use the result of (a) to show that the procedure outlined in Section 12.3 for the construction of the π -lattice indeed gives all the closed partitions.

Problem 12.6. Let λ_o and λ'_o be two output-consistent partitions on the set of states of a machine M . Prove that $\lambda_o + \lambda'_o$ and $\lambda_o \cdot \lambda'_o$ are also output-consistent partitions.

Problem 12.7

- (a) Let π be a closed partition on the set of states of a machine M . Prove that if π is also an output-consistent partition, i.e., $\pi \leq \lambda_o$, then M can be reduced to an equivalent machine that has only $\#(\pi)$ states. Conversely, if there are no closed partitions on M that are also output-consistent then M is in reduced form.
- (b) Demonstrate the above reduction procedure by first finding a closed partition that is also output-consistent for the machine shown in Table P12.7 and *then* reducing it.

Table P12.7

PS	NS		z
	$x = 0$	$x = 1$	
A	E	C	0
B	B	A	1
C	B	D	0
D	E	C	1
E	E	F	1
F	B	C	0

Problem 12.8. The incompletely specified machine in Table P12.8 has a nontrivial closed partition that is also input-consistent. Does it have an autonomous clock? If yes, show its state diagram; if no, explain why not.

Table P12.8

PS	NS		
	I_1	I_2	I_3
A	—	A	—
B	C	—	D
C	A	B	A
D	B	A	B

Problem 12.9. In each of the following sets of partitions, π_1 and π_2 designate closed partitions while λ_o and λ_i designate output-consistent and input-consistent partitions, respectively.

- (a) Construct the corresponding π -lattice for each case by obtaining all the necessary sums and products.
- (b) Show schematic diagrams, demonstrating in each case the possible machine decompositions that yield minimal interdependencies of state variables as well as of outputs.

$$\begin{aligned}
 \text{(i)} \quad \pi_1 &= \{\overline{A, B, E, F}; \overline{C, D, G, H}\}, & \lambda_o &= \{\overline{A, B, G, H}; \overline{C, D, E, F}\}, \\
 \pi_2 &= \{\overline{A, F, C, H}; \overline{B, D, E, G}\}, & \lambda_i &= \{\overline{A, C}; \overline{B, D}; \overline{E, G}; \overline{F, H}\}, \\
 \text{(ii)} \quad \pi_1 &= \{\overline{A, B}; \overline{C, D}; \overline{E, F}; \overline{G, H}\}, & \lambda_o &= \lambda_i, \\
 \pi_2 &= \{\overline{A, E}; \overline{B, F}; \overline{C, G}; \overline{D, H}\}, & \lambda_i &= \{\overline{A, B, C, D}; \overline{E, F, G, H}\}, \\
 \text{(iii)} \quad \pi_1 &= \{\overline{A, C, E, G}; \overline{B, D, F, H}\}, & \lambda_o &= \{\overline{A, C}; \overline{B, D}; \overline{E, G}; \overline{F, H}\}, \\
 \pi_2 &= \{\overline{A, G}; \overline{B, F}; \overline{C, E}; \overline{D, H}\}, & \lambda_i &= 1.
 \end{aligned}$$

Problem 12.10

- (a) For the machine shown in Table P12.10, find the π -lattice and obtain the input-consistent and output-consistent partitions.

Table P12.10

<i>PS</i>	<i>NS</i>		<i>z</i>	
	<i>x</i> = 0	<i>x</i> = 1	<i>x</i> = 0	<i>x</i> = 1
<i>A</i>	<i>D</i>	<i>C</i>	0	0
<i>B</i>	<i>C</i>	<i>D</i>	0	1
<i>C</i>	<i>E</i>	<i>F</i>	0	0
<i>D</i>	<i>F</i>	<i>F</i>	0	1
<i>E</i>	<i>G</i>	<i>H</i>	0	0
<i>F</i>	<i>H</i>	<i>G</i>	0	1
<i>G</i>	<i>B</i>	<i>A</i>	0	0
<i>H</i>	<i>A</i>	<i>B</i>	0	1

- (b) Show two assignments that result in autonomous clocks of different frequencies. In each case, determine the period of the clock and draw a schematic diagram indicating the interdependencies within the decomposed machine.

Problem 12.11

- (a) For the machine shown in Table P12.11, find λ_i and λ_o and construct the π -lattice.
- (b) Choose as a basis for your state assignment three partitions, τ_1 , τ_2 , and τ_3 (which may or may not be closed), such that the following functional dependencies result:

$$\begin{aligned}
 Y_1 &= f_1(y_1), \\
 Y_2 &= f_2(x, y_2, y_3), \\
 Y_3 &= f_3(x, y_2, y_3), \\
 z &= f_0(y_1, y_2).
 \end{aligned}$$

Specify the desired relationship between the chosen τ 's and λ_o and λ_i , and show a schematic diagram of the resulting structure.

- (c) From the chosen τ 's, obtain a state assignment and derive the corresponding logic equations.

Table P12.11

<i>PS</i>	<i>NS</i>		<i>z</i>
	<i>x</i> = 0	<i>x</i> = 1	
<i>A</i>	<i>F</i>	<i>D</i>	0
<i>B</i>	<i>D</i>	<i>E</i>	0
<i>C</i>	<i>E</i>	<i>F</i>	0
<i>D</i>	<i>A</i>	<i>B</i>	0
<i>E</i>	<i>B</i>	<i>C</i>	0
<i>F</i>	<i>C</i>	<i>A</i>	1

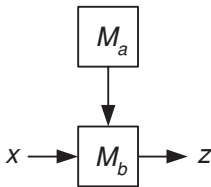


Fig. P12.12

Problem 12.12

- (a) Find a state assignment for the machine shown in Table P12.12 such that it will have the structure shown in Fig. P12.12.

Table P12.12

<i>PS</i>	<i>NS</i>		<i>z</i>	
	<i>x</i> = 0	<i>x</i> = 1	<i>x</i> = 0	<i>x</i> = 1
<i>A</i>	<i>D</i>	<i>B</i>	0	0
<i>B</i>	<i>A</i>	<i>C</i>	1	0
<i>C</i>	<i>B</i>	<i>E</i>	1	0
<i>D</i>	<i>F</i>	<i>A</i>	0	1
<i>E</i>	<i>F</i>	<i>C</i>	0	0
<i>F</i>	<i>E</i>	<i>D</i>	0	1

- (b) Obtain the logic equations for the output function and state variables.
(c) Show the state diagram of the input-independent component.

Problem 12.13

- (a) Find the π -lattice of the machine *M* shown in Table P12.13, and specify all the possible ways of decomposing the machine.

Table P12.13

<i>PS</i>	<i>NS</i>	
	<i>x</i> = 0	<i>x</i> = 1
<i>A</i>	<i>B</i>	<i>C</i>
<i>B</i>	<i>C</i>	<i>D</i>
<i>C</i>	<i>D</i>	<i>C</i>
<i>D</i>	<i>E</i>	<i>B</i>
<i>E</i>	<i>D</i>	<i>A</i>

- (b) Identify the states (A, B) and construct the implication graph. Augment the machine accordingly.
- (c) Describe all the possible ways of decomposing the augmented machine M' . Specify in each case the dependencies of state variables.

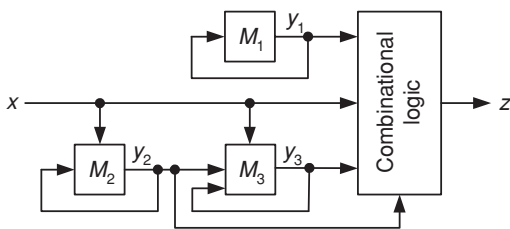
Problem 12.14. The machine shown in Table P12.14 has the closed partition $\pi = \{A, C, D, F; B, E, G\}$.

- (a) Can you find another closed partition such that a parallel decomposition is possible, without increasing the number of state variables?
- (b) Construct an implication graph, starting with the vertex (A, B), and show that there exists a machine M' , equivalent to M , that can be decomposed into the form shown in Fig. P12.14.

Table P12.14

PS	NS, z	
	$x = 0$	$x = 1$
A	$F, 1$	$C, 0$
B	$E, 0$	$B, 1$
C	$D, 0$	$C, 0$
D	$F, 1$	$C, 1$
E	$G, 0$	$B, 0$
F	$A, 1$	$F, 1$
G	$E, 1$	$G, 0$

Fig. P12.14



- (c) Show the state tables of the component machines.
- (d) Select an assignment that will lead to the structure of Fig. P12.14. Derive the corresponding logic equations.

Problem 12.15

- (a) Prove that if τ is a partition on C_1 then

$$M\{m[M(\tau)]\} = M(\tau) \quad \text{and} \quad m\{M[m(\tau)]\} = m(\tau).$$

- (b) Use the above to show that, for the partition τ of C_1 ,

$$\{M(\tau), m[M(\tau)]\} \quad \text{and} \quad \{M[m(\tau)], m(\tau)\}$$

are Mm pairs.

Problem 12.16. This problem is concerned with establishing a number of algebraic properties of Mm pairs and demonstrating that the set of all Mm pairs on a machine forms a lattice under the ordering defined in the text.

- (a) Show that if $\lambda = M(\lambda')$ and $\tau = M(\tau')$ then $\lambda \cdot \tau = M(\lambda' \cdot \tau')$.
- (b) Show that if $\lambda' = m(\lambda)$ and $\tau' = m(\tau)$ then $\lambda' + \tau' = m(\lambda + \tau)$.
- (c) Prove that if (λ, λ') and (τ, τ') are Mm pairs then their glb and lub are given by

$$\text{glb}\{(\lambda, \lambda'), (\tau, \tau')\} = [\lambda \cdot \tau, m(\lambda \cdot \tau)]$$

and

$$\text{lub}\{(\lambda, \lambda'), (\tau, \tau')\} = [M(\lambda' + \tau'), \lambda' + \tau'].$$

Problem 12.17. Find the set of all Mm pairs for the machine M_8 (Table 12.15) and draw its Mm -lattice.

Problem 12.18

- (a) Obtain the set of all Mm pairs for the machine shown in Table P12.18 and draw the corresponding Mm -lattice.
- (b) Show a state assignment that results in the following functional dependencies:

$$\begin{aligned} Y_1 &= f_1(x_1, x_2, y_1), \\ Y_2 &= f_2(x_1, x_2, y_2, y_3), \\ Y_3 &= f_3(x_1, x_2, y_1, y_2, y_3). \end{aligned}$$

Table P12.18

NS					
PS	x_1x_2				z
	00	01	10		
A	C	B	D		0
B	A	E	C		0
C	E	B	D		0
D	C	C	E		0
E	E	D	B		1

Problem 12.19

- (a) Find all the m -partitions for the machine shown in Table P12.19.

Table P12.19

NS					
	x_1x_2				
PS	00	01	11	10	z
A	A	A	D	A	1
B	C	C	D	A	0
C	D	A	A	A	0
D	B	A	D	B	0
E	E	C	A	B	0

- (b) Select a number of m -partitions and find their corresponding M -partitions, such that they yield an assignment in which every variable depends on just one variable and the external input.
- (c) Draw a schematic diagram of the resulting machine structure.

Problem 12.20. Construct an arbitrary machine with five or six states and three or four input symbols such that there exists at least one assignment that causes each state variable to be dependent only on the other variables and independent of itself, that is, Y_1 is independent of y_1 , etc.

Problem 12.21. The machine shown in Table P12.21 can be serially decomposed into three components without any increase in the number of state variables.

- (a) Determine the period of the maximal autonomous clock.
- (b) Select a set of partitions which induces an assignment such that the above serial decomposition is accomplished and the output logic is minimized.
- (c) Show the state table of each component.

Table P12.21

PS	NS		z	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
A	D	C	0	0
B	C	D	0	1
C	E	F	0	0
D	F	F	0	1
E	G	H	0	0
F	H	G	0	1
G	B	A	0	0
H	A	B	0	1

Problem 12.22. The machine shown in Table P12.22 has the following partitions:

$$\begin{aligned}\pi_1 &= \{\overline{A}, \overline{B}, \overline{C}; \overline{D}, \overline{E}, \overline{F}\}, & \lambda_o &= \{\overline{A}, \overline{D}, \overline{E}; \overline{B}, \overline{C}, \overline{F}\}, \\ \pi_2 &= \{\overline{A}, \overline{F}; \overline{B}, \overline{E}; \overline{C}, \overline{D}\}, & \lambda_i &= \{\overline{A}, \overline{C}; \overline{B}; \overline{D}, \overline{F}; \overline{E}\}.\end{aligned}$$

Table P12.22

PS	NS		NS	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
A	E	E	0	0
B	D	F	0	1
C	F	D	0	1
D	A	C	0	0
E	C	A	0	0
F	B	B	0	1

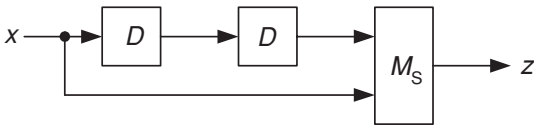
- (a) Draw a schematic diagram of the machine's structure induced by these partitions.
- (b) Show complete state tables for the component machines.

Problem 12.23. The machine of Table P12.23 is to be realized in the form shown in Fig. P12.23, where each block designated D represents a pure delay without internal feedback. Find a state table for a successor machine M_S such that the number of state variables and the functional complexity of the output are minimized.

Table P12.23

PS	NS		NS	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
A	E	A	0	0
B	D	B	0	1
C	D	B	0	0
D	F	C	1	1
E	E	C	1	0
F	F	B	1	1

Fig. P12.23



Problem 12.24. Prove that if two machines M_1 and M_2 are reduced then, for specified initial states, the composite machine is also reduced.

- Problem 12.25.** The machine M_1 shown in Table P12.25 is to be realized in a cascade form, with a machine M_2 as the predecessor component. The starting states are A and P .
- (a) Show the state table of an appropriate successor component.
 - (b) Choose a state assignment for M_1 that preserves the above structure and, at the same time, minimizes the complexity of the output function.
 - (c) Derive the logic equations for the state variables and output function.

Table P12.25

PS	NS		z		PS	NS	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$		$x = 0$	$x = 1$
A	B	E	0	1	P	R	Q
B	D	C	1	1	Q	R	P
C	G	C	0	0	R	S	Q
D	E	F	0	0	S	Q	S
E	B	A	0	1	M_2		
F	C	D	1	1			
G	F	E	0	0			
M_1							

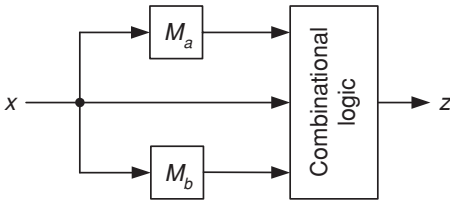
Problem 12.26. The machine M of Table P12.26 is to be realized in the form of Fig. P12.26. The state transitions of the component M_a are specified as shown. The starting state of M is A and that of M_a is G . Find the state table of M_b and specify the combinational logic that generates z .

Table P12.26

NS, z			NS		
PS	$x = 0$	$x = 1$	PS	$x = 0$	$x = 1$
A	$B, 0$	$C, 0$	G	H	G
B	$C, 0$	$D, 1$	H	G	H
C	$D, 1$	$E, 1$			
D	$E, 0$	$F, 1$	M_a		
E	$F, 1$	$A, 0$			
F	$A, 1$	$B, 1$			

M

Fig. P12.26



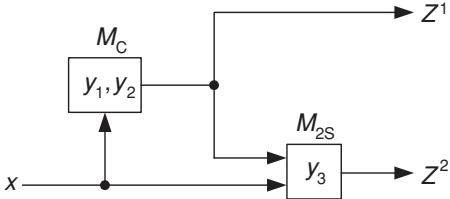
Problem 12.27. The machines M_1 and M_2 of Table P12.27 can be jointly realized in the form shown in Fig. P12.27, with only three state variables.

Table P12.27

NS				NS			
PS	$x = 0$	$x = 1$	Z^1	PS	$x = 0$	$x = 1$	Z^2
P	Q	R	0	A	B	D	0
Q	P	Q	0	B	E	C	0
R	Q	P	1	C	A	B	0
M_1				D	B	A	1
				E	C	E	1

M_2

Fig. P12.27



- (a) Construct a composite machine from M_1 and M_2 when the initial states are P and A for M_1 and M_2 , respectively.
- (b) Show the state tables for M_C and M_{2S} . Use the state names $S_1, S_2, \dots$ and $R_1, R_2, \dots$, etc.
- (c) Show the logic equations for the outputs.

Problem 12.28. Consider the machines M_1 and M_2 shown in Table P12.28. Their starting states are R_1 and S_1 , respectively.

- (a) Find the π -lattice for each machine and determine whether a common predecessor machine exists.
- (b) Show that if the state S_2 is split into S'_2 and S''_2 , a common predecessor can be found.
- (c) Realize the two machines in the form shown in Fig. 12.18. Show the state tables of the predecessor and successor machines.

Table P12.28

PS	NS		Z^1	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
R_1	R_2	R_4	1	0
R_2	R_1	R_3	0	1
R_3	R_1	R_4	0	1
R_4	R_2	R_3	1	0

M_1

PS	NS		Z^2	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
S_1	S_1	S_3	0	0
S_2	S_1	S_2	0	1
S_3	S_2	S_3	1	1

M_2

Problem 12.29. The disjoint realization of machines M_1 and M_2 shown in Table P12.29, requires six state variables. Find another realization for these machines that requires just four state variables and has the form shown in Fig. P12.29. Assume that

Table P12.29

NS				NS			
PS	$x = 0$	$x = 1$	Z^1	PS	$x = 0$	$x = 1$	Z^2
S_1	S_6	S_3	0	Q_1	Q_3	Q_4	0
S_2	S_5	S_2	0	Q_2	Q_4	Q_5	0
S_3	S_4	S_3	0	Q_3	Q_1	Q_3	0
S_4	S_6	S_2	0	Q_4	Q_2	Q_4	0
S_5	S_7	S_2	0	Q_5	Q_6	Q_5	0
S_6	S_1	S_6	0	Q_6	Q_3	Q_4	1
S_7	S_5	S_7	1				

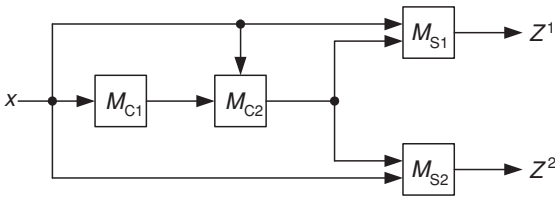
M_1

M_2

states S_1 and Q_1 are the initial states. Show the state table of each component and indicate the functional dependencies of the outputs.

Hint: You may find it necessary to split some states.

Fig. P12.29



Problem 12.30. Repeat Problem 12.29 for the machine M_1 shown in Table P12.30 and the machine M_2 shown in Table P12.29.

Hint: It is quite straightforward to find a common-factor machine that has two states. However, if you construct the composite machine for M_1 and M_2 and draw its implication graphs for the initial identifications $(S_1 Q_1, S_2 Q_1)$ and $(S_1 Q_1, S_1 Q_2)$, you can show that a common-factor machine that has four states can be found, while each of the successors has only two states.

Table P12.30

PS	NS		Z^1
	$x = 0$	$x = 1$	
S_1	S_5	S_4	1
S_2	S_5	S_3	0
S_3	S_1	S_3	0
S_4	S_2	S_4	0
S_5	S_2	S_5	1

13

State-identification experiments and testing of sequential circuits

In this chapter, we shall be concerned with experimental analysis of the behavior of finite-state machines, test generation for sequential circuits, design for testability, and built-in self-test (BIST).

A machine will be assumed to be reduced, strongly connected, and completely specified. State-identification experiments are designed to identify the unknown initial state of the machine and, whenever such an identification is unnecessary or impossible, to identify the final state of the machine. These experiments are known as distinguishing and homing experiments, respectively. Machine-identification experiments are concerned with the problem of determining whether a given n -state machine is distinguishable from all other n -state machines. This problem is shown to be, under certain conditions, equivalent to the problem of determining whether a given machine is operating correctly.

Test generation methodologies will be presented for sequential circuits under two fault models: functional and stuck-at. A *functional fault* alters the machine's state table. A *stuck-at fault* is manifested as a permanent 0, i.e., a stuck-at-0 ($s-a-0$) fault, or as a permanent 1, i.e., a stuck-at-1 ($s-a-1$) fault on some line in the circuit, as discussed in Chapter 8. Since there is no direct way to control the present state lines of a sequential circuit or observe its next state lines, sequential test generation is a difficult task. To ease the testing burden, one can use design-for-testability methods, such as scan design, to allow the control and observation of state lines. Another way to reduce the testing burden is to allow the circuit to test itself through the BIST method.

13.1 Experiments

The application of an input sequence to the input terminals of a machine is referred to as an *experiment* on the machine. An experiment designed to take the machine through all its transitions, in such a way that a definite conclusion can be reached as to whether the machine is operating correctly, is said to be a *checking experiment*. At the beginning of an experiment, the machine is said to

be in an *initial* (or *starting*) state and at the end of an experiment the machine is said to be in a *final* state. It is customary to distinguish between two types of experiments:

1. *simple experiments*, which are performed on a single copy of the machine;
2. *multiple experiments*, which are performed on two or more identical copies of the machine.

In practice, most machines are available in just a single copy, and therefore simple experiments are preferable to multiple ones.

Experiments are classified according to their performance as:

1. *adaptive experiments*, in which the input symbol at any instant of time depends on the previous output symbols;
2. *preset experiments*, in which the entire input sequence is predetermined independently of the outcome of the experiment.

Since preset experiments are simpler to perform in today's technology, we shall focus on such experiments.

A measure of the efficiency and cost of an experiment is its *length*, which is the total number of input symbols applied to the machine during the execution of the experiment.

In Chapter 10 we studied the properties of experiments used to distinguish between two nonequivalent states, S_i and S_j , of an n -state machine. We showed that if S_i and S_j are distinguishable then they can be distinguished by an experiment of length at most $n - 1$. We now consider more general problems, that of identifying the initial or final state of a given machine and that of distinguishing a given n -state machine from all other n -state machines that have the same input and output alphabets.

Introductory example

Consider the machine M_1 (Table 13.1), which may initially be in any of the states A , B , C , or D . The responses of M_1 to the input sequences 01 and 111 are listed in Table 13.2. Knowing the output sequence that M_1 produces in response to input sequence 01 is always sufficient to determine uniquely M_1 's *final* state, since each of the output sequences that might result from the application of 01 is associated with just one final state. For example, output sequence 00 indicates that the final state is B , while output sequences 11 or 01 indicate that the final state is D or A , respectively. On the other hand, the knowledge of the response of M_1 to input sequence 01 is not sufficient to determine M_1 's *initial* state, since the production of output sequence 00 could mean that the initial state was A or that it was B . In fact, if M_1 was initially in either state A or B , it is impossible to determine the initial state by an experiment which starts with a 0, since the 0-successors of both A and B are C , and the output symbol

Table 13.1 Machine M_1

PS	NS, z	
	$x = 0$	$x = 1$
A	$C, 0$	$D, 1$
B	$C, 0$	$A, 1$
C	$A, 1$	$B, 0$
D	$B, 0$	$C, 1$

Table 13.2 Responses of M_1 to the input sequences 01 and 111

Initial state	Response to 01	Final state	Initial state	Response to 111	Final state
A	00	B	A	110	B
B	00	B	B	111	C
C	11	D	C	011	D
D	01	A	D	101	A

(a)
(b)

produced in both cases is 0. No sequence following the initial 0 input symbol will yield any new information regarding the initial state.

Using the same line of argument, it is evident that the output sequence that M_1 produces in response to input sequence 111 is always sufficient to determine uniquely M_1 's final state, as well as its initial state. As shown in Table 13.2, each of the output sequences that might result from the application of 111 to M_1 is associated with just one initial state and one final state.

Before presenting techniques to be used in the design of experiments, we shall introduce some terminology and define the successor tree, which will prove to be an effective tool in the design of minimal experiments.

Uncertainties

Suppose that a machine M , which is given to the experimenter, can initially be in any of its n states. In such a case, we say that the initial uncertainty regarding the state of the machine is given by $(S_1 S_2 \cdots S_n)$. Thus, the *initial uncertainty* is the minimal subset of S (including S itself) that is known to contain the initial state. For example, if the machine M_1 can initially be in any of its four states then the initial uncertainty is $(ABCD)$.

Our aim is to perform experiments that reduce the initial uncertainty and, whenever possible, reveal the initial or final state. For example, suppose that we apply an input symbol 1 to machine M_1 and that in response it produces the output symbol 0. We may conclude that M_1 was initially in state C , since only from that state is a response of 0 to input symbol 1 possible. The final state in this case is B . However, suppose the response of M_1 to input symbol 1 is 1;

then all we can say regarding the final state of the machine is that it may be any of the states D , A , or C , depending on whether the initial state was A , B , or D , respectively. The set of states (ACD) thus represents the uncertainty regarding the final state of M_1 after the application of the input symbol 1. In general, the *uncertainty* regarding the state of M after the application of X is a specific subset of the X -successors of the states contained in the initial uncertainty. The elements of the uncertainty are not necessarily distinct.

Let U_0 be the initial uncertainty, and let input symbol I_i result in an uncertainty U_i ; then U_i is said to be the I_i -successor of U_0 . Suppose, for example, that the initial uncertainty regarding the state of M_1 is (ACD) . If an input symbol 1 is now applied to M_1 , the successor uncertainty will be (B) or (CD) , depending on whether the output symbol is 0 or 1, respectively. We thus say that the uncertainties (B) and (CD) are the 1-successors of (ACD) . Subsequently, we shall refer to a collection of uncertainties as an *uncertainty vector*. The individual uncertainties contained in the vector are called the *components* of the vector. An uncertainty vector whose components contain a single state each is said to be a *trivial uncertainty vector*. An uncertainty vector whose components contain either single states or identical repeated states is said to be a *homogeneous uncertainty vector*. Thus, for example, the vectors $(AA)(B)(C)$ and $(A)(B)(A)(C)$ are homogeneous and trivial, respectively.

The successor tree

The *successor tree*, which is defined for a specified machine M and a given initial uncertainty, displays graphically the I_i -successor uncertainties for all I_i and thus assists the experimenter in the selection of the most suitable input sequence. It is composed of branches arranged in successive *levels*, numbered $0, 1, \dots, j, \dots$. Each branch in the j th level splits into p branches, labeled $I_1, I_2, \dots, I_p$, corresponding to the input symbols of the machine. The branches emanating from the j th level form the $(j + 1)$ th level, and so on. Each node of the successor tree is associated with an uncertainty vector. The highest node (in level 0) is associated with initial uncertainty U_0 , and each of the p nodes in level 1 is associated with a successor of U_0 . The j th level of the tree consists of p^j branches, each terminating at a node. A sequence of j branches, starting at the highest node and terminating at a node in the j th level, is referred to as a *path* in the tree; j is called the *length* of the path. Each path *describes* an input sequence which, when applied to the machine, results in the uncertainty vector associated with the terminal node in the j th level. Hence, a tree with $j + 1$ levels contains p^j paths, describing the p^j input sequences of length j .

The successor tree for the machine M_1 and an initial uncertainty $(ABCD)$ is shown in Fig. 13.1. It contains four levels numbered 0 through 3. Each branch is labeled with the input symbol that it represents, and every node is associated with the corresponding uncertainty vector. The highest node is associated with

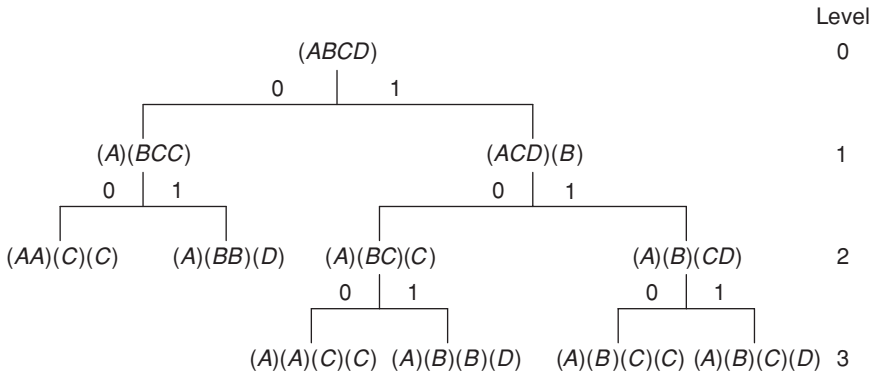


Fig. 13.1 Successor tree for M_1 .

the initial uncertainty while the nodes in level 1 are associated with its 1- and 0-successors, and so on. For example, an input symbol 1 applied to M_1 when the initial uncertainty is $(ABCD)$ results in the uncertainty vector $(ACD)(B)$, while an input symbol 0 results in the uncertainty vector $(A)(BCC)$. The 1-successor of the vector $(ACD)(B)$ is determined by obtaining the 1-successors of (ACD) and (B) separately. For example, the 1-successor of (B) is (A) , since the application of an input symbol 1 to M_1 , when in state B , takes it to state A . The 1-successor of (ACD) , however, depends on the output symbol; it is (CD) if the output symbol is 1, and (B) if it is 0. Thus, the corresponding uncertainty vector is $(A)(B)(CD)$. Similarly, the 0-successor of $(ACD)(B)$ is $(A)(BC)(C)$, since the 0-successor of (B) is (C) while that of (ACD) is $(A)(BC)$.

An uncertainty is said to be *smaller* than another uncertainty if it contains fewer elements; e.g., (BC) is smaller than (ACD) . From the way in which the tree is constructed, it is evident that an uncertainty associated with a node in the j th level is either smaller than or contains the same number of elements as its predecessor in the $(j - 1)$ th level. A homogeneous uncertainty vector will always have as its successors homogeneous uncertainty vectors. For example, in the tree of machine M_1 the successors of the uncertainty (BCC) are $(AA)(C)$ and $(A)(BB)$. The tree may be continued as far as is necessary but, for it to be of practical value, a truncated version must be defined by stipulating a number of termination rules.

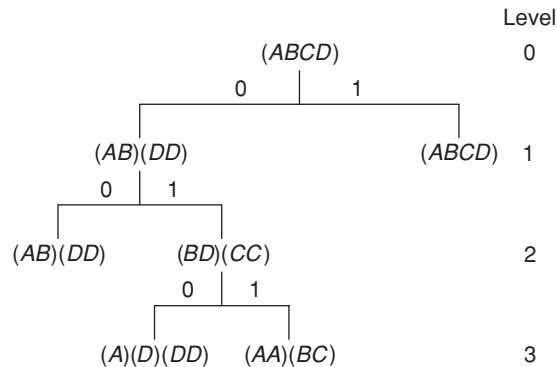
13.2 Homing experiments

The objective of this section is to develop techniques for the construction of experiments to identify the final state of a given n -state machine. It is shown that such experiments can be constructed for every reduced machine, and bounds on their lengths are derived.

Table 13.3 Machine M_2

PS	NS, z	
	$x = 0$	$x = 1$
A	$B, 0$	$D, 0$
B	$A, 0$	$B, 0$
C	$D, 1$	$A, 0$
D	$D, 1$	$C, 0$

Fig. 13.2 Homing tree for M_2 .



Definition 13.1 An input sequence Y_0 is said to be a *homing sequence* if the final state of the machine can be determined uniquely from the machine's response to Y_0 , regardless of the initial state.

The homing tree

A homing sequence for a given machine M may be obtained from a truncated version of its successor tree. Our task is to construct the tree and obtain the shortest path leading from the initial uncertainty to a trivial uncertainty or a homogeneous uncertainty. The presence of such an uncertainty at the k th level of the tree guarantees that there exists an input sequence consisting of k symbols whose application to M is sufficient to specify uniquely M 's final state.

A *homing tree* is a successor tree in which a j th-level node becomes terminal when either of the following occur:

1. the node is associated with an uncertainty vector whose nonhomogeneous components are associated with some node in a preceding level;
2. some node in the j th level is associated with a trivial or homogeneous vector.

The homing tree of a machine M_2 (Table 13.3) is shown in Fig. 13.2. The node associated with the vector $(AB)(DD)$ in level 2 is a terminal node, since its predecessor in level 1 is also associated with vector $(AB)(DD)$.

Table 13.4 The response of M_2 to the homing sequence 010

Initial state	Response to 010	Final state
A	000	A
B	001	D
C	101	D
D	101	D

Similarly, the node $(ABCD)$ in level 1 is terminated, since it is identical with the node $(ABCD)$ in level 0. The nodes in level 3 are also terminal nodes, since $(A)(D)(DD)$ is a homogeneous uncertainty vector. The shortest homing sequence is 010, since it is the shortest sequence described by a path leading from the zeroth level to a homogeneous uncertainty. The response and final states corresponding to this sequence are given in Table 13.4.

We shall now establish the existence of the homing experiment and derive a bound on its length.

Theorem 13.1 *A preset homing sequence, whose length is at most $(n - 1)^2$, exists for every reduced n -state machine M .*

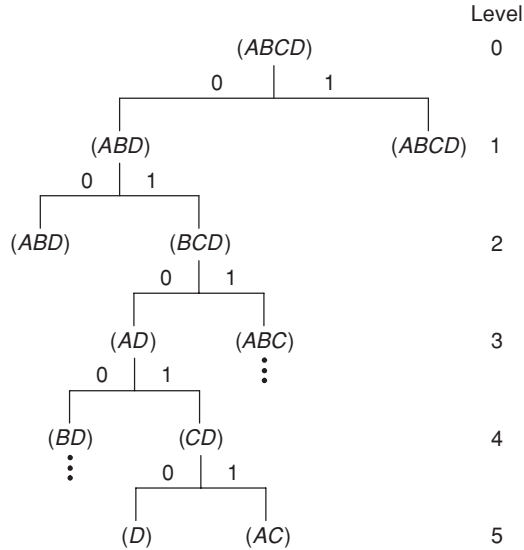
Proof Let the initial uncertainty be $(S_1 S_2 \cdots S_n)$. Since M is reduced, for every pair of states S_i, S_j there exists an experiment (i.e., a sequence) of length $n - 1$ or shorter that distinguishes S_i from S_j . Let us denote this experiment as λ_k . Starting at the initial uncertainty, application of sequence λ_1 , which distinguishes between some pair of states in M , yields the λ_1 -successor uncertainty vector, which contains at least two components. Next, we select any two states in one component and apply the appropriate sequence λ_2 , which distinguishes between them. The $\lambda_1 \lambda_2$ -successor uncertainty vector contains at least three components. In a similar manner, we obtain the $\lambda_1 \lambda_2 \cdots \lambda_{n-1}$ -successor vector, which consists of n components, each containing only one state. Therefore, the sequence $\lambda_1 \lambda_2 \cdots \lambda_{n-1}$ is a homing sequence whose length is at most $(n - 1)^2$. $\diamond$

This value is an upper bound on the length of the homing sequence, but is not the least upper bound. It can be shown that the length of the homing sequence need not exceed $\frac{1}{2}n(n - 1)$ and that this is indeed a tight bound (see Problem 13.5).

Synchronizing experiments

A *synchronizing sequence* of a machine M is a sequence that takes M to a specified final state, regardless of the output symbols or initial state. Some machines possess such sequences; others do not.

Fig. 13.3 Synchronizing tree for M_2 .



For a given machine, we can construct a successor tree by ignoring the output symbols and associating with every node in the j th level the uncertainty regarding the final state resulting from the application of the first j input symbols. For example, if the initial uncertainty of the machine M_2 is $(ABCD)$ then the 0-successor uncertainty is (ABD) , and so on. Note that, since we are interested only in the final state regardless of the output symbols, it is not necessary to write down repeated entries; e.g., $(ABDD)$ may be simply written as (ABD) , etc. A j th-level node in the tree becomes terminal whenever either of the following occurs:

1. the node is associated with an uncertainty that is also associated with some node in a preceding level;
2. some node in the j th level is associated with an uncertainty containing just a single element.

A tree so constructed will be called a *synchronizing tree*. The synchronizing tree for the machine M_2 is shown in Fig. 13.3.

A synchronizing sequence is described by (corresponds to) a path in the tree leading from the initial uncertainty to a singleton uncertainty, i.e., an uncertainty containing just a single state. For the machine M_2 , the path 01010 describes a synchronizing sequence that, when applied to M_2 , synchronizes the machine to state D regardless of the output symbols or initial state. Note that if the initial uncertainty of M_2 is (BCD) then the sequence 010 synchronizes M_2 to state D , since the 010-successors of B , C , and D are D , as shown in Table 13.4.

Theorem 13.2 *If a synchronizing sequence for an n -state machine M exists then its length is at most $\frac{1}{2}(n-1)^2n$.*

Proof Let the initial uncertainty be $(S_1 S_2 \cdots S_n)$. Select any two states S_i, S_j and apply to them a sequence ξ_1 that takes them into some state S_k . This task can always be accomplished, since M is known to possess a synchronizing sequence. The length of the sequence ξ_1 is at most $\frac{1}{2}(n-1)n$, since the longest path for the synchronization of $(S_i S_j)$ is through all possible pairs of states, i.e., $(S_1 S_2), (S_1 S_3), \dots, (S_{n-1} S_n)$. Consequently, S_k is the ξ_1 -successor of $(S_i S_j)$. Next, select a state S_p from the resultant uncertainty, and determine the sequence ξ_2 that takes $(S_k S_p)$ into some state S_q . The length of ξ_2 is also at most $\frac{1}{2}(n-1)n$. In the same way, it is possible to find sequences $\xi_3, \xi_4, \dots, \xi_{n-1}$, which, when concatenated, yield the synchronizing sequence $\xi_1 \xi_2 \cdots \xi_{n-1}$, whose length is at most $\frac{1}{2}(n-1)^2 n$. $\diamond$

The above bound is not the least upper bound. For a tighter bound, see Appendix 13.1.

13.3 Distinguishing experiments

Distinguishing experiments are concerned with the identification of the initial state of a machine whose state table is known but about which there is no other information regarding its condition.

Definition 13.2 Let M be an n -state machine. An input sequence X_0 is said to be a *distinguishing sequence* if the output sequence produced by M in response to X_0 is different for each initial state.

Knowing the output sequence that M produces in response to X_0 is sufficient to identify uniquely M 's initial state. However, knowledge of the initial state and the input sequence is always sufficient to determine uniquely the final state as well. Consequently, *every distinguishing sequence is also a homing sequence*. The converse, however, is not true, since many homing sequences do not provide all the information regarding the initial state, e.g., the sequence 010 for machine M_2 .

The distinguishing tree

A *distinguishing tree* is a successor tree in which a node in the j th level becomes terminal when any of the following occurs:

1. the node is associated with an uncertainty vector whose nonhomogeneous components are associated with some node in a preceding level;
2. the node is associated with an uncertainty vector containing a homogeneous nontrivial component;
3. some node in the j th level is associated with a trivial uncertainty vector.

A path in the tree describes a distinguishing sequence of M if and only if it starts in the initial uncertainty (which is assumed to consist of the entire set of

states S) and terminates in a node associated with a trivial uncertainty. A bound on the length of distinguishing sequences is shown in Appendix 13.2.

The distinguishing tree of the machine M_1 is obtained from the corresponding successor tree (Fig. 13.1). The node associated with the homogeneous uncertainty vector $(A)(BCC)$ is terminated, since no further experiment can split the component (CC) ; i.e., there is no way of knowing, once the machine has passed to state C , whether the initial state was A or B . The machine M_1 has four distinguishing sequences of length 3, 111, 110, 101, and 100. The response of M_1 to the sequence 111 is summarized in Table 13.2*b*. This sequence clearly causes four distinct responses, depending on the initial state.

While every machine has at least one homing sequence, not every machine has a distinguishing sequence. For example, the distinguishing tree of the machine M_2 must be terminated in level 1 (see Fig. 13.2), since the vector $(ABCD)$ is identical to the initial uncertainty and the vector $(AB)(DD)$ has a nontrivial homogeneous component. An inspection of the state table of M_2 (Table 13.3) would have revealed the same result, since no experiment that starts with an input symbol 0 will distinguish between states C and D or between states A and B , while no experiment that starts with a 1 will reduce the initial uncertainty.

The shortest distinguishing prefix

In many cases, the initial state of a machine can be determined just from the prefix of distinguishing sequence X_0 . The length of the required prefix is a function of the initial state. Consider again the machine M_1 , whose response to the distinguishing sequence 111 is given in Table 13.2*b*. It is evident that if the response of the machine to the first input symbol is 0 then the initial state must have been C , and the distinguishing experiment may be terminated at this stage. However, if the response is 1 then the initial state could have been either A , B , or D . The experiment must continue, and M_1 is supplied with a second input symbol 1. If M_1 's response is now 0 then the initial state must have been D , and the distinguishing experiment may be terminated. If, however, the response is 1 then the uncertainty regarding the initial state is (AB) and a third input symbol 1 must be applied to the machine. Thus, for the machine M_1 and the distinguishing sequence 111, the shortest distinguishing prefix for state C is 1, for state D 11, and for states A and B 111.

The shortest distinguishing prefixes can be determined by means of a modified distinguishing tree (see [9]). They are particularly useful in checking experiments and machine identification, where they lead to relatively short experiments.

13.4 Machine identification

Up to now we have been concerned with the problems of identifying the initial and final states of a known machine. We shall now address ourselves to a

more general problem – that of identifying an unknown machine. The machine identification problem is essentially that of experimentally determining the state table of an unknown machine. In its most general form, when no information is available on the unknown machine, this problem cannot be solved for several reasons. First, the experimenter must have complete information regarding the input alphabet of the machine, since otherwise he or she can never be sure that the next input symbol will not reveal new information regarding the machine. Similarly, the machine cannot be identified unless there is an upper bound on the number of its states since, for any given machine and any experiment of length L , it is possible to construct another machine that responds to the experiments of length L exactly like the given machine but will respond differently to experiments of length greater than L . Finally, if a given machine M_i is in initial state S_i then it is indistinguishable experimentally from a machine M_j whose initial state S_j is equivalent to S_i , although machines M_i and M_j may, in fact, be distinguishable. This situation clearly will not occur if both M_i and M_j are strongly connected.

To make the problem of machine identification solvable, we impose several restrictions on the machines. We assume that the input alphabet is known, as is an upper bound on the number of states of the machine. Moreover, the machine is assumed to be reduced and strongly connected.

An unknown machine with at most n states can now be identified in the following manner. Construct the direct-sum table (see Problem 10.10) from all tables that have n or fewer states and find a homing sequence for it. Clearly such a homing sequence can always be found, and its application to the machine in question will reveal which set of equivalent states from the direct-sum table contains the final state of the machine. Also, if the direct-sum table contains only those tables that correspond to reduced and strongly connected machines, the homing sequence will uniquely identify the final state of the machine and, in turn, the machine itself. This demonstrates that, under specified conditions, in principle the machine identification problem can be solved. However, as a procedure for actually designing experiments the direct-sum approach is impractical, since the number of distinct tables is staggeringly large even for relatively small n 's. It will be shown subsequently that the problem of devising checking experiments for sequential machines is directly related to the machine identification problem. More efficient procedures will be presented for the design of such experiments directly from the state table, without the use of the direct sum.

As an example, suppose that a machine is known to have two states and that its response to input sequence X is output sequence Z , as shown below.

<i>Time :</i>	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8
<i>Input, X :</i>	1	1	1	0	1	0	1	
<i>Output, Z :</i>	0	1	0	0	1	0	0	

The first step in the analysis of these sequences is the identification of the distinct states of the tested machine. Let us name these two states A and B and

Table 13.5 Machine M_3

PS	NS, z	
	$x = 0$	$x = 1$
A	$A, 0$	$B, 0$
B	$B, 0$	$A, 1$

suppose that, at the start of the experiment, the machine was in state A . The application of an input symbol 1 results in an output symbol 0 and a transition that is yet to be determined. However, since the second input symbol is also a 1 but the response is 1, the machine must have been in a state other than A at t_2 . Hence, the experimenter may conclude that at t_2 the machine was in state B .

Since state A is the only state which responds to an input symbol 1 by producing an output symbol 0, it is evident that at t_3 the machine was in state A . At t_4 , it was again in state B , since it has already been verified (at t_2) that an input symbol 1 causes a transition from state A to B . In a similar manner, it is easy to show that at t_5 the machine was again in state B , which, in turn, implies (see t_3) that at t_6 , it was in state A . Finally, at t_7 , it must have been in state A , since this is the only state in which the machine produces a 0 output symbol as a response to a 1 input symbol. As a result of the above analysis, the experimenter is able to demonstrate that the machine indeed has two states, named A and B , and that its transitions and output symbols are given by the state table of Table 13.5. Thus, the above experiment is an identification experiment for a machine M_3 .

13.5 Checking experiments

The problem of designing *checking experiments* is actually a restricted version of the problem of machine identification. An experimenter is supplied with a machine and its state table. The task is to determine from terminal experiments whether the given table accurately describes the behavior of the machine; that is, to decide whether the actual machine is isomorphic to the one described by the state table. As discussed before, we shall restrict our attention to machines that are strongly connected, completely specified, and reduced. We also assume that any faults are permanent, owing to some defect. This assumption excludes transient errors due to noise or incorrect input symbols.

First, we consider machines that possess at least one distinguishing sequence. In subsequent sections, we shall relax this restriction and discuss machines that have no distinguishing sequence. Note that these experiments are intended to detect the presence of one or more faults but will not locate or diagnose them.

Table 13.6 Machine M_4

PS	NS, z	
	$x = 0$	$x = 1$
A	$B, 0$	$C, 1$
B	$C, 0$	$D, 0$
C	$D, 1$	$C, 1$
D	$A, 1$	$B, 0$

Table 13.7 Responses of M_4

Initial state	Response to 00	Final state	Initial state	Response to 01	Final state
A	00	C	A	00	D
B	01	D	B	01	C
C	11	A	C	10	B
D	10	B	D	11	C

(a) (b)

We will make the assumption that the machine either has a synchronizing sequence or a reset input that can transfer it to the initial state.

Designing checking experiments

In the procedure we use, each checking experiment consists of two parts.

1. The first part uses the synchronizing sequence or a reset input to transfer the machine into a prespecified state, which is the initial state for the second part of the experiment.
2. The second part is a *preset* experiment in which the machine is taken through all possible transitions. This part is subdivided into two subparts. In the first subpart the machine is caused to display the response of each of its states to the distinguishing sequence, while in the second subpart the actual transitions are verified.

As an example, consider the machine M_4 whose state table is given in Table 13.6 and whose responses to the sequences 00 and 01 are summarized in Table 13.7. Suppose that the synchronizing sequence or reset input places the machine in state A , from which the preset part of the experiment can commence.

In designing the preset part of the checking experiment, the first task is to ascertain that the starting state is indeed A and that the machine being tested actually contains four distinct states. This can be accomplished by displaying the response of each state to the same distinguishing sequence. The machine M_4 has two distinguishing sequences, 00 and 01, whose applications to the machine result in the responses shown in Table 13.7. The design of experiments based

on distinguishing sequence 00 is somewhat shorter but will be left to the reader as an exercise.

To display the response of the starting state, we apply the distinguishing sequence $X_0 = 01$. If the machine has operated correctly up to this point, its output response is 00 and it is now in state D . To display the response of this state, the distinguishing sequence X_0 is applied again and, as a result, the machine goes to state C . The application of a third distinguishing sequence leaves the machine in state B and displays the response of state C . Applying X_0 twice more leaves the machine in state B , as shown below:

<i>Input :</i>	0	1	0	1	0	1	0	1	0	1
<i>State :</i>	A		D		C		B		C	B
<i>Output :</i>	0	0	1	1	1	0	0	1	1	0

The first eight symbols, by displaying four different responses to input sequence 01, i.e., 00, 11, 10, and 01, verify that the machine in question indeed has four distinct states. The last two symbols guarantee that the machine terminates in state B , since it has already been established that a response of 10 to the distinguishing sequence indicates a transition from state C to state B . The above sequence thus verifies the existence of at least four states and, since we are assuming that M_4 has no more than four states, each state must have been visited at least once, and its response to the distinguishing sequence determined. From this point on, if at any time during the course of the experiment one of the above responses to the distinguishing sequence is produced, the state of the machine at that time is uniquely identifiable. (It must be emphasized that the names given to the states are of no importance; a different set of names would result in an isomorphic machine.)

If the machine has not produced the expected output sequence up to this point, we may conclude that a fault exists. If, however, the above expected output sequence has been produced then no conclusion can be reached as to whether the machine has operated correctly and is indeed in state B or a fault exists and the actual final state is different from B . We, therefore, assume for the present that the machine actually started in state A and terminated in B . If this assumption is incorrect, it will be revealed as such in the next part of the experiment.

To complete the experiment it is now necessary to verify every state transition. The general procedure to be followed is to apply the input symbol that causes the desired transition and to identify it by applying the distinguishing sequence. Since the machine is in state B , we shall start by applying an input symbol 0, followed by a distinguishing sequence 01. This input sequence takes the machine back to state B , and thus a 101 input sequence is applied to check the transition from B to D under a 1 input symbol and verify that the machine actually has moved to state D . In each of these three-bit sequences, the first bit causes the transition, while the distinguishing sequence ascertains that the transition is indeed the assumed one. At this point we have obtained additional

information about another transition. It has earlier been shown that the application of 01 to the machine while in state B causes it to go to state C . However, since input symbol 0 itself takes the machine from B to C , we may conclude that if a 1 input symbol is applied to the machine while in state C then it stays in state C . In other words, since the 01-successor of state B is C and the 0-successor of B is also C , the 1-successor of C must be C .

At this point, the machine is in state C . If, in response to the input sequence 001, the machine produces an output sequence 111, we may conclude that the 0-successor of C is D and that the final state is again C . However, since it has already been established that the 01-successor of C is B , it means that the 1-successor of D is B . The experiment at this stage is as follows; note that the second and third rows continue the first row.

<i>Input :</i>	0	1	0	1	0	1	0	1	0
<i>State :</i>	A		D		C	D	B	C	C
<i>Output :</i>	0	0	1	1	1	0	0	1	1

<i>Input :</i>	1	0	0	1	1	0	1	0
<i>State :</i>	D	B	C	D	B	D		C
<i>Output :</i>	0	0	1	0	0	1	1	1

<i>Input :</i>	0	1
<i>State :</i>	D	C
<i>Output :</i>	1	1

Up to this point, we have checked every possible transition, except those from D to A and from A to B and C . Since the machine is presently in state C , we must apply a transfer sequence¹ to get to either state D or A . Such sequences can always be found for a strongly connected machine, and require at most $n - 1$ symbols. Furthermore, the transfer sequences should be applied in such a way that they will take the machine through “checked” transitions only. Thus, the only possible transfer sequence in this case is $T(C, D) = 0$, because, as has already been demonstrated, the machine goes from C to D under input symbol 0. The application of a 0 followed by 01 ascertains the transition from D to A and returns the machine back to state D . This sequence provides enough information to verify the transition from A to C under a 1 input symbol. This verification is achieved by inspection of the preceding sequence and observing that C is the 01-successor of D and A is the 0-successor of D . Thus, C is the 1-successor of A .

The last transition that needs to be checked is from state A to B . Since the machine is in state D , a transfer sequence $T(D, A) = 0$ is applied, followed by

¹ Recall that a *transfer sequence* $T(S_i, S_j)$ is the shortest input sequence that takes a machine from state S_i to state S_j .

001. The complete experiment is shown below:

<i>Input :</i>	0	1	0	1	0	1	0	1
<i>State :</i>	A	B	D	A	C	D	B	C
<i>Output :</i>	0	0	1	1	1	0	0	1

<i>Input :</i>	0	1	0	0	1	1	0	1
<i>State :</i>	C	D	B	C	D	B	D	A
<i>Output :</i>	1	0	0	1	0	0	1	1

<i>Input :</i>	0	0	1	0	0	0	1	0
<i>State :</i>	C	D	A	C	D	A	B	D
<i>Output :</i>	1	1	1	1	1	0	0	1

<i>Input :</i>	0	0	1
<i>State :</i>	A	B	C
<i>Output :</i>	0	0	1

The preset part of the checking experiment thus consists of the above input sequence, whose length is 27 symbols. If the machine at hand responds as shown above then it must be isomorphic to M_4 , since it has been shown to contain four states whose responses are identical to the corresponding responses of M_4 and since all state transitions, which have been verified in terms of the behavior exhibited at the beginning of the experiment, are also isomorphic to those of M_4 . Clearly, if the machine has not produced the above expected output sequence then it cannot be operating correctly. The location of the fault, however, cannot be determined merely by the above response.

Testing machines that have distinguishing sequences

The procedure can be summarized as follows. A checking experiment starts with a synchronizing sequence or a reset input, so as to maneuver the machine to the desired initial state. The machine is next supplied with an input sequence that causes it to visit each state and display its response to the distinguishing sequence. Finally, the machine is made to go through every state transition and, in each case, the transition is verified by displaying its response to the distinguishing sequence. In practice it is not necessary to display all the responses at the beginning of the experiment. Any response or transition that is verified at a later point in the experiment may be used to determine a state transition at some earlier point.

More precisely, the procedure for constructing checking experiments for machines that have distinguishing sequences is as follows. Let $S_1, S_2, \dots, S_n$ be the states of machine M , and suppose that X_0 is a distinguishing sequence for this machine. Let Q_i be the state to which M goes, when it is initially in

S_i , as a result of the input sequence X_0 . Also, let $T(S_i, S_j)$ denote an input sequence (not necessarily unique) that transfers the machine from state S_i to S_j . Now suppose that M is initially in its starting state S_1 . Then, the sequence

$$X_0T(Q_1, S_2)X_0T(Q_2, S_3)X_0T(Q_3, S_4) \cdots X_0T(Q_n, S_1)X_0$$

will serve to take the machine through each of its states and display all the different responses to the distinguishing sequence. For example, starting in S_1 , X_0 leaves the machine in Q_1 . Then $T(Q_1, S_2)$ transfers the machine to S_2 , where X_0 is applied again, leaving the machine in Q_2 . The corresponding output sequence clearly displays the response of M to X_0 , when initially in either state S_1 or S_2 . The machine is similarly led through all its n states and, at each point, the sequence X_0 is applied followed by the transfer sequence $T(Q_i, S_{i+1})$.

At the end of this part of the experiment, the machine receives the sequence $X_0T(Q_n, S_1)$. If it operates correctly, it will be in state S_1 . This is verified by applying the distinguishing sequence X_0 to it again. Clearly, if the machine's response to the last X_0 is identical to its response to the first X_0 then it will indeed be in state Q_1 at the end of this part. Thus, the next part of the experiment starts at this point, as the transitions out of state Q_1 are identified.

In the second part of the experiment, we establish various state transitions. To check, for example, the 0-transition out of state S_i , when the machine is initially in some state Q_j , the appropriate sequence is

$$T(Q_j, S_{i-1})X_0T(Q_{i-1}, S_i)0X_0$$

The sequence $T(Q_j, S_{i-1})X_0$ guarantees that the machine indeed goes to state Q_{i-1} , as it did in the previous part of the experiment. The sequence $T(Q_{i-1}, S_i)$ transfers M to state S_i , and then $0X_0$ is applied to cause the 0-transition out of S_i and also to identify it. In a similar manner the machine can be taken through every transition, in each case identifying the transition by means of the response already established in the first part of the experiment. In general, however, to reduce the length of the experiment it is possible to apply the two parts of the experiment simultaneously instead of sequentially.

The method outlined above can be applied to any reduced and strongly connected machine that has at least one distinguishing sequence. The design of checking experiments for machines that do not have any distinguishing sequence is quite complicated, and the resulting experiments are very long. To alleviate this situation, whenever a distinguishing sequence does not exist, extra output terminals can be added to make sure that such a sequence does exist for the augmented machine, as discussed next. Then the above method can be applied to the augmented machine.

Table 13.8 Testing table for M_2

	0/0	0/1	1/0	1/1
<i>A</i>	<i>B</i>	—	<i>D</i>	—
<i>B</i>	<i>A</i>	—	<i>B</i>	—
<i>C</i>	—	<i>D</i>	<i>A</i>	—
<i>D</i>	—	<i>D</i>	<i>C</i>	—
<i>AB</i>	<i>AB</i>	—	<i>BD</i>	—
<i>AC</i>	—	—	<i>AD</i>	—
<i>AD</i>	—	—	<i>CD</i>	—
<i>BC</i>	—	—	<i>AB</i>	—
<i>BD</i>	—	—	<i>BC</i>	—
<i>CD</i>	—	<i>DD</i>	<i>AC</i>	—

*13.6 Design of diagnosable machines

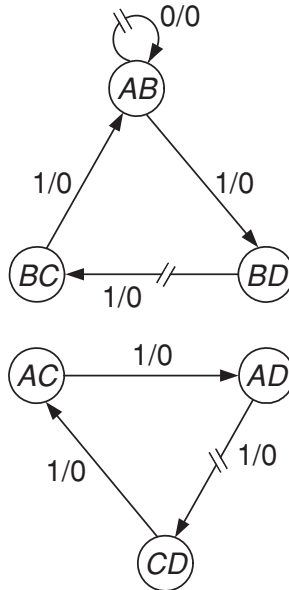
A *diagnosable* sequential machine is one that possesses one or more distinguishing sequences and thus permits us to identify uniquely the states of the machine by inspecting its response to such a sequence. In this section, we shall present a method to modify the design of sequential machines in such a way that they will possess special distinguishing sequences for which relatively short checking experiments can be constructed.

The testing graph

The machine M_2 (Table 13.3) does not possess any distinguishing sequence. We shall now show how it may be augmented by an additional output in such a way that the augmented machine will possess several distinguishing sequences.

The state table of M_2 may be rewritten as shown in the upper half of Table 13.8. The column headings consist of all input–output symbol combinations, where the pair I_k/O_l indicates a combination of input symbol I_k and output symbol O_l . The row labels in the upper half of the table are the states of the machine. The entry in column I_k/O_l , row S_i , is the I_k -successor of S_i if this successor is associated with output symbol O_l and is a dash (—) otherwise. For example, the 0-successor of A is B and the corresponding output symbol is 0. Consequently, B is entered in row A under the column 0/0 and a dash is entered in row A under the column 0/1. In a similar manner, the other next-state entries of M_2 are entered in the upper half of the table.

The lower half of the table is derived directly from the upper half. The row labels are all unordered pairs of states, while the table entries are their corresponding successors. If the entries in rows S_i and S_j , column I_k/O_l , of the upper half are S_p and S_q respectively then the entry in row S_iS_j , column I_k/O_l , of the lower half is S_pS_q . For example, since the entries in rows A and

Fig. 13.4 Testing graph for M_2 .

B , column $1/0$, are D and B respectively the corresponding entry in row AB , column $1/0$, is BD , and so on. If for some pair of states S_i and S_j , either one or both corresponding entries in some column I_k/O_l are dashes, the corresponding entry in row $S_i S_j$, column I_k/O_l , is a dash. For example, the entry in row AC , column $0/0$, is a dash, since the entry in row C , column $0/0$, is a dash. The table thus completed is referred to as a *testing table*.

We shall refer to a pair $(S_i S_j)$ as an *uncertainty pair* and to its successor $(S_p S_q)$ as the *implied pair*. Thus, for example, pair (BD) is implied by (AB) . An uncertainty pair that does not imply any other pair, so that all the entries in the corresponding row are dashes, can be omitted from the table. Whenever an entry in the testing table consists of a repeated state (e.g., DD in row CD), that entry is given in boldface. Thus the boldface entry DD means that states C and D are *merged*, under input symbol 0 , into state D and are indistinguishable by an experiment which starts with a 0 input symbol.

Let us define a directed graph G , which will be called a *testing graph*, in the following way.

1. Corresponding to each row in the lower half of the testing table, there is a vertex in G .
2. If there exists an entry $S_p S_q$, where $p \neq q$, in row $S_i S_j$, column I_k/O_l , of the testing table then G has a directed arc leading from the vertex labeled $S_i S_j$ to the vertex labeled $S_p S_q$. The arc is labeled I_k/O_l . No arc is needed if $S_i S_j$ implies $S_p S_p$, e.g., DD in row CD .

The testing graph for the machine M_2 is derived directly from the lower half of the testing table and is shown in Fig. 13.4.

Definitely diagnosable machines

A machine M is defined as a *definitely diagnosable machine of order μ* if μ is the least integer such that every sequence of length μ is a distinguishing sequence for M . In other words, a machine is definitely diagnosable if every node at the level μ of the distinguishing tree is associated with a trivial uncertainty vector. The distinguishing tree can thus serve as a tool for recognizing definitely diagnosable machines. In this section, however, we shall derive a different test by means of the testing graph.

Theorem 13.3 *A machine M is definitely diagnosable if and only if its testing graph G is loop-free and no repeated states (i.e., boldface entries) exist in the testing table.*

Proof If the testing table contains a repeated entry in row $S_i S_j$, column I_k / O_l , then state S_i cannot be distinguished from state S_j by an experiment that starts with I_k . Thus, if M is definitely diagnosable then its testing table does not contain repeated entries. Now suppose that G is not loop-free. Then, by repeatedly applying the symbols coinciding with the labels of the arcs in the loop, we find an arbitrarily long input sequence that cannot resolve the uncertainty regarding the initial state. Consequently, the machine is not definitely diagnosable. To prove sufficiency, assume that G is loop-free. If M is not definitely diagnosable then there exists an arbitrarily long path in G corresponding to some input sequence X and some pair of states $S_i S_j$, such that S_i cannot be distinguished from S_j by X . However, since the number of vertices in G cannot exceed $\frac{1}{2}(n-1)n$ (corresponding to the number of distinct pairs of states), arbitrarily long paths in G are possible only if it contains a loop. Thus, the theorem is proved. $\diamond$

The above testing procedure is clearly equivalent to testing by means of the distinguishing tree. In fact, that the graph is loop-free means that no node in the tree is associated with an uncertainty vector whose nonhomogeneous components are also associated with some node in a preceding level. Similarly, if the testing table is free of repeated entries then no node in the tree is associated with an uncertainty vector containing a homogeneous nontrivial component. Hence, every node in the μ th level of the tree is associated with a trivial uncertainty vector.

Corollary *Let the testing table of machine M be free of repeated entries, and let G be a loop-free testing graph for M . If the length of the longest path in G is l then $\mu = l + 1$.*

Proof Since G is loop-free, M is definitely diagnosable. Assume that $\mu > l + 1$; then there exists at least one uncertainty pair $(S_i S_j)$ that is transferred, by the application of an input sequence of length $l + 1$, to another pair $(S_p S_q)$. Consequently, there must exist a path, between vertices $S_i S_j$ and $S_p S_q$ in G ,

Table 13.9 Machine M'_2

PS	NS, zz_1	
	$x = 0$	$x = 1$
A	$B, 01$	$D, 00$
B	$A, 00$	$B, 00$
C	$D, 10$	$A, 01$
D	$D, 11$	$C, 01$

whose length is $l + 1$. This contradicts our assumption, and thus μ cannot exceed $l + 1$. The proof that μ cannot be smaller than $l + 1$ is trivial. $\diamond$

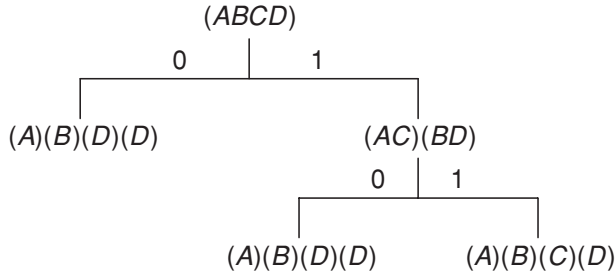
We thus arrive at the general result that *if a machine is definitely diagnosable of order μ , then $\mu \geq \frac{1}{2}(n - 1)n$* . In Problem 13.22, we show that this bound is in fact the least upper bound for μ .

Designing definitely diagnosable machines

In order to obtain a machine M'_2 that contains M_2 and possesses a distinguishing sequence, it is necessary to augment M_2 by adding to it an output terminal and assigning different output symbols to selected transitions. We shall, in fact, show that the addition of one output terminal is sufficient to make M'_2 definitely diagnosable. The first step toward this end is to assign different output symbols to each transition that may cause a repeated entry in the testing table. In the case of M_2 , this is accomplished by assigning the output symbol 10 to the transition from C to D and the output symbol 11 to the transition from D to D . Such an assignment of output values ensures that the testing table of M'_2 will be free of repeated entries.

The testing graph of M_2 contains three loops: a self-loop around AB and two other loops, each containing three vertices. Clearly, these loops must be opened if M'_2 is to be definitely diagnosable. In general, a loop is opened by the removal of any of its arcs. To remove an arc, it is necessary to assign different output symbols to the next-state entries represented by the vertex to which that arc leads. In other words, an arc leading from the vertex $S_i S_j$ to the vertex $S_p S_q$ is eliminated by assigning different output symbols to the transitions from S_i and S_j to S_p and S_q . For example, the self-loop around AB in Fig. 13.4 is opened by assigning the output symbols 01 and 00, respectively, to the next-state entries B and A in the column $x = 0$. The loop $AB - BD - BC - AB$ can be opened by the removal of the arc from BD to BC . This is achieved by assigning the output symbols 00 and 01 to the next-state entries B and C in rows B and D , column $x = 1$. In a similar manner, we open loop $AC - AD - CD - AC$ by assigning a 00 output symbol to the next-state entry D in row A , column $x = 1$, thus removing the arc from AD to CD . The resulting state table is shown in Table 13.9.

Fig. 13.5 Distinguishing tree for M'_2 .



Since the length of a checking experiment is directly proportional to the length of the distinguishing sequence for the machine, we attempt to open all loops while simultaneously minimizing the length of various paths in the graph. In opening the loops in the graph of Fig. 13.4, all the output entries, with the exception of the entry in row C , column $x = 1$, have been assigned new values. The longest path in the loop-free graph is of length 2 and, consequently, the order of the modified machine is $\mu = 3$. This result can, however, be improved by specifying the output entry in row C , column $x = 1$, as 01. This specification actually eliminates the arcs from AC to AD and from BC to AB . As a result, the length of the longest path in the graph is now 1, and M'_2 is definitely diagnosable of order 2. The distinguishing tree of machine M'_2 is shown in Fig. 13.5.

It is clear that, for any 2^k -state machine, the addition of k output terminals is sufficient to convert it into a definitely diagnosable machine. However, frequently fewer additional output terminals suffice.

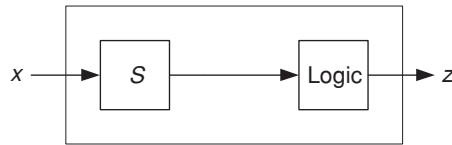
Since the procedure followed in the above example can be applied to any machine, we arrive at the following general result.

- To every reduced machine M there corresponds a definitely diagnosable machine M' , which is obtained from M by the addition of one or more output terminals.

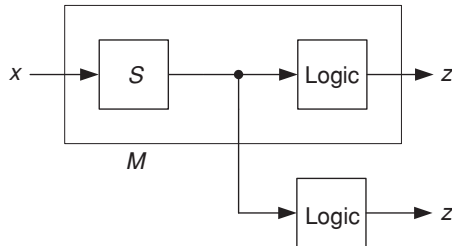
The block diagram of the definitely diagnosable machine M' that corresponds to machine M is shown in Fig. 13.6.

A question now arises regarding the purpose of designing definitely diagnosable machines. Evidently, checking experiments can be designed with just one distinguishing sequence. Moreover, even when a machine possesses two or more distinguishing sequences it is not easy to utilize them efficiently and simultaneously in an experiment. The main motivation for designing definitely diagnosable machines and studying their properties is the fact that it is possible to design checking experiments for them. Such experiments are simpler to design for definitely diagnosable machines, since it is possible to cross-check the machine with every sequence of length μ , not with just a single sequence.

Fig. 13.6 Design of a definitely diagnosable machine.



(a) M .



(b) M is modified to produce M' . The output z_1 is only used for diagnostic purposes.

13.7 Alternative approaches to the testing of sequential circuits

We saw earlier how finite-state machines can be tested using checking experiments. However, often the test sequences derived by such an approach are quite long. In this section, we shall describe two alternative test generation methods for such machines. The first method also uses a state table; however, the second method uses the sequential circuit implementation of the machine.

State-table-based test generation

This test generation approach uses a *functional fault model*. This fault model assumes that the fault is associated with a state transition in the state table. For example, a *single-state-transition* (SST) fault model assumes that the fault results in the destination state of a state transition becoming corrupted while retaining its correct input/output symbols. Test sequences derived using the SST fault model have been shown to detect a very high percentage of single stuck-at faults in the sequential circuit implementation of the machine.

We shall make the assumption that the SST fault does not increase the number of states in the state table.

We designate each state transition in a machine by the four-tuple

$\langle \text{input symbol, source state, destination state, output symbol} \rangle$.

A state transition can become corrupted if its destination state or output symbol or both are faulty. However, if a test sequence detects a corrupted destination state then it will also detect the corrupted output symbol or both the corrupted destination state and output symbol of that state transition. We can prove this as

follows. In order to detect a corrupt destination state, a test sequence needs to have three parts: an *initialization sequence* that takes the machine to the source state of the state transition in question; the input symbol of the transition to activate the fault; and a *state-pair differentiating sequence* (SPDS) that differentiates between the correct and faulty destination states, i.e., produces different output sequences starting from these states. If the output symbol associated with the state transition is faulty then the initialization sequence and the input symbol that activates the fault together detect the fault. Hence, we can limit our attention to faulty destination states only. We shall derive the three parts of the test sequence from the fault-free state table. Strictly speaking, we should employ both the fault-free and faulty state tables to derive them. However, deriving them from the fault-free state table considerably speeds up the test generation process without much loss in the ability to detect the targeted fault.

An n -state m -transition machine has $m(n - 1)$ SST faults. If the machine is large, this number can also be quite large. However, it is possible to use fault collapsing to reduce the number. For each state transition, there are $n - 1$ faulty destination states possible. However, we often need to target only a subset of these faulty states. Suppose that the four-tuple $\langle I_k, S_j, S_i, O_l \rangle$ is corrupted to $\langle I_k, S_j, S'_i, O_l \rangle$ by the SST fault f_1 and to $\langle I_k, S_j, S''_i, O_l \rangle$ by the SST fault f_2 . If we find that the SPDS of S_i and S'_i also differentiates between S_i and S''_i then fault f_2 dominates fault f_1 , and f_2 can be removed from the fault list.

Example Consider the machine M_5 shown in Table 13.10. Since the input symbol $x = 0$ differentiates between states A and B as well as A and C , $SPDS(A, B) = SPDS(A, C) = 0$. Similarly, $SPDS(B, C) = 1$. Next, consider the state transition $\langle 1, C, A, 0 \rangle$. Its destination state A can be corrupted in three ways to give $\langle 1, C, B, 0 \rangle$, $\langle 1, C, C, 0 \rangle$, or $\langle 1, C, D, 0 \rangle$. However, since $SPDS(A, B)$ is also $SPDS(A, C)$, the first two of these faulty transitions can be collapsed into just the first one.

Table 13.10 Machine M_5

PS	NS, z	
	$x = 0$	$x = 1$
A	$C, 0$	$C, 0$
B	$C, 1$	$B, 1$
C	$D, 1$	$A, 0$
D	$A, 0$	$B, 1$

Another reasonable fault-collapsing heuristic, which does not reduce the SST fault coverage much, is the following. If two state transitions have an identical source state, destination state, and output label then they are collapsed into a single transition. For example, in the machine in Table 13.10 the two

transitions from state A satisfy this condition. Hence, only one of the these faulty transitions needs to be considered, not both.

Before test generation starts, we first compute transfer sequences between every pair of states. Then, we compute the relevant SPDSs. Test generation consists of the following three steps.

1. *Initialization* In this step the machine is brought from the current state to the source state of the faulty transition using an appropriate transfer sequence.
2. *Excitation* In this step the faulty transition is executed.
3. *State differentiation* In this step the corresponding SPDS is applied to differentiate between the good and faulty states.

Example Consider an SST fault that corrupts the transition $\langle 0, D, A, 0 \rangle$ to $\langle 0, D, B, 0 \rangle$ in machine M_5 . To derive a test sequence for this SST, we first need $T(A, D) = 00$ (10 is also a valid transfer sequence). Then the activation vector $x = 0$ is applied. Finally, $SPDS(A, B) = 0$ is applied. Hence, one possible test sequence is 0000.

Sequential circuit based test generation

In this subsection, we shall show how test generation can be performed for sequential circuits with the help of the iterative array model. An iterative-array model of a sequential circuit was presented in Chapter 9. This approach generates a test sequence to activate the fault and propagate its effect to a circuit output by finding a sensitized path through multiple time frames.

Since we are targeting a faulty sequential circuit, we shall assume that the initial state of the circuits is unknown.

Extended D -algorithm

In Chapter 8, we discussed the D -algorithm, which can be used to generate test vectors for faults in combinational circuits. It is possible to extend the D -algorithm to generate test sequences for sequential circuits. We can target a fault in some time frame, say time frame 0, in the iterative array model and use the D -algorithm to generate a test vector for it. If a D or D' propagates to a circuit output, no further error propagation is required. However, if it only propagates to the next-state lines, we need to add a new time frame as the next time frame, labeled time frame 1, to try to propagate the error signal further. This process is repeated until the error signal reaches some circuit output. If the test vector contains assignments of specific logic values to any present state lines in time frame 0, we add a new time frame as the previous time frame, labeled time frame -1 . We then try to justify (trace) the current state backwards through the previous time frame. This process of line justification (Section 8.2) is repeated until no particular logic values are required at the present state lines.

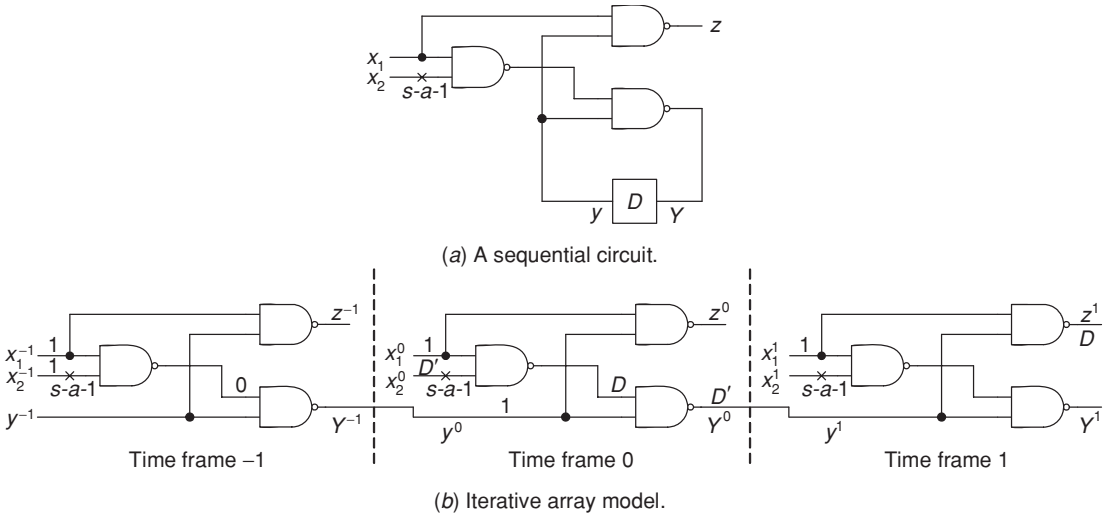


Fig. 13.7 Application of the extended D -algorithm.

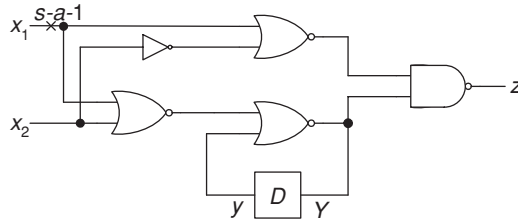
Example Consider the sequential circuit shown in Fig. 13.7a and its iterative array model in Fig. 13.7b. The signals are superscripted with i in time frame i . Suppose the target stuck-at fault is $s-a-1$ on input x_2 . This targeted stuck-at fault has to be included in every time frame. First, let us consider time frame 0. After applying the D -algorithm to the stuck-at fault in this time frame, the error signal D' propagates to the next-state line Y and the value 1 needs to be justified at the present state line y . To propagate the error further, time frame 1 is added to the right. The error signal can now be propagated to the circuit output z in this time frame. Therefore, we need to justify the value at y in time frame 0. A time frame -1 is added to the left of time frame 0 for this purpose. The signal $x_1 = x_2 = 1$ is needed at the input of this time frame to obtain $y = 1$ in time frame 0. Since the stuck-at fault is present in each time frame, we need to make sure that the fault-free and stuck-at values are the same in this state justification step. Since no value was assigned to line y in time frame -1 , there is no need to add any further time frames to the left. We thus arrive at the test sequence for the above fault, consisting of vectors at inputs (x_1, x_2) , as $\{(1, 1), (1, 0), (1, \phi)\}$.

The nine-valued logic

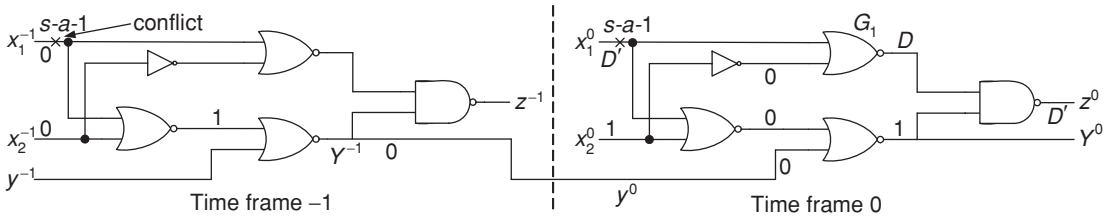
Although the above extension of the D -algorithm is straightforward, the five-valued logic $\{0, 1, \phi, D, D'\}$ used in the D -algorithm is not adequate for sequential circuits because it overspecifies the value requirements at some lines in the circuit. This may prevent the test generator from obtaining a test sequence even when one exists. This problem can be tackled by using a nine-valued logic instead. This logic accounts for the effects of the fault in each time frame correctly. The nine values in this logic each represent an ordered

pair from the ternary values 0, 1, and ϕ . The first value of the pair represents the ternary value of the fault-free circuit and the second value represents the ternary value of the faulty circuit. Hence, the nine ordered pairs are 0/0, 0/1, 0/ ϕ , 1/0, 1/1, 1/ ϕ , ϕ /0, ϕ /1, and ϕ / ϕ .

We next illustrate through an example why nine-valued logic succeeds where the extended D -algorithm may not.



(a) A sequential circuit.



(b) Application of the extended D -algorithm.

Fig. 13.8 Test generation with five-valued logic.

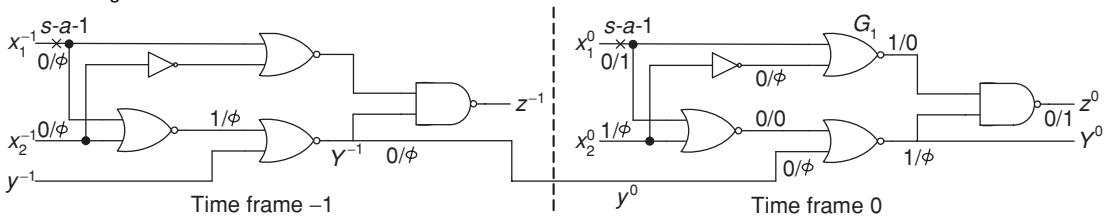


Fig. 13.9 Test generation with nine-valued logic.

Example Consider the sequential circuit shown in Fig. 13.8a. Application of the extended D -algorithm to this circuit is illustrated in Fig. 13.8b. Since, with the shown logic values in time frame 0, the error signal D' gets propagated to the circuit output, there is no need to add time frame 1 to the right. However, we need to add time frame -1 to the left in order to justify the value 0 required at y^0 . Justifying the values in time frame -1 results in a conflict at x_1^{-1} , on which a 0 is required whereas it has an $s-a-1$ fault present. Thus the algorithm concludes that no two-vector test sequence exists to detect this fault. Note that if a 1 had been placed at y^{-1} to justify a 0 at Y^{-1} , then we would need to add time frame -2 to the left.

Next, consider the application of nine-valued logic to this circuit, as shown in Fig. 13.9. In order to propagate the error from x_1 to the output of gate G_1 in time frame 0, the other input of G_1 must have a 0 for the fault-free circuit but does not require any particular value for the faulty circuit. This is denoted as $0/\phi$. Eventually, we require $0/\phi$ at the line y^0 . Owing to this relaxed requirement, there is no conflict at x_1^{-1} . The corresponding test sequence for this fault is thus $\{(0,0), (0,1)\}$.

13.8 Design for testability

Since it is difficult to control the present-state lines and observe the next-state lines of a sequential circuit, sequential test generation generally does not lead to a high fault coverage. When certain design features are added to a circuit to make it easier to derive tests or test sequences for the circuit, the corresponding approach is called *design for testability*.

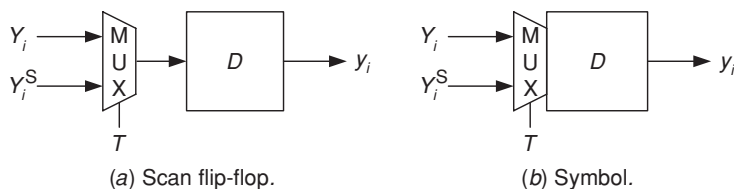
Scan design

A popular design-for-testability approach for sequential circuits is called scan design. In *scan design* there are two modes of operation, *normal* and *test*. In the normal mode, the circuit exhibits its original input–output behavior. However, in the test mode the flip-flops of the circuit are chained into a shift register. If all the flip-flops of the circuit are so chained, the circuit is said to be a *full-scan design*. If a fraction, but not all, of the flip-flops are so chained, the resultant circuit is said to be a *partial-scan design*.

Since a flip-flop may have two sources of inputs, one corresponding to its normal mode of operation and another corresponding to its test mode, a special flip-flop, called a *scan flip-flop*, is needed. Such a scan flip-flop essentially has a 2-to-1 multiplexer at its input, as shown in Fig. 13.10a. The i th D flip-flop has a normal-mode input Y_i and a test-mode input Y_i^S (where the superscript S denotes “scan”). When the *mode-select signal* T is 0, the upper input of the multiplexer is selected and this corresponds to the normal mode. However, when $T = 1$ the lower input is selected and this corresponds to the test mode. We shall, henceforth, use the compact symbol shown in Fig. 13.10b.

We are now in a position to analyze the scan chain shown in Fig. 13.11. An extra input, called the scan input (labeled *ScanIn*) and an extra output, called

Fig. 13.10 A flip-flop with an input multiplexer.



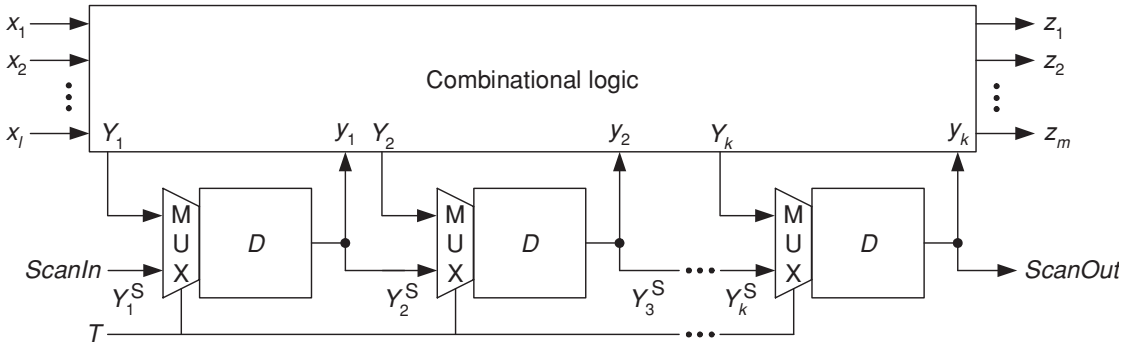


Fig. 13.11 A scan chain.

the scan output (labeled *ScanOut*), are added to the circuit. When $T = 0$, the next-state value at line Y_i , $1 \leq i \leq k$, gets transferred to the present-state line y_i after the flip-flop is clocked, as one would expect during the normal operation of a sequential circuit. However, when $T = 1$ the value at *ScanIn* is transferred to the output of the first flip-flop after clocking, the output of the first flip-flop gets transferred to the output of the second flip-flop, and so on. In other words, the value at $Y_i^S = y_{i-1}$ gets transferred to y_i . The value at y_k also gets propagated to *ScanOut*.

Testing of circuits using scan design

The scan chain enables any state of the sequential circuit to be scanned into the flip-flops in the test mode, essentially making the flip-flops fully controllable. After applying the test to the circuit, the next-state values can be captured in the flip-flops in the normal mode. Then these values can be shifted out through *ScanOut* in the test mode, thus also making the flip-flops fully observable. This reduces the sequential test generation problem to that of test generation for the combinational logic of the circuit. This logic has $x_1, x_2, \dots, x_l, y_1, y_2, \dots, y_k$ as primary inputs and $z_1, z_2, \dots, z_m, Y_1, Y_2, \dots, Y_k$ as circuit outputs. Thus, the test vectors for such a circuit will have $l + k$ bits and the resulting output response will have $m + k$ bits. The first l input bits are said to constitute the *primary input part* of the vector and last k input bits its *state part*. A test set can be obtained for such a circuit using any combinational test generation algorithm, e.g., the *D*-algorithm presented in Chapter 8 if stuck-at faults are the targeted faults.

Example Consider the sequential circuit in Fig. 13.12a. Its combinational logic is shown in Fig. 13.12b. Readers can check that the test set shown in Fig. 13.12c detects all single stuck-at faults in this combinational logic. The first two bits of each of the test vectors in this set denote the primary input part and the last two bits its state part.

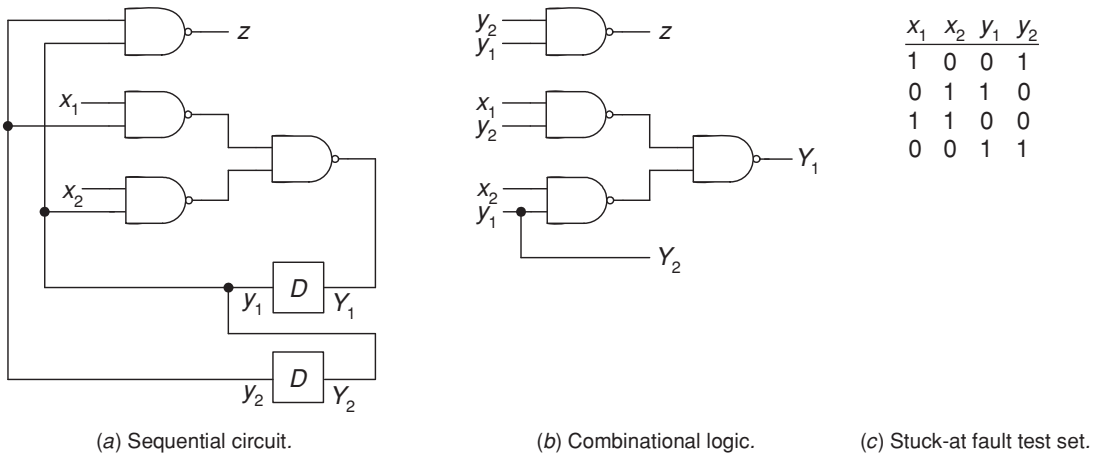


Fig. 13.12 Testing of scan designs.

To apply the test set derived for the combinational logic to the sequential circuit, the following procedure can be followed.

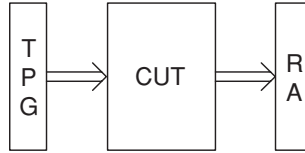
1. Make $T = 1$ to set the sequential circuit into test mode.
2. Scan in the state part of the vector through the *ScanIn* input in the next k clock cycles. In these cycles, the primary inputs can be fed arbitrary values.
3. Apply the primary input part of the vector to the primary inputs. At this point, all the $l + k$ bits of the test vector have been applied to the combinational logic. After allowing the combinational logic to settle down, observe the output response at circuit outputs $z_1, z_2, \dots, z_m$.
4. Make $T = 0$ to set the circuit into normal mode.
5. Apply a clock pulse. This results in the values on the next state lines, $Y_1, Y_2, \dots, Y_k$, being latched in the k flip-flops.
6. Make $T = 1$ and observe the values captured in the flip-flops by scanning them out through *ScanOut* while repeating this procedure for the next test vector.

The flip-flops are themselves tested beforehand by shifting through them a sequence of 1's and then a sequence of 0's to make sure that both a 1 and a 0 can be shifted through each flip-flop.

Suppose that there are n test vectors in the test set. A total of k clock cycles are required to scan-in the state part, one cycle to capture the state response, and $k - 1$ clock cycles to scan-out the captured state. Since the state part of the next test vector is scanned-in at the same time as the captured state for the previous vector is being scanned out, the total number of clock cycles needed to apply the complete test set is $n(k + 1) + k - 1$.

Example For the test set in Fig. 13.12c, $n = 4$ and $k = 2$. Thus, a total of 13 clock cycles is required for it.

Fig. 13.13 A circuit with BIST.



13.9 Built-in self-test (BIST)

The BIST approach allows the circuit to test itself. This requires that some extra circuitry be integrated on-chip. It reduces the need for expensive automatic test equipment. It allows the test vectors to be applied to the circuit under test (CUT) at the normal clock rate. This is called *at-speed testing* and has been found useful for detecting delay faults. A chip with BIST can also be tested in the field, which enhances the reliability of the system.

A CUT that incorporates BIST is shown in Fig. 13.13. It contains a test pattern generator (TPG), CUT, and response analyzer (RA). The TPG generates pseudo-random test sequences and applies them to the CUT. The RA compresses the output response of the CUT into a vector called the *signature*. When there is no fault present in the CUT, the corresponding compressed response is called the *golden signature*. When a fault is present, it is highly likely that the compressed response will not match the golden signature, thus indicating the presence of a fault.

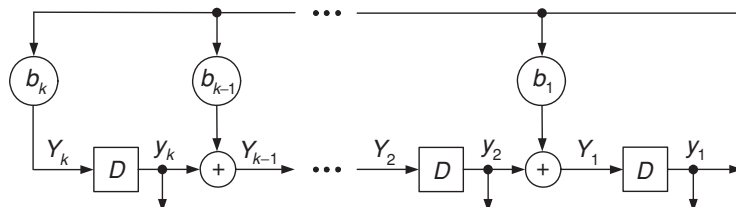
Test pattern generator

The TPG usually comprises a linear feedback shift register (LFSR). An LFSR consists of D flip-flops and XOR gates. It belongs to the class of linear sequential machines which will be discussed in detail in Chapter 15.

A k -stage LFSR is shown in Fig. 13.14 (the number of stages refers to the number of flip-flops present). In it, the output y_1 of the last flip-flop is fed back to a subset of the flip-flops determined by whether the corresponding b_j , $1 \leq j \leq k$, is 0 or 1. The presence (absence) of the feedback is indicated by $b_j = 1$ ($b_j = 0$). An LFSR is often described by a feedback polynomial:

$$p(x) = x^k + b_1x^{k-1} + \cdots + b_{k-1}x + b_k.$$

Such a polynomial is said to have *degree* k .

Fig. 13.14 A k -stage LFSR.

Example Consider the three-stage LFSR shown in Fig. 13.15. Its feedback polynomial is $p(x) = x^3 + x^2 + 1$. Note that in this case $b_3 = b_1 = 1$ and $b_2 = 0$.

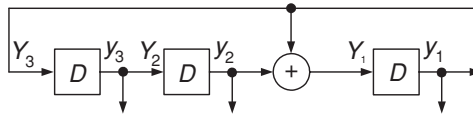


Fig. 13.15 An example of a three-stage LFSR.

A feedback polynomial is said to be *primitive* if the state diagram of the corresponding k -stage LFSR consists of two loops, a trivial loop with the all-0 state and a nontrivial loop with the remaining $2^k - 1$ states. The outputs of the k flip-flops can be directly fed to the inputs of a k -input CUT. The output patterns generated by such an LFSR are known to have very good randomness properties and hence are very useful for obtaining a high coverage of faults in the CUT.

Example For the three-stage LFSR shown in Fig. 13.15, the state diagram is shown in Fig. 13.16. Thus its feedback polynomial $p(x) = x^3 + x^2 + 1$ is primitive.

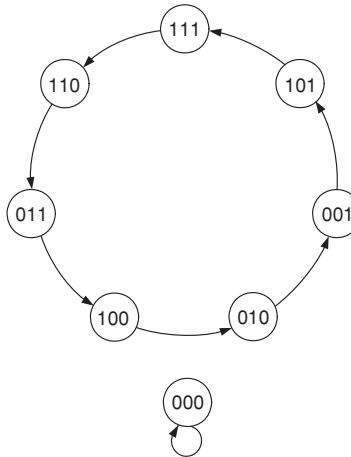


Fig. 13.16 State diagram of the LFSR in Fig. 13.15.

A list of primitive polynomials for various values of k is known. As an example, readers can verify that $p_1(x) = x^4 + x + 1$ is a primitive polynomial but that $p_2(x) = x^4 + x^2 + 1$ is not.

Usually, LFSRs based on primitive polynomials find use in BIST. Test pattern generation can start with any state in the nontrivial loop of such an LFSR. The initial state is called the *seed*. Clocking of the LFSR causes it to transition

from the seed to the next state, and so on. For example, in the state diagram in Fig. 13.16, if the seed is state 001 then the next state will be 101 and then 111, and so on. These patterns can be fed to a three-input CUT in order to test it. It is possible, however, that many patterns from this test sequence are not needed to detect any targeted faults in the CUT. Thus, if we started from different seeds and applied a few test patterns from each, we could shorten the time it takes to test the CUT. This process is called *LFSR re-seeding*.

Example Consider the circuit shown in Fig. 13.17. A possible test set for detecting all single stuck-at faults in this circuit is $(x_3, x_2, x_1) = \{(1, 0, 1), (1, 1, 1), (1, 0, 0), (0, 1, 0)\}$. Suppose that the LFSR shown in Fig. 13.15 is used to test it, with y_i connected to the input x_i , $1 \leq i \leq 3$. From the state diagram in Fig. 13.16, we can see that testing can be accomplished by applying two patterns starting with the seed (1, 0, 1) and two additional patterns starting with the seed (1, 0, 0). The two seeds can be stored on-chip and fed to the LFSR when needed. Thus, we see that four clock cycles are needed to test this circuit, which is the minimum possible. However, if only one seed were used, say (1, 0, 1), then we would have to cycle through six patterns from (1, 0, 1) to (0, 1, 0), for a total of six clock cycles.

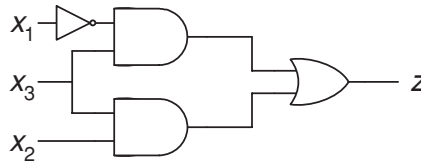


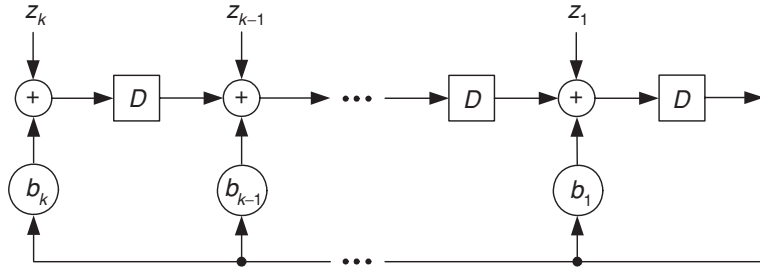
Fig. 13.17 Re-seeding example.

Response analyzer

For a k -output CUT to which ν test patterns have been applied by the TPG, we need to analyze the $k\nu$ output bits to see if any bit is erroneous, thus indicating the presence of a fault in the CUT. To do this, we would need to store the fault-free values of these bits and do a bit-by-bit comparison with the response obtained from the CUT. Since this can be quite expensive in terms of space and time, output responses are usually compressed into a signature and compared with the golden signature that would be obtained if no faults were present in the CUT. However, it is possible that, even if erroneous bits are present in the response, its signature is the same as the golden signature. This is called *aliasing*. This will lead us to declare a faulty circuit to be fault-free, which is obviously a scenario we would like to avoid. Luckily, the aliasing probability of an RA is typically extremely small.

A commonly used RA is the multiple-input signature register (MISR), which is obtained by modifying an LFSR, as shown in Fig. 13.18. The k outputs of the CUT, $z_1, z_2, \dots, z_k$, are connected to the k -stage MISR as shown. When

Fig. 13.18 A multiple-input signature register.



the CUT is being tested, in each cycle a k -bit response is fed to the MISR, leading it to a new state. When the final k -bit response is fed to the MISR, the state it enters is the signature in which we are interested. It has been shown that the aliasing probability of such a MISR is close to $1/2^k$ (note that this is independent of the CUT under test). For reasonable values of k , such as $k = 32$, this probability is negligible.

Appendix 13.1 Bounds on the length of synchronizing sequences

We shall next establish a range of values for the length of a synchronizing sequence and show that the value of the least upper bound on the length must be in this range.

Theorem 13.4 *If an n -state machine has a synchronizing sequence, or sequences, then it has one such sequence whose length is at most $\frac{1}{6}n(n+1)(n-1)$.*

Proof A necessary condition for a machine to have a synchronizing sequence is that, under at least one input symbol I_k , the I_k -successors of some two states S_i , S_j will be identical. The synchronization of a machine, whose initial state is unknown, into some state S_c can be accomplished by applying I_k to the machine in such a way that if it is in either S_i or S_j then it will go to the common successor; next, a sequence that transfers another pair of states S_p , S_q into S_i , S_j is applied, and after that I_k is again applied to the machine to take it into the common successor, and so on. This process actually reduces the initial uncertainty ($S_1 S_2 \cdots S_n$) to the singleton uncertainty (S_c).

Suppose now that $k-1$ states have already been taken out of the uncertainty, which presently consists of $n-k+1$ states. We wish to obtain an upper bound on the length of the sequence needed to reduce the uncertainty by another state, that is, to reduce it to $n-k$ states. Suppose also that S_u and S_v are the states that will now be taken by this sequence into a common successor. The present uncertainty U thus consists of S_u , S_v , and the remaining $n-k-1$ states. The length of the required sequence depends on the number of pairs of states through which $S_u S_v$ passes before reaching the common successor. This number will be maximized if $S_u S_v$ does not pass through any other pair of states contained in the remaining $n-k-1$ states of the uncertainty (because

in such a case we could use that pair of states to reduce the uncertainty). For the same reason, $S_u S_v$ should not pass through any pair of states contained in the successors of these $n - k - 1$ states.

Thus the length of the sequence to be obtained will be maximized if all the uncertainty successors of U contain the same $n - k - 1$ states and only $S_u S_v$ passes through various pairs of states. The successors of $S_u S_v$ may be any pair of states not contained in these $n - k - 1$ states. Since there are $n - (n - k - 1) = k + 1$ such states, there are $\frac{1}{2}k(k + 1)$ pairs of possible successors to $S_u S_v$. Consequently, at most $\frac{1}{2}k(k + 1)$ (which is equal to $1 + 2 + 3 + \cdots + k$) input symbols are needed to take out the k th state from the uncertainty.

To reduce the initial uncertainty ($S_1 S_2 \cdots S_n$) to a singleton uncertainty, a sequence of length $1 + (1 + 2) + (1 + 2 + 3) + \cdots + (1 + 2 + 3 + \cdots + n - 1) = \sum_{k=2}^n \frac{1}{2}k(k - 1)$ is needed. Since $\frac{1}{2}k(k - 1) = 0$ for $k = 1$, we can take the sum from 1 to n , i.e.,

$$\begin{aligned} \frac{1}{2} \sum_{k=1}^n k(k - 1) &= \frac{1}{2} \sum_{k=1}^n k^2 - \frac{1}{2} \sum_{k=1}^n k \\ &= \frac{1}{2} \left[\frac{n(n + 1)(2n + 1)}{6} - \frac{3n(n + 1)}{6} \right] \\ &= \frac{n(n + 1)(n - 1)}{6}. \end{aligned}$$

◇

Theorem 13.4 thus establishes an upper bound on the length of synchronizing sequences, which is lower by a constant factor than that in Section 13.2.

Theorem 13.5 *For every n , there exists an n -state machine that has a synchronizing sequence of length $(n - 1)^2$.*

Proof A machine that satisfies the theorem is given in Table 13.11. The proof that the shortest synchronizing sequence for this machine is of the form $0(1^{n-1}0)^{n-2}$ is left to the reader as a (nontrivial) exercise. Note that the proof must consist of two parts: first, the proof that the above is indeed a synchronizing

Table 13.11 A machine with a synchronizing sequence of length $(n - 1)^2$

PS	NS	
	$x = 0$	$x = 1$
S_1	S_1	S_n
S_2	S_1	S_1
S_3	S_3	S_2
$\vdots$	$\vdots$	$\vdots$
S_k	S_k	S_{k-1}
$\vdots$	$\vdots$	$\vdots$
S_{n-1}	S_{n-1}	S_{n-2}
S_n	S_n	S_{n-1}

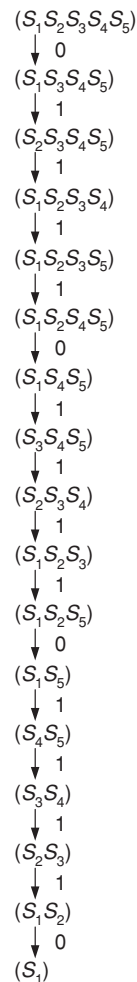
sequence, and second a demonstration that it is the shortest synchronizing sequence.

The length of the subsequence within the parentheses is n , since it consists of $n - 1$ 1's followed by a 0. There are $n - 2$ such subsequences, preceded by a single 0. Hence, the total length is $1 + (n - 2)n = n^2 - 2n + 1 = (n - 1)^2$. $\diamond$

Example A machine that illustrates Theorem 13.5 for $n = 5$ is shown in Fig. 13.19a. The corresponding path in the synchronizing tree, which leads to the singleton uncertainty, is given in Fig. 13.19b.

PS	NS	
	$x = 0$	$x = 1$
S_1	S_1	S_5
S_2	S_1	S_1
S_3	S_3	S_2
S_4	S_4	S_3
S_5	S_5	S_4

(a) Machine M_6



(b) Shortest synchronizing sequence for M_6

Fig. 13.19 Demonstrating Theorem 13.5 for $n = 5$.

Combining the results in Theorems 13.4 and 13.5, we obtain the following corollary.

Corollary *The least upper bound L on the length of synchronizing sequences is bounded by $(n - 1)^2 \leq L \leq \frac{1}{6}n(n + 1)(n - 1)$.*

Appendix 13.2 A bound on the length of distinguishing sequences

Next, we prove that the length of the distinguishing tree is bounded and, consequently, the construction of such a tree is a finite process.

Theorem 13.6 *If a preset distinguishing sequence for an n -state machine M exists then its length is at most $(n - 1)n^n$.*

Proof Let the uncertainty vector at some level in the distinguishing tree consist of m components whose sizes are $k_1, k_2, \dots, k_m$. Clearly, the sum of the sizes of all the components must be equal to n ; i.e., $k_1 + k_2 + \dots + k_m = n$. Let the numbers $k_1, k_2, \dots, k_m$ be subsets in a partition μ such that $\mu = \{k_1, k_2, \dots, k_m\}$. Clearly, μ defines the *size distribution* of the components in the uncertainty vector. The number of different uncertainty vectors with the same size distribution μ is equal to $n^{k_1}n^{k_2} \dots n^{k_m} = n^n$.

Consider now a path in the tree leading from the initial uncertainty vector to a trivial uncertainty vector. Let U_1 and U_2 be uncertainty vectors along this path, with corresponding partitions μ_1 and μ_2 . Clearly, if U_2 is a successor of U_1 then the size distribution of U_2 is either equal to that of U_1 or is a refinement of that of U_1 ; i.e., $\mu_1 \geq \mu_2$. Also, since the initial uncertainty vector contains n states, there are at most $n - 1$ possible refinements of partitions along the path leading to the distinguishing sequence. Accordingly, the length of this path is $L \leq (n - 1)n^n$. $\diamond$

The above bound is not necessarily the least upper bound.

Notes and references

The study of machine behavior from terminal experiments was first introduced by Moore [13] in 1956. He established the notions of homing, synchronizing, and distinguishing experiments and derived bounds on their lengths. Moore's ideas were further developed by Gill [5], who simplified the search for the homing and distinguishing sequences, Ginsburg [6], Hibbard [8], and Kohavi and Winograd [12]. The material on checking experiments is taken from Hennie [7], Kohavi and Lavalley [10], Kohavi and Kohavi [9], and Kohavi *et al.* [11]. State-table-based test generation using a functional fault model was presented by Cheng and Jou [2]. A survey of sequential test generation methods was presented by Cheng [3]. Sequential test generation based on nine-valued logic was first presented by Muth [14]. Scan design was first discussed by Williams and Angell [15]. A level-sensitive scan design, which is quite influential, was discussed by Eichelberger and Williams [4]. A more detailed description of BIST techniques can be found in the book by Bardell, McAnney, and Savir [1].

- [1] Bardell, P. H., W. H. McAnney, and J. Savir: *Built-in Test for VLSI: Pseudorandom Techniques*, John Wiley & Sons, 1987.
- [2] Cheng, K.-T., and J.-Y. Jou: "A functional fault model for finite state machines," *IEEE Trans. Computer-Aided Design*, vol. 11, no. 9, pp. 1065–1073, September 1992.
- [3] Cheng, K.-T.: "Gate-level test generation for sequential circuits: a survey," *ACM Trans. Design Automation of Electronic Systems*, vol. 1, no. 3, pp. 405–442, 1996.
- [4] Eichelberger, E. B., and T. W. Williams: "A logic design structure for design for testability," in *Proc. Design Automation Conf.*, pp. 462–468, June 1977.
- [5] Gill, A.: "State-identification experiments in finite automata," *Information and Control*, vol. 4, pp. 132–154, 1961.
- [6] Ginsburg, S.: "On the length of the smallest uniform experiment which distinguishes the terminal states of a machine," *J. Assoc. Computing Machinery*, vol. 5, pp. 266–280, July 1958.
- [7] Hennie, F. C.: "Fault detecting experiments for sequential circuits," in *Proc. Fifth Ann. Symp. Switching Circuit Theory and Logical Design*, pp. 95–110, November 1964.
- [8] Hibbard, T. N.: "Least upper bounds on minimal terminal state experiments for two classes of sequential machines," *J. Assoc. Computing Machinery*, vol. 8, pp. 601–612, October 1961.
- [9] Kohavi, I., and Z. Kohavi: "Variable-length distinguishing sequences and their application to the design of fault-detection experiments," *IEEE Trans. Computers*, vol. C-17, pp. 792–795, August 1968.
- [10] Kohavi, Z., and P. Lavallee: "Design of sequential machines with fault-detection capabilities," *IEEE Trans. Electron. Computers*, vol. EC-16, pp. 473–484, August 1967.
- [11] Kohavi, Z., J. A. Rivierre, and I. Kohavi: "Checking experiments for sequential machines," *Information Sciences*, vol. 7, no. 1, pp. 11–28, January 1974.
- [12] Kohavi, Z., and J. Winograd: "Establishing bounds concerning finite automata," *J. Computer & System Sciences*, vol. 7, no. 3, pp. 288–299, June 1973.
- [13] Moore, E. F.: "Gedanken-experiments on sequential machines," pp. 129–153, *Automata Studies*, Princeton University Press, 1956.
- [14] Muth, P.: "A nine-valued circuit model for test generation," *IEEE Trans. Computers*, vol. C-25, no. 6, pp. 630–636, June 1976.
- [15] Williams, M., and J. Angell: "Enhancing testability of large-scale integrated circuits via test points and additional logic," *IEEE Trans. Computers*, vol. C-32, pp. 46–60, 1973.

Problems

Problem 13.1. For each machine shown in Table P13.1:

- (a) find the shortest homing sequences;
- (b) determine whether synchronizing sequences exist, and if any do exist, find the shortest ones.

Table P13.1

NS, z			NS, z			NS, z		
PS	$x = 0$	$x = 1$	PS	$x = 0$	$x = 1$	PS	$x = 0$	$x = 1$
A	$A, 1$	$E, 0$	A	$B, 0$	$A, 0$	A	$C, 0$	$D, 1$
B	$A, 0$	$C, 0$	B	$B, 1$	$C, 1$	B	$C, 0$	$A, 1$
C	$B, 0$	$D, 1$	C	$A, 1$	$D, 0$	C	$A, 1$	$B, 0$
D	$C, 1$	$C, 0$	D	$C, 0$	$A, 1$	D	$B, 0$	$C, 1$
E	$C, 0$	$D, 0$						
M_1			M_2			M_3		

Problem 13.2. It is necessary to synchronize the machine of Table P13.2 to state A with a minimum number of input symbols. Devise such a procedure.

Table P13.2

NS, z		
PS	$x = 0$	$x = 1$
A	$C, 1$	$E, 1$
B	$A, 0$	$D, 1$
C	$E, 0$	$D, 1$
D	$F, 1$	$A, 1$
E	$B, 1$	$F, 0$
F	$B, 1$	$C, 1$

Problem 13.3. You are presented with a machine that is known to be described by one of the two state tables shown in Table P13.3. No information is available regarding the initial state of the machine. Devise a procedure for identifying the machine, and find all minimal preset experiments that can perform this task.

Hint: Construct a machine which is the *direct sum* of the two machines.

Table P13.3

NS, z			NS, z		
PS	$x = 0$	$x = 1$	PS	$x = 0$	$x = 1$
A	$A, 0$	$B, 0$	D	$E, 0$	$F, 1$
B	$C, 0$	$A, 0$	E	$F, 0$	$D, 0$
C	$A, 1$	$B, 0$	F	$E, 0$	$F, 0$

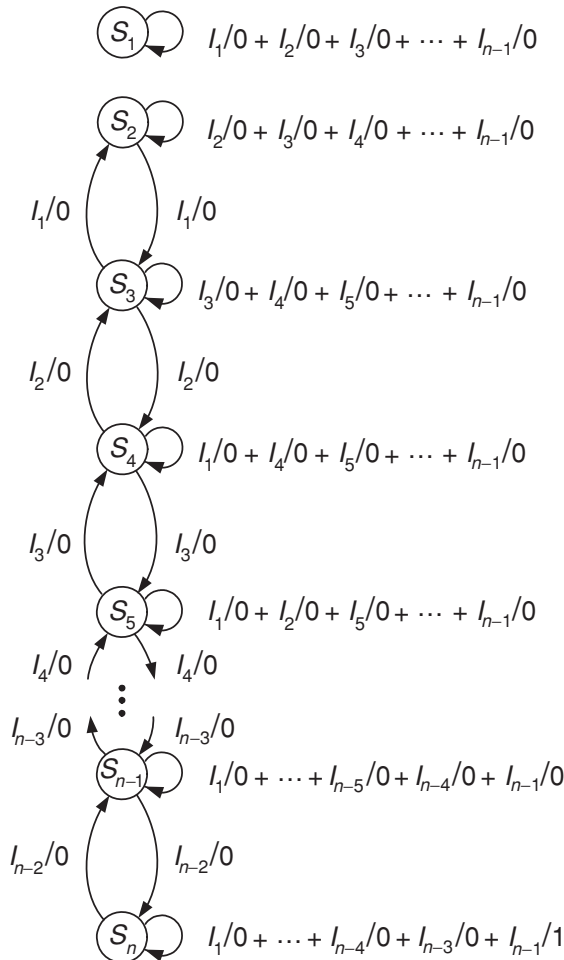
Problem 13.4. Find the shortest homing sequence for the machine shown in Table P13.4. (Note that this machine is a special case, $n = 4$, of the machine of Fig. P13.5.)

Table P13.4

PS	NS, z		
	I_1	I_2	I_3
S_1	$S_1, 0$	$S_1, 0$	$S_1, 0$
S_2	$S_3, 0$	$S_2, 0$	$S_2, 0$
S_3	$S_2, 0$	$S_4, 0$	$S_3, 0$
S_4	$S_4, 0$	$S_3, 0$	$S_4, 1$

Problem 13.5. It can be shown that every n -state machine has a preset homing sequence whose length does not exceed $\frac{1}{2}(n-1)n$. By referring to Fig. P13.5, prove that this bound cannot be lowered; i.e., there exists a class of machines the length of whose homing sequences is precisely $\frac{1}{2}(n-1)n$.

Fig. P13.5



Problem 13.6

- (a) Find a single sequence of 0's and 1's that can serve as a homing sequence for all reduced and strongly connected three-state machines whose input symbols are 0 and 1.
- (b) Can you generalize the result of part (a) to n -state machines? Show a bound on the length of such sequences.

Problem 13.7. Prove that, in a reduced n -state machine, every set of $n - k$ states ($n - 2 \geq k \geq 0$) contains at least one pair of states that is distinguishable by an experiment of length $k + 1$.

Problem 13.8. It is necessary to determine the final state of the machine shown in Table P13.8 when the initial state is unknown and only output sequences from the machine are available to the experimenter; that is, no information regarding the input to the machine is available.

- (a) Devise a procedure to determine whether a specific output sequence can be used to identify the final state of the machine.
- (b) Find a reduced standard-form state table that accepts precisely those output sequences which can be used to identify the final state of the machine. Use the state names A , B , etc.

Table P13.8

PS	NS, z	
	$x = 0$	$x = 1$
A	$B, 0$	$C, 0$
B	$A, 0$	$D, 1$
C	$D, 1$	$B, 0$
D	$A, 1$	$D, 1$

Problem 13.9. For each of the machines shown in Table P13.9, determine whether preset distinguishing sequences exist, and if any do exist then find the shortest ones.

Table P13.9

PS	NS, z	
	$x = 0$	$x = 1$
A	$C, 1$	$A, 0$
B	$D, 0$	$D, 0$
C	$A, 0$	$D, 0$
D	$B, 0$	$C, 0$
M_1		

PS	NS, z	
	$x = 0$	$x = 1$
A	$D, 0$	$C, 1$
B	$A, 0$	$B, 1$
C	$E, 0$	$B, 1$
D	$B, 0$	$D, 1$
E	$C, 1$	$E, 1$
M_2		

PS	NS, z	
	$x = 0$	$x = 1$
A	$A, 0$	$E, 1$
B	$E, 1$	$A, 0$
C	$F, 1$	$B, 0$
D	$B, 0$	$F, 1$
E	$C, 1$	$G, 0$
F	$G, 0$	$C, 1$
G	$H, 0$	$D, 1$
H	$D, 1$	$H, 0$
M_3		

Problem 13.10

- (a) Find a preset distinguishing experiment that determines the initial state of the machine shown in Table P13.10, given that it cannot initially be in state E .
- (b) Can you identify the initial state when the initial uncertainty is $(ABCDE)$?

Table P13.10

PS	NS, z	
	$x = 0$	$x = 1$
A	$B, 1$	$A, 1$
B	$E, 0$	$A, 1$
C	$A, 0$	$E, 1$
D	$C, 1$	$D, 1$
E	$E, 0$	$D, 1$

Problem 13.11. Specify the entries marked * in the machine of Table P13.11 in such a way that the machine will be strongly connected and the sequences 000 and 111 will be distinguishing sequences.

Table P13.11

PS	NS, z	
	$x = 0$	$x = 1$
A	*, 0	*, 0
B	$C, 0$	$D, 0$
C	$A, 0$	$B, 0$
D	$D, 1$	$A, 1$

Problem 13.12. Prove that the length L of the minimal distinguishing sequence for a machine with n states and q output symbols is bounded by

$$L \geq \frac{\log_2 n}{\log_2 q}.$$

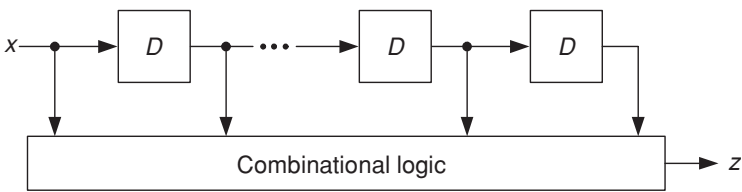
Problem 13.13. Let M be a reduced n -state machine with input alphabet $I = \{I_1, I_2, \dots, I_p\}$.

- (a) Prove that if, for every input symbol I_i in M , there exists a pair of states whose successors are identical while producing the same output symbol in response to I_i then M does not have any distinguishing sequence.
- (b) Prove that if there exists no such pair of states as that described in (a) for any input symbol I_i in M then M has a preset distinguishing sequence whose length is at most $\frac{1}{2}n(n-1)$.

Problem 13.14

- (a) Show that every machine of the form in Fig. P13.14 has a synchronizing sequence. Find such a sequence and specify its length.

Fig. P13.14



- (b) Does every machine of this form also have a distinguishing sequence? Prove that it does or show a counter-example.
- (c) Can every finite-state machine be realized in this form?

Problem 13.15. The response of the machine shown in Table P13.15 to an unknown input sequence is given to an experimenter. Devise a procedure that the experimenter may use in order to identify the initial state. What are the minimum-length sequences that will make such an identification possible?

Table P13.15

<i>PS</i>	<i>NS, z</i>	
	<i>x = 0</i>	<i>x = 1</i>
<i>A</i>	<i>A, 0</i>	<i>B, 0</i>
<i>B</i>	<i>C, 0</i>	<i>D, 0</i>
<i>C</i>	<i>D, 1</i>	<i>C, 1</i>
<i>D</i>	<i>B, 1</i>	<i>A, 1</i>

Problem 13.16. The machine shown in Table P13.16 is initially provided with an input sequence 01 to which it responds by producing an output sequence 10. It is next provided with the sequence 10101010010011010001. Assuming that no fault increases the number of states, show that this sequence is a checking experiment for this machine and find the correct output sequence.

Table P13.16

<i>PS</i>	<i>NS, z</i>	
	<i>x = 0</i>	<i>x = 1</i>
<i>A</i>	<i>A, 1</i>	<i>B, 0</i>
<i>B</i>	<i>C, 0</i>	<i>A, 0</i>
<i>C</i>	<i>B, 0</i>	<i>C, 1</i>

Problem 13.17. The initial state of the machine shown in Table P13.17 is *A*, but its entry in row *D*, column 1, is unknown. An input sequence 0110 was applied to the machine, which produced an output sequence whose last two symbols are 00. Following this sequence, a sequence 101 was applied, and this in turn produced an output sequence whose last symbol is a 0. Determine the missing entry.

Table P13.17

<i>PS</i>	<i>NS, z</i>	
	<i>x</i> = 0	<i>x</i> = 1
<i>A</i>	<i>B</i> , 0	<i>C</i> , 1
<i>B</i>	<i>A</i> , 1	<i>D</i> , 1
<i>C</i>	<i>C</i> , 0	<i>A</i> , 1
<i>D</i>	<i>E</i> , 1	*
<i>E</i>	<i>A</i> , 0	<i>E</i> , 0

Problem 13.18. The input sequence *X* shown below was applied to a reduced five-state machine whose state table is to be determined. In response, the machine produced output sequence *Z*. Give the state table of the machine in standard form if its starting state is *A*.

X : 0 0 0 0 1 0 1 0 1 0 1 0 0 1 0 1 0 0 0 1 0 0 1 0
Z : 0 1 2 0 1 3 2 1 1 0 1 3 3 2 0 1 3 3 3 2 1 2 1 1

Problem 13.19. Construct a checking experiment for the machine of Table P13.19. (Such an experiment need not require more than 24 symbols.)

Table P13.19

<i>PS</i>	<i>NS, z</i>	
	<i>x</i> = 0	<i>x</i> = 1
<i>A</i>	<i>D</i> , 0	<i>C</i> , 0
<i>B</i>	<i>C</i> , 0	<i>D</i> , 0
<i>C</i>	<i>A</i> , 0	<i>B</i> , 0
<i>D</i>	<i>D</i> , 1	<i>A</i> , 1

Problem 13.20. The following experiment was proposed as a checking experiment for the machine shown in Table P13.20, when started in state *A* and under the assumption that the number of states will not increase as a result of a fault. Either prove that it is a proper checking experiment, i.e., that it identifies the machine uniquely, or show by

Table P13.20

<i>PS</i>	<i>NS, z</i>	
	<i>x</i> = 0	<i>x</i> = 1
<i>A</i>	<i>A</i> , 2	<i>B</i> , 2
<i>B</i>	<i>C</i> , 0	<i>A</i> , 1
<i>C</i>	<i>D</i> , 1	<i>E</i> , 0
<i>D</i>	<i>E</i> , 2	<i>A</i> , 0
<i>E</i>	<i>B</i> , 1	<i>C</i> , 2

means of a counter-example that it is not such an experiment.

Input : 0 0 1 0 0 1 0 1 0 1 1 0 0 0 1 0 0
Output : 2 2 2 0 1 0 2 2 0 0 2 1 2 1 1 2 2

Problem 13.21. A four-state machine received the input sequence X shown below and, in response, produced output sequence Z .

- (a) What are the distinguishing sequences for the machine?
 (b) Assuming the machine starts in state A , do the sequences below correspond to a unique machine? If yes, show its state table; if not, show all possible state tables.

X : 0 0 0 0 0 1 0 1 0 1 0 0 0 0 0 1 0 1
 Z : 0 0 1 1 0 0 1 1 1 0 0 1 1 0 0 1 1 0

Problem 13.22. By referring to the machine in Table P13.22, where $\lfloor g \rfloor$ is the largest integer not exceeding g , prove that the bound established in Section 13.6 for definite diagnosability is the least upper bound. That is, prove that for every n there exists an n -state machine, as given in Table P13.22, which is definitely diagnosable and of order $\mu = \frac{1}{2}n(n-1)$.

Table P13.22

PS	I_1	I_2	I_3
1	2, 0	3, 0	2, 0
2	3, 0	4, 0	3, 0
3	4, 0	5, 0	4, 0
$\vdots$	$\vdots$	$\vdots$	$\vdots$
i	$i + 1, 0$	$i + 2, 0$	$i + 1, 0$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
$\lfloor n/2 \rfloor - 1$	$\lfloor n/2 \rfloor, 0$	$\lfloor n/2 \rfloor + 1, 0$	$\lfloor n/2 \rfloor, 0$
$\lfloor n/2 \rfloor$	$\lfloor n/2 \rfloor + 1, 0$	$\lfloor n/2 \rfloor + 2, 1$	$\lfloor n/2 \rfloor + 1, 1$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
j	$j + 1, 0$	$j + 2, 1$	$j + 1, 1$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
$n - 2$	$n - 1, 0$	$n, 1$	$n - 1, 1$
$n - 1$	$n, 0$	$1, 1$	$n, 0$
n	$1, 1$	$1, 0$	$n, 1$

Problem 13.23

- (a) Show the testing table and graph for the machine given in Table P13.23.
 (b) Add to the machine one output terminal such that the sequence 11 becomes a distinguishing sequence.
 (c) Design a checking experiment for the augmented machine. (Twenty four symbols are sufficient.)

Table P13.23

<i>PS</i>	<i>NS, z</i>	
	<i>x</i> = 0	<i>x</i> = 1
<i>A</i>	<i>A</i> , 0	<i>B</i> , 0
<i>B</i>	<i>A</i> , 0	<i>C</i> , 0
<i>C</i>	<i>A</i> , 0	<i>D</i> , 0
<i>D</i>	<i>A</i> , 1	<i>A</i> , 0

Problem 13.24. An unknown three-state machine with two input symbols 0 and 1 is provided with input sequence *X*, and it responds by producing output sequence *Z*. These sequences are given below:

X : 1 1 0 0 1 0 1 0 1 1 1 1 1 0 0 0 1 1 0 0 1 0 1
Z : 1 0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 1 1 1 0 0 0

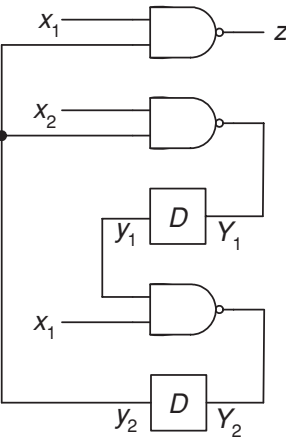
Show that this experiment is sufficient to identify the machine uniquely (up to isomorphism).

- Problem 13.25.** For the machine *M*₅ shown in Table 13.10:
- (a) obtain a minimum set of collapsed SST faults;
 - (b) derive a test sequence for the SST fault that corrupts $\langle 1, B, B, 1 \rangle$ to $\langle 1, B, C, 1 \rangle$.

- Problem 13.26.** For the circuit in Fig. 13.7(a):
- (a) find a test sequence for the fault *y s-a-1* using the extended *D*-algorithm;
 - (b) repeat (a) for the fault *y s-a-0*.

- Problem 13.27.** Consider the sequential circuit shown in Fig. P13.27. Suppose that it is to be tested for all single stuck-at faults in its combinational logic using full scan.
- (a) Find a minimal test set for its combinational logic.
 - (b) What is the minimum number of clock cycles needed to apply all vectors from your test set to the circuit using scan?

Fig. P13.27



Problem 13.28

- (a) How many loops does the state diagram of the LFSR based on feedback polynomial $p(x) = x^4 + x^2 + 1$ consist of?
- (b) Find a primitive polynomial of degree 4, and show that the state diagram of the corresponding LFSR consists of only two loops, one with the all-0 state and the other with all the remaining states.

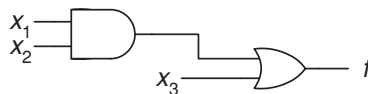
Problem 13.29

- (a) Consider an LFSR based on a primitive polynomial. Prove that if its seed is the all-0 state then it remains in the all-0 state.
- (b) Show how one can modify the design of such a k -stage LFSR such that it can generate all the 2^k states in one loop in its state diagram.
Hint: The addition of a $(k - 1)$ -input NOR gate and a two-input EXCLUSIVE-OR gate to the design shown in Fig. 13.14 is enough.
- (c) Verify that your modification of the LFSR shown in Fig. 13.15 generates all eight states in a loop.

Problem 13.30. Consider the sequence of test patterns generated by a k -stage LFSR with a feedback polynomial $p(x)$, where the values at $y_k, y_{k-1}, \dots, y_1$ are said to constitute a test pattern. The above sequence of patterns can be generated in reverse order if the k -stage LFSR is based on the feedback polynomial $x^n p(1/x)$ instead and the values at $y_1, y_2, \dots, y_k$ are said to constitute a test pattern. For example, $x^3 + x^2 + 1$ and $x^3 + x + 1$ form such a pair of polynomials. Verify the above assertion for the LFSRs based on this pair by comparing their state diagrams.

Problem 13.31. Suppose the circuit given in Fig. P13.31 is to be tested by the LFSR shown in Fig. 13.15 for all single stuck-at faults. Derive a stuck-at fault test set for this circuit such that this test set can be applied to it in four clock cycles from the LFSR, starting from a particular seed. Assume that y_1 is connected to x_1 , y_2 to x_2 , and y_3 to x_3 .

Hint: No re-seeding is necessary.

Fig. P13.31

14

Memory, definiteness, and information losslessness of finite automata

An important characteristic of a finite-state machine is that it has a “memory,” i.e., the behavior of the machine is dependent upon its past history. While the behavior of some machines depends on remote history, the behavior of others depends only on more recent events. The amount of past input and output information needed to determine the machine’s future behavior is called the *memory span* of the machine.

If the initial state of a deterministic completely specified machine and the input sequence to it are known then the corresponding final state and output sequence can be determined uniquely. However, there are special situations in which either the initial state is unknown or some past input symbols are unknown. In such situations, the behavior of the machines cannot always be predicted in advance. In this chapter, we shall try to answer the following questions. For a given machine, what is the minimum amount of past input–output information required in order to render its future behavior completely predictable? Under what conditions can the input sequence to the machine be reconstructed from its output sequence? Finally, we shall investigate some aspects of the relationship between finite-state machines and coding theory.

14.1 Memory span with respect to input–output sequences (finite-memory machines)

A finite-state machine M is defined as a *finite-memory machine of order μ* , if μ is the least integer such that the present state of M can be determined uniquely from the knowledge of the last μ input symbols and the corresponding μ output symbols. In other words, a machine is finite-memory of order μ if and only if every input sequence of length μ is a homing sequence. Consequently, the homing tree can serve as a possible tool for the detection and recognition of a finite memory for M . In this section, however, we shall derive

Table 14.1 Machine M_1

PS	NS, z	
	$x = 0$	$x = 1$
A	$B, 0$	$C, 1$
B	$D, 0$	$C, 0$
C	$D, 0$	$B, 1$
D	$C, 0$	$A, 0$

Table 14.2 Testing table for M_1

PS	0/0	0/1	1/1	1/0
A	B	—	C	—
B	D	—	—	C
C	D	—	B	—
D	C	—	—	A
AB	BD	—	—	—
AC	BD	—	BC	—
AD	BC	—	—	—
BC	DD	—	—	—
BD	CD	—	—	AC
CD	CD	—	—	—

a different test, which will be shown to be valid for all memory aspects of automata.

The testing table and testing graph¹

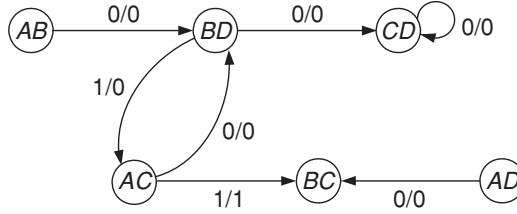
Consider a machine M_1 , whose state table is shown in Table 14.1. We may rewrite that state table as shown in the upper half of Table 14.2. The column headings of Table 14.2 consist of all input–output symbol combinations, and the entries of the upper half of the table are the next-state entries corresponding to these combinations. For example, the 1-successor of state C is B , and the corresponding output symbol is $z = 1$. Consequently, a B is entered in row C , column 1/1, of the table, and a dash (—) is entered in row C , column 1/0. The entire upper half of Table 14.2 is completed in a similar manner.

The row headings in the lower half of the table are all the unordered pairs of states, while the table entries are the corresponding successors. If the entries in rows S_i and S_j , column I_k/O_l , of the upper half are S_p and S_q respectively then the entry in row S_iS_j , column I_k/O_l , of the lower half is S_pS_q . For example, the entries in rows A and C , column 1/1, are C and B , respectively. Consequently, the entry in row AC , column 1/1, is BC . If, for some pair of states S_i and S_j , either one or both corresponding entries in some column I_k/O_l are dashes then the entry in row S_iS_j , column I_k/O_l , is a dash. For example, the entry in row AB , column 1/0, is a dash since the entry in row A , column 1/0, is a dash, and so on. The table so completed is called a *testing table for finite memory*, or simply, a *testing table*.

We shall refer to a pair of states (S_iS_j) as an *uncertainty pair*, and to its successor (S_pS_q) as the *implied pair*. Thus, for example, the pair (AC) is implied by (BD).

¹ The testing table and graph are similar to those presented in Section 13.6, but are redefined here for completeness of the presentation.

Fig. 14.1 The testing graph G_1 for M_1 .



Let us now define a directed graph G , which will be called a *testing graph* (for finite memory), in the following way.

1. Corresponding to each row in the lower half of the testing table, there is a vertex in G . The vertex label is the same as the row heading.
2. An arc is drawn leading from the vertex labeled $S_i S_j$ to the vertex labeled $S_p S_q$, where $p \neq q$, if and only if there exists an entry $S_p S_q$ in row $S_i S_j$, column I_k/O_l , of the testing table. The arc is labeled I_k/O_l . No arc is needed if $S_i S_j$ implies $S_p S_p$, e.g., DD in row BC .

The testing graph G_1 for machine M_1 is derived directly from the lower half of the testing table and is shown in Fig. 14.1.

Conditions for finite memory

Let the initial uncertainty regarding the state of machine M be $(S_1 S_2 \cdots S_n)$. M is finite-memory of order μ if the application of any input sequence of length μ transfers the machine into an identifiable state, and if there exists an input sequence of length $\mu - 1$ that, together with the corresponding output sequence, does not provide enough information for a unique identification of the final state.

Theorem 14.1 *A sequential machine M has a finite memory if and only if its testing graph G is loop-free.*

Proof Assume that G is not loop-free. Then, by repeatedly applying the symbols coinciding with the labels of the arcs in the loop, we can find an arbitrarily long input sequence that cannot resolve the uncertainty regarding the final state, thus the machine is not finite-memory. To prove sufficiency, assume that G is loop-free. If M is not finite-memory then there exists an arbitrarily long path in G corresponding to some input sequence X and some pair of states $(S_i S_j)$ such that S_i and S_j cannot be distinguished by X . However, since the number of vertices in G cannot exceed $\frac{1}{2}(n-1)n$ (corresponding to the number of distinct pairs of states), arbitrarily long paths in G are possible only if it contains a loop. Thus, the theorem is proved. $\diamond$

Table 14.3 Machine M_2

PS	NS, z	
	$x = 0$	$x = 1$
A	$B, 0$	$D, 0$
B	$C, 0$	$C, 0$
C	$D, 0$	$A, 0$
D	$D, 0$	$A, 1$

Table 14.4 Testing table for M_2

PS	0/0	0/1	1/1	1/0
A	B	—	—	D
B	C	—	—	C
C	D	—	—	A
D	D	—	A	—
AB	BC	—	—	CD
AC	BD	—	—	AD
AD	BD	—	—	—
BC	CD	—	—	AC
BD	CD	—	—	—
CD	DD	—	—	—

Example From the testing graph of M_1 (Fig. 14.1), it is evident that, since G_1 contains two loops, M_1 is not finite-memory. An arbitrarily long string of 0 input symbols will never resolve the uncertainty (CD). Similarly, if the initial uncertainty is (AC) then the input sequence $0101 \cdots 01$ will transfer the machine to (BD), (AC), (BD), $\dots$, and so on.

Corollary Let G be a loop-free testing graph for machine M . If the length of the longest path in G is l then $\mu = l + 1$.

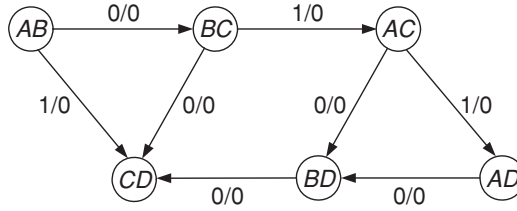
Proof Since G is loop-free, M has a finite memory. Assume that $\mu > l + 1$; then there exists at least one uncertainty pair ($S_i S_j$) that is transferred, by the application of an input sequence of length $l + 1$, to another pair ($S_p S_q$). Consequently, there must exist a path between vertices $S_i S_j$ and $S_p S_q$ in G whose length is $l + 1$. This contradicts our assumption and thus μ cannot exceed $l + 1$. The proof that μ cannot be smaller than $l + 1$ is trivial. $\diamond$

From the preceding results, it is evident that if a machine is finite-memory of order μ then $\mu \leq \frac{1}{2}(n - 1)n$.

A machine for which $\mu = \frac{1}{2}(n - 1)n$

The machine M_2 shown in Table 14.3 illustrates the case where the bound of μ is achieved. The corresponding testing table and graph are given in Table 14.4 and Fig. 14.2, respectively.

Clearly, the testing graph of M_2 is loop-free and its maximal path, emanating from AB and terminating at CD , is of length 5. Hence, $\mu = 6$. In general, it can be shown (see Problem 14.3) that there exists a class of machines for which $\mu = \frac{1}{2}(n - 1)n$ and, therefore, the bound of μ is the least upper bound and cannot be improved.

Fig. 14.2 Testing graph for M_2 .

*An algorithm to determine whether a graph is loop-free

When the number of vertices in a testing graph G is large, it is desirable to have a more systematic algorithm to determine whether it is loop-free and, if it is, the length of the longest path l . We present here one such algorithm, which does not require the actual drawing of the graph and can be easily executed by a computer.

Let G be a directed graph with p vertices. Define the *connection matrix* of G to be a $p \times p$ matrix whose (i, j) th entry is 1 if there is an arc emanating from vertex i and terminating at vertex j , and is 0 otherwise. The labels associated with the rows and columns of the matrix are the same as the labels of the vertices of G . The labels associated with corresponding rows and columns are identical; i.e., the i th row and the i th column have the same label.

The procedure for determining whether a graph is loop-free can be illustrated by means of the machine M_2 . The connection matrix of M_2 is derived directly from the testing table and is as follows:

$$\begin{array}{l}
 (AB) \\
 (AC) \\
 (AD) \\
 (BC) \\
 (BD) \\
 (CD)
 \end{array}
 \begin{bmatrix}
 0 & 0 & 0 & 1 & 0 & 1 \\
 0 & 0 & 1 & 0 & 1 & 0 \\
 0 & 0 & 0 & 0 & 1 & 0 \\
 0 & 1 & 0 & 0 & 0 & 1 \\
 0 & 0 & 0 & 0 & 0 & 1 \\
 0 & 0 & 0 & 0 & 0 & 0
 \end{bmatrix}.$$

Two arcs emanate from vertex AB : to BC and CD . Therefore, the entries in row AB , columns BC and CD , are 1, and so on.

If a directed graph G is loop-free then it has one or more terminal vertices.² Furthermore, the subgraph resulting from the removal of a terminal vertex and all arcs leading to it is also loop-free. This can be proved by observing that if G has no terminal vertex then we can construct arbitrarily long paths in G . However, since G is finite, this means that G has a loop. In the matrix representation, the removal of a vertex and all arcs leading to and from it is accomplished by the deletion from the matrix of the row and column corresponding to this vertex.

² A vertex from which no arcs emanate is called a *terminal vertex*.

The testing algorithm is summarized as follows.

1. Given a testing table, construct the corresponding connection matrix.
2. Delete all the rows having 0's in all positions and remove the corresponding columns. If there are none, go to step 4.
3. Repeat step 2.
4. If the matrix has not completely vanished then G is not loop-free. If the matrix has vanished, G is loop-free. (A "vanished" matrix has no rows or columns.)

Returning to the connection matrix of M_2 , the first application of step 2 results in the removal of the row labeled (CD) and its corresponding column. The resulting matrix is

$$\begin{array}{l} (AB) \\ (AC) \\ (AD) \\ (BC) \\ (BD) \end{array} \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

Repeated applications of step 2 result in the removal of the rows labeled (BD) , (AD) , (AC) , and so on:

$$\begin{array}{l} (AB) \\ (AC) \\ (AD) \\ (BC) \end{array} \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{array}{l} (AB) \\ (AC) \\ (BC) \end{array} \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{array}{l} (AB) \\ (BC) \end{array} \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} (AB) [0].$$

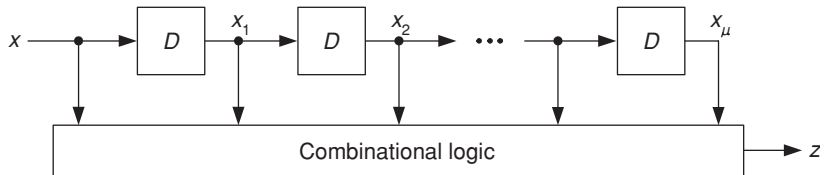
Clearly, at the next step the matrix vanishes.

We observe that at each application of step 2 we remove the terminal vertices and all arcs leading to them. Consider the terminal vertices at the end of the longest paths whose length is l . It takes $l + 1$ applications of step 2 to remove all the vertices in these paths and to eliminate the matrix. Consequently, *the number of times that step 2 is applied is equal to order μ of the memory.* In the preceding example, step 2 was applied six times; consequently, M_2 is finite-memory of order $\mu = 6$, as is already known. Note that if at some time the matrix contains two (or more) rows consisting of 0's in all their positions, all these rows and their corresponding columns must be deleted simultaneously, and this step counts as a single application of step 2.

14.2 Memory span with respect to input sequences (definite machines)

A sequential machine M is called a *definite machine of order μ* if μ is the least integer such that the present state of M can be determined uniquely from knowledge of the last μ input symbols to M . A definite machine is thus said to

Fig. 14.3 Canonical realization of a μ -definite machine.



have a finite input memory. However, for a nondefinite machine there always exists at least one input sequence of arbitrary length that does not provide enough information to identify the state of the machine. A definite machine of order μ is often called a μ -definite machine. Clearly, if a machine is μ -definite then it is also finite-memory of order equal to or smaller than μ .

The knowledge of any μ past input values is always sufficient to completely specify the present state of a μ -definite machine. Therefore, any μ -definite machine can be realized as a cascade connection of μ delay elements, which store the last μ input values, and a combinational circuit that generates the specified output value. This realization, which is often referred to as the *canonical realization of a definite machine*, is shown in Fig. 14.3.

Properties of definite machines

We shall now study some properties of definite machines, from which we shall derive tests for definiteness. The first obvious property is that a machine is definite of order μ if and only if every sequence of length μ is a synchronizing sequence. This property can be detected by means of the synchronizing tree presented in Section 13.2. The tree is terminated whenever either of the following occurs.

1. An uncertainty in the k th level is also associated with some node in a preceding level.
2. All nodes of the k th level are associated with singleton uncertainties, i.e., uncertainties that consist of a single state each.

Clearly, if the tree terminates by virtue of rule 1 then the corresponding machine is not definite. However, if the tree terminates by virtue of rule 2 then the corresponding machine is definite, since this means that every input sequence (i.e., path in the tree) leads to a unique final state. Furthermore, the length of the path determines the order of definiteness; that is, if the tree is terminated in level k and rule 2 is satisfied then the corresponding machine is k -definite. Note that if some node is associated with a singleton uncertainty then that node may become terminal, but the successors of other nodes must be determined. The order of definiteness is determined by the length of the longest path.

Example Consider the machine M_3 whose state table is given in Table 14.5. The output entries have been omitted, since only the inputs to the machine play a role in the determination of definiteness. The synchronizing tree for machine M_3 is shown in Fig. 14.4. Its length is $k = 3$ and, consequently, M_3 is definite of order 3.

Table 14.5 Machine M_3

PS	NS	
	$x = 0$	$x = 1$
A	A	B
B	C	B
C	A	D
D	C	B

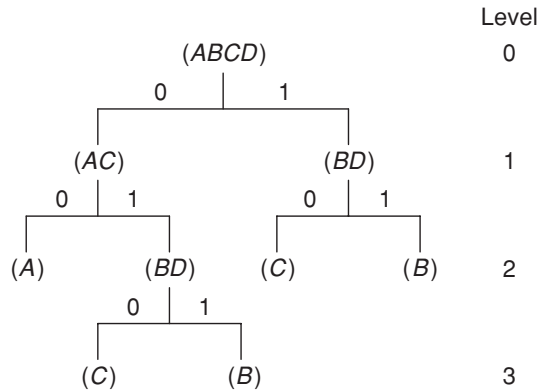


Fig. 14.4 Synchronizing tree for M_3 .

Let M be a μ -definite machine, and let $(S_i S_j)$ be a nontrivial uncertainty in the $(\mu - 1)$ th level of the corresponding synchronizing tree. Since the μ th level of the tree consists of only single states, the I_k -successors of both S_i and S_j must be identical for every possible I_k in I ; that is, every definite machine contains at least two distinct states for which $I_k S_i = I_k S_j$ for all I_k in I . Define the *contracted table* $\bar{M}$ as the table obtained by deleting row S_j and replacing in the entire table all appearances of S_j by S_i . It is easy to show that the application of any input sequence X to $\bar{M}$ or M , when initially in any state S_k such that $S_k \neq S_j$, will pass both $\bar{M}$ and M to the same final state if the final state is different from S_j and will pass $\bar{M}$ to S_i if the final state of M is S_j .

More generally, let $\bar{M}$ be the contracted table obtained from M by replacing each set of states whose I_k -successors are identical by a single member from that set. Clearly, the synchronizing tree of $\bar{M}$ has only $\mu - 1$ levels, and its

last level consists of only singleton uncertainties. However, since such a tree corresponds to a machine which is $(\mu - 1)$ -definite, we arrive at the following general result.

- If M is a μ -definite machine then the contracted machine $\overline{M}$ is $(\mu - 1)$ -definite. Conversely, if $\overline{M}$ is k -definite then M is $(k + 1)$ -definite. If $\overline{M}$ is not definite, neither is M .

Tests for definiteness

The synchronizing tree can be used to test for definiteness. In this section we shall illustrate two additional testing procedures. The first procedure, which utilizes the previously derived properties of definite machines, involves repeated derivations of contracted tables. The second procedure is based on the familiar testing graph.

The first test for the definiteness of a machine M is as follows.

1. Determine the subsets of states whose I_k -successors are identical.
2. Select one representative state in each subset.
3. Obtain the contracted table $\overline{M}$ by replacing each subset with its representative and modifying the table entries accordingly.
4. Regard $\overline{M}$ as a new table and repeat the previous steps until no new contractions are possible.

The machine M is definite if and only if the final contracted table obtained in step 4 consists of just a single state.

Example The machine M_4 of Table 14.6 will be tested for definiteness. The nontrivial subsets of states whose corresponding successors are identical are (B, F) and (C, D) . Select B and C as the representative states and obtain the contracted table $\overline{M}_4$, which consists of four states as shown in Table 14.7. States B and C in the contracted table can now be represented by

Table 14.6 Machine M_4

PS	NS	
	$x = 0$	$x = 1$
A	A	B
B	E	B
C	E	F
D	E	F
E	A	D
F	E	B

Table 14.7 The contracted machine $\overline{M}_4$

PS	NS	
	$x = 0$	$x = 1$
A	A	B
B	E	B
C	E	B
E	A	C

Table 14.8 Repeated contractions of M_4

NS			NS			NS		
PS	$x = 0$	$x = 1$	PS	$x = 0$	$x = 1$	PS	$x = 0$	$x = 1$
A	A	B	A	A	B	A	A	A
B	E	B	B	A	B	(c)		
E	A	B						
(a)			(b)					

state B , and the contracted table shown in Table 14.8a results. The fourth contraction yields a single-state machine. Thus, M_4 is definite.

We shall now show that the test for definiteness is always finite, and determine the bound on its length.

Theorem 14.2 *Given that a machine M is μ -definite, $\mu \leq n - 1$, where n is the number of states of the machine. Moreover, the order of definiteness is equal to the number of contractions needed to obtain a one-state machine.*

Proof Since M is μ -definite, $\overline{M}$ is $(\mu - 1)$ -definite. Each contracted table must contain at least one state less than its predecessor. Consequently, after at most $n - 1$ repeated contractions we obtain a one-state machine that is 0-definite, i.e., no input symbol is required in order to determine its present or final state. To determine the order of definiteness, it is necessary to count backward; that is, the last contracted table is 0-definite, its predecessor is 1-definite, and so on. $\diamond$

For machine M_4 , $\mu = 4$ since four contractions are necessary in order to obtain a one-state machine.

The second test for definiteness is based on a testing table and graph, which are defined as follows. The *testing table (for definiteness)*, which is divided into two parts, has p columns corresponding to $I_1, I_2, \dots, I_p$. The rows in the upper part of the table correspond to the states of the machine, and the table entries are the state transitions. The row headings in the lower part of the table are all unordered pairs of states, while the table entries are the corresponding successors. The *testing graph (for definiteness)* is defined as in the previous section and is derived directly from the lower part of the testing table. The arc labels, however, are now input symbols instead of input-output symbol combinations.

Example The testing table for the machine M_3 is shown in Table 14.9, and the corresponding testing graph, which is loop-free, is shown in Fig. 14.5.

Table 14.9 The testing table for M_3 (see Table 14.5)

PS	$x = 0$	$x = 1$
A	A	B
B	C	B
C	A	D
D	C	B
AB	AC	BB
AC	AA	BD
AD	AC	BB
BC	AC	BD
BD	CC	BB
CD	AC	BD

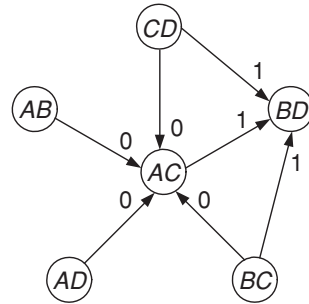


Fig. 14.5 Testing graph for M_3 .

Theorem 14.3 A machine M is μ -definite if and only if its corresponding testing graph G is loop-free. If the length of the longest path in G is l then $\mu = l + 1$.

Proof The proof is similar to that of Theorem 14.1 and is left to the reader as an exercise. $\diamond$

The machine M_3 is definite of order $\mu = 3$, since its testing graph is loop-free and the longest path in the graph is of length $l = 2$.

The relationship between the testing graph and the synchronizing tree is evident. A loop-free graph means that no uncertainty in the k th level of the tree is also associated with some node in a preceding level and, conversely, a loop in the graph means that such a situation does occur.

14.3 Memory span with respect to output sequences

A finite-state machine M is said to have an *output memory of order μ* if μ is the least integer such that the knowledge of the last μ output symbols suffices to determine the state of M at some time during the last μ transitions. In this section, emphasis is placed on the specification of the state of M at *some time* during the experiment, instead of on the identification of the final state. The case of identifying the final state is more restricted and is left to the reader as an exercise.

Test for output memory

The major tools for testing whether a given machine has a finite output memory are a modified testing table and its corresponding testing graph. The *testing table (for output memory)*, which consists of two parts, has q columns corresponding to the output symbols of the machine, i.e., $O_1, O_2, \dots, O_q$. The row names of

Table 14.10 Machine M_5

PS	NS, z	
	$x = 0$	$x = 1$
A	$B, 0$	$D, 1$
B	$C, 1$	$A, 1$
C	$B, 0$	$C, 0$
D	$C, 0$	$C, 1$

Table 14.11 Testing table for M_5

PS	$z = 0$	$z = 1$
A	B	D
B	—	(AC)
C	(BC)	—
D	C	C
AB	—	$(AD)(CD)$
AC	$(BB)(BC)$	—
AD	(BC)	(CD)
BC	—	—
BD	—	$(AC)(CC)$
CD	$(BC)(CC)$	—

the upper part of the table are the states of M . The entries in row S_i , column O_j , are the states that can be reached from S_i by single transitions associated with the output symbol O_j . We shall call these states the (*output*) O_j -successors of S_i . The entire upper half of the testing table is, actually, a listing of the output successors of the states of M and is therefore called an *output successor table*. Thus, for the machine M_5 of Table 14.10, the output 1-successors of B are A and C ; state B has no output 0-successors. This is recorded in Table 14.11 by entering AC in row B , column 1, and a dash in row B , column 0. When the reference to output successors is self-evident in the context, we shall omit the adjective “output.”

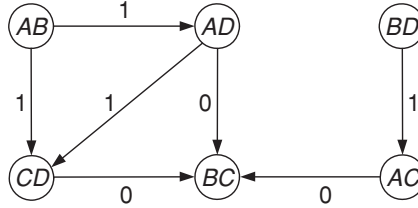
For each unordered pair of states there is a row in the lower half of the testing table. The table entries are the corresponding output successors. The output O_k -successors of $S_i S_j$ are all pairwise combinations of the output O_k -successors of S_i and S_j . For example, if the successors of S_i and S_j are $S_p S_q$ and $S_r S_t$ respectively then the corresponding successors of $S_i S_j$ are $S_p S_r$, $S_p S_t$, $S_q S_r$, $S_q S_t$. If, for some pair of states S_i and S_j , either one or both O_k -successors are dashes then the O_k -successor of $S_i S_j$ is also a dash. Thus, since the output 1-successor of C is a dash, the output 1-successor of AC is also a dash, as shown in the lower half of Table 14.11.

A *testing graph* (for output memory) G is a directed graph, such that:

1. corresponding to each row in the lower half of the testing table there is a vertex in G , whose label is the same as the row heading;
2. an arc labeled O_k is drawn from vertex $S_i S_j$ to vertex $S_p S_q$, where $p \neq q$, if and only if $S_p S_q$ is an entry at row $S_i S_j$, column O_k .

The testing graph of the machine M_5 is shown in Fig. 14.6. Note that two or more arcs having the same label may emanate from a single vertex, e.g., vertex AB .

Fig. 14.6 Testing graph G_5 for M_5 .



Theorem 14.4 *A finite-state machine M has a finite output memory if and only if its corresponding testing graph G is loop-free. Furthermore, if G is loop-free and the longest path in G is of length l then M has an output memory of order $\mu = l + 1$.*

Proof If G contains a loop, choose any two vertices in the loop, say $S_i S_j$ and $S_p S_q$; then there exist two identical output sequences, produced by M while in transition from S_i via S_p to S_i and from S_j via S_q to S_j . Since these sequences may be repeated as many times as we wish, they will never distinguish the states associated with any vertex contained in the loop and, consequently, M does not have a finite output memory. If G is loop-free but M does not have a finite output memory then, for every possible positive integer μ , there exists a path, emanating from some vertex $S_i S_j$, that does not pass M into an identifiable state. This implies arbitrarily long paths in G . However, since G is finite and loop-free, this cannot be achieved and thus M has a finite output memory.

The proof that $\mu = l + 1$ follows from the same line of argument used in the corollary in Section 14.1. $\diamond$

For example, G_5 in Fig. 14.6 is loop-free and its longest path is of length 3; this is the path from AB through AD and CD to BC . Thus, M_5 has a finite output memory of order $\mu = 4$.

Note that the testing graph does not contain any vertex corresponding to pairs consisting of repeated entries, e.g., BB , etc. The existence of such a pair means, in effect, that there is no uncertainty regarding the state of the machine. Therefore, the deletion of such pairs from the graph (or even from the testing table) does not affect the test for finite output memory.

Determining the state of the machine

If a machine M has a finite output memory, it is possible to determine the state of M at some point during any experiment of length μ . We shall now show how to identify this state when the only available information is the output sequence.

Suppose, for example, that the output sequence produced by the machine M_5 , in response to some unknown input sequence, is 1110. Initially, the machine could have been in either state A , B , or D , since no 1 output symbol can be

generated by a transition from state C . Thus, the initial uncertainty is (ABD) . From the output successor table, we find that the output 1-successor of A is D , of B is (AC) , and of D is C . Consequently, the 1-successor uncertainty of (ABD) is (ACD) . (In general, the output successor of a set of states Q is the set consisting of all output successors of the members of Q .) In a similar manner, we find that the 1-successor of (ACD) is (CD) , and so on. The next state is clearly C , as shown below:

	A	A	C	C	B
Possible uncertainties	B	C	D		C
	D	D			
Output sequence	1	1	1	0	

Note that although the state of M_5 has been identified at one point during the above experiment, the uncertainty increases to (BC) one time unit later.

The reason for suggesting the above definition of output memory, which is somewhat different from those of input–output memory or definiteness, is that the output successor table might have multivalued entries. Therefore, the identification of the state of the machine at some point during the experiment does not guarantee the identification of its successor. All we can say is that, within μ transitions corresponding to any output sequence, there must be at least one time period during which the machine is unambiguously in a certain state, regardless of the initial state.

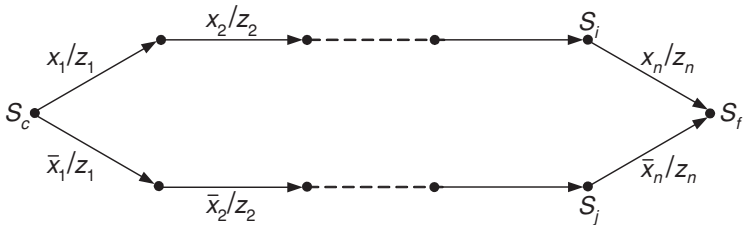
14.4 Information-lossless machines

A central problem in coding and information transmission is the determination of conditions under which it is possible to reconstruct the input sequence to the machine from the corresponding output sequence. It will be shown that whenever a machine is used as an encoding device (i.e., the machine is provided with an input sequence and its output sequence is the coded message) and when its initial and final states are known, its information losslessness guarantees that the coded message can always be deciphered. Thus, we define a machine M to be (*information*) *lossless* if the knowledge of the initial state, output sequence, and final state is sufficient to determine uniquely the input sequence.

Conditions for lossiness

A machine that is not lossless is said to be *lossy*. A simple example of a lossy machine is one in which, for some state S_i and two distinct input symbols I_p

Fig. 14.7 Condition for information loss.



and I_q , the I_p - and I_q -successors and the corresponding output symbols are identical. Clearly, in such a case, knowledge of the output sequence and the initial and final states is not sufficient to determine whether I_p or I_q was applied to the machine.

Loss of information occurs whenever two states, S_i and S_j , which can be reached from a common state S_c by means of two distinct input sequences while producing identical output sequences, merge into a final state S_f and produce the same output sequence. Clearly, once the machine has reached state S_f , no future experiment will make possible the retrieval of the input sequence that transferred M from S_c to S_f . This case, which is necessary and sufficient for a machine to be lossy, is illustrated in Fig. 14.7.

Example The machine M_6 of Table 14.12 is lossy, as demonstrated in Fig. 14.8. Two distinct input sequences (01 and 10) take the machine from state A to state B , while producing identical output sequences (00). After M_6 has reached state B , it is impossible to determine which input sequence actually occurred.

Table 14.12 Machine M_6

PS	NS, z	
	$x = 0$	$x = 1$
A	$A, 0$	$B, 0$
B	$B, 0$	$A, 1$

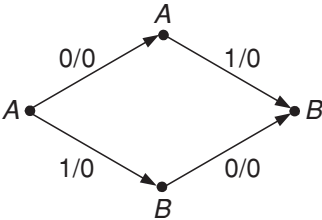


Fig. 14.8 Demonstration that M_6 is lossy.

From the foregoing discussion it is evident that in order to test a machine for losslessness, it is first necessary to determine whether, for a given state, two or more successors and their corresponding output sequences are identical or whether a merger of the type illustrated in Fig. 14.7 exists. Before presenting a test for information losslessness, we shall define an “order” of losslessness.

Table 14.13 Machine M_7

PS	NS, z	
	$x = 0$	$x = 1$
A	$C, 1$	$D, 0$
B	$D, 0$	$A, 1$
C	$D, 1$	$B, 0$
D	$C, 0$	$B, 1$

Information losslessness of finite order

Suppose that a system of lossless machines is used for encoding and decoding purposes. The “encoder” receives an input sequence and, in turn, produces an output sequence, which is transmitted to a “decoder.” Clearly, if the encoder is lossless then its input sequence can be reconstructed from its output sequence as well as the information regarding its initial and final states. The major drawback in such a decoding process lies in the fact that the information regarding the final state is transmitted by the encoder only after the entire message has been transmitted. Consequently, the entire message must be stored before the deciphering process can begin. In addition, since the output sequence may be arbitrarily long, the lossless machine cannot serve as a practical tool for encoding and decoding purposes. In view of this limitation, it becomes desirable to look for machines for which it is not necessary to store the entire message, but where the deciphering process can start when only the initial state and a finite length of the output sequence are available.

A machine is said to be (*information*) *lossless of finite order* if the knowledge of the initial state and the first μ output symbols is sufficient to determine uniquely the first input symbol. Knowledge of the initial state and the first input symbol is sufficient to determine the next state, and thus the second input symbol can be computed from the $(\mu + 1)$ th output symbol, and so on. The integer μ that is a measure of the delay in the deciphering of the input symbols is said to be the *order* of losslessness if μ is the least integer satisfying the above definition, that is, if for some initial state and a sequence of $\mu - 1$ output symbols there exist at least two possible input sequences that differ in their initial input symbols.

The simplest example of lossless machines of finite order is that of first order, where the first input symbol can be determined from knowledge of the initial state and the first output symbol. Hence, there is no delay in deciphering the input symbols for this class of machines. As an example, consider the machine M_7 shown in Table 14.13. Since for every state of M_7 , the output symbol associated with the 0-successor is different from the output symbol associated with the 1-successor, knowing the initial state and first output symbol is sufficient to identify the first input symbol. For example, if M_7 is initially

Table 14.14 Machine M_8

PS	NS, z	
	$x = 0$	$x = 1$
A	$A, 1$	$C, 1$
B	$E, 0$	$B, 1$
C	$D, 0$	$A, 0$
D	$C, 0$	$B, 0$
E	$B, 1$	$A, 0$

Table 14.15 Testing table for M_8

PS	$z = 0$	$z = 1$
A	—	(AC)
B	E	B
C	(AD)	—
D	(BC)	—
E	A	B
AC	—	—
AD	—	—
BC	$(AE)(DE)$	—
AE	—	$(AB)(BC)$
DE	$(AB)(AC)$	—
AB	—	$(AB)(BC)$

in state A and if, in response to an as yet unknown input symbol, output symbol 1 is produced then we can unambiguously identify the input symbol as a 0.

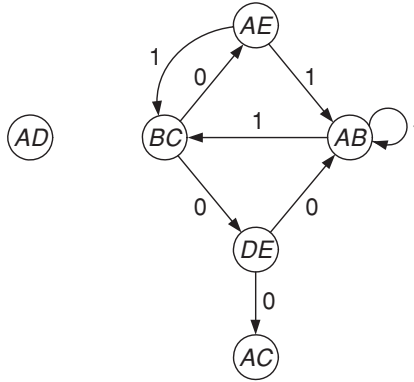
Test for information losslessness

We now derive a test to determine whether a given machine is lossless and to find its order of losslessness if it is finite. Before proceeding with the testing procedure, we introduce some terminology that facilitates discussion on information losslessness. Two states S_i and S_j are said to be (*output*) *compatible* if there exists some state S_p such that both S_i and S_j are its O_k -successors, or if there exists a compatible pair of states S_r, S_t such that S_i, S_j are their O_k -successors. In such a case, we say that the compatible pair $(S_i S_j)$ is *implied* by $(S_r S_t)$.

The first step in the testing procedure is to check each row of the state table for the appearance of two identical next-state entries associated with the same output symbol. If no identical entries appear, the next step is to construct the output successor table. A *testing table (for information losslessness)* is now constructed in two parts. The upper part consists of the output successor table, while the lower part is constructed in the following manner. Every compatible pair appearing in the successor table is made a row heading in the lower part of the testing table. The successors of these pairs are found in the usual way; they consist of all implied compatible pairs. Any implied pair that has not yet been used as a row heading is now made a row heading, its successors found, and so on. The process terminates when all compatible pairs have been used as row headings.

The machine M_8 given in Table 14.14 may be used to illustrate the testing procedure. The output successor table is shown in the upper half of Table 14.15. The pair (AC) is compatible, since both A and C are the output 1-successors

Fig. 14.9 Testing graph G_8 for M_8 .



of A . Similarly, the pairs (AD) and (BC) are compatible. Consequently, these pairs are used as row headings for the lower part of the testing table. The pairs (AE) and (DE) , which are implied by (BC) , are now made row headings, and so on.

Note that, contrary to the testing procedure for finite output memory, the testing table for information losslessness does not necessarily include all distinct pairs of states; it includes only the compatible pairs.

At this point, we are ready to derive necessary and sufficient conditions for a machine to be information lossless. Suppose that the testing table contains a compatible pair consisting of repeated entries, e.g., $(S_k S_k)$; then there exists either some compatible pair $(S_i S_j)$ that implies $(S_k S_k)$ or some state S_i that has identical output successors for two or more input symbols. However, since these cases have been shown to be necessary and sufficient for lossiness, the machine in question must be lossy. We thus arrive at the following general result.

- A machine is lossless if and only if its testing table does not contain any compatible pair consisting of repeated entries.

A testing graph (for information losslessness) G is a directed graph such that:

1. corresponding to every compatible pair there is a vertex in G ;
2. an arc labeled O_k is drawn from vertex $S_i S_j$ to vertex $S_p S_q$, where $p \neq q$, if and only if $(S_p S_q)$ is a compatible implied by $(S_i S_j)$.

The testing graph G_8 of M_8 is derived in the usual way from the lower half of the testing table and is shown in Fig. 14.9. The machine M_8 is clearly lossless, because there are no compatible pairs consisting of repeated entries. Before determining the order of losslessness, we prove the following theorem.

Theorem 14.5 *A machine M is lossless of order $\mu = l + 2$ if and only if its testing graph is loop-free and the length of the longest path in the graph is l .*

Proof Assume that M is lossless. Suppose that G is not loop-free, and let $S_i S_j$ be some vertex in the loop. Clearly, every compatible pair is accessible from some state of M by a pair of distinct input sequences that yield identical output sequences. Thus, we can find a pair of different input sequences that take M to $S_i S_j$ while producing identical output sequences. If we now observe the output symbols that the machine produces while going through all the compatible pairs in the loop, we find that the machine is back in $S_i S_j$ without supplying any additional information to make possible the identification of the first input symbol. In addition, since this loop may be repeated as many times as we wish, we may construct a pair of arbitrarily long input sequences that start in the same state of M and differ in the first symbol but produce identical output sequences. Thus, M is not lossless of finite order. The proof that the loop-free condition is indeed sufficient for finite order is trivial and follows the line of arguments used in the proof of Theorem 14.1.

To determine the order of losslessness, consider the longest path in G . It takes one input symbol to get from a state of M into the first compatible (pair), and it takes l input symbols to go through the longest path in G . Since the compatible that has been reached after $l + 1$ input symbols does not imply any other compatible, one more input symbol will yield different output symbols, depending on which state of the compatible the machine is in. This, in turn, determines the initial input symbol. Thus, $\mu = l + 2$ output symbols (plus the knowledge of the initial state) are sufficient to determine the first input symbol. $\diamond$

From Theorem 14.5 we conclude that if M is lossless of order μ then $\mu \leq 1 + \frac{1}{2}n(n - 1)$. The proof that this is indeed the least upper bound is given in Appendix 14.1.

The case $\mu = 1$ is detected by the absence of compatible pairs (see the machine M_7), while the case $\mu = 2$ is detected by the absence of arcs in the graph.

Returning to the machine M_8 , we observe that, since G_8 is not loop-free, M_8 is not lossless of finite order. It is interesting to note that M_8 is lossless even though state A can be reached by input symbol 1 from both states C and E and the output symbol produced is 0. This situation does not imply lossiness, since the pair (CE) is not compatible, i.e., C and E cannot be reached from any initial state by means of two distinct input sequences while producing identical output sequences.

Example As another illustration, the above test is applied to the machine M_9 of Table 14.16. This machine is shown to be lossless of order 3,

since its testing graph (Fig. 14.10) is loop-free and the longest path is of length 1.

Table 14.16 Machine M_9

<i>PS</i>	<i>NS, z</i>	
	<i>x = 0</i>	<i>x = 1</i>
<i>A</i>	<i>A, 0</i>	<i>B, 0</i>
<i>B</i>	<i>C, 0</i>	<i>D, 0</i>
<i>C</i>	<i>D, 1</i>	<i>C, 1</i>
<i>D</i>	<i>B, 1</i>	<i>A, 1</i>

Table 14.17 Testing table for M_9

<i>PS</i>	<i>z = 0</i>	<i>z = 1</i>
<i>A</i>	(<i>AB</i>)	—
<i>B</i>	(<i>CD</i>)	—
<i>C</i>	—	(<i>CD</i>)
<i>D</i>	—	(<i>AB</i>)
<i>AB</i>	(<i>AC</i>)(<i>AD</i>) (<i>BC</i>)(<i>BD</i>)	—
<i>CD</i>	—	(<i>AC</i>)(<i>AD</i>) (<i>BC</i>)(<i>BD</i>)

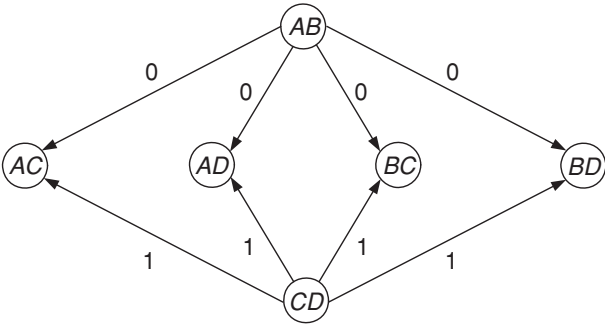


Fig. 14.10 Testing graph for machine M_9 .

Retrieval of the input sequence

Knowledge of the output sequence produced by a lossless machine, as well as its initial and final states, is sufficient to determine the input sequence applied to the machine. We shall now present a procedure to retrieve the input sequence by first reconstructing the state sequence. Since the machine is lossless, the input sequence is uniquely specified by the state sequence.

Let M be a lossless machine that is initially in a known state and, after producing a given output sequence of length r , terminates in a known final state. Suppose that we now wish to determine the state of the machine just after it has produced the j th output symbol. By applying the first j output symbols to the output successor table, starting from the known initial state, we can find a set of states in which the machine could be. In an analogous way, we can trace the predecessors of the final state by applying (in reverse order) the $r - j$ output symbols to the output predecessor table (which will be defined shortly). This last step yields a set of possible predecessors just prior to the production of

the $(j + 1)$ th output symbol. Clearly, since the machine is lossless, there is only one state in which it could have been at the time in question; the intersection of the set derived from the successor table and the set derived from the predecessor table will reveal this state.

As an example, consider the machine M_8 (see Table 14.14). Assume that this machine was initially in state A , has in response to a yet unknown input sequence produced the output sequence 110001100101, and has terminated in state B . From the output successor table (Table 14.15), we find that the 1-successors of A are A and C and the 1-successors of AC are also A and C . Just after the third output symbol, the machine could have been in either state A or D , since AD is the 0-successor of AC . Similar reasoning is used to find the states in which the machine could be after the production of every output symbol. These steps can be summarized as follows, moving from left to right:

	A	A	A	B	A	A	A	A	A	A	A	
A	C	C	D	C	D	B	B	D	B	B	D	B
					E	C	C	E	C	C	E	
	1	1	0	0	0	1	1	0	0	1	0	1

We have not yet utilized the information that can be obtained from the final state. This is best accomplished by an (*output*) *predecessor table*, which is constructed as follows. There is a column labeled O_k in the table for each output symbol O_k in O and a row for each state of the machine. The entries in row S_i , column O_k , are those states for which S_i is an output O_k -successor. These states are often referred to as the (*output*) O_k -*predecessors* of state S_i . The output predecessors of each machine state can be found directly from the state table. For convenience, the row headings of the predecessor table are placed on the right-hand side of the table. This emphasizes the fact that the row headings are the successors of the corresponding table entries.

For example, state B of the machine M_8 can be reached by a single transition from states B and E while producing output symbol 1 and from state D while producing output symbol 0. Thus, the entry in row B , column 1, of the output predecessor table (Table 14.18) is BE while the entry in row B , column 0, is D . In a similar manner, we can obtain the entire predecessor table.

Table 14.18 Output predecessor table for machine M_8

$z = 0$	$z = 1$	NS
CE	A	A
D	BE	B
D	A	C
C	—	D
B	—	E

Fig. 14.11 Retrieval of an input sequence.

Possible successors to initial state:	A	A	A	A	B	A	A	A	A	A	A	A
	C	C	D	C	D	B	B	D	B	B	D	B
					E	C	C	E	C	C	E	
Output sequence:	1	1	0	0	0	1	1	0	0	1	0	1
Possible predecessors to final state:	A	A	C	B	C	A	A	C	B	B	B	B
	D	D	D	E	A	A	D	D	E	D	E	B
State sequence:	A	A	C	D	C	A	A	C	D	B	B	E
Input sequence:	0	1	0	0	1	0	1	0	1	1	0	0

If we now wish to determine the state of M_8 just prior to the production of the last output symbol, we look for the output 1-predecessors of state B , which is known to be the final state. As shown before, the 1-predecessors of B are B and E . However, from the output successor table we have found that, at the time in question, the machine could have been in one of states A , D , or E . In addition, since it could have been in only one state at that time, this state must be given by the intersection of (B, E) and (A, D, E) . Therefore, the 1-predecessor of B is E . The entire procedure is summarized in Fig. 14.11. It is easy to verify by means of the state table that the input sequence that corresponds to the state sequence in Fig. 14.11 is 010010101100.

Whenever a given output sequence has been generated by a lossless machine, the state transitions and input sequence can be determined uniquely. If, however, at some point the intersection of the sets containing the possible successors and predecessors consists of two or more states then there exist at least two distinct input sequences that produce identical output sequences. Therefore, the machine in question is not lossless. If at some point the intersection is empty then the corresponding output sequence could not have been produced by the given machine subject to the specified initial and final states. In fact, if the intersection is empty at one point then it must be empty at all points.

Inverse machines

An *inverse* M^i is a machine which, when excited by the output sequence of a machine M , produces as its output the input sequence to M , after at most a finite delay. Evidently, a deterministic inverse can be constructed only if M is lossless, and it can be constructed such that it produces M 's input sequence after just a finite delay if and only if M is lossless of finite order.

Consider, for example, the machine M_7 of Table 14.13, which is lossless of first order. For any possible initial state and output sequence, knowledge of the initial state of M_7 and the first output symbol is sufficient to determine uniquely the first input symbol to the machine. Hence, there is no delay in deciphering the input symbols to this machine. The state transitions of the inverse machine

Table 14.19 Machine M_7^i

PS	NS, x	
	$z = 0$	$z = 1$
A	$D, 1$	$C, 0$
B	$D, 0$	$A, 1$
C	$B, 1$	$D, 0$
D	$C, 0$	$B, 1$

M_7^i are, therefore, given by the output successor table, as shown in Table 14.19. The output symbols associated with these state transitions are found by means of the state table of the machine M_7 . If M_7^i is placed in cascade with M_7 , it will produce as its output sequence an exact replica of the input sequence to M_7 .

For every lossless machine of order μ , knowledge of the state at time $t - \mu + 1$ and of the last μ output symbols, i.e., $z(t - \mu + 1), z(t - \mu + 2), \dots, z(t)$, is sufficient to determine uniquely the input symbol $x(t - \mu + 1)$. Consequently, if we send the output sequence produced by a lossless machine M of order μ into a register that consists of $\mu - 1$ delay units, we can design a combinational circuit that has as inputs the contents of that register and the state of M at time $t - \mu + 1$ and, in turn, produces the value of $x(t - \mu + 1)$.

The combinational circuit can be specified by a truth table in which the value of $x(t - \mu + 1)$ is specified for every possible combination of $S(t - \mu + 1)$ and $z(t - \mu + 1), z(t - \mu + 2), \dots, z(t)$. The information regarding the state of M can be supplied to the combinational circuit by a copy of M that is set to be at $t = \mu - 1$ in the same state that M was in at $t = 0$ and receives as its inputs a version delayed by $\mu - 1$ time units of the inputs to M . The schematic diagram of such a deciphering system is shown in Fig. 14.12.

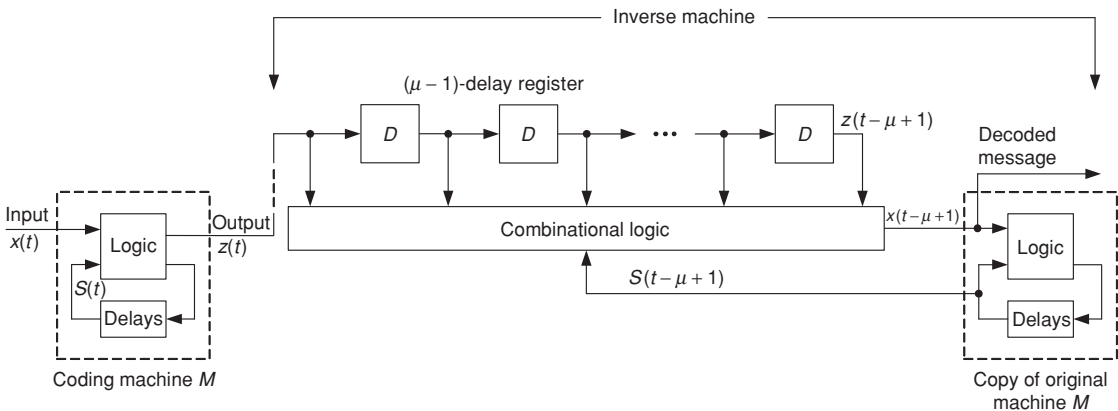


Fig. 14.12 Schematic diagram of a deciphering system.

Table 14.20 Machine M_{10}

PS	NS, z	
	$x = 0$	$x = 1$
A	$C, 0$	$D, 1$
B	$D, 0$	$C, 1$
C	$A, 0$	$B, 0$
D	$C, 1$	$D, 1$

The foregoing deciphering system does not yield an economical realization, since it requires a copy of the original machine as well as a $(\mu - 1)$ -delay register. In fact, if we were to construct a composite state table for the inverse machine (i.e., a composite table for both the register and the copy of M), we would find that in many cases it can be considerably simplified. The question that now arises is whether we can find a minimal inverse directly from M 's description, without going through the above construction procedure. Indeed, this can be accomplished, as will be shown subsequently.

*The minimal inverse machine

We shall demonstrate a construction procedure that yields a minimal inverse machine by finding the inverse of the machine M_{10} shown in Table 14.20. This machine is lossless of third order and, therefore, if we know the initial state and the values of three successive output symbols produced by transitions from this state then we can determine the first input symbol to the machine. Let us now define a set of triples, denoted $(S(t), z(t + 1), z(t + 2))$. The first member of each triple is a possible initial state of M_{10} ; the second member is one of the output symbols that can be produced by a single transition from this state; and the third member is an output symbol that can follow this initial state and the first output symbol. A triple is defined for each possible initial state and for all possible output sequences of length 2. For the machine M_{10} we obtain the following triples:

$$\begin{array}{cccc} (A, 0, 0) & (B, 0, 1) & (C, 0, 0) & (D, 1, 0) \\ (A, 1, 1) & (B, 1, 0) & (C, 0, 1) & (D, 1, 1) \end{array}$$

The triple $(A, 0, 1)$, for example, is not defined because the output sequence 01 cannot be generated by M_{10} when it is initially in state A .

The set of triples so generated contains all possible combinations of initial states and output sequences of length 2. To determine the input symbol that causes the transition from the initial state while producing the output symbol specified by the second member of the triple, all that is necessary is one additional output symbol. Accordingly, if we construct a machine, each of whose states corresponds to a triple and represents the "information" carried

Table 14.21 Machine M_{10}^i

PS	NS, x	
	$z = 0$	$z = 1$
$(A, 0, 0)$	$(C, 0, 0), 0$	$(C, 0, 1), 0$
$(A, 1, 1)$	$(D, 1, 0), 1$	$(D, 1, 1), 1$
$(B, 0, 1)$	$(D, 1, 0), 0$	$(D, 1, 1), 0$
$(B, 1, 0)$	$(C, 0, 0), 1$	$(C, 0, 1), 1$
$(C, 0, 0)$	$(A, 0, 0), 0$	$(B, 0, 1), 1$
$(C, 0, 1)$	$(B, 1, 0), 1$	$(A, 1, 1), 0$
$(D, 1, 0)$	$(C, 0, 0), 0$	$(C, 0, 1), 0$
$(D, 1, 1)$	$(D, 1, 0), 1$	$(D, 1, 1), 1$

by that triple, and if we supply the machine with the output symbols of the original machine, then it will have all the necessary information to compute the input symbols in question.

The inverse of the machine M_{10} , denoted M_{10}^i , has eight states corresponding to the eight triples derived earlier. We shall often refer to a state of the inverse machine as an *inverse state*. For every state of M_{10}^i , the next inverse state is a triple whose members are obtained in the following manner.

1. The first member is the state to which machine M_{10} goes when it is initially in the state that is the first member of the present inverse state, and when it is supplied with the first input symbol.
2. The second member is the third member of the corresponding present inverse state.
3. The third member is the present output of M_{10} .

The state table of the machine M_{10}^i is given in Table 14.21. Suppose, for example, that M_{10}^i is in the state $(A, 0, 0)$ and that its current input symbol is 0. To obtain its 0-successor, we observe that M_{10} , when initially in state A , can produce three consecutive 0 output symbols only if the first input symbol is 0; as a result, M_{10} 's first transition is to state C and the 0-successor of $(A, 0, 0)$ contains C as its first member. The second member of the triple $(C, 0, 0)$ equals the third member of $(A, 0, 0)$, while its third member is the current output symbol of M_{10} , which constitutes the current input symbol to M_{10}^i and is given by M_{10}^i 's input column heading. The output sequence of M_{10}^i is a delayed replica of the input sequence to M_{10} ; that is, the output symbol of M_{10}^i at t is equal to M_{10} 's input symbol at $t - 2$.

The set of states generated by the set of triples is clearly sufficient for a realization of the inverse machine. It does not, however, yield the smallest set of states. The machine M_{10}^i , for example, can be reduced since $(A, 0, 0)$ is equivalent to $(D, 1, 0)$ and similarly $(A, 1, 1)$ is equivalent to $(D, 1, 1)$.

Table 14.22 The minimal machine M_{10}^i

PS	NS, x	
	$z = 0$	$z = 1$
S_1	$S_5, 0$	$S_6, 0$
S_2	$S_1, 1$	$S_2, 1$
S_3	$S_1, 0$	$S_2, 0$
S_4	$S_5, 1$	$S_6, 1$
S_5	$S_1, 0$	$S_3, 1$
S_6	$S_4, 1$	$S_2, 0$

If we denote $(A, 0, 0)$ by S_1 , $(A, 1, 1)$ by S_2 , and so on, we obtain the minimal inverse, given in Table 14.22.

The foregoing procedure is applicable to any lossless machine of finite order. In general, for a machine of order μ we define a set of μ -tuples that constitutes the set of states of the inverse machine. The first member of each μ -tuple is a state of the original machine M ; the remaining members are the possible output sequences of length $\mu - 1$ that can be produced by successive transitions from that state. The fact that this procedure yields more economical realizations than the “canonic” realization of the preceding section can be explained as follows. In the canonic realization, we stored the output sequence in a shift register and used a copy of the original machine to provide the information regarding the state of the original machine. In the present realization we use the same memory devices to store information regarding both the states and output sequences, thus achieving a reduction in the number of states of the inverse machine.

Suppose that M_{10} is initially in state A and, in response to some input sequence, it produces one of the output sequences 00 or 11. Then, two units of time later, M_{10}^i must be in the state that corresponds to A and the appropriate output sequence, i.e., $(A, 0, 0)$ or $(A, 1, 1)$. However, since $S_4 = (B, 1, 0)$ is the only state from which M_{10}^i can reach $(A, 0, 0)$ and $(A, 1, 1)$ when supplied with the input sequences 00 and 11 respectively, it follows that if the initial state of M_{10} is A then the initial state of M_{10}^i must be $(B, 1, 0)$. In a similar fashion, the reader can verify that if M_{10} is initially in state B then M_{10}^i can be initially in either S_1 or S_4 and if M_{10} is initially in either state C or D then M_{10}^i can be initially in S_2, S_3, S_5 , or S_6 .

As an example demonstrating the deciphering capability of M_{10}^i let M_{10} and M_{10}^i be initially in states A and S_4 respectively and let the input sequence 010001101 be applied to M_{10} . The deciphering process is shown in Fig. 14.13. The first two output symbols of M_{10}^i , as well as the last two input symbols to M_{10} , must be ignored. In the remaining positions of both sequences, the input to M_{10} and output of M_{10}^i are identical although shifted in time.

Table 14.23 A binary code

Source symbols	Code words
<i>A</i>	00
<i>B</i>	01
<i>C</i>	11
<i>D</i>	10

Fig. 14.13 Deciphering by means of M_{10}^i .

State of M_{10} : *A* *C* *B* *D* *C* *A* *D* *D* *C* *B*
Input to M_{10} : 0 1 0 0 0 1 1 0 1
Output of M_{10} : 0 0 0 1 0 1 1 1 0
State of M_{10}^i : S_4 S_5 S_1 S_5 S_3 S_1 S_6 S_2 S_2 S_1
Output of M_{10}^i : — — 0 1 0 0 0 1 1

***14.5 Synchronizable and uniquely decipherable codes**

The objective of this section is twofold: to introduce some of the basic issues in coding theory and to demonstrate the applicability of the preceding testing techniques to the area of information transmission and codes. We do not intend to develop the entire subject of coding theory but, rather, to illustrate some aspects of this subject that are relevant to the memory and information-losslessness aspects of automata. These concepts will, therefore, be introduced without formal definitions and proofs.

Introduction

Let the symbols $\{A, B, C, \dots\}$ denote a finite *source alphabet*, and let $L = \{0, 1, 2, \dots\}$ be a *code alphabet*. We shall be concerned only with binary codes, where $L = \{0, 1\}$. A concatenation of a finite number of code symbols is referred to as a *code word*. A *code* consists of a finite number of distinct code words of finite length, each representing a source symbol. A coded message is constructed by concatenating code words without spacing or any other punctuation. For example, let the code alphabet be $L = \{0, 1\}$ and the set of code words γ_1 be $\{00, 01, 11, 10\}$. The code shown in Table 14.23 is a mapping from the source alphabet $\{A, B, C, D\}$ to γ_1 . Thus, the sequence *ABDC* would be coded as 00011011.

By using the code in Table 14.23 we may obtain a sequence of binary digits for any sequence of source symbols. We may also work backward to obtain a sequence of source symbols for any sequence of binary digits arising from this code. In fact, since each source symbol is represented by a distinct code word and all code words are of equal length, to every sequence of code words from

this code there corresponds a unique sequence of source symbols. Not in every case can we work backward and find a unique sequence of source symbols that corresponds to a given binary sequence. For example, if $\gamma_2 = \{0, 00, 01\}$ is the code representing $\{A, B, C\}$ then the sequence 0001 may be decoded as either AAC or BC .

A code is said to be *uniquely decipherable* if and only if every coded message can be decomposed into a sequence of code words in only one way. Thus, γ_1 is uniquely decipherable while γ_2 is not. Whenever the number of code symbols is not the same for all code words the code is not necessarily uniquely decipherable, as illustrated by γ_2 . However, the code $\gamma_3 = \{1, 01, 001, 0001\}$ is uniquely decipherable since the symbol 1 actually serves as a separator between successive code words. Such a separator is referred to as a *comma*, and such a code is called a *comma code*. A code in which all code words contain the same number of symbols is called a *block code*. A code in which the numbers of symbols representing different code words are not the same is called a *variable-length code*.

Whenever each code word can be deciphered without knowledge of the succeeding code words, the code is said to be an *instantaneous code*. For example, γ_1 and γ_3 are instantaneous codes while $\gamma_4 = \{1, 10, 100\}$ is not, since the sequence 10 cannot be deciphered until we verify that the next symbol is a 1.

Let $\xi = \xi_1\xi_2 \cdots \xi_n$ be a code word; then the sequence of code symbols $\xi_1\xi_2 \cdots \xi_m$, where $m \leq n$, is called a *prefix* of ξ . It can be shown that a necessary and sufficient condition for a code to be instantaneous is that no code word is a prefix of some other code word. Clearly, γ_4 is not instantaneous because 1 is a prefix of both 10 and 100.

A major reason for using variable-length codes is the consequent reduction in the average length of coded messages. Certain symbols of the source alphabet are more frequently used than others. For example, in English the letter e is more often used than the letter q . It is advantageous to assign shorter code words to those symbols that appear most often and longer code words to other symbols. If we let P_i and l_i denote, respectively, the probability of occurrence and the length of the code word representing the i th source symbol then we obtain the average length of the code, which is defined as the sum $\sum P_i l_i$ over all code words. For a given source alphabet and a given code alphabet, it is usually possible to construct many uniquely decipherable codes. In some codes, however, if an error occurs at the beginning of the coded message then it may invalidate the entire message. It is therefore desirable to have codes that are *synchronizable*, that is, for which the propagation of an error is bounded to a fixed portion of the message.

A test for unique decipherability

A code is said to be *uniquely decipherable with a finite delay μ* if and only if μ is the least integer such that knowledge of the first μ symbols of the coded

Table 14.24 Testing table for
 $\gamma = \{0, 01, 1010\}$

	0	1
S	(SB_1)	—
SB_1	—	(SC_1)
SC_1	$(SC_2)(B_1C_2)$	—
SC_2	—	(C_1C_3)
B_1C_2	—	(SC_3)
C_1C_3	(SC_2)	—
SC_3	$(SB_1)(SS)$	—

message suffices to determine its first code word. We now present a testing procedure to determine whether a code is uniquely decipherable and, if it is, the delay μ . This procedure is analogous to tests for information losslessness or for information losslessness of finite order.

Let us insert a separation symbol S at the beginning and end of each code word in γ . In addition, in every code word representing the source symbol N , we insert the symbol N_i between its i th symbol and its $(i + 1)$ th symbol. For example, if the source symbols are $\{A, B, C\}$ and $\gamma = \{0, 01, 1010\}$ then the code words with the inserted symbols are as follows:

$$\begin{array}{rcl}
 A & \rightarrow & S \ 0 \ S \\
 B & \rightarrow & S \ 0 \ B_1 \ 1 \ S \\
 C & \rightarrow & S \ 1 \ C_1 \ 0 \ C_2 \ 1 \ C_3 \ 0 \ S
 \end{array}$$

Each code symbol ξ_k is now situated between two separation symbols. We say that the separation symbol to the right of the code symbol is the ξ_k -successor, denoted R_i , of the left separation symbol. For example, C_1 is the 1-successor of S because $S1C_1$ occurs in the third code word. Two successors, R_i and R_j , are *compatible* if $S\xi_k R_i$ and $S\xi_k R_j$ occur in the code words, or if $R_p \xi_k R_i$ and $R_q \xi_k R_j$ occur, and R_p and R_q are compatible. In such a case, $(R_i R_j)$ is said to be the compatible pair *implied* by $(R_p R_q)$.

A *testing table* (for unique decipherability) can now be constructed in the following manner.

1. The column headings of the table are the symbols of the code alphabet.
2. The first row heading is S . The other row headings are the compatible pairs.
3. The entries in row $R_p R_q$, column ξ_k , are the compatible pairs implied by $(R_p R_q)$ under ξ_k .

The testing table for our example is given in Table 14.24. The entry in row S , column 0, is (SB_1) , since $S0S$ and $S0B_1$ occur in the first and second words. The compatible implied by (SB_1) is (SC_1) , since S is the 1-successor of B_1 in code word B while C_1 is the 1-successor of S in code word C ; i.e., B_11S and $S1C_1$ occur in the code words. If $(R_i R_j R_k)$ is a compatible, we enter into the

Table 14.25 The testing table for $\gamma = \{1, 10, 001\}$

		0	1
$A \rightarrow S1S$	S	—	(SB_1)
$B \rightarrow S1B_10S$	(SB_1)	(SC_1)	—
$C \rightarrow S0C_10C_21S$	(SC_1)	(C_1C_2)	—
	(C_1C_2)	—	—

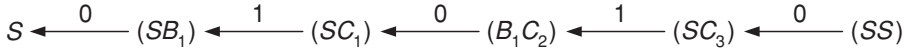
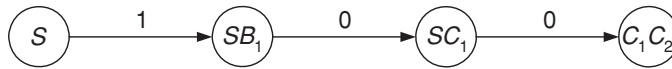
**Fig. 14.14** Determination of an ambiguous message.**Fig. 14.15** Testing graph.

table all unordered pairs $(R_i R_j)$, $(R_i R_k)$, $(R_j R_k)$. The table is complete when all the compatible pairs have been used as row headings.

If during the construction of the testing table a repeated pair (SS) occurs then the code is not uniquely decipherable. The occurrence of such a compatible pair means that there exists some compatible pair $(R_i R_j)$ such that S is the ξ -successor of both R_i and R_j . However, since both R_i and R_j (like all compatible pairs) are reachable from S by a binary sequence that corresponds to two or more different sequences of source symbols, the code is not uniquely decipherable. Moreover, by tracing back the compatible pairs that implied the pair (SS) , we can find one of the shortest ambiguous messages, which in our example is 01010, as shown in Fig. 14.14. The pair (SS) is written in the rightmost position, and its 0-predecessor is written in the next-left position, and so on. The sequence of arrow labels leading from S to (SS) is an ambiguous message. Indeed, 01010 may be interpreted as AC or as BBA .

It is easy to show that if pair (SS) is not generated then the code is uniquely decipherable. Hence, *a necessary and sufficient condition for a code to be uniquely decipherable is that a pair (SS) is not generated in the testing table.*

A testing graph (for unique decipherability) G can now be constructed as follows.

1. Corresponding to every row in the testing table, create a vertex in G .
2. Take directed arcs from each such vertex to the vertices corresponding to the implied compatible pairs.

The testing table for the code $\gamma = \{1, 10, 001\}$ is shown in Table 14.25. The corresponding testing graph is shown in Fig. 14.15. Since pair (SS) has not been generated in the testing table, the code is uniquely decipherable.

Fig. 14.16 Deciphering a coded message.

0 ' 0 , 1 ' ; 1 , 1 ' ; 0 ' 1 , 1 ' ; 0 ' 0 ' 0 ' 1 , 1 ' ; 0 1 ' ; 0 ' 0 ' 1 , 1

In analogy to Theorem 14.5, we can show that *a code is uniquely decipherable with finite delay μ if and only if its testing graph is loop-free*. The delay μ is equal to $l + 1$, where l is the length of the longest path in G . The longest path in the graph of Fig. 14.15 is 3 and thus $\mu = 4$.

Deciphering a coded message

We now describe a procedure to decipher a coded message. The decoding procedure is similar to the input-retrieval procedure for lossless machines and will be illustrated by means of an example. Consider the code $\gamma = \{11, 011, 001, 01, 00\}$, which is known to be uniquely decipherable, and suppose that we want to decode the sequence 0011101100011010011. Scanning the message from the left, we insert a lower comma whenever a sequence that corresponds to a legitimate code word is detected. For example, the first comma from the left follows the initial 00, since 00 is a code word in γ . Next, a comma follows the 1 since the sequence 001 is also a code word in γ , and so on. Although the tenth and eleventh symbols are 0's, no lower comma is inserted between the eleventh and twelfth symbols because there is no comma between the ninth and tenth symbols, and a new code word cannot start unless a comma indicates the end of the preceding code word. The procedure is illustrated in Fig. 14.16.

Next, we scan the coded message from the right and inset an upper comma whenever a sequence that corresponds to the inverse of a legitimate code word is scanned. The inverses of the code words in our example are $\{11, 110, 100, 10, 00\}$. If the code is uniquely decipherable then the message can be decoded by retaining only those commas that occur in the upper and lower spaces simultaneously. In our example, we find the following message:

001; 11; 011; 00; 011; 01; 00; 11

Although in general the above procedure will require keeping track of a number of sequences and the locations of the various commas, it is in principle a simple procedure that can be carried out by a finite-state machine.

A test for the synchronizability of codes

A code is said to be *synchronizable of order μ* if μ is the least integer such that the knowledge of any μ consecutive code symbols is sufficient to determine a separation of code words within these symbols. We shall restrict our attention to synchronizable codes that are uniquely decipherable with a finite delay, since these are the only ones of practical interest.

The problem of testing a code for synchronizability is analogous to the problem of testing a machine for finite output memory. In fact, since in both cases the objective is to specify the sequence at some point, we can use the same testing procedure. Let us construct a *testing table (for synchronizability)* in the following manner. The row headings in the upper half of the table consist of all the separation symbols. The column headings are the code symbols. The entries in row R_i , column ξ_k , of the upper half of the table are the ξ_k -successors of R_i . The row headings in the lower half of the table are all pairs of separation symbols. The entries in row $R_i R_j$, column ξ_k , are the pairs implied by $(R_i R_j)$ and symbol ξ_k . The *testing graph (for synchronizability)* has a vertex for each row in the lower half of the testing table. A directed arc labeled ξ_k leads from the vertex $R_i R_j$ to the vertex $R_p R_q$, where $p \neq q$, if and only if $(R_p R_q)$ is the ξ_k -successor of $(R_i R_j)$. We now state, without proof, the necessary and sufficient condition for a code to be synchronizable.

- A code is synchronizable if and only if it is uniquely decipherable and its testing graph is loop-free. It is synchronizable of order μ if and only if the longest path in the graph is of length $\mu - 1$.

Example Consider the code $\gamma = \{1, 10, 001\}$, whose testing table is shown in Table 14.26 and testing graph in Fig. 14.17. Since the code is uniquely decipherable and the graph is loop-free, γ is synchronizable of order 5.

Table 14.26 The testing table for
 $\gamma = \{0, 10, 001\}$

	0	1
S	C_1	(SB_1)
B_1	S	—
C_1	C_2	—
C_2	—	S
SB_1	(SC_1)	—
SC_1	$(C_1 C_2)$	—
SC_2	—	(SB_1)
$B_1 C_1$	(SC_2)	—
$B_1 C_2$	—	—
$C_1 C_2$	—	—

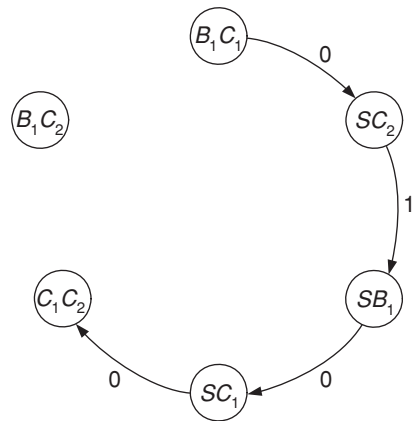


Fig. 14.17 Testing graph.

The main advantage of using a synchronizable code is that the propagation of errors within messages composed of such a code is bounded. In other words, if an error occurs in transmitting a coded message, its effect on the decipherability of the message is limited to at most μ symbols, since the knowledge of any μ code symbols is sufficient to determine a single separation within these symbols.

Table 14.27 State table of an information lossless machine of maximal order

PS	NS, z			
	I_1	I_2	I_3	I_4
1	2, 0	3, 2	2, 3	2, 5
2	3, 0	4, 2	3, 3	3, 5
3	4, 0	5, 2	4, 3	4, 5
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
i	$i + 1, 0$	$i + 2, 2$	$i + 1, 3$	$i + 1, 5$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$\lceil \frac{1}{2}n \rceil - 1$	$\lceil \frac{1}{2}n \rceil, 0$	$\lceil \frac{1}{2}n \rceil + 1, 2$	$\lceil \frac{1}{2}n \rceil, 3$	$\lceil \frac{1}{2}n \rceil, 5$
$\lceil \frac{1}{2}n \rceil$	$\lceil \frac{1}{2}n \rceil + 1, 0$	$\lceil \frac{1}{2}n \rceil + 2, 1$	$\lceil \frac{1}{2}n \rceil + 1, 3$	$\lceil \frac{1}{2}n \rceil + 1, 5$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
j	$j + 1, 0$	$j + 2, 1$	$j + 1, 3$	$j + 1, 5$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$n - 2$	$n - 1, 0$	$n, 1$	$n - 1, 3$	$n - 1, 5$
$n - 1$	$n, 0$	$1, 1$	$1, 6$	$n, 5$
n	$1, 4$	$1, 2$	$n, 3$	$2, 4$

In addition, since synchronizable codes are also uniquely decipherable with a finite delay, the determination of a single separation of code words is sufficient for the decoding of the message from that point on.

*Appendix 14.1 The least upper bound for information losslessness of finite order

In the following, we shall prove that the bound for information losslessness established by Theorem 14.5 is the least upper bound. Specifically, we shall show that, for every n , there exists a machine with four input symbols and seven output symbols which is information lossless of maximal order, that is, for which $\mu = 1 + \frac{1}{2}(n - 1)n$. Such a machine is shown in Table 14.27, where $\lceil g \rceil$ is the least integer greater than or equal to g .

Theorem 14.6 *For every n there exists an information lossless machine of order*

$$\mu = 1 + \frac{(n - 1)n}{2}.$$

Proof We prove the theorem by demonstrating that the class of machines described in Table 14.27 is information lossless of order $1 + \frac{1}{2}(n - 1)n$. The upper part of the testing table for this machine is given in Table 14.28. The testing graph is derived directly from the table and is shown for even n in

Table 14.28 Testing table for information losslessness for the machine in Table 14.27

<i>PS</i>	Output						
	0	1	2	3	4	5	6
1	2	—	3	2	—	2	—
2	3	—	4	3	—	3	—
3	4	—	5	4	—	4	—
4	5	—	6	5	—	5	—
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$\lceil \frac{1}{2}n \rceil - 1$	$\lceil \frac{1}{2}n \rceil$	—	$\lceil \frac{1}{2}n \rceil + 1$	$\lceil \frac{1}{2}n \rceil$	—	$\lceil \frac{1}{2}n \rceil$	—
$\lceil \frac{1}{2}n \rceil$	$\lceil \frac{1}{2}n \rceil + 1$	$\lceil \frac{1}{2}n \rceil + 2$	—	$\lceil \frac{1}{2}n \rceil + 1$	—	$\lceil \frac{1}{2}n \rceil + 1$	—
$\lceil \frac{1}{2}n \rceil + 1$	$\lceil \frac{1}{2}n \rceil + 2$	$\lceil \frac{1}{2}n \rceil + 3$	—	$\lceil \frac{1}{2}n \rceil + 2$	—	$\lceil \frac{1}{2}n \rceil + 2$	—
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$n - 3$	$n - 2$	$n - 1$	—	$n - 2$	—	$n - 2$	—
$n - 2$	$n - 1$	n	—	$n - 1$	—	$n - 1$	—
$n - 1$	n	1	—	—	—	n	1
n	—	—	1	n	(1, 2)	—	—

Fig. 14.18. The graph contains no vertex with repeated entries because all the entries in every column of the upper part of the testing table are distinct. The graph contains $\frac{1}{2}(n-1)n$ vertices arranged in $n-1$ columns. The maximal path, which connects all these vertices, is shown in Fig. 14.18 by the solid lines. The maximal path is constructed in the following manner. The first compatible pair (1, 2) is introduced in column 4 of the testing table. This pair, in turn, implies the pairs (2, 3), (3, 4), ..., $(n-2, n-1)$. Because of the arrangement of the entries in column 1 of Table 14.28, the pair (1, n) is implied by $(n-2, n-1)$. In addition, because of the entries in column 2, the pair (1, 3) is implied by (1, n) and similarly for every column of vertices in the graph. The path goes from the vertex (1, k), for all $2 \leq k \leq \frac{1}{2}n$, to the vertex $(n-k, n-1)$, from which it goes to the vertex $(1, n-k+2)$, as implied by the entries in column 1 of the testing table.

The path continues from the vertex (1, h), for all $(\frac{1}{2}n) + 1 < h \leq n$ to the vertex $(n-h+1, n)$, from which it goes to $(1, n-h+3)$, as implied by the entries in column 2 of the testing table. Finally, the path goes from the vertex $(\frac{1}{2}n, n)$ to $(\frac{1}{2}n+1, n)$, and so on to $(n-1, n)$, as implied by entries in column 3 of Table 14.28. The vertex $(n-1, n)$ is a terminal vertex since the corresponding compatible pair implies no other compatible pair.

It is evident from the structure of the graph that it has no loops, although it contains a number of shorter paths. The testing graph for n odd can be obtained from Table 14.28 in a similar manner, and it too has a path that connects all $\frac{1}{2}n(n-1)$ vertices. Consequently, for any given n the machine in Table 14.27 is information lossless of maximal order. $\diamond$

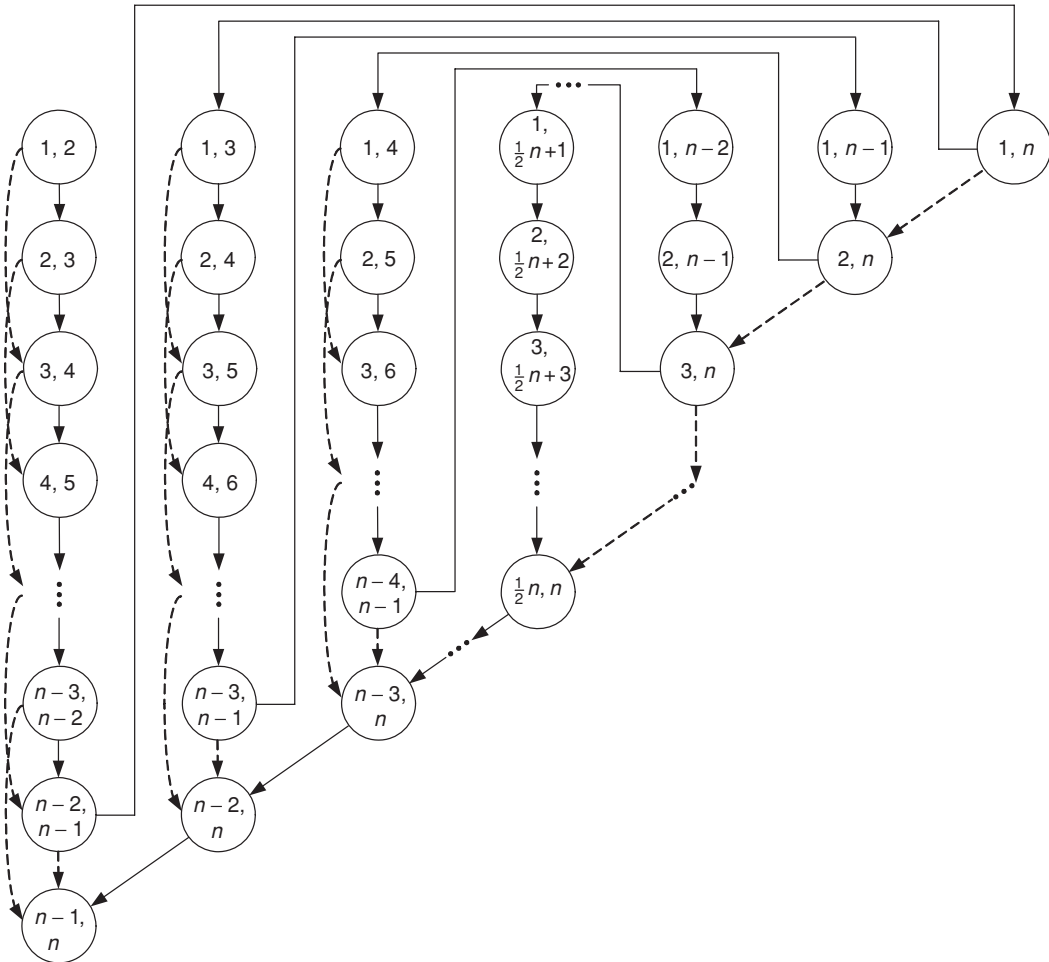


Fig. 14.18 Testing graph for even n for the lossless machine of Table 14.27.

It seems that it may be possible to find an information lossless machine of maximal order with fewer inputs or outputs. It is not clear, however, whether there exists such a machine with only two input symbols and two output symbols.

Notes and references

The various memory aspects of automata have been investigated by numerous authors, among whom are Liu [6, 7, 8], McCluskey [10], Massey [9], Simon [12], and Perles, Rabin, and Shamir [11]. Lossless machines were first studied by Huffman [4], who devised tests for losslessness and losslessness of finite order. Even [1] devised a different testing procedure, the one adopted in this chapter. The least upper bound developed in the above appendix is due to Kohavi and Winograd [5]. The tests for decipherability and synchronizability of codes are due to Even [2, 3].

- [1] Even, S.: "On information lossless automata of finite order," *IEEE Trans. Electron. Computers*, vol. EC-14, pp. 561–569, August 1965.
- [2] Even, S.: "Test for synchronizability of automata and variable length codes," *IEEE Trans. Information Theory*, vol. IT-10, pp. 185–189, July 1964.
- [3] Even, S.: "Tests for unique decipherability," *IEEE Trans. Information Theory*, vol. IT-9, pp. 109–112, April 1963.
- [4] Huffman, D. A.: "Canonical forms for information lossless finite-state machines," *IRE Trans. Circuit Theory*, vol. CT-6, Special Supplement, pp. 41–59, May 1959.
- [5] Kohavi, Z., and J. Winograd: "Establishing bounds concerning finite automata," *J. Computer and System Sciences*, vol. 7, no. 3, pp. 288–299, June 1973.
- [6] Liu, C. L.: "Some memory aspects of finite automata," MIT. Res. Lab. Electron. Tech. Rept 411, May 1963.
- [7] Liu, C. L.: "Determination of the final state of an automaton whose initial state is unknown," *IEEE Trans. Electron. Computers*, vol. EC-12, December 1963.
- [8] Liu, C. L.: "kth-order finite automaton," *IEEE Trans. Electron. Computers*, vol. EC-12, October 1963.
- [9] Massey, J. L.: "Note on finite-memory sequential machines," *IEEE Trans. Electron. Computers*, vol. EC-15, pp. 658–659, 1966.
- [10] McCluskey, E. J.: "Reduction of feedback loops in sequential circuits and carry leads in iterative networks," in *Proc. Third Ann. Symp. Switching Theory and Logical Design*, pp. 91–102, Chicago, 1962.
- [11] Perles, M., M. O. Rabin, and E. Shamir: "The theory of definite automata," *IEEE Trans. Electron. Computers*, pp. 233–243, June 1963.
- [12] Simon, S. M.: "A note on memory aspects of sequence transducers," *IRE Trans. Circuit Theory*, vol. CT-6, Special Supplement, pp. 26–29, May 1959.

Problems

Problem 14.1. For each of the machines in Table P14.1, determine whether it has a finite memory and, if it does, find its order.

Table P14.1

NS, z			NS, z			NS, z		
PS	$x = 0$	$x = 1$	PS	$x = 0$	$x = 1$	PS	$x = 0$	$x = 1$
A	$B, 0$	$B, 0$	A	$D, 0$	$C, 1$	A	$B, 0$	$E, 0$
B	$C, 0$	$D, 0$	B	$A, 0$	$E, 0$	B	$C, 0$	$D, 0$
C	$D, 0$	$C, 0$	C	$C, 1$	$E, 0$	C	$D, 0$	$C, 0$
D	$A, 0$	$C, 1$	D	$C, 1$	$C, 1$	D	$E, 0$	$A, 0$
			E	$B, 0$	$B, 1$	E	$E, 0$	$A, 1$
(a)			(b)			(c)		

Problem 14.2. The canonical realization of finite-memory machines is shown in Fig. P14.2. Verify that the machine of Table P14.2 has a finite memory, and show its canonical realization. In particular, design the combinational logic.

Table P14.2

PS	NS, z	
	x = 0	x = 1
A	A, 0	B, 1
B	C, 0	D, 1
C	B, 1	A, 0
D	D, 1	C, 0

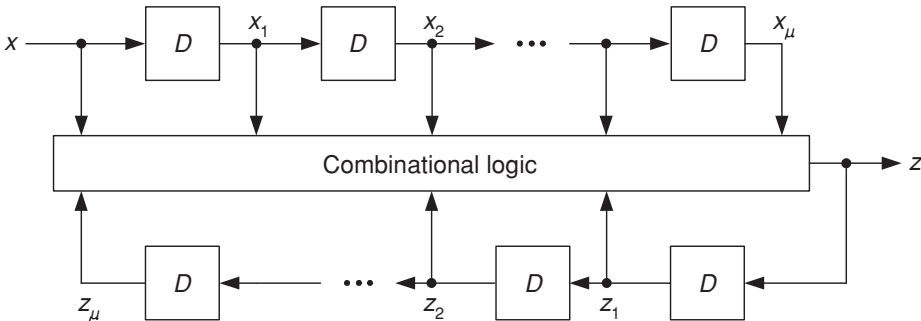


Fig. P14.2

Problem 14.3. Prove that, for every n , the machine of Table P14.3 has a finite memory of order $\mu = \frac{1}{2}(n - 1)n$. (Recall that $\lceil g \rceil$ is the least integer greater than or equal to g .)
Hint: Use a testing graph for finite memory.

Table P14.3

PS	NS		z	
	x = 0	x = 1	x = 0	x = 1
1	2	3	0	0
2	3	4	0	0
3	4	5	0	0
4	5	6	0	0
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$\lceil \frac{1}{2}(n - 3) \rceil$	$\lceil \frac{1}{2}(n - 1) \rceil$	$\lceil \frac{1}{2}(n + 1) \rceil$	0	0
$\lceil \frac{1}{2}(n - 1) \rceil$	$\lceil \frac{1}{2}(n + 1) \rceil$	$\lceil \frac{1}{2}(n + 3) \rceil$	0	1
$\lceil \frac{1}{2}(n + 1) \rceil$	$\lceil \frac{1}{2}(n + 3) \rceil$	$\lceil \frac{1}{2}(n + 5) \rceil$	0	1
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$n - 3$	$n - 2$	$n - 1$	0	1
$n - 2$	$n - 1$	n	0	1
$n - 1$	n	1	0	1
n	n	1	0	0

Problem 14.4. Let M be a p -input symbol, q -output symbol, n -state, strongly connected machine. Prove that if M has a finite memory of order μ then $(pq)^\mu \geq n$.

Problem 14.5

- (a) Test the machine of Table P14.5 for definiteness.
 (b) Show the canonical realization of this machine (see Fig. 14.3). In particular, specify the combinational logic.

Table P14.5

<i>PS</i>	<i>NS, z</i>	
	<i>x = 0</i>	<i>x = 1</i>
<i>A</i>	<i>D, 1</i>	<i>E, 0</i>
<i>B</i>	<i>A, 0</i>	<i>B, 1</i>
<i>C</i>	<i>C, 0</i>	<i>B, 0</i>
<i>D</i>	<i>C, 1</i>	<i>B, 1</i>
<i>E</i>	<i>A, 0</i>	<i>B, 0</i>

Problem 14.6

- (a) Specify the unspecified entries in Table P14.6a in such a way that the resulting machine will be definite. Is your answer unique? If not, show all possible ways to specify the table.
 (b) Is it possible to specify Table P14.6b in such a way that it corresponds to a definite machine? Justify your answer.

Table P14.6

<i>PS</i>	<i>NS</i>	
	<i>x = 0</i>	<i>x = 1</i>
<i>A</i>	<i>A</i>	<i>B</i>
<i>B</i>	—	<i>B</i>
<i>C</i>	<i>E</i>	—
<i>D</i>	—	<i>F</i>
<i>E</i>	—	<i>D</i>
<i>F</i>	<i>E</i>	—

(a)

<i>PS</i>	<i>NS</i>	
	<i>x = 0</i>	<i>x = 1</i>
<i>A</i>	<i>A</i>	<i>B</i>
<i>B</i>	<i>C</i>	<i>C</i>
<i>C</i>	—	—
<i>D</i>	—	—

(b)

Problem 14.7. Determine which of the machines in Table P14.7 has a finite output memory, and find its order.

Table P14.7

<i>PS</i>	<i>NS, z</i>	
	<i>x = 0</i>	<i>x = 1</i>
<i>A</i>	<i>A, 0</i>	<i>B, 1</i>
<i>B</i>	<i>C, 1</i>	<i>D, 0</i>
<i>C</i>	<i>D, 0</i>	<i>C, 1</i>
<i>D</i>	<i>B, 1</i>	<i>A, 0</i>

(a)

<i>PS</i>	<i>NS, z</i>	
	<i>x = 0</i>	<i>x = 1</i>
<i>A</i>	<i>C, 0</i>	<i>C, 0</i>
<i>B</i>	<i>D, 1</i>	<i>A, 0</i>
<i>C</i>	<i>C, 1</i>	<i>B, 0</i>
<i>D</i>	<i>D, 1</i>	<i>D, 1</i>

(b)

<i>PS</i>	<i>NS, z</i>	
	<i>x = 0</i>	<i>x = 1</i>
<i>A</i>	<i>B, 0</i>	<i>C, 0</i>
<i>B</i>	<i>D, 0</i>	<i>E, 1</i>
<i>C</i>	<i>F, 1</i>	<i>D, 0</i>
<i>D</i>	<i>F, 1</i>	<i>F, 1</i>
<i>E</i>	<i>B, 0</i>	<i>B, 0</i>
<i>F</i>	<i>A, 1</i>	<i>A, 1</i>

(c)

Problem 14.8. Given the state table of the machine M shown in Table P14.8, specify the missing output entries in such a way that the machine will be finite-memory of maximal order.

Table P14.8

PS	NS, z	
	$x = 0$	$x = 1$
A	$B, 0$	$C, 1$
B	$D, 0$	$D, -$
C	$C, -$	$A, 0$
D	$C, 0$	$A, 1$

Problem 14.9. Given a machine M with n states $S_1, S_2, \dots, S_n$:

- Devise a procedure to determine whether the machine has n preset sequences $X_1, X_2, \dots, X_n$ such that X_i is the shortest sequence that takes M from any unknown initial state to state S_i .
- Apply your procedure to find the appropriate sequences for the machine M in Table P14.9.
- Find an upper bound on the length of X_i .
- Does the existence of such a set of sequences imply that M must be a definite machine?

Table P14.9

PS	NS, z	
	$x = 0$	$x = 1$
A	$C, 0$	$B, 0$
B	$E, 1$	$F, 0$
C	$A, 1$	$F, 1$
D	$E, 0$	$B, 1$
E	$C, 1$	$D, 0$
F	$E, 0$	$F, 0$

Problem 14.10. Consider the class of machines that have a finite output memory of order μ such that knowledge of the last μ output symbols suffices to determine the final state of the machine.

- Devise a test to determine whether a given machine belongs to the above class.
- Find such a four- or five-state machine and apply your test to it.

Problem 14.11. For each machine in Table P14.11, determine whether it is lossless. If it is lossy, find a shortest output sequence produced by two different input sequences with the same initial and final states. If it is lossless, determine its order.

Table P14.11

NS, z			NS, z		
PS	$x = 0$	$x = 1$	PS	$x = 0$	$x = 1$
A	$B, 1$	$C, 0$	A	$B, 0$	$C, 1$
B	$A, 0$	$D, 1$	B	$D, 1$	$A, 0$
C	$B, 0$	$A, 0$	C	$E, 1$	$F, 1$
D	$C, 1$	$A, 1$	D	$F, 0$	$E, 0$
			E	$C, 1$	$A, 0$
			F	$B, 0$	$D, 1$

(a)

(b)

NS, z			NS, z		
PS	$x = 0$	$x = 1$	PS	$x = 0$	$x = 1$
A	$B, 0$	$C, 0$	A	$B, 0$	$A, 1$
B	$D, 0$	$E, 1$	B	$C, 0$	$D, 1$
C	$E, 0$	$A, 1$	C	$E, 1$	$A, 0$
D	$E, 0$	$D, 0$	D	$E, 0$	$C, 0$
E	$C, 1$	$B, 1$	E	$C, 1$	$E, 0$

(c)

(d)

Problem 14.12. In Table P14.12 you are presented with only the lower half of a testing table (for losslessness) of an unknown machine. Specify the upper half of the table and find a corresponding four-state machine. Is your answer unique?

Table P14.12

	$z = 0$	$z = 1$
A		
B		
C		
D		
AB	—	$(BC)(CC)$
AC	—	$(AB)(AC)$
AD	—	—
BC	(BD)	(AC)
BD	$(AD)(CD)$	—
CD	$(AB)(BC)$	—

Problem 14.13

- (a) The machine described in Table P14.13 has two binary outputs, z_1 and z_2 . Some output entries are incompletely specified. Specify all these output entries in such a way that the machine will be lossless of first order.

- (b) Prove that any binary-input binary-output machine can be transformed into a lossless machine of first order by adding to it a single binary output terminal.

Table P14.13

PS	$NS, z_1 z_2$	
	$x = 0$	$x = 1$
A	$B, -1$	$C, 11$
B	$D, 0-$	$D, 0-$
C	$D, 0-$	$E, --$
D	$B, 0-$	$D, -0$
E	$C, 0-$	$D, -0$

Problem 14.14. The machine described in Table P14.14 has two binary outputs, z_1 and z_2 , some of whose entries are incompletely specified. Specify all these output entries in such a way that the machine will be lossless of the least order. Is such a specification unique?

Table P14.14

PS	$NS, z_1 z_2$	
	$x = 0$	$x = 1$
A	$B, 10$	$C, 10$
B	$C, 00$	$C, 1-$
C	$A, 1-$	$D, 00$
D	$D, 1-$	$A, 00$

Problem 14.15. Prove that the machine of Table P14.15 is lossless of maximal order, i.e., $\mu = 11$.

Table P14.15

PS	NS, z	
	$x = 0$	$x = 1$
S_1	$S_2, 1$	$S_1, 1$
S_2	$S_3, 1$	$S_5, 3$
S_3	$S_4, 1$	$S_4, 3$
S_4	$S_5, 1$	$S_3, 2$
S_5	$S_1, 2$	$S_1, 3$

Problem 14.16. For the machine shown in Table P14.16:

- (a) Find in a systematic way output sequence Z_2 when output sequence Z_1 is 001001, and it is known that the initial and final states are both B .

- (b) Given the initial and final states as well as output sequence Z_1 , is it always possible to determine the output sequence Z_2 ?

Table P14.16

<i>PS</i>	<i>NS, z₁z₂</i>	
	<i>x</i> = 0	<i>x</i> = 1
<i>A</i>	<i>A</i> , 11	<i>B</i> , 10
<i>B</i>	<i>D</i> , 00	<i>A</i> , 00
<i>C</i>	<i>E</i> , 00	<i>C</i> , 10
<i>D</i>	<i>B</i> , 01	<i>C</i> , 01
<i>E</i>	<i>C</i> , 11	<i>A</i> , 01

Problem 14.17. For the machine shown in Table P14.17:

- (a) Does the machine have a finite output memory? If yes, find the order λ .
(b) Is the machine information lossless of finite order? If yes, find the order μ .
(c) The machine produced an output sequence $Z = 0101000$. What is the corresponding input sequence? Is it unique?
(d) What is the minimal length of output sequence Z that enables us to determine at least one input symbol?

Table P14.17

<i>PS</i>	<i>NS, z</i>	
	<i>x</i> = 0	<i>x</i> = 1
<i>A</i>	<i>B</i> , 0	<i>C</i> , 0
<i>B</i>	<i>D</i> , 0	<i>E</i> , 1
<i>C</i>	<i>A</i> , 1	<i>E</i> , 0
<i>D</i>	<i>E</i> , 0	<i>D</i> , 0
<i>E</i>	<i>A</i> , 1	<i>E</i> , 1

Problem 14.18. Given the cascade connection of machines M_1 and M_2 , as shown in Fig. P14.18:

- (a) For M_1 and M_2 as shown in Table P14.18, given that the output sequence $Z = 110011$ and the final state of M_2 is B , determine the initial state of M_1 .
(b) For the machines in Table P14.18 prove that, for every given output sequence Z of length L , knowledge of the final state of M_2 is sufficient to determine the state of M_1 at some time during the experiment. Find the value of L .

Fig. P14.18

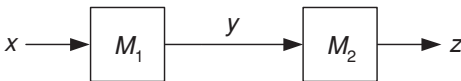


Table P14.18

<i>NS, y</i>			<i>NS, z</i>		
<i>PS</i>	<i>x = 0</i>	<i>x = 1</i>	<i>PS</i>	<i>y = 0</i>	<i>y = 1</i>
<i>A</i>	<i>B, 0</i>	<i>C, 1</i>	<i>A</i>	<i>B, 0</i>	<i>C, 1</i>
<i>B</i>	<i>C, 0</i>	<i>B, 1</i>	<i>B</i>	<i>A, 0</i>	<i>C, 0</i>
<i>C</i>	<i>D, 0</i>	<i>D, 0</i>	<i>C</i>	<i>D, 1</i>	<i>A, 1</i>
<i>D</i>	<i>D, 0</i>	<i>A, 1</i>	<i>D</i>	<i>B, 1</i>	<i>D, 0</i>

Problem 14.19. The machines M_1 and M_2 shown in Table P14.19 are connected in cascade, as shown in Fig. P14.18. The initial state of M_1 is A . Find in a systematic way all the shortest input sequences which, when applied to M_1 , make it possible to identify the initial state of M_2 by means of its response z .

Table P14.19

<i>NS, y</i>			<i>NS, z</i>		
<i>PS</i>	<i>x = 0</i>	<i>x = 1</i>	<i>PS</i>	<i>y = 0</i>	<i>y = 1</i>
<i>A</i>	<i>B, 0</i>	<i>C, 1</i>	<i>D</i>	<i>E, 1</i>	<i>D, 0</i>
<i>B</i>	<i>C, 1</i>	<i>A, 0</i>	<i>E</i>	<i>F, 1</i>	<i>G, 0</i>
<i>C</i>	<i>A, 0</i>	<i>B, 0</i>	<i>F</i>	<i>D, 1</i>	<i>E, 0</i>
			<i>G</i>	<i>F, 0</i>	<i>D, 0</i>

M_1 M_2

Problem 14.20

- (a) In response to an unknown input sequence, the machine of Table P14.20 produces the output sequence 10011. Find the input sequence if it is known that the *final* state is B .
 - (b) Prove that knowledge of the final state of this machine and the last output symbol is sufficient to determine the next-to-final state.
 - (c) Devise a test, to determine whether a given machine is lossless, such that the knowledge of the *final* state and the last μ output symbols is sufficient to identify the next-to-final state.
- Hint:* Use the output-predecessor table.

Table P14.20

<i>NS, z</i>		
<i>PS</i>	<i>x = 0</i>	<i>x = 1</i>
<i>A</i>	<i>B, 0</i>	<i>C, 1</i>
<i>B</i>	<i>A, 0</i>	<i>C, 0</i>
<i>C</i>	<i>D, 1</i>	<i>A, 1</i>
<i>D</i>	<i>B, 1</i>	<i>D, 0</i>

Problem 14.21

- (a) In response to an unknown input sequence, the machine of Table P14.21 produces the output sequence 1110000010. Find the input sequence to the machine if it is known that its initial state is A and final state is F .
- (b) Can the machine produce the output sequence 11011000 when both its initial and final states are A ?

Table P14.21

PS	NS, z	
	$x = 0$	$x = 1$
A	$B, 1$	$C, 0$
B	$D, 1$	$B, 1$
C	$E, 1$	$B, 0$
D	$A, 0$	$E, 0$
E	$F, 0$	$D, 1$
F	$D, 0$	$A, 1$

Problem 14.22. Find a reduced four-state machine that is lossless of first order and is isomorphic to its own inverse.

Problem 14.23. Design an inverse of the machine shown in Table P14.23. Give a reduced, standard-form, state table, assuming that the initial state of the lossless machine is A . For each of the other possible initial states of this machine, specify appropriate initial states of the inverse.

Table P14.23

PS	NS, z	
	$x = 0$	$x = 1$
A	$B, 1$	$C, 1$
B	$D, 0$	$E, 0$
C	$A, 1$	$F, 1$
D	$C, 0$	$B, 0$
E	$F, 1$	$A, 1$
F	$E, 0$	$D, 0$

Problem 14.24

- (a) Prove that the inverse of a lossless machine of finite order is a lossless machine of finite order.
- (b) Demonstrate, by finding the inverse of the machine M_{10}^i (Table 14.22), that the inverse of the inverse of a lossless machine of finite order is isomorphic to the original machine, i.e., show that the inverse of M_{10}^i is isomorphic to M_{10} .

Problem 14.25. The output symbol of a finite-state machine is the modulo-2 sum of the current input symbol and the second and third past input symbols, i.e.,

$$z(t) = x(t) \oplus x(t-2) \oplus x(t-3).$$

- (a) Prove that such a machine is lossless of finite order.
- (b) Realize the machine and its inverse.

Problem 14.26. Show that the code $\gamma = \{1, 110, 010, 100\}$ is uniquely decipherable. Is it also uniquely decipherable with a finite delay? If so, find the delay; if not, show a message that cannot be deciphered in a finite time.

Problem 14.27. Given the uniquely decipherable code $\gamma = \{0, 001, 101, 011\}$, decipher the message 0010100110100110001.

15

Linear sequential machines

Linear sequential machines constitute a subclass of linear systems in which the input vector, output vector, and state transitions occur in discrete steps. Consequently, the tools and techniques available for the analysis and synthesis of linear systems can be applied to linear machines as well. The numerous applications of linear machines give further incentive to the investigation of their properties and to the development of efficient synthesis procedures.

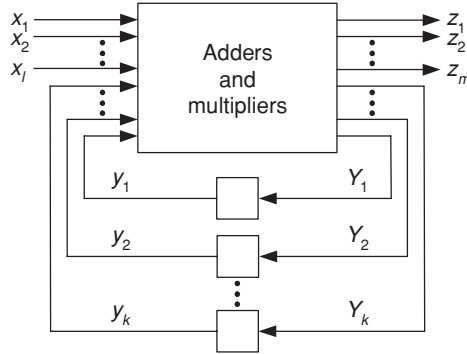
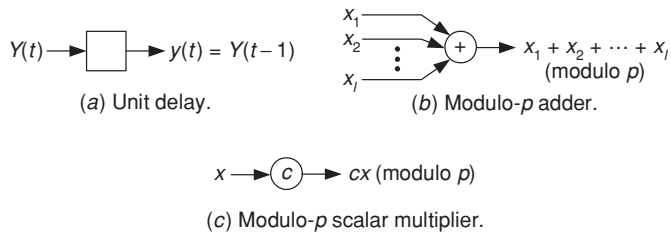
In the first few sections we present an intuitive, though well-justified, approach that requires only a limited knowledge of modern algebra. In subsequent sections (i.e., Sections 15.4 through 15.6) a matrix formulation is presented, and methods for minimizing and detecting linear machines are developed.

15.1 Introduction

A *linear sequential machine* (also called a *linear machine*) is a network that has a finite number of input and output terminals and is composed of interconnections of three types of basic components, to be introduced shortly. The input signals applied to the machine are elements of a finite field¹ $GF(p) = \{0, 1, \dots, p-1\}$, and the operations performed by the basic components on their inputs are carried out according to the rules of $GF(p)$. A block-diagram representation of a linear machine with l input terminals and m output terminals is shown in Fig. 15.1.

For a machine to be linear, its response to a linear combination of inputs must preserve the scale factor and the principle of superposition. Thus, each of the basic components used to realize a linear machine must be linear. This requirement clearly precludes the use of an AND gate whose output is the product of its inputs; e.g., if the inputs are x_1 and x_2 and the signal values

¹ Some relevant basic properties of finite fields are summarized in Appendix 15.1. The understanding of these properties is essential to the study of linear machines.

Fig. 15.1 Block diagram of a linear machine.**Fig. 15.2** Basic components of linear circuits.

are elements of $GF(2)$ then the output is $z = x_1x_2$ modulo 2. Using similar arguments, we observe that the OR gate is not linear either since, for example, the output² of a two-input gate is $z = x_1 + x_2 + x_1x_2$ modulo 2. The following three types of basic component are clearly linear.

1. *Unit delays* A unit delay is a two-terminal element whose output $y(t)$ is related to its input $Y(t)$ by $y(t) = Y(t - 1)$.
2. *Modulo- p adders* An adder has l input terminals and one output terminal. The output is the modulo- p sum of the inputs; i.e., if the inputs are $x_1, x_2, \dots, x_l$ then the output is $x_1 + x_2 + \dots + x_l$ (modulo p).
3. *Modulo- p scalar multipliers* A multiplier c (where c is an element of $GF(p)$) has one input and one output terminal. If the input is x then the output is cx (modulo p).

Modulo- p addition and scalar multiplication are assumed to be executed instantaneously. For most purposes, we shall restrict p to prime numbers. The symbols representing the above components are shown in Fig. 15.2.

Any network that is constructed by interconnecting components of the types shown in Fig. 15.2 is referred to as a linear circuit, provided that every closed loop contains at least one delay element. The unit delay is equal to the discrete

² In this chapter, the symbol $+$ represents the addition operation in accordance with the rules of $GF(p)$ (i.e., modulo p).

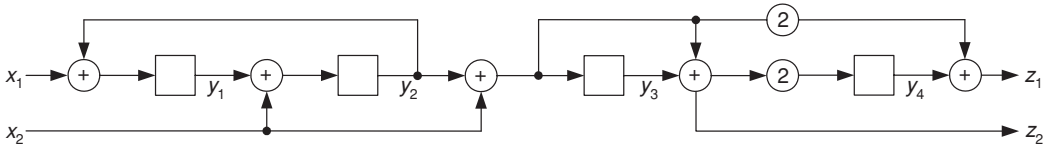


Fig. 15.3 A four-terminal four-dimensional linear machine over $GF(3)$.

interval of time between two successive clock pulses. The state variables of a linear machine are the outputs $y_1, y_2, \dots, y_k$ of the delay elements. The state of a machine at time t is specified by the value of the y 's at t , i.e., $y_1(t), y_2(t), \dots, y_k(t)$. The number of delay elements (or state variables) in a linear machine is referred to as the *dimension* of the machine. A linear machine whose components are modulo p and whose input signals are elements of $GF(p)$ is said to be a linear machine over $GF(p)$.

Example Figure 15.3 illustrates a four-terminal four-dimensional linear machine over $GF(3)$.

A linear machine over $GF(2)$ is called a *binary* machine. Binary machines are practical and simple to construct and are widely used in various applications. Consequently, although we shall develop the theory of linear machines over $GF(p)$, most examples will be selected from linear machines over the $GF(2)$ field.

15.2 Inert linear machines

A linear machine whose delay elements are initially in the zero state is referred to as an *inert* (or *quiescent*) *linear machine*. Inert linear machines are used extensively as encoding and decoding devices and in various applications that require transformations of sequences. It will subsequently be shown that the study of these machines provides insight into the problem of arbitrary linear machines, as well as some of the basic tools for the analysis of the subject.

Feedforward shift registers

The simplest type of inert linear machine is a two-terminal shift register that contains only feedforward paths and whose output is a modulo- p sum of selected input digits. The schematic representation of a feedforward shift register over $GF(p)$ is shown in Fig. 15.4.

The output z can be described by a polynomial in D over the $GF(p)$ field, i.e.,

$$z = a_0x + a_1Dx + \dots + a_kD^kx \quad (15.1)$$

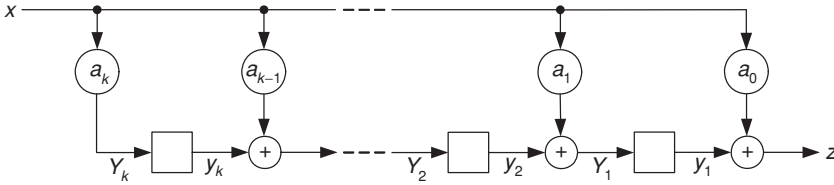


Fig. 15.4 A feedforward shift register.

where the symbol D^i is an i -unit *delay operator*, which delays by i time units the variable on which it operates. For example, equation $z = D^2x$ means that, for all $t \geq 2$, $z(t) = x(t - 2)$. The operator $D^0 = 1$ is referred to as the *identity operator*. Equation (15.1) is a valid description of the shift register of Fig. 15.4 only if initial conditions of delays are zero, i.e., $y_1(0) = y_2(0) = \dots = y_k(0) = 0$, since otherwise the output cannot be expressed for all times as only a function of the input. Equation (15.1) can be rewritten as

$$z = (a_0 + a_1D + \dots + a_kD^k)x$$

or as

$$\frac{z}{x} = a_0 + a_1D + \dots + a_kD^k = T(D), \quad (15.2)$$

where the polynomial $T(D)$, which expresses the ratio z/x , is defined as the *transfer function* of the inert linear machine.

Example Consider the inert linear machine over $GF(2)$ of Fig. 15.5, where the output digit is a modulo-2 sum of the present input digit and the first and third past input digits, i.e., $z(t) = x(t) + x(t - 1) + x(t - 3)$. The corresponding polynomial in the delay operator is

$$z = x + Dx + D^3x$$

and the transfer function is

$$T_1 = \frac{z}{x} = 1 + D + D^3.$$

Note that, for $GF(2)$, the scalar multiplier a_i is either 1 or 0, depending on whether there is or is not a connection to the i th modulo-2 adder.

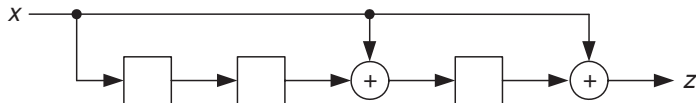


Fig. 15.5 Realization of the transfer function $T_1 = 1 + D + D^3$.

To show that the circuit represented by Eq. (15.1) and Fig. 15.4 is indeed linear, let z and z^* be the responses to two distinct input sequences x and x^*

respectively and let v and v^* be scalars taken from $GF(p)$. Then

$$z = a_0x + a_1Dx + \cdots + a_kD^kx$$

and

$$z^* = a_0x^* + a_1Dx^* + \cdots + a_kD^kx^*.$$

The response Z to a linear combination of inputs is given by

$$Z = a_0(vx + v^*x^*) + a_1D(vx + v^*x^*) + \cdots + a_kD^k(vx + v^*x^*)$$

or

$$Z = v(a_0x + a_1Dx + \cdots + a_kD^kx) + v^*(a_0x^* + a_1Dx^* + \cdots + a_kD^kx^*).$$

Hence,

$$Z = vz + v^*z^*. \quad (15.3)$$

The response of the machine to a linear combination of inputs preserves the scale factor and principle of superposition and consequently the machine is linear. As a result, we may apply the linear theory of polynomials to delay polynomials as well.

Consider now a serial connection of two linear machines of the type shown in Fig. 15.4; that is, the output of the predecessor machine is the input to the successor machine. Let x_1 , z_1 , and T_1 denote the input, output, and transfer function of the predecessor machine, and let x_2 , z_2 , and T_2 denote the input, output, and transfer function of the successor machine. The transfer function T_3 of the serial connection is given by

$$T_3 = \frac{z_2}{x_1}.$$

However, since x_2 and z_1 are identical we have

$$T_3 = \frac{z_1}{x_1} \cdot \frac{z_2}{x_2} = T_1 \cdot T_2.$$

Similarly, the transfer function of a parallel connection of the above machines is given by $T_4 = T_1 + T_2$. The multiplication and addition of polynomials are performed over the $GF(p)$ field.

Example Let $T_1 = D^2 + 2D + 1$ and $T_2 = D + 1$ be transfer functions over the field $GF(3)$. The transfer functions, which correspond to the serial and parallel connections of T_1 and T_2 , are given by

$$T_3 = (D^2 + 2D + 1)(D + 1) = D^3 + 1,$$

$$T_4 = (D^2 + 2D + 1) + (D + 1) = D^2 + 2.$$

Impulse response and null sequences

It is useful to define the *impulse response* h of an inert linear machine as its response to the input sequence $100 \cdots 0$. For example, the impulse response of the (inert) feedforward shift register of Fig. 15.4 is $a_0 a_1 a_2 \cdots a_k 0 \cdots 0$. After at most $k + 1$ time units, the output of the k -dimensional feedforward shift register will be a sequence of 0's. In analogy to linear system theory, we can determine the response of an inert linear machine to an arbitrary input sequence from its impulse response. This is accomplished by performing a discrete "convolution" in $GF(p)$.

Example The impulse response of $T_1 = 1 + D + D^3$ is $h = 110100 \cdots 0$. The response of T_1 to the input sequence 1011 is obtained by addition (modulo 2) of the sequences h , D^2h , and D^3h , as follows:

Impulse:	1	0	0	0	0	·	·	0
Impulse response h :	1	1	0	1	0	·	·	0
Input sequence:	1	0	1	1				
h :	1	1	0	1	0	0	0	· · 0
D^2h :	0	0	1	1	0	1	0	· · 0
D^3h :	0	0	0	1	1	0	1	0 · · 0
Output sequence:	1	1	1	1	1	1	1	0 · · 0

The reader can similarly verify that the response of T_1 to the input sequence 11101 is 10000001.

If the initial state at $t = 0$ of an inert linear machine is $00 \cdots 0$, i.e., $y_1(0) = y_2(0) = \cdots = y_k(0) = 0$, and the input to the machine is a sequence of 0's then the output is also a sequence of 0's. However, it is possible to generate an output sequence consisting of 0's by providing the machine with a nonzero input sequence. Such a sequence is called a *null sequence* of the linear machine T and is denoted X_0 , so that TX_0 is a sequence of 0's. If X_0 and X_0^* are null sequences for a machine T , that is, $TX_0 = 00 \cdots 0$ and $TX_0^* = 00 \cdots 0$, then $v_1TX_0 + v_2TX_0^* = T(v_1X_0 + v_2X_0^*) = 00 \cdots 0$, where v_1 and v_2 are scalars from $GF(p)$. Thus, *any linear combination of null sequences is also a null sequence for the machine*.

Example A null sequence of $T_1 = 1 + D + D^3$ is determined as follows:

$$\begin{aligned} 0 &= X_0 + DX_0 + D^3X_0, \\ X_0 &= DX_0 + D^3X_0. \end{aligned}$$

Thus, the present digit of X_0 is found by adding (modulo 2) the first and third past input digits of X_0 . The null sequence is obtained by selecting an arbitrary nonzero sequence of length 3 (in general, of length equal to

dimension k) and specifying the subsequent digits. For T_1 , the selection of 001 as the initial sequence yields the following null sequence:

$$X_0 = (0 \ 0 \ 1) \ 1 \ 1 \ 0 \ 1 \ 0 \ 0 \ 1.$$

After seven digits the null sequence, which consists of the last seven digits, repeats itself.

Example The null sequence for the polynomial $T = 1 + 2D^2 + D^3$ over $GF(3)$ is found from

$$0 = X_0 + 2D^2X_0 + D^3X_0.$$

Adding $2X_0$ to both sides and recalling that $2X_0 + X_0 = 0$ in modulo 3 yields

$$2X_0 = 2D^2X_0 + D^3X_0.$$

Multiplying both sides by 2 yields

$$X_0 = D^2X_0 + 2D^3X_0.$$

Starting with 111, we obtain the null sequence

$$X_0 = (1 \ 1 \ 1) \ 0 \ 0 \ 2 \ 0 \ 2 \ 1 \ 2 \ 2 \ 1 \ 0 \ 2 \ 2 \ 2 \ 0 \ 0 \ 1 \ 0 \ 1 \ 2 \ 1 \ 1 \ 2 \ 0 \ 1 \ 1 \ 1.$$

The preceding null sequences are known as *maximal* sequences, since each contains $(p^k - 1)$ digits and includes all possible k -tuples except $00 \cdots 0$. Additional properties of null sequences and their relationships to delay polynomials are discussed in [7].

Inverse machines

Feedforward shift registers are often used for encoding purposes. It is useful to determine whether an inverse machine that can be used as a decoder exists and, if it does, how to construct it. We shall say that a polynomial $T(D)$, where $z = Tx$, has an inverse, which will be denoted by $1/T(D)$, if there exists a network that realizes $x = (1/T)z$. We shall consider only those inverses that decode without any delay. The inverse of the feedforward shift register of Fig. 15.4 is obtained by reversing the directions of z and x in this schematic diagram and inverting the scalar multipliers, as shown in Fig. 15.6.

If we provide the inverse machine of Fig. 15.6 with the impulse response of the original machine of Fig. 15.4, i.e., $a_0a_1 \cdots a_{k-1}a_k00 \cdots 0$, its response will be the original message, $x = 100 \cdots 0$. Since the inverse machine is linear and initially inert, it will decode any message produced by the original machine. (Note that negative scalars are actually positive integers since $(-a)$ modulo $p = (p - a)$ modulo p .)

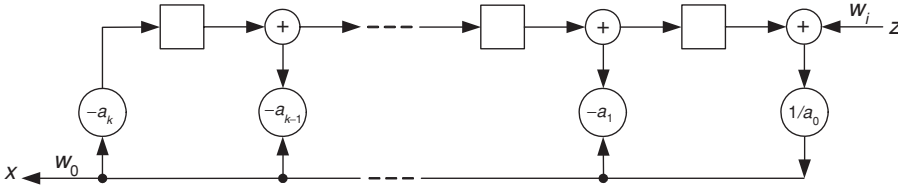


Fig. 15.6 Inverse machine for the shift register of Fig. 15.4.

From Fig. 15.6 it is evident that the inverse is realizable only if $a_0 \neq 0$. In general, an inert linear machine described by a delay polynomial T has a linear inverse described by T^{-1} , which decodes without a delay, if and only if T contains a nonzero constant term that is prime to modulo p . The general proof of this result is left to the reader as an exercise. The following demonstrates it for the case $GF(2)$.

The assertion is that an inert linear machine over the field of integers modulo 2 has an inverse, which decodes the output of the original machine without a delay, if and only if $a_0 = 1$ in T . To prove this assertion, consider the polynomial $T = a_1D + a_2D^2 + \cdots + a_kD^k$, for which $a_0 = 0$. Let the input to and the output from the inverse machine be denoted w_i and w_o , respectively; then the transfer function is given by

$$\frac{w_o}{w_i} = \frac{1}{a_1D + a_2D^2 + \cdots + a_kD^k}$$

or

$$a_1Dw_o = w_i + a_2D^2w_o + \cdots + a_kD^kw_o.$$

The above equation means that a *past* output of the inverse machine (i.e., Dw_o) is a function of past outputs as well as the *present* input to the inverse machine. Such a condition is clearly not physically realizable. (If $a_1 = 0$, the above argument holds for the term containing the lowest order $a_i \neq 0$.)

If T does not contain a nonzero constant term, no instantaneous inverse can be found. However, an “inverse” that decodes the original input after a finite delay can be found. Let a_i be the scalar associated with the lowest power of D for which $a_i \neq 0$, i.e.,

$$T = D^i + a_{i+1}D^{i+1} + \cdots + a_kD^k$$

(modulo 2). The “inverse” is given by

$$\frac{w_o}{w_i} = \frac{1}{D^i + a_{i+1}D^{i+1} + \cdots + a_kD^k} \quad (15.4)$$

or

$$\frac{D^i w_o}{w_i} = \frac{1}{1 + a_{i+1}D + \cdots + a_kD^{k-i}}. \quad (15.5)$$

Although an inverse that decodes instantaneously does not exist for T , Eq. (15.5) corresponds to a realizable inverse, which regenerates the original message after a delay of i time units. Hence, if a sufficient finite delay is allowed then the messages generated by a feedforward shift register can always be decoded. This means that the shift register of Fig. 15.4 is actually lossless of order μ , where $\mu < k$.

Example The inverse of the inert linear machine of Fig. 15.5 is given by $T_1^{-1} = 1/(1 + D + D^3)$ and is shown in Fig. 15.7. (Note that for binary inert linear machines $-a_i = a_i$.)

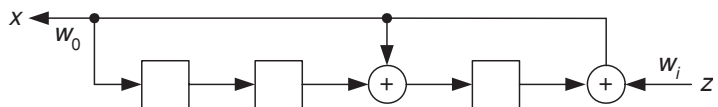


Fig. 15.7 The inverse of the machine in Fig. 15.5.

Linear machines with nonzero initial conditions

The inverse of an inert linear machine might not be inert. Consequently, its response to a sequence of zero input digits is not necessarily a sequence of zero output digits but could be a null sequence X_0 whose starting digits are determined by the initial state of the inverse. This can be shown by observing that the transfer function of the inverse is $x/z = T^{-1}$, or $z = Tx = 0$, because the input z to the inverse is assumed to be an all-0's sequence. Clearly, the solution of equation $Tx = 0$ is the null sequence X_0 .

Let the input digits to the linear machine realizing T_1 and its inverse T_1^{-1} (Figs. 15.5 and 15.7, respectively) be 0's. If the machines are inert then their respective output digits will also be 0's. If, however, they are not inert then their respective output digits will not be 0's but will depend on their initial states. Since T_1 contains only feedforward paths, its response to a sequence of 0's might initially be nonzero, depending on the initial state. However, after at most three time units the response will be a sequence of 0's. In general, for every k -dimensional feedforward shift register the response to a sequence of 0's will also be a sequence of 0's, after a transient period of at most k time units in which the output digit might be nonzero. In the case of a noninert shift register that contains feedback paths, e.g., T_1^{-1} , the response to a sequence of 0's is not necessarily a sequence of 0's. The behavior of a noninert linear machine whose input is a sequence of 0's is often referred to as *autonomous* behavior, and it can be described by the state diagram of the corresponding machine whose input terminals are ignored. The state diagrams describing the autonomous behavior of the machines realizing T_1 and T_1^{-1} are given in Fig. 15.8.

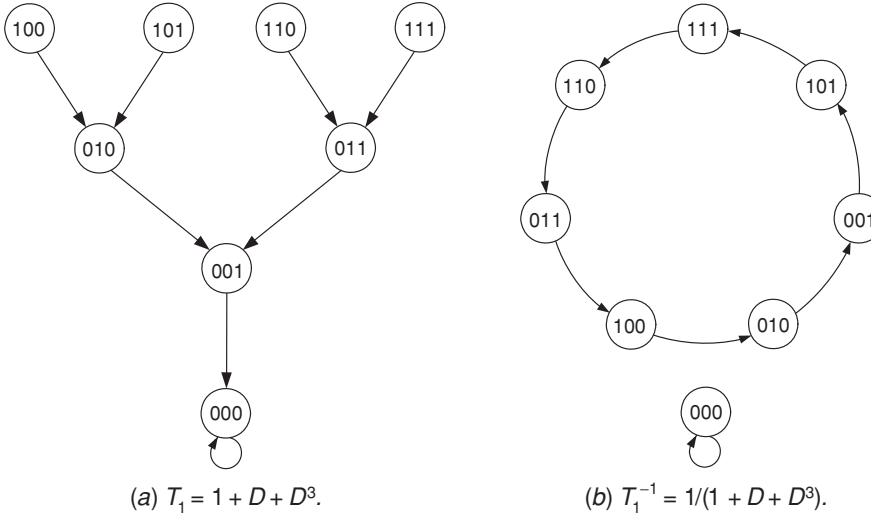


Fig. 15.8 State diagrams for autonomous behavior of linear machines.

An *autonomous linear machine* is a linear machine that contains no inputs (except a clock). A transition is caused by the clock pulse and, since the machine is deterministic, only one transition is permitted from each state. While the state diagram of T_1 contains only a single loop, corresponding to the case where the initial condition is 000, the diagram of T_1^{-1} contains two loops, which are called *cycle sets*. The nontrivial cycle in T_1^{-1} contains seven states and is maximal. (In general, the maximum number of distinct states in a k -dimensional modulo- p machine is p^k and, therefore, a maximal cycle contains $p^k - 1$ states.) For a more comprehensive study of the properties of autonomous linear machines, the reader is referred to Gill [9].

15.3 Inert linear machines and rational transfer functions

In the preceding section, the output of an inert linear machine was assumed to be a function of the present and some of the past input digits. In this section, we develop the more general case where the present output digit depends on the present and selected past input digits and also on a finite number of past output digits. In this latter case, the transfer function is a rational polynomial in the delay operator, i.e., $T = P(D)/Q(D)$.

Realization of rational polynomials

As an example, consider the inert linear machine whose output z is the modulo-2 sum of the present, first, second, and fourth previous input digits and of the first and third previous output digits, i.e.,

$$z = x + Dx + D^2x + D^4x + Dz + D^3z. \quad (15.6)$$

Equation (15.6) can be rewritten as

$$z(1 + D + D^3) = x(1 + D + D^2 + D^4)$$

and the transfer function is given by

$$T_2 = \frac{z}{x} = \frac{1 + D + D^2 + D^4}{1 + D + D^3}.$$

It can be shown that the numerator and denominator of T_2 do not contain any common factor and, thus, T_2 cannot be further simplified.

There are several methods for realizing the above transfer function. An obvious approach, although a very inefficient one, is to synthesize the inert linear machines given by the polynomials $1 + D + D^2 + D^4$ and $1/(1 + D + D^3)$ and to form a serial connection of these machines. Such a realization requires seven delay elements, four for the numerator and three for the denominator. Other synthesis procedures, which involve factoring of the numerator, partial fraction expansion, and ladder-type expansions, although useful do not necessarily yield a minimal realization. (A *minimal realization* is one that yields a machine of smallest dimension.) Clearly, the minimal possible dimension is determined by the degree of the polynomial and is equal to the highest degree in either the numerator or denominator of the transfer function. The chain realization described below yields a minimal realization in an efficient manner.

For T_2 , the number of delay elements required in the minimal realization is four, the degree of the numerator. To demonstrate this assertion, let us rewrite Eq. (15.6) in increasing powers of D as follows:

$$x + z = D(x + z) + D^2x + D^3z + D^4x$$

or

$$x + z = D\{(x + z) + D[x + D(z + Dx)]\}. \quad (15.7)$$

The realization of Eq. (15.7), which is known as a *chain realization*, and that of its inverse, which corresponds to

$$T_2^{-1} = \frac{x}{z} = \frac{(1 + D + D^3)}{(1 + D + D^2 + D^4)},$$

are shown in Fig. 15.9. The output z is generated by adding x to $x + z$, which gives $(x + z) + x = z$ (modulo 2). This realization uses only EXCLUSIVE-OR adders, i.e., two-input modulo-2 adders, which are relatively inexpensive. In general, one characteristic of the chain realization is that it employs modulo-2 adders with only two inputs.

To obtain the chain realization of an arbitrary transfer function over $GF(2)$, note that the transfer function $T = P(D)/Q(D)$ of any realizable inert linear machine over $GF(p)$ has the form

$$T = \frac{z}{x} = \frac{a_0 + a_1D + \cdots + a_kD^k}{1 + b_1D + \cdots + b_kD^k} = \frac{P(D)}{Q(D)}, \quad (15.8)$$

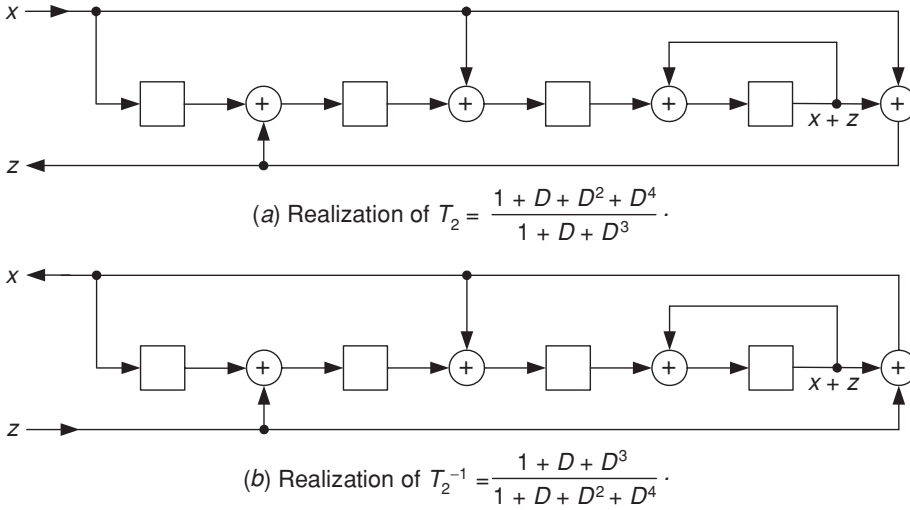


Fig. 15.9 Chain realization of an inert linear machine and its inverse.

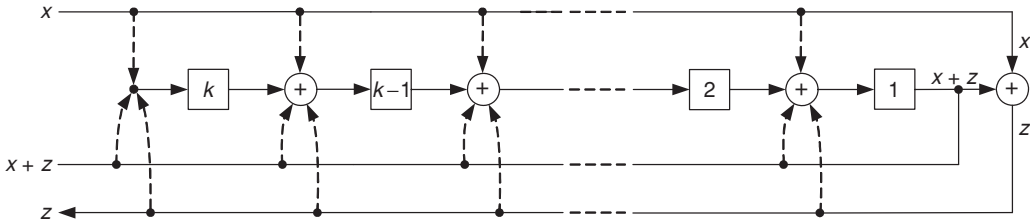
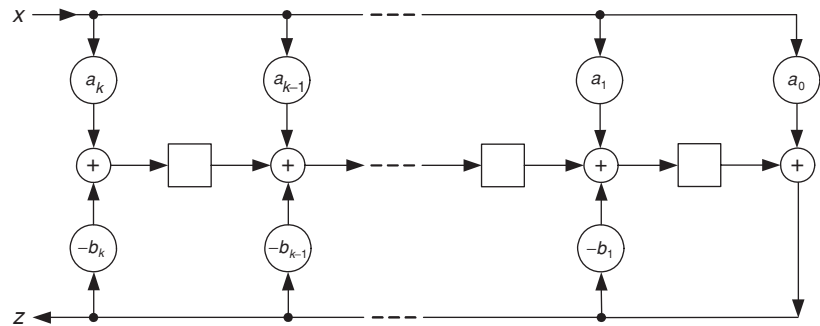


Fig. 15.10 Chain realization of an arbitrary transfer function over $GF(2)$.

where the a_i 's and b_i 's are elements of $GF(p)$. The denominator $Q(D)$ must contain the term 1 if T is to be realizable, as shown in the preceding section. Clearly, a realizable instantaneous inverse T^{-1} exists if and only if the numerator contains a nonzero constant term a_0 that is prime to modulo p . The machine T_2 has such an instantaneous inverse, as illustrated in Fig. 15.9b, since the numerator of T_2 contains a nonzero constant term, i.e., $a_0 = 1$.

For any invertible transfer function over $GF(2)$ of the form Eq. (15.8), we can write an expression for $x + z$ as a sum of *past* input and output digits, e.g., Eq. (15.7). This expression can be realized by an alternating chain of delay elements and modulo-2 adders, as shown in Fig. 15.10. In general, the chain realization of a k -dimensional inert linear machine requires k delay elements and at most k two-input modulo-2 adders. One input to the i th adder from the right (except the first adder) is the output of the i th delay element. The second input, if required, is x , z , or $x + z$, depending respectively on whether the term D^{i-1} is present in the numerator or denominator of T or both. The second input to the rightmost adder is always x , so that $x + (x + z) = z$. If D^{i-1} is absent from both $P(D)$ and $Q(D)$, i.e., $a_{i-1} = b_{i-1} = 0$, no second input is required and the i th adder may be deleted. The inverse machine

Fig. 15.11 Realization of $T = (a_0 + a_1 D + \cdots + a_k D^k) / (1 + b_1 D + \cdots + b_k D^k) \pmod{p}$.



is obtained simply by interchanging the roles of x and z , as illustrated in Fig. 15.9b.

The realization of a two-terminal k -dimensional inert linear machine, over the $GF(p)$ field, whose transfer function is given by Eq. (15.8), is shown in Fig. 15.11. Note that for $p \geq 3$ it is generally not sufficient to employ only two-terminal adders, unless the number of adders is increased. The realization of Fig. 15.11 is obtained in a direct manner from the realizations in Figs. 15.4 and 15.6. The verification that it indeed realizes Eq. (15.8) is left to the reader as an exercise.

Example The realization of

$$T_3 = \frac{1 + 2D + D^2 + 2D^4}{1 + 2D + D^2 + D^3}$$

over $GF(3)$ is shown in Fig. 15.12.

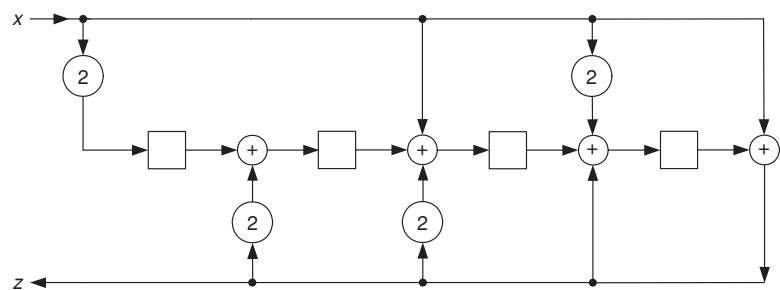


Fig. 15.12 Realization of $T_3 = (1 + 2D + D^2 + 2D^4) / (1 + 2D + D^2 + D^3) \pmod{3}$.

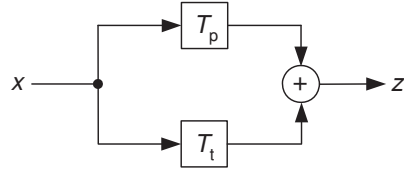
Impulse response and transfer function

The impulse response h of an inert linear machine has been defined as its response to the input sequence $100 \cdots 0$. For any given impulse response, a transfer function can always be specified and if the impulse response is realizable then a corresponding machine can be synthesized. We shall now

Fig. 15.13 Synthesis of an inert linear machine from its impulse response.

Impulse: 100000000000000000...
 h : 10101001110100111...
 h_t : 010000000000000000...
 h_p : 1110100,1110100,111...

(a) Impulse response and its components.



(b) $T = T_p + T_t$.

show how to synthesize an inert linear machine from its impulse response. In particular, we shall prove that if the impulse response is realizable then it consists of two components: a transient component denoted h_t and a periodic component denoted h_p .

In Section 10.2, it was shown that the response of an arbitrary sequential machine to a periodic excitation is periodic. In particular, the response to a sequence of 0's is periodic with period shorter than or equal to n , where n is the number of states. For a k -dimensional inert linear machine, the period of the response to a sequence of 0's is at most $p^k - 1 = n - 1$, since this is the maximal nontrivial cycle set (excluding the zero state). Consequently, a necessary condition for an impulse response h to be realizable is that it will ultimately become periodic. In addition, since the length of the transient response is at most $k + 1$, the transfer function of a realizable two-terminal k -dimensional inert linear machine can be specified uniquely by observing the first $k + p^k$ symbols of the impulse response:

As an example, consider the impulse response $h = 1010100, 1110100, 1110100, \dots$ of an inert linear machine over $GF(2)$. The impulse response can be separated into a transient and a periodic component such that $h = h_t + h_p$, as shown in Fig. 15.13a. The synthesis of the corresponding inert linear machine can be accomplished by specifying separately transfer functions T_t and T_p , corresponding, respectively, to h_t and h_p , such that the overall transfer function $T = T_t + T_p$ (see Fig. 15.13b). The transfer function T_t is found from h_t to equal D . The periodic component h_p can be described by $1 + D + D^2 + D^4$ and, since the period is 7, the entire periodic transfer function is specified by

$$T_p = (1 + D + D^2 + D^4)(1 + D^7 + D^{14} + \dots)$$

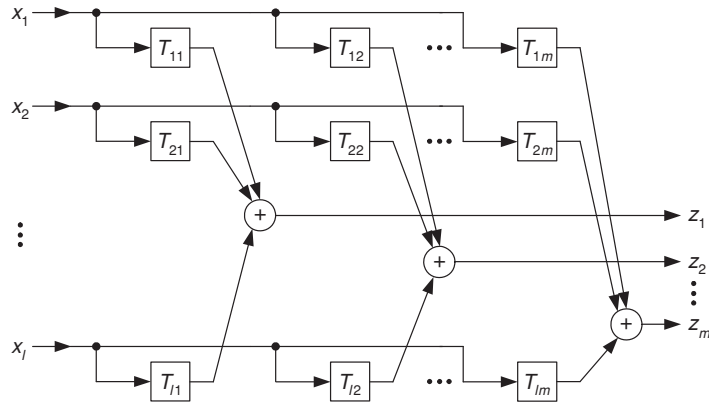
or

$$T_p = \frac{1 + D + D^2 + D^4}{1 + D^7}.$$

Hence,

$$\begin{aligned} T = T_p + T_t &= \frac{1 + D + D^2 + D^4}{1 + D^7} + D \\ &= \frac{1 + D^2 + D^4 + D^8}{1 + D^7}. \end{aligned}$$

Fig. 15.14 Schematic diagram of a multi-terminal inert linear machine.



This function can be simplified as (see Appendix 15.2)

$$T = \frac{(1 + D + D^2 + D^4)^2}{(1 + D + D^2 + D^4)(1 + D + D^3)} = \frac{1 + D + D^2 + D^4}{1 + D + D^3}.$$

A minimal realization of this transfer function is shown in Fig. 15.9a.

Multi-terminal machines

In the preceding sections we developed the properties of two-terminal inert linear machines characterized by rational polynomials in the delay operator D . A multi-terminal inert linear machine with l input terminals and m output terminals can be characterized by a set of lm transfer functions, where

$$T_{ij}(D) = \frac{z_j}{x_i} \quad \text{for all } i = 1, 2, \dots, l \quad \text{and} \quad j = 1, 2, \dots, m.$$

The transfer function T_{ij} is evaluated when $x_i = 0$ for all $i \neq j$; i.e., T_{ij} specifies the dependency of output z_j on input x_i when all other inputs are held at zero. The synthesis problem of a multi-terminal inert linear machine can thus be transformed to the well-known problem of synthesizing a set of two-terminal inert linear machines. A realization of an arbitrary multi-terminal inert linear machine from an appropriate set of two-terminal machines is shown in Fig. 15.14. It must be emphasized that this is not always a minimal realization; rather, it demonstrates that a realization exists. More efficient methods, that yield minimal realizations, are developed in subsequent sections.

15.4 The general model

The specification of the outputs z_j of an inert linear machine by means of a set of polynomials, such that $z_j = \sum_{i=1}^l T_{ij}x_i$, is actually a “black box” type

of specification; that is, each output is specified in terms of only the external inputs and the characterizing polynomials. Such a specification is possible since the machine is assumed to be initially inert, i.e., $x(t) = 0$ for all $t < 0$ and, therefore, $y_i(t) = 0$ for all $t < 0$ and $i = 1, 2, \dots, k$. The specification of an arbitrary (not necessarily inert) linear machine is accomplished by specifying the output and next-state functions in terms of the inputs as well as the present states of the machine.

The matrix formulation

Consider a k -dimensional linear machine over $GF(p)$, with l inputs and m outputs, as shown in Fig. 15.1. Since the combinational logic consists of only adders and scalar multipliers, the next state of the delay Y_i can be expressed as a function of the external inputs of the machine and its present state, as follows:

$$Y_i = (\alpha_{i1}y_1 + \alpha_{i2}y_2 + \dots + \alpha_{ik}y_k) + (\beta_{i1}x_1 + \beta_{i2}x_2 + \dots + \beta_{il}x_l)$$

or

$$Y_i = \sum_{j=1}^k \alpha_{ij}y_j + \sum_{j=1}^l \beta_{ij}x_j. \quad (15.9)$$

Equation (15.9) is called the *next-state equation* for delay Y_i . The entire set of next-state equations for a given machine can be expressed compactly in a matrix form as follows:

$$\begin{bmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_k \end{bmatrix} = \begin{bmatrix} \alpha_{11} & \alpha_{12} & \cdots & \alpha_{1k} \\ \alpha_{21} & \alpha_{22} & \cdots & \alpha_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_{k1} & \alpha_{k2} & \cdots & \alpha_{kk} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_k \end{bmatrix} + \begin{bmatrix} \beta_{11} & \beta_{12} & \cdots & \beta_{1l} \\ \beta_{21} & \beta_{22} & \cdots & \beta_{2l} \\ \vdots & \vdots & \ddots & \vdots \\ \beta_{k1} & \beta_{k2} & \cdots & \beta_{kl} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_l \end{bmatrix} \quad (15.10)$$

or

$$\mathbf{Y}(t) = \mathbf{y}(t+1) = \mathbf{A}\mathbf{y}(t) + \mathbf{B}\mathbf{x}(t).$$

The vector $\mathbf{y}(t)$ is called the *present-state vector*; its elements are the state variables. The vector $\mathbf{Y}(t)$ is the *next-state vector*, where $\mathbf{Y}(t) = \mathbf{y}(t+1)$. The vector $\mathbf{x}(t)$ is the *input vector*; its elements are the *input variables*, where $x_i(t)$ is the input applied to the i th terminal at time t . The dimensions of the state and input vectors are k and l , respectively, i.e.,

$$\mathbf{y}(t) = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_k \end{bmatrix}, \quad \mathbf{Y}(t) = \begin{bmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_k \end{bmatrix}, \quad \mathbf{x}(t) = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_l \end{bmatrix}.$$

When the dependence on t is understood, $y_i(t)$ and $x_i(t)$ are written as y_i and x_i , respectively.

In a similar manner, each output function can be specified in terms of the present state and inputs of the machine. The i th output is expressed as

$$z_i = (\gamma_{i1}y_1 + \gamma_{i2}y_2 + \cdots + \gamma_{ik}y_k) + (\delta_{i1}x_1 + \delta_{i2}x_2 + \cdots + \delta_{il}x_l)$$

or

$$z_i = \sum_{j=1}^k \gamma_{ij}y_j + \sum_{j=1}^l \delta_{ij}x_j. \quad (15.11)$$

Equation (15.11) is called the *output equation*. The entire set of output equations for a given machine can also be expressed in a matrix form, as follows:

$$\begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_m \end{bmatrix} = \begin{bmatrix} \gamma_{11} & \gamma_{12} & \cdots & \gamma_{1k} \\ \gamma_{21} & \gamma_{22} & \cdots & \gamma_{2k} \\ \vdots & \vdots & \vdots & \vdots \\ \gamma_{m1} & \gamma_{m2} & \cdots & \gamma_{mk} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_k \end{bmatrix} + \begin{bmatrix} \delta_{11} & \delta_{12} & \cdots & \delta_{1l} \\ \delta_{21} & \delta_{22} & \cdots & \delta_{2l} \\ \vdots & \vdots & \vdots & \vdots \\ \delta_{m1} & \delta_{m2} & \cdots & \delta_{ml} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_l \end{bmatrix} \quad (15.12)$$

or

$$\mathbf{z}(t) = \mathbf{C}\mathbf{y}(t) + \mathbf{D}\mathbf{x}(t),$$

where $\mathbf{z}(t)$ is the *output vector*; its i th element $z_i(t)$ is the output generated at terminal i at time t .

The matrices $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$, and $\mathbf{D}$ defined by Eqs. (15.10) and (15.12) are the *characterizing matrices* of the linear machine; $\mathbf{A}$ is referred to as the *characteristic matrix* and specifies the autonomous behavior of the machine. The matrix formulation completely characterizes any linear machine, and thus it leads to a precise definition of a linear machine in terms of the characterizing matrices, as follows.

Definition 15.1 A machine is said to be *linear* over a finite field $GF(p)$ if its states can be identified with the elements of a vector space and its next-state and output functions can be specified by a pair of matrix equations over $GF(p)$,

$$\mathbf{Y}(t) = \mathbf{A}\mathbf{y}(t) + \mathbf{B}\mathbf{x}(t), \quad (15.13)$$

$$\mathbf{z}(t) = \mathbf{C}\mathbf{y}(t) + \mathbf{D}\mathbf{x}(t). \quad (15.14)$$

The *dimension* of the machine is the dimension of its state vector.

Equations (15.13) and (15.14) represent a Moore or Mealy machine, according to whether $\mathbf{D}$ is or is not identically zero. We subsequently refer to a machine whose characterizing matrices are $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$, and $\mathbf{D}$ as the machine $\{\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}\}$.

The elements of the characterizing matrices are determined from the next-state and output equations, Eqs. (15.9) and (15.11) respectively, in the following manner. The coefficient α_{ij} denotes the product of scalar multipliers contained

in the path leading from y_j to Y_i . If there are two or more paths from y_j to Y_i , α_{ij} denotes the sum of all such products; if no path exists between y_j and Y_i , $\alpha_{ij} = 0$. The coefficient β_{ij} denotes the corresponding values for the paths leading from the input x_j to Y_i . Similarly, γ_{ij} denotes the sum of products of the scalar multipliers contained in the paths leading from y_j to output terminal z_i ; if no path exists between y_j and z_i then $\gamma_{ij} = 0$. The coefficient δ_{ij} denotes the corresponding values for paths originating at input x_j and terminating at output z_i .

Example The characterizing matrices for the four-terminal linear machine of Fig. 15.3 are

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 2 & 2 & 0 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 0 & 2 \end{bmatrix}, \quad \mathbf{C} = \begin{bmatrix} 0 & 2 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}, \quad \mathbf{D} = \begin{bmatrix} 0 & 2 \\ 0 & 1 \end{bmatrix}$$

The response of linear machines

The relationship between the input sequence to the machine $\{\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}\}$ and its corresponding output sequence is obtained by iterating Eqs. (15.13) and (15.14), i.e.,

$$\begin{aligned} \mathbf{y}(1) &= \mathbf{A}\mathbf{y}(0) + \mathbf{B}\mathbf{x}(0), \\ \mathbf{z}(0) &= \mathbf{C}\mathbf{y}(0) + \mathbf{D}\mathbf{x}(0), \\ \mathbf{z}(1) &= \mathbf{C}\mathbf{A}\mathbf{y}(0) + \mathbf{C}\mathbf{B}\mathbf{x}(0) + \mathbf{D}\mathbf{x}(1), \\ \mathbf{z}(2) &= \mathbf{C}\mathbf{A}^2\mathbf{y}(0) + \mathbf{C}\mathbf{A}\mathbf{B}\mathbf{x}(0) + \mathbf{C}\mathbf{B}\mathbf{x}(1) + \mathbf{D}\mathbf{x}(2), \\ \mathbf{z}(3) &= \mathbf{C}\mathbf{A}^3\mathbf{y}(0) + \mathbf{C}\mathbf{A}^2\mathbf{B}\mathbf{x}(0) + \mathbf{C}\mathbf{A}\mathbf{B}\mathbf{x}(1) + \mathbf{C}\mathbf{B}\mathbf{x}(2) + \mathbf{D}\mathbf{x}(3), \\ &\vdots \\ \mathbf{z}(t) &= \mathbf{C}\mathbf{A}^t\mathbf{y}(0) + \sum_{j=0}^{t-1} \mathbf{C}\mathbf{A}^{t-1-j}\mathbf{B}\mathbf{x}(j) + \mathbf{D}\mathbf{x}(t) \end{aligned}$$

or

$$\mathbf{z}(t) = \mathbf{C}\mathbf{A}^t\mathbf{y}(0) + \sum_{j=0}^t \mathbf{H}(t-j)\mathbf{x}(j) \quad (15.15)$$

where

$$\mathbf{H}(t-j) = \begin{cases} \mathbf{D} & \text{when } t-j = 0, \\ \mathbf{C}\mathbf{A}^{t-1-j}\mathbf{B} & \text{when } t-1-j \geq 0. \end{cases} \quad (15.16)$$

From Eq. (15.15) we see that the response of a linear machine consists of two components. The first component, known as the *autonomous response*, is obtained by setting $\mathbf{x}(t) = \mathbf{0}$ for all $t \geq 0$, i.e.,

$$\mathbf{z}_a(t) = \mathbf{C}\mathbf{A}^t\mathbf{y}(0). \quad (15.17)$$

The second component, known as the *forced response*, is obtained by setting $\mathbf{y}(0) = \mathbf{0}$, i.e.,

$$\mathbf{z}_f(t) = \sum_{j=0}^t \mathbf{H}(t-j)\mathbf{x}(j). \quad (15.18)$$

The total response is thus given by

$$\mathbf{z}(t) = \mathbf{z}_a(t) + \mathbf{z}_f(t). \quad (15.19)$$

Equation (15.18) actually describes the response of inert machines in matrix form. These machines have been studied extensively in earlier sections by means of the polynomial representation. The total response, Eq. (15.19), of a linear machine for a given input sequence and an arbitrary initial state can be found by separately determining the forced and autonomous responses and adding them up.

The autonomous response is generally determined from the analysis of the internal circuit.³ The state behavior of the internal circuit is completely characterized by the characteristic matrix $\mathbf{A}$, since Eq. (15.13) becomes

$$\mathbf{Y}(t) = \mathbf{y}(t+1) = \mathbf{A}\mathbf{y}(t).$$

Because the internal circuit is autonomous, the λ -successor S_j of state S_i , where $S_i = \mathbf{y}_i(t)$, is given by

$$\mathbf{y}_j(t) = \mathbf{A}^\lambda \mathbf{y}_i(t)$$

where λ denotes the number of state transitions. (Note that while y_j denotes the state of the j th delay, $\mathbf{y}_i$ denotes the state S_i of the machine.)

The sequence of predecessors of a given state is established by constructing the inverse internal circuit; such an inverse exists only if each state has a unique predecessor. For an internal circuit given by $\mathbf{A}$, the inverse is given by $\mathbf{A}^{-1}$ since

$$\mathbf{y}(t) = \mathbf{A}^{-1}\mathbf{Y}(t).$$

Thus, the inverse circuit exists if and only if $\mathbf{A}$ is nonsingular, i.e., the determinant $|\mathbf{A}|$ is nonzero.

Autonomous linear machines are best analyzed either by means of their state diagrams (as illustrated earlier in Fig. 15.8) or by means of the characteristic polynomials derived from $\mathbf{A}$. For further discussion on autonomous linear machines, see [9].

15.5 Reduction of linear machines

We now determine conditions, in terms of characterizing matrices, for linear machines to be finite-memory and definitely diagnosable. The length of

³ The *internal circuit* is that part of the circuit that can be specified by $\mathbf{A}$ alone, that is, it contains only the delay elements and their interconnections; the input and output lines have been deleted.

the shortest distinguishing sequence for arbitrary initial uncertainty will be obtained. A procedure will be presented to determine whether a given linear machine is minimal and, if it is not, how to minimize it. The techniques developed in earlier chapters for arbitrary sequential machines are valid for linear machines as well. Our current objective, however, is to develop an *analytical* procedure, rather than an enumerative one, which is valid only for linear machines and which utilizes the matrix formulation.

The diagnostic matrix

Let L be a k -dimensional linear machine over $GF(p)$. To describe an experiment of length k , Eqs. (15.15) and (15.16) can be expressed compactly as

$$\mathbf{Z}^{(k)} = \mathbf{K}_k \mathbf{y}(0) + \mathbf{V}_k \mathbf{X}^{(k)}, \quad (15.20)$$

where

$$\mathbf{Z}^{(k)} = \begin{bmatrix} \mathbf{z}(0) \\ \mathbf{z}(1) \\ \vdots \\ \mathbf{z}(k-1) \end{bmatrix}, \quad \mathbf{K}_k = \begin{bmatrix} \mathbf{C} \\ \mathbf{CA} \\ \vdots \\ \mathbf{CA}^{k-1} \end{bmatrix}, \quad \mathbf{X}^{(k)} = \begin{bmatrix} \mathbf{x}(0) \\ \mathbf{x}(1) \\ \vdots \\ \mathbf{x}(k-1) \end{bmatrix},$$

and

$$\mathbf{V}_k = \begin{bmatrix} \mathbf{D} & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{CB} & \mathbf{D} & \mathbf{0} & \cdots & \cdot \\ \mathbf{CAB} & \mathbf{CB} & \cdot & \cdots & \cdot \\ \cdot & \cdot & \cdot & \cdots & \cdot \\ \cdot & \cdot & \cdot & \cdots & \mathbf{0} \\ \mathbf{CA}^{k-2}\mathbf{B} & \cdot & \cdot & \cdots & \mathbf{D} \end{bmatrix}.$$

The vector $\mathbf{y}(0)$ denotes the initial state at $t = 0$. For initial states S_a and S_b , the corresponding state vectors are denoted $\mathbf{y}_a(0)$ and $\mathbf{y}_b(0)$, respectively. The matrix $\mathbf{K}_k$, which consists of submatrices corresponding to the different outputs, is called the *diagnostic* (or *distinguishing*) matrix.

From Eq. (15.20) it is evident that if S_a is equivalent to S_b then

$$\mathbf{K}_k \mathbf{y}_a(0) = \mathbf{K}_k \mathbf{y}_b(0), \quad (15.21)$$

since the second term $\mathbf{V}_k \mathbf{X}^{(k)}$ is independent of the initial state and depends only on the input sequence. Moreover, since the inputs enter Eq. (15.20) additively, all input sequences are equally effective in state-distinguishing experiments. Consequently, to simplify the computation $\mathbf{X}^{(k)}$ may be selected as the all-zero sequence $\mathbf{X}^{(k)} = \mathbf{0}$, reducing Eq. (15.20) to

$$\mathbf{Z}^{(k)} = \mathbf{K}_k \mathbf{y}(0). \quad (15.22)$$

The proof that Eq. (15.21) is a necessary and sufficient condition for S_a and S_b to be equivalent follows from Theorem 15.1, and is left to the reader as an exercise.

Before proceeding with the investigation of the minimal linear machines, it is necessary to show that the first r linearly independent rows of the diagnostic matrix $\mathbf{K}_k$ occur in a consecutive sequence in $\mathbf{C}, \mathbf{CA}, \dots, \mathbf{CA}^i$, where $i < r$. To prove this assertion, assume that all the rows of $\mathbf{CA}^i$ are linear combinations of the rows of $\mathbf{K}_i$, i.e., the rows of $\mathbf{C}, \mathbf{CA}, \dots, \mathbf{CA}^{i-1}$. Then the rows of $\mathbf{CA}^{i+1}$ are the same linear combinations of rows of $\mathbf{K}_i \mathbf{A}$, i.e., $\mathbf{CA}, \mathbf{CA}^2, \dots, \mathbf{CA}^i$. However, since the rows of $\mathbf{CA}^i$ are linear combinations of the rows of $\mathbf{C}, \mathbf{CA}, \dots, \mathbf{CA}^{i-1}$, the rows of $\mathbf{CA}^{i+1}$ are also linear combinations of the rows of $\mathbf{C}, \mathbf{CA}, \dots, \mathbf{CA}^{i-1}$. Consequently, the process of finding the linearly independent rows of $\mathbf{K}_k$ terminates as soon as some submatrix $\mathbf{CA}^i$ is generated whose rows are linearly dependent on the rows of the preceding submatrices.

Theorem 15.1 *A k -dimensional linear machine $\{\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}\}$ is definitely diagnosable of order k if and only if diagnostic matrix $\mathbf{K}_k$ has k linearly independent rows.*

Proof The state vector $\mathbf{y}$ is k -dimensional and consequently $\mathbf{K}_k$ has exactly k columns. Thus, the rank of $\mathbf{K}_k$ cannot exceed k . If $\mathbf{K}_k$ contains k linearly independent rows then, under a sequence of all-zero inputs, the outputs corresponding to these rows in Eq. (15.22) impose k linearly independent constraints on $\mathbf{y}(0)$. Since $\mathbf{y}(0)$ is k -dimensional, it is specified uniquely by these constraints and thus the all-zero sequence of length k is a distinguishing sequence. However, since all input sequences of a given length have been shown to be equally effective in distinguishing experiments, every input sequence of length k or more is a distinguishing sequence and the machine is definitely diagnosable.

To prove that it is definitely diagnosable of order k , it is sufficient to note that the rows of $\mathbf{CA}^k, \mathbf{CA}^{k+1}, \dots$ are linearly dependent on the rows of $\mathbf{K}_k$ and thus the length of distinguishing sequences need not exceed the rank of $\mathbf{K}_k$. If $\mathbf{K}_k$ contains fewer than k linearly independent rows, there must exist some nonzero $\mathbf{y}(0) \neq \mathbf{0}$ that is annihilated by $\mathbf{K}_k$ and, hence, results in the same input-output behavior as in the case $\mathbf{y}(0) = \mathbf{0}$. This means that the machine in question is not reduced. $\diamond$

From Theorem 15.1, it follows that a linear machine is in reduced form if and only if the rank of $\mathbf{K}_k$ is k . Moreover, *every reduced k -dimensional linear machine is definitely diagnosable of order k and is finite-memory of order less than or equal to k* . These properties are also known as the *observability* and *predictability* properties of linear machines.

Example Consider the linear machine L_1 over $GF(2)$ given by the following matrices

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{C} = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}, \quad \mathbf{D} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

The diagnostic matrix $\mathbf{K}_3$ is obtained:

$$\mathbf{K}_3 = \begin{bmatrix} \mathbf{C} \\ \mathbf{CA} \\ \mathbf{CA}^2 \end{bmatrix}.$$

Thus Eq. (15.22) becomes

$$\begin{bmatrix} z_1(0) \\ z_2(0) \\ z_1(1) \\ z_2(1) \\ z_1(2) \\ z_2(2) \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} y_1(0) \\ y_2(0) \\ y_3(0) \end{bmatrix}.$$

The rank of $\mathbf{K}_3$ is 3 and hence the dimension of L_1 cannot be reduced. For a given initial state, the values of $y_1(0)$, $y_2(0)$, and $y_3(0)$ are specified, and the matrix $\mathbf{Z}^{(t)}$ yields the response of L_1 to the distinguishing sequence 000. For example, if the initial state is (111) then in response to 000 the sequences $z_1 = 010$ and $z_2 = 100$ are produced. It is suggested that the reader should draw the circuit diagram and compare actual circuit responses with responses obtained in an analytical manner.

The minimization procedure

Let L be a k -dimensional linear machine $\{\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}\}$ over $GF(p)$ and let r be the rank of the diagnostic matrix, where $r < k$. Define an $r \times k$ matrix $\mathbf{T}$ consisting of the first r linearly independent rows of $\mathbf{K}_k$, and a $k \times r$ matrix $\mathbf{R}$ denoting the right inverse of $\mathbf{T}$, such that $\mathbf{TR} = \mathbf{I}_r$ where $\mathbf{I}_r$ is the $r \times r$ identity matrix. Define an r -dimensional machine L^* with characterizing matrices $\{\mathbf{A}^*, \mathbf{B}^*, \mathbf{C}^*, \mathbf{D}^*\}$, such that

$$\mathbf{A}^* = \mathbf{TAR}, \quad \mathbf{B}^* = \mathbf{TB}, \quad \mathbf{C}^* = \mathbf{CR}, \quad \mathbf{D}^* = \mathbf{D}. \quad (15.23)$$

At this point, we shall state and prove a major theorem that establishes the validity of the following minimization procedure.

Theorem 15.2 *The State $\mathbf{y}$ of L is equivalent to the state $\mathbf{y}^* = \mathbf{T}\mathbf{y}$ of L^* . The machine L^* is a reduced machine equivalent to L .*

*Proof*⁴ In order to prove the first part, it is necessary and sufficient to show that, for every state of L , $\mathbf{y}^*$ and $\mathbf{T}\mathbf{y}$ have equivalent successors and yield identical output digits, i.e.,

$$\mathbf{T}(\mathbf{A}\mathbf{y} + \mathbf{B}\mathbf{x}) = \mathbf{A}^*\mathbf{y}^* + \mathbf{B}^*\mathbf{x}$$

and

$$\mathbf{C}\mathbf{y} + \mathbf{D}\mathbf{x} = \mathbf{C}^*\mathbf{y}^* + \mathbf{D}^*\mathbf{x}.$$

Define $\bar{\mathbf{y}} = \mathbf{y} - \mathbf{R}\mathbf{T}\mathbf{y}$; then, since $\mathbf{TR} = \mathbf{I}_r$, we obtain

$$\mathbf{T}\bar{\mathbf{y}} = \mathbf{T}\mathbf{y} - \mathbf{TR}\mathbf{T}\mathbf{y} = \mathbf{T}\mathbf{y} - \mathbf{T}\mathbf{y} = \mathbf{0}.$$

Since $\mathbf{T}\bar{\mathbf{y}} = \mathbf{0}$, we have $\mathbf{K}_k\bar{\mathbf{y}} = \mathbf{0}$. Therefore, by Eq. (15.21), state $\bar{\mathbf{y}}$ is equivalent to state $\mathbf{0}$. In addition, since $\mathbf{A}\mathbf{0} = \mathbf{0}$,

$$\mathbf{A}\bar{\mathbf{y}} = \mathbf{0} \quad \text{and} \quad \mathbf{TA}\bar{\mathbf{y}} = \mathbf{0}.$$

Also, since the rows of $\mathbf{C}$ are spanned by those of $\mathbf{T}$, $\mathbf{C}\bar{\mathbf{y}} = \mathbf{0}$. The next-state and output equations are

$$\begin{aligned} \mathbf{T}(\mathbf{A}\mathbf{y} + \mathbf{B}\mathbf{x}) &= \mathbf{T}[\mathbf{A}(\bar{\mathbf{y}} + \mathbf{R}\mathbf{T}\mathbf{y}) + \mathbf{B}\mathbf{x}] = \mathbf{TA}\bar{\mathbf{y}} + \mathbf{TAR}\mathbf{T}\mathbf{y} + \mathbf{TB}\mathbf{x} \\ &= \mathbf{0} + (\mathbf{TAR})(\mathbf{T}\mathbf{y}) + (\mathbf{TB})\mathbf{x} = \mathbf{A}^*\mathbf{y}^* + \mathbf{B}^*\mathbf{x}, \\ \mathbf{C}\mathbf{y} + \mathbf{D}\mathbf{x} &= \mathbf{C}(\bar{\mathbf{y}} + \mathbf{R}\mathbf{T}\mathbf{y}) + \mathbf{D}\mathbf{x} = \mathbf{C}\bar{\mathbf{y}} + \mathbf{CRT}\mathbf{y} + \mathbf{D}\mathbf{x} \\ &= \mathbf{0} + (\mathbf{CR})(\mathbf{T}\mathbf{y}) + \mathbf{D}\mathbf{x} = \mathbf{C}^*\mathbf{y}^* + \mathbf{D}^*\mathbf{x}. \end{aligned}$$

Hence, $\mathbf{y}^* = \mathbf{T}\mathbf{y}$ under the transformation of Eq. (15.23). Similarly, since $\mathbf{R}\mathbf{y}^* = \mathbf{R}\mathbf{T}\mathbf{y} = \mathbf{y}$, the state $\mathbf{y}^*$ of L^* is equivalent to the state $\mathbf{y} = \mathbf{R}\mathbf{y}^*$ of L .

We shall now show that L^* is a reduced machine and thus is the minimal machine equivalent to L . Since $\mathbf{K}_k$ has rank less than k , it partitions the states of L into subsets (usually called cosets) as follows. Let G_0 denote the subset containing all states that are equivalent to the zero state $\mathbf{y} = \mathbf{0}$. From Eq. (15.21) we conclude that G_0 denotes the null space of $\mathbf{K}_k$. Let us now generate a set of subsets from G_0 such that two states $\mathbf{y}_a$ and $\mathbf{y}_b$ are in the same subset if and only if $\mathbf{y}_a - \mathbf{y}_b$ is in G_0 . Hence, $\mathbf{K}_k(\mathbf{y}_a - \mathbf{y}_b) = \mathbf{0}$ and $\mathbf{K}_k\mathbf{y}_a = \mathbf{K}_k\mathbf{y}_b$, which means that $\mathbf{y}_a$ is equivalent to $\mathbf{y}_b$ and the subsets so generated are the equivalence classes of L . Moreover, since states in different subsets are distinguishable by the all-zero sequence (or any other input sequence), the subsets generated by $\mathbf{K}_k$ correspond to states of the reduced form of the original machine. (These subsets are actually identical to the blocks of the final partition in the reduction procedure outlined in Chapter 10.)

Since G_0 generates $p^r - 1$ distinct subsets, the reduced form of L over $GF(p)$ has p^r states, where r is the rank of $\mathbf{K}_k$. Since L and L^* are equivalent and L^* has exactly p^r states, it is the minimal machine equivalent to L . $\diamond$

⁴ This proof requires some knowledge of matrix algebra and may be skipped at first reading.

Example Consider the linear machine L_2 over $GF(2)$ defined by the matrices

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 1 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \quad \mathbf{C} = [1 \ 0 \ 0], \quad \mathbf{D} = [1],$$

$$\mathbf{K}_3 = \begin{bmatrix} \mathbf{C} \\ \mathbf{CA} \\ \mathbf{CA}^2 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}.$$

The rank of $\mathbf{K}_3$ is 2 and thus L_2 is reducible. The first two rows of $\mathbf{K}_3$ are linearly independent; therefore

$$\mathbf{T} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}.$$

The right inverse $\mathbf{R}$ of $\mathbf{T}$ is constructed by selecting a set of r linearly independent columns from $\mathbf{T}$. Since the rank of $\mathbf{T}$ is r and column rank equals row rank, such a set always exists. Form an $r \times r$ matrix $\mathbf{Q}$ from these columns and find its inverse, $\mathbf{Q}^{-1}$. The right inverse $\mathbf{R}$, which is a $k \times r$ matrix, is formed by placing in it the rows of $\mathbf{Q}^{-1}$ in positions corresponding to the columns selected from $\mathbf{T}$, all other rows being set to zero.

In our case,

$$\mathbf{Q} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{Q}^{-1} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{R} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}.$$

Following the definitions of the characterizing matrices of L_2^* , we obtain

$$\mathbf{y}^* = \mathbf{T}\mathbf{y} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \mathbf{y}$$

$$\mathbf{A}^* = \mathbf{T}\mathbf{A}\mathbf{R} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix},$$

$$\mathbf{B}^* = \mathbf{T}\mathbf{B} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

$$\mathbf{C}^* = \mathbf{C}\mathbf{R} = [1 \ 0 \ 0] \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix} = [1 \ 0],$$

$$\mathbf{D}^* = \mathbf{D} = [1].$$

The circuit diagram of the reduced machine L_2^* given by $\{\mathbf{A}^*, \mathbf{B}^*, \mathbf{C}^*, \mathbf{D}^*\}$ is shown in Fig. 15.15.

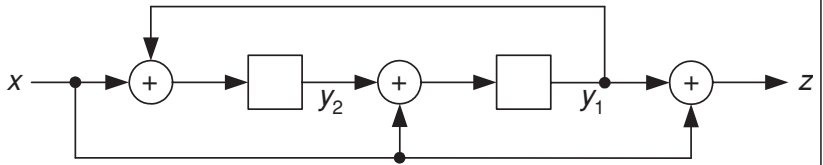


Fig. 15.15 Realization of the reduced machine L_2^* .

The minimal machine L_2^* has been obtained without explicitly constructing the equivalence classes of L_2 . We shall now find them to demonstrate the procedure outlined in the proof of Theorem 15.2. From Eq. (15.22), we have

$$\begin{bmatrix} z_1(0) \\ z_1(1) \end{bmatrix} = \mathbf{T}\mathbf{y}(0) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} y_1(0) \\ y_2(0) \\ y_3(0) \end{bmatrix}. \quad (15.24)$$

Here G_0 contains all the states, designated by their corresponding vectors, for which $\mathbf{0} = \mathbf{T}\mathbf{y}(0)$, i.e.,

$$G_0 = \left\{ \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \right\}.$$

The remaining subsets, which yield equivalence classes of L_2 , are obtained by adding to G_0 any element not contained in it and such that two states $\mathbf{y}_a$ and $\mathbf{y}_b$ are in the same subset if and only if $\mathbf{y}_a - \mathbf{y}_b$ is in G_0 . Let the first such element be the vector

$$\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad \text{which yields} \quad G_1 = \left\{ \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} \right\}.$$

Similarly, we obtain the remaining equivalence classes,

$$G_2 = \left\{ \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \right\}, \quad G_3 = \left\{ \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \right\}.$$

Note that, since $\mathbf{y}^* = \mathbf{T}\mathbf{y}$, the output vector of Eq. (15.24) actually specifies the state of L_2^* that corresponds to the equivalence class given by G_i .

Example Consider the linear machine L_3 given by $\{\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}\}$ over $GF(2)$ and shown in Fig. 15.16.

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{C} = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}, \quad \mathbf{D} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix},$$

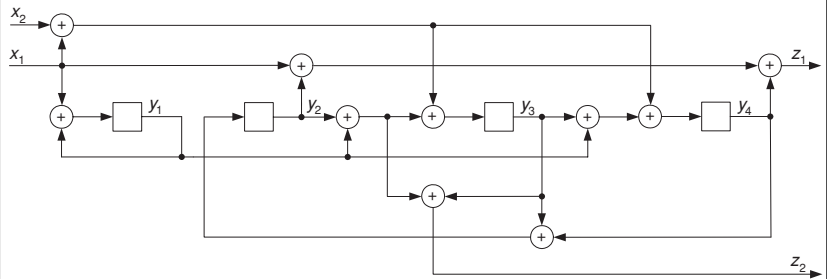


Fig. 15.16 Realization of the machine L_3 .

$$\mathbf{K}_3 = \begin{bmatrix} \mathbf{C} \\ \mathbf{CA} \\ \mathbf{CA}^2 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix},$$

$$\mathbf{T} = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{Q} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix},$$

$$\mathbf{Q}^{-1} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}, \quad \mathbf{R} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}.$$

The matrix $\mathbf{Q}$ occupies the first three columns of $\mathbf{T}$ and $\mathbf{Q}^{-1}$ the first three rows of $\mathbf{R}$, since the linearly independent columns in $\mathbf{T}$ have been selected from positions 1, 2, and 3. We have

$$\mathbf{A}^* = \mathbf{TAR} = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix},$$

$$\mathbf{B}^* = \mathbf{TB} = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 1 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 0 & 1 \end{bmatrix},$$

$$\mathbf{C}^* = \mathbf{CR} = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix},$$

$$\mathbf{D}^* = \mathbf{D} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}.$$

The reduced circuit corresponding to $\{\mathbf{A}^*, \mathbf{B}^*, \mathbf{C}^*, \mathbf{D}^*\}$ is shown in Fig. 15.17.

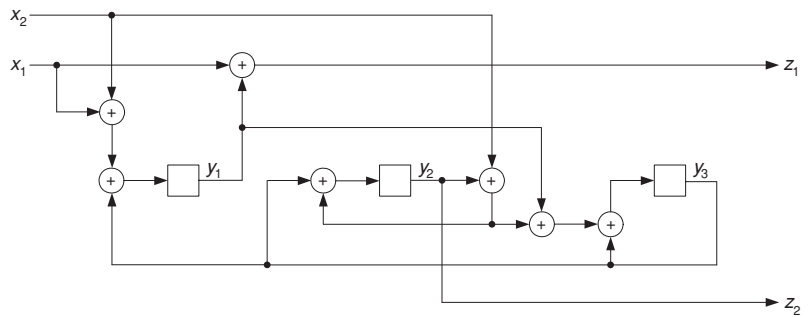


Fig. 15.17 The reduced machine L_3^* .

It is useful to note that the first three linearly independent rows of the diagnostic matrix $\mathbf{K}_3^*$ of the reduced machine L_3^* are the rows of I_3 in natural order, that is,

$$\mathbf{K}_3^* = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix} \begin{matrix} \checkmark \\ \checkmark \\ \checkmark \end{matrix}$$

From Eq. (15.23) we can show that the matrix $(\mathbf{A}^*)^t$ of the reduced machine is related to the original matrix $\mathbf{A}^t$ by

$$(\mathbf{A}^*)^t = \mathbf{TA}^t\mathbf{R}$$

and that the diagnostic matrix $\mathbf{K}^*$ is related to $\mathbf{K}$ by

$$\mathbf{K}^* = \mathbf{KR}.$$

The formal proof of the above relationships is left to the reader as an exercise (see Problem 15.23). Their immediate consequence is summarized as follows.

- The first r linearly independent rows of the matrix $\mathbf{K}_r^*$ of a reduced linear machine are the rows of the identity matrix $\mathbf{I}_r$.

Applying the above results to Eq. (15.22) suggests that for an initial state $\mathbf{y}_a^* = [\mathbf{y}_1^*, \mathbf{y}_2^*, \dots, \mathbf{y}_r^*]^T$ (where $[\mathbf{y}]^T$ denotes the transpose of $\mathbf{y}$) and under an all-0's input sequence the output values corresponding to the unit vector rows of $\mathbf{K}_r^*$ are identical to the values $y_1^*, y_2^*, \dots, y_r^*$. This result is of paramount importance in the identification problem of linear machines, which is discussed in the following section.

15.6 Identification of linear machines

We shall now establish certain conditions under which a reduced sequential machine will be linearly realizable. We shall determine an appropriate state assignment and define the characterizing matrices of a linear machine of the smallest dimension. We will assume that the input and output symbols of the machine are taken from $GF(p)$ and that the zero element of the field is specified. If a machine is not linearly realizable, one of several tests in the procedure will fail.

The identification procedure

From the discussion in Section 15.5 we know that a linearly realizable machine must have exactly p^k states for some integer k . Moreover, *a machine is equivalent to a linear machine if and only if its reduced form is linear.*

Let a sequential machine M have p^k states, denoted $S_a, S_b, \dots, S_{p^k}$, and let the l -dimensional vector $\mathbf{x}$ and the m -dimensional vector $\mathbf{z}$ denote its input and output vectors, respectively. We construct for M a *distinguishing table* that contains the *output symbols generated by M in response to a sequence of 0's*. The table contains p^k columns corresponding to the states of M . It is formed block by block; the i th block corresponds to the output vector $\mathbf{z}(t)$ at $t = i$. The table thus contains at most k blocks of m rows each, corresponding to the output vectors $\mathbf{z}(0), \mathbf{z}(1), \dots, \mathbf{z}(k-1)$. The process of adding blocks to the table is terminated when, for some t , the set of rows contained in block $\mathbf{z}(t)$ is linearly dependent on the rows in preceding blocks.

As an example, we will construct the distinguishing table for the machine M_4 of Table 15.1. It is given in Table 15.2. The entries in the column headed A are 11, 01 and correspond to the output symbols of M_4 when it is initially in state A and given the input sequence 00. The construction of Table 15.2 terminates after the second block since the rows of $\mathbf{z}(1)$ are linear combinations of those of $\mathbf{z}(0)$. We shall subsequently denote the distinguishing table by U .

Table 15.1 Machine M_4

	$NS, z_1 z_2$	
	$x = 0$	$x = 1$
PS		
A	$B, 11$	$D, 01$
B	$A, 01$	$C, 11$
C	$C, 10$	$A, 00$
D	$D, 00$	$B, 10$

Table 15.2 Distinguishing table for M_4

	A	B	C	D
$z(0)$	1	0	1	0
	1	1	0	0
$z(1)$	0	1	1	0
	1	1	0	0

Since the input and output symbols of M_4 are limited to 0 and 1, the linear realization has to be over $GF(2)$. The first test is based on the fact that, for every linear machine, the all-0's sequence is a distinguishing sequence. If M is reduced then the columns of U must be distinct, since otherwise there would be two or more states in M that are indistinguishable under the all-0's sequence, and M would not be linear. Clearly, Table 15.2 passes this test.

Let U^* be the table consisting of the first r linearly independent rows of U , and let S_i denote the i th column of U^* . Assuming that a linear realization of M is possible, let the states $A, B, \dots$ of M correspond to the state vectors $\mathbf{y}_a, \mathbf{y}_b, \dots$ of its linear realization L . This is accomplished by selecting the p^k columns of U^* as the state assignment for the p^k states of L . For the machine L_4 , which is to be the linear realization of M_4 , we have

$$\mathbf{y}_a = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{y}_b = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad \mathbf{y}_c = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \mathbf{y}_d = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

In the above step, it has been implicitly assumed that if a linear realization exists, its state assignment is given by U^* . This assertion follows directly from the result of the preceding section, in which it was shown that, under an all-0's input sequence, the output values corresponding to the r linearly independent rows of $\mathbf{K}_r^*$ are identical to the state assignment given by $(y_1^*, y_2^*, \dots, y_r^*)$. In addition, since the rows of U^* are the linearly independent output vectors associated with the states of L , they are also equal to the state assignment of L .

In order to obtain the set of characterizing matrices $\{\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}\}$ of L , we select r linearly independent columns from U^* , corresponding to the r state vectors of L , and form an $r \times r$ matrix $\mathbf{v}$ such that

$$\mathbf{v} = [\mathbf{y}_a \quad \mathbf{y}_b \quad \cdots \quad \mathbf{y}_r].$$

From Eq. (15.13), we find that the next-state function of L under input symbols 0 is

$$[\mathbf{Y}_a^0 \quad \mathbf{Y}_b^0 \quad \cdots \quad \mathbf{Y}_r^0] = \mathbf{A}\mathbf{v},$$

where $\mathbf{Y}_i^0$ denotes the 0-successor of $\mathbf{y}_i$. Since $\mathbf{v}$ is nonsingular, we can write

$$\mathbf{A} = [\mathbf{Y}_a^0 \quad \mathbf{Y}_b^0 \quad \cdots \quad \mathbf{Y}_r^0] \mathbf{v}^{-1}. \quad (15.25)$$

If all r unit vectors appear in U^* then $\mathbf{v}$ can be chosen as $\mathbf{I}_r$, which yields $\mathbf{v} = \mathbf{v}^{-1}$, and so Eq. (15.25) is reduced to

$$\mathbf{A} = [\mathbf{Y}_a^0 \quad \mathbf{Y}_b^0 \quad \cdots \quad \mathbf{Y}_r^0]. \quad (15.26)$$

Whenever the number of states $p^k = p^r$, i.e., $k = r$, $\mathbf{v}$ can be specified as $\mathbf{I}_r$.

Similarly, from Eq. (15.14) and for $\mathbf{x}(t) = \mathbf{0}$, we find that

$$[\mathbf{z}_a^0 \quad \mathbf{z}_b^0 \quad \cdots \quad \mathbf{z}_r^0] = \mathbf{C}\mathbf{v},$$

where $\mathbf{z}_i^0$ denotes the output symbol produced by L when in the state $\mathbf{y}_i$ and excited by the input symbol $\mathbf{x} = \mathbf{0}$. Thus

$$\mathbf{C} = [\mathbf{z}_a^0 \quad \mathbf{z}_b^0 \quad \cdots \quad \mathbf{z}_r^0]\mathbf{v}^{-1} \quad (15.27)$$

and so, when $\mathbf{v} = \mathbf{I}_r$,

$$\mathbf{C} = [\mathbf{z}_a^0 \quad \mathbf{z}_b^0 \quad \cdots \quad \mathbf{z}_r^0]. \quad (15.28)$$

In order to obtain $\mathbf{B}$ and $\mathbf{D}$, let us denote a unit input vector as $\mathbf{u}_i$, where the i th component of $\mathbf{u}_i$ is 1 and all other components are 0's. From Eq. (15.13) we obtain

$$\mathbf{B}\mathbf{x} = \mathbf{Y} - \mathbf{A}\mathbf{y}.$$

In order to obtain $\mathbf{B}$, we select some state $\mathbf{y}_i$ (preferably the zero state if it exists in U^*) and specify $\mathbf{B}$ in terms of the constraints imposed on it by $\mathbf{y}_i$ and the unit input vectors. Clearly, such a process does not guarantee that the selection of another $\mathbf{y}_j$ will specify the same $\mathbf{B}$ matrix, unless the machine being identified is indeed linear. For the time being, we shall specify a set of characterizing matrices and will check them for all possible input and state combinations at the end of the test.

Let the input consist of the unit vectors

$$\mathbf{u} = [\mathbf{u}_1 \quad \mathbf{u}_2 \quad \cdots \quad \mathbf{u}_l].$$

The next-state vector $\mathbf{Y}_i^{u_j}$ denotes the u_j -successor of $\mathbf{y}_i$. Thus,

$$\mathbf{Y}_i^u = [\mathbf{Y}_i^{u_1} \quad \mathbf{Y}_i^{u_2} \quad \cdots \quad \mathbf{Y}_i^{u_l}]$$

and

$$\mathbf{B}\mathbf{u} = \mathbf{Y}_i^u - \mathbf{A}\mathbf{y}_i$$

or

$$\mathbf{B} = [\mathbf{Y}_i^u - \mathbf{A}\mathbf{y}_i] \quad \mathbf{u}^{-1}. \quad (15.29)$$

Since $\mathbf{u}$ generally consists of unit vectors, when $\mathbf{y}$ is the zero state Eq. (15.29) reduces to

$$\mathbf{B} = [\mathbf{Y}_i^{u_1} \quad \mathbf{Y}_i^{u_2} \quad \cdots \quad \mathbf{Y}_i^{u_l}]. \quad (15.30)$$

Similarly, from Eq. (15.14) we obtain

$$\mathbf{D} = \{[\mathbf{z}_i^{u_1} \quad \mathbf{z}_i^{u_2} \quad \cdots \quad \mathbf{z}_i^{u_l}] - \mathbf{A}\mathbf{y}_i\}\mathbf{u}^{-1}, \quad (15.31)$$

where $\mathbf{z}_i^{u_j}$ is the output vector associated with the transition from $\mathbf{y}_i$ under an input $\mathbf{u}_j$. In analogy with Eq. (15.30) the reduced equation is

$$\mathbf{D} = [\mathbf{z}_i^{u_1} \quad \mathbf{z}_i^{u_2} \quad \cdots \quad \mathbf{z}_i^{u_l}]. \quad (15.32)$$

Returning to machine M_4 , we make the specification

$$\mathbf{v} = [\mathbf{y}_c \quad \mathbf{y}_b] = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \mathbf{I}_2.$$

From Eqs. (15.26) and (15.28), we obtain

$$\mathbf{A} = [\mathbf{Y}_c^0 \quad \mathbf{Y}_b^0] = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{C} = [\mathbf{z}_c^0 \quad \mathbf{z}_b^0] = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

The only unit input vector is $\mathbf{u} = [1]$, and hence $\mathbf{Y}_i^1$ is the 1-successor of $\mathbf{y}_i$. Since the zero state is contained in U^* , let $\mathbf{y}_i = \mathbf{y}_d$; then, by Eqs. (15.30) and (15.32), we obtain

$$\mathbf{B} = [\mathbf{Y}_d^1] = [\mathbf{y}_b] = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad \mathbf{D} = [\mathbf{z}_d^1] = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

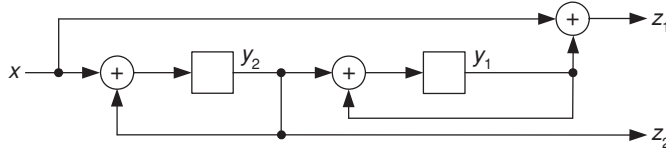
The state and output equations are

$$\begin{aligned} \mathbf{Y}(t) &= \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \mathbf{y}(t) + \begin{bmatrix} 0 \\ 1 \end{bmatrix} \mathbf{x}(t), \\ \mathbf{z}(t) &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \mathbf{y}(t) + \begin{bmatrix} 1 \\ 0 \end{bmatrix} \mathbf{x}(t). \end{aligned}$$

The final test is to verify that the above equations indeed represent the machine M_4 under *all* input and state combinations. This is accomplished by verifying each state transition and its corresponding output symbol. For example, substituting $\mathbf{y}_a$ for $\mathbf{A}$ and $\mathbf{0}$ for $\mathbf{x}(t)$, the machine should go to the state $\mathbf{y}_b$ and produce the output symbol 11, corresponding to the entry B , 11 in column 0, row A , in Table 15.1. Indeed,

$$\begin{aligned} \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} [0] &= \begin{bmatrix} 0 \\ 1 \end{bmatrix} \rightarrow \mathbf{y}_b, \\ \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \end{bmatrix} [0] &= \begin{bmatrix} 1 \\ 1 \end{bmatrix} \rightarrow \mathbf{z}_a^0. \end{aligned}$$

The characterizing matrices are thus verified, and the linear realization of Fig. 15.18 results.

Fig. 15.18 The machine L_4 .

Example The machine M_5 and its distinguishing table are given in Tables 15.3 and 15.4, respectively. The “checked” rows are linearly independent, and since U^* contains all eight possible 3-tuples, the identification procedure is continued.

Table 15.3 Machine M_5

PS	$NS, z_1 z_2$	
	$x = 0$	$x = 1$
A	$A, 00$	$E, 10$
B	$A, 10$	$E, 00$
C	$B, 11$	$F, 01$
D	$B, 01$	$F, 11$
E	$C, 01$	$G, 11$
F	$C, 11$	$G, 01$
G	$D, 10$	$H, 00$
H	$D, 00$	$H, 10$

Table 15.4 Distinguishing table for M_5

	A	B	C	D	E	F	G	H	
$\mathbf{z}(0)$	0	1	1	0	0	1	1	0	✓
	0	0	1	1	1	1	0	0	✓
$\mathbf{z}(1)$	0	0	1	1	1	1	0	0	
	0	0	0	0	1	1	1	1	✓
$\mathbf{z}(2)$	0	0	0	0	1	1	1	1	
	0	0	0	0	0	0	0	0	

Therefore, select

$$\mathbf{v} = [\mathbf{y}_b \quad \mathbf{y}_d \quad \mathbf{y}_h] = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \mathbf{I}_3.$$

From Eqs. (15.26) and (15.28), we obtain

$$\mathbf{A} = [\mathbf{Y}_b^0 \quad \mathbf{Y}_d^0 \quad \mathbf{Y}_h^0] = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}, \mathbf{C} = [\mathbf{z}_b^0 \quad \mathbf{z}_d^0 \quad \mathbf{z}_h^0] = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}.$$

Setting $\mathbf{u} = [1]$ and $\mathbf{y}_i = \mathbf{y}_a = \mathbf{0}$, Eqs. (15.30) and (15.32) yield

$$\mathbf{B} = [\mathbf{Y}_a^1] = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}, \quad \mathbf{D} = [\mathbf{z}_a^1] = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

Thus

$$\mathbf{Y}(t) = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} \mathbf{y}(t) + \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} \mathbf{x}(t),$$

$$\mathbf{z}(t) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \mathbf{y}(t) + \begin{bmatrix} 1 \\ 0 \end{bmatrix} \mathbf{x}(t).$$

The matrices are verified as corresponding to M_5 , and their linear realization is given in Fig. 15.19.

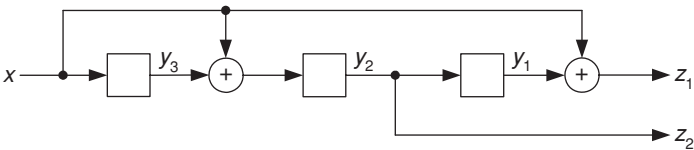


Fig. 15.19 The machine L_5 .

Example As another example, consider the four-stage up–down Gray-code counter of Table 15.5, whose distinguishing table is given in Table 15.6.

Table 15.5 The machine M_6

	NS		$z_1 z_2$
	$x = 0$	$x = 1$	
A	B	D	00
B	C	A	01
C	D	B	11
D	A	C	10

Table 15.6 Distinguishing table for M_6

	A	B	C	D	
$\mathbf{z}(0)$	0	0	1	1	✓
	0	1	1	0	✓
$\mathbf{z}(1)$	0	1	1	0	
	1	1	0	0	✓
$\mathbf{z}(2)$	1	1	0	0	
	1	0	0	1	

The state assignment is given by

$$\mathbf{y}_a = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad \mathbf{y}_b = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}, \quad \mathbf{y}_c = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{y}_d = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}.$$

Note that although M_6 has only four states, its minimal linear realization has a third dimension; that is, if M_6 is linearly realizable then it is realizable as a submachine of an eight-state linear machine. Note also that $\mathbf{v}$ cannot be chosen as the identity matrix, and the zero state $\mathbf{y}_i = \mathbf{0}$ is not contained in the state assignment. Consequently, the simplified equations cannot be used, and matrix inversion cannot be avoided. Let

$$\mathbf{v} = [\mathbf{y}_d \quad \mathbf{y}_b \quad \mathbf{y}_a] = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}; \quad \text{then} \quad \mathbf{v}^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}.$$

From Eqs. (15.25) and (15.27), we obtain

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix} \mathbf{v}^{-1} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix},$$

$$\mathbf{C} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \mathbf{v}^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}.$$

Let $\mathbf{y}_i^1 = \mathbf{y}_a$. Then from Eq. (15.29) we obtain

$$\mathbf{B} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} - \mathbf{A} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \quad \mathbf{D} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

The minimum-dimensional linear circuit realizing the counter is shown in Fig. 15.20.

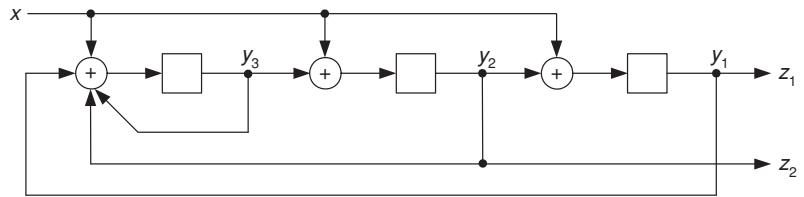


Fig. 15.20 Linear realization of the Gray-code counter.

15.7 Application of linear machines to error correction

The availability of analysis and synthesis techniques for linear machines and their economical realization by means of shift registers have made them widely

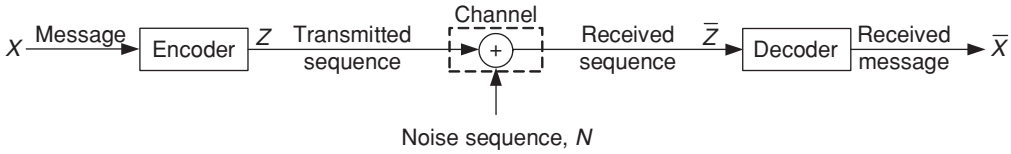


Fig. 15.21 A model for a communication system.

applicable in communication and digital computation. Linear machines are particularly useful in computations involving the multiplication and division of polynomials and in error detection and correction. In this section we describe in detail how they can be used in a simple error-correcting coding scheme. For a more complete survey of coding and digital computation applications, the reader is referred to Peterson [15] and Gill [9].

Consider the communication-system model shown in Fig. 15.21. The *message*, denoted X , consists of a sequence over $GF(p)$ of length n . The *encoder*, whose transfer function is T , transforms the message into another sequence over $GF(p)$ of length n . This sequence is referred to as the *transmitted sequence* and is designated Z , where $Z = TX$. The sequence Z is transmitted through a noisy *channel*, whose output sequence $\bar{Z}$ is called the *received sequence*. In the channel, a *noise sequence* over $GF(p)$, denoted N , is added to the transmitted sequence, so that the received sequence is equal to

$$\begin{aligned}\bar{Z} &= Z + N \\ &= TX + N.\end{aligned}$$

The *decoder*, whose transfer function is T^{-1} , processes the received sequence and produces a sequence $\bar{X}$ such that

$$\begin{aligned}\bar{X} &= T^{-1}\bar{Z} \\ &= T^{-1}(TX + N) \\ &= X + T^{-1}N.\end{aligned}$$

If the noise sequence is equal to zero, that is, $N = 0$, then the *received message* $\bar{X}$ is a replica of the original message X , that is, $X = \bar{X}$. If the noise sequence is different from zero then the received message $\bar{X}$ consists of the modulo- p sum of the original message X and the response $T^{-1}N$ of the decoder to the noise sequence.

As an illustration of the error-correction procedure, let us analyze in detail the communication system shown in Fig. 15.22, where the encoder's transfer function is given by $T = 1 + D^2 + D^3$ and the message as well as the noise are over $GF(2)$. We assume that the noise sequence contains only a single nonzero digit; i.e., the communication system is *single-error-correcting*. Suppose that a seven-bit message X is to be transmitted, where the first four digits are the *information digits* and the remaining three digits are the *checking digits*. The checking digits in X are always 0's. Consequently, if $\bar{X}$ is received with three 0's in the last three positions then it means that no noise is present in the channel and $\bar{X}$ is an identical replica of X . If, however, the received message $\bar{X}$ contains nonzero digits in the last three positions, this indicates that an error

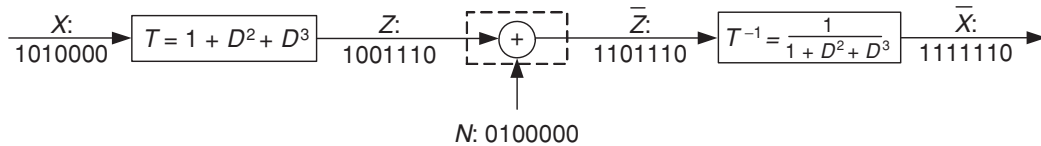


Fig. 15.22 An example of a linear single-error-correcting scheme.

has occurred during transmission and an error-correcting procedure must be employed to recover the original message.

When an error occurs, it is necessary to obtain the sequence $T^{-1}N$ and subtract it from the received message $\bar{X}$. To obtain $T^{-1}N$, we observe that since the last three digits of X were originally 0's then the last three digits of $\bar{X}$ must consist only of digits of $T^{-1}N$, without any contribution from X . In fact, if only a single error occurred at time t then the sequence $T^{-1}N$ is simply the response of decoder T^{-1} to a unit impulse occurring at t . Therefore, the checking digits of $\bar{X}$ consist of a subsequence of three digits of the impulse response of T^{-1} . (Clearly, if the error occurs in one of the checking digits, say in the second checking digit, then the first digit will be a zero and the remaining two checking digits will be the first two digits of the impulse response of T^{-1} .)

The decoder is chosen so that its impulse response has a maximal period of seven digits. This ensures that, by observing the subsequence contained in the last three digits of $\bar{X}$, we can determine uniquely the entire sequence $T^{-1}N$. Since a maximal impulse response contains all seven possible combinations of three successive nonzero digits, each noise impulse corresponds to only one pattern of checking digits and thus its location can be uniquely determined.

As an example, suppose that the sequence 1010000 is to be transmitted by means of the communication system of Fig. 15.22. The transmitted sequence Z is found to be 1001110. If an error occurs in the second digit, the received sequence $\bar{Z}$ will be 1101110. Since the impulse response of the decoder, whose transfer function is $T^{-1} = (1 + D^2 + D^3)^{-1}$, is 1011100, the received message $\bar{X}$ is equal to 1111110. The checking digits of $\bar{X}$ are identical to the fourth, fifth, and sixth digits of the impulse response. Consequently, we may conclude that the noise impulse has occurred in the second information digit. The sequence $T^{-1}N$ is thus found to be 0101110, and it may now be added (the same as subtracting modulo 2) to $\bar{X}$ to obtain the original message X , i.e.,

Decoder's impulse response: 1 0 1 1 1 0 0

$$\begin{array}{rcccccccc}
 \bar{X}: & 1 & 1 & 1 & 1 & 1 & 1 & 0 & + \\
 T^{-1}N: & 0 & 1 & 0 & 1 & 1 & 1 & 0 & \\
 \hline
 X: & 1 & 0 & 1 & 0 & 0 & 0 & 0 &
 \end{array}$$

In a similar manner, the reader can verify that if the message 1110000 is transmitted by means of the system of Fig. 15.22, and the noise N is given by 0010000, then the received message would be 1100111. The checking digits

contain the third, fourth, and fifth digits of the decoder's impulse response. Consequently, $T^{-1}N$ is equal to 0010111, and the message X can be reconstructed.

To obtain single-error correction for messages over $GF(2)$ containing m information digits and k checking digits, we need a decoder whose impulse response is of length $m + k$, with each string of k successive digits different from every other subsequence of length k . Such an impulse response can be obtained from a decoder whose transfer function is of degree k and whose impulse response is maximal, i.e., of length $m + k = 2^k - 1$. If the last k digits of received message $\bar{X}$ are not zeros then the sequence $T^{-1}N$ must be subtracted from $\bar{X}$. This can be accomplished by shifting $\bar{X}$ over the decoder's impulse response until the last k digits of $\bar{X}$ match a corresponding subsequence of the impulse response. This is always possible since the impulse response contains every nonzero subsequence of length k . The modulo-2 sum of $\bar{X}$ and the digits of the impulse response appearing directly below it yield the original message X .

Appendix 15.1 Basic properties of finite fields⁵

A set R is said to form a *ring* if two operations, addition and multiplication, are defined for every pair of elements in R , and if it satisfies the following postulates.

1. *Closure* For every a and b in R , $a + b$ and ab are in R .
2. *Associativity* For every a , b , and c in R , $(a + b) + c = a + (b + c)$ and $(ab)c = a(bc)$.
3. The set R contains a *unique zero element*, denoted 0, such that, for every a in R , $a + 0 = 0 + a = a$.
4. To each a in R , there corresponds a unique element $-a$ in R such that $a + (-a) = (-a) + a = 0$; $-a$ is called the *inverse* of a .
5. *Distributivity* Multiplication distributes over addition; that is, $a(b + c) = ab + ac$, for all a , b , and c in R .
6. *Commutativity* For all a and b in R , $a + b = b + a$.

If multiplication is also commutative, i.e., $ab = ba$, R is said to be a *commutative ring*.

Example The set of integers $\{0, 1, \dots, p - 1\}$ under modulo- p addition and multiplication operations forms a commutative ring. (Note that modulo p means that a is equal to b whenever $a - b$ is a multiple of p). The definition of modulo-4 operations is shown in Table A15.1.

⁵ This is only a short summary of several definitions and results in the area of fields. For a more complete coverage, the reader is referred to any book on algebra.

Table A15.1 Addition and multiplication modulo 4

+	0	1	2	3	·	0	1	2	3
0	0	1	2	3	0	0	0	0	0
1	1	2	3	0	1	0	1	2	3
2	2	3	0	1	2	0	2	0	2
3	3	0	1	2	3	0	3	2	1

The set F is said to be a *field* if it is a commutative ring and, in addition, satisfies the following two postulates.

1. There is a unique nonzero element 1 in F such that $a1 = a$ for every a in F .
2. To each nonzero a in F , there corresponds a unique element a^{-1} (or $1/a$) in F such that $aa^{-1} = 1$.

The set of real numbers and the set of complex numbers each forms an infinite field. Fields containing a finite number of elements are usually called *finite fields*.

Example The modulo-4 ring defined in Table A15.1 is not a field, since the element 2 does not have a multiplicative inverse; that is, the equation $2a = 1$ does not have a solution for a , as can be seen from the defining table. However, the equation $2a = 2$ (modulo 4) has two solutions, $a = 1$ and $a = 3$.

The above example illustrates the reason for restricting our discussion of linear machines to modulo p of prime numbers: multiplication by numbers that are not prime to the modulo may be irreversible and, consequently, may not preserve information. It can be shown that if p is a prime integer, then the ring of integers, modulo p , forms a field. This finite field is called a *Galois field* and is denoted $GF(p)$.

Example The set of integers $\{0, 1, 2\}$ and the operations defined in Table A15.2 form the finite field $GF(3)$.

Table A15.2 Modulo-3 operations

+	0	1	2	·	0	1	2
0	0	1	2	0	0	0	0
1	1	2	0	1	0	1	2
2	2	0	1	2	0	2	1

Any Galois field with prime characteristic p contains exactly p^k elements, for some integer k . This field is denoted $GF(p^k)$. It can also be shown that, for

any finite field, there exists a prime integer p and a positive integer k such that the given field is equivalent to $GF(p^k)$.

In this chapter the fields were defined over $GF(p)$, where p is a prime. The theory and results obtained can be generalized to include linear machines defined over any finite field. It can be shown [17] that there exists an equivalence between a linear machine defined over any finite field and a linear machine defined over $GF(p)$. Consequently, any linear machine defined over any finite field can be synthesized by the techniques developed for machines defined over $GF(p)$, where p is a prime integer.

Appendix 15.2 The Euclidean algorithm

The Euclidean algorithm provides a procedure for obtaining the greatest common divisor of two polynomials over a field F .

Let $P(D)/Q(D)$ be a rational polynomial of the following form:

$$\frac{P(D)}{Q(D)} = \frac{a_0 + a_1 D + \cdots + a_m D^m}{b_0 + b_1 D + \cdots + b_n D^n},$$

where the degree of $P(D)$ is smaller than that of $Q(D)$. (The degree of a polynomial $P(D)$ is the greatest i such that $a_i \neq 0$.) The Euclidean algorithm is based on the result that every rational polynomial can be divided in a unique manner such that

$$Q(D) = q(D)P(D) + r(D).$$

When the remainder $r(D) = 0$, $P(D)$ is said to divide $Q(D)$. To find the greatest common divisor, we use successive division as follows:

$$\begin{aligned} Q(D) &= q_1(D)P(D) + r_1(D), \\ P(D) &= q_2(D)r_1(D) + r_2(D), \\ r_1(D) &= q_3(D)r_2(D) + r_3(D), \\ &\vdots \\ r_{i-2}(D) &= q_i(D)r_{i-1}(D). \end{aligned}$$

Then $r_{i-1}(D)$ is the greatest common divisor of $P(D)$ and $Q(D)$.

Example Determine the greatest common divisor for the polynomial

$$T(D) = \frac{P(D)}{Q(D)} = \frac{1 + D + D^4 + D^6}{D + D^3 + D^4 + D^6 + D^8 + D^9} \quad (\text{over } GF(2)).$$

Proceeding by successive division,

$$\begin{array}{r}
 D^6 + D^4 + D + 1 \overline{) \begin{array}{l} D^3 + D^2 + D \\ D^9 + D^8 + D^6 + D^4 + D^3 + D \\ \underline{D^9 + D^7 + D^4 + D^3} \\ D^8 + D^7 + D^6 + D \\ \underline{D^8 + D^6 + D^3 + D^2} \\ D^7 + D^3 + D^2 + D \\ \underline{D^7 + D^5 + D^2 + D} \\ D^5 + D^3 \end{array}} \leftarrow \text{determination of } r_1(D)
 \end{array}$$

$$\begin{array}{r}
 D^5 + D^3 \overline{) \begin{array}{l} D \\ D^6 + D^4 + D + 1 \\ \underline{D^6 + D^4} \\ D + 1 \end{array}} \leftarrow \text{determination of } r_2(D)
 \end{array}$$

$$\begin{array}{r}
 D^4 + D^3 \\
 D + 1 \overline{) \begin{array}{l} D^5 + D^3 \\ \underline{D^5 + D^4} \\ D^4 + D^3 \\ \underline{D^4 + D^3} \\ \underline{D^4 + D^3} \end{array}} \leftarrow r_3(D) = 0
 \end{array}$$

Since $r_3(D) = 0$, $r_2(D) = D + 1$ is the greatest common divisor. To find the reduced polynomial, it is necessary to divide $P(D)$ and $Q(D)$ by $D + 1$. This division yields

$$T(D) = \frac{1 + D^4 + D^5}{D + D^2 + D^4 + D^5 + D^8}.$$

Notes and references

Linear machines were first investigated by Huffman in 1956 [13]. This original work, which was restricted to inert machines, was later expanded by several people, notably Cohn [3, 4], Elspas [7], Friedland [8], Hartmanis [10], and Stern and Friedland [17]. The problem of identifying linear machines was treated by numerous authors, among them Brzozowski and Davis [2], Davis and Brzozowski [6] and Hartmanis [11]. The most general minimization and identification procedure is due to Cohn and Even [5], whose approach has been followed in this chapter. Other aspects of linear machines were studied by Booth [1], Pugsley [16], and Zierler [18]. The application of linear machines to error-correcting codes is due to Huffman [12] and Peterson [15]. A good collection of papers on linear machines is available in Kautz [14]. One of the best general treatments of linear machines can be found in the book by Gill [9].

- [1] Booth, T. L.: "An analytic representation of signals in sequential networks," in *Proc. Symp. Mathematical Theory of Automata*, vol. 12, pp. 301–340, Polytechnic Institute of Brooklyn, New York, 1963.
- [2] Brzozowski, J. A., and W. A. Davis: "On the linearity of autonomous sequential machines," *Trans. IEEE*, vol. EC-13, pp. 673–679, 1964.
- [3] Cohn, M.: "Controllability in linear sequential networks," *Trans. IRE*, vol. CT-9, pp. 74–78, 1962.
- [4] Cohn, M.: "Properties of linear machines," *J. Assoc. Computing Machinery*, vol. 11, pp. 296–301, 1964.
- [5] Cohn, M., and S. Even: "Identification and minimization of linear machines," *Trans. IEEE*, vol. EC-14, pp. 367–376, 1965.
- [6] Davis, W. A., and J. A. Brzozowski: "On the linearity of sequential machines," *Trans. IEEE*, vol. EC-15, pp. 21–29, 1966.
- [7] Elspas, B.: "The theory of autonomous linear sequential networks," *Trans. IRE*, vol. CT-6, pp. 45–60, 1959.
- [8] Friedland, B.: "Linear modular sequential circuits," *Trans. IRE*, vol. CT-6, pp. 61–68, 1959.
- [9] Gill, A.: *Linear Sequential Circuits*, McGraw-Hill, New York, 1967.
- [10] Hartmanis, J.: "Linear multivalued sequential coding networks," *Trans. IRE*, vol. CT-6, pp. 69–74, 1959.
- [11] Hartmanis, J.: "Two tests for the linearity of sequential machines," *Trans. IEEE*, vol. EC-14, pp. 781–786, 1965.
- [12] Huffman, D. A.: "A linear circuit viewpoint of error-correcting codes," *Trans. IRE*, vol. IT-2, pp. 20–28, 1956.
- [13] Huffman, D. A.: "The synthesis of linear sequential coding networks," in C. Cherry (ed.), *Information Theory*, pp. 77–95, Academic Press, New York, 1956.
- [14] Kautz, W. H. (ed.): *Linear Sequential Switching Circuits: Selected Technical Papers*, Holden-Day, 1965.
- [15] Peterson, W. W.: *Error-correcting Codes*, M.I.T. Press, Cambridge MA, 1961.
- [16] Pugsley, J. H.: "Sequential functions and linear sequential machines," *Trans. IEEE*, vol. EC-14, pp. 376–382, 1965.
- [17] Stern, T. E., and B. Friedland: "The linear modular sequential circuit generalized," *Trans. IRE*, vol. CT-8, pp. 79–80, 1961.
- [18] Zierler, N.: "Linear recurring sequences," *J. Soc. Ind. Appl. Math.*, vol. 7, pp. 31–48, 1959.

Problems

Problem 15.1. A *combinational linear circuit* is a circuit constructed only of modulo- p adders and multipliers. The block diagram in Fig. P15.1 represents a combinational linear circuit over $GF(2)$. The circuit outputs can be expressed as

$$z_a = x_a,$$

$$z_b = x_a + x_b,$$

$$z_c = x_b + x_c.$$

- (a) Show the circuit diagram.

(b) Find the output sequences in response to the following input sequences:

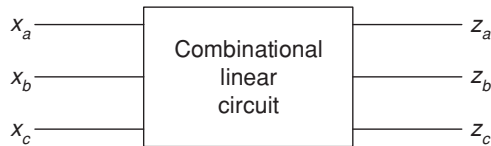
$$x_a : \quad 0 \quad 1 \quad 0 \quad 1 \quad 1 \quad 1 \quad 1 \quad 0 \quad 0 \quad 0 \quad 1 \quad 0 \quad 1 \quad 1$$

$$x_b : \quad 1 \quad 1 \quad 0 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1$$

$$x_c : \quad 0 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 1$$

(c) Design the inverse of this circuit; i.e., express the inputs as functions of the outputs and show the inverse circuit.

Fig. P15.1



Problem 15.2

(a) Determine the transfer function of the shift register shown in Fig. P15.2.

(b) Find its null sequence and show that it is maximal.

(c) Find the inverse machine.

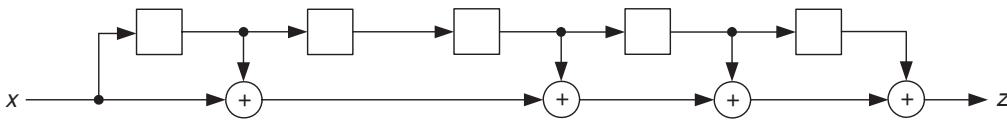


Fig. P15.2

Problem 15.3. For each of the following polynomials over $GF(2)$,

$$z_1 = x + D^3x + D^4x, \quad z_2 = x + D^2x + D^4x + D^5x:$$

(a) show the corresponding linear circuit and its inverse;

(b) find the null sequence and determine whether it is maximal;

(c) utilize the impulse response to determine the response of each circuit to the input sequence 000001101.

Problem 15.4. Show the state diagram of the linear machine whose transfer function is $T = 1 + D + D^3$.

Problem 15.5. Prove that the two circuits over $GF(3)$ of Fig. P15.5 are equivalent.

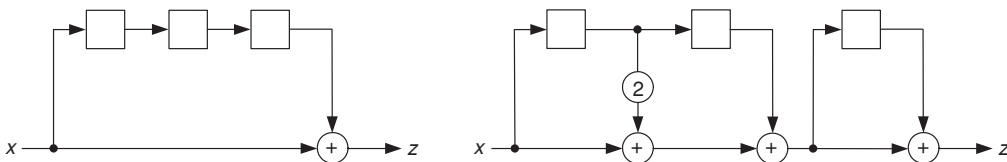


Fig. P15.5

Problem 15.6. Prove that the two circuits over $GF(16)$ of Fig. P15.6 have the same transfer functions. (Note that the use of feedback allows us in this case to construct a machine whose output symbol depends on input symbols three time units in the past, by using just a single delay element.)

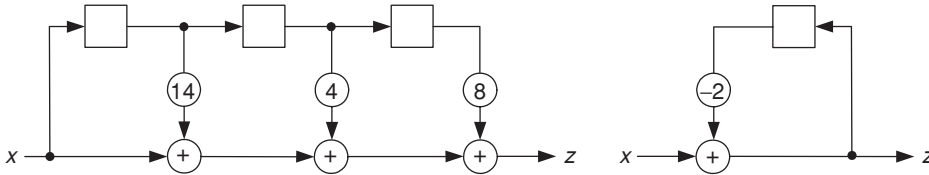


Fig. P15.6

Problem 15.7. Determine the null sequence of the linear machine over $GF(3)$ whose transfer function is $T = 2 + D^2 + 2D^3$. Prove that it is a maximal sequence.

Problem 15.8. Prove that the delay polynomial $T(D) = a_0 + a_1D + \cdots + a_kD^k$ has a linear inverse that decodes without a delay if and only if $T(D)$ has a nonzero constant term that is relatively prime to p .

Hint: Assume initially $a_0 = 1$. Expand $1/T(D)$ into the form

$$\frac{1}{T(D)} = \frac{1}{1 + \sum_1^n a_i D^i} = 1 - \sum_1^n a_i D^i + \left(\sum_1^n a_i D^i \right)^2 - \cdots$$

Problem 15.9. Figure P15.9 shows an inert linear machine over $GF(3)$. Prove that its transfer function is

$$T = \frac{z}{x} = \frac{2D + 2D^2 + D^3}{1 + D^2}.$$

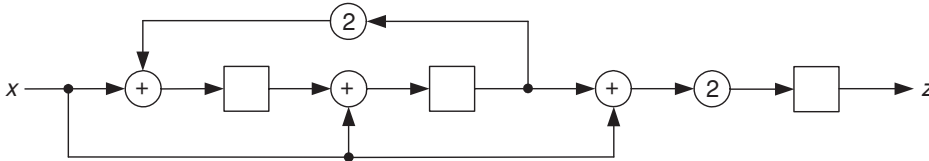


Fig. P15.9

Problem 15.10

(a) Prove that the transfer function of the inert linear machine of Fig. P15.10 is given by

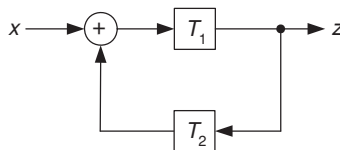
$$T = \frac{z}{x} = \frac{T_1}{1 - T_1 T_2},$$

where T_1 and T_2 are transfer functions of the individual submachines.

(b) Use the result of part (a) to find the transfer function of the machine in Fig. P15.9.

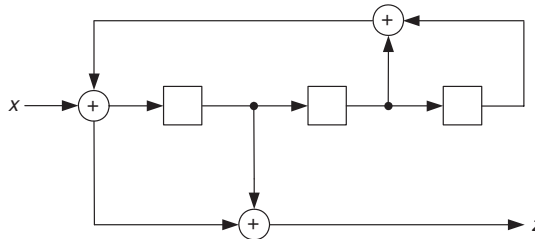
Hint: In part (b), determine first the direct paths through which the input signal can reach the output terminal.

Fig. P15.10



Problem 15.11

- (a) Determine the transfer function of the linear machine over $GF(2)$ shown in Fig. P15.11 and find its impulse response. Assume that it is initially inert.
- (b) Prove that its state table is isomorphic to Table P15.11.

Fig. P15.11**Table P15.11**

<i>PS</i>	<i>NS, z</i>	
	<i>x</i> = 0	<i>x</i> = 1
<i>A</i>	<i>A</i> , 0	<i>E</i> , 1
<i>B</i>	<i>E</i> , 1	<i>A</i> , 0
<i>C</i>	<i>F</i> , 1	<i>B</i> , 0
<i>D</i>	<i>B</i> , 0	<i>F</i> , 1
<i>E</i>	<i>C</i> , 1	<i>G</i> , 0
<i>F</i>	<i>G</i> , 0	<i>C</i> , 1
<i>G</i>	<i>H</i> , 0	<i>D</i> , 1
<i>H</i>	<i>D</i> , 1	<i>H</i> , 0

Problem 15.12. For each of the following transfer functions,

$$T_1 = \frac{1 + D^2}{1 + D + D^3} \quad \text{over } GF(2),$$

$$T_2 = \frac{D^2}{2D^2 + D + 1} \quad \text{over } GF(3),$$

- (a) show the corresponding network;
- (b) find its impulse response;
- (c) determine whether it is invertible and, if it is, show the inverse.

Problem 15.13. Given the following transfer function over $GF(2)$,

$$T = \frac{D^{10} + D^9 + D^8 + D^7 + D}{D^7 + D^4 + D^2 + D + 1},$$

- (a) determine by means of the Euclidean algorithm the greatest common divisor of the numerator and denominator, and simplify the function;
- (b) show a minimal chain realization, using no more than eight delay elements.

Problem 15.14. Show minimal realizations of the transfer function below and of its inverse.

$$T = \frac{1 + D + 2D^2 + D^3}{1 + D + D^3 + 2D^4} \quad \text{over } GF(3).$$

Problem 15.15. Design a four-dimensional linear machine over $GF(2)$ whose impulse response is

$$h = 1 \ 1 \ 1 \ 1 \ 1 \ 0 \ 0 \ 1 \ 0 \ 1 \ 1 \ 1 \ 0 \ 0 \ (1 \ 0 \ 1 \ 1 \ 1 \ 0 \ 0) \dots$$

(The sequence in parentheses repeats itself thereafter.)

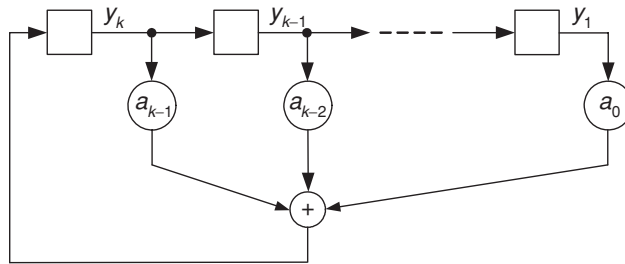
Problem 15.16. Show the linear circuit over $GF(2)$ whose characterizing matrices are

$$\mathbf{A} = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 1 & 1 \\ 0 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{C} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}, \quad \mathbf{D} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}.$$

Problem 15.17

- Find the characteristic matrix $\mathbf{A}$ that is realized by the internal circuit of Fig. P15.17.
- Determine the transpose of the matrix $\mathbf{A}$ in part (a), and show a circuit that realizes the transposed matrix.

Fig. P15.17



Problem 15.18

- Prove that a linear machine $\{\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}\}$ is μ -definite if and only if μ is the least integer such that $\mathbf{A}^\mu = 0$.
- Prove that if a k -dimensional linear machine is μ -definite then $\mu \leq k$.
Hint: See [4].

Problem 15.19

- Design the linear circuit over $GF(2)$ whose characterizing matrices are

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{bmatrix},$$

$$\mathbf{C} = \begin{bmatrix} 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 \end{bmatrix}, \quad \mathbf{D} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}.$$

- Minimize the machine of part (a), and show that it is independent of x_2 .

Problem 15.20

- (a) Minimize the linear machine over $GF(2)$ given by the following characterizing matrices:

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}, \quad \mathbf{C} = \begin{bmatrix} 1 & 0 & 1 & 0 \end{bmatrix}, \quad \mathbf{D} = [0].$$

- (b) For each state of the reduced machine, show the equivalent states of the original machine.

Problem 15.21

- (a) Design the linear circuit over $GF(2)$ whose characterizing matrices are

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{C} = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}, \quad \mathbf{D} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

- (b) Prove that no reduction in the machine dimension is possible but that the reduction procedure can be applied to obtain an equivalent machine $\{\mathbf{A}^*, \mathbf{B}^*, \mathbf{C}^*, \mathbf{D}^*\}$ that is realizable with a single modulo-2 adder.

Problem 15.22

- (a) Given a linear machine $L = \{\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}\}$ and a nonsingular matrix $\mathbf{G}$, prove that the state $\mathbf{y}$ of L is equivalent to the state $\bar{\mathbf{y}} = \mathbf{G}\mathbf{y}$ of $\bar{L}$, where $\bar{L}$ is the linear machine characterized by

$$\bar{\mathbf{A}} = \mathbf{G}\mathbf{A}\mathbf{G}^{-1}, \quad \bar{\mathbf{B}} = \mathbf{G}\mathbf{B}, \quad \bar{\mathbf{C}} = \mathbf{C}\mathbf{G}^{-1}, \quad \bar{\mathbf{D}} = \mathbf{D}.$$

- (b) Prove that the machines L and $\bar{L}$ are isomorphic.

Problem 15.23

- (a) Prove that, for all $t \geq 0$,

$$(\mathbf{A}^*)^t = \mathbf{T}\mathbf{A}^t\mathbf{R},$$

where $\mathbf{A}^*$ is the characteristic matrix of the reduced machine, defined in Eq. (15.23).

Hint: Prove the assertion for $t = 0$ and use induction on t .

- (b) Use the result of part (a) to prove that the diagnostic matrix $\mathbf{K}^*$ of the reduced machine is related to $\mathbf{K}$ by

$$\mathbf{K}^* = \mathbf{K}\mathbf{R}.$$

- (c) Prove that if $\mathbf{T}^*$ is the $r \times r$ matrix consisting of the first r linearly independent rows of $\mathbf{K}_r^*$ of a reduced linear machine then $\mathbf{T}^* = \mathbf{I}_r$, where $\mathbf{I}_r$ is the identity matrix.

Problem 15.24. A k -dimensional linear machine $\{\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}\}$ is said to be μ -controllable if for every pair of states S_i and S_j there is an input sequence of length exactly μ that takes the machine from state S_i to state S_j .

- (a) Prove that a k -dimensional machine L is μ -controllable if and only if the rank of $k \times \mu l$ matrix

$$\mathbf{G}_\mu = [\mathbf{A}^{\mu-1}\mathbf{B} \quad \mathbf{A}^{\mu-2}\mathbf{B} \quad \dots \quad \mathbf{A}\mathbf{B} \quad \mathbf{B}]$$

is k ; i.e., there are k linearly independent columns in $\mathbf{G}_\mu$.

(b) Determine whether the following machine over $GF(2)$ is μ -controllable:

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}.$$

Hint: Try the 3-controllable case first and show that $\mathbf{G}_3$ is singular.

Problem 15.25. For each machine in Table P15.25, determine whether it is linear and, if it is, show a linear realization.

Table P15.25

NS, z			NS								$z_1 z_2$			
PS	$x = 0$	$x = 1$	PS	00	01	11	10	00	01	11	10			
A	$A, 0$	$E, 1$	A	E	F	A	B	10	11	00	01			
B	$E, 1$	$A, 0$	B	G	H	C	D	11	10	01	00			
C	$F, 1$	$B, 0$	C	B	A	F	E	01	00	11	10			
D	$B, 0$	$F, 1$	D	D	C	H	G	00	01	10	11			
E	$C, 1$	$G, 0$	E	B	A	F	E	11	10	01	00			
F	$G, 0$	$C, 1$	F	D	C	H	G	10	11	00	01			
G	$H, 0$	$D, 1$	G	E	F	A	B	00	01	10	11			
H	$D, 1$	$H, 0$	H	G	H	C	D	01	00	11	10			

Problem 15.26. Test the machine of Table P15.26 for linearity. In particular, determine whether the state transitions are linear and the outputs are linear.

Table P15.26

NS, z		
PS	$x = 0$	$x = 1$
A	$A, 0$	$B, 0$
B	$C, 0$	$D, 0$
C	$A, 1$	$B, 1$
D	$C, 1$	$D, 0$

16

Finite-state recognizers

In this chapter we consider the characterization of finite-state machines and the sets of sequences that they accept. We investigate a number of generalized forms of finite-state machines and prove that these forms are equivalent, with respect to the sets of sequences that they accept, to the basic deterministic finite-state model. In Sections 16.2 and 16.3 we study the properties of nondeterministic state diagrams, called transition graphs, which will prove to be a useful tool in the study of regular expressions. Procedures are developed whereby any transition graph can be converted into a deterministic state diagram.

Section 16.4 presents the language of regular expressions, which provides a precise characterization of the sets of sequences accepted by finite-state machines. In the following two sections we prove that any finite-state machine can be characterized by a regular expression and that every regular expression can be realized by a finite-state machine. Finally, in Section 16.7 we will be concerned with a generalized form of finite-state machines known as two-way machines.

16.1 Deterministic recognizers

So far, we have regarded a finite-state machine as a *transducer* that *transforms* input sequences into output sequences. In this chapter we shall view a machine as a *recognizer* that *classifies* input strings into two classes, those that it accepts and those that it rejects. The set consisting of all the strings that a given machine accepts is said to be *recognized* by that machine.

The finite-state model that we shall use is shown in Fig. 16.1, where a *finite-state control* is coupled through a *head* to a finite linear sequence of squares, each containing a single symbol of the alphabet. Such a sequence of squares is called an (*input*) *tape*. Initially, the finite-state control is in the starting state, and the head scans the leftmost symbol of the string that appears on the tape. The head then scans the tape from left to right. In what is termed

Fig. 16.1 A finite-state recognizer.

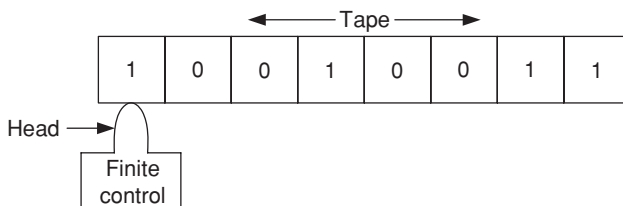
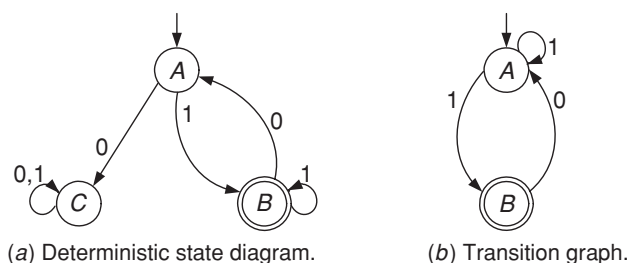


Fig. 16.2 Two ways of describing a string.



a *cycle of computation*, the machine starts in some state S_i , reads the symbol currently scanned by the head, shifts one square to the right, and then enters the state S_j .

Clearly, the concept of a head reading from left to right the symbols contained in a linear tape is equivalent to a string of input symbols entering the machine at successive times. In fact, the finite-state control is a Moore finite-state machine.¹ States whose assigned output symbol is 1 are referred to as *accepting* (or *terminal*) states while states whose assigned output symbol is 0 are called *rejecting* (or *nonterminal*) states. A string (or a tape) is *accepted* by a machine if and only if the state that the machine enters after having read the rightmost tape symbol is an accepting state. Otherwise the string is rejected. The set of strings recognized by a machine thus consists of all the input strings that take the machine from its starting state to an accepting state.

The machine of Fig. 16.1 can be described by a state diagram in which the starting state is marked by an incoming short arrow and the accepting states are indicated by double circles. For example, the state diagram of Fig. 16.2a describes a machine that accepts a string if and only if the string begins and ends with a 1 and every 0 in the string is preceded and followed by at least a single 1. The machine consists of three states, of which A is the starting state and B is an accepting state. Note that in general a starting state may also be an accepting state. In such a case, the machine is said to accept the null string.

¹ By allowing the head to write on the tape, while restricting its motion to left-to-right, we can generalize the model to include Mealy machines.

16.2 Transition graphs

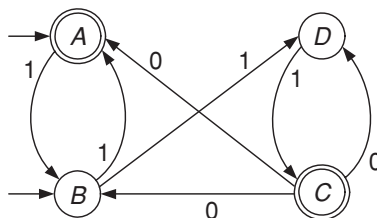
Because a state diagram describes a *deterministic* machine, the next-state transition must be determined *uniquely* by the present state and the currently scanned input symbol. No alternative behavior is allowed. Moreover, in a deterministic state diagram a transition must be specified for each input symbol. Consequently, a state diagram consists of a vertex for every state and a directed arc labeled α emanating from each vertex for every input symbol α . However, if our prime objective is to study and classify sets of sequences, some of these restrictions may be removed and different diagrams, called transition graphs, may prove more convenient.

Nondeterministic recognizers

A *transition graph* (or *transition system*) is a directed graph. It consists of a set of vertices labeled A, B, C , etc. and various directed arcs connecting them. At least one vertex is specified as a *starting vertex* and at least one is specified as an *accepting* (or *terminal*) vertex. The arcs are labeled with symbols from the (input) *alphabet* of the graph. If the graph contains an arc labeled α leading from vertex V_i to vertex V_j then V_j is said to be the α -*successor* of V_i . For a given input symbol α , a vertex may have one or more α -successors or none. Thus, for example, in the transition graph of Fig. 16.2b, vertex A has two 1-successors, namely A and B , but no 0-successor. A set of vertices S is said to be the α -successor of a set R if and only if every element of S is an α -successor of some element of R .

A sequence of directed arcs in a graph is referred to as a *path*. Every path is said to *describe* the string that consists of the symbols assigned to the arcs in the path. A string is accepted by a transition graph if it is described by at least one path that emanates from a starting vertex and terminates at an accepting vertex. Thus, for example, the string 1110 is accepted by the graph of Fig. 16.3, since it is described by a path that emanates from vertex A , passes through vertices B, D , and C , and terminates at vertex A . In the same manner, we find that the string 11011 is accepted by the graph, since it is described by a path that emanates from a starting vertex B , passes through D, C, B, D , and

Fig. 16.3 A transition graph.



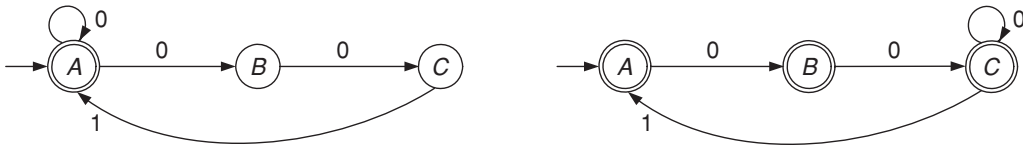


Fig. 16.4 Two equivalent transition graphs.

terminates at an accepting vertex C . However, the string 100, for example, is rejected since there is no path in the graph which describes it.

As in the case of state diagrams, the set of strings that are accepted by a transition graph is said to be *recognized* by the graph. For example, the transition graph of Fig. 16.2b recognizes the same set of strings as is recognized by the state diagram of Fig. 16.2a. If two or more graphs recognize the same set of strings then they are said to be *equivalent graphs*. Thus, the graphs in Fig. 16.4 are equivalent since each graph accepts a string if and only if each 1 in the string is preceded by at least two 0's.

Clearly, a state diagram is a special case of a transition graph and is, therefore, referred to as a *deterministic (transition) graph*. Other transition graphs are referred to as *nondeterministic (transition) graphs*. The two graphs in Fig. 16.2, for example, are equivalent although one is deterministic and the other is not. Because deterministic graphs describe the behavior of deterministic finite-state machines, we often regard nondeterministic graphs as describing the behavior of nondeterministic finite-state machines. It must, however, be emphasized that the notion of nondeterministic recognizers is useful for classifying sets of strings but should not be confused with the notion of realizable machines.

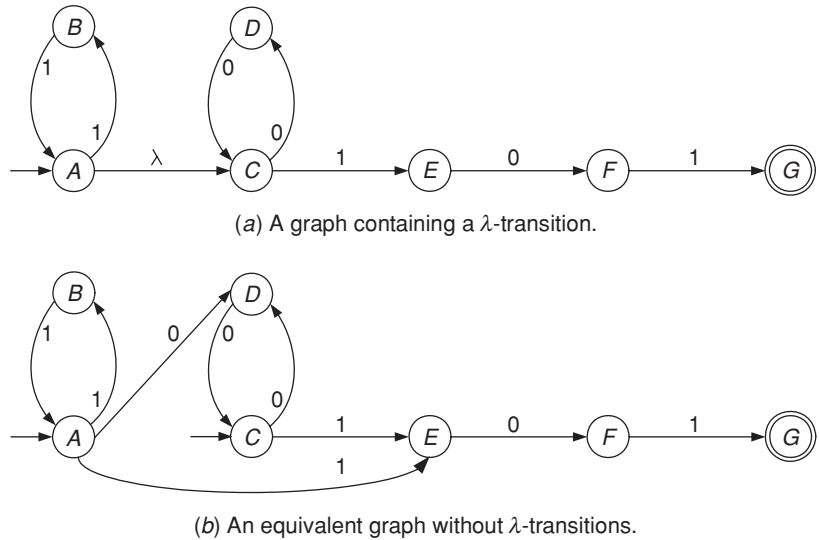
Graphs containing λ -transitions

Nondeterministic transition graphs can be generalized further by allowing transitions that are associated with a *null symbol* λ . Such transitions are referred to as λ -*transitions*, and they can occur when no input symbol is applied. When determining the string described by a path that contains arcs labeled λ , the λ -symbols are disregarded and deleted from the string.

The use of λ -transitions may sometimes simplify the transition graph by reducing the number of labeled arcs, as for the graph of Fig. 16.5a. This graph recognizes the set of strings that start with an even number of 1's, followed by an even number of 0's, and end up with substring 101. (Note that zero is considered as an even number.) Thus, for example, the strings 101, 11101, 110000101, and 00101 are accepted by the graph, while 110011101 and 0011101 are rejected.

It is a simple matter to convert a transition graph containing λ -transitions into an equivalent graph that contains no such transitions. A λ -transition from vertex V_1 to vertex V_2 of a given graph can always be replaced by a set of arcs emanating from V_1 and duplicating the transitions that emanate from V_2 . In addition, if V_1 is a starting vertex then V_2 must also be made a starting vertex. If V_2 is an accepting vertex then V_1 must also be made an accepting

Fig. 16.5 Elimination of λ -transition.



vertex. To remove the λ -transition from the graph of Fig. 16.5a it is necessary to duplicate the transitions from vertex C to vertices D and E by directing arcs, correspondingly labeled, from vertex A to vertices D and E. The equivalent graph that contains no λ -transition is shown in Fig. 16.5b.

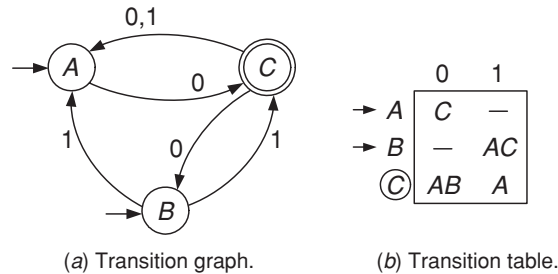
16.3 Converting nondeterministic into deterministic graphs

A natural question, which now arises, is whether a nondeterministic graph can recognize sets of strings that cannot be recognized by a deterministic graph. At first, one might suspect that the added flexibility of nondeterministic graphs increases their computational capabilities. However, as we shall now show, *there exists an effective procedure for converting a nondeterministic transition graph into an equivalent deterministic transition graph*. This leads to the conclusion that nondeterministic graphs and deterministic graphs have identical computational capabilities.

Introductory example

Consider the nondeterministic transition graph of Fig. 16.6a. A tabular description of the graph, called a *transition table*, is shown in Fig. 16.6b, where the starting vertices are indicated by the small arrows next to rows A and B, and the accepting vertex is indicated by a circle around the row heading C. The table entry in row V_i , column α , consists of the α -successors of vertex V_i .

Fig. 16.6 A nondeterministic graph to be converted to a deterministic one.



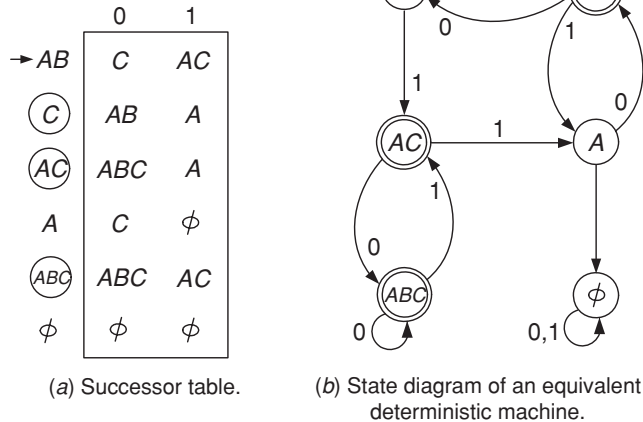
Suppose now that we wish to determine whether a given string $w = a_1a_2 \cdots a_k$ is accepted by the graph of Fig. 16.6a; that is, whether the graph contains a path that emanates from a starting vertex, terminates at an accepting vertex, and describes the string w . Since A and B are the starting vertices, any such path must include as its first arc an arc emanating from either A or B . Specifically, if the first symbol in w is a_1 then the first arc in the path can reach any vertex in the subset that consists of the a_1 -successors of $\{A, B\}$. Using similar reasoning, we find that the i th arc in a path that describes w must lead to a vertex contained in the subset which consists of the $a_1a_2 \cdots a_i$ -successors of $\{A, B\}$. If the final subset of vertices reached by the path contains an accepting vertex then the string w is accepted; otherwise, it is rejected.

For example, any path that describes string 0010 must start with the arc leading from vertex A to vertex C . Also, since the 0-successors of C are A and B , one of these vertices must be encountered next in the path describing the given string. In the same manner, since $\{AC\}$ is the 1-successor of $\{AB\}$, we find that the third arc in the path leads to either of the vertices A or C . The fourth symbol might lead to one of the vertices A , B , or C and, since vertex C is an accepting vertex, the string is accepted. A similar argument shows, for example, that the string 1100 is rejected, since it might lead to either vertex A or vertex B and neither vertex is an accepting vertex.

The foregoing example suggests a procedure for determining whether a specified string is accepted by a given graph. The procedure involves tracing the various paths that describe the given string and determining the sets of vertices that can be reached from the starting vertices by applying the symbols of the string. The procedure can be facilitated and applied to arbitrary strings by the use of a *successor table*, which lists all the subsets of vertices that are reachable from the starting vertices. The successor table for the graph of Fig. 16.6 is shown in Fig. 16.7a. Its column headings are symbols of the alphabet. The first row heading is the set of starting vertices, while the remaining row headings are subsets of vertices reachable from starting vertices. The entry in row Q , column α , is determined from the transition table and consists of the α -successor of $\{Q\}$.

The first row heading in Fig. 16.7a is AB , since A and B are the starting vertices. The entries in row AB are the 0- and 1-successors of $\{AB\}$, namely

Fig. 16.7 Deterministic form of the graph of Fig. 16.6.



$\{C\}$ and $\{AC\}$, respectively. The entries C and AC are now made row headings, their successors found, and so on. Since vertex A has no 1-successor, the 1-successor of row A must correspond to the set that contains no vertex of the transition graph. Such a set is referred to as the *empty*, or *null*, set and is denoted ϕ . Finally, the row headings of the rows C , AC , and ABC are circled to indicate that each of the sets $\{C\}$, $\{AC\}$, and $\{ABC\}$ contains the accepting vertex C of the original transition graph.

Proof of the conversion procedure

The graph in Fig. 16.7b is derived directly from the successor table. It is clearly a deterministic graph, since only one transition is allowed for each input symbol in its construction. To verify that this graph indeed accepts a given string if and only if that string is accepted by the corresponding nondeterministic graph, note that the last vertex of the deterministic graph reached by the string corresponds to the subset of vertices that can be reached by the same string in the nondeterministic graph. The string is accepted by the deterministic graph if and only if there is at least one path in the nondeterministic graph that results in the string being accepted, that is, if one vertex reachable by the string is an accepting vertex. The foregoing procedure, which is also known as *subset construction*, can be applied to any nondeterministic graph. Thus, we arrive at the following theorem.

Theorem 16.1 *Let S be a set of strings that can be recognized by a nondeterministic transition graph G_n . Then S can also be recognized by an equivalent deterministic graph G_d . Moreover, if G_n has p vertices then G_d will have at most 2^p vertices.*

Proof The existence of a deterministic graph G_d that is equivalent to the given nondeterministic graph G_n is guaranteed by the subset construction procedure developed above. If we denote the p vertices of G_n by $V_1, V_2, \dots, V_p$, then, by subset construction, the equivalent deterministic graph may have at most 2^p vertices labeled as follows: $\phi, V_1, V_2, \dots, V_p; V_1 V_2, V_1 V_3, \dots, V_2 V_3, \dots, V_{p-1} V_p; V_1 V_2 V_3, \dots, V_{p-2} V_{p-1} V_p; \dots; V_1 V_2 \dots V_p$. $\diamond$

Theorem 16.1 permits us to describe deterministic finite-state machines by means of nondeterministic transition graphs. Such descriptions will prove very convenient in the following discussion of regular expressions.

16.4 Regular expressions

In this chapter we are mainly concerned with the characterization of sets of strings recognized by finite automata. It is therefore appropriate to develop a compact language for describing such sets of strings. The language developed in this section is known as *type-3 language* or as the language of *regular expressions*.

Describing sets of strings

We shall first consider informally some sets recognized by simple graphs, leaving the formal presentation to subsequent sections. Consider the transition graph in Fig. 16.8a, which recognizes a set $\{101\}$ that contains just one string. We shall describe the set $\{101\}$ by the expression **101**.² Similarly, for an arbitrary alphabet $\{a, b\}$, the set $\{abba\}$ is described by the expression **abba**, and so on.

The graph in Fig. 16.8b recognizes the set of strings $\{01, 10\}$, that consists of two strings, 01 and 10. To represent such a set we employ the set union operation $+$, and express the set $\{01, 10\}$ as **01 + 10**. In the same manner, the set $\{abb, a, b, bba\}$ can be described by the expression **abb + a + b + bba**. Clearly, since the set union operation is commutative and associative, the union operation of expressions is also commutative and associative.

Next, consider the graph in Fig. 16.8c, which recognizes the set $\{0111, 1011\}$. This set can be described by the expression **0111 + 1011**. However, we observe that this graph recognizes precisely those strings that are recognized by the graph in Fig. 16.8b and which are followed immediately by the substring 11. In other words, the graph of Fig. 16.8c recognizes the set whose members are those strings formed by concatenating the strings in $\{01, 10\}$ and $\{11\}$. In general, the *concatenation* of two sets $\{P\}$ and $\{Q\}$ is the set

² In this chapter, boldface type is used to describe expressions.

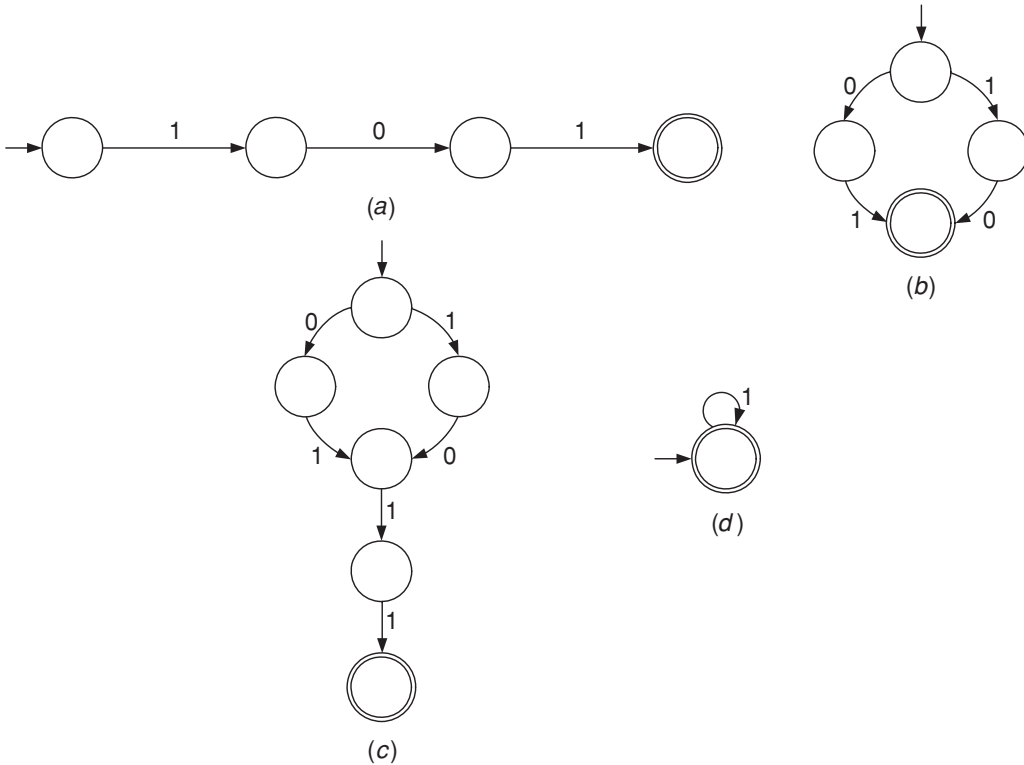


Fig. 16.8 Simple transition graphs.

consisting of strings formed by taking any string of $\{P\}$ and attaching to it any string of $\{Q\}$. The above set can thus be described by the *concatenation* of the two corresponding expressions **01** + **10** and **11**, i.e., **(01 + 10)11**. Clearly the concatenation operation is associative, that is, if **P**, **Q**, and **R** are expressions then **(PQ)R = P(QR)**, but it is not commutative, **PQ** $\neq$ **QP**. To simplify the notation, we can omit the parentheses and write the product **(PQ)R** as **PQR**.

The graph in Fig. 16.8d recognizes the set of strings whose members consist of an arbitrary number (possibly zero) of 1's, i.e., $\{\lambda, 1, 11, 111, 1111, \dots\}$. This set can be described by the infinite expression $\lambda + \mathbf{1} + \mathbf{11} + \mathbf{111} + \mathbf{1111} + \dots$ or, compactly, by $\mathbf{1}^*$, where

$$\mathbf{1}^* = \lambda + \mathbf{1} + \mathbf{11} + \mathbf{111} + \mathbf{1111} + \dots$$

The symbol $*$ is referred to as the *star* (or *closure*) operation. In general, $\mathbf{R}^*$ describes the set consisting of the null string λ and those strings that can be formed by concatenating a finite number of strings from $\{R\}$. For example, the expression **01(01)** * describes the set consisting of those strings that can be formed by concatenating one or more 01 substrings, that is,

$$\mathbf{01(01)}^* = \mathbf{01} + \mathbf{0101} + \mathbf{010101} + \mathbf{01010101} + \dots$$

For convenience, $\mathbf{RR}$ may be abbreviated as $\mathbf{R}^2$, $\mathbf{RRR}$ as $\mathbf{R}^3$, etc. Thus,

$$\mathbf{R}^* = \lambda + \mathbf{R} + \mathbf{R}^2 + \mathbf{R}^3 + \cdots$$

We are now able to describe some sets of strings on a given alphabet by means of the operations $+$, $\cdot$, $*$. For example, the set of strings on $\{0, 1\}$ beginning with a 0 and followed only by 1's can be described by $\mathbf{01}^*$ while the set of strings containing exactly two 1's can be described by $\mathbf{0*10*10*}$. An important expression is $(\mathbf{0} + \mathbf{1})^*$, which describes the set containing all the strings that can be formed on the binary alphabet; that is,

$$(\mathbf{0} + \mathbf{1})^* = \lambda + \mathbf{0} + \mathbf{1} + \mathbf{00} + \mathbf{01} + \mathbf{11} + \mathbf{10} + \mathbf{000} + \cdots$$

Thus, for example, the set of strings that begin with the substring 11 is described by the expression $\mathbf{11(0} + \mathbf{1)^*}$.

Example The transition graph of Fig. 16.9a accepts those strings that can be formed by concatenating a finite number of 01 and 10 substrings followed by a 11. Accordingly, it can be described by the expression $(\mathbf{01} + \mathbf{10})^*\mathbf{11}$. In a similar manner, the reader can verify that the set of strings recognized by the graph of Fig. 16.9b can be described by $(\mathbf{10}^*)^*$.

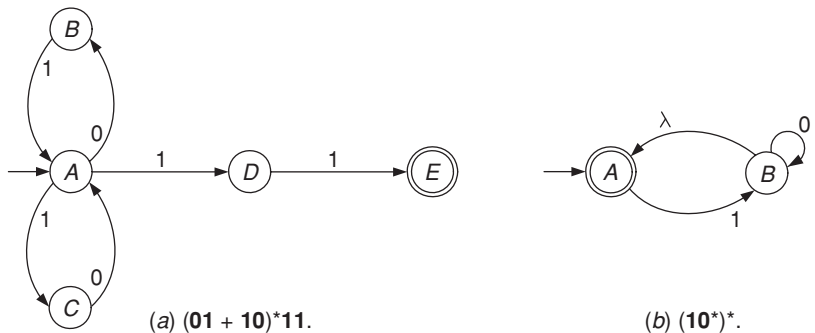


Fig. 16.9 Transition graphs and the sets of strings that they recognize.

We have thus shown that some sets of strings may be described by expressions formed of symbols from the alphabets of these sets and the operations union, concatenation, and star. We now formalize these ideas.

Definition and basic properties

Let $A = \{\alpha_1, \alpha_2, \dots, \alpha_p\}$ be a finite *alphabet*; then the class of *regular expressions over alphabet A* is defined recursively as follows.

1. Any single *symbol* $\alpha_1, \alpha_2, \dots, \alpha_p$ is a regular expression, as are the *null string* λ and the *empty set* ϕ .

Fig. 16.10 Recognizers for λ and ϕ .



(a) A graph accepting λ .

(b) A graph accepting ϕ .

2. If P and Q are regular expressions then so is their *concatenation* PQ and their *union* $P + Q$. If P is a regular expression then so is its *closure* P^* .
3. No other expressions are regular unless they can be generated in a *finite* number of applications of the above rules.

By convention, the precedence of the operations in decreasing order is $*$, $\cdot$, $+$.

At this point, it is appropriate to consider the significance of the expressions λ and ϕ . The expression λ describes the set that consists of just the null string. It can be recognized, for example, by the graph of Fig. 16.10a. Expression ϕ , however, describes the set that has no strings at all. In other words, ϕ describes the set recognized by a graph that accepts no strings, such as the graph shown in Fig. 16.10b. The reader may verify that each of the following identities, which involve the expressions ϕ and λ , exhibits different ways of describing the *same* sets of strings:

$$\phi + R = R, \quad (16.1)$$

$$\phi R = R\phi = \phi, \quad (16.2)$$

$$R\lambda = \lambda R = R, \quad (16.3)$$

$$\lambda^* = \lambda, \quad (16.4)$$

$$\phi^* = \lambda. \quad (16.5)$$

A set of strings that can be described by a regular expression is called a *regular set*. Not every set of strings is regular. For example, the set over the alphabet $\{0, 1\}$ that consists of k 0's (for all k), followed by a 1, followed in turn by k 0's is not regular, as will be proved later. This set can be described by the expression $010 + 00100 + 0001000 + \dots + 0^k 10^k + \dots$. However, such a description involves an infinite number of applications of the union operation. Consequently, it is not a regular expression. There are, however, certain infinite sums that are regular. For example, the set that consists of alternating 0's and 1's, starting and ending with a 1, i.e., $\{1, 101, 10101, 1010101, \dots\}$, can be described by the expression $1 + 101 + 10101 + \dots$, or $1(01)^*$, which is clearly regular.

Manipulating regular expressions

A regular set may be described by more than one regular expression. For example, the above set of alternating 0's and 1's can be described by the expression $1(01)^*$, as well as by $(10)^*1$. Two expressions that describe the same set of strings are said to be *equivalent*. Unfortunately, no straightforward methods are

available to determine whether two given expressions are equivalent. In certain cases, however, a regular expression can be converted into another equivalent expression by the use of simple identities. Some of these identities (whose proofs are left to the reader as an exercise) are listed as follows.

Let P , Q , and R be regular expressions; then

$$R + R = R, \quad (16.6)$$

$$PQ + PR = P(Q + R), \quad PQ + RQ = (P + R)Q, \quad (16.7)$$

$$R^*R^* = R^*, \quad (16.8)$$

$$RR^* = R^*R, \quad (16.9)$$

$$(R^*)^* = R^*, \quad (16.10)$$

$$\lambda + RR^* = R^*, \quad (16.11)$$

$$(PQ)^*P = P(QP)^*. \quad (16.12)$$

To prove the last identity, note that each of the expressions $(PQ)^*P$ and $P(QP)^*$ can be written in the form $P + PQP + PQQP + \dots$.

The set described by the expression $(P + Q)^*$ consists of all the strings that can be formed by concatenating P 's and Q 's, including the null string λ . It is easy to verify that the expression $(P^* + Q^*)^*$ describes the same set of strings, as does the expression $(P^*Q^*)^*$. Thus, we find that

$$(P + Q)^* = (P^*Q^*)^* = (P^* + Q^*)^*. \quad (16.13)$$

However, note that $(P + Q)^* \neq P^* + Q^*$.

The following identity will be proved in Section 16.5:

$$(P + Q)^* = P^*(QP^*)^* = (P^*Q)^*P^*. \quad (16.14)$$

This identity leads in turn to

$$\lambda + (P + Q)^*Q = (P^*Q)^*. \quad (16.15)$$

Indeed, by Eqs. (16.11) and (16.14),

$$\begin{aligned} (P^*Q)^* &= \lambda + (P^*Q)^*P^*Q \\ &= \lambda + (P + Q)^*Q. \end{aligned}$$

The preceding identities can sometimes be used to simplify regular expressions or demonstrate their equivalence, as illustrated in the following examples.

Example Prove that the set of strings in which every 0 is immediately followed by at least two 1's can be described by both R_1 and R_2 , where

$$\begin{aligned} R_1 &= \lambda + 1^*(011)^*(1^*(011)^*)^*, \\ R_2 &= (1 + 011)^*. \end{aligned}$$

We proceed as follows.

$$\begin{aligned} R_1 &= \lambda + 1^*(011)^*(1^*(011)^*)^* && \text{(by (16.11))} \\ &= (1^*(011)^*)^* && \text{(by (16.13))} \\ &= (1 + 011)^* = R_2. \end{aligned}$$

The reader can verify that R_2 indeed describes the set in question.

Example Prove the identity

$$(1 + 00^*1) + (1 + 00^*1)(0 + 10^*1)^*(0 + 10^*1) = 0^*1(0 + 10^*1)^*.$$

Consider the left-hand side:

$$\begin{aligned} &(1 + 00^*1) + (1 + 00^*1)(0 + 10^*1)^*(0 + 10^*1) \\ &= (1 + 00^*1)[\lambda + (0 + 10^*1)^*(0 + 10^*1)] \\ &= [(\lambda + 00^*)1][\lambda + (0 + 10^*1)^*(0 + 10^*1)] && \text{(by (16.11))} \\ &= 0^*1(0 + 10^*1)^*. \end{aligned}$$

In many situations, however, algebraic manipulations of regular expressions are extremely involved and thus are not a suitable tool for determining the equivalence of two regular expressions. As we shall see in the next section, perhaps the best approach is to convert the expressions in question into their equivalent state diagrams and to test the diagrams for equivalence by the techniques of Chapter 10. Other procedures for establishing the equivalence of regular expressions can be found in [3].

16.5 Transition graphs recognizing regular sets

We have already seen in several examples that transition graphs are capable of recognizing regular sets. We wish to show now that to every regular set there corresponds a transition graph (and hence a deterministic finite-state machine) that recognizes that set of strings.

Constructing the transition graphs

We now prove the following theorem.

Theorem 16.2 *Every regular expression R can be recognized by a transition graph.*

Proof We shall prove the theorem by constructing the required transition graph. The construction procedure is inductive on the total number of characters in R , where by a *character* we refer to an appearance of any of the expressions

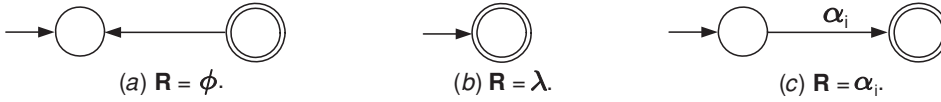


Fig. 16.11 Transition graphs recognizing elementary regular sets.

$\alpha_1, \alpha_2, \dots, \alpha_p, \lambda, \phi$ or the star operation $*$ in $\mathbf{R}$. For example, the number of characters in $\mathbf{R} = \lambda + (\mathbf{1}^* \mathbf{0})^* \mathbf{1}^*$ is seven.

Basis Let the number of characters in $\mathbf{R}$ be one. Then $\mathbf{R}$ must be either ϕ , λ , or a symbol, say α_i , from the alphabet. The graphs in Fig. 16.11 recognize these regular sets.³

Induction step Assume the theorem is true for expressions with n or fewer characters. We now show that it must also be true for any expression $\mathbf{R}$ having $n + 1$ characters. The expression $\mathbf{R}$ must be in one of the following three forms:

1. $\mathbf{R} = \mathbf{P} + \mathbf{Q}$,
2. $\mathbf{R} = \mathbf{PQ}$,
3. $\mathbf{R} = \mathbf{P}^*$,

where $\mathbf{P}$ and $\mathbf{Q}$ are each expressions having n or fewer characters. According to the induction hypothesis, the sets $\mathbf{P}$ and $\mathbf{Q}$ can be recognized by transition graphs, which we shall denote G and H , respectively, as shown in Fig. 16.12a. (Note that each graph in Fig. 16.12 contains just one starting and one accepting vertex.)

The set described by $\mathbf{P} + \mathbf{Q}$ can be recognized by a transition graph composed of G and H , as shown in Fig. 16.12b. The set described by $\mathbf{PQ}$ can be recognized by a transition graph constructed in the following manner. Coalesce the accepting vertex of G with the starting vertex of H and regard the combined vertex as one that is neither starting nor accepting. The resulting graph is shown in Fig. 16.12c. The starting vertices of this graph are the starting vertices of G , while the accepting vertices are those of H . Clearly, this graph will accept a string if and only if that string belongs to $\mathbf{R} = \mathbf{PQ}$. Finally, to recognize the set $\mathbf{P}^*$, construct the graph of Fig. 16.12d. The graphs in Fig. 16.12, which are composed of G and H , are referred to as *composite graphs*.

Since every regular set can be described by an expression obtained by a finite number of applications of operations $+$, $\cdot$, $*$ on an alphabet $\{\alpha_1, \alpha_2, \dots, \alpha_p\}$, ϕ and λ , the theorem is proved. $\diamond$

The foregoing proof makes it possible to state an upper bound on the number of vertices in a graph that recognizes a given regular expression $\mathbf{R}$. Every graph clearly contains one starting and one accepting vertex. Subexpressions connected by the $+$ operation yield a composite graph that has as many vertices as the sum of vertices in the graphs that recognize individual subexpressions.

³ Although there is a distinction between regular expressions and the sets that they describe, it is customary to speak of the regular set $\mathbf{R}$ as the set that can be described by the expression $\mathbf{R}$.

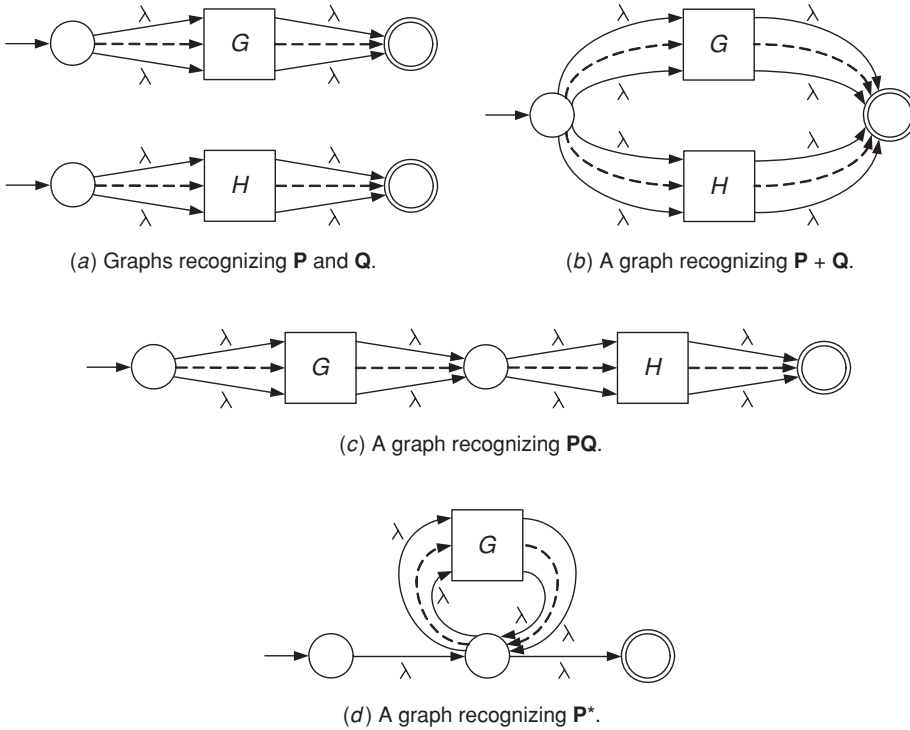


Fig. 16.12 Construction of composite graphs.

Two subexpressions connected by the concatenation operation add a new vertex to the composite graph, and similarly for the closure operation $*$. By induction on the number of vertices, we find that the number of vertices v in a graph that recognizes the given expression R need not exceed

$$v = 2 + \text{number of concatenations} + \text{number of stars}.$$

Theorem 16.2 provides us with a procedure for constructing a transition graph that recognizes a given regular expression R . Converting the graph to a deterministic form yields a state diagram of a finite-state machine that recognizes the set R .

Example Consider the regular expression $R = (0 + 1(01)^*)^*$. Since it is of the form P^* , where $P = 0 + 1(01)^*$, it is recognized by the graph of Fig. 16.13a. We now observe that $P = 0 + Q$, where $Q = 1(01)^*$, and the resulting graph is shown in Fig. 16.13b. The subexpression Q can be decomposed into $Q = ST$, where $S = 1$ and $T = (01)^*$. This yields the graph of Fig. 16.13c. The process is continued in a similar manner until each subexpression consists of only a single symbol. The final transition graph that

recognizes $\mathbf{R}$ is shown in Fig. 16.13*d*. Note that the number of vertices in the graph is six, in agreement with the value of v derived above.

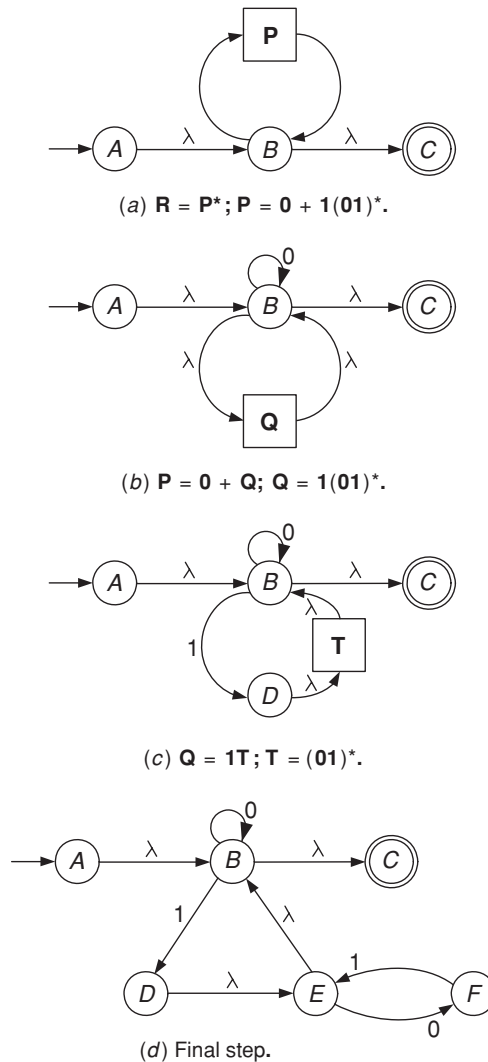
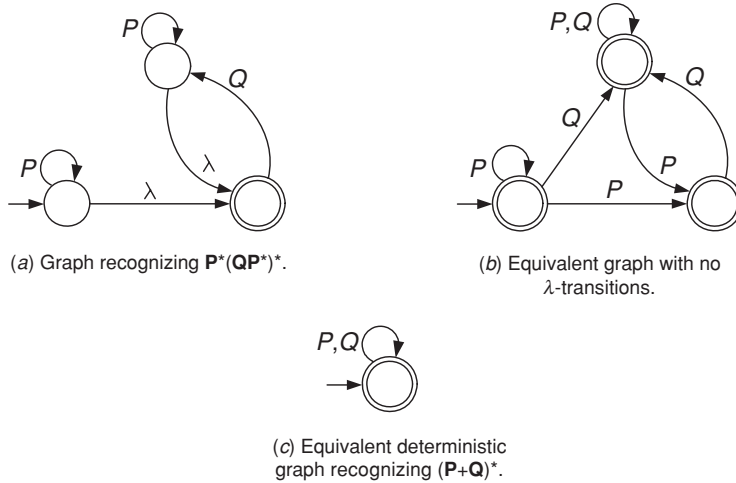


Fig. 16.13 Construction of a transition graph recognizing $\mathbf{R} = (\mathbf{0} + \mathbf{1}(\mathbf{01})^*)^*$.

We can now prove the first identity in Eq. (16.14) by demonstrating that the expressions $(\mathbf{P} + \mathbf{Q})^*$ and $\mathbf{P}^*(\mathbf{QP}^*)^*$ can be recognized by equivalent transition graphs. The graph in Fig. 16.14*a* recognizes the set described by $\mathbf{P}^*(\mathbf{QP}^*)^*$. Removal of the λ -transitions results in the graph of Fig. 16.14*b*, which can be converted to the deterministic graph of Fig. 16.14*c*. Clearly this graph recognizes set $(\mathbf{P} + \mathbf{Q})^*$, and thus the two expressions are equivalent. By Eq. (16.12), we obtain $\mathbf{P}^*(\mathbf{QP}^*)^* = (\mathbf{P}^*\mathbf{Q})^*\mathbf{P}^*$, which proves the second identity.

Fig. 16.14 Illustration of the proof that $P^*(QP^*)^* = (P + Q)^*$.



Informal techniques

In practice, in many cases it is possible to construct transition graphs from their corresponding regular expressions in a straightforward manner, without resorting to the above induction procedure.

Example Construct a graph that recognizes the regular set

$$P = (01 + (11 + 0)1^*0)11.$$

As an introduction, we shall construct a graph that recognizes the subexpression $Q = (11 + 0)1^*0$. Every string in Q starts with one of the substrings 11 and 0, followed by an arbitrary number of 1's, and ends with a 0. The graph of Fig. 16.15 clearly recognizes just this set of strings. The subexpressions 11 and 0 are represented by parallel paths between the vertices A and C , while 1^* corresponds to a self-loop around vertex C . To ensure that a string is accepted only if it ends with a 0, an arc labeled 0 leads from vertex C to accepting vertex D .

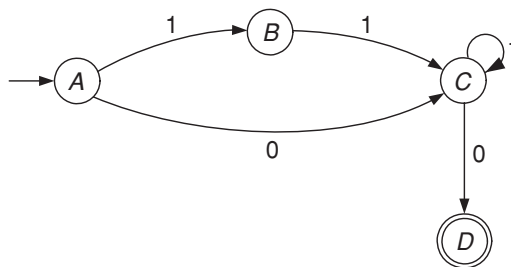


Fig. 16.15 A graph recognizing $Q = (11 + 0)1^*0$.

Now consider expression **P**. The graph that recognizes **P** is constructed in such a way that paths are provided for strings from the sets **01** and **(11 + 0)1*0**, followed by a string from the set **11**. One such possible graph is shown in Fig. 16.16.

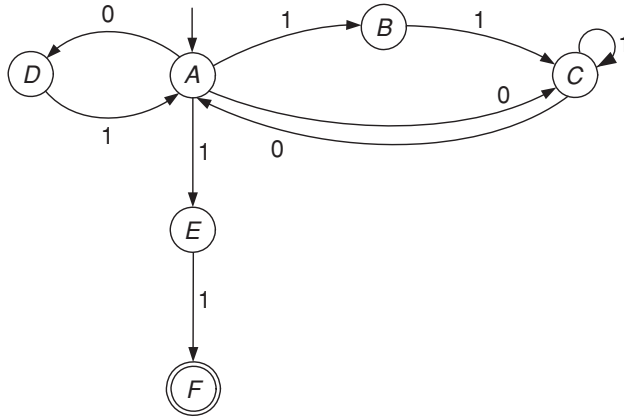


Fig. 16.16 A graph recognizing $\mathbf{P} = (\mathbf{01} + (\mathbf{11} + \mathbf{0})\mathbf{1}^*\mathbf{0})^*\mathbf{11}$.

In a number of cases it is convenient to use λ -transitions to preserve the order in which substrings appear. As an example, consider the expression $\mathbf{R} = (\mathbf{11})^*(\mathbf{00})^*\mathbf{101}$. In this expression, substrings from $(\mathbf{00})^*$ must follow substrings from $(\mathbf{11})^*$. One way of ensuring that this order is preserved is by using a λ -transition, as shown in Fig. 16.5a. This graph accepts only those strings that start with a substring from $(\mathbf{11})^*$, continue with a substring from $(\mathbf{00})^*$, and end with the substring 101.

Example Construct a transition graph that recognizes the set

$$\mathbf{R} = (\mathbf{1}(\mathbf{00})^*\mathbf{1} + \mathbf{01}^*\mathbf{0})^*.$$

We begin by setting up paths for the subexpressions $\mathbf{1}(\mathbf{00})^*\mathbf{1}$ and $\mathbf{01}^*\mathbf{0}$, as shown in Fig. 16.17a. Vertex **A** is the starting vertex, while **A**, **C**, and **F** are accepting vertices. To complete the graph, an arc labeled α_i is drawn from vertex V_j to vertex V_k if and only if a sequence leading from the starting vertex to V_j that is followed by α_i and then by a sequence that emanates from V_k to an accepting vertex is an acceptable sequence. Accordingly, for example, an arc labeled 0 is drawn from **F** to **B** since 1100 is an acceptable sequence. The graph is completed in a similar manner, as shown in Fig. 16.17b.

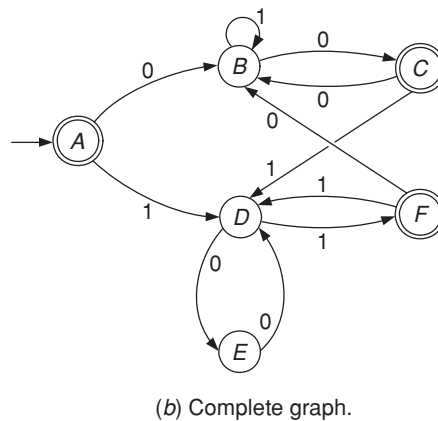
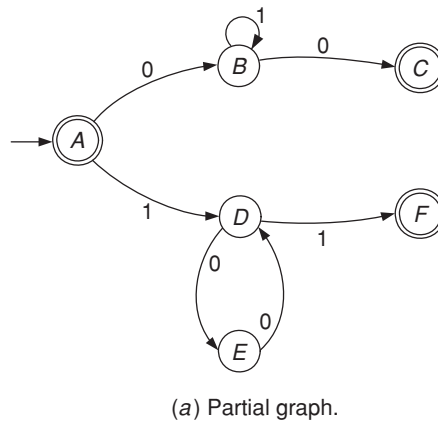


Fig. 16.17 Transition graph recognizing $R = (1(00)^*1 + 01^*0)^*$.

In conclusion, we have established that every regular set can be recognized by a finite-state machine. Moreover, there is a routine procedure for determining the machine that recognizes a given regular set. This procedure involves the use of nondeterministic transition graphs, which can later be converted into the equivalent deterministic graphs. Other methods, however, are available [6] that provide a state-diagram description of the machine directly, without the need to resort to nondeterministic graphs.

16.6 Regular sets corresponding to transition graphs

We now consider the problem of deriving regular expressions that describe specified transition graphs. Specifically, we shall show that the set of strings that can be recognized by a transition graph (and hence a finite-state machine) is a regular set.

Proof of uniqueness

Before proceeding with our main topic, we shall establish the following theorem.

Theorem 16.3 *Let $\mathbf{Q}$, $\mathbf{P}$, and $\mathbf{R}$ be regular expressions on a finite alphabet. Then, if $\mathbf{P}$ does not contain λ , the equation*

$$\mathbf{R} = \mathbf{Q} + \mathbf{R}\mathbf{P} \quad (16.16)$$

has a unique solution given by

$$\mathbf{R} = \mathbf{Q}\mathbf{P}^*. \quad (16.17)$$

Proof Clearly, $\mathbf{R} = \mathbf{Q}\mathbf{P}^*$ is a solution to the equation $\mathbf{R} = \mathbf{Q} + \mathbf{R}\mathbf{P}$, since (by substitution and Eq. (16.11))

$$\mathbf{R} = \mathbf{Q} + \mathbf{R}\mathbf{P} = \mathbf{Q} + \mathbf{Q}\mathbf{P}^*\mathbf{P} = \mathbf{Q}(\lambda + \mathbf{P}^*\mathbf{P}) = \mathbf{Q}\mathbf{P}^*.$$

To prove uniqueness, make the expansion

$$\begin{aligned} \mathbf{R} &= \mathbf{Q} + \mathbf{R}\mathbf{P} \\ &= \mathbf{Q} + (\mathbf{Q} + \mathbf{R}\mathbf{P})\mathbf{P} = \mathbf{Q} + \mathbf{Q}\mathbf{P} + \mathbf{R}\mathbf{P}^2 \\ &= \mathbf{Q} + \mathbf{Q}\mathbf{P} + (\mathbf{Q} + \mathbf{R}\mathbf{P})\mathbf{P}^2 = \mathbf{Q} + \mathbf{Q}\mathbf{P} + \mathbf{Q}\mathbf{P}^2 + \mathbf{R}\mathbf{P}^3 \\ &\vdots \\ &= \mathbf{Q}(\lambda + \mathbf{P} + \mathbf{P}^2 + \cdots + \mathbf{P}^{i-1} + \mathbf{P}^i) + \mathbf{R}\mathbf{P}^{i+1}, \end{aligned} \quad (16.18)$$

where i is any arbitrary integer. Choose some string w in $\mathbf{R}$, suppose that the length of w is k , and then substitute $i = k$ into Eq. (16.18):

$$\mathbf{R} = \mathbf{Q}(\lambda + \mathbf{P} + \mathbf{P}^2 + \cdots + \mathbf{P}^k) + \mathbf{R}\mathbf{P}^{k+1}.$$

Since $\mathbf{P}$ does not contain λ , the length of the shortest string in the set $\mathbf{R}\mathbf{P}^{k+1}$ is at least $k + 1$. Consequently, w is not contained in $\mathbf{R}\mathbf{P}^{k+1}$, but is contained in $\mathbf{Q}(\lambda + \mathbf{P} + \mathbf{P}^2 + \cdots + \mathbf{P}^k)$. However, since $\mathbf{Q}(\lambda + \mathbf{P} + \mathbf{P}^2 + \cdots + \mathbf{P}^k)$ is contained in $\mathbf{Q}\mathbf{P}^*$, w is contained in $\mathbf{Q}\mathbf{P}^*$.

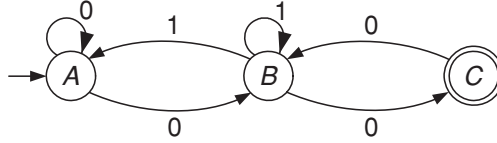
To prove the converse, suppose that w is a string in $\mathbf{Q}\mathbf{P}^*$. Then there exists some integer k such that w is in $\mathbf{Q}\mathbf{P}^k$. This, in turn, implies that w is contained in $\mathbf{Q}(\lambda + \mathbf{P} + \mathbf{P}^2 + \cdots + \mathbf{P}^k)$ and hence in $\mathbf{R} = \mathbf{Q} + \mathbf{R}\mathbf{P}$. $\diamond$

In an analogous manner, we can show that if $\mathbf{P}$ does not contain λ then $\mathbf{R} = \mathbf{P}^*\mathbf{Q}$ is the unique solution to the equation $\mathbf{R} = \mathbf{Q} + \mathbf{P}\mathbf{R}$. Note that if $\mathbf{P}$ contains λ , the solution of Eq. (16.16) is not unique. If $\mathbf{P} = \phi$ then $\mathbf{R} = \mathbf{Q}$.

*Systems of equations

Consider the transition graph of Fig. 16.18, whose starting vertex is A and accepting vertex C . The set of strings recognized by this graph consists of all the strings that can be described by paths emanating from vertex A and

Fig. 16.18 A transition graph to be analyzed.



terminating at vertex C . However, since vertex C can be reached only through vertex B , each of these strings must end with a 0 and have as prefix a string leading from A to B . Let us denote the set of strings leading from A to B by $\mathbf{B}$ and the set of strings that take the graph from A to C by $\mathbf{C}$. Set $\mathbf{C}$ can then be expressed as $\mathbf{C} = \mathbf{B0}$.

Next consider set $\mathbf{A}$, which consists of exactly those strings that take the graph from vertex A to itself. Vertex A can be reached from B with a 1, from A with a 0, and with the null string λ . Thus, $\mathbf{A}$ can be expressed as $\mathbf{A} = \lambda + \mathbf{A0} + \mathbf{B1}$. Finally, vertex B can be reached from A with a 0, from B with a 1, and from C with a 0. As a result, we obtain the equation $\mathbf{B} = \mathbf{A0} + \mathbf{B1} + \mathbf{C0}$.

The foregoing analysis yields a system of three simultaneous equations which characterize the sets of strings that take the graph from its starting vertex to each of its vertices. In Theorem 16.4 we shall prove that each of these sets of strings is regular, i.e.,

$$\mathbf{A} = \lambda + \mathbf{A0} + \mathbf{B1}, \quad (16.19)$$

$$\mathbf{B} = \mathbf{A0} + \mathbf{B1} + \mathbf{C0}, \quad (16.20)$$

$$\mathbf{C} = \mathbf{B0}. \quad (16.21)$$

These equations can now be solved for the variables $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C}$. Substituting Eq. (16.21) for $\mathbf{C}$ into Eq. (16.20) yields

$$\mathbf{B} = \mathbf{A0} + \mathbf{B1} + \mathbf{B00} = \mathbf{A0} + \mathbf{B(1 + 00)}. \quad (16.22)$$

Equation (16.22) is now of the form of Eq. (16.16),

$$\mathbf{R} = \mathbf{Q} + \mathbf{RP},$$

and its solution is given by Eq. (16.17), i.e.,

$$\mathbf{R} = \mathbf{QP}^*.$$

Applying Eq. (16.17) to Eq. (16.22), we obtain

$$\mathbf{B} = \mathbf{A0(1 + 00)}^*. \quad (16.23)$$

Now $\mathbf{B}$ can be substituted into Eq. (16.19) to give

$$\mathbf{A} = \lambda + \mathbf{A0} + \mathbf{A0(1 + 00)}^* \mathbf{1} = \lambda + \mathbf{A(0 + 0(1 + 00)}^* \mathbf{1)}. \quad (16.24)$$

Equation (16.24) is again of the general form of Eq. (16.16) and, thus, has the solution

$$\mathbf{A} = \lambda(\mathbf{0 + 0(1 + 00)}^* \mathbf{1})^* = (\mathbf{0 + 0(1 + 00)}^* \mathbf{1})^*. \quad (16.25)$$

Since the set recognized by the graph is given by $\mathbf{C}$, we want to find a solution for this variable. Substituting Eq. (16.25) for $\mathbf{A}$ into Eq. (16.23), we obtain a solution for $\mathbf{B}$ that, in turn, may be substituted into Eq. (16.21) to yield the solution for $\mathbf{C}$, i.e.,

$$\mathbf{B} = (\mathbf{0} + \mathbf{0}(\mathbf{1} + \mathbf{00})^*\mathbf{1})^*\mathbf{0}(\mathbf{1} + \mathbf{00})^*, \quad (16.26)$$

$$\mathbf{C} = (\mathbf{0} + \mathbf{0}(\mathbf{1} + \mathbf{00})^*\mathbf{1})^*\mathbf{0}(\mathbf{1} + \mathbf{00})^*\mathbf{0}. \quad (16.27)$$

The above procedure can now be applied to find a system of simultaneous equations for any transition graph that contains no λ -transitions and has a single starting vertex. (Recall that every transition graph can be converted to an equivalent graph with no λ -transitions and just one starting vertex.) Suppose that V_1 is the starting vertex in a graph containing n vertices, $V_1, V_2, \dots, V_n$. Let $\mathbf{V}_i$ denote the set of strings that take the graph from V_1 to V_i , and let α_{ij} denote the set of strings that take the graph from vertex V_i to vertex V_j without going through any other vertex; $\alpha_{ij} = \phi$ if no direct transition exists from V_i to V_j . Then we arrive at the following equations:

$$\begin{aligned} \mathbf{V}_1 &= \mathbf{V}_1\alpha_{11} + \mathbf{V}_2\alpha_{21} + \dots + \mathbf{V}_n\alpha_{n1} + \lambda, \\ \mathbf{V}_2 &= \mathbf{V}_1\alpha_{12} + \mathbf{V}_2\alpha_{22} + \dots + \mathbf{V}_n\alpha_{n2}, \\ &\vdots \\ \mathbf{V}_n &= \mathbf{V}_1\alpha_{1n} + \mathbf{V}_2\alpha_{2n} + \dots + \mathbf{V}_n\alpha_{nn}. \end{aligned} \quad (16.28)$$

This system of equations can now be solved for $\mathbf{V}_1, \mathbf{V}_2, \dots, \mathbf{V}_n$ by repeated substitution and successive applications of Eq. (16.17) in the following manner. Whenever an equation is of the form $\mathbf{V}_i = \mathbf{V}_j\alpha_{ji} + \mathbf{V}_k\alpha_{ki}$ or $\mathbf{V}_i = \mathbf{V}_j\alpha_{ji} + \mathbf{V}_k\alpha_{ki} + \lambda$, where $i \neq j \neq k$, then $\mathbf{V}_i$ can be substituted into all other equations to yield a system with fewer equations and unknowns. Whenever an equation has the form $\mathbf{V}_i = \mathbf{V}_i\alpha_{ii} + \mathbf{V}_j\alpha_{ji}$ (plus λ if appropriate), then Eq. (16.17) can be applied to yield $\mathbf{V}_i = \mathbf{V}_j\alpha_{ji}(\alpha_{ii})^*$, which can now be substituted for $\mathbf{V}_i$ in the other equations. Note that, since the graph is assumed to contain no λ -transitions, the condition in Theorem 16.3 that α_{ii} should not contain λ can always be met. This procedure will finally lead to a single equation in one variable. This variable can in turn be determined by another application of Eq. (16.17).

The set of strings recognized by a given graph can be described by the union of the $\mathbf{V}$'s that correspond to accepting vertices. For example, if vertices B and C in the graph of Fig. 16.18 were accepting vertices then the set of strings recognized by the graph could be described by $\mathbf{B} + \mathbf{C} = (\mathbf{0} + \mathbf{0}(\mathbf{1} + \mathbf{00})^*\mathbf{1})^*\mathbf{0}(\mathbf{1} + \mathbf{00})^*(\lambda + \mathbf{0})$.

Clearly, any system of equations of the form Eq. (16.28) can be uniquely solved by the procedure just outlined, provided that we prove that each of the $\mathbf{V}_i$'s and α_{ij} 's is a regular expression. This proof is given in the following theorem.

Theorem 16.4 *The set of strings that take a finite-state machine M from an arbitrary state S_i to another state S_j is a regular set.*

Proof Let Q be any subset of the states of M containing both S_i and S_j , and let R_{ij}^Q denote the set of strings that take the machine from state S_i to state S_j without passing through any state that is outside Q . Since Q may consist of all the states in M , the theorem will be proved if we can show that R_{ij}^Q is regular. The proof will be by induction on the number of states in Q .

Basis Suppose that Q consists of just a single state, which we shall call S_i . Then the set of strings that take S_i into itself without passing through any other state consists of only a finite number of single input symbols. Since by definition each such input symbol is regular, the above set of strings is regular. The corresponding regular expression will be denoted $\mathbf{T}_{ii}$.

Induction step Assume that R_{ij}^Q is regular for all subsets of states containing m or fewer states. Thus, R_{ij}^Q can be described by the regular expression $\mathbf{R}_{ij}^Q$. We shall now prove that the set of strings R_{ij}^P is also regular, where P is a set containing $m + 1$ states, including the states S_i and S_j . Suppose now that we remove state S_i from P . The resulting subset consists of only m states and will be referred to as Q ; the theorem is assumed to hold for this subset.

Consider a string from R_{ij}^P . In general, it will cause the machine to go through state transitions as follows:

$$S_i, S_t, \dots, S_u, S_i, \dots, S_i, \dots, S_j$$

where the ellipses correspond to transitions within set Q and therefore do not contain occurrences of S_i . The substrings that take the machine from S_i and back into S_i may consist of either single input symbols from the regular set $\mathbf{T}_{ii}$ or of sequences of symbols that take the machine from S_i through some states, say $S_t, \dots, S_u$, and back into S_i . Such an input sequence actually consists of a single symbol, denoted T_{it} , that takes M from S_i to S_t followed by a sequence from $\mathbf{R}_{tu}^Q$ and ending with a symbol T_{ui} that returns M to S_i . Each of the symbols T_{it} and T_{ui} is clearly regular and, consequently, the set of strings that take M from S_i into S_i can be described by the regular expression

$$\mathbf{T}_{ii} + \sum_{tu} \mathbf{T}_{it} \mathbf{R}_{tu}^Q \mathbf{T}_{ui},$$

where the sum is taken over all possible pairs of states in Q . In addition, since the machine can be taken an arbitrary number of times from S_i through states in Q and back into S_i , the set of corresponding strings can be described by the regular expression

$$\left(\mathbf{T}_{ii} + \sum_{tu} \mathbf{T}_{it} \mathbf{R}_{tu}^Q \mathbf{T}_{ui} \right)^*$$

This set of strings is followed by the set of substrings that take the machine from S_i into S_j . This latter set of substrings consists of all single symbols T_{ij} that take the machine from S_i to S_j and all other strings that take the machine from S_i to S_j via certain states $S_t, \dots, S_u$. Clearly, this set can be described by the regular expression

$$\mathbf{T}_{ij} + \sum_{tu} \mathbf{T}_{it} \mathbf{R}_{tu}^Q \mathbf{T}_{uj}.$$

Consequently, the set of strings R_{ij}^P is regular and can be described by the expression

$$\mathbf{R}_{ij}^P = \left(\mathbf{T}_{ii} + \sum_{tu} \mathbf{T}_{it} \mathbf{R}_{tu}^Q \mathbf{T}_{ui} \right)^* \left(\mathbf{T}_{ij} + \sum_{tu} \mathbf{T}_{it} \mathbf{R}_{tu}^Q \mathbf{T}_{uj} \right).$$

◇

Combining Theorems 16.2 and 16.4, we obtain the following general result, which is known as Kleene's theorem.

- A finite-state machine recognizes a set of strings if and only if it is a regular set.

Applications

The correspondence between regular sets and finite-state machines enables us to determine whether certain sets are regular. For example, let $\mathbf{R}$ denote a regular set on an alphabet A that can be recognized by a (Moore) machine M_1 . Define the complement of $\mathbf{R}$, denoted $\mathbf{R}'$, as the set containing all the strings on A that are not contained in $\mathbf{R}$. The set $\mathbf{R}'$ is regular, since it can be recognized by a machine M_2 that is obtained from M_1 by complementing the output values associated with the states of M_1 .

As another example, let us define the intersection of two sets, $\mathbf{P}$ and $\mathbf{Q}$, denoted $\mathbf{P} \& \mathbf{Q}$, as the set consisting of all the strings that are contained in both $\mathbf{P}$ and $\mathbf{Q}$. We can show that the set $\mathbf{P} \& \mathbf{Q}$ is regular by observing that each of the sets $\mathbf{P}'$ and $\mathbf{Q}'$ is regular and, consequently, $\mathbf{P}' + \mathbf{Q}'$ and $(\mathbf{P}' + \mathbf{Q}')'$ are regular. In addition, since $\mathbf{P} \& \mathbf{Q} = (\mathbf{P}' + \mathbf{Q}')'$, the set $\mathbf{P} \& \mathbf{Q}$ is regular. Regular expressions containing the complementation and intersection operations as well as union, concatenation, and closure are called *extended regular expressions*.

The added operations increase our versatility in describing regular sets. For example, consider the set of strings on the alphabet $\{0, 1\}$ such that no string in the set contains three consecutive 0's. This set can be described by the expression $[(\mathbf{0} + \mathbf{1})^* \mathbf{000} (\mathbf{0} + \mathbf{1})^*]'$, whereas a more complicated expression, such as $(\mathbf{1} + \mathbf{01} + \mathbf{001})^* (\lambda + \mathbf{0} + \mathbf{00})$, would be required if the complementation operation were not used. However, since expressions containing the complementation and intersection operations are difficult to manipulate or transform to the corresponding graphs, their usefulness is limited.

The following example will illustrate some additional techniques that can be used to determine whether certain sets are regular.

Example Let M be a finite-state machine whose input and output alphabets are $\{0, 1\}$. Assume that the machine has a designated starting state. Let $z_1 z_2 \cdots z_n$ denote the output sequence produced by M in response to the input sequence $x_1 x_2 \cdots x_n$. Define a set S_M that consists of all the strings w such that $w = z_1 x_1 z_2 x_2 \cdots z_n x_n$, for any $x_1 x_2 \cdots x_n$ in $(0 + 1)^*$. Prove that S_M is regular.

Given the state diagram of M , replace each directed arc with two directed arcs and a new state, as shown in Fig. 16.19. Retain the original starting state and designate all the original states as accepting states. The resulting nondeterministic transition graph recognizes the set S_M . Therefore, S_M must be regular.

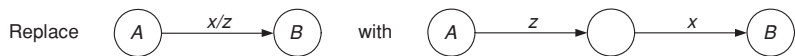


Fig. 16.19 Illustration of the procedure for designing a recognizer for S_M .

This procedure will now be applied to find a deterministic machine that recognizes the set S_N , where N is the machine described in Fig. 16.20. Replacing every arc of the machine N with two directed arcs, and following the procedure just outlined, we arrive at the transition graph in Fig. 16.21a. Converting this graph into deterministic form yields the state diagram of Fig. 16.21b.

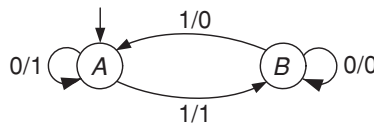


Fig. 16.20 Machine N .

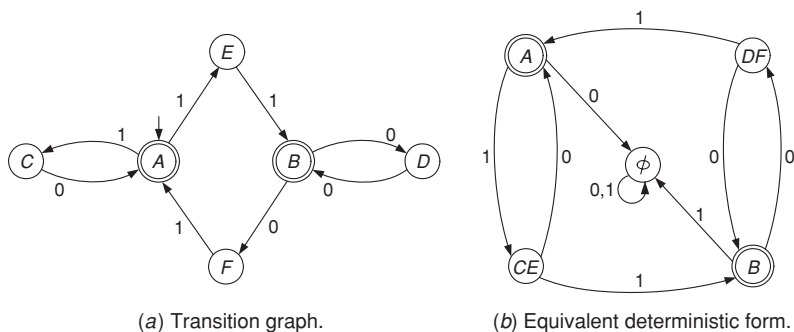


Fig. 16.21 Constructing a finite-state machine that recognizes S_N .

*16.7 Two-way recognizers

In Section 16.1, we introduced the concept of a recognizer as a finite-state control coupled through a head to a linear input tape. We assumed that the recognizer could move its head in only one direction, to the right. In an attempt to generalize the model further, we will consider recognizers that are not confined to a strict forward motion but can move two ways on their input tapes, that is, to the right and left. A natural question that now arises is whether the option given to the machine to move left and reexamine the input tape increases its computational capabilities. In other words, what characterizes the sets of tapes that are recognized by this class of machines? As we shall see, machines that can move both ways but *cannot* change the tape symbols are no more (nor less) powerful than machines that can move in only one direction.

Description of the model

A *two-way recognizer*, or *two-way machine*, consists of a finite-state control coupled through a head to a tape. Initially, the finite-state control is in its designated starting state, with its head scanning the leftmost square of the tape. The machine then proceeds to read the symbols of the tape one at a time. In each cycle of computation, the machine examines the symbol currently scanned by the head, shifts the head one square to the right or left, and then enters a new (not necessarily distinct) state.

If, when operating in this manner on a given tape, the machine eventually *moves off* the tape at the right-hand end and at that time enters an accepting state, then we shall say that the tape is *accepted* by the machine. A machine can *reject* a tape either by moving off its right-hand end while entering a rejecting state or by looping within the tape. As in the case of one-way machines, the set of tapes that are accepted by a given two-way machine is said to be *recognized* by that machine. The null string λ can be represented either by the absence of an input tape or by a completely blank tape. A machine accepts λ if and only if its starting state is an accepting state.

It is convenient to supply the two-way machine with a new symbol, ϵ , called a *left-end marker*, which is entered in the leftmost square of the tape and prevents the head from moving off the left-hand end of the tape. The end marker is not a symbol of the machine's alphabet and must not appear on any other square within the tape.

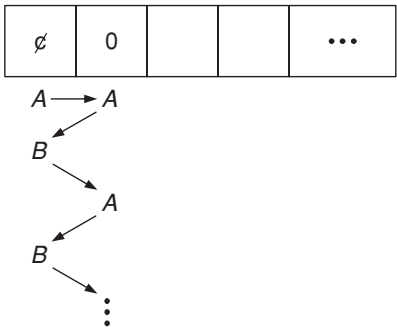
A two-way machine can be described by a state table (or diagram) that specifies, for every possible combination of present state and tape symbol being scanned, the next state that the machine should assume and the direction in which the head is to move. As directional entries, we use the letters *L* to denote a shift to the left and *R* to denote a shift to the right.

Example Table 16.1 describes a two-way machine having four states and two tape symbols, 0 and 1, plus the ϵ marker. The starting state is *A* and the accepting state is *C*. A blank tape entry indicates that the corresponding state-symbol combination cannot occur. Figure 16.22*a* illustrates the computation that the machine will perform when supplied with a tape that starts with the symbols $\epsilon 0$. The computation begins with the machine in state *A* and with its head scanning the left-end marker. According to the state table, the machine will move one square to the right while remaining in state *A*. The machine will then be scanning a 0 and, consequently, will enter state *B* and move one square to the left. From now on, the machine will oscillate between these two squares and thus all strings beginning with a 0 will be rejected.

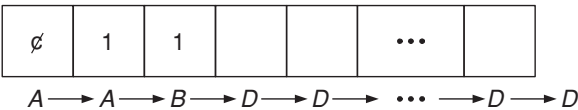
Table 16.1 A two-way-machine recognizing set **100***

	ϵ	0	1
<i>A</i>	<i>A</i> , <i>R</i>	<i>B</i> , <i>L</i>	<i>B</i> , <i>R</i>
<i>B</i>	<i>A</i> , <i>R</i>	<i>C</i> , <i>R</i>	<i>D</i> , <i>R</i>
<i>C</i>		<i>C</i> , <i>R</i>	<i>D</i> , <i>R</i>
<i>D</i>		<i>D</i> , <i>R</i>	<i>D</i> , <i>R</i>

Next, suppose that the machine is presented with a tape that starts with $\epsilon 11$. The computation is illustrated in Fig. 16.22*b*. When the third symbol is reached, the machine is in state *D*. Thereafter, it remains in state *D* regardless of the tape content until it moves off the tape. Since *D* is a rejecting state, all sets of tapes starting with 11 are rejected. Finally, let the tape consist of the string $\epsilon 10$. Again, the machine starts by moving to the right, and it goes through a succession of states until it moves off the tape in state *C*. Since *C* is an accepting state, the tape in question is accepted. By similar reasoning, we can verify that the machine recognizes the set **100***.



(a) A loop.



(b) Rejection of a tape.

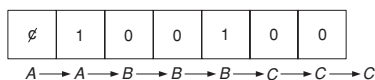
Fig. 16.22 Illustration of computations.

In the next section we shall prove that two-way machines are as powerful as one-way machines with respect to the classes of tapes that they can recognize. For some computations, however, it is convenient to use two-way recognizers since they may require fewer states than the equivalent one-way recognizers. However, for the ability of a two-way machine to reverse direction and reread its tape, we pay in terms of an increased computation time.

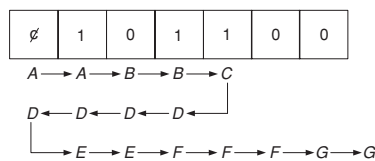
Example Consider the two-way machine shown in Table 16.2, which accepts a tape if and only if it contains at least three 1's and at least two 0's. The starting and accepting states are A and G , respectively. Some typical computations are shown in Fig. 16.23. The operation of the machine can be summarized as follows. Initially the machine is in state A and the head is scanning the left-end marker. The head then proceeds to the right to determine whether the tape contains at least three 1's. If the tape contains two or fewer 1's, it is rejected; if it contains three 1's then the head reverses its direction and moves left until it again reaches the left-end marker. The machine then proceeds to the right to determine whether the tape contains two or more 0's. If it does, the machine enters state G and will eventually accept the tape; otherwise the tape will be rejected.

Table 16.2 A two-way machine

	ϕ	0	1
A	A, R	A, R	B, R
B		B, R	C, R
C		C, R	D, L
D	E, R	D, L	D, L
E		F, R	E, R
F		G, R	F, R
G		G, R	G, R



(a) Rejecting a tape.



(b) Accepting a tape.

Fig. 16.23 Example of computations.

The minimal one-way machine that is equivalent to the two-way machine in Table 16.2 has 12 states. This larger number of states is necessary because of the way in which a one-way machine operates. Any one-way machine that recognizes the above set of tapes must examine the tapes for the proper

number of 0's and 1's *simultaneously*. This can be done, for example, by the use of two separate counters, one for the 1's and the other for the 0's. The state of the machine in such a case is the composite state of the two counters. Consequently, the number of states required to perform the above computation is proportional to the *product* of the numbers of states required to test the tapes for the number of 0's and the number of 1's separately. The two-way machine in this example tests the tapes first for the appropriate number of 1's and then for the appropriate number of 0's. Thus, the number of states is proportional to the *sum* of the numbers of states required to test the tapes for the two requirements separately.

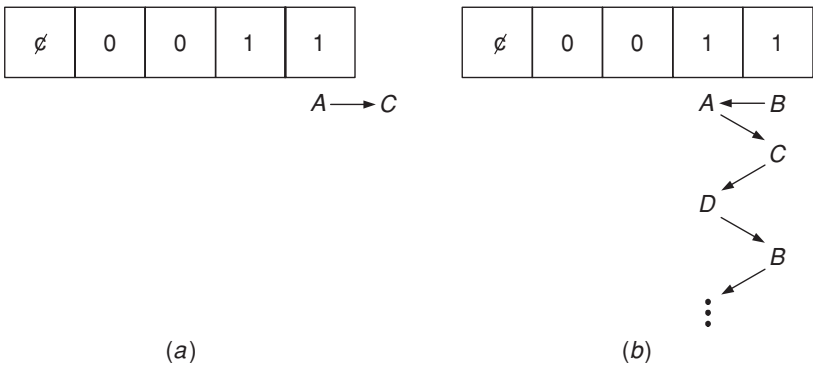
Conversion to one-way recognizers

We now turn to proving that two-way machines can recognize sets of tapes (or strings) if and only if they are regular sets. Specifically, we shall show that for every given two-way machine there is an equivalent one-way machine that recognizes the same set of tapes. Since the details of the construction procedure do not add significantly to its understanding, we shall confine our discussion to sketching the main ideas of the proof.

Since a one-way machine makes as many moves as there are symbols on the tape while a two-way machine can make moves by reversing direction, the one-way machine cannot keep track of all the moves of the two-way machine or simulate them. It is, therefore, necessary to isolate the significant information gained by a two-way machine on moving to the left from the particular sequence of moves. Consider an initial segment at the left of the input tape, and suppose that the head is scanning the rightmost square of this segment. The only way in which this segment can influence the future behavior of the two-way machine is via the state which the machine is in when (and if) it leaves this segment. Thus, when a two-way machine backs up and reexamines a segment of the tape, the state S_i in which the machine reenters the segment and the corresponding state S'_i which the machine would be in if it left the segment are the only two factors of significance in predicting the future behavior of the machine.

A two-way machine having n states can be in any of these states when it scans the rightmost square of the initial segment. Two cases must be considered. First, the machine may never leave the segment but oscillate within it. Second, the machine will ultimately leave the segment on the right in one of its n states. Thus, a reentry into a segment may have $n + 1$ outcomes, that is, leaving the segment in one of the n states or not leaving it. Consequently the effect of the segment on the computation can be determined by specifying, for each state S_i in which the machine might reenter the segment, which of the $n + 1$ outcomes would indeed result. Such a specification is accomplished by means of a *crossing function* (or *crossing table*), denoted $C(S)$.

Fig. 16.25 Illustration of computations on the segment $\emptyset 0011$.



An important property of crossing functions is that the crossing function of a $(k + 1)$ -symbol segment can be obtained from the crossing function of a k -symbol segment. The rightmost column of Table 16.4 contains the crossing function associated with the segment $\emptyset 0011$. This crossing function can be obtained from the crossing function of the segment $\emptyset 001$. Suppose, for example, that the machine is in state A and is scanning the rightmost symbol of $\emptyset 0011$. According to the state table in Table 16.3, the machine will move to the right and enter state C , as illustrated in Fig. 16.25a. Accordingly, the entry in row A in the rightmost column is C . If, however, the machine is in state B while scanning the rightmost symbol of the given segment then it will move left and enter state A . According to the crossing function associated with the segment $\emptyset 001$, the machine will leave this segment in state C , as shown in Fig. 16.25b. Again it will scan the rightmost symbol of $\emptyset 0011$ and, according to the state table, again it will move left and enter state D . According to the crossing function for $\emptyset 001$, the machine will ultimately leave this segment on the right and enter state B . Evidently such a sequence of moves indicates that the computation will never halt and, consequently, a 0 is entered in row B of Table 16.4. The same line of reasoning leads to the specification of the entries in rows C and D .

The procedure followed in this example leads to the conclusion that, *given the crossing functions associated with the initial segments containing k symbols, we can readily obtain the crossing functions associated with all initial segments containing $k + 1$ symbols*. In fact, since the number of distinct crossing functions associated with a specific two-way machine cannot exceed $(n + 1)^n$, where n is the number of states, it is possible to construct a one-way machine that will read the tape from left to right and compute with each move the crossing function associated with the corresponding initial segment. Such a machine will have as many states as there are crossing functions. Its input alphabet is the same as that of the corresponding two-way machine. The next-state entries of the one-way machine are obtained as follows. For a given state, which corresponds to a crossing function of the two-way machine, the next-state entry under the input symbol α corresponds to the new crossing function obtained from the given one and the symbol α , as illustrated in Fig. 16.25.

Once we have a one-way machine that scans the tape from left to right and computes the crossing functions associated with successive initial segments, since the starting state of the two-way machine is specified it is a simple matter to determine, after each move of the one-way machine, the corresponding next state of the two-way machine. Consequently, we can determine the state of the two-way machine when it moves off the tape. If this state is an accepting state then the one-way machine will also accept the tape; otherwise it will reject the tape. We thus have the following result.

- The sets of strings recognized by two-way finite-state machines are the same as the sets recognized by one-way finite-state machines. Moreover, there exists an effective procedure for constructing a one-way machine that recognizes the same set of strings as a given two-way machine.

Although two-way machines are no more powerful than one-way machines with respect to the sets of strings that they can recognize, it is often more convenient to describe certain computations in terms of two-way machines. The equivalence of the two models, however, makes it generally possible to use either.

Notes and references

Nondeterministic graphs were first used by Myhill [8] and further developed by numerous investigators, in particular those working on languages. The initial concept of regular expressions and the equivalence between regular expressions and finite-state machines were presented by Kleene [5]. Simpler techniques for converting regular expressions into transition graphs, and vice versa, were subsequently developed by Copi, Elgot, and Wright [4], McNaughton and Yamada [6], and Ott and Feinstein [9]. The procedure presented in this chapter of constructing transition graphs from regular expressions is due to Ott and Feinstein [9], while the procedure used to derive regular expressions that describe transition graphs is due to Arden [1]. A survey of regular expressions is available in Brzozowski [2].

Two-way machines were first investigated by Rabin and Scott [10], who provided the first proof that two-way machines are equivalent to one-way machines. Shepherdson [11] subsequently provided a simpler proof, the one outlined in Section 16.7.

- [1] Arden, D. N.: "Delay logic and finite state machines," in *Proc. Second Ann. Symp. Switching Theory and Logical Design*, pp. 133–151, October 1961.
- [2] Brzozowski, J. A.: "A survey of regular expressions and their applications," *IRE Trans. Electron. Computers*, vol. EC-11, pp. 324–335, June 1962.
- [3] Brzozowski, J. A.: "Derivatives of regular expressions," *J. Assoc. Computing Machinery*, vol. 11, pp. 481–494, 1964.
- [4] Copi, I. M., C. C. Elgot, and J. B. Wright: "Realization of events by logical nets," *J. Assoc. Computing Machinery*, vol. 5, pp. 181–196, April 1958; reprinted in Moore [7].

- [5] Kleene, S. C.: *Representation of Events in Nerve Nets and Finite Automata*, pp. 3–41, Automata Studies, Princeton University Press, 1956.
- [6] McNaughton, R., and H. Yamada: “Regular expressions and state graphs for automata,” *IRE Trans. Electron. Computers*, vol. EC-9, pp. 39–47, March 1960; reprinted in Moore [7].
- [7] Moore, E. F. (ed.): *Sequential Machines: Selected Papers*, Addison-Wesley, Reading MA, 1964.
- [8] Myhill, J.: “Finite automata and the representation of events,” WADC Technical Report 57–624, pp. 112–137, 1957.
- [9] Ott, G. H., and N. H. Feinstein: “Design of sequential machines from their regular expressions,” *J. Assoc. Computing Machinery*, vol. 8, pp. 585–600, October 1961.
- [10] Rabin, M. O., and D. Scott: “Finite automata and their decision problems,” *IBM J. Res. Develop.*, vol. 3, no. 2, pp. 114–125, April 1959; reprinted in Moore [7].
- [11] Shepherdson, J. C.: “The reduction of two-way automata to one-way automata,” *IBM J. Res. Develop.*, vol. 3, no. 2, pp. 198–200, April 1959; reprinted in Moore [7].

Problems

Problem 16.1. For each of the sets described as follows, find a transition graph that recognizes the set.

- (a) The set of strings on the alphabet $\{0, 1\}$ that start with 01 and end with 10.
- (b) The set of strings on the alphabet $\{0, 1\}$ that start and end with a 1, and in which every 0 is immediately preceded by at least two 1's.
- (c) The set of strings on the alphabet $\{0, 1, 2\}$ in which every 2 is immediately followed by exactly two 0's and every 1 is immediately followed by either 0 or else by 20.

Problem 16.2. Consider the class of transition graphs containing no λ -transitions.

- (a) Show a procedure for converting a specified transition graph with several starting vertices into a graph with just one starting vertex. Apply your procedure to the graph in Fig. P16.2.

Hint: Add a new vertex and designate it as the starting vertex.

- (b) Show a procedure for converting a given transition graph with several accepting vertices into a graph with just one accepting vertex. Apply your procedure to the graph in Fig. P16.2.
- (c) Is it always possible to convert an arbitrary transition graph into a graph with just one starting vertex and just one accepting vertex? Determine the conditions under which such a conversion is possible.

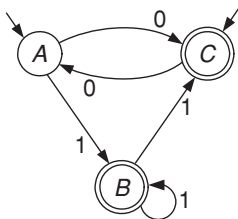
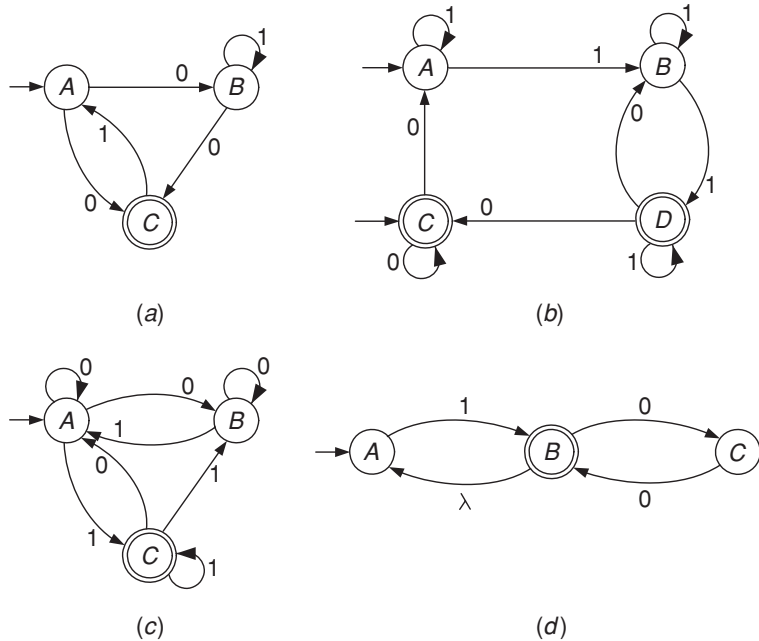


Fig. P16.2

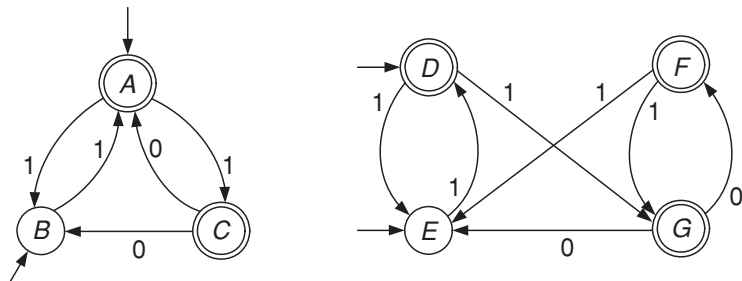
Problem 16.3. For each of the nondeterministic graphs in Fig. P16.3, find an equivalent deterministic graph (in standard form) that recognizes the same set of strings.

Fig. P16.3



Problem 16.4. Show that the two graphs in Fig. P16.4 are equivalent by converting them to deterministic forms.

Fig. P16.4



Problem 16.5. Design a finite-state machine that accepts only those input sequences that end with either 101 or 0110. First construct a nondeterministic graph that recognizes the above set of sequences and then convert this graph into an equivalent deterministic graph. Discuss the merits of this approach versus the direct approach of deriving a state diagram from a word description.

Problem 16.6. Give a word description of the sets described by the following regular expressions:

- (a) $110^*(0 + 1)$;
- (b) $1(0 + 1)^*101$;
- (c) $(10)^*(01)^*(00 + 11)^*$;
- (d) $(00 + (11)^*0)^*10$.

Problem 16.7. Find a regular expression for each set described in Problem 16.1.

Problem 16.8. Use the identities in Section 16.4 to verify the identities below:

- (a) $10 + (1010)^*[\lambda^* + \lambda(1010)^*] = 10 + (1010)^*$;
- (b) $(0^*01 + 10)^*0^* = (0 + 01 + 10)^*$;
- (c) $\lambda + 0(0 + 1)^* + (0 + 1)^*00(0 + 1)^* = [(1^*0)^*01^*]^*$.

Problem 16.9.

- (a) Use the induction procedure developed in Section 16.5 to find a transition graph that recognizes the set of strings described by

$$R = 0(11 + 0(00 + 1)^*)^*.$$

- (b) Convert the graph found in (a) to a deterministic state diagram.

Problem 16.10. For each of the following expressions, find a transition graph that recognizes the corresponding set of strings:

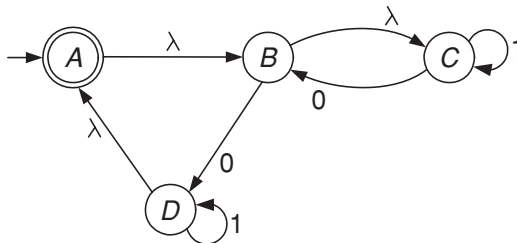
- (a) $(0 + 1)(11 + 0^*)(0 + 1)$;
- (b) $(1010^* + 1(101)^*0)^*1$;
- (c) $(0 + 11)^*(1 + (00)^*)^*11$.

Problem 16.11. The regular expression that corresponds to the transition graph in Fig. P16.11 is

$$R = [(1^*0)^*01^*]^*.$$

Find a finite-state machine that recognizes the same set of strings.

Fig. P16.11



Problem 16.12. The nondeterministic graph in Fig. P16.12 has A and B as starting vertices and C as an accepting vertex.

- (a) Find a regular expression that describes the set of strings accepted by this graph.
- (b) Derive a reduced deterministic machine equivalent to this graph.

Fig. P16.12

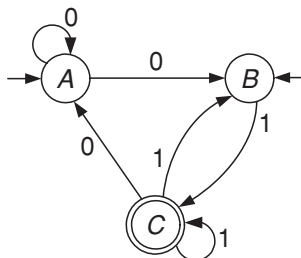


Table P16.16

PS	NS, z	
	$x = 0$	$x = 1$
A	$B, 1$	$A, 1$
B	$A, 0$	$C, 0$
C	$D, 1$	$B, 0$
D	$C, 0$	$A, 1$

Problem 16.17. The reverse $\mathbf{R}^r$ of a set $\mathbf{R}$ is the set that consists of the reverses of the strings in $\mathbf{R}$. Thus, for example, if 0101 is in $\mathbf{R}$ then 1010 is in $\mathbf{R}^r$.

(a) Prove that if $\mathbf{R}$ is regular then so is $\mathbf{R}^r$.

Hint: Develop a systematic procedure to convert a given regular expression into its reverse.

(b) Apply the above procedure to find the reverse of the expression

$$\mathbf{R} = (00)^*(0 + 10^*)^* + 10^*(01^*10^*)^*.$$

Problem 16.18. Either prove each of the following statements or show a counter example.

- (a) Every *finite* subset of a nonregular set is regular.
- (b) The expressions $\mathbf{P} = (1^*0 + 001)^*01$ and $\mathbf{Q} = (1^*001 + 00101)^*$ are equivalent.
- (c) Let $\mathbf{R}$ denote a regular set. Then the set consisting of all the strings in $\mathbf{R}$ that are identical to their own reverses is also a regular set.
- (d) Every subset of a regular set is also regular.

Problem 16.19. Consider the nondeterministic machine M^n , which is obtained from a strongly connected deterministic machine M by interchange of the sets of starting and accepting states and reversal of the arrows on the state diagram.

- (a) If the machine M recognizes the set $\mathbf{R}$, what is the set recognized by M^n ?
- (b) Prove that the deterministic machine obtained by applying “subset construction” to M^n has no equivalent states.

Problem 16.20. Let $\mathbf{P}$ be a regular set consisting of strings of even length. Define a set $\mathbf{Q}$ that consists of exactly those strings that can be formed by taking the first half of each member of $\mathbf{P}$. (For example, if 10110100 is contained in $\mathbf{P}$ then 1011 will be contained in $\mathbf{Q}$.) Prove that $\mathbf{Q}$ is a regular set.

Hint: Design a machine that recognizes $\mathbf{Q}$.

Problem 16.21. Let $\mathbf{P}$ be a regular set, and let $\mathbf{Q}$ be the set formed of all the strings from $\mathbf{P}$ with even-numbered symbols deleted; that is, if $a_1a_2a_3a_4a_5 \dots$ is a string in $\mathbf{P}$, then $a_1a_3a_5 \dots$ is a string in $\mathbf{Q}$. Prove that $\mathbf{Q}$ is a regular set.

Problem 16.22. Let $\mathbf{P}$ be an arbitrary regular set. Consider those strings w in $\mathbf{P}$ such that both w and ww are in $\mathbf{P}$. Define $\mathbf{Q}$ to be the set consisting of all the above w 's. Thus, for example, if 101 and 101101 are in $\mathbf{P}$ then 101 is in $\mathbf{Q}$. Prove that $\mathbf{Q}$ is a regular set.

Problem 16.23. Let $\mathbf{R}$ be a regular set on the alphabet $\{0, 1\}$. The *derivative of $\mathbf{R}$ with respect to x* , denoted $\mathbf{R}_x$, is defined as the set consisting of all substrings y such that xy is in $\mathbf{R}$. For example, if $\mathbf{R} = \mathbf{01}^* + \mathbf{100}^*$ then $\mathbf{R}_0 = \mathbf{1}^*$ and $\mathbf{R}_{10} = \mathbf{0}^*$.

- Prove that, for all x , $\mathbf{R}_x$ is a regular set.
- Show that there is only a finite number of distinct derivatives for any regular set (although there is an infinite number of choices for x). Find an upper bound on this number if it is known that $\mathbf{R}$ can be recognized by a transition graph with k vertices.

Problem 16.24. The *right quotient* of two sets X and Y , denoted X/Y , is defined as the set Z that consists of all strings z such that $x = zy$ is a string in X and y is a string in Y . Prove that if X is a regular set then $Z = X/Y$ is also a regular set. The set Y may or may not be regular.

Problem 16.25. Determine which of the following tapes is accepted by the two-way machine shown in Table P16.25. The starting and accepting states are A and D , respectively.

- $\emptyset 010101$
- $\emptyset 010110$
- $\emptyset 10101$

Table P16.25

	$\emptyset$	0	1
A	A, R	B, R	C, R
B		D, L	C, L
C		C, R	D, R
D		B, R	C, L

Problem 16.26. A two-way machine with n states is started at the left end of a tape containing p squares. What is the maximum number of moves that the machine can make before accepting the tape?

Problem 16.27. Construct a two-way machine whose tape may contain symbols from the alphabet $\{0, 1, 2\}$ plus the left-end marker and which accepts a string if and only if it starts and ends with a 2 and every 2 except the first is immediately preceded by a substring from the set $\mathbf{0(01)^*}$.

Problem 16.28. A given two-way machine recognizes a set of tapes A , rejects a set B , and does not accept (by never halting) a set C . Can a two-way machine be designed so that it:

- recognizes B , rejects A , does not accept C ?
- recognizes A and rejects B and C ?
- recognizes A but does not accept B and C ?
- recognizes A and C and rejects B ?
- recognizes C , rejects B , and does not accept A ?

Hint: Determine first which of the sets A , B , and C is regular.

Index

- adder
 - carry-lookahead, 131–133
 - full, 110, 129
 - half, 147
 - modulo- p , 524
 - ripple-carry, 130
 - serial-binary, 266–268
 - ternary, 148
- admissible pattern, 189
- algebraic divisor, 156, 234
 - double-cube, 234
 - multiple-cube, 234
 - single-cube, 234
- algebraic factorization, 234–236, 247–250
 - targeted, 248
- algebraic resubstitution, 234
- aliasing, 463
- alphabet
 - code, 504–506
 - input, 270, 313, 414, 432, 441
 - output, 271, 432
 - source, 504–506
- AND gate, 57
- AND operation, 38
- Arden's rule, 601
- asynchronous circuits, 109
 - sequential, 338–371
- at-speed test, 232, 461
- autonomous clock, 386
 - maximal, 387
- backtrack, 199, 215, 219, 230
- base, 3
- base function, 163
- binary arithmetic, 8–10
- binary-coded decimal (BCD) code, 10
- binary codes, 10–19
- binate function, 193, 246
- binate input, 247
- Boolean algebra, 58–60
- Boolean functions, 110
- branching, 92, 93
- bridging fault, 210
 - fault collapsing, 220
 - feedback, 211
 - gate-level, 224
 - I_{DDQ} testing, 210, 220–224
 - nonfeedback, 211
 - optimistic condition, 221
- built-in self-test (BIST), 461–464
 - aliasing, 463
 - degree of polynomial, 461
 - linear feedback shift register, 461
 - primitive polynomial, 462
 - reseeding, 463
 - response analyzer, 463
 - signature, 461
 - test pattern generator, 461
- burst
 - input, 359
 - output, 359
- burst-mode, 358–363
- canonical forms
 - product of sums, 47
 - sum of products, 47
- Cartesian product, 26
- cell library, 162
- cell table, 297
- chain-connected blocks, 32

- checking experiment, 431, 442–448
- checkpoint, 216
- clock, 109
- closed covering, 324
- code converter, 74–77
- codes
 - BCD, 10
 - binary, 10–19
 - block, 505
 - cyclic, 12
 - decipherable, 504–510
 - error-detecting and correcting, 14–19
 - Excess-3, 11
 - Gray, 12, 13
 - Hamming, 16
 - instantaneous, 505
 - reflected, 13
 - ringtail, 144
 - self-complementing, 11
 - synchronizable, 508–510
 - 2-out-of-5, 14
 - variable-length, 505
 - weighted, 10, 11
- cofactor, 243
- co-kernel, 156–161
- combinational logic, 37
- common subexpression, 153–158
- comparators, 113–115
- compatibility graph, 325–327
- compatibility relation, 27
- compatible pair, 322–329, 494–496
- compatible states, 318
 - maximal, 319
- complement
 - 1's, 5
 - 9's, 5
- complementation, 38
- complex gate, 139
- composite graph, 583
- composite machine, 405
 - decomposition of, 404–413
 - general, 411
- concatenation, 504, 577
- conflict, 215–219
- conjunctive normal form, 47
- connection matrix, 482
- consistency, 214
- contracted table, 485
- control assignment, 238
- controlling value, 226
- conversion of bases, 5–8
- counters, 284–288
- cover, 29, 78, 390
- crossing function, 598
- cube, 70
 - D*-, 218–220
 - privileged, 344
 - required, 341
 - singular, 218
 - test, 219
 - transition, 341
- cube-free expression, 156
- cube–literal incidence matrix, 157
- current monitoring, 220
- cut set, 140
- cycle, 354–356
 - of computation, 293
- cycle set, 532
- cyclic codes, 12
- D*-algorithm, 217–220
 - backtrack, 219
 - D*-drive, 219
 - D*-frontier, 219
 - D*-intersection, 218
 - implication, 219
 - line justification, 219
 - primitive *D*-cube of a fault, 218
 - propagation *D*-cube, 218
 - singular cover, 218
 - singular cube, 218
 - test cube, 219
- data selectors, 115–117
- De Morgan's theorem, 42, 43
- decoders, 119–125
 - BCD, 119
 - decimal, 119
- decomposition
 - of switching functions, 153, 161, 165
 - parallel, 393, 409–411
 - serial, 390, 404–409
 - Shannon's, 48
 - with specified components, 411–413
- definitely diagnosable machines, 450–453
- definiteness, 483–488
 - tests for, 486–488
- delay element, 268, 276, 338
- delay fault, 211
 - path, 212
 - transition, 212

- delay fault test, 224–232
 - nonrobust, 226
 - robust, 227
 - validatable nonrobust, 227
- delay operator, 526
- demultiplexer, 120–122
- design for testability, 458–460
 - full scan, 458
 - normal mode, 458
 - partial scan, 458
 - test mode, 458
- deterministic machine, 307
- dimension of a machine, 525
- direct sum, 333
- disjunctive normal form, 47
- distance, 15
 - minimal, 15
- distinguishing sequence, 312, 439
- distinguishing table, 550
- division operation, 5–9, 156
 - divisor, 156
 - quotient, 156
 - remainder, 156
- divisor, 156
 - algebraic, 156, 234
 - Boolean, 156
- don't-cares, 74
 - observability, 242
 - satisfiability, 242
- double-cube divisor, 234
- double-cube extraction, 234
- dual-expression extraction, 234
- duality, principle of, 40
- equivalence classes, 26, 313
- equivalence partition, 314
- equivalence relation, 25–27
- equivalent faults, 216
- error, 14–19
 - detection and correction of, 14–19
 - propagation, 214
- ESPRESSO, 95–97
 - expanded, 95
 - irredundant, 95
 - reduce, 95
- Euclidean algorithm, 561, 562
- Excess-3 code, 11
- excitation function, 272–280
- excitation table, 272
- excitation variables, 338
- EXCLUSIVE-OR operation, 51
- experiments, 431–435
 - adaptive, 432
 - checking, 431, 442–448
 - distinguishing, 439, 440
 - homing, 435–437
 - multiple, 432
 - preset, 432
 - synchronizing, 437–439
- expressions (*see* switching expressions)
- extended *D*-algorithm, 455
- extraction, 153, 159–161
 - cube, 159
 - dual expression, 234
 - kernel, 159
- factor, 155
 - algebraic, 156
 - Boolean, 156
- factored form, 152
- false vertex, 183
 - maximal, 183
- fanin, 109
- fanout, 109
- fanout-free circuit, 217
- fault, 206
 - bridging, 210
 - coverage, 213
 - delay, 211
 - list, 216
 - redundant, 236, 240
 - stuck-at, 206
 - stuck-on, 210
 - stuck-open, 208
- fault collapsing, 216
- dominance, 216
- equivalence, 216
- fault model, 206–212
 - bridging, 210, 211
 - delay, 211, 212
 - functional, 453
 - single-state-transition (SST), 453
 - stuck-at, 206
 - stuck-on, 210
 - stuck-open, 208
 - structural, 206–208
 - switch-level, 208–211
- fields, 559–561
 - finite, 559
 - Galois, 560
- finite memory, 478–483
 - tests for, 479–481

- finite-state machine, 265
 - deterministic, 307
 - head, 293
 - incompletely specified, 317–330
 - Mealy, 307
 - Moore, 307
 - nondeterministic, 572–577
 - nonwriting, 293
 - writing, 293
 - tape, 293
- five-valued logic, 217
- flip-flop, 276–280
 - D*, 279
 - edge-triggered, 279
 - JK*, 277
 - master–slave, 276–279
 - set–reset, 277
- flow table, 346–350
 - primitive, 348
 - reduced, 350
- four-phase clocking, 194
 - evaluate, 194
 - hold, 194
 - reset, 194
 - wait, 194
- functionally complete operations, 52
- function, 27
 - base, 163
 - binate, 193
 - crossing, 598
 - excitation, 272–280
 - linearly separable, 184
 - majority, 64, 175
 - minority, 177
 - output, 74, 307
 - self-dual, 62
 - state transition, 307
 - symmetric, 171
 - threshold, 178
 - transfer, 526
 - transmission, 54
 - unate, 104, 183
- full scan, 458
- fundamental mode, 339
 - multiple-input change, 339
 - single-input change, 339
- gate, 53
 - AND, 57
 - majority, 175
 - minority, 177
 - NAND, 125–128
 - NOR, 125–128
 - NOT, 58
 - OR, 57
 - threshold, 177
- geometric representation, 182
- Gray code, 12, 13
- greatest lower bound, 30, 381
 - of closed partitions, 381
- Hamming code, 16
- Hasse diagram, 29
- hazards, 226
 - dynamic, 226
 - function, 339
 - logic, 339
 - static, 226
- homing sequence, 436
- Huntington postulates, 65
- I_{DDQ}* testing, 210
- illegal intersection, 344
- implicant, 78
 - dynamic-hazard-free, 344
- implication, 78, 214
 - backward, 214
 - conflict, 215–219
 - forward, 214
- implication table, 229, 392, 408
- implied pair, 322, 449, 494
- impulse response, 528
- incompletely specified machines, 317–330
- information-flow inequality, 400
- information losslessness, 491–499
 - of finite order, 493
 - tests for, 494–497
- initialization sequence, 454
- initialization vector, 209
- input alphabet, 270
- input-consistent partition, 386
- input variable, 76
- integration, 113
- internal state, 176, 267
- inverse machine, 499–504
 - for a linear machine, 529–531
- inverter, 109
- irredundant circuit, 68
- isomorphic machines, 444
- isomorphic systems, 53

- iterative array model, 455
- iterative networks, 296–300
 - cell inputs, 297
 - cell outputs, 297
 - cell table, 298
 - input carries, 297
 - output carries, 297
- justification, 214, 456
 - line, 214
- Karnaugh map, 68
- kernel, 156–161
- kernel–cube incidence matrix, 160
- Kleene’s theorem, 593
- latch, 272–276
 - master, 277
 - slave, 277
- lattice, 30–33
 - of closed partitions, 380–383
 - complemented, 33
 - distributive, 32
 - Mm -, 402
 - π -, 381
- leaf-DAG, 165
- least significant digit, 4
- least upper bound, 30
- linear feedback shift registers (LFSRs), 461
 - feedback polynomial, 461
 - primitive polynomial, 462
 - seed, 462
- linear separability, 184
- linear sequential machines, 461, 523
 - autonomous, 532
 - chain realization of, 533
 - controllable, 568
 - identification of, 550–556
 - inert, 525–527
 - observability, 543
 - predictability, 543
 - reduction of, 541–550
 - response of, 528, 540, 541
- literal, 41
 - redundant, 41
- logic hazard, 339
 - static-0, 245, 341
 - static-1, 341
- logic polarity, 108
- logic transformations, 151–155
 - decomposition, 161
 - elimination, 154
 - extraction, 159–161
 - factoring, 155–159
 - hazard-nonincreasing, 345
 - substitution, 162
- logical path, 226
- machines
 - common predecessor, 410
 - composite, 405
 - concurrently operating, 404
 - definite, 483–488
 - definitely diagnosable, 450–453
 - finite-memory, 478–483
 - finite-state, 265
 - identification of, 440–442
 - information lossless, 491–499
 - inverse, 499–504
 - linear sequential, 461, 523
 - Mealy, 307
 - minimal, 319–322
 - Moore, 307
 - predecessor, 378
 - sequential, 307
 - successor, 378
 - Turing, 293
 - two-way, 595
- majority
 - function, 64, 175
 - gate, 175
- mandatory assignments, 238
- map, 68–78
 - cyclic, 93
 - of five variables, 77
 - of four variables, 69
 - Karnaugh, 68
- map-entered variables, 93–95
- marker, left-end, 595
- match, 163, 166
- matrices, 119, 482
 - characteristic, 541
 - characterizing, 544
 - connection, 482
 - cube–literal incidence, 157
 - diagnostic, 542
 - kernel–cube incidence, 160
- maxterm, 47

- Mealy machine, 307
 - transformation to Moore machine, 334
- memory span, 478
 - with respect to input–output sequences, 478–483
 - with respect to input sequences, 483–488
 - with respect to output sequences, 488–491
- merger graph, 322, 323
- merger table, 327–330
- minimal machine, 319–322
- minority
 - function, 177
 - gate, 177
- minterm, 46
- multiple-input signature register (MISR), 463
- Mm pairs, 398
- modulo-2 addition, 524
- MOBILE, 175
- Moore machine, 307
- MOS transistors and gates, 132–143
- most significant digit, 4
- multiple faults, 233
- multi-output circuit, 76
- multiplexer, 115
- multiplier, modulo- p , 524
- n -cube, 182
- NAND gate, 125–128
- NAND–NAND implementation, 233, 384
- NAND operation, 116
- networks
 - bridge, 139
 - electronic gate, 57
 - iterative, 296–300
 - non-series–parallel, 139, 140
 - series–parallel, 136–139
- network covering, 163, 167–169
- nine-valued logic, 456–458
- noncontrolling value, 226
- nonfeedback bridging faults, 211
- nonrobust test, 226
- NOR gate, 125–128
- NOR operation, 52
- NOT gate, 58
- NOT operation, 38
- null sequence, 334, 528
 - maximal, 529
- number systems, 3–10
 - binary, 4
 - decimal, 3
 - hexadecimal, 8
 - octal, 8
- observability don't-care set, 242
- observation assignment, 238
- OFF-set, 248
- on-input, 226
- ON-set, 248
- operation
 - AND, 38
 - division, 5–9, 156
 - EXCLUSIVE-OR, 51
 - functionally complete, 52
 - NAND, 116
 - NOR, 38
 - NOT, 38
 - OR, 38
 - star, 583
 - unary, 28
- optimistic condition, 221
- OR gate, 57
- OR operation, 38
- ordered pair, 25
- ordering, 28–30
 - partial, 28
 - total, 29
- output alphabet, 271
- output-consistent partition, 384
- output dependency, reduction of, 383–385
- output function, 74, 307
- output predecessor, 497
- output (compatible) states, 494
- output successor, 489
- output variable, 270
- path-delay fault, 212
- palindromes, 304, 331
- parity
 - even, 14
 - odd, 17
- parity bit, 14
- parity check, 14
 - generator, 111
- partial scan, 458

- partition, 28
 - basic, 381
 - blocks of, 397
 - closed, 376
 - equivalence, 314
 - greatest, 32
 - input-consistent, 386
 - least, 32
 - output-consistent, 384
 - refinement of, 314
 - state-consistent, 416
 - uniform, 28
- partition pair, 397
- path sensitization, 213–215
 - one-dimensional, 215
 - two-dimensional, 215
- pattern generators, 461–463
 - feedback polynomial, 461
 - linear feedback shift registers, 461
 - seed, 462
- pattern graph, 163
- perfect induction, 39
- physical path, 226
- polynomial, 3–6, 155, 461
 - feedback, 462
 - primitive, 462
- position number, 16
- positive unate function, 181
- predecessor machine, 378
- predecessor table, output, 497
- present state, 267
 - vector of, 538
- prime implicant, 78
 - essential, 79
- prime implicant chart, 86–93
 - augmented, 99
 - branching method, 92, 93
 - construction of, 86
 - cyclic, 93
 - reduction of, 90–92
- prime implicant function, 88
- primitive gates, 216
- priority encoder, 117–119
- product of sums, 47
 - canonical, 47
- propagation D -cube, 218
- propositional calculus, 88
- quantum cellular automata, 175
- Quine–McCluskey method, 81
- race, 354–356
 - critical, 355
 - noncritical, 355
- radix, 3
- radix point, 4
- rated clock scheme, 225
- realizable pattern, 196
- recognizers
 - deterministic, 570, 571
 - finite-state, 570–607
 - nondeterministic, 572–574
 - two-way, 595
 - conversion to one-way, 598–601
- rectangle covering, 157, 158
- prime, 157
- redundancy identification and removal, 236–244
 - direct, 239–244
 - don't-care-based, 242–244
 - dynamic, 241, 242
 - indirect, 237–239
 - static, 239–241
- redundant literal, 41
- regular expressions, 577–582
 - definition of, 579, 580
 - derivative of, 607
 - equivalent, 580
 - extended, 593
- regular set, 580
- relation, 25–28
 - antisymmetric, 26
 - binary, 25
 - compatibility, 27
 - equivalence, 26
 - reflexive, 26
 - symmetric, 26
 - transitive, 26
- relatively essential vertex, 248
- re-seeding, 463
- resonant tunneling diodes (RTDs), 175
- response analyzer, 461
- response compression, 461
 - aliasing, 463
 - multiple-input signature register, 463
- ring, 559
 - commutative, 559
- ripple-carry adder, 130, 131
- robust test, 227

- stuck-at fault, 206
 - multiple, 207
 - single, 207
- satisfiability don't-care set, 242
- scan design, 458–460
 - normal mode, 458
 - scan-in, 458
 - scan-out, 459
 - test mode, 458
- secondary variable, 268
- self-dual function, 62
- sensitizable path, 213
- sensitizing input values, 229–231
 - nonrobust, 231
 - robust, 229
- sequence detector, 280–283
- sequential circuit, 265
 - asynchronous, 338–371
- sequential machine, 307
 - linear, 461, 523
- serial-to-parallel converter, 112
- series–parallel switching circuits, 52
 - elementary, 53
- sets
 - Cartesian product, 26
 - complement, 24
 - disjoint, 26
 - elements of, 23
 - empty, 23
 - equal, 24
 - intersection of, 24
 - null, 23
 - partially ordered, 28
 - totally ordered, 29
 - universe, 24
 - union of, 24
- seven-segment display, 123
- Shannon's expansion theorem, 48
- shift register, 278
 - feedback, 461
 - feedforward, 525–527
- side input, 214
- signature, 461
 - golden, 461
- sine generator, 124, 125
- single-cube divisor (*see* algebraic divisor)
- single-electron box (SEB), 175, 176
- single-state-transition (SST) fault, 453
- singular cover, 218
- slack, 231
- sneak path, 141
- stable state, 338
- standard form, 316
- states, 267
 - adjacent, 356
 - accepting, 571
 - compatible, 318
 - closed set of, 324
 - complete, 360
 - distinguishable, 312
 - equivalent, 312
 - final, 272
 - initial, 272
 - input, 338
 - rejecting, 595
 - secondary, 338
 - stable, 338
 - total, 338
 - unstable, 339
- state assignment,
 - in asynchronous circuits, 356–358
 - race-free, 361
 - using partitions, 375–380
 - valid, 356
- state diagram, 267
- state justification, 456
- state-pair differentiating sequence (SPDS), 454
- state splitting, 320
 - application to parallel decomposition, 393–395
- state table, 266–268
- state transition, 267
 - function, 307
- state variables, 268
 - reduction of functional dependency of, 377–380
- static hazard, 226
- status
 - uncontrollability, 239
 - unobservability, 239
- strongly connected machine, 309
- structural testing, 206
- stuck-on fault, 210
- stuck-open fault, 208
- subject graph, 163
- subset, 24
 - proper, 24
 - self-dependent, 375
- subset construction procedure, 576

- subtractor, 147
 - half, 147
 - full, 147
- successor, 309
- successor machine, 378
- successor table, 489
 - output, 489
- sum of products, 46
 - canonical, 47
- switching algebra, 37–44
- switching expressions, 40
 - algebraic, 155
 - Boolean, 155
 - cube-free, 156
 - irredundant, 68
 - simplification of, 40
- switching functions, 44
 - canonical forms of, 46–49
 - decomposition of, 153
 - minimization of, 67–107
 - number of, 50
 - of two variables, 50, 51
- symmetric functions, 171
- synchronizing sequence, 437
- synchronous circuits, 266, 274
- synthesis for testability, 232–250

- table of combinations, 39
- tabulation procedure, 81–86
- tape, 293
- tautology, 183
- technology mapping, 162
- test
 - application time, 213
 - delay, 212
 - generation time, 213
 - I_{DDQ} , 210
 - nonrobust, 226
 - robust, 227
 - set, 213
 - transition, 212
 - two-pattern, 209
- test modes, 458
- test pattern, 461
- test vector, 209
- testing graph and testing table
 - for definiteness, 486–488
 - for diagnosability, 448–451
 - for finite memory, 479–483
 - for information losslessness, 494–497
 - for output memory, 488–490
 - for synchronizability, 508–510
 - for unique decipherability, 505–508
- theorem
 - absorption, 40
 - combining, 79
 - consensus, 41
 - De Morgan's, 42, 43
 - dual, 32, 40
 - involution, 42
 - Kleene's, 593
 - Shannon's expansion, 48
- three-pattern test, 232
- threshold element, 173
- threshold function, 178
 - identification of, 186–189
- threshold network, 181
- tie set, 139
- transducer, 570
- transfer function, 526
- transfer sequence, 332
- transition, λ , 573
- transition diagram, 357
- transition faults, 212
 - slow-to-fall, 212
 - slow-to-rise, 212
- transition graph, 572
 - conversion to deterministic form, 574–577
- transition table, 269
- transmission function, 54
- tree
 - distinguishing, 439, 440
 - homing, 436, 437
 - successor, 433
 - synchronizing, 438
- tree matching, 166
- true vertex, 183
 - minimal, 183
- truth table, 39
- truth values, 38
- tunneling phase logic (TPL), 177
- Turing machine, 293
- two-level realization, 76
- two-pattern test, 209
 - initialization vector, 209
 - test vector, 209

- unate function, 181, 182
- uncertainty, 433

- uncertainty vector, 434
 - homogeneous, 434
 - initial, 433
 - nonhomogeneous, 436
 - trivial, 434
- uncontrollability analysis, 239
 - 0-uncontrollable, 239
 - 1-uncontrollable, 239
 - status, 239
- unobservability status, 239
- unstable state, 339
- untestable fault, 215
- validatable nonrobust test, 227
- variable clock scheme, 224, 225
- Venn diagram, 24
- weight, 10
- weight–threshold vector, 180
- wired-AND, 110
- wired-OR, 110
- writing machine, 293
- X -successor, 309